



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For



Kubernetes and Cloud Native Associate (KCNA) Training

TGS Ref No: TGS-2023039343

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 6.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	31 August 2023	Legacy LP: first version.	Tertiary Infotech Academy
2.0	1 October 2024	Legacy LP: second version.	Tertiary Infotech Academy
3.0	17 August 2025	Legacy LP: third version.	Tertiary Infotech Academy
4.0	17 October 2025	Legacy LP: company name updated.	Tertiary Infotech Academy
6.0	5 September 2026	Full rebuild under the Kubernetes and Cloud Native Associate (KCNA) title. Deck rebuilt to the house visual system with native editable charts and mechanism-level diagrams; 25 self-contained labs with runnable manifests, synthetic datasets and verification checks; coverage realigned to the CNCF KCNA curriculum effective 24 Nov 2025 (44/28/16/12) while preserving the registered WSQ outcomes for TSC Solution Architecture ICT-DES-4006-1.1.	Dr. Alfred Ang

TABLE OF CONTENTS

How to Use This Guide.....	5
What this guide is, and what it is not.....	5
The three artefacts, and how they relate.....	5
How to work through it.....	5
What you are being certified against.....	6
Conventions used in this guide.....	6
Before You Start — Cluster Setup.....	7
What you need.....	7
Step 1 — Install a container runtime.....	7
Step 2 — Install kubectl.....	8
Step 3 — Install kind (the cluster tool this course is written against).....	8
Step 4 — Write a cluster configuration file.....	9
Step 5 — Create the cluster.....	9
Step 6 — Verify the cluster.....	10
Step 7 — Clone the course repository.....	10
The alternative the registered outline names: minikube.....	10
Setup troubleshooting.....	11
Tearing the cluster down.....	11
Day 1 — Cloud Native Foundations & Kubernetes Core Concepts.....	12
Day 1 Learning Focus.....	12
Day 1 Concepts.....	12
Day 1 Labs.....	16
Lab 01 — Cluster Reconnaissance with kubectl.....	17
Lab 02 — Authoring Your First Pod Manifest.....	30
Lab 03 — Commands, Arguments and Environment.....	41
Lab 04 — Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador.....	51
Lab 05 — Container Images, OCI and the Runtime Interface.....	61
Day 2 — Workloads, Scheduling & Container Orchestration.....	71
Day 2 Learning Focus.....	71
Day 2 Concepts.....	71
Day 2 Labs.....	74
Lab 06 — Labels, Selectors and ReplicaSets.....	76
Lab 07 — Deployments, Rollout Strategy and Rollback.....	86
Lab 08 — DaemonSets, Jobs and CronJobs.....	96
Lab 09 — Scheduling: nodeSelector, Affinity, Taints and Tolerations.....	107
Lab 10 — Resource Requests, Limits, QoS and Autoscaling.....	119
Day 3 — Services, Networking & Cluster Security.....	133
Day 3 Learning Focus.....	133
Day 3 Concepts.....	133
Day 3 Labs.....	136
Lab 11 — Services, Endpoints and Cluster DNS.....	138

Lab 12 — Service Types: ClusterIP, NodePort and LoadBalancer.....	151
Lab 13 — Ingress Resources and HTTP Routing.....	163
Lab 14 — Namespaces, ServiceAccounts and RBAC.....	186
Lab 15 — Secrets, ConfigMaps and Safe Injection.....	199
Lab 16 — NetworkPolicy and Default-Deny Isolation.....	214
Day 4 — Storage, Cluster Architecture & Application Delivery.....	228
Day 4 Learning Focus.....	228
Day 4 Concepts.....	228
Day 4 Labs.....	231
Lab 17 — Volumes: emptyDir, hostPath and the Container Filesystem.....	232
Lab 18 — PersistentVolumes, Claims and StorageClasses.....	244
Lab 19 — StatefulSets, Headless Services and Stable Identity.....	260
Lab 20 — Control Plane Anatomy and etcd Backup/Restore.....	275
Lab 21 — Packaging and Delivery: Helm, Kustomize and GitOps.....	294
Day 5 — Observability, Troubleshooting & Assessment.....	315
Day 5 Learning Focus.....	315
Day 5 Concepts.....	315
Day 5 Labs.....	318
Lab 22 — Probes, Health and Self-Healing.....	320
Lab 23 — Events, Logs, Field Selectors and the Metrics Server.....	331
Lab 24 — Metrics, Prometheus Exposition and the Observability Pipeline.....	345
Lab 25 — Troubleshooting Triage: Application, Node and Control Plane.....	361
Quick Command Reference.....	378
Context, identity and discovery.....	378
Creating, reading and changing objects.....	378
Workloads, controllers and rollouts.....	379
Scheduling and capacity.....	380
Services, networking and DNS.....	380
Security: identity, RBAC and configuration.....	380
Storage.....	381
Observability and troubleshooting.....	381
Packaging, delivery and the control plane.....	382
Assessment Preparation.....	383
The two instruments.....	383
What each Written Assessment question covers.....	383
What each Practical Performance task covers.....	383
How to prepare.....	384
The assessment flow, in order.....	384
A note on the external KCNA exam.....	385
Support.....	386
Contact.....	386
Links you will need.....	386
Course identity.....	386

How to Use This Guide

This Learner Guide is the study text for **Kubernetes and Cloud Native Associate (KCNA) Training** (TGS-2023039343), a 40-hour WSQ programme delivered over 5 days (37 hours of instruction plus 3 hours of assessment). It is written so that you can read it before the class, work from it during the class, and revise from it afterwards without needing the trainer present.

What this guide is, and what it is not

The guide is **the authoritative written record of the course**. It contains the concepts in continuous prose, and it contains the *complete* procedure for every one of the 25 hands-on labs — every command, every expected output, every verification step, the deliberate failure you are asked to inject, the troubleshooting table and the cleanup. Nothing is abbreviated to "see the slides".

It is **not** a transcript of the slide deck. The deck moves quickly, uses diagrams and live demonstrations, and is designed for the room. This guide is designed for reading. Where the two overlap, the guide carries the detail.

The three artefacts, and how they relate

Artefact	What it is for	Where it lives
Slide deck — <i>Kubernetes and Cloud Native Associate (KCNA) Training-v6.0.pptx</i>	Delivery in the classroom: diagrams, worked examples, decision trees, failure walkthroughs	LMS · https://lms-tms.tertiaryinfotech.com/
This Learner Guide	Study text and the authoritative lab procedures	LMS · https://lms-tms.tertiaryinfotech.com/
Lab folders — <code>courseware/labs/<slug>/</code>	The runnable material: <code>manifests/</code> , <code>data/</code> , <code>verification/</code> and a <code>README.md</code> identical in substance to the lab section here	Course repository

Every lab section in this guide names the **deck slide** it is introduced on and the **repository path** of its files. If a command in this guide and a command in the lab `README.md` ever disagree, the lab folder is the one that was executed against a live cluster — raise it with your trainer.

How to work through it

1. **Read the day chapter first.** Each of the five day chapters opens with a learning focus and then works through that day's concepts as continuous teaching prose. Read it before the lab sections that follow it.
2. **Do the labs in order.** Later labs assume namespaces, habits and vocabulary established by earlier ones. Each lab is self-contained in its own namespace, so a lab you get wrong cannot poison the next one.
3. **Type the commands.** Do not copy-paste blindly. The muscle memory for `kubectl -n <ns> describe pod <name>` is worth more in the assessment than any note you can write.
4. **Run the failure injection.** Every lab deliberately breaks something and asks you to diagnose it from the evidence. That section is where the Practical Performance skills actually come from.

5. **Run the verification script.** Each lab ends with `bash verification/checks.sh` and a pass count. Treat a failed check as a finding, not as an inconvenience.
6. **Clean up.** Every lab deletes only its own namespace. Never widen that.

What you are being certified against

This programme is registered against the SkillsFuture Singapore Technical Skills and Competency **Solution Architecture** (ICT-DES-4006-1.1). The six registered learning outcomes are:

- **LO1** — Develop a Kubernetes architectural proof of concept.
- **LO2** — Identify the technical and practical requirements in a Kubernetes setup.
- **LO3** — Develop a solution architecture within Kubernetes.
- **LO4** — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
- **LO5** — Demonstrate Kubernetes solution for a specific business problem.
- **LO6** — Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.

The course additionally covers the current CNCF **Kubernetes and Cloud Native Associate (KCNA)** curriculum so that the same five days prepare you for the external KCNA examination. Both maps are honoured: the WSQ outcomes govern the assessment you sit on Day 5, and the KCNA domains govern the technical breadth.

Conventions used in this guide

- **Monospace shaded blocks are things you run or files you read.** Shell commands, YAML manifests and command output appear in a shaded monospace block. Everything a block contains is literal.
- **Instructions are written as ordinary prose.** If a step tells you to open a browser, look at a column, or decide something, it is written as a normal sentence — it is not a command and must not be typed.
- **Placeholders are angle-bracketed**, for example `<pod-name>` or `<ns>`. Replace them, including the brackets.
- **Output blocks are illustrative.** IP addresses, UIDs, ages, node names and timestamps will differ on your cluster. Structure and column names will not.
- The cluster used to capture the outputs in this guide ran Kubernetes **v1.37.0**. Anything from v1.30 onwards behaves as described unless a lab says otherwise.

Before You Start — Cluster Setup

Every lab in this guide runs against a Kubernetes cluster you create on your own machine and own completely. You need three things: a **container runtime**, a **local cluster tool**, and **kubectl**. Allow 30–40 minutes the first time.

A standing rule for this course. Only ever operate on a cluster you created for this course. Do not point these labs at a shared development cluster, a company staging cluster, or anything reachable from a kubeconfig you were given. Several labs deliberately break application resources; Lab 20 inspects control-plane and backup evidence without restoring etcd. Check `kubectl config current-context` before you start, every time.

What you need

Requirement	Minimum	Comfortable
CPU	2 cores	4 cores
RAM free for the cluster	4 GB	8 GB
Free disk	15 GB	30 GB
Network	Able to pull from <code>docker.io</code> , <code>registry.k8s.io</code> and <code>ghcr.io</code>	—
Operating system	macOS 13+, Windows 10/11 with WSL 2, or a current Linux distribution	—

Step 1 — Install a container runtime

`kind` runs each Kubernetes node as a container, so it needs a working container runtime first. Choose **one** of the two below. Both are fine for this course; Rancher Desktop is the usual choice where Docker Desktop's licence terms are a problem for a corporate laptop.

Option A — Docker Desktop. Download the installer for your platform from <https://www.docker.com/products/docker-desktop/>, install it, launch it, and wait for the whale icon to report *Engine running*. On Windows, accept the prompt to use the **WSL 2 based engine** — this course assumes WSL 2, not Hyper-V.

Option B — Rancher Desktop. Download from <https://rancherdesktop.io/>, install, and on first launch open **Preferences** → **Container Engine** and select **dockerd (moby)**. `kind` talks to the Docker API, so the containerd/nerdctl engine is not a drop-in substitute.

Verify, on any platform:

```
docker version --format '{{.Server.Version}}'
docker run --rm hello-world
```

You should get a server version number and then the *Hello from Docker!* message. If `docker version` reports only a client, the engine is not running yet — start Docker Desktop or Rancher Desktop and wait.

Windows users: run every command in this course from inside a **WSL 2 Ubuntu shell**, not from PowerShell or CMD. Open PowerShell once, run `wsl --install -d Ubuntu`, reboot if asked, then

do everything else in the Ubuntu terminal. Paths, `bash` verification scripts and the `kubectl` binaries in this guide all assume a Linux shell.

Step 2 — Install kubectl

`kubectl` must be within one minor version of your cluster. The labs were captured against **v1.37.0**.

macOS — with Homebrew:

```
brew install kubectl
```

Or without Homebrew, for Apple Silicon:

```
curl -LO "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/darwin/arm64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/kubectl
```

Replace `arm64` with `amd64` on an Intel Mac.

Linux (and Windows via WSL 2 Ubuntu):

```
curl -LO "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/kubectl
```

On an ARM64 Linux host or ARM WSL, substitute `linux/arm64`.

Windows, native PowerShell (only if you are not using WSL 2):

```
winget install -e --id Kubernetes.kubectl
```

Verify on every platform:

```
kubectl version --client
Client Version: v1.37.0
Kustomize Version: v5.8.1
```

Step 3 — Install kind (the cluster tool this course is written against)

Pin the version. Newer `kind` releases change the default node image, and the labs were captured against a known pair.

Use kind v0.33.0 with node image kindest/node:v1.36.4.

macOS — Homebrew, then confirm the version:

```
brew install kind
kind version
```

If Homebrew installs something other than v0.33.0, install the pinned binary directly instead (Apple Silicon shown):

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-darwin-arm64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

Linux / WSL 2 Ubuntu:

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

Windows, native PowerShell (only if not using WSL 2):


```
winget install -e --id Kubernetes.kind
```

Verify:

```
kind version  
kind v0.33.0 go1.26.6 darwin/arm64
```

Step 4 — Write a cluster configuration file

Create a file called `kcna-cluster.yaml` somewhere you can find it — the root of the course repository is a good place. This is the cluster every lab in this guide assumes.

```
kind: Cluster  
apiVersion: kind.x-k8s.io/v1alpha4  
name: kcna  
nodes:  
- role: control-plane  
  image: kindest/node:v1.36.4  
  labels:  
    ingress-ready: "true"  
  extraPortMappings:  
    - containerPort: 30080  
      hostPort: 30080  
      protocol: TCP  
    - containerPort: 30443  
      hostPort: 30443  
      protocol: TCP  
- role: worker  
  image: kindest/node:v1.36.4  
- role: worker  
  image: kindest/node:v1.36.4
```

Three things in that file matter and are worth understanding rather than copying:

- **Two workers plus a control plane.** Day 2's scheduling labs (Lab 09) need more than one schedulable node to demonstrate `nodeSelector`, affinity, taints and topology spread. A single-node cluster will run every other lab, but Lab 09 becomes a reading exercise.
- **extraPortMappings.** `kind` nodes are containers, so a NodePort is open on the node container's network, not on your laptop. Publishing 30080 and 30443 to the host is what makes `curl http://localhost:30080` work in Lab 12.
- **The pinned image.** Fixing the node image fixes the Kubernetes version, which is what makes the outputs in this guide reproducible.

Step 5 — Create the cluster

```
kind create cluster --config kcna-cluster.yaml
```

Expected output (abridged — the emoji and timings vary):

```
Creating cluster "kcna" ...  
✓ Ensuring node image (kindest/node:v1.36.4)   
✓ Preparing nodes   
✓ Writing configuration   
✓ Starting control-plane   
✓ Installing CNI   
✓ Installing StorageClass   
✓ Joining worker nodes   
Set kubectl context to "kind-kcna"
```

The first run downloads roughly 1 GB of node image and takes several minutes. Subsequent runs take under two.

Step 6 — Verify the cluster

Run all four. Every one of them should succeed before you start Lab 01.

```
kubectl config current-context
kind-kcna
kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-control-plane	Ready	control-plane	93s	v1.36.4
kcna-worker	Ready	<none>	78s	v1.36.4
kcna-worker2	Ready	<none>	78s	v1.36.4

All three must read `Ready`. If a node sits at `NotReady` for more than two minutes, see the troubleshooting table below.

```
kubectl get pods -n kube-system
```

Every Pod should be `Running` or `Completed`. You should see `etcd-*`, `kube-apiserver-*`, `kube-controller-manager-*`, `kube-scheduler-*`, two `coredns-*` Pods, a `kindnet-*` and a `kube-proxy-*` per node, and `local-path-provisioner-*`.

```
kubectl version
```

Client and server minor versions must be within one of each other.

Step 7 — Clone the course repository

```
git clone https://github.com/tertiarycourses/TGS-2023039343-Kubernetes-and-Cloud-Native-Associate-KCNA-Training.git
cd TGS-2023039343-Kubernetes-and-Cloud-Native-Associate-KCNA-Training
```

Every relative path in every lab — `manifests/00-namespace.yaml`, `data/*.csv`, `verification/checks.sh` — is relative to that lab's own folder under `courseware/labs/` in the cloned repository. For example, run `cd courseware/labs/lab-01-kubectl-and-cluster-recon` before Lab 01. Each lab tells you to work inside its own folder.

The alternative the registered outline names: minikube

The registered course outline names **minikube** as the local cluster tool, and it remains fully supported for this course. Every lab except the ones that inspect `kind`'s node containers directly (Lab 20's `docker exec` into the control-plane container, and the optional appendices in Labs 12, 13 and 16) runs unchanged on minikube.

Install:

```
brew install minikube # macOS
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
chmod +x minikube && sudo mv minikube /usr/local/bin/ # Linux / WSL 2
winget install -e --id Kubernetes.minikube # Windows native
```

Create a cluster equivalent to the kind one above:

```
minikube start --profile kcna --nodes 3 --kubernetes-version v1.36.4 --driver docker --cpus 2 --memory 3g
kubectl config use-context kcna
kubectl get nodes
```

Two differences to keep in mind while you work:

- **NodePort access.** There is no `extraPortMappings`. Use `minikube -p kcna service <svc> -n <ns> --url` or `minikube -p kcna ip` plus the node port, instead of `localhost:30080`.
- **Add-ons instead of manual installs.** `minikube` ships an ingress add-on (`minikube -p kcna addons enable ingress`) and a metrics-server add-on (`minikube -p kcna addons enable metrics-server`). Where a lab installs one of these by hand, the add-on is an acceptable substitute — say so in your lab notes.

Setup troubleshooting

Symptom	Likely cause	What to do
<code>kind create cluster</code> hangs at <i>Starting control-plane</i>	Not enough memory given to the Docker VM	Docker Desktop → Settings → Resources: give it 6 GB or more, then <code>kind delete cluster --name kcna</code> and recreate
ERROR: failed to create cluster: node(s) already exist for a cluster with the name "kcna"	A previous cluster is still there	<code>kind get clusters</code> , then <code>kind delete cluster --name kcna</code> and recreate
Cannot connect to the Docker daemon	Runtime not started, or Rancher Desktop is on the containerd engine	Start Docker/Rancher Desktop; in Rancher Desktop set Container Engine to dockerd (moby)
The connection to the server localhost:8080 was refused	No kubeconfig — <code>kubectl</code> is guessing	<code>kind export kubeconfig --name kcna</code> , then <code>kubectl config current-context</code>
A worker stays <code>NotReady</code>	CNI still starting, or the host is starved	Wait two minutes, then <code>kubectl -n kube-system get pods -o wide</code> and describe any <code>kindnet</code> Pod that is not Running
error: exec plugin: invalid apiVersion "client.authentication.k8s.io/v1alpha1"	kubeconfig written by a much older CLI	Recreate the kubeconfig with the current tool, or align <code>kubectl</code> to within one minor of the server
Windows: <code>bash verification/checks.sh</code> fails with <code>\r: command not found</code>	The repository was cloned on Windows with CRLF line endings	Clone inside WSL 2, or run <code>git config --global core.autocrlf input</code> and re-clone
Image pulls fail behind a corporate proxy	Registry access blocked	Configure the proxy in Docker Desktop → Settings → Resources → Proxies, then recreate the cluster

Tearing the cluster down

At the end of the course, or any time you want a clean start:

```
kind delete cluster --name kcna
minikube delete --profile kcna
```

Both commands remove **only** the cluster named on the command line. Never run `kind delete clusters --all` or `minikube delete --all` on a machine that also carries work clusters.

Day 1 — Cloud Native Foundations & Kubernetes Core Concepts

Day 1 Learning Focus

Establish what 'cloud native' actually means, what a container really is, and why the Kubernetes API — not the kubelet, not Docker — is the product. By the end of the day you can read a cluster you did not build, author a Pod manifest you would defend in review, configure a container entirely from outside its image, and compose multi-container Pods.

Field	Value
Instructional hours	8 hours
Class window	9:30 AM – 6:30 PM
Registered topics	1. Core Concepts
KCNA modules	Cloud Native Architecture & the CNCF Ecosystem
Learning outcomes	LO1
Abilities / Knowledge	A1 / K1
Labs	5 — Lab 01, Lab 02, Lab 03, Lab 04, Lab 05
Hands-on minutes	220 minutes
Deck slides	15–132

Session structure. The table below is the timetable the trainer works to. Breaks are not counted as instructional time.

Time	Duration	Session	Mode
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

Day 1 Concepts

Cloud native is a set of operating properties, not a product list

The CNCF definition is worth reading clause by clause, because every clause is a requirement you can test a system against. Cloud native technologies "empower organisations to build and run **scalable** applications in **modern, dynamic environments** such as public, private and hybrid clouds", using approaches such as "containers, service meshes, microservices, immutable infrastructure and declarative APIs". They "enable **loosely coupled** systems that are **resilient, manageable and observable**", and combined with "**robust automation**" they let engineers "make high-impact changes frequently and predictably with minimal toil".

Nothing in that definition names a vendor, and nothing in it names Kubernetes. Kubernetes is one implementation of those properties, and it is the one this course teaches, but a system can be cloud native without it and can run on it while being nothing of the sort. When you are asked in the assessment to justify an architecture, justify it against those properties.

The four deployment eras — physical servers, virtual machines, containers, and orchestrated containers — each solved the previous era's problem and left a new one. Physical servers wasted capacity and coupled applications to hardware. Virtual machines fixed utilisation but kept a whole guest operating system per workload. Containers removed the guest OS and made the unit of deployment an image, but left three jobs undone: **scheduling** (which machine runs this?), **lifecycle** (who restarts it, and how many should there be?) and **networking and discovery** (how does anything find it?). Orchestration is the era that does those three jobs, and Kubernetes is how this course does them.

The CNCF, maturity levels and why they matter to you

The CNCF hosts projects at three maturity levels. **Sandbox** is for early projects that need a neutral home; adoption is expected to be experimental. **Incubating** requires demonstrable production use by at least three independent adopters, a healthy committer base and a documented governance process. **Graduated** adds a completed independent security audit, an adopted CNCF Code of Conduct, an explicit governance and committer-diversity record, and the Core Infrastructure Initiative / OpenSSF best-practices badge.

The practical reading is: graduation is a statement about *project health and process*, not about whether the tool suits you. Kubernetes, Prometheus, Envoy, etcd, containerd, CoreDNS, Helm, Argo, Flux, Istio, Cilium, Harbor and many others are graduated. That tells you the project will still be there and has been audited. It does not tell you it is the right choice for your problem.

The CNCF also deliberately does *not* do certain things: it does not pick winners between competing projects, does not ship a distribution, and does not provide commercial support. Governance of Kubernetes itself sits with the project's Steering Committee and its SIGs, not with the CNCF board.

A container is a process with a restricted view

There is no such thing as a "container" object in the Linux kernel. A container is an ordinary process to which three kernel facilities have been applied.

Namespaces restrict what a process can see. A PID namespace makes the process believe it is PID 1 in an empty process table. A mount namespace gives it its own filesystem tree. A network namespace gives it its own interfaces, routing table and port space — this is why every Pod can bind port 80 without colliding. UTS, IPC, user and cgroup namespaces complete the set. Namespaces are about *visibility*.

cgroups v2 account for and limit what a process may consume. CPU, memory, IO and PIDs are metered and capped per cgroup. cgroups are about *consumption*, not visibility — a container can be limited to 100 MiB of memory and still, without the right settings, read the host's total memory from `/proc/meminfo`. That distinction is examined directly.

Union filesystems make images cheap. An image is an ordered stack of read-only layers; the runtime adds a thin writable layer on top at start. Twenty containers from the same image share one copy of every read-only layer on disk. The writable layer is per-container and dies with the container — which is the whole reason Day 4 exists.

The **Open Container Initiative** standardises three of these things: the **image specification** (what a set of layers plus a manifest and config looks like), the **runtime specification** (what a bundle plus a `config.json` means, and what `create/start/kill/delete` must do), and the **distribution specification** (the registry HTTP API). Kubernetes never talks OCI directly; it talks the **Container Runtime Interface (CRI)** over gRPC to a runtime such as **containerd** or **CRI-O**, which then uses an OCI runtime such as **runc** to do the actual `clone()` and `pivot_root()`. Docker is not a CRI runtime and has not been callable by the kubelet since the dockershim was removed in Kubernetes v1.24; Docker Desktop still works for this course because it is what builds and stores images and what runs the `kind` nodes.

An image is not a container

An **image** is an immutable, content-addressed artefact identified by a digest — `sha256:...` — and usually also by one or more mutable **tags**. A **container** is a running instance created from an image. `nginx:1.27` may point at a different digest tomorrow; `nginx@sha256:abc...` cannot. That is the whole reproducibility trade: tags are readable and mutable, digests are opaque and immutable. In a Pod spec, `imagePullPolicy` defaults to `IfNotPresent` unless the tag is `:latest` or absent, in which case it defaults to `Always` — a default that surprises people, and a favourite exam question.

The API is the product

Everything Kubernetes does is an HTTP request to the API server. `kubectl` is an HTTPS client and nothing more; running any command with `--v=6` prints the exact `GET/POST` it issued. Resources live at `/api/v1/...` for the historical **core** group, which has an empty group name, and at `/apis/<group>/<version>/...` for every named group — `apps/v1`, `batch/v1`, `networking.k8s.io/v1`, `discovery.k8s.io/v1` and so on. That `/api` versus `/apis` asymmetry is a historical artefact and it appears on the exam.

Two consequences follow. First, **anything `kubectl` can do, a controller, a CI job or a script can do** — there is no privileged back channel. Second, the API server can tell you about itself: `kubectl api-versions` lists the group/version pairs it serves, `kubectl api-resources` lists the kinds including the critical **namespaced versus cluster-scoped** column, and `kubectl explain` walks the server's **live OpenAPI v3 schema** rather than a bundled cheat sheet, so it is never out of date for the cluster in front of you.

A Pod object has five main top-level fields: `apiVersion`, `kind`, `metadata`, `spec` and `status`. You write `spec`. Controllers write `status`. The control plane writes defaulted fields such as `creationTimestamp`, `uid` and `resourceVersion`. `kubectl get -o yaml` returns all three mixed together; only what you wrote belongs in Git.

Reconciliation, and the two planes

A Kubernetes controller runs one loop forever: **observe** the current state, **compare** it with the desired state in `spec`, **act** to close the gap. This is *level-triggered* control — it responds to the current level of the signal, not to the edge of a change event. That is why a controller that was down for an hour catches up correctly when it comes back, and why a missing ConfigMap does not require you to recreate the Pod that references it: the kubelet simply keeps retrying the mount and succeeds the moment the object appears.

The **control plane** runs `kube-apiserver` (the only component that talks to `etcd`), `etcd` itself, `kube-scheduler` and `kube-controller-manager`. The **node plane** runs the `kubelet`, `kube-proxy` and a

CRI runtime on every node. Between `kubectl apply` and a running process, the sequence is: `kubectl` POSTs the object; the API server authenticates, authorises, runs admission and persists to etcd; the scheduler notices a Pod with an empty `spec.nodeName`, filters and scores nodes and writes a binding; the kubelet on that node sees a Pod bound to it, asks the CRI runtime to pull images and create the sandbox and containers, and reports status back.

Namespaces, labels, annotations and ownership

A **namespace** is a scope for names and a boundary for quota, RBAC and NetworkPolicy. It is *not* a security boundary by itself, it does *not* isolate the network by default, and cluster-scoped objects — Nodes, PersistentVolumes, StorageClasses, ClusterRoles, CRDs — are not in one at all.

Labels are indexed key/value pairs used for *selection*; selectors are how one object finds another.

Annotations are unindexed and arbitrary, used to carry data for tools and humans. Never select on an annotation; never put a kilobyte of JSON in a label. The recommended `app.kubernetes.io/*` label set exists so that different tools can agree on what "name", "instance", "component" and "part-of" mean.

ownerReferences wire the object graph: a ReplicaSet owns its Pods, a Deployment owns its ReplicaSets. Deleting an owner triggers cascading deletion of its dependents through garbage collection, with `Foreground`, `Background` and `Orphan` propagation policies available.

The Pod is the unit of scheduling

Kubernetes schedules Pods, never containers. A Pod is one or more containers that **share a network namespace** — the same IP address and port space, so they reach each other on `localhost` — and that **can share volumes**, but that do **not** share a filesystem root or a mount namespace by default. Every mount must be declared per container. "But they're in the same Pod!" is the single most common multi-container mistake, and Lab 04 makes you produce it.

`status.phase` has exactly five values — `Pending`, `Running`, `Succeeded`, `Failed`, `Unknown` — and it is a coarse summary, not a health signal. A Pod in `Running` may have a container in `CrashLoopBackOff`. **Conditions** (`PodScheduled`, `Initialized`, `ContainersReady`, `Ready`) and per-container state (`waiting`, `running`, `terminated` with a reason) carry the real detail.

`CrashLoopBackOff` is a *waiting reason describing a back-off timer*, not an error in itself — the error is whatever made the container exit, and it is in the previous container's logs.

`spec.restartPolicy` is Pod-wide with three values: `Always` (the default, and the only one a Deployment permits), `OnFailure` and `Never`. `containerPort` in a Pod spec is **documentation** — it does not publish anything, does not open a firewall and is not required for a Service to reach the container.

Configuring a container from outside its image

An image carries `ENTRYPOINT` and `CMD`. A Pod spec overrides them with `command` and `args` respectively — the naming is deliberately confusing and is examined. If you set `command`, the image's `ENTRYPOINT` is discarded; if you set only `args`, the image's `ENTRYPOINT` runs with your arguments.

Configuration reaches a container by five routes: literal `env` values, `env` sourced from a ConfigMap or Secret key via `valueFrom`, whole objects via `envFrom`, mounted volumes, and the **downward API** for the Pod's own metadata. The critical operational difference is that **environment variables are a snapshot taken at container start and never change**, whereas a mounted ConfigMap or Secret is

refreshed by the kubelet in place — via an atomic symlink flip on the `..data` directory — without restarting anything.

The three multi-container patterns

All three use the same mechanism — extra containers in one Pod — and differ only in intent. A **sidecar** adds capability to the main container (log shipping, certificate renewal, a cache warmer). An **adapter** normalises the main container's output into a standard format something else expects. An **ambassador** proxies the main container's outbound connections so the application can address `localhost` and know nothing about discovery, sharding or TLS. **Init containers** run to completion, in order, before any app container starts. Since the feature graduated, a container declared in `initContainers` with `restartPolicy: Always` is a **native sidecar**: it starts before the app containers, keeps running alongside them, and no longer blocks Job completion.

Day 1 Labs

Day 1 has 5 labs, listed below and then set out in full. Work through them in order; each runs in its own namespace and cleans up after itself.

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 01	Cluster Reconnaissance with kubectl	kcna-lab01	45	LO1 / A1 / K1	72
Lab 02	Authoring Your First Pod Manifest	kcna-lab02	45	LO1 / A1 / K1	92
Lab 03	Commands, Arguments and Environment	kcna-lab03	40	LO1 / A1 / K1	105
Lab 04	Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador	kcna-lab04	50	LO1 / A1 / K1	118
Lab 05	Container Images, OCI and the Runtime Interface	kcna-lab05	40	LO1 / A1 / K1	127

Lab 01 — Cluster Reconnaissance with kubectl

Field	Value
Lab ID	Lab 01
Title	Cluster Reconnaissance with kubectl
Day / Topic	Day 1 · Core Concepts
Duration	45 minutes
Namespace	kcna-lab01
Learning outcome	LO1 — Develop a Kubernetes architectural proof of concept.
Ability	A1 Develop an architectural proof of concept
Knowledge	K1 Process for developing proof of concepts
Deck slide	Slide 72
Repository path	courseware/labs/lab-01-kubectl-and-cluster-recon/

Goal. Profile an unfamiliar Kubernetes cluster with kubectl alone — contexts, nodes, API groups, resource schemas and the raw REST calls behind every command — and record the findings as reusable evidence.

What you will produce:

- A namespace kcna-lab01 with a ResourceQuota and LimitRange, discovered and read back as YAML
- A recon-shell Pod that serves the Meridian Freight reconnaissance checklist from a ConfigMap built out of data/recon-checklist.csv
- A verified core-API inventory diff proving every resource in data/core-api-inventory.tsv is served by this cluster

Lab 01 · Objective

By the end of this lab you will be able to:

- Establish **which cluster, which user and which namespace** your `kubectl` is actually talking to, and change that scope deliberately.
- Enumerate a cluster's **API surface** with `kubectl api-versions` and `kubectl api-resources`, and tell namespaced from cluster-scoped kinds.
- Read the **live OpenAPI schema** with `kubectl explain` instead of guessing field names.
- Prove that **kubectl is nothing but an HTTP client** by exposing the raw REST request with `--v=6`.
- Retrieve the **server-side truth** of an object with `kubectl get -o yaml` and distinguish what *you* wrote from what the *control plane* added.
- Record a reconnaissance baseline and diff a live cluster against it — the first deliverable of any architectural proof of concept.

Lab 01 · Prerequisites

- A running Kubernetes cluster, v1.30 or later. A single-node [kind](https://kind.sigs.k8s.io/) cluster is sufficient and is what this lab is written against. <<https://kind.sigs.k8s.io/>>

- `kubectl` on your PATH, within one minor version of the server.
- No cluster-admin rights are required beyond create/read/delete inside your own namespace.

Confirm access before you start:

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0

kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	47m	v1.37.0

If `kubectl version` prints only a client version and then The connection to the server localhost:8080 was refused - did you specify the right host or port?, your kubeconfig is not being found. Stop here and fix it (see **Troubleshooting**).

Change into the lab directory — every relative path below is from there:

```
cd courseware/labs/lab-01-kubectl-and-cluster-recon
```

Lab 01 · Scenario

Meridian Freight Pte Ltd is a Singapore-based regional logistics operator. Its **Depot Portal** — the web application that depot supervisors use to book inbound trailers — has been running on a pair of hand-managed VMs for six years. The platform team has been asked to produce an architectural **proof of concept** for moving Depot Portal onto Kubernetes.

You have just been handed a kubeconfig for a cluster nobody on your team built. Before a single line of Depot Portal YAML is written, the PoC has to answer one question honestly: **what is actually on this cluster, and how do I find out?**

That is this lab. You will build a reconnaissance record for the cluster and compare it against Meridian's standard baseline in `data/core-api-inventory.tsv`.

Lab 01 · Step-by-step procedure

Step 1 — Find out who you are and where you are pointing

`kubectl` reads a kubeconfig file. That file contains *clusters* (an API server URL plus a CA), *users* (credentials) and *contexts* (a named pairing of the two, plus an optional default namespace).

List every context available to you:

```
kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	kind-kcna-qa	kind-kcna-qa	kind-kcna-qa	

The * marks the current context. Now print only the current context's details, with credentials redacted:

```
kubectl config view --minify

apiVersion: v1
clusters:
```

```
- cluster:
  certificate-authority-data: DATA+OMITTED
  server: https://127.0.0.1:52913
  name: kind-kcna-qa
contexts:
- context:
  cluster: kind-kcna-qa
  user: kind-kcna-qa
  name: kind-kcna-qa
current-context: kind-kcna-qa
kind: Config
preferences: {}
users:
- name: kind-kcna-qa
  user:
    client-certificate-data: DATA+OMITTED
    client-key-data: DATA+OMITTED
```

Two things to notice:

- `server`: is an ordinary HTTPS URL. Everything `kubectl` does is an HTTP request to that URL.
- There is **no** `namespace`: **key** in this context, so every namespaced command you run defaults to the namespace `default`. That is the single most common source of "my Pod disappeared".

Print just the current context name (useful in scripts):

```
kubectl config current-context  
kind-kcna-qa
```

Step 2 — Survey the nodes

```
kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME		
kcna-qa-20260905-control-plane	Ready	control-plane	47m	v1.37.0	172.18.0.2
<none>	Debian GNU/Linux 12 (bookworm)	6.10.14-linuxkit		containerd://2.1.4	

Read the last column. `containerd://2.1.4` tells you the **container runtime** behind the CRI socket on this node. Kubernetes never runs containers itself; it asks a CRI-conformant runtime to do it. You will come back to this in Lab 05.

Now look at what the node advertises to the scheduler:

```
kubectl get node -o jsonpath='{.items[0].status.allocatable}' | tr ',' '\n'
{"cpu":"10"
"ephemeral-storage":"61202244Ki"
"hugepages-1Gi":"0"
"hugepages-2Mi":"0"
"hugepages-32Mi":"0"
"hugepages-64Ki":"0"
"memory":"8039412Ki"
"pods":"110"}
```

`allocatable` — not `capacity` — is the pool the scheduler may spend. Everything you will learn about requests and limits on Day 2 is arithmetic against these numbers.

Step 3 — Enumerate the API surface

A Kubernetes cluster is a set of REST APIs grouped into *API groups*. List the group/version pairs this server serves:

```
kubectl api-versions | sort | head -n 12
```

```

admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1

```

Count them:

```

kubectl api-versions | wc -l

29

```

Now list the *kinds* those groups expose. The core group is the historical one and has an empty group name — that is why core objects use `apiVersion: v1` while Deployments use `apiVersion: apps/v1`:

```

kubectl api-resources --api-group=''

```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pv	v1	false	PersistentVolume
Pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service

The `NAMESPACED` column is the one that matters architecturally. Ask the API directly for everything that is **not** namespaced:

```

kubectl api-resources --namespaced=false --no-headers | awk '{print $1}' | head -n 12

componentstatuses
namespaces
nodes
persistentvolumes
mutatingwebhookconfigurations
validatingadmissionpolicies
validatingadmissionpolicybindings
validatingwebhookconfigurations
customresourcedefinitions
apiservices
selfsubjectreviews
tokenreviews

```

Cluster-scoped objects cannot be isolated per tenant with a namespace. That is a design constraint you must record in the PoC.

Finally, which verbs may you use on Pods?

```

kubectl api-resources --api-group='' -o wide | grep '^pods '

```

```
 pods    po    v1    true    Pod    [create delete deletecollection get list patch update watch]
all
```

Step 4 — Read the schema instead of guessing

`kubectl explain` does not read a bundled cheat sheet. It downloads the server's **OpenAPI v3 schema** and walks it. That means it is always correct for *this* cluster's version.

```
kubectl explain pod

GROUP:
KIND:    Pod
VERSION: v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <ObjectMeta>
  spec          <PodSpec>
  status        <PodStatus>
```

Every Kubernetes object has these five top-level fields. Drill into `spec`:

```
kubectl explain pod.spec --recursive | head -n 20

GROUP:
KIND:    Pod
VERSION: v1

FIELD: spec <PodSpec>

DESCRIPTION:
  Specification of the desired behavior of the pod.
FIELDS:
  activeDeadlineSeconds <integer>
  affinity              <Affinity>
    nodeAffinity        <NodeAffinity>
      preferredDuringSchedulingIgnoredDuringExecution <[]PreferredSchedulingTerm>
      preference      <NodeSelectorTerm>
        matchExpressions <[]NodeSelectorRequirement>
          key <string>
          operator <string>
          values <[]string>
        matchFields <[]NodeSelectorRequirement>
          key <string>
          operator <string>
          values <[]string>
      weight <integer>
```

Now the exact field you will need in Lab 02:

```
kubectl explain pod.spec.containers.resources

GROUP:
KIND:    Pod
VERSION: v1

FIELD: resources <ResourceRequirements>

DESCRIPTION:
  Compute Resources required by this container. Cannot be updated.
```

```
FIELDS:
  claims      <[]ResourceClaim>
  limits      <map[string]Quantity>
  requests    <map[string]Quantity>
```

Try a field that does not exist — this is how you learn to trust the tool:

```
kubectl explain pod.spec.container
error: field "container" does not exist
```

Singular vs plural matters. `containers` is a list.

Step 5 — Create the lab namespace and its guardrails

Read the manifest first:

```
cat manifests/00-namespace.yaml

apiVersion: v1
kind: Namespace
metadata:
  name: kcna-lab01
  labels:
    app.kubernetes.io/part-of: depot-portal
    kcna.tertiaryinfotech.com/lab: lab-01
    kcna.tertiaryinfotech.com/day: "1"
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: latest
```

Apply it:

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab01 created
```

Apply the namespaced policy objects. These give you something non-trivial to inspect, and they cap what this lab can consume:

```
kubectl apply -f manifests/10-namespace-guardrails.yaml
resourcequota/depot-recon-quota created
limitrange/depot-recon-defaults created

kubectl -n kcna-lab01 get resourcequota,limitrange

NAME                                     AGE    REQUEST
LIMIT
resourcequota/depot-recon-quota        8s     count/configmaps: 0/10, pods: 0/10, requests.cpu:
0/1, requests.memory: 0/1Gi           limits.cpu: 0/2, limits.memory: 0/2Gi

NAME                                     CREATED AT
limitrange/depot-recon-defaults        2026-09-05T07:44:11Z
```

Step 6 — Turn the dataset into a ConfigMap

`data/recon-checklist.csv` is Meridian's standard recon checklist. Look at it:

```
head -n 5 data/recon-checklist.csv

# Meridian Freight Pte Ltd - Depot Portal cluster reconnaissance checklist
# Synthetic dataset for KCNA Lab 01. Consumed as ConfigMap `recon-checklist`.
check_id,area,question,kubectl_probe,evidence_expected
RC-001,identity,Which cluster and user is my context bound to?,kubectl config view --
minify,current-context and user name
RC-002,version,Do client and server minor versions match?,kubectl version,clientVersion
and serverVersion blocks
```

Before creating it for real, use `--dry-run=client -o yaml` to see exactly what `kubectl` would POST. This flag is the single most useful thing in the tool:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv \
  --dry-run=client -o yaml | head -n 12

apiVersion: v1
data:
  recon-checklist.csv: |
    # Meridian Freight Pte Ltd - Depot Portal cluster reconnaissance checklist
    # Synthetic dataset for KCNA Lab 01. Consumed as ConfigMap `recon-checklist`.
    check_id,area,question,kubectl_probe,evidence_expected
    RC-001,identity,Which cluster and user is my context bound to?,kubectl config view --
minify,current-context and user name
    RC-002,version,Do client and server minor versions match?,kubectl
version,clientVersion and serverVersion blocks
    RC-003,capacity,How many nodes are Ready and what is their role?,kubectl get nodes -o
wide,STATUS Ready and ROLES column
    RC-004,api,Which API groups and versions does the server serve?,kubectl api-
versions,list of group/version strings
    RC-005,api,Which resource kinds exist and are they namespaced?,kubectl api-
resources,NAMESPACED and KIND columns
    RC-006,schema,What fields may a Pod spec legally contain?,kubectl explain
pod.spec,field list with types
```

`--from-file=<path>` produces one key named after the **file's basename**. Now create it for real:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv

configmap/recon-checklist created

kubectl -n kcna-lab01 get configmap recon-checklist \
  -o jsonpath='{.data.recon-checklist\.csv}' | wc -l

15
```

The `\.` escape is required: `jsonpath` treats `.` as a path separator, so a key containing a dot must be escaped.

Step 7 — Deploy the reconnaissance Pod

Do not run this yet if you want to see the failure injection first — skip to section 7, then come back. If you are running straight through:

```
kubectl apply -f manifests/20-recon-shell.yaml

pod/recon-shell created

kubectl -n kcna-lab01 wait --for=condition=Ready pod/recon-shell --timeout=90s

pod/recon-shell condition met

kubectl -n kcna-lab01 get pod recon-shell

NAME          READY   STATUS    RESTARTS   AGE
recon-shell   1/1     Running   0           14s
```

The container read the mounted CSV at start-up. Prove it:

```
kubectl -n kcna-lab01 logs recon-shell | tail -n 3

RC-012,events,What has the control plane recently done here?,kubectl get events -n kcna-
lab01 --sort-by=.lastTimestamp,Scheduled Pulled Created Started
[recon-shell] checklist rows: 13
[recon-shell] holding open for kubectl exec
```

13 is the count of non-comment lines: the CSV header row plus the 12 RC-0nn records. The container computed it with `grep -vc '^#'` against the mounted file, so this line is direct evidence that the ConfigMap really was mounted and read.

And from inside the container:

```
kubectl -n kcna-lab01 exec recon-shell -- ls -l /etc/depot
total 0
lrwxrwxrwx    1 root    root                26 Sep  5 07:45 recon-checklist.csv ->
..data/recon-checklist.csv
```

That symlink is how the kubelet does atomic ConfigMap updates — the real file lives under a timestamped `..2026_09_05_07_45_10.123456789` directory and `..data` is flipped between them.

Step 8 — Read the server-side truth of an object

```
kubectl -n kcna-lab01 get pod recon-shell -o yaml | head -n 30
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{...}}
  creationTimestamp: "2026-09-05T07:45:09Z"
  labels:
    app.kubernetes.io/name: recon-shell
    app.kubernetes.io/part-of: depot-portal
    kcna.tertiaryinfotech.com/lab: lab-01
  name: recon-shell
  namespace: kcna-lab01
  resourceVersion: "4127"
  uid: 8f1cb0f2-6d9b-4b8f-9f0a-4b0c8alf2d31
spec:
  containers:
  - args:
    - |
      echo "[recon-shell] Meridian Freight depot reconnaissance checklist"
    ...
    image: busybox:1.36
    imagePullPolicy: IfNotPresent
    name: shell
```

You wrote none of `creationTimestamp`, `resourceVersion`, `uid`, or the whole `status`: block at the bottom. Those are **control-plane defaulted and controller-written** fields. The discipline for a PoC is: your Git repository holds *desired state*; `kubectl get -o yaml` shows *desired + defaulted + observed*.

Isolate just the observed part:

```
kubectl -n kcna-lab01 get pod recon-shell \
-o jsonpath='{.status.phase}{"\t"}{.status.podIP}{"\t"}{.spec.nodeName}{"\n"}'
Running    10.244.0.8    kcna-qa-20260905-control-plane
```

Step 9 — Prove kubectl is just an HTTP client

Raise the log verbosity to 6. Level 6 prints one line per HTTP request:

```
kubectl -n kcna-lab01 get pods --v=6
I0905 15:44:02.118374    62311 loader.go:395] Config loaded from file:  /root/.kube/config
I0905 15:44:02.146902    62311 round_tripper.go:553] GET
https://127.0.0.1:52913/api/v1/namespaces/kcna-lab01/pods?limit=500 200 OK in 21
milliseconds
```


NAME	READY	STATUS	RESTARTS	AGE
recon-shell	1/1	Running	0	64s

Read the URL carefully:

Segment	Meaning
/api/v1	the core group (no group name) at version v1
/namespaces/kcna-lab01	the namespace scope
/pods	the resource, plural, lower-case — exactly the NAME column of api-resources
?limit=500	server-side pagination

Now do the same for a resource in a named group:

```
kubectl -n kcna-lab01 get deployments --v=6 2>&1 | grep round_tripplers
I0905 15:45:31.402118 62410 round_tripplers.go:553] GET
https://127.0.0.1:52913/apis/apps/v1/namespaces/kcna-lab01/deployments?limit=500 200 OK in
8 milliseconds
```

/apis/apps/v1/... — note /apis (plural) for named groups versus /api for the core group. This asymmetry is a historical artefact and it appears in the KCNA exam.

You can skip `kubectl`'s object model entirely and hit the API through the built-in proxy:

```
kubectl get --raw /api/v1/namespaces/kcna-lab01/pods/recon-shell | head -c 300; echo
{"kind":"Pod","apiVersion":"v1","metadata":{"name":"recon-shell","namespace":"kcna-lab01","uid":"8f1cb0f2-6d9b-4b8f-9f0a-4b0c8a1f2d31","resourceVersion":"4127","creationTimestamp":"2026-09-05T07:45:09Z","labels":{"app.kubernetes.io/name":"recon-shell",
```

Step 10 — Diff the cluster against Meridian's baseline

```
kubectl api-resources --api-group='' --no-headers | awk '{print $1}' | sort > /tmp/live-core.txt
grep -v '^#' data/core-api-inventory.tsv | cut -f1 | sort > /tmp/baseline-core.txt
comm -23 /tmp/baseline-core.txt /tmp/live-core.txt
```

Empty output is the pass condition: nothing Meridian expects is missing. Look at the other direction:

```
comm -13 /tmp/baseline-core.txt /tmp/live-core.txt
bindings
endpoints
```

`endpoints` is served but excluded from the baseline on purpose: **v1 Endpoints was deprecated in Kubernetes v1.33** in favour of `discovery.k8s.io/v1 EndpointSlice`. You will use `EndpointSlice` on Day 3.

Step 11 — Look at what the control plane has been doing

```
kubectl -n kcna-lab01 get events --sort-by=.lastTimestamp

LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
2m          Normal    Scheduled    pod/recon-shell Successfully assigned kcna-lab01/recon-shell to kcna-qa-20260905-control-plane
2m          Normal    Pulled       pod/recon-shell Container image "busybox:1.36" already present on machine
2m          Normal    Created      pod/recon-shell Created container: shell
2m          Normal    Started      pod/recon-shell Started container shell
```

Events are ordinary API objects with a TTL (one hour by default). They are the first place to look when something did not happen.

Lab 01 · Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script asserts, in order: API reachability; the namespace, ResourceQuota and LimitRange; the ConfigMap and its key and content; the Pod's phase, readiness and log evidence that the mounted CSV was read; the full baseline diff against `data/core-api-inventory.tsv`; that `kubectl explain` returns live schema; and that `--v=6` prints the expected REST path.

Expected tail:

```
Result: 13 passed, 0 failed
```

The full annotated evidence is in `verification/expected-output.md`.

Lab 01 · Failure injection — the Pod that never starts

Do this deliberately. Delete the ConfigMap and recreate the Pod so the volume source is missing.

```
kubectl -n kcna-lab01 delete pod recon-shell --ignore-not-found
kubectl -n kcna-lab01 delete configmap recon-checklist --ignore-not-found
kubectl apply -f manifests/20-recon-shell.yaml

pod/recon-shell deleted
configmap "recon-checklist" deleted
pod/recon-shell created
```

Now watch:

```
kubectl -n kcna-lab01 get pod recon-shell -w
```

NAME	READY	STATUS	RESTARTS	AGE
recon-shell	0/1	ContainerCreating	0	5s
recon-shell	0/1	ContainerCreating	0	35s

Press **Ctrl+C**. The Pod is stuck. `kubectl logs` is useless here — there is no container yet:

```
kubectl -n kcna-lab01 logs recon-shell

Error from server (BadRequest): container "shell" in pod "recon-shell" is waiting to
start: ContainerCreating
```

The evidence lives in the events, so describe the Pod:

```
kubectl -n kcna-lab01 describe pod recon-shell | tail -n 12
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	62s	default-scheduler	Successfully assigned kcna-lab01/recon-shell to kcna-qa-20260905-control-plane
Warning	FailedMount	30s (x8 over 62s)	kubelet	MountVolume.SetUp failed for volume "checklist" : configmap "recon-checklist" not found
Warning	FailedMount	17s	kubelet	Unable to attach or mount volumes: unmounted volumes=[checklist], unattached volumes=[], failed to process volumes=[]: timed out waiting for the condition

Diagnosis

Read the three facts in order:

1. `Scheduled` succeeded — so this is **not** a scheduling problem. The API server accepted the object and the scheduler placed it on a node. Nothing is wrong with resources, taints or selectors.
2. The failure is reported **by kubelet**, not by the scheduler or the API server. The node is trying and failing to build the container's filesystem.
3. `MountVolume.SetUp failed ... configmap "recon-checklist" not found` names the volume (`checklist`) and the missing object exactly.

The API server accepted a Pod that references a ConfigMap which does not exist. **Kubernetes does not validate cross-object references at admission time.** The object is syntactically valid; the reconciliation loop just cannot converge. It will keep retrying forever, and it will succeed the moment the ConfigMap appears — you do **not** need to recreate the Pod.

Fix it without touching the Pod:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv

configmap/recon-checklist created

kubectl -n kcna-lab01 wait --for=condition=Ready pod/recon-shell --timeout=120s
kubectl -n kcna-lab01 get pod recon-shell

pod/recon-shell condition met
NAME          READY   STATUS    RESTARTS   AGE
recon-shell   1/1     Running   0           2m14s
```

Note the `AGE` — it never restarted. The kubelet simply retried the mount. This is the level-triggered, eventually-consistent behaviour that defines Kubernetes controllers, and it is worth more to your PoC than any diagram.

Lab 01 · Troubleshooting

Symptom	Likely cause	Fix
The connection to the server localhost:8080 was refused	No kubeconfig found — <code>\$KUBECONFIG</code> unset and <code>~/.kube/config</code> absent	<code>export KUBECONFIG=/path/to/kubeconfig</code> , then re-run <code>kubectl version</code>
Error from server (NotFound): pods "recon-shell" not found	You omitted <code>-n kcna-lab01</code> ; the request went to namespace default	Add <code>-n kcna-lab01</code> , or set a default with <code>kubectl config set-context --current --namespace=kcna-lab01</code>
error: field "container" does not exist from kubectl explain	Singular/plural or path typo	Walk the path one level at a time: <code>kubectl explain pod.spec</code> first, then pick the exact field name from the <code>FIELDS</code> list
Pod stuck <code>ContainerCreating</code> , logs returns <code>BadRequest ... waiting to start</code>	Volume source (ConfigMap/Secret/PVC) missing or misnamed	<code>kubectl -n kcna-lab01 describe pod <name></code> and read the <code>FailedMount</code> event; create the missing object
error: exec plugin: invalid apiVersion "client.authentication.k8s.io/v1alpha1"	kubeconfig written by a much older CLI than your kubectl	Regenerate the kubeconfig with the current tool, or upgrade/downgrade kubectl to within one minor of the server

Symptom	Likely cause	Fix
Error from server (Forbidden): pods is forbidden: User "... cannot list resource "pods"	RBAC — your identity has no permission in that namespace	kubectl auth can-i list pods -n kcna-lab01; ask for a Role/RoleBinding (covered in Lab 14)
kubectl api-resources prints error: unable to retrieve the complete list of server APIs	An APIService (usually a metrics or custom-API extension) is unavailable	kubectl get apiservices grep -v True to find the broken one; the rest of the output is still valid
ConfigMap created but the Pod sees an empty directory	--from-file pointed at a directory, or the key name does not match the file you cat	kubectl -n kcna-lab01 get cm recon-checklist -o yaml and check the keys under data:

Lab 01 · Cleanup

Delete **only** this lab's namespace. Everything created above is namespaced, so this removes all of it.

```
kubectl delete namespace kcna-lab01
namespace "kcna-lab01" deleted
kubectl get namespace kcna-lab01
Error from server (NotFound): namespaces "kcna-lab01" not found
```

Do not run `kubectl delete` without `-n/--namespace` scoping, and never with `--all-namespaces`.

Lab 01 · What you learned

- A **context** binds a cluster, a user and (optionally) a default namespace. `kubectl config view --minify` is the fastest way to answer "where am I?".
- The API is organised into **groups**; the historical *core* group has an empty name and lives at `/api/v1`, while every named group lives at `/apis/<group>/<version>`.
- `kubectl api-resources` is the authoritative map of a cluster's kinds, including the **namespaced vs cluster-scoped** distinction that governs multi-tenant design.
- `kubectl explain` reads the **live OpenAPI schema**, so it is never out of date for the cluster in front of you.
- `--v=6` and `kubectl get --raw` show that `kubectl` is a thin, well-behaved **HTTP client**; anything it can do, a controller or a CI job can do.
- `kubectl get -o yaml` returns desired **plus** defaulted **plus** observed state. Only the first of those belongs in Git.
- Missing cross-object references are **not** rejected at admission. They surface as kubelet events and resolve themselves when the referenced object appears.

Lab 01 · Further reading

- [Command line tool \(kubectl\)](https://kubernetes.io/docs/reference/kubectl/) <https://kubernetes.io/docs/reference/kubectl/>
- [kubectl Quick Reference](https://kubernetes.io/docs/reference/kubectl/quick-reference/) <https://kubernetes.io/docs/reference/kubectl/quick-reference/>
- [Configure Access to Multiple Clusters](https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/) <https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/>

- [Kubernetes API Concepts](https://kubernetes.io/docs/reference/using-api/api-concepts/) <https://kubernetes.io/docs/reference/using-api/api-concepts/>
- [API Groups and Versioning](https://kubernetes.io/docs/reference/using-api/#api-groups) <https://kubernetes.io/docs/reference/using-api/#api-groups>
- [Namespaces](https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/) <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
- [Resource Quotas](https://kubernetes.io/docs/concepts/policy/resource-quotas/) and [Limit Ranges](https://kubernetes.io/docs/concepts/policy/limit-range/) <https://kubernetes.io/docs/concepts/policy/resource-quotas/>
<https://kubernetes.io/docs/concepts/policy/limit-range/>
- [Kubernetes Object Management](https://kubernetes.io/docs/concepts/overview/working-with-objects/object-management/) <https://kubernetes.io/docs/concepts/overview/working-with-objects/object-management/>
- [Endpoints deprecation \(v1.33\)](https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/) <https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>

Lab 02 — Authoring Your First Pod Manifest

Field	Value
Lab ID	Lab 02
Title	Authoring Your First Pod Manifest
Day / Topic	Day 1 · Core Concepts
Duration	45 minutes
Namespace	kcna-lab02
Learning outcome	L01 — Develop a Kubernetes architectural proof of concept.
Ability	A1 Develop an architectural proof of concept
Knowledge	K1 Process for developing proof of concepts
Deck slide	Slide 92
Repository path	courseware/labs/lab-02-first-pod-manifest/

Goal. Hand-author a valid Pod manifest field by field — using `kubectl explain` and `--dry-run=client` as the authoring tools — then run it, serve real content from a ConfigMap, and diagnose an `ImagePullBackOff` caused by a single mistyped tag.

What you will produce:

- A depot-web Pod in `kcna-lab02` serving `data/depot-status.html` and `data/depots.csv` from the ConfigMap `depot-content`
- A learner-authored manifest in `/tmp` that round-trips cleanly against `manifests/20-depot-web-pod.yaml`
- A documented `ImagePullBackOff` diagnosis with the exact kubelet Failed event text

Lab 02 · Objective

By the end of this lab you will be able to:

- Write a **Pod manifest from scratch** and explain every field you typed.
- Use `kubectl explain` and `kubectl run --dry-run=client -o yaml` as **authoring tools** rather than guessing YAML.
- Distinguish what **you** declared from what the **API server defaulted** and what the **kubelet observed**.
- Serve real content into a container from a **ConfigMap volume**.
- Explain precisely what `containerPort` does — and, more importantly, what it does *not* do.
- Recognise, reproduce and diagnose `ErrImagePull` / `ImagePullBackOff`.

Lab 02 · Prerequisites

- Kubernetes v1.30+ cluster (single-node kind is fine) and `kubectl` on PATH.
- Lab 01 completed, or equivalent comfort with `kubectl get/describe`.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0

kubectl get nodes

NAME                                STATUS    ROLES    AGE   VERSION
kcna-qa-20260905-control-plane    Ready    control-plane    58m   v1.37.0

cd courseware/labs/lab-02-first-pod-manifest
```

Lab 02 · Scenario

Meridian Freight Pte Ltd has approved the Kubernetes proof of concept from Lab 01. The first workload to be modelled is the smallest, least risky piece of the Depot Portal: the **static status page** that depot supervisors load before their shift. It shows the depot register (`data/depots.csv`) and a build banner.

Your job in this lab is to express that workload as a **Pod** — the smallest deployable unit in Kubernetes — and to be able to defend every line of the manifest at the architecture review. No Deployment yet; that comes on Day 2. A bare Pod first, so that you understand what a Deployment is actually creating.

Lab 02 · Step-by-step procedure

Step 1 — Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml

namespace/kcna-lab02 created
```

Make it your default for the rest of the lab so you stop typing `-n`:

```
kubectl config set-context --current --namespace=kcna-lab02

Context "kind-kcna-qa" modified.
```

Every command below still shows `-n kcna-lab02` explicitly, so the lab works either way. Remember to set it back to `default` at cleanup.

Step 2 — Look at the dataset you are going to serve

```
head -n 4 data/depots.csv

depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28

grep -c '^MF-' data/depots.csv

10
```

Step 3 — Turn the dataset into a ConfigMap

nginx serves whatever is in `/usr/share/nginx/html`. We will project two files there. Preview the object first:

```
kubectl -n kcna-lab02 create configmap depot-content \
  --from-file=index.html=data/depot-status.html \
  --from-file=depots.csv=data/depots.csv \
  --dry-run=client -o yaml | head -n 8
```

```

apiVersion: v1
data:
  depots.csv: |
    depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
    MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
    MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
    MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28
    MF-SIN-04,Pasir Panjang Consolidation,Singapore South,32,yes,3x8,2026-08-19

```

--from-file=<key>=<path> lets you **rename the key**. Without the `index.html=` prefix the key would have been `depot-status.html`, and nginx would return 403 because there would be no index file. Create it for real:

```

kubectl -n kcna-lab02 create configmap depot-content \
  --from-file=index.html=data/depot-status.html \
  --from-file=depots.csv=data/depots.csv

configmap/depot-content created

kubectl -n kcna-lab02 get configmap depot-content \
  -o go-template='{{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{end}}'

depots.csv
index.html

```

Step 4 — Ask the API what a Pod may contain

Do not open a browser. Ask the cluster:

```

kubectl explain pod

GROUP:
KIND:    Pod
VERSION: v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <ObjectMeta>
  spec          <PodSpec>
  status        <PodStatus>

```

Those five fields are universal. **status is written by the control plane** — you never author it.

```

kubectl explain pod.spec.containers | head -n 24

GROUP:
KIND:    Pod
VERSION: v1

FIELD: containers <[]Container>

DESCRIPTION:
  List of containers belonging to the pod. Containers cannot currently be
  added or removed. There must be at least one container in a Pod. Cannot be
  updated.
  Container represents a single container that is expected to be run within a
  pod.

FIELDS:
  args    <[]string>
  command <[]string>
  env     <[]EnvVar>
  envFrom <[]EnvFromSource>

```



```

image <string>
imagePullPolicy <string>
lifecycle <Lifecycle>
livenessProbe <Probe>
name <string>
ports <[]ContainerPort>

```

Note <[]Container> — a list. Two of these fields are required:

```

kubectl explain pod.spec.containers.name

GROUP:
KIND:    Pod
VERSION: v1

FIELD: name <string> -required-

DESCRIPTION:
  Name of the container specified as a DNS_LABEL. Each container in a pod must
  have a unique name (DNS_LABEL). Cannot be updated.

```

And the field that will bite you in section 6:

```

kubectl explain pod.spec.containers.ports.containerPort

GROUP:
KIND:    Pod
VERSION: v1

FIELD: containerPort <integer> -required-

DESCRIPTION:
  Number of port to expose on the pod's IP address. This must be a valid port
  number, 0 < x < 65536.

```

Step 5 — Generate a skeleton, then finish it by hand

`kubectl run` with `--dry-run=client` never contacts the cluster to create anything; it renders the object the generator would have POSTed:

```

kubectl run depot-web --image=nginx:1.27-alpine --port=80 \
  --dry-run=client -o yaml

apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: depot-web
  name: depot-web
spec:
  containers:
  - image: nginx:1.27-alpine
    name: depot-web
    ports:
    - containerPort: 80
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}

```

This is a **starting point, not a deliverable**. Three defects for a production PoC: `resources: {}`, a meaningless `run: label`, and no volume. Save it and fix it:

```

kubectl run depot-web --image=nginx:1.27-alpine --port=80 \
  --dry-run=client -o yaml > /tmp/my-depot-web.yaml

```

Now open `/tmp/my-depot-web.yaml` in your editor and make it look like this (type it; do not copy-paste if you can avoid it):

```
apiVersion: v1
kind: Pod
metadata:
  name: depot-web
  namespace: kcna-lab02
  labels:
    app.kubernetes.io/name: depot-web
    app.kubernetes.io/component: frontend
    app.kubernetes.io/part-of: depot-portal
    app.kubernetes.io/version: 0.1.0-poc
spec:
  containers:
    - name: web
      image: nginx:1.27-alpine
      imagePullPolicy: IfNotPresent
      ports:
        - name: http
          containerPort: 80
          protocol: TCP
      volumeMounts:
        - name: content
          mountPath: /usr/share/nginx/html
          readOnly: true
      resources:
        requests:
          cpu: 50m
          memory: 64Mi
        limits:
          cpu: 200m
          memory: 128Mi
  volumes:
    - name: content
      configMap:
        name: depot-content
```

Field-by-field, what you just wrote:

Field	Why it is there
<code>apiVersion: v1</code>	Pod lives in the core group, so no group prefix
<code>metadata.namespace</code>	pins the object to this lab; a manifest that relies on the ambient namespace is not reproducible
<code>metadata.labels</code>	the <code>app.kubernetes.io/*</code> set is the recommended label convention; selectors on Day 2 depend on them < https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/ >
<code>containers[].name</code>	required, DNS-label, unique in the Pod; it is what <code>kubectl logs -c</code> takes
<code>image</code> with a real tag	never <code>:latest</code> — see Lab 05 for why
<code>imagePullPolicy: IfNotPresent</code>	with a fixed tag there is nothing to re-pull
<code>ports[].name: http</code>	lets probes and Services reference the port by name
<code>resources</code>	without <code>requests</code> the scheduler is flying blind; without <code>limits</code> one Pod can starve a node
<code>volumes[].configMap</code>	projects the ConfigMap keys into the container filesystem as files

Step 6 — Validate before you apply

Client-side (schema only, no network):

```
kubectl apply -f /tmp/my-depot-web.yaml --dry-run=client
pod/depot-web created (dry run)
```

Server-side (full admission chain, defaulting and validation — but nothing is persisted):

```
kubectl apply -f /tmp/my-depot-web.yaml --dry-run=server
pod/depot-web created (server dry run)
```

`--dry-run=server` is the stronger check: it runs admission webhooks, quota and Pod Security Admission. Use it in CI.

Deliberately break it once to see validation fire. Change `containerPort: 80` to `containerPort: 80000` and re-run:

```
sed 's/containerPort: 80$/containerPort: 80000/' /tmp/my-depot-web.yaml \
| kubectl apply -f - --dry-run=server

The Pod "depot-web" is invalid: spec.containers[0].ports[0].containerPort: Invalid value:
80000: must be between 1 and 65535, inclusive
```

Note the **field path** `spec.containers[0].ports[0].containerPort`. Kubernetes validation errors always tell you exactly where to look.

Step 7 — Apply the reference manifest and inspect the result

```
kubectl apply -f manifests/20-depot-web-pod.yaml
pod/depot-web created

kubectl -n kcna-lab02 get pod depot-web -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS GATES				
depot-web	1/1	Running	0	18s	10.244.0.11	kcna-qa-20260905-control-plane
	<none>		<none>			

Read `READY 1/1`: one container, one ready. Readiness comes from the `readinessProbe` in the reference manifest — `httpGet /index.html` on the port **named** `http`.

Now compare what you wrote with what the server stored:

```
kubectl -n kcna-lab02 get pod depot-web \
-o jsonpath='{.spec.dnsPolicy}{"\t"}{.spec.serviceAccountName}{"\t"}{.spec.schedulerName}{"\n"}'
ClusterFirst      default      default-scheduler
```

You wrote none of those. The API server **defaulted** them at admission time. The scheduler then wrote `.spec.nodeName`, and the kubelet wrote `.status`:

```
kubectl -n kcna-lab02 get pod depot-web \
-o jsonpath='{range .status.conditions[*]}{.type}={.status}{"\n"}{end}'

PodReadyToStartContainers=True
Initialized=True
Ready=True
ContainersReady=True
PodScheduled=True
```

Step 8 — Understand containerPort

`containerPort` publishes nothing and blocks nothing. Prove it. The reference manifest declares port 80 only. Reach a port that was **never declared**:

```
kubectl -n kcna-lab02 exec depot-web -c web -- \
  sh -c 'wget -qO- http://127.0.0.1:80/index.html | head -n 2'

<!DOCTYPE html>
<html lang="en">
```

Now reach the Pod IP from another Pod, on a port you did declare, and note that nothing about the declaration was load-bearing — nginx opened 80 because its own configuration told it to, not because your YAML did:

```
POD_IP=$(kubectl -n kcna-lab02 get pod depot-web -o jsonpath='{.status.podIP}')
echo "$POD_IP"

10.244.0.11

kubectl -n kcna-lab02 run curl-probe --rm -it --restart=Never \
  --image=busybox:1.36 --command -- wget -qO- "http://${POD_IP}/depots.csv"

depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28
MF-SIN-04,Pasir Panjang Consolidation,Singapore South,32,yes,3x8,2026-08-19
MF-JHB-01,Senai Cross-Dock,Johor,20,no,2x12,2026-05-30
MF-JHB-02,Pasir Gudang Yard,Johor,28,yes,3x8,2026-07-21
MF-KUL-01,Port Klang North Gate,Selangor,36,yes,3x8,2026-08-11
MF-BKK-01,Laem Chabang Transit,Chonburi,18,no,2x12,2026-04-17
MF-HAN-01,Hai Phong Bonded Store,Hai Phong,22,yes,3x8,2026-06-09
MF-JKT-01,Tanjung Priok Staging,Jakarta,26,no,2x12,2026-07-03
pod "curl-probe" deleted
```

So what is `containerPort` for? Three real uses: it names a port so probes and Services can use `port: http` instead of a magic number; it feeds `kubectl describe`/tooling and dashboards; and `kubectl port-forward` and some service meshes read it. It is *documentation with a name attached*.

Step 9 — Reach the Pod from your workstation

```
kubectl -n kcna-lab02 port-forward pod/depot-web 8080:80

Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Leave that running. In a **second terminal**:

```
curl -s http://127.0.0.1:8080/ | grep STATUS

<p class="ok">STATUS: OK - served from a Pod in namespace kcna-lab02</p>

curl -s http://127.0.0.1:8080/depots.csv | wc -l

11
```

Return to the first terminal and press `Ctrl+C` to stop the forward.

```
Handling connection for 8080
Handling connection for 8080
^C
```

`port-forward` is a debugging tool. It tunnels over the API server, it is single-user and it dies with your terminal. It is not how you expose a service — that is Day 3.

Lab 02 · Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace; that `depot-content` exists with **both** keys and that its `depots.csv` really carries the ten rows from `data/depots.csv`; that `depot-web` is Running, Ready, pinned to `nginx:1.27-alpine`, declares `containerPort: 80`, and has explicit requests **and** limits; that HTTP GETs for `/index.html` and `/depots.csv` return the seeded content; that server-side defaulting happened; and — if you have not cleaned it up yet — that `depot-web-badtag` is in `ImagePullBackOff`.

Expected tail:

```
Result: 14 passed, 0 failed
```

Full annotated evidence: `verification/expected-output.md`.

Lab 02 · Failure injection — one character, one broken deployment

`manifests/90-depot-web-badtag.yaml` is identical to a working Pod except for the image tag: `nginx:1.27-alpin` (the trailing `e` is missing). Apply it:

```
kubectl apply -f manifests/90-depot-web-badtag.yaml
pod/depot-web-badtag created
```

The API server accepted it. Schema validation cannot know which tags exist in a registry. Watch what happens next:

```
kubectl -n kcna-lab02 get pod depot-web-badtag
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ErrImagePull	0	8s

Wait about 40 seconds and look again:

```
kubectl -n kcna-lab02 get pod depot-web-badtag
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ImagePullBackOff	0	45s

`kubectl logs` cannot help — no container was ever created:

```
kubectl -n kcna-lab02 logs depot-web-badtag
```

Error from server (BadRequest): container "web" in pod "depot-web-badtag" is waiting to start: trying and failing to pull image

The evidence is in the events:

```
kubectl -n kcna-lab02 describe pod depot-web-badtag | tail -n 9
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	62s	default-scheduler	Successfully assigned kcna-lab02/depot-web-badtag to kcna-qa-20260905-control-plane
Normal	Pulling	22s (x3 over 62s)	kubelet	Pulling image "nginx:1.27-alpin"
Warning	Failed	21s (x3 over 61s)	kubelet	Failed to pull image "nginx:1.27-alpin": failed to pull and unpack image "docker.io/library/nginx:1.27-alpin": failed to resolve reference "docker.io/library/nginx:1.27-alpin": docker.io/library/nginx:1.27-alpin: not found
Warning	Failed	21s (x3 over 61s)	kubelet	Error: ErrImagePull
Normal	BackOff	8s (x4 over 60s)	kubelet	Back-off pulling image "nginx:1.27-alpin"
Warning	Failed	8s (x4 over 60s)	kubelet	Error: ImagePullBackOff

Diagnosis

Four things to read, in this order:

1. **Scheduled succeeded.** Not a scheduling problem — no taint, selector or resource issue.
2. **Pulling image "nginx:1.27-alpin"** — the kubelet is echoing the reference *you* asked for. Compare it character by character with what you meant. This is where the bug is visible.
3. **failed to resolve reference "docker.io/library/nginx:1.27-alpin": ...: not found.** Containerd expanded the short name to its fully-qualified form (`docker.io/library/...` — see Lab 05) and the registry returned a 404 for that tag. `not found` means the *tag* does not exist. Contrast with 401 `Unauthorized`, which would mean a credentials problem, and `x509: certificate signed by unknown authority`, which would mean a TLS/proxy problem.
4. **ErrImagePull then ImagePullBackOff.** These are two stages of the same thing. `ErrImagePull` is a single failed attempt. After repeated failures the kubelet switches to `ImagePullBackOff` and applies exponential back-off (10s, 20s, 40s ... capped at 5 minutes), which is why the status flips and the `x3/x4` counters in the events climb slowly.

Note `RESTARTS 0` throughout: nothing restarted, because nothing ever started.

Repair

A Pod's `image` field **is** mutable, so you can patch in place:

```
kubectl -n kcna-lab02 set image pod/depot-web-badtag web=nginx:1.27-alpine
pod/depot-web-badtag image updated
kubectl -n kcna-lab02 get pod depot-web-badtag -w
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ImagePullBackOff	0	2m10s
depot-web-badtag	0/1	ContainerCreating	0	2m38s
depot-web-badtag	1/1	Running	0	2m40s

Press `Ctrl+C`. The kubelet noticed the spec change, resolved the correct tag and started the container — again without you deleting anything.

In real life you almost never patch a bare Pod: you fix the manifest in Git and let a Deployment roll it out. That is Lab 07.

Remove the injected Pod when you are done (namespace-scoped, by name):

```
kubectl -n kcna-lab02 delete pod depot-web-badtag
pod "depot-web-badtag" deleted
```

Lab 02 · Troubleshooting

Symptom	Likely cause	Fix
error: error validating "...": ... unknown field "spec.container"	Field name typo (singular/plural, wrong nesting level)	Check the exact path with <code>kubectl explain pod.spec</code> ; re-run <code>kubectl apply --dry-run=server</code> before applying
The Pod "depot-web" is invalid: spec.containers[0].ports[0].containerPort: Invalid value: 80000	Value outside the schema's allowed range	Read the field path in the error; fix that exact field

Symptom	Likely cause	Fix
ErrImagePull / ImagePullBackOff, event says not found	Tag or repository name is wrong	Correct the tag; verify with <code>crictl images</code> or by pulling on your workstation
ImagePullBackOff, event says 401 Unauthorized or pull access denied	Private registry with no <code>imagePullSecrets</code>	Create a <code>kubernetes.io/dockerconfigjson</code> Secret and reference it in <code>spec.imagePullSecrets</code> (Lab 15)
Pod Running but READY 0/1	<code>readinessProbe</code> failing — wrong path, port or the app is slow to start	<code>kubectl describe pod</code> → look for Readiness probe failed:; raise <code>initialDelaySeconds</code> or fix the path
nginx returns 403 Forbidden	ConfigMap mounted but no key named <code>index.html</code>	Recreate with <code>--from-file=index.html=...</code> ; confirm with <code>kubectl get cm depot-content -o yaml</code>
Error from server (BadRequest): container "web" ... is waiting to start from kubectl logs	There is no container yet, so there is no log stream	Use <code>kubectl describe pod</code> and read the Events; logs only works once a container has been created
unable to forward port because pod is not running	port-forward started against a Pod that is not Running	Wait for <code>kubectl wait --for=condition=Ready pod/depot-web</code> , then retry
Pod stuck Pending with FailedScheduling ... Insufficient cpu	Your requests exceed the node's allocatable	Lower <code>resources.requests</code> ; check <code>kubectl describe node</code>

Lab 02 · Cleanup

Reset your default namespace first if you changed it in Step 1:

```
kubectl config set-context --current --namespace=default
Context "kind-kcna-qa" modified.
```

Then delete only this lab's namespace:

```
kubectl delete namespace kcna-lab02
namespace "kcna-lab02" deleted
```

Lab 02 · What you learned

- A Pod manifest is four authored fields — `apiVersion`, `kind`, `metadata`, `spec` — plus a `status` you never write.
- `kubectl explain` and `--dry-run=client -o yaml` turn manifest authoring from recall into discovery. `--dry-run=server` additionally runs admission.
- Generators produce **skeletons**: no resources, no volumes, poor labels. Always finish the file by hand.
- The API server **defaults** fields (`dnsPolicy`, `serviceAccountName`, `schedulerName`, `restartPolicy`) at admission; the scheduler and kubelet then write more. `kubectl get -o yaml` is desired + defaulted + observed.

- `containerPort` is **named documentation**. It does not open, publish or restrict anything; the process inside the container decides what it listens on.
 - A ConfigMap volume projects keys as files, and `--from-file=<key>=<path>` lets you control the filename.
 - `ErrImagePull` is one failure; `ImagePullBackOff` is the back-off state after several. `logs` is useless before a container exists — `describe` is the tool.
-

Lab 02 · Further reading

- [Pods](https://kubernetes.io/docs/concepts/workloads/pods/) <https://kubernetes.io/docs/concepts/workloads/pods/>
- [Pod Lifecycle](https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/) <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- [Kubernetes Objects — Understanding Kubernetes objects](https://kubernetes.io/docs/concepts/overview/working-with-objects/) <https://kubernetes.io/docs/concepts/overview/working-with-objects/>
- [Recommended Labels](https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/) <https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>
- [Images and imagePullPolicy](https://kubernetes.io/docs/concepts/containers/images/) <https://kubernetes.io/docs/concepts/containers/images/>
- [Configure a Pod to Use a ConfigMap](https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/) <https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/>
- [Managing Resources for Containers](https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/) <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- [Use Port Forwarding to Access Applications in a Cluster](https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/) <https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>
- [Debug Pods](https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/) <https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/>

Lab 03 — Commands, Arguments and Environment

Field	Value
Lab ID	Lab 03
Title	Commands, Arguments and Environment
Day / Topic	Day 1 · Core Concepts
Duration	40 minutes
Namespace	kcna-lab03
Learning outcome	LO1 — Develop a Kubernetes architectural proof of concept.
Ability	A1 Develop an architectural proof of concept
Knowledge	K1 Process for developing proof of concepts
Deck slide	Slide 105
Repository path	courseware/labs/lab-03-commands-args-env/

Goal. Configure a container entirely from outside its image — override ENTRYPOINT and CMD with command/args, inject settings with env, envFrom and the downward API, and mount a JSON dataset — then diagnose a CrashLoopBackOff caused by one missing variable.

What you will produce:

- An entrypoint-override matrix of three Pods proving how command and args map onto image ENTRYPOINT and CMD
- A booking-config Pod driven by ConfigMaps built from data/route-defaults.env and data/tariff-rates.json
- A CrashLoopBackOff diagnosis read from kubectl logs --previous and lastState.terminated.exitCode

Lab 03 · Objective

By the end of this lab you will be able to:

- State exactly how `spec.containers[].command` and `spec.containers[].args` map onto a container image's **ENTRYPOINT** and **CMD**, and prove it by reading logs.
- Inject configuration with `env`, `envFrom` (including `prefix` and `optional`), `configMapKeyRef`, and the **downward API** (`fieldRef`, `resourceFieldRef`).
- Build ConfigMaps from a real `.env` file and a real JSON file, and mount the JSON into a container.
- Explain where `$(VAR)` expansion happens and what it can and cannot reference.
- Diagnose a **CrashLoopBackOff** from a missing variable using `kubectl logs --previous` and `lastState.terminated.exitCode`.

Lab 03 · Prerequisites

- Kubernetes v1.30+ cluster; `kubectl` on PATH.
- Labs 01 and 02 completed.

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	72m	v1.37.0

```
cd courseware/labs/lab-03-commands-args-env
```

Lab 03 · Scenario

Meridian Freight Pte Ltd runs one container image for the Depot Portal booking API, but it must run in four places: the Singapore West depots, the Johor cross-docks, the staging environment and a developer laptop. Today the team solves this by baking a `config.properties` into the image and rebuilding it four times — four images, four SHAs, four things to audit.

The PoC has to demonstrate the cloud native answer: **one immutable image, configuration supplied at run time**. `data/route-defaults.env` is the settings file the operations team maintains, and `data/tariff-rates.json` is the pricing table that finance publishes monthly. Neither may ever be baked into an image.

Lab 03 · Step-by-step procedure

Step 1 — Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab03 created
```

Step 2 — The ENTRYPOINT / CMD rule, before you run anything

Every OCI image can carry two defaults:

- **ENTRYPOINT** — the executable.
- **CMD** — the default arguments to that executable (or, if there is no ENTRYPOINT, the whole command).

Kubernetes overrides them with two *different* field names. This mismatch is the single most common confusion in the KCNA syllabus:

You set in the Pod spec	Container runs	Image ENTRYPOINT	Image CMD
neither <code>command</code> nor <code>args</code>	ENTRYPOINT CMD	used	used
<code>args</code> only	ENTRYPOINT <code>args</code>	used	replaced
<code>command</code> only	<code>command</code>	replaced	discarded
both <code>command</code> and <code>args</code>	<code>command args</code>	replaced	replaced

Memorise the mapping: `command` → **ENTRYPOINT**, `args` → **CMD**. And note the third row: setting `command` alone silently throws the image's CMD away.

Confirm the field names against the live schema:

```
kubectl explain pod.spec.containers.command
```

```
GROUP:
KIND:    Pod
VERSION: v1

FIELD: command <[]string>
```

DESCRIPTION:

Entrypoint array. Not executed within a shell. The container image's ENTRYPOINT is used if this is not provided. Variable references \$(VAR_NAME) are expanded using the container's environment.

Two facts in that text you must not skip:

1. **"Not executed within a shell."** `command: ["echo hello && echo world"]` does not work; there is no shell to interpret `&&`. If you need shell syntax you must ask for one explicitly: `command: ["/bin/sh", "-c"]`.
2. **"Variable references \$(VAR_NAME) are expanded"** — by the kubelet, before the process starts. Not `$(VAR)`; not backticks. Only `$(VAR_NAME)`.

Step 3 — Run the override matrix

```
kubectl apply -f manifests/10-entrypoint-matrix.yaml
pod/em-a-image-defaults created
pod/em-b-args-only created
pod/em-c-command-override created

kubectl -n kcna-lab03 get pods
```

NAME	READY	STATUS	RESTARTS	AGE
em-a-image-defaults	1/1	Running	0	31s
em-b-args-only	0/1	Completed	0	31s
em-c-command-override	0/1	Completed	0	31s

Case A — neither field set. The image's ENTRYPOINT (`/docker-entrypoint.sh`) runs with the image's CMD (`nginx -g "daemon off;"`), so nginx serves and the Pod stays Running:

```
kubectl -n kcna-lab03 logs em-a-image-defaults | head -n 3
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
```

Case B — args: ["nginx", "-v"], no command. The ENTRYPOINT is untouched; only the arguments changed:

```
kubectl -n kcna-lab03 logs em-b-args-only
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of
/etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
/etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
nginx version: nginx/1.27.5
```

The banner is the ENTRYPOINT; `nginx version: is your args` running instead of the CMD. The container then exits 0 — hence Completed, because this Pod uses `restartPolicy: Never`.

Case C — command set. The ENTRYPOINT is gone:

```
kubectl -n kcna-lab03 logs em-c-command-override
```

```

case C: image ENTRYPOINT was replaced by spec.command
case C: pid 1 is now: /bin/sh -c echo "case C: image ENTRYPOINT was replaced by
spec.command" ...
case C: no /docker-entrypoint.sh banner above == proof

```

No banner. That absence is the evidence: `/docker-entrypoint.sh` never ran, so any configuration work it would have done (IPv6 listen, template substitution, worker tuning) silently did not happen. This is why overriding `command` on an unfamiliar image is risky — you may be skipping its initialisation.

Inspect what the API stored for each case:

```

kubectl -n kcna-lab03 get pods -o custom-columns=\
'NAME:.metadata.name,COMMAND:.spec.containers[0].command,ARGS:.spec.containers[0].args'

```

NAME	COMMAND	ARGS
em-a-image-defaults	<none>	<none>
em-b-args-only	<none>	[nginx -v]
em-c-command-override	[/bin/sh -c]	[case C: image ENTRYPOINT was replaced by spec.command...]

<none> means "fall back to the image". Kubernetes does not copy the image's ENTRYPOINT into the Pod spec — the runtime resolves it at start-up.

Step 4 — Build a ConfigMap from an env file

```

cat data/route-defaults.env

# Meridian Freight Pte Ltd — Depot Portal booking-service defaults
# Synthetic dataset for KCNA Lab 03.
# Loaded with: kubectl create configmap route-defaults --from-env-file=<this file>
# Every line must be KEY=VALUE. Blank lines and #-comments are ignored.
DEPOT_REGION=sg-west
DEPOT_CODE=MF-SIN-02
BOOKING_WINDOW_MINUTES=45
MAX_TRAILERS_PER_BAY=3
COLD_CHAIN_REQUIRED=true
CURRENCY=SGD
LOG_LEVEL=info
BOOKING_API_HOST=booking.depot-portal.svc.cluster.local
BOOKING_API_PORT=8080

```

`--from-env-file` parses `KEY=VALUE` lines into **separate keys**. Contrast with `--from-file`, which makes **one key holding the whole file**. Preview both:

```

kubectl -n kcna-lab03 create configmap route-defaults \
  --from-env-file=data/route-defaults.env --dry-run=client -o yaml

```

```

apiVersion: v1
data:
  BOOKING_API_HOST: booking.depot-portal.svc.cluster.local
  BOOKING_API_PORT: "8080"
  BOOKING_WINDOW_MINUTES: "45"
  COLD_CHAIN_REQUIRED: "true"
  CURRENCY: SGD
  DEPOT_CODE: MF-SIN-02
  DEPOT_REGION: sg-west
  LOG_LEVEL: info
  MAX_TRAILERS_PER_BAY: "3"
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: route-defaults

```

Nine keys; the comment lines are gone. Note "8080" is quoted — **ConfigMap values are always strings**. Now create both ConfigMaps for real:

```
kubectl -n kcna-lab03 create configmap route-defaults \
--from-env-file=data/route-defaults.env
kubectl -n kcna-lab03 create configmap tariff-rates \
--from-file=data/tariff-rates.json

configmap/route-defaults created
configmap/tariff-rates created

kubectl -n kcna-lab03 get configmaps
```

NAME	DATA	AGE
kube-root-ca.crt	1	4m
route-defaults	9	6s
tariff-rates	1	6s

DATA 9 versus DATA 1 is the whole difference between the two flags.

Step 5 — Deploy the configured booking Pod

```
kubectl apply -f manifests/20-booking-config.yaml
pod/booking-config created

kubectl -n kcna-lab03 wait --for=condition=Ready pod/booking-config --timeout=90s
pod/booking-config condition met

kubectl -n kcna-lab03 logs booking-config

=== 1. envFrom: every key of ConfigMap route-defaults ===
DEPOT_REGION=sg-west
DEPOT_CODE=MF-SIN-02
BOOKING_WINDOW_MINUTES=45
CURRENCY=SGD
LOG_LEVEL=info
=== 2. envFrom with prefix: MF_ ===
MF_DEPOT_CODE=MF-SIN-02
=== 3. explicit env, incl. downward API ===
SERVICE_NAME=booking
POD_NAME=booking-config
NODE_NAME=kcna-qa-20260905-control-plane
MEM_LIMIT_MI=64
=== 4. dependent env var expanded by the kubelet ===
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
=== 5. dataset mounted from ConfigMap tariff-rates ===
TARIFF_TABLE=/etc/depot/tariff-rates.json
tariff entries: 10
rate row for MF-SIN-02:
  { "depot_code": "MF-SIN-02", "bay_hour": 51.00, "cold_chain_surcharge": 21.50,
    "after_hours_multiplier": 1.40 },
=== booking-config ready ===
```

Step 6 — Read the five injection mechanisms in the manifest

Open manifests/20-booking-config.yaml alongside the log above.

Mechanism	Manifest excerpt	Result
envFrom + configMapRef	- configMapRef: {name: route-defaults}	all 9 keys become env vars with their original names
envFrom + prefix	- prefix: MF_	the same 9 keys again as MF_DEPOT_CODE, ... — used to keep two config sources from colliding
envFrom + optional: true	name: route-overrides optional: true	the ConfigMap does not exist and the Pod still starts . Without optional, the Pod would hang in CreateContainerConfigError

Mechanism	Manifest excerpt	Result
env + configMapKeyRef	key: MAX_TRAILERS_PER_BAY → name: TRAILERS_PER_BAY	one key, renamed on the way in
env + fieldRef / resourceFieldRef	metadata.name, spec.nodeName, limits.memory with divisor: 1Mi	the downward API — the Pod learns facts about itself without calling the API server

Verify the dependent-variable expansion. Nothing in the ConfigMap contains that URL; the kubelet built it:

```
kubectl -n kcna-lab03 get pod booking-config \
-o jsonpath='{.spec.containers[0].env[?(@.name=="BOOKING_ENDPOINT")].value}{"\n"}'
http://$(SERVICE_NAME).kcna-lab03.svc.cluster.local:8080
```

The stored spec still contains the literal `$(SERVICE_NAME)`. Expansion is a run-time behaviour of the kubelet, not a mutation of the object. Compare with what the process sees:

```
kubectl -n kcna-lab03 exec booking-config -- env | grep BOOKING_ENDPOINT
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
```

`$(VAR)` can only reference variables defined **earlier in the same `env` list**. It cannot reference anything that arrived through `envFrom`. That is why `SERVICE_NAME` is an explicit `env` entry and not a ConfigMap key.

Look at the full environment:

```
kubectl -n kcna-lab03 exec booking-config -- env | sort | head -n 12
BOOKING_API_HOST=booking.depot-portal.svc.cluster.local
BOOKING_API_PORT=8080
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
BOOKING_WINDOW_MINUTES=45
COLD_CHAIN_REQUIRED=true
CURRENCY=SGD
DEPOT_CODE=MF-SIN-02
DEPOT_REGION=sg-west
HOME=/
HOSTNAME=booking-config
LOG_LEVEL=info
MAX_TRAILERS_PER_BAY=3
```

And the mounted dataset:

```
kubectl -n kcna-lab03 exec booking-config -- head -n 6 /etc/depot/tariff-rates.json
{
  "schema": "meridianfreight/tariff-rates/v1",
  "generated": "2026-09-01T00:00:00Z",
  "currency": "SGD",
  "rates": [
    { "depot_code": "MF-SIN-01", "bay_hour": 42.50, "cold_chain_surcharge": 18.00,
      "after_hours_multiplier": 1.35 },
```

Step 7 — Env vars are a snapshot; mounted files are not

Change a value in the ConfigMap:

```
kubectl -n kcna-lab03 patch configmap route-defaults \
--type merge -p '{"data":{"LOG_LEVEL":"debug"}}'
configmap/route-defaults patched
kubectl -n kcna-lab03 exec booking-config -- env | grep LOG_LEVEL
```

```
LOG_LEVEL=info
```

Still `info`. **Environment variables are injected once, at container start.** They never update. A mounted ConfigMap volume, by contrast, is refreshed by the kubelet (typically within a minute):

```
kubectl -n kcna-lab03 patch configmap tariff-rates \
  --type merge -p '{"data":{"note.txt":"reviewed 2026-09-05"}}'
configmap/tariff-rates patched
```

Wait up to 60 seconds, then:

```
kubectl -n kcna-lab03 exec booking-config -- ls /etc/depot
note.txt
tariff-rates.json
```

The new file appeared without restarting the Pod. This asymmetry drives real architecture decisions: **secrets and hot-reloadable settings go in volumes; start-up-only settings go in env.**

Lab 03 · Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script asserts the namespace; that `route-defaults` has exactly as many keys as `data/route-defaults.env` has `KEY=VALUE` lines and that `tariff-rates` carries the shipped JSON; all three `ENTRYPOINT/CMD` cases including the *absence* of the banner in case C; the `booking-config` log markers for `envFrom`, prefix, dependent expansion, `resourceFieldRef` and the 10 tariff rows; the `optional: true` reference; and the full `CrashLoopBackOff` evidence chain.

Expected tail:

```
Result: 23 passed, 0 failed
```

Full annotated evidence: `verification/expected-output.md`.

Lab 03 · Failure injection — one missing variable, endless restarts

`manifests/90-booking-missing-env.yaml` runs the same preflight script but never sets `TARIFF_TABLE` and never references `route-defaults`.

```
kubectl apply -f manifests/90-booking-missing-env.yaml
pod/booking-missing-env created
```

Watch it for about a minute:

```
kubectl -n kcna-lab03 get pod booking-missing-env -w
```

NAME	READY	STATUS	RESTARTS	AGE
booking-missing-env	0/1	ContainerCreating	0	2s
booking-missing-env	0/1	Error	0	4s
booking-missing-env	0/1	Error	1 (2s ago)	6s
booking-missing-env	0/1	CrashLoopBackOff	1 (5s ago)	11s
booking-missing-env	0/1	Error	2 (16s ago)	27s
booking-missing-env	0/1	CrashLoopBackOff	2 (12s ago)	39s
booking-missing-env	0/1	CrashLoopBackOff	3 (28s ago)	72s

Press `Ctrl+C`. Try the obvious thing:

```
kubectl -n kcna-lab03 logs booking-missing-env
```

```
Error from server (BadRequest): container "app" in pod "booking-missing-env" is waiting to start: CrashLoopBackOff
```

This is the trap. At the moment you ran `logs`, the container was not running — it had already exited and the kubelet was waiting out the back-off. The log stream you want belongs to the *previous* instance:

```
kubectl -n kcna-lab03 logs booking-missing-env --previous
[preflight] booking-api starting
[preflight] FATAL: TARIFF_TABLE is not set - refusing to start
```

There is the cause, in the application's own words. Corroborate with the container status:

```
kubectl -n kcna-lab03 get pod booking-missing-env \
-o jsonpath='{range .status.containerStatuses[0]}.{.state.waiting.reason}{"\t"}
{.lastState.terminated.exitCode}{"\t"}{.restartCount}{"\n"}{end} '
CrashLoopBackOff      1      3

kubectl -n kcna-lab03 describe pod booking-missing-env | sed -n '/Last State/,/Restart
Count/p'
      Last State:      Terminated
        Reason:      Error
        Exit Code:    1
        Started:      Sat, 05 Sep 2026 16:03:11 +0800
        Finished:     Sat, 05 Sep 2026 16:03:11 +0800
        Ready:        False
        Restart Count: 3
```

Diagnosis

- **CrashLoopBackOff is not an error condition; it is a *waiting reason*.** It means "this container has exited repeatedly and I am pausing before the next attempt". The back-off doubles — 10s, 20s, 40s, 80s — capped at 5 minutes.
- The real signal is `lastState.terminated.exitCode: 1` plus `Reason: Error`. Exit 1 is the application deciding to stop. Contrast with exit **137** (SIGKILL, usually the OOM killer, `Reason: OOMKilled`) and **143** (SIGTERM).
- `Started` and `Finished` are the **same second** — the process died during start-up, not under load. That points at configuration, not traffic.
- `--previous` (`-p`) is mandatory for any crash-looping container. Without it `kubectl logs` targets the container that does not exist yet.

Note what did **not** happen: the API server accepted the Pod, the scheduler placed it, the image pulled fine. Nothing upstream of the process complained, because nothing upstream knows this application requires `TARIFF_TABLE`.

Repair

A Pod's `env` is **immutable** — unlike `image`, you cannot patch it:

```
kubectl -n kcna-lab03 set env pod/booking-missing-env TARIFF_TABLE=/etc/depot/tariff-
rates.json

error: Pod "booking-missing-env" is invalid: spec: Forbidden: pod updates may not change
fields other than
`spec.containers[*].image`, `spec.initContainers[*].image`, `spec.activeDeadlineSeconds`, `sp
ec.tolerations` (only additions to existing
tolerations), `spec.terminationGracePeriodSeconds` (allow it to be set to 1 if it was
previously negative)
```


Read that error carefully — it is the definitive list of what you may change on a running Pod. The fix is to replace the object:

```
kubectl -n kcna-lab03 delete pod booking-missing-env  
pod "booking-missing-env" deleted
```

The corrected workload is already running as `booking-config`. In production, a Deployment would have done this delete-and-recreate for you — which is exactly the argument for not managing bare Pods, and the subject of Lab 07.

Run `bash verification/checks.sh` **before** deleting the Pod if you want section 4 of the script to pass rather than skip.

Lab 03 · Troubleshooting

Symptom	Likely cause	Fix
CrashLoopBackOff and kubectl logs returns BadRequest ... waiting to start	You are asking for the logs of a container that is not currently running	<code>kubectl logs <pod> --previous (or -p)</code> ; also read <code>lastState.terminated.exitCode</code>
Pod stuck in CreateContainerConfigError	<code>envFrom/env</code> references a ConfigMap or Secret that does not exist and is not marked <code>optional: true</code>	<code>kubectl describe pod</code> → Error: configmap "X" not found; create it or add <code>optional: true</code>
<code>standard_init_linux.go: exec user process caused: exec format error</code> or <code>command not found</code>	<code>command:</code> was given shell syntax (<code>&&</code> , pipes, globs) but is not executed in a shell	Use <code>command: ["/bin/sh", "-c"]</code> and put the script in <code>args</code>
Container exits immediately with code 0, status Completed	The image CMD is a one-shot program, or your <code>args</code> replaced a long-running CMD	Use a long-running process, or <code>restartPolicy: Never</code> and treat it as a Job (Lab 08)
Env var is empty inside the container but the key exists in the ConfigMap	Key name mismatch, or the ConfigMap changed after the container started	<code>kubectl exec <pod> -- env sort</code> ; <code>env</code> is a start-up snapshot — recreate the Pod
<code>\$ (VAR)</code> appears literally in the process's arguments	The referenced name is not defined earlier in the same <code>env</code> list (e.g. it came from <code>envFrom</code>)	Promote it to an explicit <code>env</code> entry above the one that references it
Exit code 137 with Reason: OOMKilled	The container exceeded <code>resources.limits.memory</code>	Raise the limit or reduce the workload; see Lab 10
pod updates may not change fields other than <code>spec.containers[*].image ...</code>	You tried to patch <code>env</code> , <code>command</code> or <code>volumes</code> on a live Pod	Delete and recreate the Pod, or manage it with a Deployment
Mounted ConfigMap file does not reflect a recent edit	Volume refresh is asynchronous (kubelet sync period, typically under a minute)	Wait, then re-check; if you used <code>subPath</code> , the file will never update — mount the directory instead

Lab 03 · Cleanup

```
kubectl delete namespace kcna-lab03  
namespace "kcna-lab03" deleted
```

Lab 03 · What you learned

- **command** overrides **ENTRYPOINT**; **args** overrides **CMD**. Setting **command** alone also discards the image's CMD — and skips whatever initialisation the image's entrypoint script would have done.
 - **command/args** are **not run in a shell**. Ask for one explicitly if you need shell syntax.
 - **--from-env-file** produces one ConfigMap key per line; **--from-file** produces one key holding the whole file. ConfigMap values are always **strings**.
 - **envFrom** bulk-imports a ConfigMap; **prefix** prevents collisions; **optional: true** stops a missing map from blocking start-up.
 - The **downward API** (**fieldRef**, **resourceFieldRef**) lets a Pod learn its own name, node, IP and resource limits with no API call and no RBAC.
 - **\$ (VAR)** expansion happens in the **kubelet**, only against earlier entries of the same **env** list, and the stored object keeps the literal reference.
 - **Env is a start-up snapshot; volumes are live**. That asymmetry decides where each piece of configuration should go.
 - **CrashLoopBackOff** is a back-off **state**. The diagnosis lives in **logs --previous**, **lastState.terminated.exitCode** and **Reason**.
-

Lab 03 · Further reading

- [Define a Command and Arguments for a Container](https://kubernetes.io/docs/tasks/inject-data-application/define-command-argument-container/) <https://kubernetes.io/docs/tasks/inject-data-application/define-command-argument-container/>
- [Define Environment Variables for a Container](https://kubernetes.io/docs/tasks/inject-data-application/define-environment-variable-container/) <https://kubernetes.io/docs/tasks/inject-data-application/define-environment-variable-container/>
- [Define Dependent Environment Variables](https://kubernetes.io/docs/tasks/inject-data-application/define-interdependent-environment-variables/) <https://kubernetes.io/docs/tasks/inject-data-application/define-interdependent-environment-variables/>
- [ConfigMaps](https://kubernetes.io/docs/concepts/configuration/configmap/) <https://kubernetes.io/docs/concepts/configuration/configmap/>
- [Configure a Pod to Use a ConfigMap](https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/) <https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/>
- [Expose Pod Information to Containers Through Environment Variables \(downward API\)](https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/) <https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/>
- [Pod Lifecycle — container states and restart back-off](https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/) <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- [Debug Running Pods](https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/) <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>

Lab 04 — Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador

Field	Value
Lab ID	Lab 04
Title	Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador
Day / Topic	Day 1 · Core Concepts
Duration	50 minutes
Namespace	kcna-lab04
Learning outcome	LO1 — Develop a Kubernetes architectural proof of concept.
Ability	A1 Develop an architectural proof of concept
Knowledge	K1 Process for developing proof of concepts
Deck slide	Slide 118
Repository path	courseware/labs/lab-04-multi-container-patterns/

Goal. Build and read the three classic multi-container Pod patterns — sidecar, adapter and ambassador — plus a v1.29+ native sidecar, and prove what containers in a Pod really share: one network namespace and named volumes, nothing else.

What you will produce:

- Four running Pods (sidecar, adapter, ambassador, native sidecar) that all consume data/access-seed.log and data/tariff-quote.json
- Prometheus-format metrics produced by an adapter container from a proprietary key=value log it did not write
- An admission-time validation error proving that each container has its own independent volumeMounts list

Lab 04 · Objective

By the end of this lab you will be able to:

- State exactly **what containers in a Pod share** — one network namespace and named volumes — and what they do **not** share.
- Build and explain the three classic composite-container patterns: **sidecar**, **adapter** and **ambassador**.
- Use `kubectl logs -c <container>` and `kubectl exec -c <container>`, and recognise the error you get when you forget `-c`.
- Use a **native sidecar** — an `initContainer` with `restartPolicy: Always`, stable since Kubernetes v1.29 — and explain the ordering guarantee it buys.
- Diagnose a Pod rejected at admission because two containers did not really share the volume the author thought they shared.

Lab 04 · Prerequisites

- Kubernetes **v1.29 or later** (native sidecars); this lab is written against v1.30+. `kubectl` on PATH.
- Labs 01–03 completed.

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0

kubectl get nodes

NAME                                STATUS    ROLES    AGE   VERSION
kcna-qa-20260905-control-plane    Ready    control-plane   88m   v1.37.0

cd courseware/labs/lab-04-multi-container-patterns
```

Lab 04 · Scenario

The Depot Portal booking API at **Meridian Freight Pte Ltd** has three requirements that the application team refuses to build into the application itself, and they are right to refuse:

1. **Logs** must reach the central platform. Today they are written to a file on disk in Meridian's own key=value format.
2. **Metrics** must be scrapeable by the platform's Prometheus. The application emits no metrics at all and will not be rewritten this quarter.
3. **Outbound calls** to the tariff service must gain retries and, later, mutual TLS. The application hard-codes `http://127.0.0.1:9000` and cannot be changed.

Every one of those is solvable **outside** the application, in the same Pod. That is what composite-container patterns are for, and it is one of the strongest arguments in the PoC for adopting Kubernetes.

Lab 04 · Step-by-step procedure

Step 1 — What a Pod actually is

A Pod is not "a container". It is a **shared execution context**: one or more containers that share

- **one network namespace** — one Pod IP, one loopback interface, one port space;
- **one set of named volumes**, each mounted where and how each container asks;
- one lifecycle, one node, one scheduling decision.

They do **not** share a filesystem root, and by default they do **not** share a PID namespace. That is the entire mental model, and everything below follows from it.

```
kubectl explain pod.spec.containers.volumeMounts.name

GROUP:
KIND:    Pod
VERSION:  v1

FIELD: name <string> -required-

DESCRIPTION:
  This must match the Name of a Volume.
```

"This must match the Name of a Volume" — remember that sentence; it becomes the failure injection in section 6.

Step 2 — Namespace, datasets and the remote service

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab04 created
```

Look at the seed dataset — 16 rows of Meridian's proprietary access log:

```
head -n 3 data/access-seed.log
2026-09-05T08:00:01Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=41
2026-09-05T08:00:04Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=38
2026-09-05T08:00:07Z depot=MF-SIN-02 path=/api/bays status=200 bytes=1544 latency_ms=52

kubectl -n kcna-lab04 create configmap access-seed --from-file=data/access-seed.log
kubectl -n kcna-lab04 create configmap tariff-quote --from-file=data/tariff-quote.json

configmap/access-seed created
configmap/tariff-quote created
```

Bring up the remote tariff service that the ambassador will proxy to:

```
kubectl apply -f manifests/10-tariff-backend.yaml
pod/tariff-backend created
service/tariff-backend created

kubectl -n kcna-lab04 wait --for=condition=Ready pod/tariff-backend --timeout=120s
kubectl -n kcna-lab04 get endpointslices -l kubernetes.io/service-name=tariff-backend

pod/tariff-backend condition met
NAME                                ADDRESSSTYPE  PORTS    ENDPOINTS    AGE
tariff-backend-9k4xd               IPv4          80      10.244.0.19  22s
```

Use `endpointslices`, not `endpoints`. The `v1 Endpoints` resource was deprecated in Kubernetes v1.33 in favour of `discovery.k8s.io/v1 EndpointSlice`.

Step 3 — Pattern 1: SIDECAR (a log shipper on a shared volume)

Definition. A sidecar *adds a capability* the application does not have, without changing the application.

```
kubectl apply -f manifests/20-sidecar-logship.yaml
pod/depot-sidecar created

kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-sidecar --timeout=120s
kubectl -n kcna-lab04 get pod depot-sidecar

pod/depot-sidecar condition met
NAME      READY   STATUS    RESTARTS   AGE
depot-sidecar  2/2     Running   0          25s
```

READY 2/2 — the first number is ready containers, the second is total. Now ask for the logs the way you did in Lab 02:

```
kubectl -n kcna-lab04 logs depot-sidecar
error: a container name must be specified for pod depot-sidecar, choose one of: [app shipper]
```

`kubectl logs` streams **one container**. With more than one it cannot guess:

```
kubectl -n kcna-lab04 logs depot-sidecar -c app --tail=3
[app] seeded 16 historical rows

kubectl -n kcna-lab04 logs depot-sidecar -c shipper --tail=6
2026-09-05T08:00:37Z depot=MF-SIN-02 path=/api/trailers status=404 bytes=64 latency_ms=11
2026-09-05T08:00:40Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=37
```

```
2026-09-05T08:00:43Z depot=MF-SIN-02 path=/api/tariff status=503 bytes=0 latency_ms=3002
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:12:07Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
2026-09-05T16:12:10Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

Stop and read that. The `shipper` container never wrote a single one of those lines. It is emitting content produced by a *different container* — the first four rows came from `data/access-seed.log`, the last two were generated live — because both containers mount the same `emptyDir` volume named `varlog`.

That log stream is now an ordinary container log, so anything that collects container logs (a DaemonSet log agent, the cloud provider's collector) picks it up for free. That is the sidecar's whole value.

Prove the two containers have separate filesystems but one shared directory:

```
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls -l /var/log/depot
total 4
-rw-r--r--    1 1000      3000          2814 Sep  5 16:12 access.log
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls /seed
ls: /seed: No such file or directory
command terminated with exit code 1
```

`/seed` exists only in the `app` container — it declared that mount, the `shipper` did not. Same Pod, different filesystems.

Now try to write from the `shipper`:

```
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- \
  sh -c 'echo tampered >> /var/log/depot/access.log'
sh: can't create /var/log/depot/access.log: Read-only file system
command terminated with exit code 1
```

The same volume is mounted `readOnly: true` in the `shipper` and read-write in the `app`. **Mount options are per container**, not per volume.

Finally, look at how the `emptyDir` is bounded:

```
kubectl -n kcna-lab04 get pod depot-sidecar \
  -o jsonpath='{.spec.volumes[?(@.name=="varlog")].emptyDir.sizeLimit}{"\n"}'
16Mi
```

An unbounded `emptyDir` can fill the node's disk and get your Pod evicted. Always set `sizeLimit`.

Step 4 — Pattern 2: ADAPTER (reshape an output to a standard)

Definition. An adapter *changes the shape* of something the application already produces, so that a standard consumer can read it. The application is unchanged and unaware.

```
kubectl apply -f manifests/30-adapter-metrics.yaml
pod/depot-adapter created
kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-adapter --timeout=120s
pod/depot-adapter condition met
```

The `app` still speaks Meridian's private format:

```
kubectl -n kcna-lab04 logs depot-adapter -c app --tail=2
2026-09-05T16:14:22Z depot=MF-SIN-02 path=/api/bookings status=404 bytes=812 latency_ms=44
2026-09-05T16:14:24Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

Wait about 20 seconds for the adapter's loop to fire, then:

```
kubectl -n kcna-lab04 logs depot-adapter -c adapter --tail=12

# HELP depot_http_requests_total Depot Portal requests by status code.
# TYPE depot_http_requests_total counter
depot_http_requests_total{depot="MF-SIN-02",code="200"} 24
depot_http_requests_total{depot="MF-SIN-02",code="201"} 4
depot_http_requests_total{depot="MF-SIN-02",code="404"} 4
depot_http_requests_total{depot="MF-SIN-02",code="503"} 4
# HELP depot_http_requests_observed_total All parsed log rows.
# TYPE depot_http_requests_observed_total counter
depot_http_requests_observed_total{depot="MF-SIN-02"} 36
```

That is the **Prometheus text exposition format** — # HELP, # TYPE, then `metric{label="value"} number`. You will scrape exactly this shape in Lab 24. The application produced none of it.

Sidecar or adapter? The mechanics are identical — an extra container on a shared volume. The *intent* differs, and that is what you write in the design document:

	Sidecar	Adapter	Ambassador
Direction	augments the app	reshapes app output	owns app outbound traffic
App awareness	none	none	none (it just calls localhost)
Typical use	log shipping, config reload, cert rotation	metrics exporter, log reformatter, protocol translation	proxy, TLS origination, retries, sharding
Real-world example	Fluent Bit shipping a log file	*_exporter translating to Prometheus	Envoy in a service mesh

Step 5 — Pattern 3: AMBASSADOR (own the outbound connection)

Definition. An ambassador *represents the outside world to the application on localhost*. The application makes the simplest possible call; the ambassador deals with discovery, TLS, retries and topology.

```
kubectl apply -f manifests/40-ambassador-proxy.yaml

configmap/ambassador-conf created
pod/depot-ambassador created

kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-ambassador --timeout=120s
kubectl -n kcna-lab04 get pod depot-ambassador

pod/depot-ambassador condition met
NAME                READY    STATUS    RESTARTS   AGE
depot-ambassador    2/2     Running   0           41s

kubectl -n kcna-lab04 logs depot-ambassador -c app --tail=4

[app] I only know about http://127.0.0.1:9000
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
```

The `bay_hour` value came from `data/tariff-quote.json`, which lives in a **different Pod** (`tariff-backend`), reached through a **Service DNS name** the application has never heard of. Look at the proxy's own log:

```
kubectl -n kcna-lab04 logs depot-ambassador -c ambassador --tail=3

127.0.0.1 - - [05/Sep/2026:16:16:02 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-"
"Wget"
127.0.0.1 - - [05/Sep/2026:16:16:12 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-"
```

```
"Wget"
127.0.0.1 - - [05/Sep/2026:16:16:22 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-"
"Wget"
```

The client address is **127.0.0.1**. Two containers, one loopback interface — because they share a network namespace. Confirm that they also share one IP:

```
kubectl -n kcna-lab04 exec depot-ambassador -c app -- \
  wget -qO- http://127.0.0.1:9000/ambassador-health
ambassador ok
```

Because the port space is shared, **two containers in one Pod may not both bind the same port**. Read the proxy configuration that made this work:

```
kubectl -n kcna-lab04 get configmap ambassador-conf \
  -o go-template='{{index .data "default.conf"}}' | head -n 12
server {
    listen          9000;
    server_name     localhost;

    # Everything the application asks for on localhost:9000 is forwarded
    # to the real, cluster-internal service.
    location / {
        proxy_pass          http://tariff-backend.kcna-lab04.svc.cluster.local:80;
        proxy_set_header    Host                $host;
        proxy_set_header    X-Meridian-Via      ambassador;
        proxy_connect_timeout 2s;
        proxy_read_timeout  5s;
    }
}
```

Adding mutual TLS, retries or a canary split later means editing that ConfigMap — not the application. This is precisely how a service mesh sidecar (Envoy in Istio or Linkerd) works, injected automatically instead of written by hand.

Step 6 — Native sidecars (Kubernetes v1.29+)

The three patterns above all use ordinary `spec.containers`, and they share one weakness: **start-up order is not guaranteed**. If the shipper starts after the app, early log lines can be lost; if an ambassador starts after the app, the app's first outbound calls fail.

Kubernetes v1.29 made **restartable init containers** — "native sidecars" — available by default, and the feature reached GA in v1.33. An `initContainer` with `restartPolicy: Always`:

- starts **before** any regular container, in list order;
- keeps running for the life of the Pod (it does not have to exit);
- is **not** counted in the Pod's `READY` `n/m` ratio;
- does not stop a Job's Pod from reaching `Completed`;
- is terminated **after** the regular containers, so it can flush buffers.

```
kubectl apply -f manifests/50-native-sidecar.yaml
pod/depot-native-sidecar created

kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-native-sidecar --timeout=120s
kubectl -n kcna-lab04 get pod depot-native-sidecar

pod/depot-native-sidecar condition met
NAME                                READY    STATUS    RESTARTS   AGE
depot-native-sidecar                1/1      Running   0           33s
```

1/1, not 2/2 — the sidecar is not in `spec.containers`. It is running all the same:


```
kubectl -n kcna-lab04 get pod depot-native-sidecar \
-o jsonpath='{range .status.initContainerStatuses[*]}{.name}{ "\t" }{.state}{ "\n" }{end} '
prepare      {"terminated":{"exitCode":0,"finishedAt":"2026-09-05T16:17:41Z","reason":"Completed","startedAt":"2026-09-05T16:17:41Z"}}
shipper      {"running":{"startedAt":"2026-09-05T16:17:43Z"}}
```

Two init containers, two completely different outcomes:

- **prepare has no restartPolicy**, so it is a classic init container: it ran to completion (exitCode: 0) and the Pod moved on.
- **shipper has restartPolicy: Always**, so it is **running** and will keep running.

```
kubectl -n kcna-lab04 logs depot-native-sidecar -c shipper --tail=3
[shipper] native sidecar up BEFORE the app container
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:17:48Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

`-c shipper` works for init containers too — no special flag needed.

	Classic init container	Native sidecar	Regular container
Field	initContainers[]	initContainers[] + restartPolicy: Always	containers[]
Must exit	yes, 0	no	no
Start order	before everything	before regular containers, in list order	no guarantee among peers
Counted in <code>READY</code> n/m	no	no	yes
Blocks Job completion	no	no	yes
Since	v1.0	v1.29 (GA v1.33)	v1.0

Choose native sidecars for anything the application depends on at start-up — log shippers, proxies, credential agents — and especially inside Jobs, where an old style sidecar would keep the Job running forever.

Lab 04 · Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace and both ConfigMaps; that `tariff-backend` is Ready and has an EndpointSlice; that `depot-sidecar` has two containers and that the shipper re-emitted the 16 seeded rows it never wrote; that the adapter is producing Prometheus # `TYPE` metadata and a 503 series; that the ambassador's app got a quote through `127.0.0.1:9000` and the proxy logged a 200; that the native sidecar declares `restartPolicy: Always` and is in a *running* state while the app runs; and finally that the broken manifest is rejected at admission (via `--dry-run=server`, so nothing is created).

Expected tail:

```
Result: 20 passed, 0 failed
```

Full annotated evidence: `verification/expected-output.md`.

Lab 04 · Failure injection — "but they're in the same Pod!"

The most common multi-container bug is assuming that two containers in one Pod automatically see each other's files. They do not. **Each container has its own `volumeMounts` list**, and a shared volume only exists where both containers explicitly mount the same volume *name*.

`manifests/90-sidecar-broken-volume.yaml` has the app mounting `varlog` and the shipper mounting `applogs` — a name that is never declared:

```
kubectl apply -f manifests/90-sidecar-broken-volume.yaml

The Pod "depot-sidecar-broken" is invalid: spec.containers[1].volumeMounts[0].name: Not found: "applogs"

kubectl -n kcna-lab04 get pod depot-sidecar-broken

Error from server (NotFound): pods "depot-sidecar-broken" not found
```

Diagnosis

- The error came from the **API server at admission time**, before any node was involved. Contrast with Lab 01, where a missing *ConfigMap* was accepted and only failed later at the kubelet. The difference: a `volumeMounts.name` is an **intra-object** reference, so the API server can validate it without looking anything up. A *ConfigMap* name is an **inter-object** reference, and Kubernetes does not do referential integrity across objects.
- Read the field path: `spec.containers[**1**].volumeMounts[**0**].name`. Index 1 is the *second* container (*shipper*); index 0 is its first mount. The message tells you the offending container without naming it.
- **Nothing was created.** There is no Pod to delete, no events to read, no half-broken state. Admission-time errors are the cheapest class of failure — which is the argument for running `kubectl apply --dry-run=server` in CI.

Now consider the *silent* version of this bug, which is far worse. If the author had instead mounted the **right volume at the wrong path** — say `varlog` at `/var/log/app` in the shipper while the app writes to `/var/log/depot` — the Pod would be perfectly valid, would start, and the shipper would simply tail a file that never appears. You would see:

NAME	READY	STATUS	RESTARTS	AGE
depot-sidecar-quiet	2/2	Running	0	3m

Two containers, both "healthy", and zero logs shipped. The diagnostic habit:

```
# 1. Does the writer actually see its file?
kubectl -n kcna-lab04 exec depot-sidecar -c app -- ls -l /var/log/depot
# 2. Does the reader see the SAME file at the path it is reading?
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls -l /var/log/depot
# 3. Compare the mounts the API server actually stored.
kubectl -n kcna-lab04 get pod depot-sidecar \
  -o jsonpath='{range .spec.containers[*]}{.name}{": "}{.volumeMounts[*].mountPath}{"\n"}{end}'}

total 4
-rw-r--r--    1 1000    3000           3122 Sep  5 16:20 access.log
total 4
-rw-r--r--    1 1000    3000           3122 Sep  5 16:20 access.log
app: /var/log/depot /seed
shipper: /var/log/depot
```

Same size, same timestamp, same path — that is what "sharing a volume" looks like when it is genuinely working.

Fix the broken manifest by making the names agree (either rename the volume to `applogs` in `spec.volumes`, or the mount to `varlog`), then re-apply. The working equivalent is already deployed as `depot-sidecar`.

Lab 04 · Troubleshooting

Symptom	Likely cause	Fix
error: a container name must be specified for pod X, choose one of: [...]	<code>kubectl logs/exec</code> on a multi-container Pod without <code>-c</code>	Add <code>-c <container></code> ; use <code>kubectl logs X --all-containers=true --prefix</code> to see all at once
<code>spec.containers[N].volumeMounts[M].name: Not found: "..."</code> at apply time	A container mounts a volume name that <code>spec.volumes</code> does not declare	Make the names match; catch it earlier with <code>kubectl apply --dry-run=server</code>
Both containers Running, sidecar log is empty	Same volume mounted at different paths , or the writer buffers and never flushes	<code>exec</code> into each container and <code>ls -l</code> the path; compare <code>.spec.containers[*].volumeMounts</code>
Sidecar container CrashLoopBackOff with tail: can't open ...: No such file or directory	The sidecar started before the app created the file	Wait for the file in a loop (as <code>20-sidecar-logship.yaml</code> does), or make it a native sidecar with an init container that creates the file
Error: failed to create containerd task: ... address already in use or nginx exits with bind() to 0.0.0.0:80 failed	Two containers in the Pod bound the same port — the network namespace is shared	Give each container a distinct port
Ambassador returns 502 Bad Gateway	The upstream Service has no ready endpoints, or the DNS name is wrong	<code>kubectl get endpointslices -l kubernetes.io/service-name=<svc></code> ; check the FQDN in the proxy config
Pod evicted with The node was low on resource: ephemeral-storage	An unbounded <code>emptyDir</code> filled the node disk	Set <code>emptyDir.sizeLimit</code> , and rotate or cap the log inside the container
Native sidecar behaves like a classic init container (Pod never starts the app)	<code>restartPolicy</code> : Always missing, or the cluster is older than v1.29	<code>kubectl get pod X -o jsonpath='{.spec.initContainers[*].restartPolicy}'</code> ; check <code>kubectl version</code>
A Job with a sidecar never completes	The sidecar is a regular container, so the Pod stays Running forever	Move it to <code>initContainers</code> with <code>restartPolicy: Always</code> (Lab 08)

Lab 04 · Cleanup

```
kubectl delete namespace kcna-lab04
namespace "kcna-lab04" deleted
```

Lab 04 · What you learned

- A Pod is a **shared execution context**: one network namespace, one set of named volumes, one lifecycle — not "a container".
- Containers in a Pod **do not** share a filesystem root. Sharing happens only where two containers mount the **same volume name**, and mount options (`readOnly`, `mountPath`) are set **per container**.

- **Sidecar** adds a capability; **adapter** reshapes an output to a standard; **ambassador** owns outbound connectivity on localhost. The mechanics are the same; the intent is what you document.
- `kubectl logs/exec` need `-c` on multi-container Pods; the error helpfully lists the container names.
- One shared loopback means containers reach each other on `127.0.0.1` — and means they cannot both bind the same port.
- **Native sidecars** (`initContainers` + `restartPolicy: Always`, v1.29+, GA v1.33) give the start-order and shutdown-order guarantees that ordinary sidecars lack, and they let Jobs complete.
- Intra-object references are validated at **admission**; inter-object references are not validated at all. `--dry-run=server` catches the first class in CI.

Lab 04 · Further reading

- [Pods — Workload resources](https://kubernetes.io/docs/concepts/workloads/pods/) <https://kubernetes.io/docs/concepts/workloads/pods/>
- [Sidecar Containers](https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/) <https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/>
- [Init Containers](https://kubernetes.io/docs/concepts/workloads/pods/init-containers/) <https://kubernetes.io/docs/concepts/workloads/pods/init-containers/>
- [Kubernetes 1.28: Introducing native sidecar containers](https://kubernetes.io/blog/2023/08/25/native-sidecar-containers/) <https://kubernetes.io/blog/2023/08/25/native-sidecar-containers/>
- [Communicate Between Containers in the Same Pod Using a Shared Volume](https://kubernetes.io/docs/tasks/access-application-cluster/communicate-containers-same-pod-shared-volume/) <https://kubernetes.io/docs/tasks/access-application-cluster/communicate-containers-same-pod-shared-volume/>
- [Volumes — emptyDir](https://kubernetes.io/docs/concepts/storage/volumes/#emptydir) <https://kubernetes.io/docs/concepts/storage/volumes/#emptydir>
- [Logging Architecture — sidecar container patterns](https://kubernetes.io/docs/concepts/cluster-administration/logging/) <https://kubernetes.io/docs/concepts/cluster-administration/logging/>
- [EndpointSlices](https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/) <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>

Lab 05 — Container Images, OCI and the Runtime Interface

Field	Value
Lab ID	Lab 05
Title	Container Images, OCI and the Runtime Interface
Day / Topic	Day 1 · Cloud Native Architecture & the CNCF Ecosystem
Duration	40 minutes
Namespace	kcna-lab05
Learning outcome	LO1 — Develop a Kubernetes architectural proof of concept.
Ability	A1 Develop an architectural proof of concept
Knowledge	K1 Process for developing proof of concepts
Deck slide	Slide 127
Repository path	courseware/labs/lab-05-container-images-oci/

Goal. Take apart what an image reference really means — registry, repository, tag, digest and OCI manifest — see how the kubelet hands it to a CRI runtime, and prove why a moved tag breaks reproducibility while a digest cannot.

What you will produce:

- An image-catalog Pod serving data/image-inventory.csv and data/reference-forms.tsv, with its resolved status.imageID captured
- A digest-pinned Pod repaired from a stale sha256 reference to the digest this cluster actually resolved
- An imagePullPolicy matrix showing Always, IfNotPresent and an ErrImageNeverPull for a Never-policy image

Lab 05 · Objective

By the end of this lab you will be able to:

- Decompose an **image reference** into registry, repository, tag and digest, and state the two defaults Kubernetes applies silently.
- Describe the **OCI Image Specification** stack — index → manifest → config → content-addressed layers — and why layers are shared between images.
- Distinguish `spec.containers[].image` (what you **asked for**) from `status.containerStatuses[].imageID` (what the runtime **resolved**).
- State the **CRI** boundary: kubelet → CRI → containerd/CRI-O → OCI runtime.
- Apply `imagePullPolicy` correctly, including its version-dependent **default**, and recognise `ErrImageNeverPull`.
- Explain and demonstrate why a **moved tag** breaks reproducibility and a **digest** cannot.

Lab 05 · Prerequisites

- Kubernetes v1.30+ cluster; `kubectl` on PATH.
- Labs 01–04 completed.

- Outbound access to Docker Hub from the cluster node (the lab pulls `nginx:1.27-alpine` and `busybox:1.36`).

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0

kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
kcna-qa-20260905-control-plane    Ready    control-plane    102m   v1.37.0

cd courseware/labs/lab-05-container-images-oci
```

Lab 05 · Scenario

Meridian Freight Pte Ltd had an outage last quarter that nobody could reproduce. The Depot Portal ran fine in staging and failed in production, from the *same* manifest, with the *same* image reference: `nginx:1.27-alpine`.

The post-incident review found the cause. The two environments had pulled that tag six weeks apart, and in between the publisher had re-pointed it at a new build. Two nodes, one reference, **two different images**.

The architecture board has asked the PoC to answer three questions:

1. What exactly does an image reference name?
2. How do we prove which bytes are running?
3. What is the minimum change that makes deployments reproducible?

Meridian's approved image list is `data/image-inventory.csv`, and the reference normalisation card the team keeps on the wall is `data/reference-forms.tsv`. You will serve both from the cluster and use them to answer all three questions.

Lab 05 · Step-by-step procedure

Step 1 — Namespace

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab05 created
```

Step 2 — Publish the inventory datasets

```
kubectl -n kcna-lab05 create configmap image-catalog \
  --from-file=data/image-inventory.csv \
  --from-file=data/reference-forms.tsv
configmap/image-catalog created

kubectl -n kcna-lab05 get configmap image-catalog \
  -o go-template='{{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{ end }}'

image-inventory.csv
reference-forms.tsv
```

Step 3 — What is actually running containers on this node

Kubernetes does not run containers. The kubelet speaks the **Container Runtime Interface (CRI)** — a gRPC API — to a runtime, and that runtime speaks the **OCI Runtime Spec** to a low-level runtime that creates the process.

```
kubelet —CRI (gRPC)→ containerd / CRI-O —OCI runtime spec→ runc / crun
                                     |
                                     — OCI distribution spec → registry (Docker Hub,
registry.k8s.io, ...)
```

Ask the node which runtime it has:

```
kubectl get nodes -o jsonpath='{.items[0].status.nodeInfo.containerRuntimeVersion}{"\n"}'
containerd://2.1.4
```

Everything before `://` is the CRI implementation. Since Kubernetes v1.24 there is no built-in Docker shim: Docker Engine is not a CRI runtime, though images built by Docker are ordinary OCI images and run fine.

On a node you could inspect the same thing with `crictl images`, `crictl inspecti <ref>` and `crictl ps` — `crictl` is the CRI-level equivalent of `docker`. It needs shell access to the node, which this lab deliberately does not require. Every fact `crictl` would show you is available through the API, as the next command demonstrates.

The kubelet reports the node's whole image store back to the API server:

```
kubectl get nodes \
  -o jsonpath='{range .items[0].status.images[*]}{.sizeBytes}{"\t"}{.names[0]}{"\n"}{end}' \
  | sort -rn | head -n 6
147718971
registry.k8s.io/etcd@sha256:e78e30bb5c2ea9d2b0f8b1a2f9e1bd5e9e9b3f18ea9d1c0b2b74f6c3e2e9b40d2
92632544 registry.k8s.io/kube-
apiserver@sha256:1a1f05a2cd7c2b2e3e0f1a45c7c7f74a4e3fdc7fbc25da8b0e0d84a8b7f6cf12
21674089
docker.io/library/nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
4264178
docker.io/library/alpine@sha256:beefdbd8a1da6d2915566fde36db9db0b524eb737fc57cd1367effd16dc0d06d
4042190
docker.io/library/busybox@sha256:2376a0c12759aa1214ba83e771ff252c7b1663216b192f5e0fb364e952f85c
744000
registry.k8s.io/pause@sha256:7031c1b283388d2c2e09b57badb803c05ebed362dc88d84b480cc47f72a21097
```

Two things to take away:

- **Every entry is addressed by digest**, not by tag. The node's store is a content-addressed cache; tags are labels pointing into it.
- `registry.k8s.io/pause` is the **sandbox (infrastructure) container**. Every Pod gets one; it holds the network namespace open so that your containers — and the sidecars from Lab 04 — can join it and share a Pod IP.

Step 4 — Anatomy of an image reference

```
grep -v '^#' data/reference-forms.tsv | column -t -s $'\t'
```

nginx	docker.io/library/nginx:latest	weak
- tag is mutable AND implicit		
nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak
- tag can be re-pointed		
library/nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak
- same as above		
docker.io/library/nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak
- fully qualified but still a tag		
nginx@sha256:<64 hex>	docker.io/library/nginx@sha256:<64 hex>	
strong - content addressed, immutable		
registry.k8s.io/pause:3.9	registry.k8s.io/pause:3.9	weak
- no docker.io default applied		
ghcr.io/meridian/depot-api:2.4.1	ghcr.io/meridian/depot-api:2.4.1	weak
- private registry, still a tag		
localhost:5000/depot-api:dev	localhost:5000/depot-api:dev	weak
- host:port prefix means "registry", not "repo"		

A full reference is:

```
[REGISTRY[:PORT]/]REPOSITORY[:TAG] [@DIGEST]
docker.io / library/nginx : 1.27-alpine @ sha256:3d69...
```

Two defaults are applied **silently** when you write a short name:

1. No registry host → `docker.io`, and for a single-segment repository also `library/`. This is a Docker Hub convention that other registries do not share: `registry.k8s.io/pause` gets no `library/`.
2. No tag and no digest → `:latest`. `latest` is **not** "the newest"; it is just the default tag name, and it may be years old or may not exist at all.

The rule of thumb Kubernetes decides on: a reference is *anything* the runtime can resolve. How much you can trust it is entirely up to you.

Step 5 — Run the tag-pinned catalog and read what it resolved

```
kubectl apply -f manifests/10-image-catalog.yaml
pod/image-catalog created
kubectl -n kcna-lab05 wait --for=condition=Ready pod/image-catalog --timeout=180s
pod/image-catalog condition met
```

Now read the three fields that matter:

```
kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.spec.containers[0].image}{"\n"}{.status.containerStatuses[0].image}{"\n"}{.status.containerStatuses[0].imageID}{"\n"}'

nginx:1.27-alpine
docker.io/library/nginx:1.27-alpine
docker.io/library/nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Field	Meaning
<code>spec.containers[0].image</code>	what you wrote — never rewritten by Kubernetes
<code>status.containerStatuses[0].image</code>	the normalised reference the runtime used
<code>status.containerStatuses[0].imageID</code>	the digest that actually ran

That third line is your answer to "which bytes are in production?". Record it at every release. Save it now — you will need it in section 6:


```
CATALOG_ID=$(kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.status.containerStatuses[0].imageID}')
echo "$CATALOG_ID"

docker.io/library/
nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Confirm the datasets are being served:

```
kubectl -n kcna-lab05 exec image-catalog -c web -- \
  wget -qO- http://127.0.0.1/image-inventory.csv | grep -v '^#' | head -n 3

component,image_ref,registry,repository,tag,pin_style,approx_size_mb,base,notes
depot-web,nginx:1.27-alpine,docker.io,library/nginx,1.27-alpine,tag,22,alpine,static
content only
depot-api,busybox:1.36,docker.io,library/busybox,1.36,tag,2,scratch+busybox,PoC stand-in
for the Go API
```

Step 6 — What is inside an OCI image

An OCI image is not a file. It is a small graph of JSON documents plus a set of compressed tarballs, all addressed by the SHA-256 of their own content:

```
image index      (optional, multi-platform: "which manifest for linux/arm64?")
├─ image manifest (one platform: lists the config + the layers)
│   └─ image config (JSON: ENTRYPOINT, CMD, Env, WorkingDir, User, rootfs.diff_ids,
history)
└─ layers[]        (gzipped tar of filesystem CHANGES, each with its own digest)
```

Consequences you can reason about without a shell on the node:

- **Layers are shared.** `nginx:1.27-alpine` and `alpine:3.20` have the same Alpine base layer, stored once. The node's image store above lists the *image* sizes, not the sum of unique bytes on disk.
- **The image config is where ENTRYPOINT and CMD live** — the exact fields you overrode with `command` and `args` in Lab 03. `kubectl` never shows them, because Kubernetes does not read them; the runtime does.
- **Multi-platform is an index, not a different image.** The same `busybox:1.36` reference resolves to a different manifest on arm64 and amd64. That is why a digest you copied from an amd64 laptop can fail on an arm64 node: you pinned the *platform manifest* instead of the *index*.
- **A digest is a checksum of the manifest.** If any byte of any layer changed, the layer digest changes, so the manifest changes, so the image digest changes. There is no way to alter content behind a digest.

Look at the immutable identity Kubernetes itself keeps for the sandbox image:

```
kubectl get nodes \
-o jsonpath='{range .items[0].status.images[*]}{.names[0]}{"\n"}{end}' | grep pause

registry.k8s.io/
pause@sha256:7031c1b283388d2c2e09b57badb803c05ebed362dc88d84b480cc47f72a21097
```

Step 7 — imagePullPolicy semantics

```
kubectl apply -f manifests/30-pullpolicy-matrix.yaml

pod/pp-always created
pod/pp-ifnotpresent created
pod/pp-never created

kubectl -n kcna-lab05 get pods -o custom-columns=\
'NAME:.metadata.name,IMAGE:.spec.containers[0].image,POLICY:.spec.containers[0].imagePullP
olicy,STATUS:.status.phase'
```

NAME	IMAGE	POLICY
STATUS		
image-catalog	nginx:1.27-alpine	
IfNotPresent	Running	
pp-always	busybox:1.36	Always
Running		
pp-ifnotpresent	busybox:1.36	
IfNotPresent	Running	
pp-never	meridianfreight.example.com/depot-portal/depot-api:0.1.0-poc	Never
Pending		

Compare the events:

```
kubectl -n kcna-lab05 describe pod pp-always | grep -E 'Pulling|Pulled'
Normal Pulling 30s kubelet Pulling image "busybox:1.36"
Normal Pulled 28s kubelet Successfully pulled image "busybox:1.36" in 1.842s
(1.842s including waiting). Image size: 4042190 bytes.
kubectl -n kcna-lab05 describe pod pp-ifnotpresent | grep -E 'Pulling|Pulled'
Normal Pulled 30s kubelet Container image "busybox:1.36" already present on machine
```

No Pulling line at all for IfNotPresent — the node had it. And the third:

```
kubectl -n kcna-lab05 get pod pp-never
NAME      READY   STATUS             RESTARTS   AGE
pp-never  0/1     ErrImageNeverPull  0          19s
kubectl -n kcna-lab05 describe pod pp-never | tail -n 4
Events:
  Type      Reason             Age           From          Message
  ----      -
Warning    ErrImageNeverPull  6s (x3 over 21s) kubelet       Container image
"meridianfreight.example.com/depot-portal/depot-api:0.1.0-poc" is not present with pull
policy of Never
```

Note the distinction from Lab 02: this is **ErrImageNeverPull**, not **ErrImagePull**. No registry was contacted; there is no network failure to investigate. **Never** is used with pre-loaded images (`kind load docker-image`) and in air-gapped clusters.

Policy	Behaviour	Use when
Always	resolve the reference at the registry on every container start	mutable tags, or you must guarantee freshness; fails if the registry is down
IfNotPresent	use the node's copy if the reference is present	fixed tags and digests — the normal choice
Never	never contact a registry	air-gapped nodes, pre-loaded images

Defaults, if you omit the field: `:latest` or no tag at all → **Always**; any other explicit tag → **IfNotPresent**; a digest → **IfNotPresent**. State the policy explicitly anyway, so a reviewer never has to recall this table.

Lab 05 · Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace and both ConfigMap keys; that the node reports a CRI runtime; that `image-catalog` is **Ready**, that its `spec.image` is the tag you wrote and its `status.imageID` is a well-

formed sha256:<64 hex> digest; that both datasets are actually served over HTTP; that the running catalog image is on data/image-inventory.csv while pp-never's image deliberately is not; the three imagePullPolicy values and pp-never's ErrImageNeverPull; and finally that depot-web-locked is digest-pinned and — once you have repaired it — that its digest matches image-catalog's exactly.

Expected tail:

```
Result: 19 passed, 0 failed
```

Full annotated evidence: verification/expected-output.md.

Lab 05 · Failure injection — the tag moved, or the digest went stale

manifests/20-digest-pinned.yaml carries a **deliberately stale** digest: a syntactically valid sha256: reference that no registry can resolve. This is exactly what a hand-copied digest looks like after the manifest it named was replaced or the operator mistyped one character.

```
kubectl apply -f manifests/20-digest-pinned.yaml
pod/depot-web-locked created
kubectl -n kcna-lab05 get pod depot-web-locked
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-locked	0/1	ErrImagePull	0	7s

```
kubectl -n kcna-lab05 describe pod depot-web-locked | tail -n 8
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	48s	default-scheduler	Successfully assigned kcna-lab05/depot-web-locked to kcna-ga-20260905-control-plane
Normal	Pulling	18s (x3 over 48s)	kubelet	Pulling image "nginx@sha256:9c1b5b6f0f6cc1e7d0d2a3c4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718"
Warning	Failed	17s (x3 over 47s)	kubelet	Failed to pull image "nginx@sha256:9c1b5b6f...": failed to pull and unpack image "docker.io/library/nginx@sha256:9c1b5b6f...": failed to resolve reference "docker.io/library/nginx@sha256:9c1b5b6f...": docker.io/library/nginx@sha256:9c1b5b6f...: not found
Warning	Failed	17s (x3 over 47s)	kubelet	Error: ErrImagePull
Normal	BackOff	3s (x4 over 46s)	kubelet	Back-off pulling image "nginx@sha256:9c1b5b6f..."
Warning	Failed	3s (x4 over 46s)	kubelet	Error: ImagePullBackOff

Diagnosis

- The reference is **syntactically valid**, so nothing rejected it: the API server accepted the Pod and the scheduler placed it. Only the registry can say whether a digest exists.
- failed to resolve reference ... not found after docker.io/library/nginx@ tells you the repository was found and the **digest** was not. Compare with Lab 02, where the same phrasing appeared after nginx:1.27-alpin — there the *tag* was missing. Same error string, different last path segment; read it carefully.
- If instead you had seen 401 Unauthorized, the digest may well exist but your node has no credentials. If you had seen no match for platform in manifest, you pinned a single-platform manifest digest and this node has a different architecture.

- Note what a stale digest **cannot** do: silently give you different content. That is the entire trade — a digest fails loudly; a moved tag succeeds quietly with different bytes, which is what caused Meridian's outage.

Repair — pin to the digest this cluster resolved

```
CATALOG_ID=$(kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.status.containerStatuses[0].imageID}')
echo "resolved digest: ${CATALOG_ID##*@}"

resolved digest: sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

image is one of the very few mutable Pod fields, so patch it in place:

```
kubectl -n kcna-lab05 set image pod/depot-web-locked "web=nginx@${CATALOG_ID##*@}"
pod/depot-web-locked image updated

kubectl -n kcna-lab05 wait --for=condition=Ready pod/depot-web-locked --timeout=120s
kubectl -n kcna-lab05 get pod depot-web-locked

pod/depot-web-locked condition met
NAME                READY   STATUS    RESTARTS   AGE
depot-web-locked    1/1     Running   0           2m11s

kubectl -n kcna-lab05 get pod depot-web-locked \
-o jsonpath='{.spec.containers[0].image}{"\n"}{.status.containerStatuses[0].imageID}{"\n"}'

nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

spec.image and status.imageID now name the same content. With the tag-pinned image-catalog Pod they do not, and cannot be made to. That single difference is the answer to the architecture board's third question, and it costs nothing at run time.

To make this practical rather than manual, Meridian's release pipeline should resolve the tag to a digest **once at build time** and commit the digest into the manifest:

```
# Illustrative — resolve once in CI, commit the result:
# crane digest nginx:1.27-alpine
# -> sha256:3d696e83...
# then render: image: nginx@sha256:3d696e83...
sed "s|nginx@sha256:[0-9a-f]\{64\}|nginx@${CATALOG_ID##*@}|" \
manifests/20-digest-pinned.yaml | grep 'image:'

image: nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Keep the human-readable tag in a label or annotation so the digest is still traceable to a version, e.g. `app.kubernetes.io/version: "1.27-alpine"`.

Lab 05 · Troubleshooting

Symptom	Likely cause	Fix
ErrImagePull... failed to resolve reference "...@sha256:...": not found	The digest does not exist in that repository (stale, mistyped, or garbage-collected)	Re-resolve the digest from a known-good source and patch image; verify with status.imageID
ErrImagePull... docker.io/library/x:tag: not found	The tag does not exist	Fix the tag (Lab 02); check for a typo or a retired release

Symptom	Likely cause	Fix
<code>ErrImageNeverPull</code>	<code>imagePullPolicy: Never</code> and the image is not on the node	Pre-load it (<code>kind load docker-image <ref></code>) or change the policy
<code>ImagePullBackOff ... 401</code> <code>Unauthorized/pull access denied</code>	Private registry with no credentials	Create a <code>kubernetes.io/dockerconfigjson</code> Secret and add <code>spec.imagePullSecrets</code> (Lab 15)
<code>no match for platform in manifest/exec format error on start</code>	You pinned a single-platform manifest digest, or the image has no build for this node's architecture	Pin the index digest, or use a multi-arch tag; check <code>kubectl get node -o jsonpath='{.items[0].status.nodeInfo.architecture}'</code>
<code>x509: certificate signed by unknown authority during pull</code>	Corporate TLS-intercepting proxy, or a private registry with a private CA	Install the CA on the nodes ; this cannot be fixed from a Pod spec
Two environments behave differently from the same manifest	A mutable tag resolved to different digests at different times	Compare <code>status.imageID</code> in both; move to digest pinning
Pod keeps re-pulling and start-up is slow	<code>imagePullPolicy: Always</code> (explicitly, or implied by <code>:latest</code>)	Use an explicit non-latest tag or a digest, and <code>IfNotPresent</code>
<code>kubectl describe</code> shows the image pulled but the container exits instantly	The image's ENTRYPOINT/CMD is not what you assumed	Revisit Lab 03; inspect the image config with a tool such as <code>crane config <ref></code>

Lab 05 · Cleanup

```
kubectl delete namespace kcna-lab05
namespace "kcna-lab05" deleted
```

Lab 05 · What you learned

- An image reference is `[REGISTRY/]REPOSITORY[:TAG] [@DIGEST]`, and Kubernetes applies two silent defaults: `docker.io/library/` and `:latest`. `latest` means "the default tag", never "the newest".
- An **OCI image** is an index → manifest → config + content-addressed layers. Layers are shared between images; the config is where ENTRYPOINT and CMD live.
- **`spec.image` is a request; `status.imageID` is the answer.** Record the `imageID` at every release — it is the only durable statement of what ran.
- **Tags are mutable pointers; digests are content addresses.** A stale digest fails loudly; a moved tag succeeds quietly with different bytes.
- Kubernetes talks to a runtime over the **CRI**; the runtime talks to `runc` over the OCI runtime spec and to registries over the OCI distribution spec. Every Pod also runs a `pause` sandbox container that owns the network namespace.
- `imagePullPolicy` has three values and a **default that depends on your tag**. `ErrImageNeverPull` is a policy outcome, not a network failure.

Lab 05 · Further reading

- [Images](https://kubernetes.io/docs/concepts/containers/images/) — reference format, `imagePullPolicy`, defaults and pull secrets
<https://kubernetes.io/docs/concepts/containers/images/>
- [Container Runtime Interface \(CRI\)](https://kubernetes.io/docs/concepts/architecture/cri/) <https://kubernetes.io/docs/concepts/architecture/cri/>
- [Container Runtimes](https://kubernetes.io/docs/setup/production-environment/container-runtimes/) <https://kubernetes.io/docs/setup/production-environment/container-runtimes/>
- [Debugging Kubernetes nodes with crictl](https://kubernetes.io/docs/tasks/debug/debug-cluster/crictl/) <https://kubernetes.io/docs/tasks/debug/debug-cluster/crictl/>
- [Don't Panic: Kubernetes and Docker \(dockershim removal\)](https://kubernetes.io/blog/2020/12/02/dont-panic-kubernetes-and-docker/) <https://kubernetes.io/blog/2020/12/02/dont-panic-kubernetes-and-docker/>
- [OCI Image Specification](https://github.com/opencontainers/image-spec) <https://github.com/opencontainers/image-spec>
- [OCI Distribution Specification](https://github.com/opencontainers/distribution-spec) <https://github.com/opencontainers/distribution-spec>
- [OCI Runtime Specification](https://github.com/opencontainers/runtime-spec) <https://github.com/opencontainers/runtime-spec>
- [Pod Lifecycle — container images and the sandbox](https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/) <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

Day 2 — Workloads, Scheduling & Container Orchestration

Day 2 Learning Focus

Move from single Pods to controllers. Understand the reconciliation loop that every Kubernetes controller runs, drive Deployments through rollouts and rollbacks, choose correctly between the five workload controllers, and make the scheduler place Pods where you intend using selectors, affinity, taints and resource arithmetic.

Field	Value
Instructional hours	8 hours
Class window	9:30 AM – 6:30 PM
Registered topics	2. Workloads & Scheduling
KCNA modules	Container Orchestration Fundamentals
Learning outcomes	LO2
Abilities / Knowledge	A3 / K3
Labs	5 — Lab 06, Lab 07, Lab 08, Lab 09, Lab 10
Hands-on minutes	250 minutes
Deck slides	133–239

Session structure. The table below is the timetable the trainer works to. Breaks are not counted as instructional time.

Time	Duration	Session	Mode
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

Day 2 Concepts

Kubernetes runs a loop, not your application

Day 1 ended with the reconciliation loop. Day 2 is that loop applied repeatedly. Every workload controller in Kubernetes does the same three things — watch the API for objects it owns, compare observed against desired, issue the minimum API calls to close the gap — and differs only in *what* it considers desired.

`kube-controller-manager` is a single binary running dozens of these loops: the Deployment controller, the ReplicaSet controller, the Job and CronJob controllers, the node lifecycle controller, the endpoint and EndpointSlice controllers, the garbage collector, the ServiceAccount token controller and more. They never talk to each other. They communicate only by writing objects that the others are watching. That indirection is the architecture.

Level-triggered beats edge-triggered because the system is allowed to miss events. If a controller crashes and restarts, it re-lists the world and reconciles from the current level; nothing needs to be replayed. The cost is that everything is *eventually* consistent — there is a window in which the cluster does not match your YAML, and `kubectl get` during that window is telling you the truth, not lying to you.

Choosing the right workload controller

Five controllers, five different jobs, and picking wrongly is the most expensive early mistake:

- **Deployment** — stateless, interchangeable replicas. You want *N* of something and you do not care which is which. Rolling updates, revision history and rollback come with it.
- **StatefulSet** — replicas that need stable network identity and stable per-replica storage. Ordinal names (`db-0`, `db-1`), ordered rollout, and a `PersistentVolumeClaim` per replica that survives rescheduling. Day 4.
- **DaemonSet** — exactly one Pod per matching node. Log shippers, node exporters, CNI agents. **You do not set replicas** — the node count is the replica count, which is why scaling a `DaemonSet` is not your decision.
- **Job** — run to completion, then stop. `completions` and `parallelism` control how many and how concurrently; `backoffLimit` caps retries.
- **CronJob** — a Job factory on a schedule, with `concurrencyPolicy` (`Allow`, `Forbid`, `Replace`), `startingDeadlineSeconds`, and history limits.

Labels, selectors and ReplicaSets

A `ReplicaSet`'s `spec.selector` is what it uses to find its Pods; its `spec.template.metadata.labels` is what it stamps on the Pods it creates. If those two disagree, the API server rejects the object — because the `ReplicaSet` would create Pods it could not then see, and would create them forever.

Selectors come in two flavours. **Equality-based** (`tier=backend`) is what `spec.selector.matchLabels` and older objects such as `Services` use. **Set-based** (`tier in (backend, worker)`, `env notin (prod)`, `key, !key`) is available in `matchExpressions` on newer objects. A selector with multiple terms is an **AND** across all of them.

Two behaviours follow from selectors that surprise people. **Adoption**: a bare Pod whose labels happen to match a `ReplicaSet`'s selector, and which has no controller owner, is adopted — the `ReplicaSet` writes an `ownerReference` into it and counts it towards the replica total. **Orphaning**: relabel a Pod so it no longer matches, and the `ReplicaSet` drops it and immediately creates a replacement, leaving you with an unmanaged Pod still consuming resources. That is the standard technique for quarantining a misbehaving Pod for debugging.

Deployments, rollouts and rollback

A `Deployment` does not manage Pods. It manages **ReplicaSets**, one per revision of the Pod template, and each `ReplicaSet` manages Pods. That three-level chain — `Deployment` owns `ReplicaSet` owns Pod — is what makes rollback cheap: rolling back means scaling an old `ReplicaSet` back up and the new one down.

`strategy.type` is `RollingUpdate` (the default) or `Recreate`. Under `RollingUpdate`, `maxSurge` is how many Pods above `replicas` may exist during the rollout and `maxUnavailable` is how many below `readyReplicas` you will tolerate. Both default to 25% and both accept an absolute number or a

percentage. Percentages round **up** for `maxSurge` and **down** for `maxUnavailable`, so at `replicas: 4` the defaults give surge 1 and unavailable 1. Setting `maxUnavailable: 0` guarantees no loss of capacity but requires headroom for the surge; setting `maxSurge: 0` guarantees no extra capacity but must take Pods down first. You cannot set both to zero.

Only a change to `spec.template` creates a new revision. Scaling does not. Changing a label on the Deployment itself does not. `kubectl rollout history` shows the revisions, `--revision=N` shows one revision's template, and `kubectl rollout undo` reverses to the previous one — which, note carefully, creates a *new* revision number rather than deleting the bad one.

When a rollout is stuck, `progressDeadlineSeconds` (default 600) eventually flips the `Progressing` condition to `False` with reason `ProgressDeadlineExceeded`. That is the signal to read; `kubectl rollout status` will otherwise sit there indefinitely. The usual cause is a bad image tag: the new ReplicaSet's Pods never become ready, `maxUnavailable` stops the old ones being removed, and the Deployment correctly refuses to proceed.

The scheduler: filter, score, bind

A Pod with an empty `.spec.nodeName` is the scheduler's entire input. The scheduling framework runs in two phases. **Filtering** eliminates nodes that cannot run the Pod at all — insufficient allocatable resources, a `nodeSelector` or node affinity that does not match, a taint without a matching toleration, a port conflict, a volume that cannot be attached. **Scoring** ranks the survivors with plugins that reward spreading across zones, image locality, balanced resource allocation and inter-pod affinity preferences. The highest-scoring node wins and the scheduler writes a **Binding** — which is just another API object.

The placement controls, from bluntest to most expressive:

- **`nodeSelector`** — a flat map of labels that must all match. Hard, simple, no expressiveness.
- **Node affinity** — `requiredDuringSchedulingIgnoredDuringExecution` (a hard filter, but with set-based operators: `In`, `NotIn`, `Exists`, `Gt`, `Lt`) and `preferredDuringSchedulingIgnoredDuringExecution` (weighted soft preference). "IgnoredDuringExecution" means an already-running Pod is not evicted if the node's labels later change.
- **Pod affinity / anti-affinity** — place near, or away from, *other Pods* matching a selector, within a `topologyKey` such as `kubernetes.io/hostname` or `topology.kubernetes.io/zone`. Anti-affinity is how you stop all replicas landing on one node.
- **Taints and tolerations** — the inverse direction. A taint on a node *repels* Pods; a toleration on a Pod says it will accept that taint. Effects are `NoSchedule`, `PreferNoSchedule` and `NoExecute` — the last of which evicts already-running Pods that do not tolerate it. Control-plane nodes carry `node-role.kubernetes.io/control-plane:NoSchedule` by default.
- **Topology spread constraints** — `maxSkew` across a `topologyKey`, with `whenUnsatisfiable: DoNotSchedule` or `ScheduleAnyway`. The modern, declarative way to say "spread evenly".

A Pod can also bypass the scheduler entirely by setting `spec.nodeName` directly. It will be run by that node's kubelet without any filtering — no resource check, no taint check. It is a debugging tool, not a design.

Requests, limits, QoS and eviction

`requests` is what the **scheduler** reserves. `limits` is what the **kubelet and kernel enforce**. They do different jobs at different times, and confusing them causes most capacity incidents.

CPU is **compressible**: exceeding a CPU limit gets the container throttled by the cgroup CPU controller, which is slow but survivable. Memory is **incompressible**: exceeding a memory limit gets the container **OOMKilled** by the kernel, immediately, with exit code 137. There is no gradual degradation for memory.

Quality of Service class is derived, never set:

- **Guaranteed** — every container has requests equal to limits for both CPU and memory. Last to be evicted.
- **Burstable** — at least one request is set, but the Guaranteed condition is not met.
- **BestEffort** — no requests and no limits anywhere in the Pod. First to be evicted.

Under node-pressure, the kubelet evicts BestEffort first, then Burstable Pods exceeding their requests (ordered by how far over they are and by Pod priority), and Guaranteed last. Note that node-pressure eviction is the kubelet's decision on one node, and is a different mechanism from an OOMKill of one container.

LimitRange and **ResourceQuota** are namespace-level and also distinct. A LimitRange sets *defaults* and per-object *min/max* for containers in the namespace — it mutates objects that omit values. A ResourceQuota sets an *aggregate cap* for the namespace and *rejects* objects that would exceed it. A ResourceQuota that constrains `requests.cpu` makes requests mandatory: without a LimitRange supplying a default, Pods without requests are simply refused.

The **HorizontalPodAutoscaler** changes `replicas` based on observed metrics against a target, computing `desired = ceil(current * currentMetric / targetMetric)` and clamping to `minReplicas/maxReplicas`, with a stabilisation window to damp flapping. It needs a metrics source — `metrics-server` for CPU and memory — and, for CPU targets expressed as a utilisation percentage, it needs CPU **requests** to be set, because utilisation is a percentage *of the request*.

Day 2 Labs

Day 2 has 5 labs, listed below and then set out in full. Work through them in order; each runs in its own namespace and cleans up after itself.

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 06	Labels, Selectors and ReplicaSets	kcna-lab06	45	LO2 / A3 / K3	155
Lab 07	Deployments, Rollout Strategy and Rollback	kcna-lab07	55	LO2 / A3 / K3	175
Lab 08	DaemonSets, Jobs and CronJobs	kcna-lab08	45	LO2 / A3 / K3	190
Lab 09	Scheduling: nodeSelector, Affinity, Taints and Tolerations	kcna-lab09	55	LO2 / A3 / K3	211

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 10	Resource Requests, Limits, QoS and Autoscaling	kcna-lab10	50	LO2 / A3 / K3	237

Lab 06 — Labels, Selectors and ReplicaSets

Field	Value
Lab ID	Lab 06
Title	Labels, Selectors and ReplicaSets
Day / Topic	Day 2 · Workloads & Scheduling
Duration	45 minutes
Namespace	kcna-lab06
Learning outcome	LO2 — Identify the technical and practical requirements in a Kubernetes setup.
Ability	A3 Identify technical and practical requirements as well as stakeholders' demands
Knowledge	K3 Objectives of solution architecture
Deck slide	Slide 155
Repository path	courseware/labs/lab-06-labels-selectors-replicasets/

Goal. Use labels and selectors as the binding mechanism between controllers and Pods, and prove it by watching a ReplicaSet adopt a hand-created Pod and replace one you relabel out of its selector.

What you will produce:

- A tracking-api ReplicaSet (matchLabels) that adopts an existing Pod instead of creating a third replica
- A berth-status ReplicaSet driven by matchExpressions using In, NotIn and Exists
- A label-audit Pod that reads the MFS service inventory and prints the standard label set

Lab 06 · Objective

By the end of this lab you will be able to:

1. Apply the Marina Freight Systems label standard to Kubernetes objects and query the cluster with both **equality-based** (`-l app=tracking-api`) and **set-based** (`-l 'app in (a,b)'`) selectors.
2. Explain that a ReplicaSet has **no pointer** to its Pods — the only binding is `.spec.selector` matching Pod labels.
3. Distinguish `matchLabels` from `matchExpressions` and use the `In`, `NotIn` and `Exists` operators.
4. Demonstrate **adoption** (a controller takes ownership of a pre-existing Pod) and **orphaning** (a relabelled Pod is released and replaced).
5. Diagnose the API-server rejection that occurs when a selector does not match its own Pod template.

Lab 06 · Prerequisites

A running single-node `kind` cluster and a `kubectl` that can reach it.

```
kubectl version --output=yaml | grep -E 'gitVersion|major|minor' | head -6
```

Expected output (your patch version will differ; anything v1.30 or newer is fine):

```
major: "1"
minor: "31"
```

```
gitVersion: v1.31.0
major: "1"
minor: "31"
gitVersion: v1.31.0
```

Confirm you have exactly one node and that it is `Ready`:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	21m	v1.31.0

Change into the lab directory — every path in this lab is relative to it:

```
cd courseware/labs/lab-06-labels-selectors-replicasets
ls
```

Expected output:

```
README.md  brief.json  data        manifests   verification
```

Lab 06 · Scenario

Marina Freight Systems (MFS) is a Singapore port-logistics SaaS provider. Its platform team runs the *Harbour* platform on Kubernetes and is preparing release **r24**.

An incident review found that a routine `kubectl delete pod -l app=tracking` command took down two unrelated services, because nobody could say with confidence which Pods a given label selector actually matched. The platform lead has published a **label standard** in `data/service-inventory.csv` and asked you to:

- load the inventory into the cluster so it is queryable,
- stand up the `tracking-api` and `berth-status` workloads under the standard,
- and prove — with evidence, not assertion — exactly which Pods each selector binds.

There is one complication. A Pod named `tracking-api-legacy` was created by hand during a previous incident and is still serving traffic. Management will not allow it to be deleted. You must show what happens when a ReplicaSet is introduced whose selector already matches it.

Lab 06 · Step-by-step procedure

Step 1 — Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab06 created
```

Confirm it, and note the labels the manifest set:

```
kubectl get ns kcna-lab06 --show-labels
```

Expected output:

NAME	STATUS	AGE	LABELS
kcna-lab06	Active	4s	course=kcna,kubernetes.io/metadata.name=kcna-lab06,lab=lab-06,owner=mfs-platform

kubernetes.io/metadata.name is added automatically by the API server on every namespace. You will use it in Lab 16 to write NetworkPolicies that select namespaces.

Step 2 — Load the MFS service inventory into the cluster

Look at the dataset first. This CSV is the source of truth for every label value you apply in this lab.

```
cat data/service-inventory.csv
```

Expected output:

```
service_name,app,tier,env,release,owner_team,replicas
Container Tracking API,tracking-api,edge,staging,r24,mfs-platform,3
Berth Status Service,berth-status,internal,staging,r24,mfs-platform,2
Cargo Manifest Store,cargo-manifest,internal,staging,r24,mfs-data,2
Customs Declaration Gateway,customs-gateway,edge,staging,r24,mfs-compliance,3
Tariff Rate Cache,tariff-cache,internal,staging,r24,mfs-data,2
Vessel ETA Predictor,vessel-eta,experimental,staging,r23,mfs-labs,1
Yard Crane Telemetry,crane-telemetry,internal,staging,r24,mfs-ops,2
Driver Mobile Backend,driver-backend,edge,staging,r24,mfs-platform,3
Warehouse Slotting Engine,slotting-engine,internal,staging,r23,mfs-ops,2
Partner Webhook Relay,webhook-relay,experimental,staging,r23,mfs-labs,1
```

Publish it as a ConfigMap so workloads in the cluster can read it:

```
kubectl -n kcna-lab06 create configmap service-inventory \
--from-file=data/service-inventory.csv
```

Expected output:

```
configmap/service-inventory created
```

Verify the key name — it is the **file's base name**, which is what the audit Pod mounts:

```
kubectl -n kcna-lab06 get configmap service-inventory -o jsonpath='{.data}' | head -c 120;
echo
```

Expected output:

```
{"service-inventory.csv":"service_name,app,tier,env,release,owner_team,replicas\nContainer
Tracking API,tracki
```

Step 3 — Run the data-driven label audit

```
kubectl apply -f manifests/40-pod-label-audit.yaml
```

Expected output:

```
pod/label-audit created
```

Wait for it to finish (it runs once and exits):

```
kubectl -n kcna-lab06 wait --for=jsonpath='{.status.phase}'=Succeeded pod/label-audit --
timeout=90s
```

Expected output:

```
pod/label-audit condition met
```

Read what it computed from the CSV:

```
kubectl -n kcna-lab06 logs label-audit
```

Expected output:

```
== MFS service inventory: tier=edge services ==
Container Tracking API app=tracking-api tier=edge env=staging release=r24
Customs Declaration Gateway app=customs-gateway tier=edge env=staging release=r24
```

```
Driver Mobile Backend app=driver-backend tier=edge env=staging release=r24
== counts by tier ==
edge=3
experimental=2
internal=5
AUDIT-COMPLETE
```

Three edge services, two experimental, five internal — ten in total. **tracking-api** is an edge service on release **r24**. That is the label set you will use next; it is not invented, it comes from the inventory.

Step 4 — Create the hand-made Pod (the "legacy" workload)

This Pod carries `app=tracking-api,tier=edge` — precisely what the ReplicaSet in Step 5 will select on.

```
kubectl apply -f manifests/10-pod-adoption-candidate.yaml
```

Expected output:

```
pod/tracking-api-legacy created
kubectl -n kcna-lab06 get pod tracking-api-legacy --show-labels
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	LABELS
tracking-api-legacy	1/1	Running	0	12s	app=tracking-api,env=staging,provenance=hand-created,release=r24,tier=edge

Confirm it has **no owner** — nothing in the cluster manages it:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences}'; echo
```

Expected output (an empty line — the field is absent):

Step 5 — Create the tracking-api ReplicaSet and watch adoption

Read the selector before you apply it:

```
grep -A5 'selector:' manifests/20-replicaset-tracking-api.yaml
```

Expected output:

```
selector:
  matchLabels:
    app: tracking-api
    tier: edge
template:
```

It asks for **3** replicas. There is already 1 matching Pod. Predict the outcome, then apply:

```
kubectl apply -f manifests/20-replicaset-tracking-api.yaml
```

Expected output:

```
replicaset.apps/tracking-api created
kubectl -n kcna-lab06 get rs tracking-api
```

Expected output:

NAME	DESIRED	CURRENT	READY	AGE
tracking-api	3	3	3	15s

Now the key observation — list the Pods with their `provenance` label:

```
kubectl -n kcna-lab06 get pods -l app=tracking-api \
-L provenance,tier,release
```

Expected output (the two generated Pod name suffixes will differ):

NAME	READY	STATUS	RESTARTS	AGE	PROVENANCE	TIER	RELEASE
tracking-api-4m2ql	1/1	Running	0	20s	replicaset	edge	r24
tracking-api-legacy	1/1	Running	0	75s	hand-created	edge	r24
tracking-api-x7ntb	1/1	Running	0	20s	replicaset	edge	r24

Only two new Pods were created, not three. The ReplicaSet counted the pre-existing Pod towards its replica goal. Prove ownership changed:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}
controller={.metadata.ownerReferences[0].controller}'; echo
```

Expected output:

```
ReplicaSet/tracking-api controller=true
```

This is **adoption**: the ReplicaSet controller wrote an `ownerReference` into a Pod it did not create, purely because the labels matched.

Step 6 — Orphan a Pod and watch the replacement appear

Open a watch in a second terminal:

```
kubectl -n kcna-lab06 get pods -l app=tracking-api -w
```

In your first terminal, relabel the legacy Pod out of the selector — this is the standard "quarantine a Pod for debugging" technique:

```
kubectl -n kcna-lab06 label pod tracking-api-legacy tier=quarantine --overwrite
```

Expected output:

```
pod/tracking-api-legacy labeled
```

The watch terminal shows a new Pod being created within a second or two:

tracking-api-9kdd8	0/1	Pending	0	0s
tracking-api-9kdd8	0/1	ContainerCreating	0	0s
tracking-api-9kdd8	1/1	Running	0	2s

Stop the watch with `Ctrl-C`. Now confirm the legacy Pod was **released, not deleted**:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences}'; echo
```

Expected output (empty again — the `ownerReference` was removed):

```
kubectl -n kcna-lab06 get pods --show-labels
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	LABELS
label-audit	0/1	Completed	0	5m	app=label-
audit,env=staging,release=r24,tier=tooling					
tracking-api-4m2ql	1/1	Running	0	3m	app=tracking-
api,env=staging,provenance=replicaset,release=r24,tier=edge					
tracking-api-9kdd8	1/1	Running	0	40s	app=tracking-
api,env=staging,provenance=replicaset,release=r24,tier=edge					
tracking-api-legacy	1/1	Running	0	4m	app=tracking-
api,env=staging,provenance=hand-created,release=r24,tier=quarantine					
tracking-api-x7ntb	1/1	Running	0	3m	app=tracking-
api,env=staging,provenance=replicaset,release=r24,tier=edge					

The quarantined Pod is still `Running` and still serving — it is simply no longer counted. This is the safest way to pull a misbehaving Pod out of a Service for inspection.

Restore it so the ReplicaSet re-adopts it and scales itself back down:

```
kubectl -n kcna-lab06 label pod tracking-api-legacy tier=edge --overwrite
sleep 5
kubectl -n kcna-lab06 get pods -l app=tracking-api
```

Expected output — the ReplicaSet now owns 4 Pods for a desired count of 3, so it deletes the newest one and settles back to 3:

NAME	READY	STATUS	RESTARTS	AGE
tracking-api-4m2ql	1/1	Running	0	4m
tracking-api-legacy	1/1	Running	0	5m
tracking-api-x7ntb	1/1	Running	0	4m

Step 7 — Set-based selectors with matchExpressions

```
kubectl apply -f manifests/30-replicaset-berth-status.yaml
```

Expected output:

```
replicaset.apps/berth-status created
kubectl -n kcna-lab06 get rs
```

Expected output:

NAME	DESIRED	CURRENT	READY	AGE
berth-status	2	2	2	18s
tracking-api	3	3	3	6m

Inspect the compiled selector — `kubectl` renders set-based expressions in the same syntax you type on the command line:

```
kubectl -n kcna-lab06 get rs berth-status -o jsonpath='{.spec.selector}' | tr ',' '\n'
```

Expected output:

```
{"matchExpressions":[{"key":"tier"
"operator":"NotIn"
"values":["experimental"]}
{"key":"release"
"operator":"Exists"}
{"key":"app"
"operator":"In"
"values":["berth-status"]}]}
"matchLabels":{"app":"berth-status"}}
```

Now run the equivalent queries yourself. Equality-based:

```
kubectl -n kcna-lab06 get pods -l app=berth-status
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
berth-status-hn6bq	1/1	Running	0	65s
berth-status-tzc4k	1/1	Running	0	65s

Set-based, spanning both workloads and excluding the experimental tier:

```
kubectl -n kcna-lab06 get pods \
-l 'app in (tracking-api,berth-status),tier notin (experimental)'
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
berth-status-hn6bq	1/1	Running	0	2m

berth-status-tzc4k	1/1	Running	0	2m
tracking-api-4m2ql	1/1	Running	0	8m
tracking-api-legacy	1/1	Running	0	9m
tracking-api-x7ntb	1/1	Running	0	8m

The `Exists` form, and its negation:

```
kubectl -n kcna-lab06 get pods -l 'release' --no-headers | wc -l
kubectl -n kcna-lab06 get pods -l '!provenance' --no-headers | wc -l
```

Expected output:

```
6
1
```

Six Pods carry a `release` key of any value; exactly one Pod (`label-audit`) has no `provenance` key at all.

Step 8 — Cross-check the selector against the inventory

Everything in this lab traces back to the CSV. Confirm the count you queried matches what the dataset declares:

```
awk -F, 'NR>1 && $2=="tracking-api" {print "inventory says replicas="$7}' data/service-
inventory.csv
kubectl -n kcna-lab06 get rs tracking-api -o jsonpath='cluster says
replicas={.spec.replicas}'; echo
```

Expected output:

```
inventory says replicas=3
cluster says replicas=3
```

Lab 06 · Verification

Run the bundled check script:

```
bash verification/checks.sh
```

The full expected transcript is in `verification/expected-output.md`. The script is read-only — it creates and deletes nothing. It exits `0` only when every assertion passes.

Lab 06 · Failure injection — a selector that cannot match its own template

`manifests/50-replicaset-broken-selector.yaml` asks for `tier: edge` in `.spec.selector.matchLabels`, but its Pod template stamps `tier: internal`. Such a ReplicaSet would create Pods it could never own, then create more, forever. The API server refuses it.

```
kubectl apply -f manifests/50-replicaset-broken-selector.yaml
```

Real error text:

```
The ReplicaSet "cargo-manifest-broken" is invalid: spec.template.metadata.labels: Invalid
value: map[string]string{"app":"cargo-manifest", "env":"staging", "release":"r24",
"tier":"internal"}: `selector` does not match template `labels`
```

Diagnosis. Validation for ReplicaSets, Deployments, StatefulSets, DaemonSets and Jobs requires the selector to be a **subset** of the template labels. Read the message backwards: it prints the *template* labels and tells you the *selector* does not match them. Compare the two blocks directly:

```
grep -A4 'matchLabels:' manifests/50-replicaset-broken-selector.yaml
grep -A5 'template:' manifests/50-replicaset-broken-selector.yaml | grep -E 'app:|tier:'
```

Expected output:

```
matchLabels:
  app: cargo-manifest
  tier: edge
template:
  app: cargo-manifest
  tier: internal
```

tier is edge in the selector and internal in the template. Fix it by making them agree — the inventory says cargo-manifest is an internal service, so the **selector** is the wrong one:

```
sed 's/      tier: edge/      tier: internal/' manifests/50-replicaset-broken-
selector.yaml \
| kubectl apply -f -
```

Expected output:

```
replicaset.apps/cargo-manifest-broken created
kubectl -n kcna-lab06 get rs cargo-manifest-broken
```

Expected output:

NAME	DESIRED	CURRENT	READY	AGE
cargo-manifest-broken	2	2	2	20s

Second failure to try if time allows. Delete one of the ReplicaSet's Pods (`kubectl -n kcna-lab06 delete pod <a tracking-api pod>`) and watch a replacement appear in under two seconds. The ReplicaSet is a *reconciliation loop over a label query*, not a list of Pod names — that is the whole idea.

Lab 06 · Troubleshooting

Symptom	Cause	Fix
The ReplicaSet "x" is invalid: spec.template.metadata.labels .. . \selector\ does not match template \labels\``	.spec.selector is not a subset of .spec.template.metadata.labels	Make every selector key/value appear identically in the template labels; the template may carry extra labels, the selector may not
ReplicaSet reports DESIRED 3 / CURRENT 5 and keeps deleting Pods	Another Pod or controller in the same namespace carries labels that also satisfy this selector	Run <code>kubectl -n kcna-lab06 get pods -l <selector> --show-labels</code> ; make selectors more specific (add app and tier), or relabel the intruder
You delete a Pod and it "comes back"	The owning ReplicaSet reconciled the label query and recreated it	Delete or scale the ReplicaSet , not the Pod: <code>kubectl -n kcna-lab06 scale rs/tracking-api --replicas=0</code>
kubectl label returns error: 'tier' already has a value (edge), and --overwrite is false	kubectl label refuses to change an existing key by default	Add <code>--overwrite</code>
Set-based query returns nothing and no error	Shell expanded the parentheses or ! in the selector	Quote the whole selector in single quotes: <code>-l 'app in (a,b)', -l '!provenance'</code>

Symptom	Cause	Fix
label-audit Pod is CreateContainerConfigError	The ConfigMap service-inventory was not created, or was created with a different key name	kubect1 -n kcna-lab06 get cm service-inventory -o jsonpath='{.data}'; recreate with --from-file=data/service-inventory.csv so the key is service-inventory.csv

Lab 06 · Cleanup

Delete **only** this lab's namespace. Everything you created lives inside it.

```
kubect1 delete namespace kcna-lab06
```

Expected output:

```
namespace "kcna-lab06" deleted
```

Confirm:

```
kubect1 get ns kcna-lab06
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab06" not found
```

Lab 06 · What you learned

- A label is **arbitrary key/value metadata**; a selector is a **query** over it. Kubernetes controllers are built almost entirely out of this one pairing — there are no Pod-name lists anywhere in a ReplicaSet spec.
- `matchLabels` is equality-based and **ANDed**. `matchExpressions` adds the set operators `In`, `NotIn`, `Exists`, `DoesNotExist`; both may be present and are **ANDed** together.
- On the command line, `-l k=v` is equality-based, `-l 'k in (a,b)'` / `-l 'k notin (a)'` is set-based, `-l k` is `Exists` and `-l '!k'` is `DoesNotExist`.
- **Adoption**: a controller writes an `ownerReference` into any unowned Pod whose labels satisfy its selector. **Orphaning**: change the labels and the controller removes the `ownerReference` and creates a replacement. The Pod itself is untouched — this is how you quarantine a Pod without losing it.
- A selector that is not a subset of the template labels is rejected at admission, before any Pod is created.
- `--show-labels` and `-L <key>` are the two flags that make label debugging tractable.

Lab 06 · Further reading

- Labels and Selectors — <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>
- ReplicaSet — <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>
- Owners and Dependents — <https://kubernetes.io/docs/concepts/overview/working-with-objects/owners-dependents/>
- Recommended Labels — <https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>

- `kubectl label` **reference** — https://kubernetes.io/docs/reference/kubectl/generated/kubectl_label/

Lab 07 — Deployments, Rollout Strategy and Rollback

Field	Value
Lab ID	Lab 07
Title	Deployments, Rollout Strategy and Rollback
Day / Topic	Day 2 · Workloads & Scheduling
Duration	55 minutes
Namespace	kcna-lab07
Learning outcome	LO2 — Identify the technical and practical requirements in a Kubernetes setup.
Ability	A3 Identify technical and practical requirements as well as stakeholders' demands
Knowledge	K3 Objectives of solution architecture
Deck slide	Slide 175
Repository path	courseware/labs/lab-07-deployments-rollout-rollback/

Goal. Drive a Deployment through three revisions of the MFS Freight Portal, make maxSurge and maxUnavailable arithmetic visible during the update, then wedge the rollout with a bad image and recover with rollout undo.

What you will produce:

- A freight-portal Deployment at revision 2 serving the r24.2 page, with a readable rollout history and change-cause column
- A wedged revision 3 stuck in ImagePullBackOff while the old ReplicaSet keeps 4 Pods Available
- A verified rollback to revision 2 confirmed by curl through the ClusterIP Service

Lab 07 · Objective

By the end of this lab you will be able to:

1. Explain the three-level ownership chain **Deployment** → **ReplicaSet** → **Pod** and show it with `ownerReferences`.
2. Compute, in advance, the **ceiling** (`replicas + maxSurge`) and **floor** (`replicas - maxUnavailable`) of a rolling update, then observe the cluster obey both.
3. Use `kubectl rollout status, history, undo` and `--to-revision`, and annotate revisions with `kubernetes.io/change-cause`.
4. Diagnose a rollout wedged by an unpullable image, read `ProgressDeadlineExceeded`, and recover without downtime.
5. State why `.spec.selector` is immutable on an existing Deployment and recognise the rejection message.

Lab 07 · Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```

gitVersion: v1.31.0
NAME          STATUS    ROLES    AGE   VERSION
kind-control-plane Ready    control-plane 34m   v1.31.0

```

Lab 06 should be complete (labels and selectors are assumed knowledge). Work from the lab directory:

```

cd courseware/labs/lab-07-deployments-rollout-rollback
ls manifests data

```

Expected output:

```

data:
release-plan.csv      site-r24.1.html      site-r24.2.html

manifests:
00-namespace.yaml      30-deployment-portal-r24.3-broken.yaml
10-deployment-portal-r24.1.yaml      40-deployment-selector-change.yaml
20-deployment-portal-r24.2.yaml      50-service-freight-portal.yaml

```

Two terminals are helpful for this lab. Terminal B is used only to watch.

Lab 07 · Scenario

Marina Freight Systems (MFS) must ship release **r24.2** of the *Freight Portal* — the web application customs brokers use to file declarations. The Port Authority contract sets a hard availability requirement: **at no point may fewer than 4 portal replicas be serving traffic**, because brokers file continuously during the 05:00–23:00 window.

The release manager has published `data/release-plan.csv`. It is the authority for the replica count and the rollout arithmetic; you are not free to invent values. Your job:

1. Deploy r24.1 and confirm the baseline.
2. Roll forward to r24.2 under a **zero-unavailable** strategy and prove the floor held.
3. When the on-call engineer pushes a hotfix tag that does not exist (r24.3), diagnose the wedge and roll back — with evidence for the incident review.

Lab 07 · Step-by-step procedure

Step 1 — Read the release plan

```
column -s, -t data/release-plan.csv
```

Expected output:

revision	release	image	configmap	replicas	maxSurge
maxUnavailable	max_pods_during_rollout	min_available_during_rollout	change_cause		
1	r24.1	nginx:1.27-alpine	portal-site-r24-1	4	1
1		5		3	r24.1 - initial
portal release					
2	r24.2	nginx:1.27-alpine	portal-site-r24-2	4	1
0		5		4	r24.2 - customs
banner zero-unavailable rollout					
3	r24.3	nginx:1.27-alpine-mfs-hotfix	portal-site-r24-2	4	1
0		5		4	r24.3 - BROKEN
image tag (deliberate)					

Read the two arithmetic columns before you touch the cluster:

- **Revision 1** — replicas 4, maxSurge 1, maxUnavailable 1. Ceiling 4+1=5. Floor 4-1=3. Kubernetes may take one Pod down before a replacement is Ready.
- **Revision 2** — replicas 4, maxSurge 1, maxUnavailable 0. Ceiling 5. Floor 4. Kubernetes **must** add a Ready Pod before it removes an old one. This satisfies the Port Authority contract.

maxSurge and maxUnavailable may not both be zero — that specification can make no progress, and the API server rejects it.

Step 2 — Namespace, site content and Service

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab07 created
```

Turn the two HTML files in `data/` into ConfigMaps. The key must be `index.html` because that is the filename nginx serves and the readiness probe requests:

```
kubectl -n kcna-lab07 create configmap portal-site-r24-1 \
  --from-file=index.html=data/site-r24.1.html
kubectl -n kcna-lab07 create configmap portal-site-r24-2 \
  --from-file=index.html=data/site-r24.2.html
```

Expected output:

```
configmap/portal-site-r24-1 created
configmap/portal-site-r24-2 created
```

Confirm the two releases really differ:

```
diff <(kubectl -n kcna-lab07 get cm portal-site-r24-1 -o jsonpath='{.data.index\.html}')} \
  <(kubectl -n kcna-lab07 get cm portal-site-r24-2 -o jsonpath='{.data.index\.html}')
```

Expected output:

```
6c6,7
< <p id="release">RELEASE=r24.1</p>
---
> <p id="release">RELEASE=r24.2</p>
> <p><strong>New:</strong> customs declaration status banner.</p>
11,12c12,13
< </ul>
< <p>build=r24.1 revision=1 strategy=RollingUpdate maxSurge=1 maxUnavailable=1</p>
---
> <li>Customs declaration status</li>
> </ul>
> <p>build=r24.2 revision=2 strategy=RollingUpdate maxSurge=1 maxUnavailable=0</p>
```

Create the Service now so it can act as a stable front door across the whole rollout:

```
kubectl apply -f manifests/50-service-freight-portal.yaml
```

Expected output:

```
service/freight-portal created
```

Step 3 — Deploy revision 1 (r24.1)

```
kubectl apply -f manifests/10-deployment-portal-r24.1.yaml
```

Expected output:

```
deployment.apps/freight-portal created
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=120s
```


Expected output:

```
Waiting for deployment "freight-portal" rollout to finish: 0 of 4 updated replicas are available...
Waiting for deployment "freight-portal" rollout to finish: 2 of 4 updated replicas are available...
deployment "freight-portal" successfully rolled out
kubectl -n kcna-lab07 get deploy,rs,pods
```

Expected output (hash suffixes will differ):

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/freight-portal	4/4	4	4	35s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/freight-portal-6c9f4b8d57	4	4	4	35s

NAME	READY	STATUS	RESTARTS	AGE
pod/freight-portal-6c9f4b8d57-8kq2m	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-hd7wn	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-p4xtl	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-zr9vc	1/1	Running	0	35s

Note there is one ReplicaSet, not four Pods managed directly. Prove the ownership chain:

```
POD=$(kubectl -n kcna-lab07 get pods -l app=freight-portal -o name | head -1)
kubectl -n kcna-lab07 get "$POD" -o
jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}'; echo
kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o
jsonpath='{.items[0].metadata.ownerReferences[0].kind}/{.items[0].metadata.ownerReferences[0].name}'; echo
```

Expected output:

```
ReplicaSet/freight-portal-6c9f4b8d57
Deployment/freight-portal
```

Confirm the served content is r24.1:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
grep RELEASE /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.1</p>
```

Step 4 — Watch the ceiling and floor during the r24.2 rollout

In **Terminal B**, start a watch that prints the Deployment status every second:

```
kubectl -n kcna-lab07 get deploy freight-portal \
-o custom-
columns=DESIRED:.spec.replicas,UPDATED:.status.updatedReplicas,TOTAL:.status.replicas,AVAILABLE:.status.availableReplicas \
--watch
```

In **Terminal A**, roll forward:

```
kubectl apply -f manifests/20-deployment-portal-r24.2.yaml
```

Expected output:

```
deployment.apps/freight-portal configured
```

Terminal B prints a sequence like this (exact interleaving varies, the **bounds do not**):

DESIRED	UPDATED	TOTAL	AVAILABLE
4	4	4	4
4	1	5	4
4	2	5	4
4	2	5	5
4	3	5	4
4	4	5	4
4	4	4	4

Read the two invariants straight off the columns:

- **TOTAL never exceeds 5** — that is the ceiling `replicas + maxSurge = 4 + 1`.
- **AVAILABLE never drops below 4** — that is the floor `replicas - maxUnavailable = 4 - 0`.

Stop the watch with `Ctrl-C`. Back in Terminal A:

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal
```

Expected output:

```
deployment "freight-portal" successfully rolled out
kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o custom-
columns=NAME:.metadata.name,DESIRED:.spec.replicas,READY:.status.readyReplicas,REVISION:.m
etadata.annotations.deployment\\.kubernetes\\.io/revision
```

Expected output:

NAME	DESIRED	READY	REVISION
freight-portal-6c9f4b8d57	0	<none>	1
freight-portal-7d84c65f9b	4	4	2

The old ReplicaSet is retained at 0 replicas. That empty ReplicaSet *is* the rollback mechanism — there is no snapshot store, just a scaled-down controller.

Confirm the new content is being served:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
grep -E 'RELEASE|customs' /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.2</p>
<p><strong>New:</strong> customs declaration status banner.</p>
```

Step 5 — Read the rollout history

```
kubectl -n kcna-lab07 rollout history deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1         r24.1 - initial portal release
2         r24.2 - customs banner, zero-unavailable rollout
```

The `CHANGE-CAUSE` column is populated from the `kubernetes.io/change-cause` annotation that each manifest sets. Without it you get `<none>` and a history that tells you nothing.

Inspect one revision in detail:

```
kubectl -n kcna-lab07 rollout history deployment/freight-portal --revision=1 | head -20
```

Expected output:

```
deployment.apps/freight-portal with revision #1
Pod Template:
```

```

Labels:      app=freight-portal
            pod-template-hash=6c9f4b8d57
            release=r24.1
            tier=edge
Annotations:  kubernetes.io/change-cause: r24.1 - initial portal release
Containers:
  web:
    Image:     nginx:1.27-alpine
    Port:      80/TCP
    Host Port: 0/TCP
    Limits:
      cpu:      200m
      memory:   128Mi
    Requests:
      cpu:      50m
      memory:   64Mi
    Liveness:   http-get http://:http/index.html delay=10s timeout=1s period=10s
#success=1 #failure=3
    Readiness: http-get http://:http/index.html delay=2s timeout=1s period=3s
#success=1 #failure=3

```

Note the `pod-template-hash` label. The Deployment controller adds it to the selector of every ReplicaSet it creates — that is how two ReplicaSets with otherwise identical labels stay disjoint.

```

kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o jsonpath='{range .items[*]}{.metadata.name}{ " hash="}
{.spec.selector.matchLabels.pod-template-hash}{ "\n"}{end} '

```

Expected output:

```

freight-portal-6c9f4b8d57 hash=6c9f4b8d57
freight-portal-7d84c65f9b hash=7d84c65f9b

```

Step 6 — Verify the current state

```
bash verification/checks.sh
```

You should see `ALL CHECKS PASSED`. Now break it.

Lab 07 · Verification

The bundled script `verification/checks.sh` reads `data/release-plan.csv`, asserts the live Deployment matches the plan's `replicas/maxSurge/maxUnavailable`, confirms the rollback landed on `r24.2`, and confirms the Service has four `EndpointSlice` backends.

```
bash verification/checks.sh; echo "exit=$?"
```

The full expected transcript is in `verification/expected-output.md`.

Lab 07 · Failure injection

6a. A hotfix tag that does not exist wedges the rollout

The on-call engineer builds `nginx:1.27-alpine-mfs-hotfix` locally but never pushes it. Apply revision 3:

```
kubectl apply -f manifests/30-deployment-portal-r24.3-broken.yaml
```

Expected output:

```
deployment.apps/freight-portal configured
```

Watch the rollout refuse to progress:

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=150s
```

Real output — it prints the stall, then fails on the progress deadline (progressDeadlineSeconds: 120):

```
Waiting for deployment "freight-portal" rollout to finish: 1 out of 4 new replicas have
been updated...
error: deployment "freight-portal" exceeded its progress deadline
```

Look at the partial state:

```
kubectl -n kcna-lab07 get deploy,rs,pods
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/freight-portal	4/4	1	4	9m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/freight-portal-6c9f4b8d57	0	0	0	9m
replicaset.apps/freight-portal-7d84c65f9b	4	4	4	6m
replicaset.apps/freight-portal-5f7b9c4d68	1	1	0	2m

NAME	READY	STATUS	RESTARTS	AGE
pod/freight-portal-5f7b9c4d68-w2kfz	0/1	ImagePullBackOff	0	2m
pod/freight-portal-7d84c65f9b-4rt8x	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-9lmzq	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-mn5vd	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-tq6jw	1/1	Running	0	6m

This is the rolling update working exactly as designed. maxUnavailable: 0 forbade the controller from removing any r24.2 Pod until a new Pod became Ready. No new Pod ever became Ready. So the update stopped after the single surge Pod — and **AVAILABLE is still 4**. The customs brokers never noticed.

Read the real error on the wedged Pod:

```
BAD=$(kubectl -n kcna-lab07 get pods -l release=r24.3 -o name | head -1)
kubectl -n kcna-lab07 describe "$BAD" | sed -n '/^Events:$/, $p'
```

Real output:

```
Events:
  Type     Reason      Age          From              Message
  ----     -
  Normal   Scheduled   2m14s        default-scheduler Successfully assigned kcna-lab07/freight-portal-5f7b9c4d68-w2kfz to kind-control-plane
  Normal   Pulling     43s (x4 over 2m13s)  kubelet           Pulling image "nginx:1.27-alpine-mfs-hotfix"
  Warning  Failed      41s (x4 over 2m11s)  kubelet           Failed to pull image "nginx:1.27-alpine-mfs-hotfix": failed to pull and unpack image "docker.io/library/nginx:1.27-alpine-mfs-hotfix": failed to resolve reference "docker.io/library/nginx:1.27-alpine-mfs-hotfix": docker.io/library/nginx:1.27-alpine-mfs-hotfix: not found
  Warning  Failed      41s (x4 over 2m11s)  kubelet           Error: ErrImagePull
  Normal   BackOff     5s (x7 over 2m11s)  kubelet           Back-off pulling image "nginx:1.27-alpine-mfs-hotfix"
  Warning  Failed      5s (x7 over 2m11s)  kubelet           Error: ImagePullBackOff
```

And the Deployment-level condition:

```
kubectl -n kcna-lab07 get deploy freight-portal \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}{ "\n" }{end}'
```

Expected output:

```
Available=True reason=MinimumReplicasAvailable
Progressing=False reason=ProgressDeadlineExceeded
```

Diagnosis. Available=True and Progressing=False together are the signature of a *stalled but safe* rollout: the service is up, the new version cannot start. The kubelet event names the exact cause — the tag `nginx:1.27-alpine-mfs-hotfix` does not resolve in `docker.io/library/nginx`. Nothing is wrong with the cluster, the manifest, the ConfigMap or the probes.

6b. Recover with rollout undo

```
kubectl -n kcna-lab07 rollout undo deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal rolled back
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=120s
```

Expected output:

```
deployment "freight-portal" successfully rolled out
kubectl -n kcna-lab07 rollout history deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1          r24.1 - initial portal release
3          r24.3 - BROKEN image tag (deliberate)
4          r24.2 - customs banner, zero-unavailable rollout
```

Revision 2 has vanished and revision 4 has appeared. This surprises people. A rollback does not create a copy — it *re-scales the existing ReplicaSet* and re-stamps it with the next revision number. The r24.2 ReplicaSet was revision 2 and is now revision 4. Confirm:

```
kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o custom-columns=NAME:.metadata.name,REVISION:.metadata.annotations.deployment\
\.kubernetes\io/
revision,DESIRED:.spec.replicas,IMAGE:.spec.template.spec.containers[0].image
```

Expected output:

NAME	REVISION	DESIRED	IMAGE
freight-portal-5f7b9c4d68	3	0	nginx:1.27-alpine-mfs-hotfix
freight-portal-6c9f4b8d57	1	0	nginx:1.27-alpine
freight-portal-7d84c65f9b	4	4	nginx:1.27-alpine

To roll back to a *specific* revision instead of the previous one:

```
kubectl -n kcna-lab07 rollout undo deployment/freight-portal --to-revision=1 --dry-
run=server -o jsonpath='{.spec.template.metadata.labels.release}'; echo
```

Expected output (a server-side dry run — nothing is changed):

```
r24.1
```

Confirm the portal is serving r24.2 again:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
grep RELEASE /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.2</p>
```

6c. .spec.selector is immutable

MFS wants to rename the tier label from `edge` to `frontend`. Try it on the live Deployment:

```
kubectl apply -f manifests/40-deployment-selector-change.yaml
```

Real error text:

```
The Deployment "freight-portal" is invalid: spec.selector: Invalid value:
v1.LabelSelector{MatchLabels:map[string]string{"app":"freight-portal", "tier":"frontend"},
MatchExpressions:[]v1.LabelSelectorRequirement(nil)}: field is immutable
```

Diagnosis. `apps/v1` froze `.spec.selector` after creation. If it could change, the Deployment would instantly orphan every Pod it owns and start again from zero — an unannounced full outage. The supported procedure is to create a **new** Deployment under the new selector, shift the Service to it, then delete the old one. Label renames are not free.

Lab 07 · Troubleshooting

Symptom	Cause	Fix
error: deployment "x" exceeded its progress deadline	No new Pod became Ready within <code>progressDeadlineSeconds</code> ; usually a bad image, a failing readiness probe or an unschedulable Pod	<code>kubectl -n <ns> describe pod -l <new release label></code> and read the Events block; fix the root cause or <code>kubectl rollout undo</code>
The Deployment "x" is invalid: <code>spec.selector: ... field is immutable</code>	Attempt to change <code>.spec.selector</code> on an existing Deployment	Create a new Deployment with the new selector, repoint the Service, then delete the old Deployment
Rollout appears to do nothing; <code>rollout status</code> returns immediately	<code>kubectl apply</code> produced no change to <code>.spec.template</code> ; only template changes create a new revision	Confirm with <code>kubectl diff -f <manifest></code> ; changing <code>.spec.replicas</code> alone scales but never creates a revision
<code>CHANGE-CAUSE</code> shows <code><none></code> for every revision	The <code>kubernetes.io/change-cause</code> annotation was not set on the Deployment	Add the annotation in the manifest, or <code>kubectl annotate deploy/x kubernetes.io/change-cause="..." --overwrite</code> before the next apply
error: no rollout history found for deployment "x" after several updates	<code>revisionHistoryLimit</code> is too low (0 keeps nothing) and old ReplicaSets were garbage-collected	Raise <code>.spec.revisionHistoryLimit</code> ; there is nothing to recover for revisions already collected
Pods stay <code>ContainerCreating</code> with configmap "portal-site-r24-2" not found	The ConfigMap was not created before the Deployment referenced it	<code>kubectl -n kcna-lab07 create configmap portal-site-r24-2 --from-file=index.html=data/site-r24.2.html</code> ; the kubelet retries automatically
The Deployment is invalid: <code>spec.strategy.rollingUpdate.maxUnavailable: Invalid value: intstr.IntOrString{...}: may not be 0 when maxSurge is 0</code>	Both surge and unavailable set to zero — no legal move exists	Set at least one of them above zero

Lab 07 · Cleanup

Delete **only** this lab's namespace.

```
kubectl delete namespace kcna-lab07
```

Expected output:

```
namespace "kcna-lab07" deleted
kubectl get ns kcna-lab07
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab07" not found
```

Lab 07 · What you learned

- A Deployment does not manage Pods. It manages **ReplicaSets**, each identified by a `pod-template-hash` the controller injects into the selector. Rolling forward and rolling back are both just *scaling two ReplicaSets in opposite directions*.
- `maxSurge` is the ceiling above `replicas`; `maxUnavailable` is the floor below it. `maxUnavailable: 0` buys a zero-downtime rollout at the cost of needing capacity for one extra Pod. They may not both be zero.
- `minReadySeconds` and the readiness probe together define what "available" means — without a readiness probe, "Ready" means only "the container process started".
- `kubernetes.io/change-cause` is the difference between a usable `rollout history` and a column of `<none>`.
- `rollout undo` re-scales an existing old ReplicaSet and gives it a **new, higher** revision number; the old revision number disappears from the history.
- `revisionHistoryLimit` bounds how many scaled-to-zero ReplicaSets are retained — and therefore how far back you can roll.
- `.spec.selector` is immutable. Plan label schemes before the first apply.

Lab 07 · Further reading

- Deployments — <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Rolling update strategy and `maxSurge/maxUnavailable` — <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment>
- Rolling back a Deployment — <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-back-a-deployment>
- `kubectl rollout` reference — https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/
- Pod lifecycle and container image pull policy — <https://kubernetes.io/docs/concepts/containers/images/>

Lab 08 — DaemonSets, Jobs and CronJobs

Field	Value
Lab ID	Lab 08
Title	DaemonSets, Jobs and CronJobs
Day / Topic	Day 2 · Workloads & Scheduling
Duration	45 minutes
Namespace	kcna-lab08
Learning outcome	LO2 — Identify the technical and practical requirements in a Kubernetes setup.
Ability	A3 Identify technical and practical requirements as well as stakeholders' demands
Knowledge	K3 Objectives of solution architecture
Deck slide	Slide 190
Repository path	courseware/labs/lab-08-daemonsets-jobs-cronjobs/

Goal. Run the three non-Deployment workload controllers against real MFS port data: a per-node DaemonSet agent, an Indexed Job that splits a work queue across parallel Pods, and a CronJob on a schedule - then fail a Job past its backoffLimit and read the reason.

What you will produce:

- A berth-agent DaemonSet reporting DESIRED=CURRENT=1 on the single-node kind cluster, reading berth-readings.csv
- A completed Indexed Job (completions 6, parallelism 2) whose six Pods each processed a distinct queue line
- A tariff-sync CronJob with concurrencyPolicy Forbid and a retained Job history

Lab 08 · Objective

By the end of this lab you will be able to:

1. Choose correctly between a Deployment, a DaemonSet, a Job and a CronJob for a given MFS workload.
2. Explain why a DaemonSet has **no .spec.replicas field** and predict its Pod count from the node set — **on this single-node kind cluster that count is exactly 1.**
3. Configure a Job with `completions`, `parallelism` and `backoffLimit`, and use `completionMode: Indexed` so parallel Pods split a work queue without coordination.
4. Configure a CronJob's `schedule`, `timeZone`, `concurrencyPolicy` and history limits.
5. Recognise and diagnose a Job that exhausts its `backoffLimit`.

Lab 08 · Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:


```

gitVersion: v1.31.0
NAME          STATUS    ROLES    AGE   VERSION
kind-control-plane Ready    control-plane 52m   v1.31.0

```

Read that node count. It is 1. Everything the DaemonSet does in this lab is scaled by that number. On the MFS production cluster the same manifest produces 14 Pods; here it produces 1. That is the point of the controller, not a limitation of the lab.

Confirm `Indexed Jobs` and `timeZone` on CronJobs are available (both are stable from v1.24 and v1.27 respectively):

```
kubectl explain job.spec.completionMode | head -8
```

Expected output:

```

GROUP:      batch
KIND:       Job
VERSION:    v1

FIELD: completionMode <string>

DESCRIPTION:
  completionMode specifies how Pod completions are tracked. It can be

```

Work from the lab directory:

```

cd courseware/labs/lab-08-daemonsets-jobs-cronjobs
ls manifests data

```

Expected output:

```

data:
berth-readings.csv      manifest-queue.txt      tariff-rates.csv

manifests:
00-namespace.yaml      30-cronjob-tariff-sync.yaml
10-daemonset-berth-agent.yaml  40-job-customs-export-broken.yaml
20-job-manifest-reconcile.yaml

```

Lab 08 · Scenario

Marina Freight Systems (MFS) runs three workloads that are *not* long-lived web services, and the platform team keeps trying to model them as Deployments:

MFS workload	What it actually is	Right controller
Berth telemetry agent — must run on every node to scrape the node's local sensor feed	one instance per node, forever	DaemonSet
Overnight cargo-manifest reconciliation — 6 manifests to process, run to completion, then stop	finite parallel batch	Job
Tariff rate synchronisation — pull the published rate card on a schedule	recurring batch	CronJob

Modelling the reconciliation as a Deployment produced an infinite restart loop last quarter (a Deployment's `restartPolicy` is always `Always`, so a container that exits 0 is restarted forever). You will build each one with the correct controller and prove it behaves as intended, using the real port datasets in `data/`.

Lab 08 · Step-by-step procedure

Step 1 — Namespace and the shared dataset

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab08 created
```

Inspect the three datasets — every controller in this lab reads one of them:

```
head -4 data/berth-readings.csv
cat data/manifest-queue.txt
head -4 data/tariff-rates.csv
```

Expected output:

```
reading_ts,berth,vessel_imo,draft_m,occupancy_pct,crane_moves_per_hr
2026-09-05T02:00:00Z,B01,9321483,14.2,92,31
2026-09-05T02:00:00Z,B02,9456712,12.8,74,28
2026-09-05T02:00:00Z,B03,9187004,15.1,96,34
MFS-MAN-24001 berth=B01 containers=418 consignee=SEMBAWANG-COLD-CHAIN
MFS-MAN-24002 berth=B03 containers=602 consignee=JURONG-PETROCHEM
MFS-MAN-24003 berth=B05 containers=275 consignee=CHANGI-AIRFREIGHT-LINK
MFS-MAN-24004 berth=B02 containers=511 consignee=TUAS-MEGA-RETAIL
MFS-MAN-24005 berth=B06 containers=733 consignee=PASIR-PANJANG-AUTOS
MFS-MAN-24006 berth=B08 containers=344 consignee=KEPPEL-MARINE-PARTS
lane_code,lane_name,rate_sgd,status
SG-CN,Singapore to Shanghai,412.50,active
SG-MY,Singapore to Port Klang,118.00,active
SG-ID,Singapore to Tanjung Priok,164.25,review
```

The queue has exactly 6 lines. That is why the Job asks for `completions: 6`. Load all three files into one ConfigMap:

```
kubectl -n kcna-lab08 create configmap berth-data \
  --from-file=data/berth-readings.csv \
  --from-file=data/manifest-queue.txt \
  --from-file=data/tariff-rates.csv
```

Expected output:

```
configmap/berth-data created
```

List the keys — each is the base name of the file it came from, which is what the containers mount:

```
kubectl -n kcna-lab08 get configmap berth-data \
  -o go-template='{{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{end}}'
```

Expected output:

```
berth-readings.csv
manifest-queue.txt
tariff-rates.csv
```

Step 2 — The DaemonSet: one Pod per node

```
kubectl apply -f manifests/10-daemonset-berth-agent.yaml
```

Expected output:

```
daemonset.apps/berth-agent created
kubectl -n kcna-lab08 get daemonset berth-agent
```

Expected output — **note the columns are entirely different from a Deployment's:**

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
berth-agent	1	1	1	1	1	<none>	18s

DESIRED 1 **was computed**, not configured. Prove there is no `replicas` field to configure:

```
kubectl -n kcna-lab08 get ds berth-agent -o jsonpath='{.spec.replicas}'; echo "-- empty"
kubectl explain daemonset.spec | grep -c 'replicas' || echo "no replicas field in the
DaemonSet spec"
```

Expected output:

```
-- empty
no replicas field in the DaemonSet spec
```

Where the count comes from:

```
kubectl get nodes --no-headers | wc -l
kubectl -n kcna-lab08 get ds berth-agent \
-o jsonpath='desired={.status.desiredNumberScheduled} ready={.status.numberReady}'; echo
```

Expected output:

```
1
desired=1 ready=1
```

One node, one Pod. On a 14-node cluster this manifest would report `desired=14` with no edit. Confirm which node the single Pod landed on:

```
kubectl -n kcna-lab08 get pods -l app=berth-agent -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
berth-agent-lq7dw	1/1	Running	0	52s	10.244.0.9	kind-control-plane
<none>	<none>					

kind and the control-plane taint. In a multi-node cluster the control-plane node normally carries `node-role.kubernetes.io/control-plane:NoSchedule`, which would keep ordinary Pods off it. `kind` removes that taint when the cluster has no worker nodes — otherwise nothing could ever be scheduled. Check rather than assume:

```
kubectl get node kind-control-plane -o jsonpath='{.spec.taints}'; echo "-- empty means untainted"
```

Expected output:

```
-- empty means untainted
```

The DaemonSet still declares a toleration for that taint. That costs nothing here and is what makes the same manifest correct on a real cluster. A toleration grants no privileges — it only permits scheduling. You will add and remove a taint yourself in Lab 09.

Read what the agent computed from the dataset:

```
kubectl -n kcna-lab08 logs -l app=berth-agent --tail=3
```

Expected output (timestamps will differ):

```
02:14:07 node=kind-control-plane berths=8 high_occupancy=3
02:14:27 node=kind-control-plane berths=8 high_occupancy=3
02:14:47 node=kind-control-plane berths=8 high_occupancy=3
```

Cross-check against the raw file — three berths are at or above 90 % occupancy (B01 92, B03 96, B06 90):

```
awk -F, 'NR>1 && $5+0 >= 90 {print $2, $5"%"}' data/berth-readings.csv
```

Expected output:

```
B01 92%
B03 96%
B06 90%
```

The agent also stamped its own node name into the log, via the downward API (`fieldRef: spec.nodeName`) — that is how node agents self-identify without a config file per node.

Step 3 — The Indexed Job: six work items, two at a time

```
kubectl apply -f manifests/20-job-manifest-reconcile.yaml
```

Expected output:

```
job.batch/manifest-reconcile created
```

Watch the parallelism cap in action:

```
kubectl -n kcna-lab08 get pods -l job-name=manifest-reconcile -w
```

Expected output — **never more than two Pods are Running at once:**

NAME	READY	STATUS	RESTARTS	AGE
manifest-reconcile-0-4k9tp	0/1	ContainerCreating	0	0s
manifest-reconcile-1-x7c2m	0/1	ContainerCreating	0	0s
manifest-reconcile-0-4k9tp	1/1	Running	0	2s
manifest-reconcile-1-x7c2m	1/1	Running	0	2s
manifest-reconcile-0-4k9tp	0/1	Completed	0	7s
manifest-reconcile-2-p8dhq	0/1	ContainerCreating	0	0s
manifest-reconcile-1-x7c2m	0/1	Completed	0	7s
manifest-reconcile-3-m2wvb	0/1	ContainerCreating	0	0s
...				

Note the Pod names: `manifest-reconcile-<index>-<random>`. Indexed Jobs put the completion index in the Pod name. Stop the watch with `Ctrl-C` once six Pods are Completed, then:

```
kubectl -n kcna-lab08 get job manifest-reconcile
```

Expected output:

NAME	STATUS	COMPLETIONS	DURATION	AGE
manifest-reconcile	Complete	6/6	26s	30s

Confirm each Pod took a **different** line of the queue:

```
kubectl -n kcna-lab08 logs -l job-name=manifest-reconcile --tail=-1 | grep RECONCILED | sort
```

Expected output:

```
index=0 RECONCILED MFS-MAN-24001 berth=B01 containers=418 consignee=SEMBAWANG-COLD-CHAIN
index=1 RECONCILED MFS-MAN-24002 berth=B03 containers=602 consignee=JURONG-PETROCHEM
index=2 RECONCILED MFS-MAN-24003 berth=B05 containers=275 consignee=CHANGI-AIRFREIGHT-LINK
index=3 RECONCILED MFS-MAN-24004 berth=B02 containers=511 consignee=TUAS-MEGA-RETAIL
index=4 RECONCILED MFS-MAN-24005 berth=B06 containers=733 consignee=PASIR-PANJANG-AUTOS
index=5 RECONCILED MFS-MAN-24006 berth=B08 containers=344 consignee=KEPPEL-MARINE-PARTS
```

Six items, six Pods, no duplicates and no locking. Each Pod read `JOB_COMPLETION_INDEX` from its environment and pulled line `index+1`. Inspect where that index is recorded:

```
kubectl -n kcna-lab08 get pods -l job-name=manifest-reconcile \
-o custom-columns=POD:.metadata.name,INDEX:.metadata.annotations.batch\\.kubernetes\\
.io/job-completion-index,STATUS:.status.phase
```

Expected output:

POD	INDEX	STATUS
manifest-reconcile-0-4k9tp	0	Succeeded
manifest-reconcile-1-x7c2m	1	Succeeded
manifest-reconcile-2-p8dhq	2	Succeeded
manifest-reconcile-3-m2wvb	3	Succeeded
manifest-reconcile-4-jn6rt	4	Succeeded
manifest-reconcile-5-zb3hf	5	Succeeded

And the Job's own record of which indexes finished:

```
kubectl -n kcna-lab08 get job manifest-reconcile \
-o jsonpath='{completed={.status.completedIndexes} succeeded={.status.succeeded}}'; echo
```

Expected output:

```
completed=0-5 succeeded=6
```

`restartPolicy: Never` is mandatory here. A Job template may only use `Never` or `OnFailure`; `Always` is rejected, because a Job that restarts forever can never complete.

Step 4 — The CronJob

```
kubectl apply -f manifests/30-cronjob-tariff-sync.yaml
```

Expected output:

```
cronjob.batch/tariff-sync created
kubectl -n kcna-lab08 get cronjob tariff-sync
```

Expected output (before the first firing):

NAME	SCHEDULE	TIMEZONE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
tariff-sync	* / 2 * * * *	Asia/Singapore	False	0	<none>	8s

Wait for the next even minute (up to two minutes), then:

```
kubectl -n kcna-lab08 get jobs -l app=tariff-sync
```

Expected output:

NAME	STATUS	COMPLETIONS	DURATION	AGE
tariff-sync-29342316	Complete	1/1	4s	35s

The Job name suffix is the **schedule time in minutes since the Unix epoch** — that is how the CronJob controller keeps runs idempotent.

```
kubectl -n kcna-lab08 logs -l app=tariff-sync --tail=-1
```

Expected output:

```
tariff-sync run at 2026-09-05T02:20:03Z
lanes=8 mean_rate_sgd=683.66
FLAGGED SG-ID Singapore to Tanjung Priok
FLAGGED SG-AE Singapore to Jebel Ali
SYNC-OK
```

Cross-check the two flagged lanes against the dataset:

```
awk -F, 'NR>1 && $4=="review" {print $1, $2}' data/tariff-rates.csv
```

Expected output:

```
SG-ID Singapore to Tanjung Priok
SG-AE Singapore to Jebel Ali
```

Inspect the scheduling controls:

```
kubectl -n kcna-lab08 get cronjob tariff-sync -o jsonpath='{.spec.concurrencyPolicy}
{.spec.startingDeadlineSeconds}
{.spec.successfulJobsHistoryLimit}/{.spec.failedJobsHistoryLimit}'; echo
```

Expected output:

```
Forbid 60 3/1
```

- **Forbid** — if the previous run is still active when the next fire time arrives, **skip** it. **Allow** (the default) would run both; **Replace** would kill the old one. A tariff sync that ran twice concurrently would double-write the rate cache, so **Forbid** is the correct choice here.
- **startingDeadlineSeconds: 60** — a run more than 60 s late (for example, because the control plane was restarting) is abandoned rather than fired late.
- **3/1** — keep the last three successful Jobs and the last failed one, so you can read their logs.

Suspend and resume it, which is what you do during a maintenance window:

```
kubectl -n kcna-lab08 patch cronjob tariff-sync -p '{"spec":{"suspend":true}}'
kubectl -n kcna-lab08 get cronjob tariff-sync -o jsonpath='suspend={.spec.suspend}'; echo
kubectl -n kcna-lab08 patch cronjob tariff-sync -p '{"spec":{"suspend":false}}'
```

Expected output:

```
cronjob.batch/tariff-sync patched
suspend=true
cronjob.batch/tariff-sync patched
```

Trigger a run immediately without waiting for the schedule — the standard incident-response move:

```
kubectl -n kcna-lab08 create job tariff-sync-manual --from=cronjob/tariff-sync
kubectl -n kcna-lab08 wait --for=condition=complete job/tariff-sync-manual --timeout=90s
kubectl -n kcna-lab08 logs job/tariff-sync-manual
```

Expected output:

```
job.batch/tariff-sync-manual created
job.batch/tariff-sync-manual condition met
tariff-sync run at 2026-09-05T02:22:41Z
lanes=8 mean_rate_sgd=683.66
FLAGGED SG-ID Singapore to Tanjung Priok
FLAGGED SG-AE Singapore to Jebel Ali
SYNC-OK
```

Step 5 — Compare the three controllers side by side

```
kubectl -n kcna-lab08 get ds,job,cronjob
```

Expected output:

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
SELECTOR AGE						
daemonset.apps/berth-agent 1	1	1	1	1	1	<none>
9m						

NAME	STATUS	COMPLETIONS	DURATION	AGE
job.batch/manifest-reconcile	Complete	6/6	26s	7m
job.batch/tariff-sync-29342316	Complete	1/1	4s	3m
job.batch/tariff-sync-manual	Complete	1/1	4s	40s

NAME	SCHEDULE	TIMEZONE	SUSPEND	ACTIVE	LAST
SCHEDULE AGE					
cronjob.batch/tariff-sync 4m	*/* * * * *	Asia/Singapore	False	0	2m

Three different `STATUS`/count vocabularies for three different lifecycles: *per-node*, *run-to-completion*, *scheduled*.

Lab 08 · Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script reads all three files in `data/`, derives the expected berth count, queue length and review-lane count from them, and asserts the cluster agrees. It also compares `desiredNumberScheduled` against the live node count rather than hard-coding 1, so it is correct on a multi-node cluster too. Full transcript: `verification/expected-output.md`.

Lab 08 · Failure injection — a Job that exhausts its `backoffLimit`

The customs export job was written against a file that is not in the ConfigMap. Apply it:

```
kubectl apply -f manifests/40-job-customs-export-broken.yaml
```

Expected output:

```
job.batch/customs-export created
```

Watch the retries:

```
kubectl -n kcna-lab08 get pods -l job-name=customs-export -w
```

Expected output — three Pods, each `Error`, then it stops:

NAME	READY	STATUS	RESTARTS	AGE
customs-export-9wq4k	0/1	Pending	0	0s
customs-export-9wq4k	0/1	Running	0	2s
customs-export-9wq4k	0/1	Error	0	3s
customs-export-cn2xr	0/1	Running	0	13s
customs-export-cn2xr	0/1	Error	0	14s
customs-export-4tbvz	0/1	Running	0	38s
customs-export-4tbvz	0/1	Error	0	39s

Stop the watch. The real error from the container:

```
kubectl -n kcna-lab08 logs -l job-name=customs-export --tail=-1 | head -6
```

Real output:

```
customs-export starting
wc: /data/customs-export.csv: No such file or directory
EXPORT-FAILED: source file missing
```

The Job's own verdict:

```
kubectl -n kcna-lab08 get job customs-export
```

Expected output:

NAME	STATUS	COMPLETIONS	DURATION	AGE
customs-export	Failed	0/1	41s	2m

```
kubectl -n kcna-lab08 describe job customs-export | sed -n '/^Events:/, $p'
```

Real output:

Events:	Type	Reason	Age	From	Message
----	----	-----	----	----	-----

```

Normal    SuccessfulCreate    2m10s    job-controller    Created pod: customs-export-9wq4k
Normal    SuccessfulCreate    2m       job-controller    Created pod: customs-export-cn2xr
Normal    SuccessfulCreate    95s      job-controller    Created pod: customs-export-4tbvz
Warning   BackoffLimitExceeded 89s      job-controller    Job has reached the specified
backoff limit

kubectl -n kcna-lab08 get job customs-export \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}
msg={.message}{"\n"}{end} '

```

Expected output:

```
Failed=True reason=BackoffLimitExceeded msg=Job has reached the specified backoff limit
```

Diagnosis. Three facts explain the whole event trail:

1. `backoffLimit: 2` means **1 initial attempt plus 2 retries = 3 Pods**, then stop. Count the `SuccessfulCreate` events: exactly three.
2. `restartPolicy: Never` means each failure creates a **new Pod** rather than restarting the container in place. With `restartPolicy: OnFailure` you would instead see one Pod with a rising `RESTARTS` count.
3. The retries were 10 s, then 25 s apart — the controller applies exponential back-off (capped at 6 minutes) between attempts.

The root cause is in the container log, not in Kubernetes: the mounted ConfigMap has no `customs-export.csv` key. Confirm:

```
kubectl -n kcna-lab08 exec -it "$(kubectl -n kcna-lab08 get pods -l app=berth-agent -o
name | head -1)" -- ls -l /data
```

Expected output:

```
berth-readings.csv
manifest-queue.txt
tariff-rates.csv
```

Fix. A Job's `.spec.template` is immutable, so you cannot patch this Job — you must delete it and re-create with a corrected spec. Point it at a file that exists:

```
kubectl -n kcna-lab08 delete job customs-export
sed 's#/data/customs-export.csv#/data/manifest-queue.txt#' \
manifests/40-job-customs-export-broken.yaml | kubectl apply -f -
kubectl -n kcna-lab08 wait --for=condition=complete job/customs-export --timeout=90s
kubectl -n kcna-lab08 logs job/customs-export
```

Expected output:

```
job.batch "customs-export" deleted
job.batch/customs-export created
job.batch/customs-export condition met
customs-export starting
      6 /data/manifest-queue.txt
EXPORT-OK
```

Re-run `bash verification/checks.sh` **before** this fix if you want the `BackoffLimitExceeded` assertions to pass; after the fix the Job is `Complete` and the script reports the optional-skip line instead.

Lab 08 · Troubleshooting

Symptom	Cause	Fix
Job has reached the specified backoff limit / Failed=True reason=BackoffLimitExceeded	The container exited non-zero more times than backoffLimit allows	Read <code>kubectl logs -l job-name=<job> --tail=-1</code> for the application error; a Job spec is immutable, so delete and re-create with the fix
DaemonSet shows DESIRED 0	No node satisfies the Pod's nodeSelector/affinity, or every node carries a taint the Pod does not tolerate	<code>kubectl describe ds <name></code> ; check <code>kubectl get nodes --show-labels</code> and <code>kubectl get node <n> -o jsonpath='{.spec.taints}'</code>
The Job "x" is invalid: spec.template.spec.restartPolicy : Unsupported value: "Always"	A Job template may only use Never or OnFailure	Set restartPolicy: Never (new Pod per failure) or OnFailure (restart in place)
Indexed Job Pods all process the same item	completionMode was left at the default NonIndexed, so JOB_COMPLETION_INDEX is not set	Set completionMode: Indexed; the index is also available as the batch.kubernetes.io/job-completion-index annotation
CronJob shows LAST SCHEDULE <none> long after its fire time	suspend: true, or every fire was older than startingDeadlineSeconds, or the schedule is being read in a different time zone	<code>kubectl get cronjob x -o jsonpath='{.spec.suspend}{.spec.timeZone}'</code> ; set .spec.timeZone explicitly rather than relying on the controller's zone
CronJob Jobs pile up and overlap	concurrencyPolicy: Allow (the default) with runs longer than the interval	Set Forbid to skip, or Replace to cancel the running Job; also set activeDeadlineSeconds on the jobTemplate
error: cannot patch Job ... spec.template: field is immutable	Attempt to edit a live Job's Pod template	Delete the Job and apply a corrected manifest
Job logs are gone minutes after completion	ttlSecondsAfterFinished elapsed, or the CronJob history limit evicted the Job	Raise ttlSecondsAfterFinished / successfulJobsHistoryLimit, or ship logs off-cluster

Lab 08 · Cleanup

Delete **only** this lab's namespace. The DaemonSet, all Jobs, the CronJob and the ConfigMap are inside it.

```
kubectl delete namespace kcna-lab08
```

Expected output:

```
namespace "kcna-lab08" deleted
kubectl get ns kcna-lab08
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab08" not found
```

Lab 08 · What you learned

- **DaemonSet** = one Pod per matching node. There is no `replicas` field; `status.desiredNumberScheduled` is derived from the node set, so a **single-node kind cluster yields exactly one Pod** and a 14-node cluster yields 14 from the identical YAML.
- A node agent should carry a toleration for the control-plane taint so it also runs on control-plane nodes. `kind` untaints the control plane on single-node clusters — check with `kubectl get node <n> -o jsonpath='{.spec.taints}'` rather than assuming either way.
- **Job**: `completions` is how many successes are required, `parallelism` is how many may run at once, `backoffLimit` is how many failures are tolerated (initial attempt + retries).
- `completionMode: Indexed` gives each Pod a unique `JOB_COMPLETION_INDEX`, which turns a static work list into a parallel batch with no queue broker and no locking.
- A Job Pod template may only use `restartPolicy: Never` or `OnFailure`, and a live Job's template is immutable.
- **CronJob** wraps a jobTemplate with `schedule`, an explicit `timeZone`, `concurrencyPolicy` (`Allow/Forbid/Replace`), `startingDeadlineSeconds` and `history limits`. `kubectl create job --from=cronjob/<name>` fires one immediately.
- Choosing the controller is a design decision, not a formality: a run-to-completion task modelled as a Deployment restarts forever.

Lab 08 · Further reading

- DaemonSet — <https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/>
- Job — <https://kubernetes.io/docs/concepts/workloads/controllers/job/>
- Indexed Job for parallel processing — <https://kubernetes.io/docs/tasks/job/indexed-parallel-processing-static/>
- CronJob — <https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>
- Automatic cleanup for finished Jobs (`ttlSecondsAfterFinished`) — <https://kubernetes.io/docs/concepts/workloads/controllers/ttlafterfinished/>
- Taints and Tolerations — <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>

Lab 09 — Scheduling: nodeSelector, Affinity, Taints and Tolerations

Field	Value
Lab ID	Lab 09
Title	Scheduling: nodeSelector, Affinity, Taints and Tolerations
Day / Topic	Day 2 · Workloads & Scheduling
Duration	55 minutes
Namespace	kcna-lab09
Learning outcome	LO2 — Identify the technical and practical requirements in a Kubernetes setup.
Ability	A3 Identify technical and practical requirements as well as stakeholders' demands
Knowledge	K3 Objectives of solution architecture
Deck slide	Slide 211
Repository path	courseware/labs/lab-09-scheduling-affinity-taints/

Goal. Create the placement conditions yourself on a single-node kind cluster - label the node, taint it, untaint it - and prove how nodeSelector, required vs preferred nodeAffinity and tolerations decide where a Pod lands, including one Pod left Pending forever.

What you will produce:

- A node labelled from data/node-placement-policy.csv and temporarily tainted `mfs.io/maintenance=window:NoSchedule`
- Four Pods placed by nodeSelector, required affinity, preferred affinity and a toleration
- Two Pods stuck Pending with readable FailedScheduling events: one unsatisfiable affinity, one missing toleration

Lab 09 · Objective

By the end of this lab you will be able to:

1. Describe scheduling as **filter then score**: hard constraints eliminate nodes, soft preferences only rank the survivors.
2. Place a Pod with `nodeSelector` and with `nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution`, and explain what affinity buys you over `nodeSelector`.
3. Show that an unsatisfied `preferredDuringSchedulingIgnoredDuringExecution` term costs a node points but **never** blocks placement.
4. Add and remove a **taint** on a node, and write the matching **toleration**.
5. Read a `FailedScheduling` event and name the exact predicate that rejected every node.

Read this before you start — single-node reality

This cluster has **one node**. You cannot demonstrate placement by comparing node A with node B, so **you will create the conditions yourself**: you label the node, you taint the node, you remove the taint. Everything you do here is exactly what you would do on a 50-node cluster; only the node count differs.

Two consequences to keep in front of you:

- When a Pod goes `Pending`, the scheduler's message says *"0/1 nodes are available"*. On a production cluster it would read *"0/50 nodes are available"* with a breakdown per predicate. The mechanism is identical.
- `kind` **removes** the `node-role.kubernetes.io/control-plane:NoSchedule` taint on a single-node cluster, because otherwise nothing could ever run. Do not take that on trust — Step 1 checks it.

Lab 09 · Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME                STATUS    ROLES    AGE   VERSION
kind-control-plane  Ready    control-plane  68m   v1.31.0
```

You need permission to label and taint nodes (both are cluster-scoped writes). Confirm:

```
kubectl auth can-i patch nodes
```

Expected output:

```
yes
```

Work from the lab directory and capture the node name in a shell variable — every command below uses it, so the lab works whatever your cluster is called:

```
cd courseware/labs/lab-09-scheduling-affinity-taints
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "NODE=$NODE"
```

Expected output:

```
NODE=kind-control-plane
```

Lab 09 · Scenario

Marina Freight Systems (MFS) is preparing to move the *Harbour* platform onto hardware that is not uniform. The production cluster will have three classes of node:

- `sg-harbourfront` nodes wired directly to the gate-scanner camera network,
- SSD/NVMe nodes for the cargo-manifest store,
- a small `gpu-a100` pool that has been **ordered but not yet delivered**.

The platform lead has written `data/node-placement-policy.csv` — the contract between the workload owners and the platform team. Your task is to encode that contract in Kubernetes placement primitives and, crucially, to demonstrate what each one does **when it cannot be satisfied**, because half of the policy refers to hardware that does not exist yet.

You must also rehearse the **monthly maintenance window**: the node is tainted, batch work must stay off it, and the safety-critical crane controller must keep running.

Lab 09 · Step-by-step procedure

Step 1 — Inspect the node you are about to steer

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab09 created
```

Look at the labels the node already has:

```
kubectl get node "$NODE" --show-labels
```

Expected output (one long line, wrapped here):

```
NAME                STATUS    ROLES    AGE   VERSION   LABELS
kind-control-plane   Ready     control-plane   69m   v1.31.0   beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,kubernetes.io/arch=arm64,kubernetes.io/hostname=kind-control-plane,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=
```

Every one of those is a legal `nodeSelector` target. Note `kubernetes.io/os=linux` — the `eta-predictor` Pod uses it as a hard floor later.

Now check the taint situation instead of assuming it.

```
kubectl get node "$NODE" -o jsonpath='{.spec.taints}'; echo " <-- empty means no taints"
```

Expected output on a single-node kind cluster:

```
<-- empty means no taints
kubectl describe node "$NODE" | grep -A2 '^Taints:'
```

Expected output:

```
Taints:                <none>
Unschedulable:         false
```

`kind` removed the control-plane taint when it built this single-node cluster. On a multi-node cluster (or a `kubeadm` cluster) you would instead see:

```
Taints:                node-role.kubernetes.io/control-plane:NoSchedule
```

and ordinary Pods would be kept off it. **Check, do not assume** — this is the single most common reason a lab "works on my cluster and not yours".

Step 2 — Read the placement policy and label the node from it

```
column -s, -t data/node-placement-policy.csv
```

Expected output:

workload	manifest	mechanism
node_label_key	node_label_value	toleration_key
toleration_effect	expected_phase	toleration_value
gate-scanner	10-pod-nodeselector-gate-scanner.yaml	nodeSelector
mfs.io/zone	sg-harbourfront	yes
Running		
manifest-store	20-pod-affinity-required-manifest-store.yaml	requiredNodeAffinity
mfs.io/disk	ssd	yes
Running		
eta-predictor	30-pod-affinity-preferred-eta-predictor.yaml	preferredNodeAffinity
mfs.io/zone	sg-tuas	no
Running		
draft-optimiser	40-pod-affinity-unsatisfiable.yaml	requiredNodeAffinity
mfs.io/hardware	gpu-a100	no

Pending				
yard-report	50-pod-no-toleration-yard-report.yaml		none	
no			Pending	
crane-controller	60-pod-toleration-crane-controller.yaml		toleration	
no	mfs.io/maintenance window	NoSchedule	Running	

The `apply_label` column tells you exactly which labels this node is supposed to carry. **Two rows say yes — apply only those:**

```
awk -F, 'NR>1 && $6=="yes" {print $4=="$5}" data/node-placement-policy.csv
```

Expected output:

```
mfs.io/zone=sg-harbourfront
mfs.io/disk=ssd
```

Apply them:

```
kubectl label node "$NODE" mfs.io/zone=sg-harbourfront mfs.io/disk=ssd
```

Expected output:

```
node/kind-control-plane labeled
```

Confirm — note that `mfs.io/hardware` and `mfs.io/zone=sg-tuas` are deliberately **absent**, because that hardware has not arrived:

```
kubectl get node "$NODE" -o jsonpath='{.metadata.labels["mfs.io/zone"]}'
disk={.metadata.labels["mfs.io/disk"]} hardware={.metadata.labels["mfs.io/hardware"]}";
echo
```

Expected output:

```
zone=sg-harbourfront disk=ssd hardware=
kubectl get nodes -l mfs.io/zone=sg-harbourfront
kubectl get nodes -l mfs.io/hardware=gpu-a100
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	71m	v1.31.0
No resources found				

One node satisfies the zone selector; zero satisfy the GPU selector. That second result is the whole of Step 6 in advance.

Step 3 — *nodeSelector: the blunt instrument*

```
grep -A2 'nodeSelector:' manifests/10-pod-nodeselector-gate-scanner.yaml
kubectl apply -f manifests/10-pod-nodeselector-gate-scanner.yaml
```

Expected output:

```
nodeSelector:
  mfs.io/zone: sg-harbourfront
containers:
  pod/gate-scanner created
kubectl -n kcna-lab09 get pod gate-scanner -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
gate-scanner	1/1	Running	0	8s	10.244.0.12	kind-control-plane
<none>						

`nodeSelector` is a flat map of equality matches, ANDed. There is no `In`, no `NotIn`, no `OR` and no weighting. It is the right tool when the rule really is "this exact label".

Step 4 — Required node affinity: the same idea with operators

```
sed -n '/affinity:/,/containers:/p' manifests/20-pod-affinity-required-manifest-store.yaml
```

Expected output:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: mfs.io/disk
              operator: In
              values:
                - ssd
                - nvme
            - key: mfs.io/decommission
              operator: DoesNotExist
containers:
```

Read it precisely:

- `nodeSelectorTerms` are **ORed** — a node matching any one term is a candidate.
- `matchExpressions` inside a single term are **ANDed** — this node must have `mfs.io/disk` in `{ssd, nvme}` **and** must not carry `mfs.io/decommission` at all.

```
kubectl apply -f manifests/20-pod-affinity-required-manifest-store.yaml
kubectl -n kcna-lab09 get pod manifest-store -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/manifest-store created						
NOMINATED NODE	READYNESS	GATES				
manifest-store	1/1	Running	0	6s	10.244.0.13	kind-control-plane
<none>	<none>					

The node has `mfs.io/disk=ssd` (in the value list) and no `mfs.io/decommission` key, so it survives both expressions.

The suffix **IgnoredDuringExecution** matters. If you now removed `mfs.io/disk` from the node, this Pod would **keep running** — node affinity is evaluated at scheduling time only. Prove it if you like, then re-apply the label:

```
kubectl label node "$NODE" mfs.io/disk-
kubectl -n kcna-lab09 get pod manifest-store
kubectl label node "$NODE" mfs.io/disk=ssd
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
node/kind-control-plane unlabeled						
NOMINATED NODE	READYNESS	GATES				
manifest-store	1/1	Running	0	70s	10.244.0.13	kind-control-plane
<none>	<none>					
node/kind-control-plane labeled						

Step 5 — Preferred node affinity: a wish, not a rule

```
sed -n '/preferredDuringScheduling/,/containers:/p' manifests/30-pod-affinity-preferred-eta-predictor.yaml
```

Expected output:

```
preferredDuringSchedulingIgnoredDuringExecution:
- weight: 80
  preference:
    matchExpressions:
      - key: mfs.io/zone
        operator: In
        values:
          - sg-tuas
- weight: 20
  preference:
    matchExpressions:
      - key: mfs.io/disk
        operator: In
        values:
          - ssd
          - nvme

containers:
```

The **sg-tuas** zone does not exist on this cluster. The weight-80 preference therefore cannot be satisfied. Predict what happens, then:

```
kubectl apply -f manifests/30-pod-affinity-preferred-eta-predictor.yaml
kubectl -n kcna-lab09 get pod eta-predictor -o wide
```

Expected output:

```
pod/eta-predictor created
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
NOMINATED NODE   READINESS GATES
eta-predictor    1/1     Running   0           7s    10.244.0.14     kind-control-plane
<none>          <none>
```

It scheduled anyway. The node scored 20 out of a possible 100 on the affinity plugin and lost 80 points it could have had — but scoring only ranks candidates, it never eliminates them. The `required` term (`kubernetes.io/os in (linux)`) was satisfied, and that is the only part that could have blocked it.

This is the single most important distinction in the whole lab. Say it back before moving on:

Term	If satisfied	If not satisfied
<code>requiredDuringSchedulingIgnoredDuringExecution</code>	node stays a candidate	node eliminated — Pod may go <code>Pending</code> forever
<code>preferredDuringSchedulingIgnoredDuringExecution</code>	node gains <code>weight</code> points	node loses those points, still a candidate

Step 6 — Taints: repel work from a node

The monthly maintenance window is starting. Batch jobs must stay off the node; the crane controller must not.

Apply the taint from the policy file:

```
awk -F, 'NR>1 && $7!="" {print $7"="$8":"$9}' data/node-placement-policy.csv
```

Expected output:

```
mfs.io/maintenance=window:NoSchedule
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule
```

Expected output:


```
node/kind-control-plane tainted
kubectl describe node "$NODE" | grep -A1 '^Taints:'
```

Expected output:

```
Taints:          mfs.io/maintenance=window:NoSchedule
Unschedulable:   false
```

First, confirm what a NoSchedule taint does **not** do:

```
kubectl -n kcna-lab09 get pods -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
eta-predictor	1/1	Running	0	3m	10.244.0.14	kind-control-plane
<none>	<none>					
gate-scanner	1/1	Running	0	8m	10.244.0.12	kind-control-plane
<none>	<none>					
manifest-store	1/1	Running	0	5m	10.244.0.13	kind-control-plane
<none>	<none>					

Nothing was evicted. NoSchedule affects *future* scheduling decisions only. The effect that evicts running Pods is NoExecute; the third effect, PreferNoSchedule, is a soft version that only lowers the node's score.

Now try to schedule a Pod with no toleration:

```
kubectl apply -f manifests/50-pod-no-toleration-yard-report.yaml
sleep 5
kubectl -n kcna-lab09 get pod yard-report
```

Expected output:

```
pod/yard-report created
NAME      READY   STATUS    RESTARTS   AGE
yard-report 0/1     Pending   0           5s
```

Read why:

```
kubectl -n kcna-lab09 describe pod yard-report | sed -n '/^Events:/, $p'
```

Real output:

```
Events:
  Type     Reason             Age   From                  Message
  ----     -
Warning   FailedScheduling   12s   default-scheduler     0/1 nodes are available: 1 node(s)
had untolerated taint {mfs.io/maintenance: window}. preemption: 0/1 nodes are available: 1
Preemption is not helpful for scheduling.
```

The message names the taint, its value and how many nodes it eliminated. Now the crane controller, which tolerates it:

```
grep -A5 'tolerations:' manifests/60-pod-toleration-crane-controller.yaml
kubectl apply -f manifests/60-pod-toleration-crane-controller.yaml
kubectl -n kcna-lab09 get pod crane-controller -o wide
```

Expected output:

```
tolerations:
- key: mfs.io/maintenance
  operator: Equal
  value: window
  effect: NoSchedule
pod/crane-controller created
NAME      READY   STATUS    RESTARTS   AGE   IP      NODE
```

NOMINATED NODE	READINESS	GATES				
crane-controller	1/1	Running	0	6s	10.244.0.16	kind-control-plane
<none>	<none>					

Side by side:

```
kubectl -n kcna-lab09 get pods -l 'placement in (no-toleration,toleration)' \
-o custom-
columns=POD:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName,TOLERATES:.spec.tolera
tions[0].key
```

Expected output:

POD	STATUS	NODE	TOLERATES
crane-controller	Running	kind-control-plane	mfs.io/maintenance
yard-report	Pending	<none>	<none>

A toleration is permission, not attraction. `crane-controller` was not *drawn* to the tainted node; it was merely *allowed* onto it. If you needed to both allow and steer, you would add a `nodeSelector` or `nodeAffinity` as well. Taints and node affinity are complementary controls, not alternatives — affinity is written by the *workload* owner ("I need SSD"), taints are written by the *node* owner ("stay off unless invited").

The maintenance window is over. Remove the taint — note the trailing minus:

```
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule-
sleep 5
kubectl -n kcna-lab09 get pod yard-report -o wide
```

Expected output:

node/kind-control-plane untainted							
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
NOMINATED NODE	READINESS	GATES					
yard-report	1/1	Running	0	2m	10.244.0.17	kind-control-plane	<none>
<none>							

The scheduler retried on its own — a `Pending` Pod stays in the scheduling queue and is re-evaluated whenever cluster state changes. Nothing was recreated; check the `AGE` column.

Re-apply the taint before running the verification script, because `checks.sh` asserts the maintenance-window end state:

```
kubectl -n kcna-lab09 delete pod yard-report
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule
kubectl apply -f manifests/50-pod-no-toleration-yard-report.yaml
```

Expected output:

```
pod "yard-report" deleted
node/kind-control-plane tainted
pod/yard-report created
```

Lab 09 · Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script parses `data/node-placement-policy.csv` and, for every row, asserts (a) the node carries the labels whose `apply_label` is `yes`, and (b) the Pod reached the `expected_phase` in the policy.

Run it after section 6 and before section 8. The policy has six rows, and the sixth Pod (`draft-optimiser`) is only created in the failure-injection section. The script also requires the `mfs.io/maintenance` taint to still be applied, which is the end state of Step 6. Running it earlier will fail on the missing Pod and the missing taint — that is the script working, not the script being wrong.

Full transcript: `verification/expected-output.md`.

Lab 09 · Failure injection — an affinity that can never be satisfied

The GPU nodes were ordered in Q2 and have not shipped. The analytics team deploys against them anyway.

```
kubectl apply -f manifests/40-pod-affinity-unsatisfiable.yaml
```

Expected output:

```
pod/draft-optimiser created
sleep 10
kubectl -n kcna-lab09 get pod draft-optimiser
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
draft-optimiser	0/1	Pending	0	10s

Wait a minute and check again — it will still be `Pending`. **This is a terminal state, not a transient one.** Diagnose it:

```
kubectl -n kcna-lab09 describe pod draft-optimiser | sed -n '/^Events:/, $p'
```

Real output:

```
Events:
  Type            Reason             Age           From              Message
  ----            -
  Warning         FailedScheduling   63s          default-scheduler  0/1 nodes are
available: 1 node(s) didn't match Pod's node affinity/selector.
preemption: 0/1 nodes are available: 1 Preemption is not helpful
for scheduling.
```

The scheduling condition:

```
kubectl -n kcna-lab09 get pod draft-optimiser \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}
msg={.message}{"\n"}{end}'
```

Expected output:

```
PodScheduled=False reason=Unschedulable msg=0/1 nodes are available: 1 node(s) didn't
match Pod's node affinity/selector. preemption: 0/1 nodes are available: 1 Preemption is
not helpful for scheduling.
```

And no node was ever assigned:

```
kubectl -n kcna-lab09 get pod draft-optimiser -o jsonpath='{.spec.nodeName}';
echo
```

Expected output:

```
nodeName=[]
```

Diagnosis — read the message in three parts.

1. `0/1 nodes are available` — the scheduler considered every node in the cluster. One node, zero survivors. On a 50-node cluster this would read `0/50` with a per-reason tally.

- 1 node(s) didn't match Pod's node affinity/selector — the *filter* phase eliminated it. This is a **hard** constraint failing, so no amount of waiting will help.
3. preemption: ... Preemption is not helpful — the scheduler also checked whether evicting lower-priority Pods would free up a *suitable* node. It would not, because the node is not suitable at all — it is missing a label, not short of capacity. **That last clause is how you distinguish "wrong node" from "full node"**. A capacity failure instead says `Insufficient cpu` or `Insufficient memory`.

Find the offending term yourself:

```
kubectl -n kcna-lab09 get pod draft-optimiser \
-o
jsonpath='{.spec.affinity.nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution.node
SelectorTerms[0].matchExpressions[0]}'; echo
kubectl get nodes -l mfs.io/hardware=gpu-a100
```

Expected output:

```
{"key":"mfs.io/hardware","operator":"In","values":["gpu-a100"]}
No resources found
```

Fix — pick one, and understand the difference.

Option A: the hardware genuinely arrived, so label the node. (In this lab that would be a lie, so undo it afterwards.)

```
kubectl label node "$NODE" mfs.io/hardware=gpu-a100
sleep 5
kubectl -n kcna-lab09 get pod draft-optimiser -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
draft-optimiser	1/1	Running	0	3m	10.244.0.18	kind-control-plane
<none>	<none>					

The Pod was never recreated — its `AGE` is unchanged. It sat in the scheduling queue and was placed the moment the cluster could satisfy it.

Restore the honest state before verifying:

```
kubectl -n kcna-lab09 delete pod draft-optimiser
kubectl label node "$NODE" mfs.io/hardware-
kubectl apply -f manifests/40-pod-affinity-unsatisfiable.yaml
```

Expected output:

```
pod "draft-optimiser" deleted
node/kind-control-plane unlabeled
pod/draft-optimiser created
```

Option B: downgrade the requirement to a preference. This is what the analytics team should have written, because the model runs on CPU too — just slower:

```
sed
's/requiredDuringSchedulingIgnoredDuringExecution:/preferredDuringSchedulingIgnoredDuringE
xecution:/' \
manifests/40-pod-affinity-unsatisfiable.yaml | kubectl diff -f - | head -20
```

This prints the diff without applying it, so the `Pending` evidence stays intact for the verification step. **A hard requirement for hardware you do not own is the most common self-inflicted `Pending` in production.**

Lab 09 · Troubleshooting

Symptom	Cause	Fix
0/N nodes are available: N node(s) didn't match Pod's node affinity/selector.	A <code>nodeSelector</code> or a required <code>nodeAffinity</code> term matches no node	<code>kubectl get nodes -l <key>=<value></code> to confirm; label a node, or downgrade the term to <code>preferred</code>
0/N nodes are available: N node(s) had untolerated taint {key: value}.	The node is tainted and the Pod has no matching toleration	Add a toleration with matching <code>key</code> , <code>value</code> and <code>effect</code> , or remove the taint with <code>kubectl taint node <n> key=value:Effect-</code>
0/N nodes are available: N Insufficient cpu.	Capacity, not placement — the node's allocatable CPU is already committed	Lower <code>resources.requests</code> , scale down other workloads, or add a node (see Lab 10)
Pod is <code>Pending</code> with no events at all	Events older than roughly one hour are garbage-collected	<code>kubectl -n <ns> delete pod <p></code> and re-apply to regenerate the event, or read <code>.status.conditions[?(@.type=="PodScheduled")].message</code> which never expires
A <code>preferred</code> affinity term is ignored	Working as designed — preferences only contribute to the node score	If the placement is mandatory, move the term to <code>requiredDuringSchedulingIgnoredDuringExecution</code>
Pod keeps running after you delete the node label it required	<code>IgnoredDuringExecution</code> — node affinity is evaluated only at scheduling time	Use a <code>NoExecute</code> taint if you need running Pods evicted; there is no <code>RequiredDuringExecution</code> affinity
error: at least one taint update is required	The <code>kubectl taint</code> argument was malformed — the effect or the trailing <code>-</code> is missing	Add: <code>key=value:NoSchedule</code> ; remove: <code>key=value:NoSchedule-</code>
Tainting the node makes system Pods (CoreDNS, CNI) go <code>Pending</code> too	On a single-node cluster your taint applies to the only node, and add-ons without that toleration are affected	Untaint promptly: <code>kubectl taint node <n> mfs.io/maintenance=window:NoSchedule-</code> ; never leave a lab taint in place

Lab 09 · Cleanup

Delete **only** this lab's namespace — and then revert the two cluster-scoped changes you made to the node. Node labels and taints live **outside** the namespace, so deleting the namespace does not remove them, and leaving a `NoSchedule` taint behind will break every lab that follows.

```
kubectl delete namespace kcna-lab09
```

Expected output:

```
namespace "kcna-lab09" deleted
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
kubectl taint node "$NODE" mfs.io/maintenance=window:NoSchedule-
kubectl label node "$NODE" mfs.io/zone- mfs.io/disk-
```

Expected output:

```
node/kind-control-plane untainted
node/kind-control-plane unlabeled
```

Confirm the node is back to how you found it:

```
kubectl get node "$NODE" -o jsonpath='taints={.spec.taints}'; echo
kubectl get node "$NODE" --show-labels | grep -c 'mfs.io' || echo "no mfs.io labels remain"
kubectl get ns kcna-lab09
```

Expected output:

```
taints=
no mfs.io labels remain
Error from server (NotFound): namespaces "kcna-lab09" not found
```

Lab 09 · What you learned

- Scheduling is **filter then score**. Filtering (predicates) eliminates nodes; scoring ranks whatever survives. A Pod goes `Pending` only when filtering leaves zero nodes.
- `nodeSelector` is a flat AND of equality matches. `nodeAffinity` does the same job with operators (`In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`), OR across `nodeSelectorTerms` and AND within `matchExpressions`.
- `requiredDuringSchedulingIgnoredDuringExecution` can leave a Pod `Pending` **forever**. `preferredDuringSchedulingIgnoredDuringExecution` never can — it only moves points.
- `IgnoredDuringExecution` means the constraint is checked once, at scheduling time. Removing the label later does not evict the Pod.
- **Taints repel, tolerations permit**. `NoSchedule` blocks new placements, `NoExecute` also evicts running Pods, `PreferNoSchedule` merely lowers the score. A toleration never *attracts* a Pod — pair it with affinity when you also need to steer.
- Node affinity is written by the workload owner; taints are written by the node owner. They are complementary, not interchangeable.
- The `FailedScheduling` message is a complete diagnosis: node tally, failing predicate, and whether preemption would help. Preemption is not helpful means the node is *unsuitable*; `Insufficient cpu` means it is merely *full*.
- Node labels and taints are **cluster-scoped**. Deleting a namespace does not undo them — clean them up explicitly.

Lab 09 · Further reading

- Assigning Pods to Nodes — <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>
- Taints and Tolerations — <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>
- Kubernetes Scheduler — <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>
- Pod Priority and Preemption — <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/>
- Well-Known Labels, Annotations and Taints — <https://kubernetes.io/docs/reference/labels-annotations-taints/>
- Pod Topology Spread Constraints — <https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

Lab 10 — Resource Requests, Limits, QoS and Autoscaling

Field	Value
Lab ID	Lab 10
Title	Resource Requests, Limits, QoS and Autoscaling
Day / Topic	Day 2 · Container Orchestration Fundamentals
Duration	50 minutes
Namespace	kcna-lab10
Learning outcome	LO2 — Identify the technical and practical requirements in a Kubernetes setup.
Ability	A3 Identify technical and practical requirements as well as stakeholders' demands
Knowledge	K3 Objectives of solution architecture
Deck slide	Slide 237
Repository path	courseware/labs/lab-10-resources-requests-limits-autoscaling/

Goal. Apply the MFS workload resource profiles, prove all three QoS classes from the live API, watch a container get OOMKilled by its own memory limit, then bound the namespace with a LimitRange and ResourceQuota and read an HPA honestly on a cluster with no metrics-server.

What you will produce:

- Three Pods proved to be Guaranteed, Burstable and BestEffort via `.status.qosClass`
- An OOMKilled container showing `reason=OOMKilled` and exit code 137
- A LimitRange plus ResourceQuota that reject an over-budget Pod at admission, and an autoscaling/v2 HPA on the quote-engine Deployment

Lab 10 · Objective

By the end of this lab you will be able to:

1. State precisely what a **request** does (scheduling, and the denominator of HPA utilisation) and what a **limit** does (CPU throttling, memory OOMKill).
2. Prove all three **QoS classes** — `Guaranteed`, `Burstable`, `BestEffort` — from the live API with `kubect1 get pod -o jsonpath='{.status.qosClass}'`, and explain the rule that produced each.
3. Cause and diagnose an **OOMKill**: `reason=OOMKilled`, exit code 137.
4. Apply a **LimitRange** (per-container defaults and bounds) and a **ResourceQuota** (aggregate namespace budget), and read the admission rejection each produces.
5. Author an `autoscaling/v2 HorizontalPodAutoscaler` and interpret its status **honestly on a cluster with no metrics-server**.

Read this before you start — the metrics-server caveat

A stock `kind` cluster does not run metrics-server. There is no `metrics.k8s.io` API, so:

- `kubect1 top pod` fails with error: Metrics API not available, and
- your HPA will show `TARGETS: cpu: <unknown>/60%` with `ScalingActive=False`, `reason=FailedGetResourceMetric`.

That is the **correct and expected** result, and this lab treats it as the primary path. It is not a broken manifest and not a broken cluster — the measurement pipeline is simply absent. You will read the HPA's own conditions to prove that diagnosis rather than guess at it.

Section 7 gives the optional install if your instructor wants live metrics. It modifies `kube-system`, which is **outside this lab's namespace**, so it is instructor-run and explicitly opt-in.

Lab 10 · Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME          STATUS    ROLES          AGE   VERSION
kind-control-plane Ready    control-plane  84m   v1.31.0
```

Check how much the single node actually has to give out — this is the budget every request in this lab is drawn against:

```
kubectl get node -o custom-
columns=NODE:.metadata.name,CPU_ALLOC:.status.allocatable.cpu,MEM_ALLOC:.status.allocatable.memory,PODS:.status.allocatable.pods
```

Expected output (values depend on how much you gave Docker Desktop):

```
NODE          CPU_ALLOC  MEM_ALLOC  PODS
kind-control-plane 8          8039488Ki  110
```

Confirm the metrics API really is absent, so the rest of the lab is grounded in fact:

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

Expected output on a stock kind cluster:

```
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io"
not found
kubectl top nodes
```

Expected output:

```
error: Metrics API not available
```

Write that down. It is the evidence for section 6b.

```
cd courseware/labs/lab-10-resources-requests-limits-autoscaling
ls manifests
```

Expected output:

```
00-namespace.yaml          31-resourcequota.yaml
10-pod-qos-guaranteed.yaml  40-deployment-quote-engine.yaml
11-pod-qos-burstable.yaml   50-hpa-quote-engine.yaml
12-pod-qos-besteffort.yaml  60-pod-quota-violation.yaml
20-pod-oomkill-ledger-compactor.yaml
30-limitrange.yaml
```


Lab 10 · Scenario

Marina Freight Systems (MFS) has been running its staging namespace with no resource declarations at all. Last month a runaway ledger compaction consumed 6 GiB on a shared node and evicted the container-tracking API during the morning gate rush.

The platform lead's remediation plan is in `data/workload-profiles.csv`: every Harbour workload now has an agreed CPU/memory profile and a **required QoS class**. Finance additionally wants a hard ceiling on what the staging namespace can consume, so no single team can repeat the incident.

Your job:

1. Apply the profiles exactly as written and **prove** each workload landed in its required QoS class.
2. Reproduce the compaction incident safely, at 32 MiB instead of 6 GiB.
3. Install the namespace guardrails and show one of them rejecting an over-budget Pod.
4. Wire up an HPA for the rate-quote engine and report truthfully what it can and cannot do on this cluster.

Lab 10 · Step-by-step procedure

Step 1 — Read the workload profiles

```
column -s, -t data/workload-profiles.csv
```

Expected output:

workload	manifest	cpu_request	cpu_limit	
mem_request	mem_limit	expected_qos	note	
berth-ledger	10-pod-qos-guaranteed.yaml	100m	100m	128Mi
128Mi	Guaranteed	requests equal limits for cpu and memory		
rate-quoter	11-pod-qos-burstable.yaml	50m	200m	64Mi
192Mi	Burstable	requests below limits so the container may burst		
notice-board	12-pod-qos-besteffort.yaml	none	none	none
none	BestEffort	no resources declared at all - apply before the LimitRange		
ledger-compactor	20-pod-oomkill-ledger-compactor.yaml	100m	100m	32Mi
32Mi	Guaranteed	memory limit deliberately below the working set - OOMKilled		
quote-engine	40-deployment-quote-engine.yaml	100m	300m	96Mi
256Mi	Burstable	HPA target - 60% utilisation of the 100m request is 60m		
cargo-modeller	60-pod-quota-violation.yaml	900m	1	128Mi
256Mi	Rejected	within the LimitRange max but over the namespace requests.cpu		
quota				

The `expected_qos` column is the contract. `verification/checks.sh` reads this file and compares every value against the live API — so if you edit the manifests, the checks will catch the drift.

Before applying anything, be able to state the rules:

QoS class	Rule
Guaranteed	Every container sets both cpu and memory, and for each, <code>requests == limits</code>
Burstable	At least one container sets a request or limit, but the Pod is not Guaranteed
BestEffort	No container sets any request or any limit

You never write `qosClass`. The API server derives it and writes it into `.status`.

Step 2 — Namespace and the three QoS Pods

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab10 created
```

Apply the three QoS Pods now, before the LimitRange. The order matters and Step 5 explains why.

```
kubectl apply -f manifests/10-pod-qos-guaranteed.yaml
kubectl apply -f manifests/11-pod-qos-burstable.yaml
kubectl apply -f manifests/12-pod-qos-besteffort.yaml
```

Expected output:

```
pod/berth-ledger created
pod/rate-quoter created
pod/notice-board created
kubectl -n kcna-lab10 wait --for=condition=Ready pod --all --timeout=90s
```

Expected output:

```
pod/berth-ledger condition met
pod/notice-board condition met
pod/rate-quoter condition met
```

Now prove the classes:

```
kubectl -n kcna-lab10 get pods \
-o custom-
columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu,MEM_REQ:.spec.containers[0].resources.requests.memory,MEM_LIM:.spec.containers[0].resources.limits.memory
```

Expected output:

POD	QOS	CPU_REQ	CPU_LIM	MEM_REQ	MEM_LIM
berth-ledger	Guaranteed	100m	100m	128Mi	128Mi
notice-board	BestEffort	<none>	<none>	<none>	<none>
rate-quoter	Burstable	50m	200m	64Mi	192Mi

Read each row against the rule table. `berth-ledger` has `100m == 100m` and `128Mi == 128Mi` → **Guaranteed**. `rate-quoter` has requests strictly below limits → **Burstable**. `notice-board` declares nothing → **BestEffort**.

Single-Pod form, which is the one to memorise for the exam:

```
kubectl -n kcna-lab10 get pod berth-ledger -o jsonpath='{.status.qosClass}'; echo
```

Expected output:

```
Guaranteed
```

Confirm you cannot set it yourself — `.status` is not writable through `apply`:

```
grep -c 'qosClass' manifests/10-pod-qos-guaranteed.yaml || echo "qosClass appears nowhere in the manifest"
```

Expected output:

```
0
qosClass appears nowhere in the manifest
```

Why the class matters. Under node memory pressure the kubelet evicts in order: **BestEffort** first, then **Burstable** that exceed their requests, and **Guaranteed** last. `notice-board` is the first thing the

cluster will sacrifice; berth-ledger is the last. That ordering is the whole reason MFS assigned classes per workload.

```
kubectl -n kcna-lab10 describe node "$(kubectl get nodes -o
jsonpath='{.items[0].metadata.name}')" \
| sed -n '/Non-terminated Pods/,/Allocated resources/p' | grep -E 'kcna-lab10|NAMESPACE'
```

Expected output:

NAMESPACE	NAME	CPU Requests	CPU Limits	Memory
Requests	Memory Limits	Age		
kcna-lab10	berth-ledger	100m (1%)	100m (1%)	128Mi
(1%)	128Mi (1%)	3m		
kcna-lab10	notice-board	0 (0%)	0 (0%)	0 (0%)
0 (0%)	3m			
kcna-lab10	rate-quoter	50m (0%)	200m (2%)	64Mi
(0%)	192Mi (2%)	3m		

notice-board books zero. The scheduler treats a BestEffort Pod as free, which is exactly how a node ends up oversubscribed.

Step 3 — Cause an OOMKill

Reproduce the MFS incident, scaled down to 32 MiB:

```
sed -n '/resources:/,/securityContext:/p' manifests/20-pod-oomkill-ledger-compactor.yaml
```

Expected output:

```
resources:
  requests:
    cpu: 100m
    memory: 32Mi
  limits:
    cpu: 100m
    memory: 32Mi
securityContext:
```

The container will try to write a 64 MiB file into a memory-backed `emptyDir`, whose pages are charged to its own cgroup. It has a 32 MiB ceiling.

```
kubectl apply -f manifests/20-pod-oomkill-ledger-compactor.yaml
sleep 15
kubectl -n kcna-lab10 get pod ledger-compactor
```

Expected output:

```
pod/ledger-compactor created
NAME          READY   STATUS    RESTARTS   AGE
ledger-compactor 0/1     OOMKilled 0           15s
```

The evidence, from the container status rather than the summary column:

```
kubectl -n kcna-lab10 get pod ledger-compactor \
-o jsonpath='reason={.status.containerStatuses[0].state.terminated.reason}
exitCode={.status.containerStatuses[0].state.terminated.exitCode}
limit={.spec.containers[0].resources.limits.memory}'; echo
```

Expected output:

```
reason=OOMKilled exitCode=137 limit=32Mi
kubectl -n kcna-lab10 describe pod ledger-compactor | sed -n '/^ *State:/,/^ *Ready:/p'
```

Expected output:

```
State:      Terminated
Reason:     OOMKilled
```

```
Exit Code:      137
Started:        Sat, 05 Sep 2026 10:41:02 +0800
Finished:       Sat, 05 Sep 2026 10:41:04 +0800
Ready:         False
```

```
kubectl -n kcna-lab10 logs ledger-compactor
```

Expected output — the work never finished:

```
compactor: memory limit is 32Mi; attempting a 64 MiB working set
```

Diagnosis.

- **137 = 128 + 9.** The container was killed by signal 9 (`SIGKILL`). That is the kernel OOM killer, not Kubernetes deciding to stop it.
- The `COMPACTION-COMPLETE` line never printed, so the process died mid-dd.
- Its QoS class is still `Guaranteed` — **being Guaranteed does not exempt you from your own limit.** Guaranteed protects you from *other* workloads causing your eviction; nothing protects you from exceeding a ceiling you declared yourself.

```
kubectl -n kcna-lab10 get pod ledger-compactor -o jsonpath='{.status.qosClass} / { .status.phase}'; echo
```

Expected output:

```
Guaranteed / Failed
```

Memory and CPU limits behave completely differently. Memory is incompressible: exceed the limit and you are killed. CPU is compressible: exceed the limit and you are *throttled* — the container runs slower but survives. Never treat the two as interchangeable.

Step 4 — Set namespace guardrails

```
kubectl apply -f manifests/30-limitrange.yaml
kubectl apply -f manifests/31-resourcequota.yaml
```

Expected output:

```
limitrange/mfs-container-limits created
resourcequota/mfs-lab-quota created

kubectl -n kcna-lab10 describe limitrange mfs-container-limits
```

Expected output:

```
Name:          mfs-container-limits
Namespace:     kcna-lab10
Type          Resource  Min    Max    Default Request  Default Limit  Max Limit/Request
Ratio
-----
Container     cpu          10m    1      50m          500m          -
Container     memory       16Mi   512Mi  64Mi         256Mi         -

kubectl -n kcna-lab10 describe resourcequota mfs-lab-quota
```

Expected output:

```
Name:          mfs-lab-quota
Namespace:     kcna-lab10
Resource      Used    Hard
-----
count/deployments.apps  0      5
limits.cpu             300m    3
limits.memory          320Mi   2Gi
pods                   3       15
```

```
requests.cpu          150m    1
requests.memory       192Mi   1Gi
```

ledger-compactor is in a terminal phase (Failed), so the quota controller no longer counts it. notice-board counts as 3 in pods but 0 in every compute resource — a BestEffort Pod consumes quota *slots*, not quota *capacity*.

Step 5 — Watch the LimitRange rewrite a Pod

This is why the ordering in Step 2 mattered. Create a Pod that declares nothing — the same shape as notice-board:

```
kubectl -n kcna-lab10 run late-arrival --image=registry.k8s.io/pause:3.9 --restart=Never sleep 5
kubectl -n kcna-lab10 get pod late-arrival \
  -o custom-
columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu,MEM_REQ:.spec.containers[0].resources.requests.memory,MEM_LIM:.spec.containers[0].resources.limits.memory
```

Expected output:

```
pod/late-arrival created
POD      QOS      CPU_REQ  CPU_LIM  MEM_REQ  MEM_LIM
late-arrival  Burstable  50m      500m     64Mi     256Mi
```

It is **Burstable**, not **BestEffort**, and the manifest never said so. The LimitRange admission plugin injected `defaultRequest` and `default` into the Pod spec before it was persisted. Compare with notice-board, created *before* the LimitRange existed:

```
kubectl -n kcna-lab10 get pods -l qos-demo=besteffort -o
jsonpath='{.items[0].metadata.name}={.items[0].status.qosClass}'; echo
kubectl -n kcna-lab10 get pod late-arrival -o jsonpath='late-arrival={.status.qosClass}';
echo
```

Expected output:

```
notice-board=BestEffort
late-arrival=Burstable
```

Once a LimitRange with defaults exists, BestEffort is impossible in that namespace. For most platform teams that is the point: it makes "declared nothing" unrepresentable. Note also that the LimitRange applied retroactively to *nothing* — notice-board was untouched. Admission plugins only see creates and updates.

Remove the temporary Pod:

```
kubectl -n kcna-lab10 delete pod late-arrival
```

Expected output:

```
pod "late-arrival" deleted
```

Step 6 — Deploy the HPA target

```
kubectl apply -f manifests/40-deployment-quote-engine.yaml
kubectl -n kcna-lab10 rollout status deploy/quote-engine --timeout=120s
```

Expected output:

```
deployment.apps/quote-engine created
Waiting for deployment "quote-engine" rollout to finish: 0 of 2 updated replicas are
available...
deployment "quote-engine" successfully rolled out
```

```
kubectl -n kcna-lab10 get pods -l app=quote-engine \
-o custom-
columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].resources.requ
ests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu
```

Expected output:

POD	QOS	CPU_REQ	CPU_LIM
quote-engine-6b7f9c8d54-kx2mp	Burstable	100m	300m
quote-engine-6b7f9c8d54-w9tqz	Burstable	100m	300m

The quota now reflects the Deployment:

```
kubectl -n kcna-lab10 get resourcequota mfs-lab-quota \
-o jsonpath='requests.cpu used={.status.used.requests\.cpu} of
{.status.hard.requests\.cpu}'; echo
```

Expected output:

```
requests.cpu used=350m of 1
```

100m (berth-ledger) + 50m (rate-quoter) + 0 (notice-board) + 200m (quote-engine ×2) = 350m. Remember that figure — Step 6 of the failure injection uses it.

Step 7 — Create the HPA and read it honestly

```
kubectl apply -f manifests/50-hpa-quote-engine.yaml
```

Expected output:

```
horizontalpodautoscaler.autoscaling/quote-engine created
sleep 20
kubectl -n kcna-lab10 get hpa quote-engine
```

Expected output on a stock kind cluster (this is the honest result — do not expect numbers here):

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
quote-engine	Deployment/quote-engine	cpu: <unknown>/60%, memory: <unknown>/200Mi	2	2
6	20s			

Do not guess why. Ask the object:

```
kubectl -n kcna-lab10 get hpa quote-engine \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}{"\n"}{end}'
```

Expected output:

```
AbleToScale=True reason=SucceededGetScale
ScalingActive=False reason=FailedGetResourceMetric
kubectl -n kcna-lab10 describe hpa quote-engine | sed -n '/^Conditions:/, $p'
```

Real output:

```
Conditions:
  Type           Status  Reason
  ----           -
  AbleToScale    True    SucceededGetScale
the HPA controller was able to get the
target's current scale
  ScalingActive  False   FailedGetResourceMetric
the HPA was unable to compute the
replica count: failed to get cpu utilization: unable to get metrics for resource cpu:
unable to fetch metrics from resource metrics API: the server could not find the requested
resource (get pods.metrics.k8s.io)
Events:
  Type           Reason           Age           From
  Message
  ----           -
  -----
```

```
Warning FailedGetResourceMetric 5s (x3 over 35s) horizontal-pod-autoscaler
failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch
metrics from resource metrics API: the server could not find the requested resource (get
pods.metrics.k8s.io)
Warning FailedComputeMetricsReplicas 5s (x3 over 35s) horizontal-pod-autoscaler the
server could not find the requested resource (get pods.metrics.k8s.io)
```

Read the condition pair, it is a complete diagnosis:

- `AbleToScale=True` — the HPA found the Deployment and *could* scale it. The `scaleTargetRef` is correct.
- `ScalingActive=False, reason=FailedGetResourceMetric` — it has no numbers to scale on. The message names the exact missing API: `pods.metrics.k8s.io`.

That API is served by **metrics-server**, which `kind` does not install. Confirm once more, from the API registry rather than from inference:

```
kubectl get apiservice | grep -c metrics.k8s.io || echo "no metrics.k8s.io APIService is
registered"
kubectl -n kcna-lab10 top pod
```

Expected output:

```
0
no metrics.k8s.io APIService is registered
error: Metrics API not available
```

What the HPA would do if metrics existed. The arithmetic is worth knowing even without a live pipeline:

```
desired = ceil( currentReplicas x ( currentMetricValue / desiredMetricValue ) )
```

With `averageUtilization: 60` against a `100m` request, the target is `60m` of CPU per Pod. If the two Pods averaged `120m`:

```
desired = ceil( 2 x (120 / 60) ) = 4
```

bounded by `minReplicas: 2 / maxReplicas: 6`, and damped by the `behavior` block: at most 2 Pods added per 60 s after a 30 s stabilisation window, and at most 50 % removed per 60 s after a 300 s window. Read those values off the object:

```
kubectl -n kcna-lab10 get hpa quote-engine \
-o jsonpath='{target={.spec.metrics[0].resource.target.averageUtilization}%
min={.spec.minReplicas} max={.spec.maxReplicas}
scaleUpStabilisation={.spec.behavior.scaleUp.stabilizationWindowSeconds}s
scaleDownStabilisation={.spec.behavior.scaleDown.stabilizationWindowSeconds}s'; echo
```

Expected output:

```
target=60% min=2 max=6 scaleUpStabilisation=30s scaleDownStabilisation=300s
```

Notice what this proves about requests: **an HPA on utilization is meaningless without a CPU request**, because the request is the denominator. A BestEffort Pod cannot be autoscaled on utilisation at all.

Optional (instructor-run) — install metrics-server so the HPA reports real numbers

This step writes to the `kube-system` namespace, outside this lab's namespace, and is therefore opt-in. Skip it unless your instructor asks for it; every assertion in `verification/checks.sh` passes either way.

`kind` nodes serve the kubelet API with a self-signed certificate that metrics-server will not trust by default, so the `--kubelet-insecure-tls` argument is mandatory here. It is **not** appropriate for a production cluster.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml
kubectl -n kube-system patch deployment metrics-server --type=json \
  -p ' [{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"} ] '
kubectl -n kube-system rollout status deploy/metrics-server --timeout=180s
```

Expected output:

```
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
...
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
deployment.apps/metrics-server patched
deployment "metrics-server" successfully rolled out
```

Then, after roughly 60 seconds of scrape warm-up:

```
kubectl -n kcna-lab10 top pod
kubectl -n kcna-lab10 get hpa quote-engine
```

Expected output — now with real figures, which will differ on your machine:

NAME	CPU (cores)	MEMORY (bytes)
berth-ledger	1m	4Mi
notice-board	0m	0Mi
quote-engine-6b7f9c8d54-kx2mp	1m	7Mi
quote-engine-6b7f9c8d54-w9tqz	1m	7Mi
rate-quoter	1m	4Mi

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
quote-engine	Deployment/quote-engine	cpu: 1%/60%, memory: 7Mi/200Mi	2	6

The Pods are idle, so utilisation sits far below the 60 % target and the HPA holds at `minReplicas`:

2. To remove metrics-server afterwards, delete exactly what you installed: `kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml`.

Lab 10 · Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script reads `data/workload-profiles.csv` and asserts that each workload's live `.status.qosClass` equals the `expected_qos` column. It also checks the OOMKill evidence, the LimitRange values, the ResourceQuota, and the HPA spec.

For the metrics pipeline it **branches on reality**: it queries the `v1beta1.metrics.k8s.io` APIService, and passes either when metrics-server is absent and the HPA correctly reports `FailedGetResourceMetric`, or when metrics-server is present and the HPA is scaling. It never asserts that `kubectl top` worked.

Run it after section 6 — the `cargo-modeller` rejection check requires that the failure injection has been attempted. Full transcript: `verification/expected-output.md`.

Lab 10 · Failure injection

6a. A Pod that breaches the namespace quota

Finance's ceiling is `requests.cpu: 1` for the whole namespace. The analytics team asks for `900m` on top of the `350m` already committed.

```
kubectl apply -f manifests/60-pod-quota-violation.yaml
```

Real error text:

```
Error from server (Forbidden): error when creating "manifests/60-pod-quota-violation.yaml": pods "cargo-modeller" is forbidden: exceeded quota: mfs-lab-quota, requested: requests.cpu=900m, used: requests.cpu=350m, limited: requests.cpu=1
```

Confirm nothing was created — this is an **admission** rejection, so there is no `Pending` Pod to inspect:

```
kubectl -n kcna-lab10 get pod cargo-modeller
```

Expected output:

```
Error from server (NotFound): pods "cargo-modeller" not found
```

Diagnosis. The message is fully self-describing: `requested + used > limited`, i.e. `900m + 350m > 1000m`. Note what did **not** happen:

- It was not rejected by the `LimitRange` — `900m` is within the per-container `max` of `1`. Confirm the distinction:

```
kubectl -n kcna-lab10 get limitrange mfs-container-limits -o jsonpath='per-container max cpu={.spec.limits[0].max.cpu}'; echo
kubectl -n kcna-lab10 get resourcequota mfs-lab-quota -o jsonpath='namespace-wide requests.cpu={.status.hard.requests\.cpu} used={.status.used.requests\.cpu}'; echo
```

Expected output:

```
per-container max cpu=1
namespace-wide requests.cpu=1 used=350m
```

- It never reached the scheduler. Compare this with Lab 09, where an unschedulable Pod **was** created and sat `Pending`. Admission rejects; scheduling defers. Different failure, different evidence, different fix.

Fix — pick one and justify it to Finance:

```
sed 's/cpu: 900m/cpu: 400m/' manifests/60-pod-quota-violation.yaml | kubectl apply -f -
kubectl -n kcna-lab10 get pod cargo-modeller -o jsonpath='{.metadata.name} qos={.status.qosClass} cpuRequest={.spec.containers[0].resources.requests.cpu}'; echo
```

Expected output:

```
pod/cargo-modeller created
cargo-modeller qos=Burstable cpuRequest=400m
```

The alternative is to raise the quota (`kubectl -n kcna-lab10 patch resourcequota mfs-lab-quota -p '{"spec":{"hard":{"requests.cpu":"2"}}}'`) — but that is a budget decision, not an engineering one, which is precisely why the quota exists.

Restore the rejected state before verifying:

```
kubectl -n kcna-lab10 delete pod cargo-modeller
```

Expected output:

```
pod "cargo-modeller" deleted
```

6b. A Pod that breaches the LimitRange

A different admission plugin, a different message:

```
kubectl -n kcna-lab10 run oversized --image=registry.k8s.io/pause:3.9 --restart=Never \
--overrides='{ "spec": { "containers": [ { "name": "oversized", "image": "registry.k8s.io/
pause:3.9", "resources": { "requests": { "cpu": "50m", "memory": "64Mi", "limits":
{ "cpu": "2", "memory": "128Mi" } } } ] } }'
```

Real error text:

```
Error from server (Forbidden): pods "oversized" is forbidden: maximum cpu usage per
Container is 1, but limit is 2
```

Diagnosis. This one names a **per-container** bound (maximum cpu usage per Container), not a namespace total. LimitRange polices the shape of each container; ResourceQuota polices the sum across the namespace. Both are admission plugins, both reject before persistence, and a Pod must satisfy both.

6c. The HPA that cannot scale

Already produced in Step 7 and worth restating as a failure to recognise on sight:

```
ScalingActive False FailedGetResourceMetric ... unable to fetch metrics from resource
metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)
```

Diagnosis. AbleToScale=True with ScalingActive=False means *the HPA is wired correctly and blind*. The fix is never in the HPA manifest — it is to install metrics-server (or a custom-metrics adapter for type: Pods/type: External metrics). If instead you see ScalingActive=False with reason=InvalidSelector or AbleToScale=False with FailedGetScale, the fault is in the manifest: the scaleTargetRef names an object that does not exist.

Lab 10 · Troubleshooting

Symptom	Cause	Fix
STATUS: OOMKilled, Exit Code: 137	The container's working set exceeded resources.limits.memory; the kernel OOM killer sent SIGKILL	Raise the memory limit to the real working set, or reduce what the process allocates. Memory is incompressible — there is no throttling fallback
Container runs but is inexplicably slow, no restarts	CPU limit throttling. CPU is compressible, so exceeding the limit slows the container instead of killing it	Raise limits.cpu, or remove the CPU limit and rely on the request for scheduling
pods "x" is forbidden: exceeded quota: <name>, requested: ..., used: ..., limited: ...	Namespace ResourceQuota breached in aggregate	Lower the Pod's requests, delete unused Pods, or have the quota raised
pods "x" is forbidden: maximum cpu usage per Container is 1, but limit is 2	Per-container LimitRange max breached	Bring the container within min/max, or change the LimitRange
pods "x" is forbidden: failed quota: <name>: must specify limits.cpu	A ResourceQuota names a compute resource, so every new Pod must declare it	Declare the resource on the container, or add a LimitRange default/defaultRequest so it is injected automatically
A Pod that declared nothing is Burstable, not BestEffort	A LimitRange with default/defaultRequest injected values at admission	Expected behaviour. BestEffort cannot exist in a namespace with LimitRange defaults

Symptom	Cause	Fix
error: Metrics API not available from <code>kubectl top</code>	metrics-server is not installed — the stock kind default	Install metrics-server with <code>--kubelet-insecure-tls</code> (see the optional step), or accept that <code>top</code> and utilisation-based HPAs are unavailable
HPA TARGETS shows <code><unknown>/60%</code>	No metrics source; the HPA cannot compute a ratio	Check <code>ScalingActive</code> — <code>FailedGetResourceMetric</code> means missing metrics-server, not a bad manifest
HPA never scales even with metrics, replicas stuck at <code>minReplicas</code>	Utilisation is genuinely below target, or the <code>scaleUp</code> stabilisation window has not elapsed	<code>kubectl describe hpa</code> and read the events; check <code>.spec.behavior.scaleUp.stabilizationWindowSeconds</code>
HPA <code>AbleToScale=False</code> <code>reason=FailedGetScale</code>	<code>scaleTargetRef</code> points at an object that does not exist or is a kind without a scale subresource	Correct <code>scaleTargetRef.kind/name</code> ; <code>DaemonSets</code> have no scale subresource and cannot be an HPA target

Lab 10 · Cleanup

Delete **only** this lab's namespace. The `LimitRange`, `ResourceQuota`, `HPA`, `Deployment` and all `Pods` are namespaced objects and go with it.

```
kubectl delete namespace kcna-lab10
```

Expected output:

```
namespace "kcna-lab10" deleted
kubectl get ns kcna-lab10
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab10" not found
```

If you took the optional metrics-server step, remove exactly what you installed: `kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml`

Lab 10 · What you learned

- **Requests** are a scheduling reservation and the denominator of HPA utilisation. **Limits** are an enforcement ceiling. They are different fields answering different questions, and a workload profile must set both deliberately.
- **CPU is compressible, memory is not.** Exceed a CPU limit and you are throttled; exceed a memory limit and you are `OOMKilled` with exit code 137 (128 + `SIGKILL`).
- The three **QoS classes** are *derived* by the API server, never declared: `Guaranteed` (every container sets `cpu` and `memory` with `requests == limits`), `Burstable` (something set, not `Guaranteed`), `BestEffort` (nothing set). Read it with `kubectl get pod -o jsonpath='{.status.qosClass}'`.

- QoS drives eviction order under node pressure: BestEffort first, Guaranteed last. It does **not** protect you from your own limit.
- A **LimitRange** shapes each container at admission — `defaultRequest/default` inject values, `min/max` reject outliers. Its side effect is that BestEffort becomes impossible in that namespace.
- A **ResourceQuota** is an aggregate namespace budget, and naming a compute resource in it makes that resource mandatory on every new Pod.
- Both are **admission** controls: a rejected Pod is never created, which is a different failure mode from Lab 09's `Pending` Pod that was created but could not be placed.
- An `autoscaling/v2` HPA needs a metrics source. On a stock `kind` cluster there is none, so the honest reading is `TARGETS: <unknown>` with `AbleToScale=True / ScalingActive=False, reason=FailedGetResourceMetric`. That condition pair — wired correctly, but blind — is the signature to recognise.

Lab 10 · Further reading

- Resource Management for Pods and Containers — <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Pod Quality of Service Classes — <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>
- Assign Memory Resources / troubleshoot OOMKill — <https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/>
- Limit Ranges — <https://kubernetes.io/docs/concepts/policy/limit-range/>
- Resource Quotas — <https://kubernetes.io/docs/concepts/policy/resource-quotas/>
- Horizontal Pod Autoscaling — <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- HPA walkthrough (includes the metrics-server requirement) — <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>
- Resource Metrics Pipeline — <https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>
- Node-pressure Eviction — <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

Day 3 — Services, Networking & Cluster Security

Day 3 Learning Focus

Make Pods reachable and then make them safe. Work through the three address spaces of a cluster, Services and EndpointSlices, cluster DNS, the Service types and Ingress; then authenticate, authorise and isolate with ServiceAccounts, RBAC, Secrets, ConfigMaps and NetworkPolicy.

Field	Value
Instructional hours	8 hours
Class window	9:30 AM – 6:30 PM
Registered topics	3. Services and Networking, 4. Security
KCNA modules	—
Learning outcomes	LO3, LO4
Abilities / Knowledge	A2, A4 / K2, K4, K6
Labs	6 — Lab 11, Lab 12, Lab 13, Lab 14, Lab 15, Lab 16
Hands-on minutes	300 minutes
Deck slides	240–359

Session structure. The table below is the timetable the trainer works to. Breaks are not counted as instructional time.

Time	Duration	Session	Mode
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

Day 3 Concepts

Three address spaces, and one rule

Every Pod gets its own IP address, and every Pod can reach every other Pod without NAT. That is the Kubernetes network model, and a cluster has **three separate address spaces** that people routinely confuse:

- **The node network** — the addresses your machines actually have.
- **The Pod CIDR** — allocated per node by the CNI plugin and carved into per-Pod addresses. Pod IPs are ephemeral: they change on every reschedule.
- **The Service CIDR** — virtual addresses that exist only as rules in the kernel. Nothing ever answers ARP for a ClusterIP; no interface has one.

The **CNI plugin** is what makes the Pod network real. When the kubelet asks the runtime to create a Pod sandbox, the runtime invokes the CNI plugin, which creates a network namespace, allocates an

IP from the node's Pod CIDR, wires a veth pair between the namespace and the node, and installs routes. Plugins differ on many axes, but the one that matters this week is whether the plugin **implements NetworkPolicy**. `kindnet`, the default in `kind`, does not. Calico and Cilium do. A NetworkPolicy applied on a cluster whose CNI ignores policy is stored happily by the API server and enforces nothing — Lab 16 makes you measure exactly that.

A Service stores a selector; the cluster derives everything else

A Service is a stable name and a stable virtual IP in front of a changing set of Pods. You write a **selector**. You do not write the backend list. The EndpointSlice controller watches Pods matching that selector and maintains **EndpointSlice** objects listing the ready backends. `kube-proxy` (or an eBPF datapath) watches those and programs the node's kernel so that traffic to the ClusterIP is DNAT'd to one of the backends.

Endpoints versus EndpointSlice. The original `v1 Endpoints` object put every backend in a single object, which does not scale, and it was **deprecated in Kubernetes v1.33**. Use `kubectl get endpointslices` for diagnostics on any current cluster. `kubectl get endpoints` still works and still appears in older material, so recognise both — but reach for EndpointSlice.

An empty backend list is the single most common Service fault, and it has exactly three causes: the selector does not match any Pod's labels; the Pods match but are not **Ready** (readiness gates endpoint membership, which is the whole point of readiness probes); or the port names and numbers in the Service do not line up with the container. `kubectl get endpointslices -l kubernetes.io/service-name=<svc>` answers all three in one command.

Also learn the port vocabulary precisely, because four different numbers are all called "port": `port` is the Service's own port; `targetPort` is the container port it forwards to and may be a *name* defined in the Pod spec; `nodePort` is the port opened on every node for `type: NodePort`; and `containerPort` in the Pod spec is documentation only.

`kube-proxy` has run in **iptables** mode for years and now increasingly in **nftables** or **IPVS** mode; the mode stopped being an implementation detail once cluster sizes made iptables rule evaluation costs visible. All modes implement the same API; they differ in how the kernel stores and matches the rules.

Cluster DNS

CoreDNS gives every Service a name. The full form is `<service>.<namespace>.svc.cluster.local`. Within the same namespace, `<service>` alone resolves; across namespaces, `<service>.<namespace>` does. A normal Service resolves to its single ClusterIP. A **headless** Service — `clusterIP: None` — has no virtual IP and resolves instead to the **A/AAAA records of every ready Pod**, which is exactly what a client that wants to do its own load balancing, or a StatefulSet client that wants a specific member, needs.

The `ndots:5` setting in every Pod's `/etc/resolv.conf` means any name with fewer than five dots is first tried against each search domain in turn. A lookup of `example.com` therefore issues several failing queries before the correct one. Appending a trailing dot — `example.com.` — makes it absolute and skips the search list.

Service types, and what each one actually requires

- **ClusterIP** (default) — reachable only from inside the cluster. This is the right answer for almost everything.

- **NodePort** — allocates a port in `30000–32767` on **every** node and forwards it to the Service. A port outside that range is rejected by the API server. On `kind`, a NodePort is open on the node *container*, so reaching it from your laptop requires `extraPortMappings`.
- **LoadBalancer** — a NodePort plus a request for an external load balancer. The request is fulfilled by a **cloud controller manager** or a bare-metal implementation such as MetalLB. On a stock `kind` or minikube cluster there is nothing to fulfil it, so `EXTERNAL-IP` stays `<pending>` forever. That is correct behaviour, not a fault.
- **ExternalName** — no proxying at all, just a CNAME to a DNS name outside the cluster.

Ingress: a declaration, and a controller that implements it

An **Ingress** object is HTTP/HTTPS routing rules — host, path, `pathType` and a backend Service. The API server will happily accept it on a cluster with no controller at all, and it will route nothing. `pathType` has three values: `Exact`, `Prefix` (which matches on **path elements**, so `/api` matches `/api/v1` but not `/apifoo`), and `ImplementationSpecific`. `ingressClassName` selects which controller adopts the object.

A note on the ecosystem, current as of September 2026: the **community ingress-nginx project was retired in March 2026**. The `networking.k8s.io/v1` Ingress API itself was *not* retired and remains the stable API this course teaches — but for a runnable cluster today, choose a maintained controller (Traefik, Contour, HAProxy, Envoy Gateway or a commercial NGINX offering). Do not confuse the retired `ingress-nginx` controller project with the NGINX web server image, which is unaffected and is used throughout these labs.

The **Gateway API** is the successor design: role-oriented (`GatewayClass`, `Gateway`, `HTTPRoute`), portable and expressive about traffic splitting, header matching and cross-namespace delegation. Ingress remains stable and widely deployed; Gateway API is where new capability is going. KCNA expects you to know both exist and what distinguishes them.

Security: four layers, and one request path

The "4C" model — **Cloud, Cluster, Container, Code** — is the framing: each layer can only be as secure as the one outside it.

Inside the cluster, **every API request passes through the same three gates in order: authentication** (who are you?), **authorisation** (may you do this?), then **admission control** (should this specific object be allowed, and should it be modified first?). Stop at the first refusal and you know which gate rejected you, which is the whole diagnostic technique.

Kubernetes has no User object. There is nothing to create, list or delete. Human identity comes from outside — a client certificate whose `CN` is the username and whose `O` entries are groups, an OIDC token, or an authenticating proxy. What *does* exist as an API object is the **ServiceAccount**, which is the identity for workloads. Every Pod gets one (`default` if unspecified) and its projected, audience-bound, expiring token is mounted at `/var/run/secrets/kubernetes.io/serviceaccount/`.

RBAC is four object kinds and one rule. A **Role** grants verbs on resources *within one namespace*; a **ClusterRole** grants them cluster-wide or on cluster-scoped resources. A **RoleBinding** grants a Role — or a ClusterRole — to subjects *within one namespace*; a **ClusterRoleBinding** grants a ClusterRole cluster-wide. The useful combination people miss is **ClusterRole + RoleBinding**: define the permission set once, grant it in one namespace. RBAC is **purely additive** — there are no deny

rules, and permissions are the union of everything bound to you. `kubectl auth can-i <verb> <resource> -n <ns>`, optionally with `--as`, is the authoritative way to test it.

ConfigMaps and Secrets

Both are namespaced key/value objects, both cap at roughly 1 MiB, and both can be consumed as environment variables or mounted as volumes. A **Secret** is **base64-encoded, not encrypted** — `kubectl get secret -o yaml | base64 -d` reads it back, and anyone who can read Secrets in a namespace has the credentials. Encryption at rest for Secrets is a separate API-server configuration (`EncryptionConfiguration`) applied to the etcd write path, and it is off unless someone turned it on.

The practical guidance: prefer **mounted files over environment variables** for secrets. Environment variables are a start-time snapshot, they leak into `kubectl describe`, crash dumps, child processes and logs, and they cannot be rotated without a restart. Mounted secrets update in place. Also set `automountServiceAccountToken: false` on Pods that never call the API.

NetworkPolicy

By default, **all Pod-to-Pod traffic in a cluster is allowed**. A NetworkPolicy is *additive allow-listing with an implicit deny side effect*: the moment any policy selects a Pod for a direction (`Ingress` or `Egress`), that Pod becomes default-deny **for that direction**, and only explicitly allowed traffic passes. A policy with `podSelector: {}` and `policyTypes: [Ingress, Egress]` and no rules is the canonical **default-deny** for a namespace.

Peers are selected by `podSelector` (within the policy's namespace), `namespaceSelector` (whole namespaces), the two combined in one list element (a specific Pod set *in* a specific namespace set — note that two separate list elements mean OR, one element with both keys means AND), or `ipBlock` with `cidr` and `except` for addresses outside the cluster. Ports are matched on the **target** of the connection.

Three honest caveats: policies are **namespaced**; they act on **connections, not packets**, so reply traffic on an allowed connection is permitted without a matching reverse rule; and — most importantly — **enforcement is the CNI plugin's job**, so on a plugin that does not implement policy, nothing you write has any effect.

Pod Security admission

The built-in **Pod Security admission** controller applies one of three profiles per namespace via labels: **privileged** (no restrictions), **baseline** (blocks the well-known escapes — host namespaces, privileged containers, most `hostPath`), and **restricted** (baseline plus enforced non-root, seccomp, and dropped capabilities). Each profile can be set in three modes — `enforce`, `audit`, `warn` — so you can measure the impact of a policy before turning it on.

Day 3 Labs

Day 3 has 6 labs, listed below and then set out in full. Work through them in order; each runs in its own namespace and cleans up after itself.

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 11	Services, Endpoints and Cluster DNS	kcna-lab11	50	LO3 / A2 / K2, K4	273

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 12	Service Types: ClusterIP, NodePort and LoadBalancer	kcna-lab12	45	LO3 / A2 / K2, K4	283
Lab 13	Ingress Resources and HTTP Routing	kcna-lab13	50	LO3 / A2 / K2, K4	305
Lab 14	Namespaces, ServiceAccounts and RBAC	kcna-lab14	55	LO4 / A4 / K6	332
Lab 15	Secrets, ConfigMaps and Safe Injection	kcna-lab15	50	LO4 / A4 / K6	341
Lab 16	NetworkPolicy and Default-Deny Isolation	kcna-lab16	50	LO4 / A4 / K6	350

Lab 11 — Services, Endpoints and Cluster DNS

Field	Value
Lab ID	Lab 11
Title	Services, Endpoints and Cluster DNS
Day / Topic	Day 3 · Services and Networking
Duration	50 minutes
Namespace	kcna-lab11
Learning outcome	LO3 — Develop a solution architecture within Kubernetes.
Ability	A2 Develop a solution architecture utilising appropriate tools, techniques and models of system components and interfaces
Knowledge	K2 Components of solution architecture · K4 Steps for developing solution architecture
Deck slide	Slide 273
Repository path	courseware/labs/lab-11-services-endpoints/

Goal. Give the Meridian Freight quote-api a stable cluster identity by putting a ClusterIP Service in front of it, then prove what that name actually points at by reading the EndpointSlice the selector produced - and break it with a one-character selector typo.

What you will produce:

- A quote-api Deployment (3 replicas) serving data/quote-api-index.html from a ConfigMap on named container port http/8080
- A ClusterIP Service mapping port 80 to the named targetPort http, with an EndpointSlice holding exactly the three Ready Pod IPs
- A dns-client Pod that resolves every name form in data/dns-fixtures.csv, including the cross-namespace case that correctly fails

Lab 11 · Objective

By the end of this lab you will be able to:

1. Explain a Service as a **selector plus a port map**, and the EndpointSlice as the *product* that the endpoints controller derives from that selector.
2. Distinguish `port`, `targetPort` and a **named** container port, and change the Pod listener without touching the Service.
3. Resolve a Service through cluster DNS in all four name forms (`<svc>`, `<svc>.<ns>`, `<svc>.<ns>.svc`, `<svc>.<ns>.svc.cluster.local`) and explain the `search` list in the Pod's `/etc/resolv.conf` that makes the short forms work.
4. Read `kubectl get endpointslices` as the primary Service-backend diagnostic, and recognise `v1 Endpoints` as the deprecated predecessor.
5. Diagnose a Service that has **zero endpoints** because of a selector typo, and name the exact symptom the client sees.

Lab 11 · Prerequisites

- A running single-node `kind` cluster and a `kubectl` whose minor version is within one of the cluster (`kubectl` version).
- Labs 01–10 completed (Pods, labels, selectors, Deployments).
- Your shell's working directory is the **lab folder**:

```
cd courseware/labs/lab-11-services-endpoints
pwd
```

Expected output (your absolute prefix will differ):

```
/.../TGS-2023039343-Kubernetes and Cloud Native Associate (KCNA)
Training/courseware/labs/lab-11-services-endpoints
```

- Confirm the cluster answers and CoreDNS is running — every DNS step below depends on it:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Expected output (names and ages will differ):

NAME	READY	STATUS	RESTARTS	AGE
coredns-668d6bf9bc-4nzt8	1/1	Running	0	41m
coredns-668d6bf9bc-r7wqk	1/1	Running	0	41m

If this returns `No resources found`, stop. Without CoreDNS, steps 6–8 cannot work and no amount of Service debugging will help.

Lab 11 · Scenario

Meridian Freight Pte Ltd runs a container-shipping booking platform. The `quote-api` service prices a shipment for a given lane and container type. It is being moved off a fixed VM address onto Kubernetes.

The VM version listened on `:8080` and other teams hard-coded `http://10.20.4.11:8080` into their configuration. Every time the VM was rebuilt the IP changed and three downstream teams broke. Your job as platform engineer is to give `quote-api` a **stable name** — `quote-api.kcna-lab11.svc.cluster.local` — that keeps working while the Pods behind it are replaced, scaled and rescheduled, and to be able to prove *which* Pods a caller is actually reaching.

Lab 11 · Step-by-step procedure

Step 1 — Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab11 created
```

Confirm it exists and is `Active`:

```
kubectl get namespace kcna-lab11
```

Expected output:

NAME	STATUS	ACTIVE	AGE
kcna-lab11	Active	5s	

Note: on some releases the column header set is `NAME STATUS AGE`. The value that matters is `Active`.

Step 2 — Understand where the Pod content comes from

The `quote-api` Pods serve a static page held in a ConfigMap. That ConfigMap is the declarative form of the file in `data/`. Look at the source file first:

```
cat data/quote-api-index.html
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>
<p>service: quote-api</p>
<p>namespace: kcna-lab11</p>
<p>listener: container port 8080 (named "http")</p>
<p>build: 2026.09.05-lab11</p>
</body>
</html>
```

`manifests/10-configmap-quote-content.yaml` is exactly what this command produces — generate it yourself to prove it, without applying anything:

```
kubectl create configmap quote-api-content \
  --from-file=index.html=data/quote-api-index.html \
  -n kcna-lab11 --dry-run=client -o yaml | head -8
```

Expected output:

```
apiVersion: v1
data:
  index.html: |
    <!DOCTYPE html>
    <html lang="en">
    <head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
    <body>
    <h1>Meridian Freight Pte Ltd</h1>
```

Now apply the checked-in version (it also carries the nginx server block that moves the listener to port 8080):

```
kubectl apply -f manifests/10-configmap-quote-content.yaml
```

Expected output:

```
configmap/quote-api-content created
configmap/quote-api-nginx created
```

Step 3 — Deploy the workload

```
kubectl apply -f manifests/20-deployment-quote-api.yaml
```

Expected output:

```
deployment.apps/quote-api created
```

Wait for all three replicas to become **Ready** — readiness, not merely Running, is what puts a Pod into the EndpointSlice:

```
kubectl rollout status deployment/quote-api -n kcna-lab11 --timeout=120s
```

Expected output:

```
Waiting for deployment "quote-api" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "quote-api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "quote-api" rollout to finish: 2 of 3 updated replicas are available...
deployment "quote-api" successfully rolled out
```

List the Pods with their IPs — these are the addresses the Service will collect:

```
kubectl get pods -n kcna-lab11 -l app=quote-api -o wide
```

Expected output (Pod name suffixes and IPs are allocated at runtime and **will** differ on your cluster):

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS GATES					
quote-api-6c9b4f7d55-7lqm2	1/1	Running	0	38s	10.244.0.14	kind-control-plane <none>
quote-api-6c9b4f7d55-jd8vx	1/1	Running	0	38s	10.244.0.15	kind-control-plane <none>
quote-api-6c9b4f7d55-x2t9k	1/1	Running	0	38s	10.244.0.16	kind-control-plane <none>

Record these three Pod IPs. Step 5 shows the identical set appearing inside the EndpointSlice — that is the whole point of this lab.

Step 4 — Create the Service and read its three port fields

```
kubectl apply -f manifests/30-service-quote-api.yaml
```

Expected output:

```
service/quote-api created
kubectl get service quote-api -n kcna-lab11
```

Expected output (the CLUSTER-IP is allocated from the cluster's service CIDR and will differ):

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
quote-api	ClusterIP	10.96.114.23	<none>	80/TCP	6s

Read the port fields deliberately:

Field	Value here	Meaning
spec.ports[0].port	80	The port the Service listens on. This is what clients dial: <code>quote-api:80</code> .
spec.ports[0].targetPort	http	The port on the Pod . Because it is a <i>name</i> , kubelet resolves it against the container's <code>ports[].name</code> .
container ports[0].containerPort	8080	The port the process actually binds.
spec.ports[0].name	http	The Service port's own name. Required once a Service has more than one port; also what an Ingress <code>backend.service.port.name</code> refers to.

Confirm the name resolution really happened by asking the API server for the resolved target:

```
kubectl get service quote-api -n kcna-lab11 -o jsonpath='{.spec.ports[0].targetPort}'
```

Expected output:

```
http
```

The Service object keeps the *name*; the resolution to 8080 happens per-Pod, in the EndpointSlice. You will see the number in the next step.

Step 5 — The EndpointSlice: what the selector produced

`kubectl get endpointslices` is the current, supported way to inspect Service backends. (v1 Endpoints was deprecated in Kubernetes v1.33; it is still served for compatibility — see Step 5b.)

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api
```

Expected output (the slice name gets a random suffix):

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-t8hkb	IPv4	8080	10.244.0.14,10.244.0.15,10.244.0.16	45s

Two things to notice:

- PORTS is **8080**, not 80. The named `targetPort: http` was resolved to the real container port when the slice was written.
- ENDPOINTS is exactly the three Pod IPs from Step 3.

Print the same information in a form you can compare against Step 3:

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api \
-o jsonpath='{range .items[*].endpoints[*]}{.addresses[0]}{"\t"}{.conditions.ready}{"\t"}{.targetRef.name}{"\n"}{end}'
```

Expected output:

10.244.0.14	true	quote-api-6c9b4f7d55-7lqm2
10.244.0.15	true	quote-api-6c9b4f7d55-jd8vx
10.244.0.16	true	quote-api-6c9b4f7d55-x2t9k

`conditions.ready` is the readiness probe's verdict. A Pod that is `Running` but not `Ready` still appears in the slice with `ready: false`, and kube-proxy will **not** send it traffic. This is why "the Pod is running but the Service is broken" is such a common incident.

Step 5b — The deprecated Endpoints view

```
kubectl get endpoints quote-api -n kcna-lab11
```

Expected output:

NAME	ENDPOINTS	AGE
quote-api	10.244.0.14:8080,10.244.0.15:8080,10.244.0.16:8080	70s

Same data, older shape. The v1 Endpoints API was deprecated in Kubernetes v1.33 because a single object could not scale past a few thousand addresses and every update rewrote the whole list. EndpointSlice shards them (default 100 endpoints per slice) and adds topology and per-endpoint conditions. **For the KCNA exam and for real diagnostics, reach for endpointslices.**

Step 6 — Start a client Pod and read its resolver configuration

```
kubectl apply -f manifests/40-pod-dns-client.yaml
```

Expected output:

```
pod/dns-client created
kubectl wait --for=condition=Ready pod/dns-client -n kcna-lab11 --timeout=60s
```

Expected output:

```
pod/dns-client condition met
```

Before resolving anything, look at *why* short names work:

```
kubectl exec -n kcna-lab11 dns-client -- cat /etc/resolv.conf
```

Expected output:

```
search kcna-lab11.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

Read it line by line:

- `nameserver 10.96.0.10` — the **ClusterIP of the kube-dns Service** in `kube-system` (the Service is named `kube-dns` even though CoreDNS serves it). kubelet injects this into every Pod with `dnsPolicy: ClusterFirst`, the default.
- `search ...` — appended in order to any name with fewer than `ndots` dots. This is what turns `quote-api` into `quote-api.kcna-lab11.svc.cluster.local` on the first try.
- `options ndots:5` — a name containing fewer than 5 dots is treated as *relative* and pushed through the search list first. `quote-api.kcna-lab11.svc.cluster.local` has 4 dots, so even the "FQDN" is walked through the search list before being tried absolute — the classic source of extra DNS lookups. Add a trailing dot to force an absolute query.

Step 7 — Resolve the Service in all four name forms

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api.kcna-lab11.svc.cluster.local
```

Expected output:

```
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   quote-api.kcna-lab11.svc.cluster.local
Address: 10.96.114.23
```

Honest note on BusyBox `nslookup`: BusyBox also issues an AAAA (IPv6) query. On an IPv4-only kind cluster you will often see an extra `*** Can't find quote-api.kcna-lab11.svc.cluster.local: No answer` line before or after the answer above, and the command may exit non-zero. That is **not** a failure of your Service — it is the missing IPv6 record. Judge the result by the `Address:` line.

Now the shorter forms, from a Pod in the same namespace:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api
```

Expected output (same ClusterIP, reached via the first search-path entry):

```
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   quote-api.kcna-lab11.svc.cluster.local
Address: 10.96.114.23
```

Note that the *answer* is the FQDN even though you asked for the short name — proof the search path was applied.

Now the two **partial** forms. Use `ping` rather than `nslookup` here, and read only the resolution shown in `ping`'s first line:

```
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11
```

Expected output (the ClusterIP will differ; the packet loss is expected and irrelevant):

```
PING quote-api.kcna-lab11 (10.96.114.23): 56 data bytes

--- quote-api.kcna-lab11 ping statistics ---
1 packets transmitted, 0 packets received, 100% packet loss
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11.svc
```

Expected output:

```
PING quote-api.kcna-lab11.svc (10.96.114.23): 56 data bytes

--- quote-api.kcna-lab11.svc ping statistics ---
1 packets transmitted, 0 packets received, 100% packet loss
```

Two honest points here.

1. **0 packets received is correct.** A ClusterIP is a kube-proxy DNAT rule, not a host — it is not obliged to answer ICMP. The line that matters is the resolved address in parentheses. Never use "ping fails" as evidence that a Service is down.
2. **`nslookup` is the wrong tool for these two names.** BusyBox `nslookup` sends the name it was given without walking `/etc/resolv.conf`'s search list once the name already contains a dot, so `nslookup quote-api.kcna-lab11` returns `*** Can't find quote-api.kcna-lab11: No answer` on a cluster where the name resolves perfectly well for real clients. `ping`, `wget` and any application using the standard resolver **do** apply the search list. `verification/checks.sh` therefore uses `ping`'s resolution line, not `nslookup`, to test the fixtures.

Confirm point 2 for yourself with the tool that a real caller would use:

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api.kcna-lab11/ | head -1
```

Expected output:

```
<!DOCTYPE html>
```

Work through the fixture list in `data/dns-fixtures.csv`, which records every form and whether it should resolve:

```
cat data/dns-fixtures.csv
```

Expected output:

```
name,kind,resolves,note
quote-api,short,yes,resolved via the search path of the client Pod in the same namespace
quote-api.kcna-lab11,ns-qualified,yes,second search-path entry svc.cluster.local is appended
quote-api.kcna-lab11.svc,partial-fqdn,yes,cluster.local is appended from the search path
quote-api.kcna-lab11.svc.cluster.local,fqdn,yes,fully qualified - no search path needed
quote-api.default.svc.cluster.local,wrong-namespace,no,the Service does not exist in the default namespace
kubernetes.default.svc.cluster.local,control-plane,yes,the always-present API server Service
```

Prove the negative case — the same Service name in the wrong namespace:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api.default.svc.cluster.local
```


Expected output (non-zero exit code is correct here):

```
Server:      10.96.0.10
Address:     10.96.0.10:53

*** Can't find quote-api.default.svc.cluster.local: No answer

command terminated with exit code 1
```

The same negative with `ping`, which is what `checks.sh` uses:

```
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-
api.default.svc.cluster.local
```

Expected output:

```
ping: bad address 'quote-api.default.svc.cluster.local'
command terminated with exit code 1
```

`bad address` means **resolution itself failed** — there is no line with a resolved IP in parentheses. Contrast that with the partial names above, which resolved and then merely dropped ICMP.

A Service name is namespaced. Cross-namespace callers must use at least `<svc>.<ns>`; the bare name only works inside the owning namespace.

Step 8 — Actually use the Service

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- http://quote-api/
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>
<p>service: quote-api</p>
<p>namespace: kcna-lab11</p>
<p>listener: container port 8080 (named "http")</p>
<p>build: 2026.09.05-lab11</p>
</body>
</html>
```

The client dialled port **80** (the Service port, the default for `http://`) and was delivered to port **8080** in a Pod. That translation is the Service's job.

Now watch the endpoint set follow the Pod set. Scale down:

```
kubectl scale deployment/quote-api -n kcna-lab11 --replicas=1
```

Expected output:

```
deployment.apps/quote-api scaled
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api
```

Expected output (one address remains; the ClusterIP has not changed):

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-t8hkb	IPv4	8080	10.244.0.14	4m12s

Scale back up before continuing:

```
kubectl scale deployment/quote-api -n kcna-lab11 --replicas=3
kubectl rollout status deployment/quote-api -n kcna-lab11 --timeout=120s
```

Expected output:

```
deployment.apps/quote-api scaled
deployment "quote-api" successfully rolled out
```

The stable thing is the name and the ClusterIP. Everything behind it is disposable.

Lab 11 · Verification

Run the automated checks:

```
bash verification/checks.sh
```

Expected output:

```
== Lab 11 verification: Services, Endpoints and Cluster DNS ==
PASS namespace kcna-lab11 exists
PASS ConfigMap quote-api-content matches data/quote-api-index.html byte-for-byte
PASS deployment quote-api has 3/3 ready replicas
PASS service quote-api is ClusterIP with port 80
PASS service quote-api targetPort is the named port "http"
PASS EndpointSlice resolved the named port to 8080
PASS EndpointSlice holds 3 ready addresses
PASS EndpointSlice addresses match the quote-api Pod IPs exactly
PASS dns fixture: quote-api resolves
PASS dns fixture: quote-api.kcna-lab11 resolves
PASS dns fixture: quote-api.kcna-lab11.svc resolves
PASS dns fixture: quote-api.kcna-lab11.svc.cluster.local resolves
PASS dns fixture: quote-api.default.svc.cluster.local correctly does NOT resolve
PASS dns fixture: kubernetes.default.svc.cluster.local resolves
PASS HTTP GET through the Service returns the Meridian Freight page
-- 15 passed, 0 failed --
```

See [verification/expected-output.md](#) for the full annotated reference output.

Lab 11 · Failure injection — the selector typo

This is the single most common Service fault in production. You will create it deliberately.

6.1 Break it

```
kubectl apply -f manifests/50-service-broken-selector.yaml
```

Expected output:

```
service/quote-api-broken created
```

The API server accepted it without complaint. **There is no validation that a Service selector matches anything** — a Service with zero backends is a legal object.

6.2 Observe the symptom the caller sees

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api-broken/
```

Expected output (the ClusterIP will differ):

```
wget: can't connect to remote host (10.96.203.77): Connection refused
command terminated with exit code 1
```

Note carefully: **DNS worked**. The name resolved to a ClusterIP. Confirm it:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api-broken
```

Expected output:

```
Server:      10.96.0.10
Address:     10.96.0.10:53
```

```
Name:      quote-api-broken.kcna-lab11.svc.cluster.local
Address:   10.96.203.77
```

This is the diagnostic fork every engineer must be able to take:

- **Name does not resolve** → DNS / CoreDNS / wrong namespace / wrong Service name.
- **Name resolves but the connection is refused** → the Service exists but has **no endpoints**. In iptables mode kube-proxy installs a REJECT rule for a ClusterIP with an empty endpoint set, which is why you get an immediate *Connection refused* rather than a timeout.

6.3 Diagnose

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api-broken
```

Expected output:

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-broken-4zq6n	IPv4	<unset>	<unset>	40s

An empty slice. Now the classic confirmation:

```
kubectl describe service quote-api-broken -n kcna-lab11
```

Expected output (abridged; look at the Endpoints line):

```
Name:      quote-api-broken
Namespace:  kcna-lab11
Labels:     app=quote-api
            kcna.tertiaryinfotech.com/intent=failure-injection
Selector:   app=quote-apis
Type:       ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP:         10.96.203.77
IPs:        10.96.203.77
Port:       http 80/TCP
TargetPort: http/TCP
Endpoints:
Session Affinity:   None
Events:             <none>
```

Endpoints: is empty and Selector: app=quote-apis is the culprit.

6.4 Prove the root cause

Ask the cluster what the selector actually matches. This is the decisive step — run the Service's own selector as a label query:

```
kubectl get pods -n kcna-lab11 -l app=quote-apis
```

Expected output:

```
No resources found in kcna-lab11 namespace.
```

Compare with the correct label:

```
kubectl get pods -n kcna-lab11 -l app=quote-api --no-headers | wc -l
```

Expected output:

```
3
```

Diagnosis: the Service selector `app=quote-apis` matches zero Pods. The Pods carry `app=quote-api`. The endpoints controller has nothing to put in the EndpointSlice, so kube-proxy has no destination to DNAT to and rejects the connection at the ClusterIP.

6.5 Fix and re-verify

Patch the selector to the correct label:

```
kubectl patch service quote-api-broken -n kcna-lab11 \
  --type=merge -p '{"spec":{"selector":{"app":"quote-api"}}}'
```

Expected output:

```
service/quote-api-broken patched
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api-broken
```

Expected output:

```
NAME                                ADDRESSTYPE  PORTS  ENDPOINTS                                AGE
quote-api-broken-4zq6n             IPv4         8080   10.244.0.14,10.244.0.15,10.244.0.16    2m5s
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api-broken/ |
head -1
```

Expected output:

```
<!DOCTYPE html>
```

The endpoint set repopulated within a second of the patch, with no Pod restart. The Service was never "down" — it was simply pointing at nothing.

Lab 11 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
wget: can't connect to remote host (10.96.x.y): Connection refused and the name resolves	Service has zero endpoints : selector matches no Pod, or no Pod is Ready	<code>kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=<svc></code> shows no addresses	Correct <code>spec.selector</code> to match the Pod template labels, or fix the readiness probe
nslookup: can't resolve 'quote-api' from a Pod in another namespace	Short names are namespaced; the search path of the <i>client</i> Pod is used	<code>kubectl exec <pod> -- cat /etc/resolv.conf</code> — first search entry is the client's namespace	Use <code>quote-api.kcna-lab11</code> or the full FQDN
nslookup <code>quote-api.kcna-lab11</code> says No answer, but <code>wget http://quote-api.kcna-lab11/</code> works	BusyBox <code>nslookup</code> does not walk the search list for a name that already contains a dot	<code>kubectl exec dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11</code> shows the resolved IP in its header	Diagnose with <code>ping/wget</code> , or query the full FQDN with <code>nslookup</code> . Nothing to fix on the cluster
ping to a ClusterIP shows 100% packet loss	A ClusterIP is a kube-proxy DNAT rule, not a host; it need not answer ICMP	The <code>PING <name> (<ip>)</code> header still shows a resolved address	Test with TCP (<code>wget</code>), never with ICMP
Connection times out instead of being refused	Not an empty-endpoint problem — a NetworkPolicy, a wrong port, or the app not listening on <code>containerPort</code>	<code>kubectl get netpol -n kcna-lab11</code> ; <code>kubectl exec <backend-pod> -- netstat -ltn</code>	Align <code>containerPort</code> with the real listener; see Lab 16 for policy

Symptom	Likely cause	Command that confirms it	Fix
EndpointSlice lists an address with <code>ready: false</code>	Pod is Running but failing its readiness probe	<code>kubectl describe pod <pod> -n kcna-lab11 → Readiness probe failed: events</code>	Fix the probe path/port or the app's health handler
Error: couldn't find port "http" on Pod start, or <code>TargetPort: http</code> never resolves	The named <code>targetPort</code> has no matching <code>ports[].name</code> in the container spec	<code>kubectl get deploy quote-api -n kcna-lab11 -o jsonpath='{.spec.template.spec.containers[0].ports}'</code>	Add name: <code>http</code> to the container port, or use the numeric <code>targetPort: 8080</code>
<code>kubectl get endpoints</code> prints a deprecation notice or is missing on a future cluster	<code>v1 Endpoints</code> is deprecated as of Kubernetes v1.33	<code>kubectl api-resources grep -E 'endpoint'</code>	Use <code>kubectl get endpointslices</code>

Lab 11 · Cleanup

Delete **only** this lab's namespace. Every object you created lives inside it.

```
kubectl delete namespace kcna-lab11
```

Expected output:

```
namespace "kcna-lab11" deleted
```

Confirm:

```
kubectl get namespace kcna-lab11
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab11" not found
```

Do **not** run an unscoped `kubectl delete svc --all` — without `-n kcna-lab11` that targets your current namespace and can remove the `kubernetes` Service.

Lab 11 · What you learned

- A **Service** is two things: a stable virtual IP + DNS name, and a *label selector*. It holds no list of Pods.
- The **endpoints controller** watches Pods, evaluates the selector, and writes an **EndpointSlice**. That slice — not the Service — is the answer to "what is actually behind this name right now".
- Only **Ready** Pods are given `conditions.ready: true` and receive traffic.
- `port` (Service) → `targetPort` (Pod) is a translation, and using a **named** port decouples the Service from the container's port number.
- Cluster DNS gives `<svc>.<ns>.svc.cluster.local`; the Pod's `search` path and `ndots:5` make the shorter forms work **within** the client's namespace only.
- The signature of an empty Service is: **DNS resolves, TCP is refused**. Always run the Service's selector as a `kubectl get pods -l query`.

- `v1 Endpoints` is deprecated (Kubernetes v1.33); `discovery.k8s.io/v1 EndpointSlice` is the current API.

Lab 11 · Further reading

- Service — <https://kubernetes.io/docs/concepts/services-networking/service/>
- EndpointSlices — <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- DNS for Services and Pods — <https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>
- Debug Services — <https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>
- Endpoints API deprecation — <https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>
- Virtual IPs and Service proxies — <https://kubernetes.io/docs/reference/networking/virtual-ips/>

Lab 12 — Service Types: ClusterIP, NodePort and LoadBalancer

Field	Value
Lab ID	Lab 12
Title	Service Types: ClusterIP, NodePort and LoadBalancer
Day / Topic	Day 3 · Services and Networking
Duration	45 minutes
Namespace	kcna-lab12
Learning outcome	LO3 — Develop a solution architecture within Kubernetes.
Ability	A2 Develop a solution architecture utilising appropriate tools, techniques and models of system components and interfaces
Knowledge	K2 Components of solution architecture · K4 Steps for developing solution architecture
Deck slide	Slide 283
Repository path	courseware/labs/lab-12-service-types-nodeport-lb/

Goal. Expose one Meridian Freight backend through all four Service shapes at once - ClusterIP, NodePort, LoadBalancer and headless - and record honestly which ones a stock kind cluster can actually deliver.

What you will produce:

- Four Services over a single booking-edge Deployment, proven by `kubectl get svc -o wide` to share one selector and one endpoint set
- A pinned NodePort on 30080 reached on the node InternalIP from inside the cluster, plus the kind `extraPortMappings` config in `data/` that would expose it to the host
- A LoadBalancer Service documented and asserted as permanently `<pending>`, with `kubectl describe` showing Events: `<none>` as the proof no controller is watching

Lab 12 · Objective

By the end of this lab you will be able to:

1. Create and compare all four Service shapes — **ClusterIP**, **NodePort**, **LoadBalancer** and **headless** (`clusterIP: None`) — against a single backend.
2. Explain the **superset relationship**: NodePort *contains* a ClusterIP; LoadBalancer *contains* a NodePort.
3. State the default `nodePort` range (**30000–32767**) and the API-server flag that sets it, and read the exact rejection message when you go outside it.
4. Explain **why type: LoadBalancer stays <pending> forever on kind** and name what would have to be installed to change that.
5. Explain why a NodePort is reachable from *inside* the cluster immediately but needs kind `extraPortMappings` to be reachable from your laptop.
6. Show that a headless Service returns **Pod IPs** from DNS instead of a virtual IP.

Lab 12 · Prerequisites

- Lab 11 completed — you must already be comfortable with `port` vs `targetPort` and with reading `EndpointSlices`.
- A running single-node `kind` cluster.
- Working directory:

```
cd courseware/labs/lab-12-service-types-nodeport-lb
pwd
```

Expected output (your absolute prefix will differ):

```
/.../courseware/labs/lab-12-service-types-nodeport-lb
```

- Confirm you are on a **kind** cluster, because half this lab is about kind's limitations:

```
kubectl get nodes -o wide
```

Expected output (one node, name starts with your kind cluster name):

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
OS-IMAGE		KERNEL-VERSION		CONTAINER-RUNTIME		
kind-control-plane	Ready	control-plane	52m	v1.31.0	172.18.0.2	<none>
Debian GNU/Linux 12 (bookworm)		6.10.14-linuxkit		containerd://1.7.18		

The `INTERNAL-IP` (172.18.0.2 here) is the address of the kind **node container** on the Docker bridge network. Write yours down — Step 4 uses it.

Lab 12 · Scenario

Meridian Freight's `booking-edge` service is the public entry point that partner freight forwarders call to lodge a booking. Three environments need three different exposure models and the platform team keeps arguing about which Service type to use:

- **In-cluster callers** (the `quote-api` from Lab 11) just need a name → ClusterIP.
- **The QA team** wants to hit it from a laptop against the local kind cluster → NodePort.
- **Production on a managed cloud** is supposed to get a real external address → LoadBalancer.
- **A future sharded pricing engine** needs to address individual Pods directly → headless.

You will build all four against the same two Pods and record honestly which ones actually work on the training cluster and which ones cannot.

Lab 12 · Step-by-step procedure

Step 1 — Namespace and backend

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab12 created
```

The backend is `hashicorp/http-echo:1.0`, which returns one fixed line so you can always tell it answered. That line is held in `data/edge-banner.txt`:

```
cat data/edge-banner.txt
```


Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

The Deployment passes exactly this string as `-text=`:

```
kubectl apply -f manifests/10-deployment-booking-edge.yaml
kubectl rollout status deployment/booking-edge -n kcna-lab12 --timeout=120s
```

Expected output:

```
deployment.apps/booking-edge created
Waiting for deployment "booking-edge" rollout to finish: 0 of 2 updated replicas are
available...
deployment "booking-edge" successfully rolled out
kubectl get pods -n kcna-lab12 -o wide
```

Expected output (names and IPs will differ):

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS GATES					
booking-edge-79c4b5f9d8-h4kpz	1/1	Running	0	25s	10.244.0.31	kind-
control-plane	<none>	<none>				
booking-edge-79c4b5f9d8-tzq7v	1/1	Running	0	25s	10.244.0.32	kind-
control-plane	<none>	<none>				

Start the diagnostic client now — every reachability test in this lab is run from it:

```
kubectl apply -f manifests/60-pod-net-client.yaml
kubectl wait --for=condition=Ready pod/net-client -n kcna-lab12 --timeout=60s
```

Expected output:

```
pod/net-client created
pod/net-client condition met
```

Step 2 — ClusterIP: the base case

```
kubectl apply -f manifests/20-service-clusterip.yaml
```

Expected output:

```
service/booking-edge-clusterip created
kubectl get service booking-edge-clusterip -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-clusterip	ClusterIP	10.96.51.7	<none>	80/TCP	4s

Use it from inside the cluster:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 http://booking-edge-
clusterip/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Now try it the way a beginner tries it — from your laptop. **This is expected to fail**; run it so you have seen the failure:

```
kubectl get service booking-edge-clusterip -n kcna-lab12 -o jsonpath='{.spec.clusterIP}
{"\n"}'
```

Expected output:

```
10.96.51.7
```

```
curl --max-time 5 http://10.96.51.7/ ; echo "exit=$?"
```

Expected output:

```
curl: (28) Connection timed out after 5001 milliseconds
exit=28
```

10.96.0.0/12 is a **virtual** range that exists only as kube-proxy rules inside the cluster's network namespaces. Your laptop has no route to it. The supported way in is a port-forward:

```
kubectl port-forward -n kcna-lab12 service/booking-edge-clusterip 18080:80 >/tmp/pf-
lab12.log 2>&1 &
sleep 2
curl -s http://127.0.0.1:18080/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Stop the port-forward before continuing:

```
kill %1 2>/dev/null; sleep 1; echo "port-forward stopped"
```

Expected output:

```
port-forward stopped
```

Step 3 — NodePort: ClusterIP plus a port on every node

```
kubectl apply -f manifests/30-service-nodeport.yaml
```

Expected output:

```
service/booking-edge-nodeport created
kubectl get service booking-edge-nodeport -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-nodeport	NodePort	10.96.77.140	<none>	80:30080/TCP	5s

Read `PORT(S)` carefully: `80:30080/TCP` means *ClusterIP port 80 and node port 30080*. A NodePort Service still has a fully working ClusterIP — prove it:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 http://booking-edge-
nodeport/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Step 4 — Reach the NodePort on the node's own IP

Capture the node's internal IP into a shell variable:

```
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?
(@.type=="InternalIP")].address}')
echo "NODE_IP=$NODE_IP"
```

Expected output (yours will differ):

```
NODE_IP=172.18.0.2
```

Dial the node IP on port 30080 **from inside the cluster**. This works on stock kind — no special cluster configuration required:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 "http://${NODE_IP}:30080/"
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Now the honest part. Try the same thing from your laptop:

```
curl --max-time 5 "http://${NODE_IP}:30080/" ; echo "exit=$?"
```

Expected output on **macOS or Windows** (Docker Desktop runs the Docker network inside a VM, so 172.18.0.0/16 is not routable from the host):

```
curl: (28) Connection timed out after 5002 milliseconds
exit=28
```

On a **Linux** host with Docker running natively, the same `curl` usually **succeeds** and prints the banner, because the Docker bridge network is reachable from the host. Both results are correct for their platform — record which one you got.

The portable answer, on every platform, is to have created the kind cluster with an `extraPortMappings` entry. Look at the configuration that does it:

```
cat data/kind-cluster-nodeport.yaml
```

Expected output:

```
# kind cluster configuration used by README Appendix A.
...
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080    # the Service nodePort inside the node
    hostPort: 30080        # the port on your laptop
    listenAddress: "127.0.0.1"
    protocol: TCP
```

`extraPortMappings` can only be set when the cluster is created. You cannot add it to a running kind cluster; you would have to create a new one (Appendix A). For today, `kubectl port-forward` is the correct tool and it works against any Service type.

Step 5 — LoadBalancer: the one that will not complete

```
kubectl apply -f manifests/40-service-loadbalancer.yaml
```

Expected output:

```
service/booking-edge-lb created
```

Watch it for 30 seconds. Nothing will change:

```
kubectl get service booking-edge-lb -n kcna-lab12 -w --request-timeout=30s
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-lb	LoadBalancer	10.96.184.221	<pending>	80:31544/TCP	3s

error: Timeout exceeded while reading body

<pending> is the correct and permanent result on kind. It will never resolve. Do not wait for it, and do not treat it as a broken lab.

Confirm there is genuinely nothing in status:

```
kubectl get service booking-edge-lb -n kcna-lab12 -o jsonpath='{.status.loadBalancer}{"\n"}'
```

Expected output:

```
{}
```

And that no controller has said anything about it:

```
kubectl describe service booking-edge-lb -n kcna-lab12 | tail -6
```

Expected output:

```
Session Affinity:      None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:                <none>
```

Why. `type: LoadBalancer` is a *request*, not an implementation. Something has to watch for these Services and write `.status.loadBalancer.ingress[]`:

Environment	What fulfils the request
EKS / GKE / AKS	The cloud-controller-manager , which calls the cloud's load-balancer API
Bare metal / on-prem	MetalLB (ARP/BGP), or a vendor controller
Local <code>kind</code>	cloud-provider-kind , a separate binary you run alongside <code>kind</code>
Stock <code>kind</code> (this cluster)	Nothing . No cloud-controller-manager is installed.

`Events: <none>` is the diagnostic signature: with MetalLB installed you would see `IPAllocated/nodeAssigned` events here; an empty event list means *nobody is even looking at this object*.

Note what the API server **did** do: it allocated a ClusterIP *and* a nodePort (`80:31544/TCP`). That is the superset relationship — the object is a working NodePort Service that is additionally waiting for an external address that will never arrive:

```
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}')
LB_NODEPORT=$(kubectl get service booking-edge-lb -n kcna-lab12 -o jsonpath='{.spec.ports[0].nodePort}')
echo "auto-allocated nodePort: ${LB_NODEPORT}"
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 "http://${NODE_IP}:${LB_NODEPORT}"/"
```

Expected output (your nodePort will differ; it is allocated from 30000–32767):

```
auto-allocated nodePort: 31544
Meridian Freight booking-edge :: kcna-lab12
```

Step 6 — Headless: no virtual IP at all

```
kubectl apply -f manifests/50-service-headless.yaml
```

Expected output:

```
service/booking-edge-headless created
kubectl get service booking-edge-headless -n kcna-lab12
```

Expected output — note `CLUSTER-IP` is the literal **None**:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-headless	ClusterIP	None	<none>	80/TCP	4s

Compare the DNS answers. First the normal ClusterIP Service — **one** address, the virtual IP:

```
kubectl exec -n kcna-lab12 net-client -- nslookup booking-edge-clusterip
```

Expected output:

```
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   booking-edge-clusterip.kcna-lab12.svc.cluster.local
Address: 10.96.51.7
```

Now the headless Service — **one A record per ready Pod**:

```
kubectl exec -n kcna-lab12 net-client -- nslookup booking-edge-headless
```

Expected output (these are the Pod IPs from Step 1, not a service IP):

```
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.31
Name:   booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.32
```

As in Lab 11, BusyBox may add a ***** Can't find ...: No answer** line for the AAAA query. Judge by the **Address:** lines.

Because there is no virtual IP, **there is no load balancing**. The client library picks one of the returned addresses. This is exactly what a StatefulSet needs so that `pod-0` and `pod-1` are individually addressable (Lab 19).

Step 7 — Side-by-side comparison

```
kubectl get services -n kcna-lab12 -o wide
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
SELECTOR					
booking-edge-clusterip app=booking-edge	ClusterIP	10.96.51.7	<none>	80/TCP	6m
booking-edge-headless app=booking-edge	ClusterIP	None	<none>	80/TCP	1m
booking-edge-lb app=booking-edge	LoadBalancer	10.96.184.221	<pending>	80:31544/TCP	3m
booking-edge-nodeport app=booking-edge	NodePort	10.96.77.140	<none>	80:30080/TCP	5m

Four Services, **one** set of Pods, **one** identical selector. The Service type changes only *how the address is exposed*, never *what is behind it*. Confirm that all four share the same backends:

```
kubectl get endpointslices -n kcna-lab12 \
-o custom-columns='SLICE:.metadata.name,SERVICE:.metadata.labels.kubernetes\.io/service-name,PORTS:.ports[*].port,ENDPOINTS:.endpoints[*].addresses[0]'
```

Expected output:

SLICE	SERVICE	PORTS	ENDPOINTS
booking-edge-clusterip-9xnv	booking-edge-clusterip	5678	10.244.0.31,10.244.0.32
booking-edge-headless-2rk4d	booking-edge-headless	5678	10.244.0.31,10.244.0.32
booking-edge-lb-lm7ct	booking-edge-lb	5678	10.244.0.31,10.244.0.32
booking-edge-nodeport-vv8sq	booking-edge-nodeport	5678	10.244.0.31,10.244.0.32

The recorded expectations for each type live in `data/service-matrix.csv`:

```
cat data/service-matrix.csv
```

Expected output:

```
name,type,cluster_ip,port,target_port,node_port,external_ip_on_kind,reachable_from_laptop
booking-edge-clusterip,ClusterIP,allocated,80,5678,none,none,no - use kubectl port-forward
booking-edge-nodeport,NodePort,allocated,80,5678,30080,none,only if the kind cluster was
created with extraPortMappings
booking-edge-lb,LoadBalancer,allocated,80,5678,auto,pending-forever,no - kind has no
cloud-controller-manager
booking-edge-headless,ClusterIP,None,80,5678,none,none,no - DNS returns Pod IPs and there
is no virtual IP
```

`verification/checks.sh` asserts the live cluster against every row of this file — including asserting that the LoadBalancer is pending.

Lab 12 · Verification

```
bash verification/checks.sh
```

Expected output:

```
== Lab 12 verification: Service types on kind ==
PASS namespace kcna-lab12 exists
PASS deployment booking-edge has 2/2 ready replicas
PASS matrix row booking-edge-clusterip: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-clusterip: clusterIP allocated as expected
PASS matrix row booking-edge-clusterip: no nodePort allocated, as expected
PASS matrix row booking-edge-nodeport: type=NodePort port=80 targetPort=5678
PASS matrix row booking-edge-nodeport: clusterIP allocated as expected
PASS matrix row booking-edge-nodeport: nodePort is 30080
PASS matrix row booking-edge-nodeport: nodePort 30080 is inside 30000-32767
PASS matrix row booking-edge-lb: type=LoadBalancer port=80 targetPort=5678
PASS matrix row booking-edge-lb: clusterIP allocated as expected
PASS matrix row booking-edge-lb: auto-allocated nodePort 31544 is inside 30000-32767
PASS EXPECTED ON kind: booking-edge-lb EXTERNAL-IP is still <pending> (no cloud-
controller-manager)
PASS matrix row booking-edge-headless: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-headless: clusterIP is None as expected
PASS matrix row booking-edge-headless: no nodePort allocated, as expected
PASS all four Services resolve to the same 2 endpoint addresses
PASS ClusterIP Service serves the exact text in data/edge-banner.txt
PASS NodePort 30080 answers on the node InternalIP from inside the cluster
PASS headless Service DNS returns Pod IPs, not a virtual IP
-- 20 passed, 0 failed --
```

See `verification/expected-output.md` for the annotated reference.

Lab 12 · Failure injection — a nodePort outside the allowed range

6.1 Break it

```
kubectl apply -f manifests/70-service-nodeport-invalid.yaml
```

Expected output (this is a **rejection**, and the command exits non-zero):

```
The Service "booking-edge-nodeport-invalid" is invalid: spec.ports[0].nodePort: Invalid
value: 8080: provided port is not in the valid range. The range of valid ports is 30000-
32767
```

6.2 Diagnose

Three facts are packed into that one line:

1. **The object was never created.** Confirm:

```
kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
```

Expected output:

```
Error from server (NotFound): services "booking-edge-nodeport-invalid" not found
```

1. **The rejection came from the API server, not from your YAML tooling.** The file is schema-valid — `kubeconform` accepts it, because "is 8080 inside the node-port range" is *API-server validation*, not OpenAPI schema validation. This is the difference between a manifest that is *well-formed* and one that is *admissible*.
2. **The range is a cluster setting, not a constant.** It comes from the API server flag `--service-node-port-range`, default 30000-32767. Read it off this cluster's control plane:

```
kubectl get pod -n kube-system -l component=kube-apiserver \
-o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep -i 'node-port'
|| echo "flag not set - cluster is using the default 30000-32767"
```

Expected output on stock kind (the flag is not passed, so the default applies):

```
flag not set - cluster is using the default 30000-32767
```

6.3 Fix and re-verify

```
kubectl apply -f manifests/70-service-nodeport-invalid.yaml --dry-run=client -o yaml \
| sed 's/nodePort: 8080/nodePort: 30081/' \
| kubectl apply -f -
```

Expected output:

```
service/booking-edge-nodeport-invalid created
kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-nodeport-invalid	NodePort	10.96.9.201	<none>	80:30081/TCP	4s

6.4 Second failure to be aware of — port already allocated

Try to claim 30080 a second time:

```
kubectl create service nodeport clash --tcp=80:5678 --node-port=30080 -n kcna-lab12
```

Expected output:

```
The Service "clash" is invalid: spec.ports[0].nodePort: Invalid value: 30080: provided
port is already allocated
```

Node ports are a **cluster-wide** shared resource. Hard-coding them, as this lab does for teaching purposes, does not scale past a handful of Services — which is exactly the argument for Ingress (Lab 13).

Lab 12 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
EXTERNAL-IP stuck at <pending>	No load-balancer implementation in the cluster	kubectl describe svc <name> → Events: <none>	Expected on kind. Use NodePort or port-forward; install cloud-provider-kind or MetalLB if you truly need it
curl http://<clusterIP>/ from the laptop times out	ClusterIP range is virtual and cluster-internal	curl exits 28; the same URL works from net-client	kubectl port-forward svc/<name> 18080:80
curl http://<nodeIP>:30080/ times out from a macOS/Windows laptop	Docker's bridge network is inside a VM	Same request from net-client succeeds	Recreate kind with extraPortMappings (Appendix A) or use port-forward
provided port is not in the valid range	nodePort outside --service-node-port-range	Read the error; check the flag on kube-apiserver	Choose a port in 30000–32767, or omit nodePort and let the API server allocate
provided port is already allocated	Another Service in any namespace holds that nodePort	kubectl get svc -A -o jsonpath='{range .items [*]} {spec.ports[*].nodePort}{"\n"}{end}'	Pick a free port or omit nodePort
Headless Service returns No answer	Headless DNS only publishes ready endpoints; zero ready Pods means zero records	kubectl get endpointslices -n kcna-lab12 -l kubernetes.io/service-name=booking-edge-headless	Fix Pod readiness first
ImagePullBackOff on hashicorp/http-echo:1.0	No registry access from the cluster	kubectl describe pod <pod> -n kcna-lab12 → Failed to pull image	Substitute nginx:1.27-alpine with targetPort: 80 in all four Services; the Service-type behaviour is unchanged

Lab 12 · Cleanup

Delete **only** this lab's namespace:

```
kubectl delete namespace kcna-lab12
```

Expected output:

```
namespace "kcna-lab12" deleted
```

If you left a port-forward running, stop it:

```
kill %1 2>/dev/null; echo "done"
```

Expected output:

```
done
```

Never run `kubectl delete svc --all` without `-n kcna-lab12`.

Lab 12 · What you learned

- The four shapes are **nested**, not parallel: ClusterIP $\subset$ NodePort $\subset$ LoadBalancer; headless is ClusterIP with the VIP deliberately removed.
- The Service **type only changes the front door**. All four of your Services produced identical EndpointSlices from the identical selector.
- `nodePort` comes from a **cluster-wide** pool, default **30000–32767** (`--service-node-port-range`), and collides across namespaces.
- On kind, **type: LoadBalancer never leaves <pending>**, because no cloud-controller-manager or MetalLB is installed. `Events: <none>` on the Service is the proof that no controller is watching it.
- A NodePort works *inside* the cluster on stock kind; reaching it from the host needs `extraPortMappings` chosen at cluster-creation time (or a Linux host with native Docker).
- `clusterIP: None` removes the virtual IP; DNS then returns one A record per ready Pod and the client chooses.
- API-server validation (`nodePort` range, port already allocated) is stricter than schema validation — a file `kubeconform` accepts can still be rejected on apply.

Lab 12 · Further reading

- Service, and the publishing types — <https://kubernetes.io/docs/concepts/services-networking/service/#publishing-services-service-types>
- Headless Services — <https://kubernetes.io/docs/concepts/services-networking/service/#headless-services>
- Virtual IPs and Service proxies — <https://kubernetes.io/docs/reference/networking/virtual-ips/>
- kube-apiserver flags (`--service-node-port-range`) — <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>
- Cloud controller manager — <https://kubernetes.io/docs/concepts/architecture/cloud-controller/>
- Use Port Forwarding to Access Applications in a Cluster — <https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

Lab 12 · Appendix A — a kind cluster whose NodePort is reachable from the host

Optional, and it creates a second cluster. Only do this if you want to see

`http://localhost:30080` work. It does not delete your existing cluster.

```
kind create cluster --name kcna-nodeport --config data/kind-cluster-nodeport.yaml
```

Expected output (abridged):

```
Creating cluster "kcna-nodeport" ...
✓ Ensuring node image (kindest/node:v1.31.0) 
✓ Preparing nodes 
✓ Writing configuration 
✓ Starting control-plane 
✓ Installing CNI 
✓ Installing StorageClass 
Set kubectl context to "kind-kcna-nodeport"
```

Redeploy this lab into that cluster and `curl http://127.0.0.1:30080/` will return the banner, because Docker is publishing the node container's port 30080 on your loopback interface. `type:`

LoadBalancer **still stays** <pending> there — extraPortMappings solves NodePort host access, not the missing cloud provider.

When finished, switch back to your training cluster:

```
kubect1 config use-context kind-kind
```

Lab 13 — Ingress Resources and HTTP Routing

Field	Value
Lab ID	Lab 13
Title	Ingress Resources and HTTP Routing
Day / Topic	Day 3 · Services and Networking
Duration	50 minutes
Namespace	kcna-lab13
Learning outcome	LO3 — Develop a solution architecture within Kubernetes.
Ability	A2 Develop a solution architecture utilising appropriate tools, techniques and models of system components and interfaces
Knowledge	K2 Components of solution architecture · K4 Steps for developing solution architecture
Deck slide	Slide 305
Repository path	courseware/labs/lab-13-ingress-routing/

Goal. Model Meridian Freight's host and path routing with Ingress, then verify the rules through a pinned Traefik controller. Distinguish accepted API configuration from working HTTP routing.

What you will produce:

- Ingress host, Prefix and Exact rules aligned with data/routing-table.csv
- A TLS declaration and lab TLS Secret
- Controller inventory and observed routing evidence
- Optional Traefik 41.4.0 chart installation with an explicit IngressClass
- Seven live HTTP checks, including element boundaries and the standalone fallback Ingress

Read this before you start

An **Ingress** is a **declaration, not an implementation**. The object is served by the API server; the *routing* is performed by a separately installed **ingress controller**. A stock `kind` cluster has **no ingress controller and no IngressClass**, so everything you create in Steps 1–7 will be accepted, will look correct, and will **route nothing**.

That is the central lesson of this lab. This README runs in **two phases**:

Phase	What you do	Where
Phase 1 — required	Author the Ingress, verify it as a <i>declaration</i> , and then prove <i>measurably</i> that this cluster has no controller to execute it	Section 4, Steps 1–8
Phase 2 — optional	Install a pinned, maintained controller into a lab-owned namespace, reach it with <code>kubect1 port-forward</code> , and exercise every routing rule for real	Appendix C

Phase 1 never shows you a `curl` transcript of routed traffic, because none is produced. Phase 2 gives you exact, reproducible commands and the output you should expect — **run them and**

compare against your own cluster. Appendix C needs **no new cluster**: it works on the training cluster you already have.

Lab 13 · Objective

By the end of this lab you will be able to:

1. Author a `networking.k8s.io/v1` Ingress with **host-based** and **path-based** rules over multiple backend Services.
2. Choose correctly between `pathType: Prefix`, `Exact` and `ImplementationSpecific`, and state the element-boundary rule that makes `/catalog` **not** match `/catalogue`.
3. Explain `ingressClassName` and the IngressClass object, and recognise the **silent failure** when neither is present.
4. Read the `tls` block and say precisely what the Ingress does and does not do about TLS.
5. **Prove whether a cluster has an ingress controller at all**, ranking the evidence correctly: the absence of any **IngressClass** and the absence of any **controller Pod** are the reliable checks; a blank `ADDRESS` column and `Events: <none>` are *suggestive but not proof*.
6. (Optional, Appendix C) Install a pinned ingress controller into a lab-owned namespace, reach it with `kubectrl port-forward`, and verify each routing rule — including the `/catalog` vs `/catalogue` boundary — with real HTTP requests.

Lab 13 · Prerequisites

- Labs 11 and 12 completed. **This lab routes to Service backends**, so you must already be fluent with ClusterIP Services and EndpointSlices. (For completeness: the `networking.k8s.io/v1` Ingress API defines *two* backend forms — `backend.service`, used here, and `backend.resource`, a typed reference to another object. `backend.resource` is out of scope for KCNA and for this lab, and support for it varies by controller, but "an Ingress can only point at a Service" is not what the API says.)
- A running single-node `kind` cluster.
- *Optional, for Appendix C only*: `helm` (v3 or later) and `curl` on your workstation. Appendix C also documents a Helm-free path.
- Working directory:

```
cd courseware/labs/lab-13-ingress-routing
pwd
```

Expected output:

```
/.../courseware/labs/lab-13-ingress-routing
```

- Confirm the Ingress API is served by your cluster:

```
kubectrl api-resources --api-group=networking.k8s.io
```

Expected output:

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
ingressclasses		networking.k8s.io/v1	false	IngressClass
ingresses	ing	networking.k8s.io/v1	true	Ingress
networkpolicies	netpol	networking.k8s.io/v1	true	NetworkPolicy

Note that `ingressclasses` is **cluster-scoped** (`NAMESPACED false`) while `ingresses` is namespaced. That asymmetry matters in Step 5.

Lab 13 · Scenario

Meridian Freight's customer portal is being consolidated. Today three services are each exposed on their own NodePort, and partners have to remember three different `:3xxxx` URLs. Worse, Lab 12 showed that node ports come from one cluster-wide pool of 2,768 numbers — that does not scale.

The target design is a single HTTP entry point:

Request	Should reach
<code>http://shop.meridianfreight.internal/catalog</code> and anything below it	<code>catalog-svc</code>
<code>http://shop.meridianfreight.internal/checkout/status</code> — that URL exactly	<code>checkout-svc</code>
<code>http://tracking.meridianfreight.internal/</code> and anything below it	<code>catalog-svc</code>
anything else	the default backend, <code>route-fallback-svc</code>

You will express that design as an Ingress object, verify the object is correct, and then determine honestly whether this training cluster can execute it.

Lab 13 · Step-by-step procedure

Step 1 — Namespace and the routing table

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab13 created
```

The intended routing is recorded as data before you write any YAML:

```
cat data/routing-table.csv
```

Expected output:

```
host,path,pathType,service,port
shop.meridianfreight.internal,/catalog,Prefix,catalog-svc,80
shop.meridianfreight.internal,/checkout/status,Exact,checkout-svc,80
tracking.meridianfreight.internal/,Prefix,catalog-svc,80
```

`verification/checks.sh` diffs the live Ingress against exactly this file, so the file is the specification and the Ingress is the implementation.

Step 2 — Backends

```
kubectl apply -f manifests/10-configmap-catalog.yaml
kubectl apply -f manifests/20-deployments.yaml
kubectl apply -f manifests/30-services.yaml
kubectl apply -f manifests/35-pod-net-client.yaml
```

Expected output:

```

configmap/catalog-content created
configmap/catalog-nginx created
deployment.apps/catalog created
deployment.apps/checkout created
deployment.apps/route-fallback created
service/catalog-svc created
service/checkout-svc created
service/route-fallback-svc created
pod/net-client created

kubectl wait --for=condition=Available deployment --all -n kcna-lab13 --timeout=120s
kubectl wait --for=condition=Ready pod/net-client -n kcna-lab13 --timeout=60s

```

Expected output:

```

deployment.apps/catalog condition met
deployment.apps/checkout condition met
deployment.apps/route-fallback condition met
pod/net-client condition met

```

Step 3 — Prove the backends work before involving Ingress

This is the discipline that saves hours later: never debug an Ingress until you have proved the Service behind it answers.

```

kubectl exec -n kcna-lab13 net-client -- wget -qO- --timeout=5 http://catalog-svc/ | head -3

```

Expected output:

```

<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Container Catalog</title></head>

kubectl exec -n kcna-lab13 net-client -- wget -qO- --timeout=5 http://checkout-svc/

```

Expected output:

```

checkout status OK :: kcna-lab13

kubectl exec -n kcna-lab13 net-client -- wget -qO- --timeout=5 http://route-fallback-svc/

```

Expected output:

```

no ingress rule matched - default backend :: kcna-lab13

```

All three backends are healthy. **Remember this:** anything that fails from here on is an Ingress-layer problem, not an application problem.

Step 4 — Create the Ingress

```

kubectl apply -f manifests/40-ingress-shop.yaml

```

Expected output:

```

ingress.networking.k8s.io/meridian-shop created

kubectl get ingress -n kcna-lab13

```

Expected output on a stock kind cluster:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE	
meridian-shop	meridian-edge					
shop.meridianfreight.internal,tracking.meridianfreight.internal					80	8s

The ADDRESS column is empty, and on this cluster it will stay empty. ADDRESS is rendered from `.status.loadBalancer.ingress[]`, which only a controller writes. No controller, no address.

But read the implication in the right direction. No controller $\Rightarrow$ blank ADDRESS. The converse does **not** hold: blank ADDRESS does *not* prove there is no controller. A controller that is reachable in-cluster or through a port-forward — the Appendix C setup is exactly that — routes traffic correctly while publishing no external address at all, so ADDRESS stays blank. Treat a blank ADDRESS as a *prompt to investigate*, and settle the question with `kubectl get ingressclass` and a controller-Pod search (Step 8).

Note also that PORTS shows 80 only — 443 appears once an Ingress in the namespace has a `tls` block (you will see that change in Step 7).

Step 5 — Read the rules the way a controller would

```
kubectl describe ingress meridian-shop -n kcna-lab13
```

Expected output:

```
Name: meridian-shop
Labels: app.kubernetes.io/part-of=meridian-freight
Namespace: kcna-lab13
Address:
Ingress Class: meridian-edge
Default backend: route-fallback-svc:80 (10.244.0.44:5678)
Rules:
  Host      Path      Backends
  ----      -
  shop.meridianfreight.internal
            /catalog   catalog-svc:80 (10.244.0.42:8080)
            /checkout/status checkout-svc:80 (10.244.0.43:5678)
  tracking.meridianfreight.internal
            /          catalog-svc:80 (10.244.0.42:8080)
Annotations: <none>
Events: <none>
```

Three diagnostics live in this one output — note how strong each one actually is:

Line	Reading	Strength of evidence
Address: empty	Nothing has published a load-balancer address for this Ingress	Weak. Suggestive only. A controller reached via ClusterIP + port-forward, or any controller configured without a published service or <code>ingressEndpoint</code> , routes perfectly and still leaves this blank
Backends show real Pod IPs, e.g. (10.244.0.42:8080)	<code>kubectl</code> resolved each Service to its endpoints. If a backend showed (<code><error: endpoints "catalog-svc" not found></code>) the Service name is wrong or has no endpoints — a Lab 11 problem, not an Ingress problem	Strong, about the <i>backends</i>
Events: <code><none></code>	No controller has written an event on this object	Weak. Some controllers emit <code>Sync</code> -style events, others emit none at all in normal operation. Absence of events is not absence of a controller

The two checks that *do* settle it are in Step 8: is there an **IngressClass**, and is there a **controller Pod**.

Verify the object matches the specification in `data/routing-table.csv`:

```
kubectl get ingress meridian-shop -n kcna-lab13 -o go-template='{{range .spec.rules}}
{{ $h := .host }}{{range .http.paths}}{{ $h }}, {{.path}}, {{.pathType}},
{{.backend.service.name}}, {{.backend.service.port.number}} {{ "\n" }} {{ end }} {{ end }}'
```

Expected output — identical to the CSV body, in order:

```
shop.meridianfreight.internal,/catalog,Prefix,catalog-svc,80
shop.meridianfreight.internal,/checkout/status,Exact,checkout-svc,80
tracking.meridianfreight.internal/,Prefix,catalog-svc,80
```

Step 6 — *pathType*, precisely

There are exactly three legal values. Confirm from the live schema rather than memory:

```
kubectl explain ingress.spec.rules.http.paths.pathType
```

Expected output (abridged — the description is long):

```
KIND:      Ingress
VERSION:   networking.k8s.io/v1

FIELD: pathType <string>

DESCRIPTION:
  PathType determines the interpretation of the Path matching. PathType can be
  one of the following values:
  * Exact: Matches the URL path exactly.
  * Prefix: Matches based on a URL path prefix split by '/'. Matching is done on
    a path element by element basis. ...
  * ImplementationSpecific: Interpretation of the Path matching is up to the
    IngressClass. ...
```

Internalise the element-boundary rule, because it is examinable and it is the source of real outages:

Rule	Request path	Match?	Why
/catalog Prefix	/catalog	yes	exact element match
/catalog Prefix	/catalog/	yes	trailing separator
/catalog Prefix	/catalog/containers/ 20ft	yes	/catalog is a complete leading element sequence
/catalog Prefix	/catalogue	no	catalogue ≠ the element catalog; prefix matching is not string matching
/checkout/status Exact	/checkout/status	yes	identical
/checkout/status Exact	/checkout/status/	no	trailing slash makes it a different path
/checkout/status Exact	/checkout/status?id=7	yes	the query string is not part of the path

When two rules match, the **longest matching path wins**; where an `Exact` and a `Prefix` rule are equally long, `Exact` takes precedence. `ImplementationSpecific` hands the decision to the controller — for example, some controllers interpret the path as a regular expression. Portable manifests avoid it.

Step 7 — *The TLS block*

```
kubectl apply -f manifests/50-ingress-tls-example.yaml
```

Expected output:

```
ingress.networking.k8s.io/meridian-shop-tls created
```



```
kubectl get ingress -n kcna-lab13
```

Expected output — the new object advertises 80, 443:

NAME	ADDRESS	PORTS	CLASS	AGE	HOSTS
meridian-shop			meridian-edge		
shop.meridianfreight.internal,tracking.meridianfreight.internal				80	4m
meridian-shop-tls			meridian-edge		secure.meridianfreight.internal
80, 443		6s			

Look at the structure — this is the **entire** TLS surface of the Ingress API:

```
kubectl get ingress meridian-shop-tls -n kcna-lab13 -o jsonpath='{.spec.tls}' ; echo
```

Expected output:

```
[{"hosts":["secure.meridianfreight.internal"],"secretName":"meridian-shop-tls-cert"}]
```

Two fields, and that is all:

- `hosts` — the SNI names this certificate is presented for.
- `secretName` — a Secret of type `kubernetes.io/tls` **in the same namespace**, holding `tls.crt` and `tls.key`.

The Ingress object does not terminate TLS. The controller does, by reading that Secret. Note that the Secret does not exist yet and the object was still accepted — another silent dependency. Create it so you have seen the required shape (a throwaway self-signed certificate, lab use only):

```
openssl req -x509 -nodes -newkey rsa:2048 -days 1 \
  -keyout /tmp/lab13-tls.key -out /tmp/lab13-tls.crt \
  -subj "/CN=secure.meridianfreight.internal" 2>/dev/null
kubectl create secret tls meridian-shop-tls-cert -n kcna-lab13 \
  --cert=/tmp/lab13-tls.crt --key=/tmp/lab13-tls.key
```

Expected output:

```
secret/meridian-shop-tls-cert created
kubectl get secret meridian-shop-tls-cert -n kcna-lab13
```

Expected output:

NAME	TYPE	DATA	AGE
meridian-shop-tls-cert	kubernetes.io/tls	2	5s

TYPE `kubernetes.io/tls` and DATA 2 (`tls.crt` + `tls.key`) is the exact shape an ingress controller looks for. In production you would never hand-roll this — use `cert-manager` or your organisation's CA.

Step 8 — Determine, honestly, whether this cluster can route

Combine the known cluster configuration, controller inventory and live routing probes. No single empty inventory query proves absence on an arbitrary cluster; controllers may use different names, run externally or accept class-less Ingresses.

Check 1 (inventory) — does any IngressClass exist? The object identifies a controller type but does not prove a running controller. Ask which classes exist:

```
kubectl get ingressclass
```

Expected output on a **stock kind** cluster:

```
No resources found
```

Check 2 (inventory) — is a known controller Pod running? Label conventions differ between projects, so search on the workload name too, not just on one label:

```
kubectl get pods --all-namespaces -l app.kubernetes.io/component=controller
kubectl get pods --all-namespaces -o custom-
columns=NS:.metadata.namespace,POD:.metadata.name --no-headers \
| grep -Ei 'ingress|traefik|contour|haproxy|envoy' || echo "no controller-shaped Pod
found"
```

Expected output on a **stock kind cluster**:

```
No resources found
no controller-shaped Pod found
```

Check 3 (corroborating only) — did anything write a status onto your Ingress?

```
kubectl get ingress meridian-shop -n kcna-lab13 -o jsonpath='{.status.loadBalancer}' ;
echo
```

Expected output:

```
{}
```

Remember Step 4: an empty `.status.loadBalancer` on its own would **not** prove anything. It only corroborates checks 1 and 2. Indeed, when you install the controller in Appendix C, routing will start working and this field will *still* print `{}` — because that controller is deliberately configured with a ClusterIP Service and no published address.

Conclusion, stated plainly: checks 1 and 2 both come back empty, so this cluster has no ingress controller. Your Ingress objects are syntactically correct, semantically correct, and completely inert. No host, port or IP on this cluster will exercise these rules — and if any tutorial shows you a `curl` transcript without installing a controller first, it is wrong.

An IngressClass is what a controller installs to advertise itself. Here is the shape you would see afterwards, so you can recognise it (this is **not** created by Steps 1–8; Appendix C creates a real one):

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: meridian-edge
  annotations:
    # Optional. It is ONE mechanism by which a controller may pick up an Ingress
    # that names no class. It is not the only one - see section 6.2.
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: example.com/ingress-controller # the controller's own identifier
```

`spec.controller` is an opaque identifier that a specific controller binary watches for. Your `ingressClassName: meridian-edge` is a **forward reference** to an object that does not exist yet — which is legal, and silent. Appendix C makes that reference resolve by installing a controller that names its IngressClass `meridian-edge`.

Lab 13 · Verification

`verification/checks.sh` has **two modes** and prints which one it ran:

Mode	Trigger	What it asserts
MODE 1 — declaration-only	no IngressClass exists on the cluster	the Ingress objects are correct <i>declarations</i> . Live routing is reported SKIP — never PASS, never FAIL

Mode	Trigger	What it asserts
MODE 2 — controller	an IngressClass exists	everything MODE 1 checks, plus seven real HTTP routing assertions driven through <code>kubect1 port-forward</code>

```
bash verification/checks.sh
```

Expected output on a stock kind cluster (**MODE 1**):

```
== Lab 13 verification: Ingress resources and HTTP routing ==
PASS namespace kcna-lab13 exists
PASS ConfigMap catalog-content matches data/catalog-index.html byte-for-byte
PASS all 3 backend Deployments are Available
PASS catalog-svc, checkout-svc and route-fallback-svc all have endpoints
PASS backend catalog-svc answers directly (Ingress bypassed)
PASS backend checkout-svc answers directly (Ingress bypassed)
PASS backend route-fallback-svc answers directly (Ingress bypassed)
PASS ingress meridian-shop exists with apiVersion networking.k8s.io/v1
PASS ingress rules match data/routing-table.csv exactly
PASS every pathType is one of Exact / Prefix / ImplementationSpecific
PASS ingress meridian-shop sets ingressClassName (meridian-edge)
PASS ingress meridian-shop declares a defaultBackend (route-fallback-svc:80)
PASS ingress meridian-shop-tls declares tls[0].secretName
PASS TLS Secret meridian-shop-tls-cert is type kubernetes.io/tls with 2 keys
NOTE MODE 1 (declaration-only): no IngressClass exists on this cluster.
PASS DECISIVE: 0 IngressClasses exist, so no controller can claim the Ingress
PASS DECISIVE: no Ready controller Pod in namespace kcna-lab13-ingress
NOTE CORROBORATING: .status.loadBalancer is empty (ADDRESS column blank). This does not
establish whether routing works.
SKIP live HTTP routing not exercised: this cluster has no ingress controller (MODE 1)
NOTE The Ingress objects are verified as CORRECT DECLARATIONS only.
NOTE README Appendix C installs a pinned controller and switches this script into MODE 2.
-- 16 passed, 0 failed, 1 skipped --
```

Read the last four lines carefully:

- The blank ADDRESS is now a **NOTE**, not a **PASS**. It is corroboration, not proof. The two DECISIVE lines are what actually settle the question.
- Routing is **SKIP**, not **FAIL**. The script did not test it, so it reports neither success nor failure — that is the honesty contract.

MODE 2 appears once you complete Appendix C. See Appendix C step C6 for the exact expected transcript.

Lab 13 · Failure injection — an invalid pathType, then a silent one

6.1 Break it — the loud failure

```
kubect1 apply -f manifests/60-ingress-invalid-pathtype.yaml
```

Expected output (rejected; non-zero exit):

```
The Ingress "meridian-shop-broken" is invalid: spec.rules[0].http.paths[0].pathType:
Unsupported value: "prefix": supported values: "Exact", "ImplementationSpecific", "Prefix"
```

Diagnosis. pathType is a case-sensitive enum. prefix is not Prefix. The error names the field path (spec.rules[0].http.paths[0].pathType), the rejected value, and the complete legal set — read the whole line before guessing.

Confirm nothing was created:

```
kubectl get ingress meridian-shop-broken -n kcna-lab13
```

Expected output:

```
Error from server (NotFound): ingresses.networking.k8s.io "meridian-shop-broken" not found
```

Note again that the file is **schema-valid**:

```
kubeconform -strict -summary manifests/60-ingress-invalid-pathtype.yaml
```

Expected output:

```
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

`pathType` is typed as a plain string in the OpenAPI schema; the enum is enforced by API-server validation. Static linting cannot catch this class of defect.

6.2 Fix defect 1 — and meet the silent failure

```
sed 's/pathType: prefix/pathType: Prefix/' manifests/60-ingress-invalid-pathtype.yaml \
| kubectl apply -f -
```

Expected output:

```
ingress.networking.k8s.io/meridian-shop-broken created
```

It was accepted. Now look at it:

```
kubectl get ingress meridian-shop-broken -n kcna-lab13
```

Expected output — note the empty `CLASS` column:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
meridian-shop-broken	<none>	shop.meridianfreight.internal		80	12s

```
kubectl describe ingress meridian-shop-broken -n kcna-lab13 | grep -E 'Ingress Class|Address|Events'
```

Expected output:

```
Ingress Class:    <none>
Address:
Events:           <none>
```

Diagnosis of the silent failure. `CLASS <none>` means this Ingress named no class. What happens next is **controller-dependent**, and this is a distinction worth getting right:

- On a cluster with **no controller at all** — this cluster, right now — nothing claims it. Guaranteed silence.
- The `ingressclass.kubernetes.io/is-default-class: "true"` annotation is **one** mechanism by which a controller may adopt a class-less Ingress. It is a common one, but it is not the only one, and it is not a universal rule.
- Some controllers additionally claim class-less Ingresses on their own terms. Traefik, for instance, documents that when its `providers.kubernetesIngress.ingressClass` option is empty it processes "resources missing the annotation, having an empty value, or the value `traefik`" (<https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/>).
- Other controllers ignore class-less Ingresses outright unless configured otherwise.

The operational rule that survives all of that: **name the class explicitly**. Never rely on a default, because you cannot predict which controller will or will not pick your object up.

Check whether this cluster has a default IngressClass:

```
kubectl get ingressclass -o jsonpath='{range .items[*]}{.metadata.name}{ "\t"}
{.metadata.annotations.ingressclass\.kubernetes\.io/is-default-class}{ "\n"}{end}'
echo "(no output above means: no IngressClass at all)"
```

Expected output:

```
(no output above means: no IngressClass at all)
```

On this cluster there is no controller and no IngressClass, so the object is certainly ignored. On a cluster that *did* have a controller, whether it is adopted depends on that controller's documented class-matching behaviour. A controller may emit events or status; neither is required for successful routing. Test the configured host and path through the controller endpoint. This is the single most common Ingress support ticket:

"I applied the Ingress and nothing happens."

Your diagnostic checklist:

1. `kubectl get ingressclass` — does any controller advertise itself? (*inventory*)
2. Is a controller **Pod** running anywhere? (*inventory*)
3. `kubectl get ingress -n <ns>` — is the `CLASS` column populated, and does that name match an installed IngressClass?
4. If `CLASS` is `<none>`: read **your controller's** documentation on class-less Ingresses. Do not assume the default-class annotation is the only rule.
5. `kubectl describe ingress` — do the backends resolve to real endpoints?
6. `kubectl describe ingress` — any controller `Events`? (*weak signal; many controllers emit none*)
7. `ADDRESS` blank? (*weakest signal; a correctly routing ClusterIP controller leaves it blank forever*)

6.3 Clean up the broken object

```
kubectl delete ingress meridian-shop-broken -n kcna-lab13
```

Expected output:

```
ingress.networking.k8s.io "meridian-shop-broken" deleted
```

Lab 13 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
<code>ADDRESS</code> column stays empty forever	Often no controller — but not necessarily . A controller with a ClusterIP Service, or one configured without a published service / <code>ingressEndpoint</code> , routes correctly and never fills this in	Inspect the configured controller using <code>kubectl get ingressclass</code> and a controller-Pod search. <code>Events: <none></code> is only a weak extra hint	If the known stock-kind configuration has no controller: expected on stock kind — install a controller (Appendix C). If a controller <i>is</i> present, ignore the blank <code>ADDRESS</code> and test routing directly
Routing works but <code>ADDRESS</code> is still blank	The controller is not publishing a status address (ClusterIP Service, or publishing disabled)	<code>curl</code> through a port-forward succeeds while <code>kubectl get ingress</code> shows no <code>ADDRESS</code>	Nothing to fix. This is the Appendix C configuration by design

Symptom	Likely cause	Command that confirms it	Fix
CLASS shows <none> and nothing routes, with no error	ingressClassName omitted; whether any controller adopts it is controller-dependent	kubectl get ingressclass -o jsonpath=...is-default-class, plus your controller's own class-matching documentation	Set spec.ingressClassName explicitly. Prefer explicit over default, always
/catalogue unexpectedly reaches catalog-svc on a controller you installed	That controller is doing character-by-character prefix matching instead of the spec's path-element matching	Compare /catalog and /catalogue responses through the port-forward	Turn on the controller's spec-conformance option — for Traefik, providers.kubernetesIngress.strictPrefixMatching=true, already set in data/traefik-lab-values.yaml
Unsupported value: "prefix" on apply	pathType enum is case-sensitive	Read the error — it lists all three legal values	Use Prefix, Exact or ImplementationSpecific
spec.rules[0].http.paths[0].path: Invalid value: "catalog": must be an absolute path	Path is missing its leading /	The API-server error names the exact field path	Write /catalog
describe shows <error: endpoints "catalog-svc" not found> under Backends	The named Service does not exist, or has no endpoints	kubectl get endpointslices -n kcna-lab13	Fix the Service name/selector first — this is a Lab 11 fault surfacing in an Ingress
Requests to /catalogue unexpectedly 404	Prefix matches whole path elements, not strings	Re-read the pathType table in Step 6	Add an explicit rule, or restructure the URL
/checkout/status/ 404s while /checkout/status works	Exact does not tolerate a trailing slash	Compare the two requests	Use Prefix, or add a second Exact rule
Controller logs secret ... not found for TLS	The tls[].secretName Secret is missing or is not type kubernetes.io/tls	kubectl get secret <name> -n kcna-lab13 -o jsonpath='{.type}'	Create it with kubectl create secret tls ..., or use cert-manager

Lab 13 · Cleanup

Delete **only** this lab's namespace. The Ingress, Services, Deployments, ConfigMaps and the TLS Secret all live inside it.

```
kubectl delete namespace kcna-lab13
```

Expected output:

```
namespace "kcna-lab13" deleted
```

Remove the throwaway certificate files from /tmp:

```
rm -f /tmp/lab13-tls.key /tmp/lab13-tls.crt && echo "temp certificate removed"
```

Expected output:

```
temp certificate removed
```

Steps 1–8 create no cluster-scoped objects at all.

If you also did **Appendix C**, remove the controller — it *does* create cluster-scoped objects (an IngressClass plus a ClusterRole and ClusterRoleBinding):

```
helm uninstall meridian-edge -n kcna-lab13-ingress
kubectl delete namespace kcna-lab13-ingress
kubectl get ingressclass
```

Expected output — `helm uninstall` removes the IngressClass, ClusterRole and ClusterRoleBinding it created, so the cluster returns to its stock state:

```
release "meridian-edge" uninstalled
namespace "kcna-lab13-ingress" deleted
No resources found
```

If you installed the vendored manifest instead of using Helm, delete it with `kubectl delete -f data/traefik-41.4.0-rendered.yaml` before deleting the namespace.

If you additionally built the optional cluster in **Appendix A**, delete it separately with `kind delete cluster --name kcna-ingress`. Appendix C does **not** need that cluster.

Lab 13 · What you learned

- An **Ingress is data, not behaviour**. The API server stores it; an ingress controller implements it. No controller means no routing, no error and no event.
- `ingressClassName` binds an Ingress to a controller. Omitting it makes the outcome **controller-dependent**: the `is-default-class` annotation is one adoption mechanism among several, some controllers claim class-less Ingresses on their own terms, and others ignore them entirely. Name the class explicitly and the ambiguity disappears.
- `pathType` has exactly three values, is case-sensitive, and `Prefix` matches whole **path elements** — `/catalog` never matches `/catalogue`. Longest match wins; `Exact` beats `Prefix` at equal length. Controllers can and do deviate from this, so verify it against the controller you actually run (Appendix C does exactly that).
- `defaultBackend` expresses the unmatched-request fallback. Appendix C supplies a separate rule-free Ingress because the pinned Traefik version requires that form.
- The `tls` block is only `hosts` + `secretName`, pointing at a `kubernetes.io/tls` Secret in the same namespace. The **controller** terminates TLS.
- **This lab routes to Service backends** (`backend.service`) — so an Ingress fault is very often a Lab 11 endpoint fault in disguise. Always test the backend Service directly first. The API also defines `backend.resource` for typed object references; it is out of scope here, but "Ingress can only point at a Service" is inaccurate.
- **Establish actual routing behaviour**: combine the known cluster configuration, IngressClass/controller inventory, readiness and HTTP probes. Empty inventory queries or blank `ADDRESS` alone are insufficient; a working controller behind a ClusterIP Service may leave status and events empty.

Lab 13 · Further reading

- Ingress — <https://kubernetes.io/docs/concepts/services-networking/ingress/>
- Ingress API reference (`backend.service` **and** `backend.resource`) — <https://kubernetes.io/docs/reference/kubernetes-api/networking/ingress-v1/>
- Ingress Controllers — <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>

- `pathType` and path matching — <https://kubernetes.io/docs/concepts/services-networking/ingress/#path-types>
- IngressClass and the default-class annotation — <https://kubernetes.io/docs/concepts/services-networking/ingress/#default-ingress-class>
- TLS in Ingress — <https://kubernetes.io/docs/concepts/services-networking/ingress/#tls>
- ingress-nginx **controller project** retirement announcement — <https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>
- Gateway API — <https://kubernetes.io/docs/concepts/services-networking/gateway/>
- kind: Ingress setup — <https://kind.sigs.k8s.io/docs/user/ingress/>
- `kubectrl port-forward` — <https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>
- Traefik: getting started on Kubernetes (Appendix C install method) — <https://doc.traefik.io/traefik/getting-started/kubernetes/>
- Traefik: Kubernetes Ingress provider options (`strictPrefixMatching`, `namespaces`, `publishedService`) — <https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/>
- Traefik Helm chart source and examples — <https://github.com/traefik/traefik-helm-chart>

Lab 13 · Appendix A — a kind cluster with host ports (optional, and not required)

You do not need this appendix to get live routing. Appendix C runs a real controller on the cluster you already have, using `kubectrl port-forward`. Read this one only to understand what `extraPortMappings` buys you, and when it is worth recreating a cluster for.

Optional. Creates a second cluster; does not touch your training cluster.

```
cat data/kind-cluster-ingress.yaml
```

Expected output (abridged):

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    listenAddress: "127.0.0.1"
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    listenAddress: "127.0.0.1"
    protocol: TCP

kind create cluster --name kcna-ingress --config data/kind-cluster-ingress.yaml
```

Both settings are **create-time only** — a running kind cluster cannot be retrofitted with either:

- `node-labels: ingress-ready=true` — some controller manifests packaged for kind pin the controller Pod to a node carrying this label.
- `extraPortMappings` for 80/443 — publishes the node container's ports on your laptop so `http://localhost/` reaches the controller on the real, unprefixed port.

What these settings are *not*. They are **not** the only way to reach a cluster workload from the host, and their absence does **not** prevent you from exercising an ingress controller:

Access method	Needs cluster recreation?	Reaches the controller on
<code>kind extraPortMappings + hostPort</code>	Yes — create-time only	<code>http://localhost/</code> (real port 80/443)
<code>kubectl port-forward</code>	No — works on any running cluster	<code>http://127.0.0.1:<local-port>/</code>
NodePort Service	No	the node's <code>:3xxxx</code> port (still needs a host route into the node on kind)

Appendix C uses `kubectl port-forward` precisely so that **no new cluster is needed**. The only thing you give up is the cosmetic convenience of port 80: you send `Host:` headers to `127.0.0.1:18080` instead of `127.0.0.1:80`. The Ingress rules under test are identical.

Lab 13 · Appendix B — the controller landscape, stated carefully (checked 5 September 2026)

The `networking.k8s.io/v1` **Ingress API is stable and not deprecated** — everything you wrote in this lab remains correct. What changed is the *controller* landscape.

Two different things called "nginx" — do not conflate them

Thing	Status	Where it appears in these labs
The Kubernetes community ingress-nginx controller project	Retired in March 2026 (announced at https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/). Do not select it for a new deployment	Nowhere. This lab does not use it
The NGINX web server container image , <code>nginx:1.27-alpine</code>	Unrelated and unaffected. It is an ordinary web server image	The <code>catalog</code> backend Pod in this lab, and backends in several other labs

The retirement of a *controller project* says nothing about the *web server image*. They share a name and nothing else.

What this lab verifies about controllers, and what it does not.

Claim	Status
Traefik chart <code>41.4.0 / app v3.7.12</code> installs on a single-node kind cluster and serves <code>networking.k8s.io/v1</code> Ingress	The path is pinned, reproducible and documented in Appendix C. Run it and confirm on your own cluster
Traefik's default prefix matching is character-by-character, and <code>providers.kubernetesIngress.strictPrefixMatching=true</code> makes it comply with the Ingress spec	Documented by the project: https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/
The community <code>ingress-nginx</code> controller project is retired	Announced by the Kubernetes project (link above)
Any <i>ranking</i> of which other controllers are "best" or "most maintained"	Not asserted here. Maintenance status changes; check it yourself at install time

How to evaluate a controller yourself, rather than trusting a table in any courseware:

1. Start from the upstream list — <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/> — and then go to each project's own site.
2. Check the release cadence and the latest release date in its own repository.
3. Check that it explicitly supports the `networking.k8s.io/v1` **Ingress** API, not only Gateway API.
4. Read its documentation on `pathType` conformance and on how it treats an Ingress with no `ingressClassName`. Both are controller-dependent (sections 6.2 and 7).
5. On a managed cluster, evaluate the provider's own controller first (AWS Load Balancer Controller, GKE Ingress, AGIC and similar), because it integrates with that platform's load balancers.

A precision point about Gateway API. `gateway.networking.k8s.io` — `GatewayClass` / `Gateway` / `HTTPRoute` — is the newer, role-oriented design that expresses natively much of what Ingress needed vendor annotations for. Note carefully:

- Gateway API is a **separate API family**, not a new version of Ingress.
- Projects such as **Envoy Gateway** are **Gateway API implementations**. Do not describe them as drop-in Ingress-controller replacements; if you need `networking.k8s.io/v1` Ingress, confirm from the project's own documentation whether and how it serves that API before choosing it.
- KCNA expects you to know **Ingress**, and to know that Gateway API exists and why.

Lab 13 · Appendix C — install a real controller and exercise the routes (optional, ~15 min)

Status of the transcripts in this appendix. They are the **expected result of the documented path**, derived from the pinned chart's own rendered manifests and the project's documentation. They are **not** a recording of a run on your cluster. Execute the commands and compare. If your output differs, your output is the truth — investigate, and treat any mismatch as a defect in this appendix.

What you are about to install

Property	Value
Controller	Traefik Proxy
Helm chart	traefik 41.4.0 , repo https://traefik.github.io/charts
Chart sha256	1470656b1c93e8637daaa19cdb30633c5ea691768a9aab3214da3ee822c4a9f0
App version	v3.7.12 → image <code>docker.io/traefik:v3.7.12</code>
Chart <code>kubeVersion</code> constraint	<code>>=1.25.0-0</code>
Install method	Helm, per the project's own docs: https://doc.traefik.io/traefik/getting-started/kubernetes/
Namespace	<code>kcna-lab13-ingress</code> — lab-owned, not kube-system
Ingress watch scope	<code>kcna-lab13</code> only
Service type	ClusterIP (no LoadBalancer, which would hang <pending> on kind)
Host access	<code>kubectl port-forward</code> — no new cluster, no extraPortMappings

Property	Value
Traefik CRDs	none installed (<code>--skip-crds + providers.kubernetesCRD=false</code>)
Gateway API	disabled
IngressClass created	meridian-edge, not marked default

Every value is set in `data/traefik-lab-values.yaml`, which is commented line by line.

The cluster-scoped footprint, stated up front

Steps 1–8 create nothing cluster-scoped. **This appendix does**, because an ingress controller genuinely cannot work without it. Exactly three cluster-scoped objects:

Object	Why it must be cluster-scoped
IngressClass/meridian-edge	IngressClass is a cluster-scoped kind by API design (you confirmed this in section 2: <code>NAMESPACED false</code>). A controller advertises itself with one; there is no namespaced equivalent
ClusterRole/meridian-edge-traefik-kcna-lab13-ingress	The controller must read that cluster-scoped IngressClass to know which Ingresses are its own. The chart's own documentation states that namespaced RBAC (<code>rbac.namespaced=true</code>) is incompatible with using IngressClass, which is why this lab leaves it <code>false</code>
ClusterRoleBinding/...	Binds the above ClusterRole to the controller's ServiceAccount in <code>kcna-lab13-ingress</code>

The ClusterRole grants **read-only** access, plus one write:

Rule	Verbs	Why the controller needs it
<code>"" configmaps, nodes, services</code>	get, list, watch	resolve Service backends and node addresses
<code>discovery.k8s.io endpointslices</code>	list, watch	find the Pod endpoints behind each Service (your Lab 11/12 knowledge)
<code>"" pods</code>	get	correlate endpoints with Pods
<code>"" secrets</code>	get, list, watch	load <code>kubernetes.io/tls</code> Secrets named by <code>spec.tls[].secretName</code>
<code>networking.k8s.io ingresses, ingressclasses</code>	get, list, watch	the core job
<code>networking.k8s.io ingresses/status</code>	update	the only write verb: publishing <code>ADDRESS</code> . This lab disables publishing anyway
<code>"" namespaces</code>	list, watch	discover namespaces to watch

Everything else — ServiceAccount, Service, Deployment — is namespaced inside `kcna-lab13-ingress` and disappears with the namespace.

C1 — Preconditions

Complete Steps 1–8 first, so the Ingress objects exist and you have seen the no-controller diagnosis. Then:

```
kubectl config current-context
helm version --short
```

Expected output (**your context name will differ** — record it, you will not need to change it, because everything below happens on this same cluster):

```
kind-<your-training-cluster>
v3.x.y+g...      # or v4.x.y+g... - both work
```

The chart is `apiVersion: v2`, so Helm 3 and Helm 4 both install it, and every flag used below (`--repo`, `--version`, `--skip-crds`, `--wait`, `--timeout`) exists in both.

Do not switch context. Appendix C deliberately runs on the training cluster you already have.

C2 — Create the lab-owned controller namespace

```
kubectl apply -f manifests/70-namespace-ingress.yaml
```

Expected output:

```
namespace/kcna-lab13-ingress created
```

C3 — Install the pinned controller

`--repo` is used instead of `helm repo add` so nothing in your global Helm configuration is modified.

```
helm install meridian-edge traefik \
  --repo https://traefik.github.io/charts \
  --version 41.4.0 \
  --namespace kcna-lab13-ingress \
  --skip-crds \
  -f data/traefik-lab-values.yaml \
  --wait --timeout 5m
```

Expected output (the `LAST DEPLOYED` timestamp will differ):

```
NAME: meridian-edge
LAST DEPLOYED: ...
NAMESPACE: kcna-lab13-ingress
STATUS: deployed
REVISION: 1
```

`--skip-crds` matters: the chart ships `traefik.io` and `hub.traefik.io` CRDs that this lab has no use for. Combined with `providers.kubernetesCRD: false` in the values file, the install adds **no CustomResourceDefinitions at all**.

<details> <summary>No Helm? Use the vendored manifest instead</summary>

`data/traefik-41.4.0-rendered.yaml` is the byte-for-byte output of

```
helm template meridian-edge traefik \
  --repo https://traefik.github.io/charts --version 41.4.0 \
  --namespace kcna-lab13-ingress --skip-crds --kube-version 1.31.0 \
  -f data/traefik-lab-values.yaml
```

so it is the same six objects, same pinned image `docker.io/traefik:v3.7.12`:

```
kubectl apply -f data/traefik-41.4.0-rendered.yaml
```

Expected output:

```
serviceaccount/meridian-edge-traefik created
clusterrole.rbac.authorization.k8s.io/meridian-edge-traefik-kcna-lab13-ingress created
clusterrolebinding.rbac.authorization.k8s.io/meridian-edge-traefik-kcna-lab13-ingress created
service/meridian-edge-traefik created
deployment.apps/meridian-edge-traefik created
ingressclass.networking.k8s.io/meridian-edge created
```

It also passes static validation with no CRD schemas required:

```
kubeconform -strict -summary -kubernetes-version 1.31.0 data/traefik-41.4.0-rendered.yaml
Summary: 6 resources found in 1 file - Valid: 6, Invalid: 0, Errors: 0, Skipped: 0
```

Uninstall with `kubectl delete -f data/traefik-41.4.0-rendered.yaml` instead of `helm uninstall`.

The chart is Apache-2.0 licensed; its LICENSE is vendored as `data/TRAEFIK-CHART-LICENSE.txt`.
</details>

C4 — Readiness: three checks, in order

C4.1 — the Deployment rolled out.

```
kubectl rollout status deployment/meridian-edge-traefik -n kcna-lab13-ingress --
timeout=180s
```

Expected output:

```
deployment "meridian-edge-traefik" successfully rolled out
```

C4.2 — the controller Pod is Ready, running the pinned image.

```
kubectl wait --for=condition=Ready pod \
  -l app.kubernetes.io/name=traefik -n kcna-lab13-ingress --timeout=180s
kubectl get pods -n kcna-lab13-ingress
kubectl get deployment meridian-edge-traefik -n kcna-lab13-ingress \
  -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
```

Expected output (Pod name suffix and age will differ):

```
pod/meridian-edge-traefik-... condition met
NAME                                READY    STATUS    RESTARTS    AGE
meridian-edge-traefik-...          1/1     Running   0           40s
docker.io/traefik:v3.7.12
```

C4.3 — the IngressClass now exists, and your forward reference resolves.

```
kubectl get ingressclass
```

Expected output:

NAME	CONTROLLER	PARAMETERS	AGE
meridian-edge	traefik.io/ingress-controller	<none>	45s

That is the object Step 8 proved was missing. `spec.controller` is `traefik.io/ingress-controller` — Traefik's own opaque identifier — and the name `meridian-edge` is exactly what `manifests/40-ingress-shop.yaml` has been referencing all along.

Confirm it is **not** the cluster default, which is what keeps section 6.2's class-less Ingress teaching intact:

```
kubectl get ingressclass meridian-edge \
  -o jsonpath='{.metadata.annotations.ingressclass\.kubernetes\.io/is-default-class}' ;
echo
```

Expected output:

```
false
```

C4.4 — and now the point of the whole appendix.

```
kubectl get ingress -n kcna-lab13
```

Expected output — **ADDRESS** is still blank:

NAME	ADDRESS	PORTS	CLASS	AGE	HOSTS		
meridian-shop			meridian-edge				
shop.meridianfreight.internal,					tracking.meridianfreight.internal	80	20m
meridian-shop-tls			meridian-edge		secure.meridianfreight.internal		
80, 443		16m					

A controller is installed, running and about to route your traffic correctly — and ADDRESS is blank, because its Service is a ClusterIP and publishedService is disabled, so there is no external address to publish. **This is the concrete counter-example to "blank ADDRESS means no controller."** Do not let anyone tell you otherwise, including an earlier draft of this lab.

C4.4 — Configure this controller's fallback route

Traefik **v3.7.12** creates its default router only for an Ingress whose `spec.rules` is empty. The `defaultBackend` declared alongside the rules in `meridian-shop` does not activate that router in this implementation. Without the additional object below, unmatched requests return Traefik's 404 page not found.

Apply the supplied standalone fallback Ingress before running the routing checks:

```
kubectl apply -f manifests/75-ingress-traefik-fallback.yaml
kubectl get ingress meridian-fallback -n kcna-lab13
```

Expected result: `ingress.networking.k8s.io/meridian-fallback` created on first application. This object has `ingressClassName: meridian-edge`, no host/path rules, and `defaultBackend.service.name: route-fallback-svc` on port 80. Its fallback router handles requests that match none of the more specific routes.

This configuration passed all seven HTTP route checks on the training test cluster. The implementation condition is visible in the [version-pinned Traefik provider source](https://github.com/traefik/traefik/blob/v3.7.12/pkg/provider/kubernetes/ingress/kubernetes.go#L261).
<https://github.com/traefik/traefik/blob/v3.7.12/pkg/provider/kubernetes/ingress/kubernetes.go#L261>

C5 — Reach the controller from the host and test every rule

Start the port-forward in a **second terminal**, or in the background:

```
kubectl port-forward -n kcna-lab13-ingress deployment/meridian-edge-traefik 18080:8000 &
PF_PID=$!
sleep 3
```

Expected output:

```
Forwarding from 127.0.0.1:18080 -> 8000
Forwarding from [::1]:18080 -> 8000
```

Why 8000: the container's `web` entrypoint listens on **8000**, and the Service maps port 80 → `targetPort: web`. Forwarding to the Deployment addresses the container port directly. The equivalent through the Service is `kubectl port-forward -n kcna-lab13-ingress svc/meridian-edge-traefik 18080:80`.

Now exercise the routing table. Every request goes to the same `127.0.0.1:18080`; only the `Host:` header and the path change — which is exactly how host-based and path-based Ingress routing works.

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog | head -5
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head><meta charset="utf-8"><title>Meridian Freight - Container Catalog</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>

curl -sS -H 'Host: shop.meridianfreight.internal'
http://127.0.0.1:18080/catalog/containers/20ft | grep '<h1>'
```

Expected output — Prefix matches the deeper path too:

```
<h1>Meridian Freight Pte Ltd</h1>
```

The element-boundary rule, now measurable:

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalogue
```

Expected output — /catalogue **does not match** Prefix: /catalog, so the defaultBackend answers:

```
no ingress rule matched - default backend :: kcna-lab13
```

If you instead get the catalog page here, your controller is doing string-prefix matching rather than the spec's path-element matching. For Traefik that means `providers.kubernetesIngress.strictPrefixMatching` is not set — it is `true` in `data/traefik-lab-values.yaml` for exactly this reason. This is a live demonstration that controller behaviour must be *verified*, not assumed.

Exact, and its intolerance of a trailing slash:

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status/
```

Expected output:

```
checkout status OK :: kcna-lab13
no ingress rule matched - default backend :: kcna-lab13
```

Host fan-out — a different host, the same Ingress object:

```
curl -sS -H 'Host: tracking.meridianfreight.internal' http://127.0.0.1:18080/ | grep
'<h1>'
```

Expected output:

```
<h1>Meridian Freight Pte Ltd</h1>
```

An unmapped host falls through to defaultBackend:

```
curl -sS -H 'Host: unmapped.meridianfreight.internal' http://127.0.0.1:18080/
```

Expected output:

```
no ingress rule matched - default backend :: kcna-lab13
```

Stop the port-forward when you are done experimenting by hand:

```
kill "$PF_PID"
```

C6 — Re-run the verifier: it switches to MODE 2 by itself

`verification/checks.sh` detects the IngressClass, sets up its own port-forward, and asserts the routes. Stop your manual port-forward first so port 18080 is free.

```
bash verification/checks.sh
```

Expected output (tail):


```

NOTE   MODE 2 (controller present): 1 IngressClass(es) found on this cluster.
PASS   DECISIVE: 1 IngressClass(es) exist, so a controller is installed
PASS   IngressClass 'meridian-edge' exists and names controller 'traefik.io/ingress-
controller', so meridian-shop is claimable
PASS   DECISIVE: 1 Ready controller Pod(s) in namespace kcna-lab13-ingress
NOTE   CORROBORATING: ADDRESS is blank. That is NOT a failure - a ClusterIP controller
reached by port-forward publishes no address and still routes.
PASS   port-forward 127.0.0.1:18080 -> kcna-lab13-ingress/meridian-edge-traefik:8000 is up
PASS   ROUTED shop.../catalog (Prefix) -> catalog-svc
PASS   ROUTED shop.../catalog/containers/20ft (Prefix, deeper path) -> catalog-svc
PASS   ROUTED shop.../catalogue does NOT match Prefix /catalog (element boundary) ->
defaultBackend
PASS   ROUTED shop.../checkout/status (Exact) -> checkout-svc
PASS   ROUTED shop.../checkout/status/ trailing slash breaks Exact -> defaultBackend
PASS   ROUTED tracking.../ (host fan-out, Prefix /) -> catalog-svc
PASS   ROUTED unmapped host -> defaultBackend route-fallback-svc
-- 25 passed, 0 failed, 0 skipped --

```

If your controller lives elsewhere or is named differently, point the script at it:

```

ING_NS=my-ns ING_DEPLOY=my-controller ING_CONTAINER_PORT=8000 LOCAL_PORT=18081 \
  bash verification/checks.sh

```

If anything the script needs is missing — `curl`, the Deployment, a free local port — it reports `SKIP` for the routing suite and still passes the declaration checks. It never converts "I could not test this" into `FAIL`.

C7 — Optional: watch the controller-dependent class behaviour for yourself

Section 6.2 said the fate of a class-less Ingress is controller-dependent. With a controller installed you can now test the claim instead of reading about it:

```

sed 's/pathType: prefix/pathType: Prefix/' manifests/60-ingress-invalid-pathtype.yaml |
kubect1 apply -f -
kubect1 get ingress meridian-shop-broken -n kcna-lab13
kubect1 port-forward -n kcna-lab13-ingress deployment/meridian-edge-traefik 18080:8000 &
PF_PID=$!
sleep 3
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog | grep
'<h1>'
kill "$PF_PID"
kubect1 delete ingress meridian-shop-broken -n kcna-lab13

```

`meridian-shop-broken` still shows `CLASS <none>`, and `meridian-edge` is **not** the default `IngressClass` — yet Traefik documents that it processes Ingresses that carry no class annotation when its own `ingressClass` option is unset. **Record what your cluster actually does.** Whichever way it goes, the lesson is the same and it is the one the exam and production both reward: *name the class explicitly and this question never arises.*

C8 — Remove the controller

```

helm uninstall meridian-edge -n kcna-lab13-ingress
kubect1 delete namespace kcna-lab13-ingress
kubect1 get ingressclass
kubect1 get clusterrole,clusterrolebinding -o name | grep meridian-edge || echo "no
meridian-edge cluster-scoped objects remain"

```

Expected output:

```

release "meridian-edge" uninstalled
namespace "kcna-lab13-ingress" deleted
No resources found
no meridian-edge cluster-scoped objects remain

```


The cluster is back to its stock state, and `bash verification/checks.sh` returns to MODE 1. Nothing this appendix created outlives it.

Lab 14 — Namespaces, ServiceAccounts and RBAC

Field	Value
Lab ID	Lab 14
Title	Namespaces, ServiceAccounts and RBAC
Day / Topic	Day 3 · Security
Duration	55 minutes
Namespace	kcna-lab14
Learning outcome	LO4 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 332
Repository path	courseware/labs/lab-14-namespaces-serviceaccounts-rbac/

Goal. Grant Meridian Freight's night-shift operators read-only access to one namespace and nothing else, then prove every allow and every deny with `kubectl auth can-i` instead of asserting it.

What you will produce:

- A namespaced Role/RoleBinding giving ops-reader get, list, watch on pods plus the separate pods/log subresource
- A read-only ClusterRole bound by a RoleBinding - reusable rules, namespaced grant, and zero ClusterRoleBindings
- A 12-row permission matrix in `data/rbac-matrix.csv` replayed through `kubectl auth can-i --as=system:serviceaccount:kcna-lab14:<sa>`

Lab 14 · Objective

By the end of this lab you will be able to:

1. State precisely what a Namespace does and does **not** isolate.
2. Create ServiceAccounts and explain that an unbound ServiceAccount has **zero** permissions.
3. Read the **projected ServiceAccount token** volume that kubelet mounts into a Pod, and disable it with `automountServiceAccountToken: false`.
4. Distinguish **Role vs ClusterRole** and **RoleBinding vs ClusterRoleBinding**, and explain the crucial third combination: a *RoleBinding* pointing at a *ClusterRole*.
5. Use `kubectl auth can-i --as=system:serviceaccount:<ns>:<sa>` as the authoritative proof tool, and drive a whole permission matrix through it.
6. Diagnose the real `Forbidden` error, the silent **dangling roleRef**, and the **immutable roleRef** rejection.

Lab 14 · Prerequisites

- A running single-node `kind` cluster where your `kubeconfig` user is `cluster-admin` (the `kind` default). You need the `impersonate` permission for `--as`.

- Working directory:

```
cd courseware/labs/lab-14-namespaces-serviceaccounts-rbac
pwd
```

Expected output:

```
/.../courseware/labs/lab-14-namespaces-serviceaccounts-rbac
```

- Confirm RBAC is the active authorization mode. If it is not, nothing in this lab will deny anything:

```
kubectl api-versions | grep rbac
```

Expected output:

```
rbac.authorization.k8s.io/v1
kubectl auth can-i create clusterrolebindings
```

Expected output (you are cluster-admin on kind):

```
yes
```

Lab 14 · Scenario

Meridian Freight has one shared cluster. The night-shift operations team must be able to **look at** the freight platform's Pods and read their logs when a booking job fails at 03:00 — and must not be able to change anything, read any credential, or see into any other team's namespace.

Separately, a CI identity called `deploy-bot` has been created in advance for a pipeline that is not built yet. It must have **no** access until someone deliberately grants it.

You will implement both, and prove each claim with a permission matrix rather than an opinion.

Lab 14 · Step-by-step procedure

Step 1 — Create the namespace and read what it actually gives you

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab14 created
kubectl get namespace kcna-lab14 --show-labels
```

Expected output:

NAME	STATUS	AGE	LABELS
kcna-lab14	Active	6s	app.kubernetes.io/part-of=meridian-freight, kcna.tertiaryinfotech.com/day=3, kcna.tertiaryinfotech.com/lab=lab-14, kubernetes.io/metadata.name=kcna-lab14, pod-security.kubernetes.io/enforce=baseline, pod-security.kubernetes.io/warn=restricted

Note `kubernetes.io/metadata.name=kcna-lab14` — the API server adds this label to every Namespace automatically. **Lab 16 depends on it**, because a NetworkPolicy `namespaceSelector` can only select on labels.

What a Namespace is and is not:

A Namespace gives you	A Namespace does not give you
A unique name scope for namespaced objects	Network isolation — by default any Pod can reach any Pod in any namespace (Lab 16)
A target for <code>RoleBinding</code> — the unit of RBAC scoping	Protection from any subject holding a <code>ClusterRoleBinding</code>
A target for <code>ResourceQuota</code> and <code>LimitRange</code>	Node, kernel, CPU or memory isolation on its own
A target for Pod Security Admission labels	Isolation of cluster-scoped objects: Nodes, PersistentVolumes, StorageClasses, ClusterRoles
A deletion unit — delete the namespace, delete its contents	Any guarantee about what a workload's <i>image</i> does

Confirm the last row of the left column — the cluster-scoped kinds that a Namespace can never contain:

```
kubectl api-resources --namespaced=false --api-group='' -o name
```

Expected output:

```
componentstatuses
namespaces
nodes
persistentvolumes
```

Step 2 — ServiceAccounts, and the fact that they grant nothing

```
kubectl apply -f manifests/10-serviceaccounts.yaml
```

Expected output:

```
serviceaccount/ops-reader created
serviceaccount/deploy-bot created
kubectl get serviceaccounts -n kcna-lab14
```

Expected output — note the third one you did not create:

```
NAME          SECRETS  AGE
default       0        55s
deploy-bot    0        4s
ops-reader    0        4s
```

Two observations:

- **default was created for you.** Every namespace gets one, and every Pod that does not name a ServiceAccount runs as it.
- **SECRETS is 0.** Before Kubernetes v1.24 each ServiceAccount had an auto-created, never-expiring Secret token. That behaviour was removed; tokens are now short-lived and projected into Pods, or minted on demand with `kubectl create token`.

Now prove the important point — a brand-new ServiceAccount can do nothing:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
```

Expected output (exit code 1):

```
no
```

The identity exists. It has no permissions. **Creating a ServiceAccount is not granting access.**

Note the subject string format, which you will type all lab:

`system:serviceaccount:<namespace>:<name>`. Every ServiceAccount is also a member of the group `system:serviceaccounts:<namespace>`.

Step 3 — Grant read-only Pod access with a namespaced Role

```
kubectl apply -f manifests/20-role-pod-reader.yaml
```

Expected output:

```
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/ops-reader-can-read-pods created
kubectl describe role pod-reader -n kcna-lab14
```

Expected output:

```
Name:      pod-reader
Labels:    app.kubernetes.io/part-of=meridian-freight
Annotations: <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  pods      []                 []              [get list watch]
  pods/log   []                 []              [get]
```

Read the rule model out loud: **apiGroups × resources × verbs**, optionally narrowed by **resourceNames**. RBAC is **purely additive** — there is no **deny** rule. A subject can do something if *any* bound rule allows it, and nothing else can take that away.

Note that **pods/log** needed **its own rule**. Granting **pods** does not grant **pods/log**, **pods/exec** or **pods/portforward**. **pods/exec** is effectively shell access to the container — never grant it casually.

Now re-run the same question as Step 2:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
```

Expected output:

```
yes
```

And the scoping claim — the same subject, a different namespace:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n default
```

Expected output:

```
no
```

The RoleBinding, not the Role, decides the namespace.

Step 4 — A ClusterRole scoped by a RoleBinding

Meridian Freight wants the same "view config" rule set in many namespaces without copying a Role into each. That is what a ClusterRole is for.

```
kubectl apply -f manifests/30-clusterrole-configmap-viewer.yaml
```

Expected output:

```
clusterrole.rbac.authorization.k8s.io/kcna-lab14-configmap-viewer created
rolebinding.rbac.authorization.k8s.io/ops-reader-can-view-configmaps created
```

This ClusterRole is the only cluster-scoped object this lab creates. It is read-only (**get**, **list**, **watch** on **configmaps**), has no wildcards, and — this is the part that matters — is bound by a **RoleBinding**, not a **ClusterRoleBinding**. No privilege is widened beyond **kcna-lab14**. Section 8 deletes it by exact name.

```
kubectl describe clusterrole kcna-lab14-configmap-viewer
```

Expected output:

```
Name:          kcna-lab14-configmap-viewer
Labels:        app.kubernetes.io/part-of=meridian-freight
               kcna.tertiaryinfotech.com/lab=lab-14
Annotations:   <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
  configmaps    []                  []              [get list watch]
```

The four combinations, which are examinable:

Role kind	Binding kind	Where the permissions apply
Role	RoleBinding	The RoleBinding's namespace only
ClusterRole	RoleBinding	The RoleBinding's namespace only — the rule set is reusable, the grant is not cluster-wide
ClusterRole	ClusterRoleBinding	Every namespace, plus cluster-scoped resources
Role	ClusterRoleBinding	Illegal. A ClusterRoleBinding's <code>roleRef.kind</code> must be <code>ClusterRole</code>

Prove row 2 — the grant did **not** leak out of the namespace:

```
kubectl auth can-i list configmaps --as=system:serviceaccount:kcna-lab14:ops-reader -n
kcna-lab14
kubectl auth can-i list configmaps --as=system:serviceaccount:kcna-lab14:ops-reader -n
default
```

Expected output:

```
yes
no
```

Confirm no ClusterRoleBinding was created anywhere that references it:

```
kubectl get clusterrolebindings -o jsonpath='{range .items[?(@.roleRef.name=="kcna-lab14-
configmap-viewer")]}{.metadata.name}{"\n"}{end}'
echo "(empty above = no cluster-wide grant exists)"
```

Expected output:

```
(empty above = no cluster-wide grant exists)
```

For contrast, inspect a **built-in** read-only ClusterRole that ships with every cluster — do not bind it, just read it:

```
kubectl describe clusterrole view | head -14
```

Expected output (abridged; the full list is long):

```
Name:          view
Labels:        kubernetes.io/bootstrapping=rbac-defaults
               rbac.authorization.k8s.io/aggregate-to-edit=true
Annotations:   rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
  bindings       []                  []              [get
list watch]
  configmaps     []                  []              [get
list watch]
  endpoints      []                  []              [get
list watch]
```

events	[]	[]	[get
list watch]			
limitranges	[]	[]	[get
list watch]			
namespaces/status	[]	[]	[get
list watch]			
namespaces	[]	[]	[get
list watch]			
persistentvolumeclaims/status	[]	[]	[get
list watch]			

view **deliberately excludes Secrets**. edit adds write verbs. admin adds RBAC management within a namespace. cluster-admin is * on * — bound to the system:masters group, which is what your kubeconfig uses on kind.

Step 5 — Drive the whole permission matrix from data/

```
cat data/rbac-matrix.csv
```

Expected output:

```
serviceaccount,verb,resource,namespace,expected,granted_by
ops-reader,get,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,list,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,watch,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,get,pods/log,kcna-lab14,yes,Role/pod-reader subresource rule
ops-reader,list,configmaps,kcna-lab14,yes,ClusterRole/kcna-lab14-configmap-viewer scoped
by a RoleBinding
ops-reader,delete,pods,kcna-lab14,no,verb not present in any bound rule
ops-reader,list,secrets,kcna-lab14,no,resource not present in any bound rule
ops-reader,list,pods,default,no,RoleBinding only grants inside kcna-lab14
ops-reader,list,configmaps,default,no,a RoleBinding to a ClusterRole still only grants in
its own namespace
deploy-bot,list,pods,kcna-lab14,no,ServiceAccount exists but has no binding at all
deploy-bot,create,deployments,kcna-lab14,no,ServiceAccount exists but has no binding at
all
default,list,pods,kcna-lab14,no,the auto-created default ServiceAccount is unprivileged
```

Spot-check three rows by hand before letting the script do all twelve:

```
kubectl auth can-i get pods/log --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-
lab14
kubectl auth can-i delete pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-
lab14
kubectl auth can-i create deployments --as=system:serviceaccount:kcna-lab14:deploy-bot -n
kcna-lab14
```

Expected output:

```
yes
no
no
```

Ask the API server to enumerate everything the subject can do — the fastest way to audit an identity:

```
kubectl auth can-i --list --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
```

Expected output (row order varies; the last block is granted to **every** authenticated identity by the built-in system:discovery and system:basic-user ClusterRoles):

Resources	Non-Resource URLs	Resource Names	Verbs
selfsubjectreviews.authentication.k8s.io	[]	[]	
[create]			
selfsubjectaccessreviews.authorization.k8s.io	[]	[]	
[create]			
selfsubjectrulesreviews.authorization.k8s.io	[]	[]	
[create]			

configmaps	[]	[]	[get
list watch]			
Pods	[]	[]	[get
list watch]			
Pods/log	[]	[]	[get]
	[/api/*]	[]	[get]
	[/api]	[]	[get]
	[/healthz]	[]	[get]
	[/livez]	[]	[get]
	[/openapi/*]	[]	[get]
	[/openapi]	[]	[get]
	[/readyz]	[]	[get]
	[/version/]	[]	[get]
	[/version]	[]	[get]

Exactly the three resource lines you granted, plus the harmless discovery URLs. No secrets. No write verbs.

Step 6 — The projected ServiceAccount token inside a Pod

```
kubectl apply -f manifests/40-pod-token-inspector.yaml
kubectl apply -f manifests/50-pod-no-token.yaml
kubectl wait --for=condition=Ready pod/token-inspector pod/no-token -n kcna-lab14 --
timeout=90s
```

Expected output:

```
pod/token-inspector created
pod/no-token created
pod/token-inspector condition met
pod/no-token condition met
```

Look at what kubelet mounted:

```
kubectl exec -n kcna-lab14 token-inspector -- ls -l
/var/run/secrets/kubernetes.io/serviceaccount/
```

Expected output (timestamps differ):

```
total 0
lrwxrwxrwx 1 root root 13 Sep 5 10:02 ca.crt -> ../data/ca.crt
lrwxrwxrwx 1 root root 16 Sep 5 10:02 namespace -> ../data/namespace
lrwxrwxrwx 1 root root 12 Sep 5 10:02 token -> ../data/token
```

Three files, and the symlink-to-../data pattern that lets kubelet swap the contents atomically when the token is rotated.

```
kubectl exec -n kcna-lab14 token-inspector -- cat
/var/run/secrets/kubernetes.io/serviceaccount/namespace
```

Expected output (no trailing newline):

```
kcna-lab14
kubectl exec -n kcna-lab14 token-inspector -- sh -c 'wc -c <
/var/run/secrets/kubernetes.io/serviceaccount/token'
```

Expected output (a JWT; the exact length varies):

```
1012
```

Now the part learners rarely see — where that mount came from. You never wrote a volume:

```
kubectl get pod token-inspector -n kcna-lab14 -o jsonpath='{.spec.volumes[0]}' ; echo
```

Expected output (the volume name suffix is random):

```
{ "name": "kube-api-access-7v2qd", "projected": { "defaultMode": 420, "sources":
[ { "serviceAccountToken": { "expirationSeconds": 3607, "path": "token" } }, { "configMap": { "items":
```



```
[{"key":"ca.crt","path":"ca.crt"}], "name":"kube-root-ca.crt"}}, {"downwardAPI":{"items": [{"fieldRef": {"apiVersion":"v1", "fieldPath":"metadata.namespace"}, "path":"namespace"}]}]}
```

Read the three sources:

Source	Produces	Note
serviceAccountToken	token	expirationSeconds: 3607 — roughly one hour. kubelet rotates it and rewrites the file. The token is also audience-bound and bound to this Pod's UID , so it stops working the moment the Pod is deleted
configMap: kube-root-ca.crt	ca.crt	The cluster CA, so the workload can verify the API server's certificate
downwardAPI	namespace	The Pod's own namespace

This is *BoundServiceAccountTokenVolume*: short-lived, audience- and object-bound credentials replacing the old forever-valid Secret tokens.

Now the contrast:

```
kubectl exec -n kcna-lab14 no-token -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Expected output (exit code 1 — the directory does not exist):

```
ls: /var/run/secrets/kubernetes.io/serviceaccount/: No such file or directory
command terminated with exit code 1
kubectl get pod no-token -n kcna-lab14 -o jsonpath='{.spec.volumes}' ; echo
```

Expected output:

No volumes at all. **automountServiceAccountToken: false is free defence in depth** for every workload that does not call the Kubernetes API — which is most of them. You can also set it on the ServiceAccount to make it the default for all Pods using it.

Finally, mint a token by hand — the supported replacement for the removed auto-generated Secret:

```
kubectl create token ops-reader -n kcna-lab14 --duration=10m | cut -c1-45
```

Expected output (a different token every time):

```
eyJhbGciOiJSUzI1NiIsImtpZCI6IiFrTndFcXBGYlZo
```

A JWT is three base64url segments separated by dots — header, payload, signature. The payload carries **aud**, **exp**, and a **kubernetes.io** claim naming the namespace, ServiceAccount and (for projected tokens) the Pod. It is **signed, not encrypted** — anyone holding it can read those claims, and anyone holding it can *use* it. Treat a token like a password.

Lab 14 · Verification

```
bash verification/checks.sh
```

Expected output:

```
== Lab 14 verification: Namespaces, ServiceAccounts and RBAC ==
PASS namespace kcna-lab14 exists
PASS namespace enforces the baseline Pod Security profile
```

```

PASS ServiceAccounts ops-reader and deploy-bot exist
PASS Role pod-reader is read-only (no create/update/patch/delete verbs)
PASS RoleBinding ops-reader-can-read-pods binds Role/pod-reader to ops-reader
PASS ClusterRole kcna-lab14-configmap-viewer is read-only with no wildcards
PASS NO ClusterRoleBinding references kcna-lab14-configmap-viewer (grant stays
namespaced)
PASS can-i get pods as ops-reader in kcna-lab14 = yes
PASS can-i list pods as ops-reader in kcna-lab14 = yes
PASS can-i watch pods as ops-reader in kcna-lab14 = yes
PASS can-i get pods/log as ops-reader in kcna-lab14 = yes
PASS can-i list configmaps as ops-reader in kcna-lab14 = yes
PASS can-i delete pods as ops-reader in kcna-lab14 = no
PASS can-i list secrets as ops-reader in kcna-lab14 = no
PASS can-i list pods as ops-reader in default = no
PASS can-i list configmaps as ops-reader in default = no
PASS can-i list pods as deploy-bot in kcna-lab14 = no
PASS can-i create deployments as deploy-bot in kcna-lab14 = no
PASS can-i list pods as default in kcna-lab14 = no
PASS token-inspector has a projected serviceAccountToken volume
PASS token-inspector namespace file contains kcna-lab14
PASS no-token Pod has no ServiceAccount volume mounted
-- 22 passed, 0 failed --

```

Lab 14 · Failure injection

6.1 The real Forbidden — a subject missing a verb or a resource

```
kubectl get secrets -n kcna-lab14 --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output (exit code 1):

```
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:kcna-lab14:ops-reader" cannot list resource "secrets" in API group "" in the namespace "kcna-lab14"
```

Parse the message field by field — it tells you everything you need:

Fragment	Meaning
User "system:serviceaccount:kcna-lab14:ops-reader"	The subject the request was evaluated as
cannot list	The verb . <code>list</code> and <code>get</code> are different verbs; granting <code>get</code> alone does not allow <code>kubectl get pods</code> without a name
resource "secrets"	The resource , plural, as it appears in a rule
in API group ""	The apiGroup . "" is the core group. A Deployment would say in API group "apps"
in the namespace "kcna-lab14"	The scope . If a request against a cluster-scoped resource fails, this clause is absent

Those five fields map exactly onto the five fields of an RBAC rule. Now the same subject, same verb, same resource, different namespace:

```
kubectl get pods -n default --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output:

```
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kcna-lab14:ops-reader" cannot list resource "pods" in API group "" in the namespace "default"
```

Compare with the same command inside the granted namespace:

```
kubectl get pods -n kcna-lab14 --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
no-token	1/1	Running	0	3m
token-inspector	1/1	Running	0	3m

6.2 The silent failure — a dangling roleRef

```
kubectl apply -f manifests/60-rolebinding-dangling.yaml
```

Expected output — **created without complaint**:

```
rolebinding.rbac.authorization.k8s.io/deploy-bot-dangling created
kubectl auth can-i create deployments --as=system:serviceaccount:kcna-lab14:deploy-bot -n
kcna-lab14
```

Expected output:

```
no
```

The RoleBinding exists. The permission does not. Diagnose it:

```
kubectl describe rolebinding deploy-bot-dangling -n kcna-lab14
```

Expected output:

```
Name:          deploy-bot-dangling
Labels:        kcna.tertiaryinfotech.com/intent=failure-injection
Annotations:   <none>
Role:
  Kind:  Role
  Name:  deployment-manager
Subjects:
  Kind      Name          Namespace
  ----      -
  ServiceAccount  deploy-bot  kcna-lab14
```

Nothing here says the Role is missing. Ask directly:

```
kubectl get role deployment-manager -n kcna-lab14
```

Expected output:

```
Error from server (NotFound): roles.rbac.authorization.k8s.io "deployment-manager" not
found
```

Diagnosis: `roleRef` is not validated on write. RBAC treats a binding to a non-existent Role as granting nothing, and emits no error, warning or event. The triage routine for "I made a RoleBinding and it still says Forbidden":

```
kubectl get rolebindings -n kcna-lab14 -o custom-
columns='NAME:.metadata.name,ROLEKIND:.roleRef.kind,ROLE:.roleRef.name,SUBJECT:.subjects[0
].name'
```

Expected output:

NAME	ROLEKIND	ROLE	SUBJECT
deploy-bot-dangling	Role	deployment-manager	deploy-bot
ops-reader-can-read-pods	Role	pod-reader	ops-reader
ops-reader-can-view-configmaps	ClusterRole	kcna-lab14-configmap-viewer	ops-reader

Then confirm each `ROLE` actually exists:

```
kubectl get roles -n kcna-lab14 -o name
```

Expected output:

```
role.rbac.authorization.k8s.io/pod-reader
```

`deployment-manager` is absent — that is the fault. The three other classic causes of the same symptom, in the order worth checking:

1. `subjects[].namespace` omitted for a `ServiceAccount` subject (it is **required**; it does not default to the `RoleBinding`'s namespace).
2. `subjects[].name` misspelled — RBAC does not verify subjects exist either.
3. The `RoleBinding` is in the wrong namespace.

Remove the broken binding:

```
kubectl delete rolebinding deploy-bot-dangling -n kcna-lab14
```

Expected output:

```
rolebinding.rbac.authorization.k8s.io "deploy-bot-dangling" deleted
```

6.3 The immutable `roleRef`

Try to repoint an existing binding at a different Role:

```
kubectl patch rolebinding ops-reader-can-read-pods -n kcna-lab14 --type=merge \
-p '{"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"Role","name":"pod-reader-
v2"}}'
```

Expected output (rejected):

```
The RoleBinding "ops-reader-can-read-pods" is invalid: roleRef: Invalid value:
rbac.RoleRef{APIGroup:"rbac.authorization.k8s.io", Kind:"Role", Name:"pod-reader-v2"}:
cannot change roleRef
```

Diagnosis: `roleRef` is immutable by design. Silently repointing a binding is a privilege-escalation vector, so the API server forces you to delete and recreate, which is auditable. `subjects` **can** be edited freely.

Confirm the original binding is untouched:

```
kubectl get rolebinding ops-reader-can-read-pods -n kcna-lab14 -o
jsonpath='{.roleRef.name}'
```

Expected output:

```
pod-reader
```

Lab 14 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
Error from server (Forbidden): ... cannot list resource "secrets" in API group ""	No bound rule covers that verb+resource+group	<code>kubectl auth can-i --list --as=<subject> -n <ns></code>	Add the resource/verb to a Role, or bind an existing one. Never reach for <code>cluster-admin</code>
RoleBinding created but access still denied, no error anywhere	<code>roleRef</code> names a Role that does not exist	<code>kubectl get role <roleRef.name> -n <ns></code> → <code>NotFound</code>	Create the Role, or delete and recreate the binding pointing at the right one
Same, but the Role exists	<code>subjects[].namespace</code> missing on a <code>ServiceAccount</code> subject, or the name is misspelled	<code>kubectl get rolebinding <n> -o jsonpath='{.subjects}'</code>	Add <code>namespace;</code> RBAC does not validate subjects
cannot change <code>roleRef</code>	<code>roleRef</code> is immutable	Read the error	<code>kubectl delete rolebinding <n> -n <ns></code> then recreate

Symptom	Likely cause	Command that confirms it	Fix
kubectl get pods fails but kubectl get pod <name> works	list and get are separate verbs	kubectl auth can-i list pods --as=<subject> → no	Add list (and watch for -w and for informers)
kubectl logs fails though pods is granted	pods/log is a separate subresource	kubectl auth can-i get pods/log --as=<subject>	Add a rule for pods/log
Deployment access denied though the rule says deployments	Wrong apiGroups — Deployments are in apps, not ""	The Forbidden message prints in API group "apps"	Use apiGroups: ["apps"]
error: You must be logged in to the server from --as=...	Your own user lacks impersonate	kubectl auth can-i impersonate serviceaccounts	Run as cluster-admin, or test from inside a Pod instead
Permissions appear cluster-wide when you wanted them namespaced	A ClusterRoleBinding was used instead of a RoleBinding	kubectl get clusterrolebindings -o jsonpath='...roleRef.name...'	Replace with a RoleBinding in the target namespace

Lab 14 · Cleanup

```
kubectl delete namespace kcna-lab14
```

Expected output:

```
namespace "kcna-lab14" deleted
```

That removes the ServiceAccounts, Role, both RoleBindings and both Pods.

One cluster-scoped object remains. `kcna-lab14-configmap-viewer` is a ClusterRole and therefore lives outside every namespace, so deleting the namespace cannot remove it. Delete it **by exact name** — never with a selector or `--all`:

```
kubectl delete clusterrole kcna-lab14-configmap-viewer
```

Expected output:

```
clusterrole.rbac.authorization.k8s.io "kcna-lab14-configmap-viewer" deleted
```

Confirm nothing of this lab is left cluster-wide:

```
kubectl get clusterrole kcna-lab14-configmap-viewer
kubectl get namespace kcna-lab14
```

Expected output:

```
Error from server (NotFound): clusterroles.rbac.authorization.k8s.io "kcna-lab14-configmap-viewer" not found
Error from server (NotFound): namespaces "kcna-lab14" not found
```

Never run `kubectl delete clusterrole --all` or `kubectl delete rolebinding --all`. The first would destroy the cluster's built-in RBAC and the cluster with it.

Lab 14 · What you learned

- A Namespace is a **scope**, not a security boundary. It scopes names, RBAC bindings, quotas and Pod Security labels — it does not isolate the network, the node, or a subject holding a ClusterRoleBinding.
- A ServiceAccount is an **identity with no permissions**. Creating one grants nothing.
- Permissions come from a **binding**. `Role+RoleBinding` is namespaced;
`ClusterRole+RoleBinding` is a reusable rule set applied **within one namespace**;
`ClusterRole+ClusterRoleBinding` is cluster-wide and is what you avoid by default.
- RBAC is **additive only**; there is no deny rule and no ordering.
- Subresources (`pods/log`, `pods/exec`) need their own rules, and the `apiGroup` matters ("" for Pods, `apps` for Deployments).
- kubelet projects a **short-lived, audience- and Pod-bound token** into every Pod unless you set `automountServiceAccountToken: false` — which you should, by default.
- `kubectl auth can-i --as=system:serviceaccount:<ns>:<sa>` is the proof tool, and `--list` audits an identity in one command.
- The two silent RBAC failures — a **dangling `roleRef`** and a missing `subjects[].namespace` — produce no error at all. The immutable `roleRef` is the one that *does* shout, and it does so to prevent silent escalation.

Lab 14 · Further reading

- Using RBAC Authorization — <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- ServiceAccounts — <https://kubernetes.io/docs/concepts/security/service-accounts/>
- Configure Service Accounts for Pods — <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>
- Namespaces — <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
- Authorization overview — <https://kubernetes.io/docs/reference/access-authn-authz/authorization/>
- Pod Security Admission — <https://kubernetes.io/docs/concepts/security/pod-security-admission/>
- Kubernetes API access-control good practices — <https://kubernetes.io/docs/concepts/security/rbac-good-practices/>

Lab 15 — Secrets, ConfigMaps and Safe Injection

Field	Value
Lab ID	Lab 15
Title	Secrets, ConfigMaps and Safe Injection
Day / Topic	Day 3 · Security
Duration	50 minutes
Namespace	kcna-lab15
Learning outcome	LO4 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 341
Repository path	courseware/labs/lab-15-secrets-configmaps/

Goal. Split the Meridian Freight booking-api configuration into ConfigMaps and a Secret, inject each by the safest route, and prove with your own hands that a Kubernetes Secret is base64-encoded rather than encrypted.

What you will produce:

- A booking-app-config ConfigMap built three ways - from-file, from-env-file and from-literal - kept byte-identical to data/app-settings.json and data/feature-flags.env
- A booking-credentials Secret generated from data/dummy-credentials.env and deliberately never committed as a manifest, decoded live with base64 -d
- Two consumer Pods contrasting env injection with volume injection, including a subPath mount that provably never updates

Safety notice

Every credential-shaped value in this lab is an obvious placeholder such as `DUMMY-not-a-real-password-change-me`. Nothing here authenticates to anything. **Do not substitute a real credential at any point**, including in `data/`.

Lab 15 · Objective

By the end of this lab you will be able to:

1. Create ConfigMaps three ways — `--from-literal`, `--from-file` and `--from-env-file` — and predict the resulting keys.
2. Inject configuration as **environment variables** and as **volumes**, and choose between them for a given requirement.
3. Demonstrate, with your own hands, that a **Secret is base64-encoded, not encrypted**, and name the three mitigations that actually protect it.
4. Explain `immutable: true` and the version-the-name pattern it forces.
5. Explain why a **subPath mount never receives updates** while a whole-directory mount does.

6. Diagnose `CreateContainerConfigError` from a key-name mismatch, using the real kubelet event text.

Lab 15 · Prerequisites

- Lab 14 completed — you need to understand *who* can read a Secret before you learn how one is stored.
- A running single-node `kind` cluster.
- Working directory:

```
cd courseware/labs/lab-15-secrets-configmaps
pwd
```

Expected output:

```
/.../courseware/labs/lab-15-secrets-configmaps
```

- A `base64` binary that can decode. Check which flag yours uses:

```
printf 'hello' | base64 | base64 -d
```

Expected output on Linux and on macOS (both accept `-d`; older macOS releases need `-D`):

```
hello
```

Lab 15 · Scenario

Meridian Freight's `booking-api` reads a JSON settings file, a set of feature flags, and a database credential. On the legacy VMs all three lived in `/opt/booking/.env`, world-readable, in the same Git repository as the code.

Your task is to split them correctly: **non-secret configuration into ConfigMaps, credentials into a Secret**, injected in the safest form for each. You will then show the security team exactly how much protection a Kubernetes Secret does and does not provide — because they have been told "Kubernetes encrypts secrets", and that is not true by default.

Lab 15 · Step-by-step procedure

Step 1 — Namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab15 created
```

Step 2 — Look at the source data

```
cat data/app-settings.json
```

Expected output:

```
{
  "service": "booking-api",
  "environment": "training",
  "region": "ap-southeast-1",
  "listen_port": 8080,
```



```

"log_level": "info",
"quote_cache_ttl_seconds": 300,
"downstream": {
  "quote_api": "http://quote-api.kcna-lab11.svc.cluster.local:80",
  "tracking_api": "http://tracking-api.kcna-lab15.svc.cluster.local:80"
},
"limits": {
  "max_containers_per_booking": 40,
  "max_weight_kg": 26000
}
}
cat data/feature-flags.env

```

Expected output:

```

# Consumed by: kubectl create configmap ... --from-env-file=data/feature-flags.env
# Each line becomes ONE ConfigMap key. Blank lines and # comments are ignored.
FEATURE_MULTILEG_QUOTES=true
FEATURE_LIVE_VESSEL_ETA=false
FEATURE_BULK_UPLOAD=true
FEATURE_LEGACY_EDI=false

```

Step 3 — Three ways to build a ConfigMap

Generate — do not yet apply — the ConfigMap, so you can see how each flag maps:

```

kubectl create configmap booking-app-config -n kcna-lab15 \
--from-file=app-settings.json=data/app-settings.json \
--from-env-file=data/feature-flags.env \
--from-literal=APP_GREETING="Meridian Freight booking API" \
--dry-run=client -o yaml | grep -vE '^s{4}' | head -20

```

Expected output (the `grep -v` hides the JSON body so the key list is readable):

```

apiVersion: v1
data:
  APP_GREETING: Meridian Freight booking API
  FEATURE_BULK_UPLOAD: "false"
  app-settings.json: |
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: booking-app-config
  namespace: kcna-lab15

```

The abridged view above only proves the *shape*. Run the command without the `grep/head` filters to see all seven keys.

The mapping rule for each flag:

Flag	Key becomes	Value becomes
<code>--from-file=app-settings.json=data/app-settings.json</code>	the name left of = (app-settings.json); without key=, the basename of the file	the entire file contents
<code>--from-env-file=data/feature-flags.env</code>	one key per line , the part before =	the part after =. # comments and blank lines are ignored
<code>--from-literal=APP_GREETING=...</code>	exactly what you typed	exactly what you typed

Now apply the checked-in declarative version, which is the same object:

```

kubectl apply -f manifests/10-configmap-booking-app-config.yaml

```

Expected output:

```
configmap/booking-app-config created
kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{range .data}{"{""}
{end}}' 2>/dev/null
kubectl describe configmap booking-app-config -n kcna-lab15 | head -22
```

Expected output:

```
Name:          booking-app-config
Namespace:     kcna-lab15
Labels:        app=booking-api
Annotations:   <none>
```

```
Data
====
APP_GREETING:
----
Meridian Freight booking API
```

```
FEATURE_BULK_UPLOAD:
----
true
```

```
FEATURE_LEGACY_EDI:
----
false
```

```
FEATURE_LIVE_VESSEL_ETA:
----
false
```

Look at what `describe` just did. It printed every ConfigMap value in full, in plaintext, to your terminal. Hold that thought until Step 6.

Confirm the ConfigMap value is byte-identical to the file it came from:

```
diff <(kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{.data.app-
settings\.json}') data/app-settings.json && echo "IDENTICAL"
```

Expected output:

```
IDENTICAL
```

Step 4 — Immutability

```
kubectl apply -f manifests/15-configmap-immutable.yaml
```

Expected output:

```
configmap/booking-app-config-v1 created
kubectl get configmap booking-app-config-v1 -n kcna-lab15 -o jsonpath='{.immutable}{"\n"}'
```

Expected output:

```
true
```

Try to change it:

```
kubectl patch configmap booking-app-config-v1 -n kcna-lab15 \
--type=merge -p '{"data":{"RELEASE_CHANNEL":"beta"}}'
```

Expected output (rejected):

```
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid: data:
Field is immutable
```

Why you would want this:

- A running Pod cannot be broken by someone editing shared config.
- kubelet stops watching the object, which measurably reduces API server and etcd load on large clusters.

What it costs: the only way to change the content is to delete and recreate. So you **version the name** (...-v1, ...-v2) and roll the Deployment to point at the new one — which also gives you a real rollout and a real rollback, instead of a silent in-place mutation.

Note that `immutable` itself can be set on an existing ConfigMap, but never unset:

```
kubectl patch configmap booking-app-config-v1 -n kcna-lab15 \
--type=merge -p '{"immutable":false}'
```

Expected output:

```
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid:
immutable: Field is immutable
```

Step 5 — Create the Secret from data/

Look at the fixture first. Note how loudly it announces that it is fake:

```
cat data/dummy-credentials.env | tail -3
```

Expected output:

```
BOOKING_DB_USERNAME=DUMMY-not-a-real-username
BOOKING_DB_PASSWORD=DUMMY-not-a-real-password-change-me
BOOKING_API_TOKEN=DUMMY-not-a-real-token-0000000000
```

Create the Secret **imperatively**:

```
kubectl create secret generic booking-credentials -n kcna-lab15 \
--from-env-file=data/dummy-credentials.env
```

Expected output:

```
secret/booking-credentials created
```

Notice what is *not* in `manifests/`. There is no `booking-credentials` YAML file in this repository, on purpose: **a Secret manifest must never be committed to a Git repository**, because a base64 string is not a protection (Step 6 proves it). The only Secret manifest checked in — `20-secret-declarative-example.yaml` — exists to teach the object's *shape* and contains nothing but placeholders. Apply it now so you can compare `stringData` with `data`:

```
kubectl apply -f manifests/20-secret-declarative-example.yaml
```

Expected output:

```
secret/booking-api-example created
kubectl get secrets -n kcna-lab15
```

Expected output:

NAME	TYPE	DATA	AGE
booking-api-example	Opaque	2	4s
booking-credentials	Opaque	3	30s

Step 6 — Prove that a Secret is encoded, not encrypted

This is the most important five minutes of the lab.

```
kubectl get secret booking-credentials -n kcna-lab15 -o yaml
```

Expected output:

```
apiVersion: v1
data:
  BOOKING_API_TOKEN: RFVNTVktbm90LWEtcVhbC10b2tlbi0wMDAwMDAwMDAw
  BOOKING_DB_PASSWORD: RFVNTVktbm90LWEtcVhbC1wYXNzd29yZC1jaGFuZ2UtbWU=
  BOOKING_DB_USERNAME: RFVNTVktbm90LWEtcVhbC1lc2VybmFtZQ==
kind: Secret
metadata:
  creationTimestamp: "2026-09-05T10:31:07Z"
  name: booking-credentials
  namespace: kcna-lab15
  resourceVersion: "4821"
  uid: 6a0f4a1e-6d64-4f9e-b2ab-27c0e0b1f0c2
type: Opaque
```

That looks reassuringly unreadable. It is not. Decode it:

```
kubectl get secret booking-credentials -n kcna-lab15 \
-o jsonpath='{.data.BOOKING_DB_PASSWORD}' | base64 -d ; echo
```

Expected output:

```
DUMMY-not-a-real-password-change-me
```

Decode every key at once:

```
kubectl get secret booking-credentials -n kcna-lab15 -o json \
| grep -E '"BOOKING_' \
| sed 's/[,]/g' \
| while read -r k v; do printf '%s %s\n' "$k" "$(printf '%s' "$v" | base64 -d)"; done
```

Expected output:

```
BOOKING_API_TOKEN: DUMMY-not-a-real-token-0000000000
BOOKING_DB_PASSWORD: DUMMY-not-a-real-password-change-me
BOOKING_DB_USERNAME: DUMMY-not-a-real-username
```

And confirm that `stringData` is not stored as you typed it:

```
kubectl get secret booking-api-example -n kcna-lab15 -o
jsonpath='{.data.EXAMPLE_PASSWORD}' ; echo
kubectl get secret booking-api-example -n kcna-lab15 -o jsonpath='{.stringData}' ; echo
"<- stringData is write-only"
```

Expected output:

```
RFVNTVktbm90LWEtcVhbC1wYXNzd29yZC1jaGFuZ2UtbWU=
<- stringData is write-only
```

`stringData` is a **write-only convenience field**. The API server base64-encodes it into `data` and it never comes back on a read.

State the conclusion precisely

Base64 is an encoding, not a cipher. It has no key. Anyone — or any process — that can get the Secret object has the plaintext. By default the value is also written to **etcd in plaintext**.

Contrast this with the *one* thing Kubernetes does do for you:

```
kubectl describe secret booking-credentials -n kcna-lab15
```

Expected output:

```
Name:          booking-credentials
Namespace:     kcna-lab15
Labels:        <none>
Annotations:   <none>

Type: Opaque

Data
====
BOOKING_API_TOKEN:    33 bytes
BOOKING_DB_PASSWORD: 35 bytes
BOOKING_DB_USERNAME: 25 bytes
```

`describe` masks the values (it shows byte counts) — unlike `describe configmap` in Step 3, which printed everything. That is a courtesy against shoulder-surfing, not a security control: `kubectl get -o yaml` bypasses it entirely.

The mitigations that actually work

Mitigation	What it addresses	How
RBAC on secrets	Who can read the object at all	Lab 14. Note that the built-in <code>view</code> ClusterRole deliberately excludes Secrets. Grant <code>get secrets</code> with <code>resourceNames</code> where you can
Encryption at rest	The plaintext sitting in etcd	<code>--encryption-provider-config</code> on <code>kube-apiserver</code> , with an <code>aescbc/aesgcm</code> provider or, better, a <code>kms v2</code> provider backed by an external KMS
An external secret store	Removing the secret from Kubernetes entirely	HashiCorp Vault, cloud secret managers, the External Secrets Operator, or the Secrets Store CSI Driver — the material is fetched at mount time and never persisted in etcd
<code>automountServiceAccountToken: false</code>	Stopping a compromised Pod from reading <i>other</i> Secrets via the API	Lab 14, Step 6
Prefer volumes over env	Leakage through <code>/proc/<pid>/environ</code> , child processes and crash dumps	Step 8 below

Check whether **this** cluster encrypts at rest — the honest answer on kind:

```
kubectl get pod -n kube-system -l component=kube-apiserver \
-o jsonpath='{.items[0].spec.containers[0].command}' \
| tr ',' '\n' | grep -i 'encryption-provider' \
|| echo "NOT SET -> Secrets are stored in etcd in plaintext on this cluster"
```

Expected output on a stock kind cluster:

```
NOT SET -> Secrets are stored in etcd in plaintext on this cluster
```

That is the true state of the training cluster, and of a great many production clusters. Say so to your security team rather than repeating the myth.

Step 7 — Inject as environment variables

```
kubectl apply -f manifests/30-pod-env-injection.yaml
kubectl wait --for=condition=Ready pod/env-consumer -n kcna-lab15 --timeout=90s
```

Expected output:

```
pod/env-consumer created
pod/env-consumer condition met

kubectl exec -n kcna-lab15 env-consumer -- sh -c 'env | grep -E "^(GREETING|
MULTILEG_ENABLED|DB_USERNAME|OPTIONAL_TUNING|CFG_)" | sort'
```

Expected output:

```
CFG_APP_GREETING=Meridian Freight booking API
CFG_FEATURE_BULK_UPLOAD=true
CFG_FEATURE_LEGACY EDI=false
CFG_FEATURE_LIVE_VESSEL_ETA=false
CFG_FEATURE_MULTILEG_QUOTES=true
DB_USERNAME=DUMMY-not-a-real-username
GREETING=Meridian Freight booking API
MULTILEG_ENABLED=true
```

Four things to read out of that list:

1. **OPTIONAL_TUNING is absent.** Its `configMapKeyRef set optional: true` and the key does not exist, so the variable was simply not created — no error. Compare with section 6, where the same situation without `optional` kills the container.
2. **CFG_app-settings.json is absent.** `envFrom` skips keys that are not valid shell identifiers. kubelet records a warning event instead of failing:

```
kubectl get events -n kcna-lab15 --field-selector involvedObject.name=env-consumer
```

Expected output (an `InvalidEnvironmentVariableNames` warning; wording may vary slightly by release):

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
39s	Normal	Scheduled	pod/env-consumer	Successfully assigned kcna-lab15/env-consumer to kind-control-plane
38s	Warning	InvalidEnvironmentVariableNames	pod/env-consumer	Keys [app-settings.json] from the EnvFrom configMap kcna-lab15/booking-app-config were skipped since they are considered invalid environment variable names.
38s	Normal	Pulled	pod/env-consumer	Container image "busybox:1.36" already present on machine
38s	Normal	Created	pod/env-consumer	Created container: shell
38s	Normal	Started	pod/env-consumer	Started container shell

1. **prefix:** `CFG_` namespaces the bulk import so it cannot collide with the application's own variables.
2. **DB_USERNAME holds the decoded secret value.** By the time it reaches the process it is plaintext — there is no such thing as an "encrypted environment variable".

Now see how leaky an environment variable is:

```
kubectl exec -n kcna-lab15 env-consumer -- sh -c 'tr "\0" "\n" < /proc/1/environ | grep DB_USERNAME'
```

Expected output:

```
DB_USERNAME=DUMMY-not-a-real-username
```

Anything that can read `/proc` inside that container — a sidecar sharing the process namespace, a debugging tool, a crash handler writing a core dump — can read every credential you injected as an environment variable.

Step 8 — Inject as volumes, and meet the `subPath` trap

```
kubectl apply -f manifests/40-pod-volume-injection.yaml
kubectl wait --for=condition=Ready pod/volume-consumer -n kcna-lab15 --timeout=90s
```

Expected output:

```
pod/volume-consumer created
pod/volume-consumer condition met

kubectl exec -n kcna-lab15 volume-consumer -- ls -l /etc/booking/config
/etc/booking/credentials
```

Expected output (`..data` symlinks again — the atomic-update mechanism):

```
/etc/booking/config:
total 0
lrwxrwxrwx    1 root    root          25 Sep  5 10:40 app-settings.json -> ..data/app-
settings.json
lrwxrwxrwx    1 root    root          19 Sep  5 10:40 greeting.txt ->
..data/greeting.txt

/etc/booking/credentials:
total 0
lrwxrwxrwx    1 root    root          24 Sep  5 10:40 BOOKING_API_TOKEN ->
..data/BOOKING_API_TOKEN
lrwxrwxrwx    1 root    root          26 Sep  5 10:40 BOOKING_DB_PASSWORD ->
..data/BOOKING_DB_PASSWORD
lrwxrwxrwx    1 root    root          26 Sep  5 10:40 BOOKING_DB_USERNAME ->
..data/BOOKING_DB_USERNAME
```

Note that `items`: projected **only two** of the ConfigMap's seven keys, and renamed `APP_GREETING` to `greeting.txt`. Without `items`, every key becomes a file.

Check the file permissions the `fsGroup` made readable:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'ls -lL
/etc/booking/credentials/BOOKING_DB_PASSWORD; id'
```

Expected output:

```
-r--r-----    1 root    1000          35 Sep  5 10:40
/etc/booking/credentials/BOOKING_DB_PASSWORD
uid=1000 gid=1000 groups=1000
```

Mode 0440, owner root, **group 1000** — set by `fsGroup`: 1000. Without `fsGroup`, that file would be `root:root` and this non-root container could not read it. That is one of the most common "I hardened my Pod and it broke" incidents.

Read the mounted secret:

```
kubectl exec -n kcna-lab15 volume-consumer -- cat
/etc/booking/credentials/BOOKING_DB_PASSWORD ; echo
```

Expected output:

```
DUMMY-not-a-real-password-change-me
```

Confirm the Secret volume is `tmpfs` — never written to the node's disk:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'mount | grep booking/credentials'
```

Expected output (the source device name varies):

```
tmpfs on /etc/booking/credentials type tmpfs (ro,relatime,size=...)
```

The subPath demonstration

Both mounts below come from the **same** ConfigMap. They are currently identical:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'grep log_level  
/etc/booking/config/app-settings.json /etc/booking/pinned-settings.json'
```

Expected output:

```
/etc/booking/config/app-settings.json: "log_level": "info",  
/etc/booking/pinned-settings.json: "log_level": "info",
```

Change the ConfigMap:

```
kubectl patch configmap booking-app-config -n kcna-lab15 --type=merge \  
-p '{"data":{"app-settings.json":{"\n  \"service\": \"booking-api\", \n  \"log_level\":  
\"debug\"\n}\n}}'
```

Expected output:

```
configmap/booking-app-config patched
```

Now wait for kubelet's sync. **This takes up to about 90 seconds** — that is normal, not a hang:

```
for i in $(seq 1 20); do  
  if kubectl exec -n kcna-lab15 volume-consumer -- grep -q debug /etc/booking/config/app-  
settings.json 2>/dev/null; then  
    echo "directory mount updated after ~$(i * 5)s"; break  
  fi  
  sleep 5  
done
```

Expected output (the exact number of seconds varies with the kubelet sync period):

```
directory mount updated after ~65s
```

Now compare the two mounts again:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'grep log_level  
/etc/booking/config/app-settings.json /etc/booking/pinned-settings.json'
```

Expected output:

```
/etc/booking/config/app-settings.json: "log_level": "debug",  
/etc/booking/pinned-settings.json: "log_level": "info",
```

There is the trap. The directory mount updated. The `subPath` mount did not, and never will for the life of the container.

Why: a whole-directory ConfigMap mount is managed by kubelet as an atomically swapped symlink tree, so refreshing it is a symlink flip. A `subPath` mount is a bind mount of one file made at container-creation time; kubelet has no way to re-point it without restarting the container.

And note what did *not* happen anywhere: no environment variable changed. `env-consumer` still reports the old values, because environment variables are frozen at process start. If your application must pick up config changes without a restart, it must read a **whole-directory volume mount** and re-read the file.

Restore the ConfigMap before verification:

```
kubectl apply -f manifests/10-configmap-booking-app-config.yaml
```

Expected output:

```
configmap/booking-app-config configured
```


Lab 15 · Verification

```
bash verification/checks.sh
```

Expected output:

```
== Lab 15 verification: Secrets, ConfigMaps and safe injection ==
PASS namespace kcna-lab15 exists
PASS ConfigMap booking-app-config app-settings.json matches data/app-settings.json byte-for-byte
PASS ConfigMap booking-app-config has all 4 keys from data/feature-flags.env with matching values
PASS ConfigMap booking-app-config-v1 is immutable
PASS Secret booking-credentials has all 3 keys from data/dummy-credentials.env
PASS Secret booking-credentials values base64-decode to the data/dummy-credentials.env values
PASS Secret booking-credentials contains only obvious DUMMY placeholders
PASS Secret booking-api-example stores stringData as base64 under .data and returns no stringData
PASS env-consumer resolved GREETING from a configMapKeyRef
PASS env-consumer resolved DB_USERNAME from a secretKeyRef
PASS env-consumer bulk-imported the FEATURE_* keys with the CFG_ prefix
PASS env-consumer omitted the optional key that does not exist
PASS volume-consumer projected only the 2 requested ConfigMap keys, with greeting.txt renamed
PASS volume-consumer secret file mode is 0440 with group 1000 (fsGroup)
PASS volume-consumer secret volume is tmpfs (never written to node disk)
PASS volume-consumer subPath mount exists alongside the directory mount
PASS RECORDED: this cluster has NO --encryption-provider-config (Secrets are plaintext in etcd)
-- 17 passed, 0 failed --
```

Lab 15 · Failure injection — a key name that does not exist

6.1 Break it

```
kubectl apply -f manifests/50-pod-broken-key.yaml
```

Expected output — **accepted**, because nothing validates key names at admission:

```
pod/broken-key-consumer created
sleep 10; kubectl get pod broken-key-consumer -n kcna-lab15
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
broken-key-consumer	0/1	CreateContainerConfigError	0	12s

CreateContainerConfigError is specific and diagnostic: the image pulled fine, the Pod was scheduled fine, and kubelet then failed while **assembling the container's configuration**.

6.2 Get the real error text

```
kubectl describe pod broken-key-consumer -n kcna-lab15 | tail -14
```

Expected output:

```
Events:
  Type     Reason      Age    From                      Message
  ----     -
  Normal   Scheduled   25s    default-scheduler        Successfully assigned kcna-lab15/broken-key-consumer to kind-control-plane
```

Normal	Pulled	10s (x3 over 24s)	kubelet	Container image "busybox:1.36"
Warning	Failed	10s (x3 over 24s)	kubelet	Error: couldn't find key APP_TIER in ConfigMap kcna-lab15/booking-app-config

The one line that matters:

```
Error: couldn't find key APP_TIER in ConfigMap kcna-lab15/booking-app-config
```

It names the **key**, the **namespace** and the **object**. The same shape appears for a missing Secret key (couldn't find key ... in Secret ...) and, if the object itself is absent, configmap "..." not found.

6.3 Diagnose

List the keys that actually exist:

```
kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{range .data}{"\n"}{end}'
kubectl get configmap booking-app-config -n kcna-lab15 -o go-template='{{range $k, $v := .data}}{{ $k }}{\n"}{{end}}' | sort
```

Expected output:

```
APP_GREETING
FEATURE_BULK_UPLOAD
FEATURE_LEGACY_EDI
FEATURE_LIVE_VESSEL_ETA
FEATURE_MULTILEG_QUOTES
app-settings.json
```

There is no APP_TIER. Compare with what the Pod asked for:

```
kubectl get pod broken-key-consumer -n kcna-lab15 \
-o jsonpath='{range .spec.containers[0].env[*]}{"\n"}{end}'
```

Expected output:

```
APP_TIER <- booking-app-config/APP_TIER
```

Diagnosis: the reference is to a key that was never defined. Nothing catches this until kubelet tries to build the container — not kubectl apply, not admission, not kubeconform:

```
kubeconform -strict -summary manifests/50-pod-broken-key.yaml
```

Expected output:

```
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

6.4 Fix — two valid choices

Choice A — add the key (the config really was missing):

```
kubectl patch configmap booking-app-config -n kcna-lab15 \
--type=merge -p '{"data":{"APP_TIER":"training"}}'
kubectl delete pod broken-key-consumer -n kcna-lab15
kubectl apply -f manifests/50-pod-broken-key.yaml
kubectl wait --for=condition=Ready pod/broken-key-consumer -n kcna-lab15 --timeout=90s
kubectl exec -n kcna-lab15 broken-key-consumer -- printenv APP_TIER
```

Expected output:

```
configmap/booking-app-config patched
pod "broken-key-consumer" deleted
pod/broken-key-consumer created
pod/broken-key-consumer condition met
training
```

The Pod had to be **recreated**. A Pod's `env` is immutable; patching the ConfigMap alone would have left it stuck in `CreateContainerConfigError` forever.

Choice B — mark the reference **optional** (the config is genuinely not always present): add `optional: true` under the `configMapKeyRef`, exactly as `OPTIONAL_TUNING` does in `manifests/30-pod-env-injection.yaml`. The container then starts with the variable simply unset.

Restore the ConfigMap and remove the failure-injection Pod before cleanup:

```
kubectl delete pod broken-key-consumer -n kcna-lab15
kubectl apply -f manifests/10-configmap-booking-app-config.yaml
```

Expected output:

```
pod "broken-key-consumer" deleted
configmap/booking-app-config configured
```

Lab 15 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
CreateContainerConfigError	configMapKeyRef/secretKeyRef names a key that does not exist	<code>kubectl describe pod <p> → couldn't find key <K> in ConfigMap <ns>/<name></code>	Add the key, or set <code>optional: true</code> , then recreate the Pod
CreateContainerConfigError with configmap "x" not found	The whole object is missing, or is in another namespace	<code>kubectl get cm -n <ns></code>	Create it in the same namespace — ConfigMaps and Secrets never cross namespaces
Field is immutable on patch	The ConfigMap/Secret has <code>immutable: true</code>	<code>kubectl get cm <n> -o jsonpath='{.immutable}'</code>	Create a new versioned object (<code>...v2</code>) and roll the workload
A ConfigMap change is not visible in the container	Injected as <code>env</code> (frozen at start), or mounted with <code>subPath</code>	<code>kubectl get pod <p> -o yaml grep -A2 subPath</code>	Use a whole-directory volume mount and re-read the file, or roll the Deployment
Config change visible in the container but the app ignores it	The application caches the file at startup	Read the app's own logs	Roll the Deployment; volume updates do not restart processes
Permission denied reading a mounted Secret as a non-root user	Files are <code>root:root</code> ; <code>defaultMode</code> excludes <code>other</code>	<code>kubectl exec <p> -- ls -lL <path>; id</code>	Set <code>securityContext.fsGroup</code> to the container's GID, or widen <code>defaultMode</code>
Key from <code>envFrom</code> never appears	Not a valid shell identifier (contains <code>.</code> , <code>-</code> , or starts with a digit)	<code>kubectl get events → InvalidEnvironmentVariableNames</code>	Use an explicit <code>configMapKeyRef</code> with a valid <code>name:</code> , or a volume mount
Secret value looks like gibberish in <code>-o yaml</code>	It is base64, not encryption	<code>... -o jsonpath='{.data.<K>}' base64 -d</code>	Nothing to fix — but stop treating base64 as protection
<code>base64: invalid option -- 'd'</code>	Older macOS <code>base64</code>	<code>base64 --help</code>	Use <code>base64 -D</code>

Lab 15 · Cleanup

Delete **only** this lab's namespace. ConfigMaps, Secrets and Pods are all namespaced.

```
kubectl delete namespace kcna-lab15
```

Expected output:

```
namespace "kcna-lab15" deleted
```

Confirm:

```
kubectl get namespace kcna-lab15
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab15" not found
```

This lab creates **no cluster-scoped objects**. Never run `kubectl delete secret --all` without `-n kcna-lab15`.

Lab 15 · What you learned

- `--from-literal`, `--from-file` and `--from-env-file` produce **different key layouts** from the same data: one key, one key holding a whole file, or one key per line.
- **Environment injection** is simple but frozen at start, leaks through `/proc/<pid>/environ`, and hard-fails on a missing key unless `optional: true`.
- **Volume injection** updates live (whole-directory mounts), keeps values out of the process environment, and is the right default for credentials.
- **subPath mounts are point-in-time copies and never update**. The directory mount next to it does.
- `fsGroup` is what makes a `0440` Secret file readable by a non-root container.
- Secret volumes are **tmpfs** — they never touch the node's disk. `data` in `etcd`, however, is plaintext by default.
- **A Secret is base64-encoded, not encrypted**. `kubectl get -o jsonpath | base64 -d` is a one-liner. The real controls are RBAC, encryption at rest (`--encryption-provider-config`, ideally a KMS v2 provider) and an external secret store.
- `describe secret` masks values; `describe configmap` does not. Neither is a security boundary.
- `immutable: true` protects running Pods and reduces API-server load, at the cost of forcing you to version the object's **name**.
- Missing-key failures are invisible to `kubectl apply` and to static validation. They surface only as `CreateContainerConfigError` at container start.

Lab 15 · Further reading

- ConfigMaps — <https://kubernetes.io/docs/concepts/configuration/configmap/>
- Secrets — <https://kubernetes.io/docs/concepts/configuration/secret/>
- Configure a Pod to Use a ConfigMap — <https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/>
- Distribute Credentials Securely Using Secrets — <https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/>

- Encrypting Confidential Data at Rest — <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>
- Using a KMS provider for data encryption — <https://kubernetes.io/docs/tasks/administer-cluster/kms-provider/>
- Good practices for Kubernetes Secrets — <https://kubernetes.io/docs/concepts/security/secrets-good-practices/>
- Configure a Security Context (`fsGroup`) — <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

Lab 16 — NetworkPolicy and Default-Deny Isolation

Field	Value
Lab ID	Lab 16
Title	NetworkPolicy and Default-Deny Isolation
Day / Topic	Day 3 · Security
Duration	50 minutes
Namespace	kcna-lab16
Learning outcome	LO4 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 350
Repository path	courseware/labs/lab-16-networkpolicy-isolation/

Goal. Close the auditors' finding that any Pod can reach Meridian Freight's payments-api by building default-deny plus one selective allow - and then determine honestly whether this cluster's CNI enforces any of it.

What you will produce:

- A default-deny-all NetworkPolicy (podSelector {}, Ingress and Egress) plus the DNS egress rule that default-deny otherwise breaks
- A matched ingress/egress pair admitting only role=frontend to app=payments-api on the POD port 8080, diffed against data/traffic-matrix.csv
- A recorded enforcement determination: the CNI DaemonSet inventory and a live probe whose result is reported rather than assumed

⚠ Read this before you start — the honesty statement for this lab

kind's default CNI (`kindnet`) does not enforce NetworkPolicy.

The `networking.k8s.io/v1` NetworkPolicy API is served by the API server on every conformant cluster, so every object in this lab will be **accepted and stored**. But a NetworkPolicy is enforced by the **CNI plugin**, and `kindnet` — the CNI kind installs by default — implements Pod networking without a policy engine.

Consequence: after you apply `default-deny-all`, the `reporting` Pod will *still* be able to reach `payments-api` on a stock kind cluster. **That is not a mistake in this lab, and it is not a mistake in your YAML.** It is the CNI.

This lab therefore teaches you to verify a policy **by reasoning about the object** — selectors, ports, direction, namespace scope — and gives you an explicit enforcement probe whose result you record honestly either way. Appendix A shows how to build a kind cluster with Calico if you want real enforcement.

Nothing in this README shows a connection being blocked on a cluster where it was not blocked.

Lab 16 · Objective

By the end of this lab you will be able to:

1. Read every field of a `networking.k8s.io/v1 NetworkPolicy`: `podSelector`, `policyTypes`, `ingress`, `egress`, `namespaceSelector`, `ports`.
2. Explain the **additive, deny-by-default-on-selection** model — that `NetworkPolicy` has no `deny` verb, and that isolation is a *consequence* of being selected.
3. Build the **default-deny then selective-allow** pattern, including the DNS egress rule that default-deny otherwise breaks.
4. State which port number a `NetworkPolicy` matches (the **Pod** port, not the `Service` port) and why.
5. **Determine whether your cluster's CNI enforces `NetworkPolicy` at all**, and interpret the result correctly instead of assuming.
6. Diagnose a policy that is stored but ineffective because a selector or port does not match anything.

Lab 16 · Prerequisites

- Labs 11 and 14 completed. You need `Services/EndpointSlices`, and the Lab 14 point that a namespace is **not** a network boundary.
- A running single-node `kind` cluster.
- Working directory:

```
cd courseware/labs/lab-16-networkpolicy-isolation
pwd
```

Expected output:

```
/.../courseware/labs/lab-16-networkpolicy-isolation
```

- Confirm the API is served (it always is — this proves nothing about enforcement):

```
kubectl api-resources --api-group=networking.k8s.io | grep networkpolicies
```

Expected output:

```
networkpolicies  netpol  networking.k8s.io/v1  true  NetworkPolicy
```

- **Identify your CNI now**, because it determines what you will observe:

```
kubectl get daemonsets -A
```

Expected output on a stock `kind` cluster:

NAMESPACE	NAME	AGE	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
kube-system	kindnet	58m	1	1	1	1	1	<none>
kube-system	kube-proxy	58m	1	1	1	1	1	
kubernetes.io/os=linux								

On the documented stock `kind` setup, `kindnet` provides no `NetworkPolicy` enforcement. Inventory all namespaces and confirm the actual behaviour in Step 5; names alone do not establish every possible CNI or external policy configuration.

Lab 16 · Scenario

Meridian Freight's auditors have flagged a finding: **any Pod in the cluster can open a TCP connection to any other Pod**. The `payments-api` holds settlement data and should only be

reachable from the shipper portal front end. The finance `reporting` job runs in the same namespace and currently has full access to it.

You will implement the standard remediation — **default-deny, then allow exactly one flow** — and then do the thing most tutorials skip: *check whether the cluster you are running on can actually enforce what you wrote*.

Lab 16 · Step-by-step procedure

Step 1 — Namespace, backend and two clients

```
kubectl apply -f manifests/00-namespace.yaml
kubectl apply -f manifests/10-configmap-payments.yaml
kubectl apply -f manifests/20-deployment-payments-api.yaml
kubectl apply -f manifests/30-pods-clients.yaml
```

Expected output:

```
namespace/kcna-lab16 created
configmap/payments-content created
configmap/payments-nginx created
deployment.apps/payments-api created
service/payments-api created
pod/frontend created
pod/reporting created

kubectl wait --for=condition=Available deployment/payments-api -n kcna-lab16 --
timeout=120s
kubectl wait --for=condition=Ready pod/frontend pod/reporting -n kcna-lab16 --timeout=90s
```

Expected output:

```
deployment.apps/payments-api condition met
pod/frontend condition met
pod/reporting condition met
```

Look carefully at the labels — they are the entire mechanism of this lab:

```
kubectl get pods -n kcna-lab16 --show-labels
```

Expected output (Pod-name suffix will differ):

NAME	READY	STATUS	RESTARTS	AGE	LABELS
frontend	1/1	Running	0	32s	app=shipper-
portal,role=frontend					
payments-api-5f6d84b7c9-gq2xn	1/1	Running	0	32s	app.kubernetes.io/part-
of=meridian-freight,app=payments-api,pod-template-hash=5f6d84b7c9,tier=backend					
reporting	1/1	Running	0	32s	app=finance-
reporting,role=reporting					

`frontend` and `reporting` are the same image with the same command. **The only difference between them is `role=frontend` versus `role=reporting`.**

Step 2 — Establish the baseline: everything can reach everything

```
kubectl exec -n kcna-lab16 frontend -- wget -qO- --timeout=5 http://payments-api/ | grep
sensitivity
```

Expected output:

```
<p>sensitivity: RESTRICTED - only role=frontend should reach this</p>

kubectl exec -n kcna-lab16 reporting -- wget -qO- --timeout=5 http://payments-api/ | grep
sensitivity
```


Expected output:

```
<p>sensitivity: RESTRICTED - only role=frontend should reach this</p>
```

That second result is the audit finding. With no NetworkPolicy anywhere, the Kubernetes network model is deliberately flat: every Pod can reach every Pod, across namespaces, without NAT.

Confirm there is genuinely no policy yet:

```
kubectl get networkpolicies -n kcna-lab16
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

Step 3 — Apply default-deny

```
kubectl apply -f manifests/40-networkpolicy-default-deny.yaml
```

Expected output:

```
networkpolicy.networking.k8s.io/default-deny-all created
kubectl describe networkpolicy default-deny-all -n kcna-lab16
```

Expected output:

```
Name:          default-deny-all
Namespace:     kcna-lab16
Created on:    2026-09-05 11:04:22 +0800 +08
Labels:        app.kubernetes.io/part-of=meridian-freight
Annotations:   <none>
Spec:
  PodSelector:  <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress connectivity)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types: Ingress, Egress
```

Read those three parenthesised phrases — they are `kubectl`'s own summary of the model:

Line	Meaning
PodSelector: <none> (... to all pods in this namespace)	podSelector: {} selects every Pod. <none> here means "no <i>constraint</i> ", not "no Pods"
Allowing ingress traffic: <none> (Selected pods are isolated...)	Zero allow rules. Being <i>selected</i> is what causes isolation
Policy Types: Ingress, Egress	Both directions are now governed for those Pods

The model, stated exactly:

- NetworkPolicy has **no deny rule**. You cannot write one.
- A Pod is *non-isolated* for a direction until **some** policy selects it for that direction. Then it becomes **deny-by-default** in that direction.
- What gets through is the **union of the allow rules of every policy** that selects the Pod. Policies are purely additive and unordered.
- Consequently, you cannot "block one client". You deny everything and re-allow the flows you want.

Step 4 — Restore DNS, then allow exactly one flow

A default-deny **egress** policy also blocks DNS, because a DNS query is just UDP/53 to CoreDNS in kube-system. On an enforcing CNI, forgetting this rule turns every `wget http://payments-api/`

into bad address 'payments-api' — which looks like a DNS outage and sends people to debug CoreDNS for an hour.

```
kubectl apply -f manifests/50-networkpolicy-allow-dns.yaml
kubectl apply -f manifests/60-networkpolicy-allow-frontend.yaml
```

Expected output:

```
networkpolicy.networking.k8s.io/allow-dns-egress created
networkpolicy.networking.k8s.io/allow-frontend-to-payments created
networkpolicy.networking.k8s.io/allow-frontend-egress-to-payments created
kubectl get networkpolicies -n kcna-lab16
```

Expected output:

NAME	POD-SELECTOR	AGE
allow-dns-egress	<none>	12s
allow-frontend-egress-to-payments	role=frontend	12s
allow-frontend-to-payments	app=payments-api	12s
default-deny-all	<none>	3m

```
kubectl describe networkpolicy allow-frontend-to-payments -n kcna-lab16
```

Expected output:

```
Name:      allow-frontend-to-payments
Namespace:  kcna-lab16
Labels:     app.kubernetes.io/part-of=meridian-freight
Spec:
  PodSelector:  app=payments-api
  Allowing ingress traffic:
    To Port: 8080/TCP
    From:
      PodSelector:  role=frontend
  Not affecting egress traffic
  Policy Types:  Ingress
```

Four things to internalise from this one object:

1. **PodSelector: app=payments-api** — the policy is attached to the **server**, not to the client. NetworkPolicy always selects the Pods it *protects*.
2. **To Port: 8080/TCP** — this is the **Pod** port (`containerPort: 8080`), **not** the Service port 80. Policy is evaluated after kube-proxy has already DNATed the destination to a Pod IP and Pod port. Writing `port: 80` here is one of the most common silent mistakes, and section 6 makes you do it.
3. **From: PodSelector: role=frontend** — a bare `podSelector` means "in **this** namespace". To allow another namespace you need a `namespaceSelector`, as `allow-dns-egress` does for `kube-system`.
4. **Not affecting egress traffic** — `policyTypes: [Ingress]` here. The client's outbound permission is a separate decision, which is why `allow-frontend-egress-to-payments` exists.

Look at the namespace-crossing rule too:

```
kubectl get networkpolicy allow-dns-egress -n kcna-lab16 -o
jsonpath='{.spec.egress[0].to}' ; echo
```

Expected output:

```
[{"namespaceSelector":{"matchLabels":{"kubernetes.io/metadata.name":"kube-
system"}}, "podSelector":{"matchLabels":{"k8s-app":"kube-dns"}}}]
```

The single subtlest rule in the whole API. `namespaceSelector` and `podSelector` in the **same list item** are **ANDed** — "a Pod labelled `k8s-app=kube-dns` *in* `kube-system`". Split them into two list items and they become **ORed** — "anything in `kube-system`, **or** any `k8s-app=kube-dns` Pod anywhere". A misplaced hyphen changes the meaning completely and nothing warns you.

Step 5 — Determine whether this cluster enforces any of it

Now the honest part. Re-run the two probes from Step 2 and record what actually happens.

```
echo "--- frontend -> payments-api (policy INTENDS: allow) ---"
kubectl exec -n kcna-lab16 frontend -- wget -qO- --timeout=5 http://payments-api/
>/dev/null 2>&1 \
  && echo "REACHABLE" || echo "BLOCKED"
echo "--- reporting -> payments-api (policy INTENDS: deny) ---"
kubectl exec -n kcna-lab16 reporting -- wget -qO- --timeout=5 http://payments-api/
>/dev/null 2>&1 \
  && echo "REACHABLE" || echo "BLOCKED"
```

Expected output on a stock `kind` cluster (`kindnet`):

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
REACHABLE
```

Expected output on a cluster with an enforcing CNI (Calico, Cilium, Antrea, and the managed CNIs on EKS/GKE/AKS):

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
BLOCKED
```

If you saw `REACHABLE` for `reporting`, **your policy is not wrong — your CNI is not enforcing it.** Confirm the cause rather than guessing:

```
kubectl get daemonsets -A -o custom-
columns='NAME:.metadata.name,IMAGE:.spec.template.spec.containers[0].image'
```

Expected output on stock `kind`:

NAME	IMAGE
kindnet	docker.io/kindest/kindnetd:v20240813-c6f155d6
kube-proxy	registry.k8s.io/kube-proxy:v1.31.0

There is no policy engine in that list. The reasoning chain, which you should be able to state in an interview:

1. The `NetworkPolicy` API is part of Kubernetes; **enforcement is not**.
2. Enforcement is delegated to the CNI plugin.
3. `kindnet` provides Pod networking (IPAM, routes, port mapping) but implements **no `NetworkPolicy` controller**.
4. Therefore, on stock `kind`, a `NetworkPolicy` is an **inert, correctly stored object**.
5. Calico, Cilium and Antrea can enforce it. Managed-cloud enforcement depends on the selected CNI and enabled policy features or add-ons; verify that setup explicitly.

There is no event, condition, status field or warning anywhere that tells you this. `kubectl get netpol` looks identical on both kinds of cluster. The only ways to know are to inspect the CNI, or to probe — which is exactly what you just did.

Step 6 — Verify the policy by reasoning, not by observation

Since the training cluster cannot demonstrate enforcement, you verify the *design*. This is a genuinely useful professional skill: policy review before rollout.

The intended traffic is recorded in `data/traffic-matrix.csv`:

```
cat data/traffic-matrix.csv
```

Expected output:

```
source,source_selector,destination,dest_selector,port,protocol,intended,expressed_by
frontend,role=frontend,payments-api,app=payments-api,8080,TCP,allow,allow-frontend-to-
payments
reporting,role=reporting,payments-api,app=payments-api,8080,TCP,deny,no allow rule matches
- default-deny-all applies
frontend,role=frontend,kube-dns,k8s-app=kube-dns,53,UDP,allow,allow-dns-egress
reporting,role=reporting,kube-dns,k8s-app=kube-dns,53,UDP,allow,allow-dns-egress
frontend,role=frontend,internet,none,443,TCP,deny,default-deny-all egress
payments-api,app=payments-api,frontend,role=frontend,8080,TCP,deny,frontend has no ingress
allow rule
```

Trace row 2 by hand — the flow the auditors care about, `reporting` → `payments-api:8080`:

Question	Command	Answer
Is the destination selected by any Ingress policy?	<code>kubectl get netpol -n kcna-lab16 -o jsonpath=...</code>	Yes — <code>default-deny-all({})</code> and <code>allow-frontend-to-payments(app=payments-api)</code>
So is it isolated for ingress?	—	Yes . Selected ⇒ <code>deny-by-default</code>
Does any allow rule admit the source?	<code>allow-frontend-to-payments</code> allows <code>role=frontend</code>	<code>reporting</code> is <code>role=reporting</code> ⇒ no
Verdict	—	DENY — which matches the matrix

Verify each link of that chain against the live cluster:

```
kubectl get pods -n kcna-lab16 -l app=payments-api -o name
```

Expected output — the destination selector does match a real Pod:

```
pod/payments-api-5f6d84b7c9-gq2xn
kubectl get pods -n kcna-lab16 -l role=frontend -o name
kubectl get pods -n kcna-lab16 -l role=reporting -o name
```

Expected output — the allow-list selector matches exactly one Pod, and it is not `reporting`:

```
pod/frontend
pod/reporting
kubectl get networkpolicy allow-frontend-to-payments -n kcna-lab16 \
-o jsonpath='{.spec.ingress[0].ports[0].port}/{ "/" }{.spec.ingress[0].ports[0].protocol} {"\n"}'
kubectl get pods -n kcna-lab16 -l app=payments-api \
-o jsonpath='{.items[0].spec.containers[0].ports[0].containerPort}{ "\n"}'
```

Expected output — **these two numbers must be equal**, and they are:

```
8080/TCP
8080
```

That three-way check — *destination selector matches a Pod, source selector matches the intended client and no one else, policy port equals the container port* — catches the overwhelming majority of real NetworkPolicy defects, and it works on any cluster, enforcing or not.

Lab 16 · Verification

```
bash verification/checks.sh
```

Expected output on a stock kind cluster:

```
== Lab 16 verification: NetworkPolicy and default-deny isolation ==
PASS namespace kcna-lab16 exists
PASS ConfigMap payments-content matches data/payments-index.html byte-for-byte
PASS deployment payments-api is Available and Service payments-api has endpoints
PASS frontend and reporting run the same image and differ by the role label
PASS default-deny-all selects ALL pods ({})) for both Ingress and Egress
PASS default-deny-all declares no allow rules (isolation by selection)
PASS allow-dns-egress permits UDP/53 and TCP/53 to kube-system
PASS allow-dns-egress ANDs namespaceSelector with podSelector in one peer
PASS allow-frontend-to-payments protects app=payments-api on ingress
PASS allow-frontend-to-payments admits role=frontend only
PASS allow-frontend-to-payments port 8080 equals the payments-api containerPort
PASS allow-frontend-egress-to-payments covers the client side of the same flow
PASS matrix allow: frontend -> payments-api:8080 is expressed by a policy
PASS matrix deny: reporting -> payments-api:8080 is expressed by NO policy
PASS matrix deny: payments-api -> frontend:8080 is expressed by NO policy
NOTE -----
NOTE CNI DaemonSet(s) detected: kindnet
NOTE kindnet does NOT implement NetworkPolicy enforcement.
NOTE Enforcement probe: reporting -> payments-api was REACHABLE.
NOTE That is the EXPECTED result on stock kind and is NOT a policy defect.
NOTE Appendix A shows how to build a kind cluster with Calico for real
NOTE enforcement. The policy OBJECTS above are verified as correct.
NOTE -----
PASS RECORDED: enforcement state of this cluster determined and reported honestly
-- 16 passed, 0 failed --
```

On a cluster with an enforcing CNI the **NOTE** block changes and two extra assertions run — see [verification/expected-output.md](#).

Lab 16 · Failure injection — a policy that is stored and does nothing

6.1 Break it

```
kubectl apply -f manifests/70-networkpolicy-broken-selector.yaml
```

Expected output — **accepted, with no complaint whatsoever**:

```
networkpolicy.networking.k8s.io/allow-frontend-to-payments-broken created
kubectl get networkpolicies -n kcna-lab16
```

Expected output — it sits in the list looking exactly as legitimate as the others:

NAME	POD-SELECTOR	AGE
allow-dns-egress	<none>	6m
allow-frontend-egress-to-payments	role=frontend	6m
allow-frontend-to-payments	app=payments-api	6m
allow-frontend-to-payments-broken	app=payments	8s
default-deny-all	<none>	9m

6.2 Diagnose — three independent defects

```
kubectl describe networkpolicy allow-frontend-to-payments-broken -n kcna-lab16
```

Expected output:

```
Name:          allow-frontend-to-payments-broken
Namespace:     kcna-lab16
```

```
Labels:          kcna.tertiaryinfotech.com/intent=failure-injection
Spec:
  PodSelector:    app=payments
  Allowing ingress traffic:
    To Port: 80/TCP
    From:
      PodSelector: role=frontends
  Not affecting egress traffic
  Policy Types: Ingress
```

Run each selector as a live query — the only reliable way to check a NetworkPolicy:

Defect 1 — the policy protects nothing.

```
kubectl get pods -n kcna-lab16 -l app=payments
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

The label is `app=payments-api`. `app=payments` matches zero Pods, so this policy attaches to nothing.

Defect 2 — the port is the Service port, not the Pod port.

```
kubectl get networkpolicy allow-frontend-to-payments-broken -n kcna-lab16 \
-o jsonpath='netpol port: {.spec.ingress[0].ports[0].port}{"\n"}'
kubectl get service payments-api -n kcna-lab16 \
-o jsonpath='service port: {.spec.ports[0].port} -> pod port:
{.spec.ports[0].targetPort}{"\n"}'
```

Expected output:

```
netpol port: 80
service port: 80 -> pod port: http
```

The policy says 80; traffic arrives at the Pod on **8080**, because kube-proxy already translated it. NetworkPolicy sits **below** the Service abstraction.

Defect 3 — the source selector matches nothing.

```
kubectl get pods -n kcna-lab16 -l role=frontends
```

Expected output:

```
No resources found in kcna-lab16 namespace.
```

`role=frontends` $\neq$ `role=frontend`.

Diagnosis. Three defects, three silent failures, zero errors, zero events, zero status. And note the specific danger: because NetworkPolicy is **additive**, a broken *allow* policy never opens a hole — it simply fails to open the one you intended, so your legitimate traffic breaks while you stare at a policy that "looks right". The mirror-image danger is worse: a broken **default-deny** (for example, a `podSelector` that matches nothing) leaves you believing you are isolated when you are not.

Confirm that last point:

```
kubectl get networkpolicy default-deny-all -n kcna-lab16 -o jsonpath='{.spec.podSelector}'
; echo " <- {} means ALL pods"
kubectl get pods -n kcna-lab16 --no-headers | wc -l
```

Expected output:

```
{} <- {} means ALL pods
3
```

An empty `podSelector` covers all 3 Pods. If it had said `matchLabels: {app: payments}` it would cover 0, and your "default deny" would deny nothing at all.

6.3 Fix and re-verify

```
kubectl delete networkpolicy allow-frontend-to-payments-broken -n kcna-lab16
```

Expected output:

```
networkpolicy.networking.k8s.io "allow-frontend-to-payments-broken" deleted
```

Re-run the three-way reasoning check on the *correct* policy:

```
kubectl get pods -n kcna-lab16 -l "$(kubectl get netpol allow-frontend-to-payments -n kcna-lab16 -o jsonpath='{.spec.podSelector.matchLabels.app}')" -o name
kubectl get pods -n kcna-lab16 -l "$(kubectl get netpol allow-frontend-to-payments -n kcna-lab16 -o jsonpath='{.spec.ingress[0].from[0].podSelector.matchLabels.role}')" -o name
```

Expected output:

```
pod/payments-api-5f6d84b7c9-gq2xn
pod/frontend
```

Both selectors resolve to exactly the intended Pod. The policy is correct — whether or not this cluster will act on it.

Lab 16 · Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
Policy applied but traffic still flows	The CNI does not enforce NetworkPolicy (kindnet, and some minimal CNIs)	<code>kubectl get ds -A</code> and the cluster configuration show stock kindnet without a policy engine	Expected on stock kind. Use Calico/Cilium (Appendix A) if you need enforcement
Policy applied and <i>still</i> flows on Calico/Cilium	<code>podSelector</code> matches no Pod	<code>kubectl get pods -n <ns> -l <selector> → No resources found</code>	Fix the label; re-run the selector as a query
Legitimate traffic blocked after adding an allow rule	Policy <code>port</code> is the Service port, not the container port	Compare <code>netpol ... ports[0].port</code> with <code>pod ... containers[0].ports[0].containerPort</code>	Use the Pod port
Everything resolves as <code>bad address</code> after default-deny	Egress deny also blocks DNS	<code>kubectl get netpol -n <ns> -o yaml grep -A5 egress</code>	Add the <code>allow-dns-egress</code> policy (UDP and TCP 53 to kube-system)
Cross-namespace traffic blocked though a <code>podSelector</code> allows it	A bare <code>podSelector</code> only means "this namespace"	<code>kubectl get netpol <n> -o jsonpath='{.spec.ingress[0].from}'</code>	Add a <code>namespaceSelector</code>
A rule allows far more than intended	<code>namespaceSelector</code> and <code>podSelector</code> were put in separate list items (OR) instead of one item (AND)	Count the - entries under <code>from:/to:</code>	Merge them into a single list item
Client allowed by the server's ingress rule but still cannot connect	Default-deny egress also governs the client	<code>kubectl get netpol -n <ns> -o custom-columns=NAME:.metadata.name,TYPES:.spec.policyTypes</code>	Add an egress allow policy for the client Pod

Symptom	Likely cause	Command that confirms it	Fix
kubectl describe netpol shows PodSelector: <none> and you panic	<none> means "no constraint" — i.e. all Pods	Read the parenthesised text kubectl prints beside it	Nothing to fix
Policy has no effect on traffic from outside the cluster	Node-local and some LB traffic can bypass Pod-level policy depending on CNI and externalTrafficPolicy	Test from a Pod, not from the host	Test in-cluster; treat host-network sources separately

Lab 16 · Cleanup

Delete **only** this lab's namespace. NetworkPolicies are namespaced, so they go with it.

```
kubectl delete namespace kcna-lab16
```

Expected output:

```
namespace "kcna-lab16" deleted
```

Confirm:

```
kubectl get namespace kcna-lab16
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab16" not found
```

This lab creates **no cluster-scoped objects**. Never run `kubectl delete netpol --all` without `-n kcna-lab16`; in a production namespace that removes every isolation guarantee at once.

If you built the Appendix A cluster, remove it separately: `kind delete cluster --name kcna-netpol`.

Lab 16 · What you learned

- **The NetworkPolicy API is universal; enforcement is not.** The API server accepts and stores policies on every cluster. The **CNI plugin** enforces them — and **kindnet**, **kind's default CNI**, **does not**. There is no status, event or warning that reveals this; you must inspect the CNI or probe.
- The default Kubernetes network model is **flat**: with no policy, every Pod can reach every Pod in every namespace. A Namespace is not a network boundary (Lab 14).
- NetworkPolicy has **no deny rule**. A Pod becomes deny-by-default in a direction the moment *any* policy selects it for that direction; allowed traffic is the **union** of all matching allow rules.
- `podSelector: {}` means **all Pods in the namespace** — and a `podSelector` that matches nothing produces a policy that silently does nothing.
- The `ports` in a policy are **Pod ports**, not Service ports. Policy is evaluated after kube-proxy's translation.
- **Default-deny egress breaks DNS**. Always pair it with an allow rule for UDP *and* TCP 53 to `kube-system`.

- namespaceSelector + podSelector in one peer item is **AND**; in two items it is **OR**. One hyphen changes the security posture.
- Ingress on the server and egress on the client are **separate decisions**; a flow needs both.
- Verify a policy by running its selectors as live label queries and by comparing its port with the container port. That review works on any cluster, enforcing or not.

Lab 16 · Further reading

- Network Policies — <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Declare Network Policy (walkthrough) — <https://kubernetes.io/docs/tasks/administer-cluster/declare-network-policy/>
- The Kubernetes network model — <https://kubernetes.io/docs/concepts/services-networking/#the-kubernetes-network-model>
- Network Plugins (CNI) — <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>
- kind — Configuration, disableDefaultCNI — <https://kind.sigs.k8s.io/docs/user/configuration/#disable-default-cni>
- Security concepts / cluster hardening — <https://kubernetes.io/docs/concepts/security/>

Lab 16 · Appendix A — a complete policy-enforcing kind cluster

This optional path was tested with **kind v0.33.0**, **Kubernetes v1.36.4** and **Calico v3.32.2**. Calico's published tested range is Kubernetes 1.34–1.36; do not replace the pinned node image with kind's default version without checking compatibility. The exact operator manifests used in verification are supplied in `data/calico-v3.32.2/`, with provenance, checksums and the upstream licence. Internet access is required to pull container images. Allow several minutes for first-time pulls; all six Calico status resources became Available in the test.

Run these commands from **this lab folder**, using Docker, kind and kubectl on macOS/Linux or a Linux shell such as WSL2 on Windows. This creates a separate single-node training cluster. The explicit kubeconfig leaves your existing kubectl context unchanged. If `kcna-netpol` already exists, inspect it before creating or reusing anything; do not apply this bootstrap to an unrelated cluster.

A1 — create the isolated cluster

```
KCNA_POLICY_DIR="$(mktemp -d)"
KCNA_POLICY_CFG="$KCNA_POLICY_DIR/kubeconfig"
kind create cluster --name kcna-netpol \
  --config data/kind-cluster-calico.yaml \
  --image
kindest/node:v1.36.4@sha256:099e049362a1526b2db71494e1947aae99bd16290d7c895f2b7ea312e3cbfa
ed \
  --kubeconfig "$KCNA_POLICY_CFG"
chmod 600 "$KCNA_POLICY_CFG"
kubectl --kubeconfig "$KCNA_POLICY_CFG" get nodes
```

NotReady is expected at this stage: the supplied kind configuration disables kindnet, and Calico has not been installed. Keep this terminal open so the two variables remain available. Treat the kubeconfig as a credential; do not copy it into the lab repository or evidence submission.

A2 — install the supplied, pinned Calico resources

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f
data/calico-v3.32.2/v1_crd_projectcalico_org.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f data/calico-v3.32.2/tigera-operator.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" rollout status deployment/tigera-operator -n
tigera-operator --timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" create -f data/calico-v3.32.2/custom-
resources.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=create tigerastatus/calico --
timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Available tigerastatus/calico
--timeout=600s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Ready node --all --
timeout=300s
kubectl --kubeconfig "$KCNA_POLICY_CFG" get tigerastatus
kubectl --kubeconfig "$KCNA_POLICY_CFG" get daemonsets -A
```

Expect `calico-node` in **calico-system**, not necessarily `kube-system`, and the node to be `Ready`. The optional API server, `Goldmane` and `Whisker` components may finish after core networking. Before using those components, check their own `Available` status. If a wait times out, inspect `get pods -A`, `get events -A` and the affected Pod's `describe` output for pull or readiness failures; do not assume an accepted `Installation` object proves working networking.

A3 — establish a working baseline before policy

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/00-namespace.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/10-configmap-payments.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/20-deployment-payments-api.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/30-pods-clients.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Available
deployment/payments-api -n kcna-lab16 --timeout=180s
kubectl --kubeconfig "$KCNA_POLICY_CFG" wait --for=condition=Ready pod/frontend
pod/reporting -n kcna-lab16 --timeout=180s
kubectl --kubeconfig "$KCNA_POLICY_CFG" exec -n kcna-lab16 frontend -- wget -qO- --
timeout=5 http://payments-api/
kubectl --kubeconfig "$KCNA_POLICY_CFG" exec -n kcna-lab16 reporting -- wget -qO- --
timeout=5 http://payments-api/
```

Both clients must return the payments mock HTML. If either fails, repair the backend, Service, DNS or CNI before adding policy. A pre-existing connection failure would make a later deny result inconclusive.

A4 — apply isolation and prove the intended exception

```
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/40-networkpolicy-default-
deny.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/50-networkpolicy-allow-dns.yaml
kubectl --kubeconfig "$KCNA_POLICY_CFG" apply -f manifests/60-networkpolicy-allow-
frontend.yaml
KUBECONFIG="$KCNA_POLICY_CFG" bash verification/checks.sh
```

The tested result was **18 passed, 0 failed**. In particular:

- `frontend` -> `payments-api` remains **REACHABLE**.
- `reporting` -> `payments-api` is **BLOCKED**.
- DNS permits both UDP and TCP port 53; the allow flow covers both ingress and the frontend's egress, because the default-deny policy isolates both directions.

This environment assignment applies only to the verification command. Other `kubectl` commands continue to use your original configuration. Save the baseline, policy inventory and post-policy outcomes as evidence; submit no `kubeconfig`.

A5 — finish the optional environment

The existing training context was never changed, so no hard-coded context switch is needed. Stop this optional cluster's container when finished to release CPU and memory while retaining its state:

```
docker stop kcna-netpol-control-plane
```

To resume the same cluster, start that container and use the retained kubeconfig; confirm readiness before continuing. Keep its private temporary directory only for as long as you need this training environment.

Sources: [Calico requirements](https://docs.tigera.io/calico/latest/getting-started/kubernetes/requirements), [Calico quickstart](https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart), [kind v0.33.0 node image release](https://github.com/kubernetes-sigs/kind/releases/tag/v0.33.0).

<<https://docs.tigera.io/calico/latest/getting-started/kubernetes/requirements>>

<<https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart>>

<<https://github.com/kubernetes-sigs/kind/releases/tag/v0.33.0>>

Day 4 — Storage, Cluster Architecture & Application Delivery

Day 4 Learning Focus

Give workloads state and give the cluster an architecture. Cover the volume model from `emptyDir` up through `PersistentVolumes`, `StorageClasses` and `CSI`; `StatefulSets` and stable identity; the anatomy of a `kubeadm` control plane and an `etcd` backup/restore you can actually test; then package and deliver it all with `Helm`, `Kustomize` and `GitOps`.

Field	Value
Instructional hours	8 hours
Class window	9:30 AM – 6:30 PM
Registered topics	5. Storage, 6. Cluster Architecture, Installation & Configuration
KCNA modules	Cloud Native Application Delivery
Learning outcomes	LO4, LO5
Abilities / Knowledge	A4, A5 / K5, K6
Labs	5 — Lab 17, Lab 18, Lab 19, Lab 20, Lab 21
Hands-on minutes	260 minutes
Deck slides	360–478

Session structure. The table below is the timetable the trainer works to. Breaks are not counted as instructional time.

Time	Duration	Session	Mode
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

Day 4 Concepts

A container's filesystem dies with the container

The writable layer a runtime adds on top of an image's read-only layers is **not storage**. It is scratch space with the lifetime of the container, and it disappears on restart — not on Pod deletion, on *container* restart. That single fact is the reason the entire volume model exists.

A **volume** in Kubernetes has the lifetime of the **Pod**, and is mounted into one or more containers at paths each container declares for itself. The two you meet first:

- **`emptyDir`** — created empty when the Pod is assigned to a node, deleted permanently when the Pod is removed from that node. It survives container restarts but not rescheduling. Setting `medium: Memory` backs it with tmpfs, which is fast, and which **counts against the container's**

memory limit. It is the correct choice for scratch space and for sharing files between containers in the same Pod.

- **hostPath** — mounts a path from the node's own filesystem. It is a well-known escape route and a genuine security risk, it ties the Pod to one node, and **baseline** Pod Security blocks most of its uses. Legitimate uses are node-level agents (a log shipper reading `/var/log`, a monitoring agent reading `/proc`). It is not application storage.

PersistentVolumes, Claims and StorageClasses

Persistent storage separates the **consumer** from the **supply**. A **PersistentVolumeClaim (PVC)** is namespaced and is written by the application team: "I need 5 GiB, ReadWriteOnce, of class `standard`". A **PersistentVolume (PV)** is cluster-scoped and represents a real piece of storage. A **StorageClass** names a provisioner and its parameters, and is what turns a PVC into a PV automatically.

Static provisioning: an administrator pre-creates PVs; the control plane binds a PVC to a suitable one. **Dynamic provisioning:** the PVC names a StorageClass and the provisioner creates the PV on demand. Dynamic is the norm; static is for pre-existing storage you must import.

Binding is **one-to-one and exclusive** — once a PVC is bound to a PV, no other claim can use that PV, even if the PV is much larger than requested.

Access modes are properties of the volume and are **node-scoped, not Pod-scoped**, which is the part people get wrong. `ReadWriteOnce` (RWO) means the volume can be mounted read-write by **one node** — several Pods on that same node can share it. `ReadOnlyMany` (ROX) is read-only from many nodes. `ReadWriteMany` (RWX) is read-write from many nodes and requires a filesystem that supports it (NFS, CephFS); most block storage cannot do it. `ReadWriteOncePod` restricts to a single Pod. Access modes are *not* enforced as a lock by Kubernetes — they are a scheduling and binding hint.

Reclaim policy decides what happens to the data when the PVC is deleted. `Delete` (the default for most dynamic classes) destroys the PV and the underlying storage. `Retain` keeps the PV in a `Released` state with the data intact, requiring manual cleanup before it can be reused. On anything holding real data, choose `Retain` deliberately.

volumeBindingMode: `WaitForFirstConsumer` is worth understanding rather than memorising. With the default `Immediate`, a PV is provisioned as soon as the PVC exists — possibly in a zone where the Pod cannot then be scheduled. With `WaitForFirstConsumer`, provisioning waits until a Pod actually needs the claim, so the volume is created where the Pod is going. A PVC sitting in `Pending` with no Pod is therefore the **correct** state for such a class, not a fault.

CSI

The **Container Storage Interface** replaced the old in-tree volume plugins, which required storage vendors to get code merged into Kubernetes itself. A CSI driver is split into a **controller** component (provision, delete, attach, detach, snapshot — runs anywhere) and a **node** component (stage, publish, unpublish — runs as a DaemonSet on every node), with a set of Kubernetes-supplied **sidecars** (`external-provisioner`, `external-attacher`, `external-resizer`, `external-snapshotter`, `node-driver-registrar`) translating API objects into gRPC calls. What CSI unlocked in practice: out-of-tree drivers on a vendor's own release cadence, volume snapshots, volume expansion, raw block volumes and ephemeral inline volumes.

StatefulSets and stable identity

A Deployment's Pods are interchangeable and get random names. A **StatefulSet** gives each replica three things a Deployment cannot:

- **A stable ordinal name** — `web-0`, `web-1`, `web-2` — that is reused when the Pod is rescheduled.
- **A stable network identity** through a **headless Service**, giving each replica the DNS name `<pod>.<service>.<namespace>.svc.cluster.local`.
- **Stable per-replica storage** through `volumeClaimTemplates`, which creates one PVC per replica, named `<claim>-<pod>`, and **binds it back to the same ordinal** on reschedule.

Rollout and scale-up are ordered 0, 1, 2 ..., and scale-down is reverse-ordered. Crucially, **scaling down does not delete the PVCs** — the data waits for the replica to come back. That is deliberate and it is the correct default, but it means storage cost does not fall when you scale in, and cleaning up requires deleting the PVCs explicitly.

The control plane, kubeadm and version skew

`kubeadm init` runs a sequence of phases: preflight checks, generating the **PKI**, writing kubeconfigs for the control-plane components, writing the **static Pod manifests** into `/etc/kubernetes/manifests/`, waiting for the kubelet to start them, uploading cluster configuration to a ConfigMap, marking and tainting the node, configuring bootstrap tokens and RBAC, and installing CoreDNS and kube-proxy.

Two of those deserve attention. **Static Pods** are Pods the kubelet runs from files on disk, with no scheduler and no API server involvement — which is how the API server itself can be a Pod. They appear in the API as read-only mirror Pods named `<pod>-<node>`. And the **PKI**: a kubeadm cluster runs several certificate authorities — the cluster CA, an etcd CA, and a front-proxy CA — issuing certificates for the API server, the kubelet client, etcd peers and service-account token signing. Certificate expiry is a real and regular cause of cluster outages.

etcd is the consistent key-value store for Kubernetes API data. The API server accesses it and also communicates with components such as kubelets and admission webhooks. etcd uses the **Raft** consensus algorithm: members elect a leader, the leader appends entries to a log, an entry commits once a **majority** has acknowledged it. A cluster of n members tolerates $(n-1)/2$ failures, which is why member counts are odd — three tolerates one failure, four also tolerates only one, so the fourth member buys nothing and adds latency.

Backup is `etcdctl snapshot save`; restore is `etcdctl snapshot restore` into a **new data directory**, followed by pointing the etcd static Pod at it. Restore is a **node-level file operation, not an API call** — you cannot restore etcd through `kubectl`. A backup you have never restored is a hypothesis, so the testable version of "we back up etcd" is a documented, timed, rehearsed restore.

Upgrade order is control plane first, then nodes, and never skip a minor version. The skew rules: `kube-apiserver` is the reference; the controller manager and scheduler may be up to one minor **behind** it; the kubelet may be up to three minors behind; `kubectl` may be one minor either side.

Application delivery: CI, CD and GitOps

Three different things, routinely conflated. **CI** builds and tests a commit and produces an artefact. **CD** takes that artefact to an environment. **GitOps** is a specific way of doing the second half: the desired state lives in Git, and an **in-cluster controller continuously reconciles the cluster towards it**. The distinction that matters is push versus pull. A pipeline that runs `kubectl apply` is push delivery — the

CI system holds cluster credentials and acts once. A GitOps controller is pull delivery — the cluster holds the credentials, nothing outside needs access, and drift is corrected continuously rather than at deploy time. **Argo CD** and **Flux** are both CNCF graduated implementations.

Helm packages a set of templated manifests as a **chart**, renders them with **values**, and tracks the result as a **release** with a revision history stored in the cluster. Values precedence runs from the chart's `values.yaml`, through `-f` files in order, to `--set` on the command line. `helm template` renders locally and installs nothing; `helm install` creates a release; `helm upgrade` creates a new revision; `helm rollback` returns to a previous one.

Kustomize takes the opposite approach: no templating language at all. A `kustomization.yaml` composes plain YAML **bases** and applies **overlays** — patches, name prefixes, common labels, image tag substitutions, ConfigMap and Secret generators with content hashes. It is built into `kubectl` as `kubectl apply -k`. Helm wins where you are *distributing* software to people you do not know; Kustomize wins where you are *operating* your own manifests across environments. They compose — rendering a chart and patching the output is a common pattern.

Release strategies trade blast radius against cost. *Recreate* is cheapest and has downtime. *Rolling* has no downtime and both versions serve at once. *Blue/green* holds a full second environment and switches traffic atomically, giving instant rollback at double the cost. *Canary* sends a small percentage to the new version and promotes on evidence — which requires that you have the metrics to decide, which is Day 5.

Day 4 Labs

Day 4 has 5 labs, listed below and then set out in full. Work through them in order; each runs in its own namespace and cleans up after itself.

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 17	Volumes: emptyDir, hostPath and the Container Filesystem	kcna-lab17	45	LO4 / A4 / K6	379
Lab 18	PersistentVolume s, Claims and StorageClasses	kcna-lab18	55	LO4 / A4 / K6	397
Lab 19	StatefulSets, Headless Services and Stable Identity	kcna-lab19	50	LO4 / A4 / K6	411
Lab 20	Control Plane Anatomy and etcd Backup/Restore	kcna-lab20	55	LO5 / A5 / K5	437
Lab 21	Packaging and Delivery: Helm, Kustomize and GitOps	kcna-lab21	55	LO5 / A5 / K5	474

Lab 17 — Volumes: emptyDir, hostPath and the Container Filesystem

Field	Value
Lab ID	Lab 17
Title	Volumes: emptyDir, hostPath and the Container Filesystem
Day / Topic	Day 4 · Storage
Duration	45 minutes
Namespace	kcna-lab17
Learning outcome	L04 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 379
Repository path	courseware/labs/lab-17-volumes-emptydir-hostpath/

Goal. Prove that a container's writable layer is discarded with the Pod, then use emptyDir to share scratch state between containers in one Pod and hostPath to reach the node's filesystem — while learning exactly why hostPath is a security liability.

What you will produce:

- A destroyed-and-recreated Pod that demonstrably loses its container-layer data, and an emptyDir-backed report served over HTTP by a sibling container.
- A memory-backed emptyDir (tmpfs) with an enforced sizeLimit, evidenced by mount output and an ENOSPC write failure.
- A read-only hostPath mount of the node's /var/log plus a diagnosed FailedMount event from a hostPath type assertion.

Lab 17 · Objective

By the end of this lab you will be able to:

1. **Demonstrate** that a container's writable layer is destroyed with the Pod, and state precisely which lifecycle event destroys it.
2. **Author** a multi-container Pod in which two containers exchange files through a single emptyDir volume, and explain why that is the only zero-config way to share a filesystem between containers.
3. **Distinguish** node-disk-backed emptyDir from emptyDir.medium: Memory (tmpfs), and show that sizeLimit is genuinely enforced.
4. **Mount** a hostPath volume with the correct type assertion and a readOnly mount, and articulate at least three concrete reasons hostPath is restricted or banned in production clusters.
5. **Diagnose** a FailedMount event caused by a hostPath type assertion, from kubectl describe pod alone.

Lab 17 · Prerequisites

- A single-node **kind** cluster running **Kubernetes v1.30 or later**, and a `kubectl` whose minor version is within one of the server.
- `kubectl` configured with a context that can create namespaces.
- Completion of Lab 04 (multi-container Pod patterns) and Lab 15 (ConfigMaps) — this lab assumes you can read a Pod spec fluently.
- A terminal opened at this lab folder:

```
cd courseware/labs/lab-17-volumes-emptydir-hostpath
pwd
.../courseware/labs/lab-17-volumes-emptydir-hostpath
```

Confirm your cluster and client:

```
kubectl version
Client Version: v1.31.0
Kustomize Version: v5.4.2
Server Version: v1.31.0
```

Version strings will differ on your machine. Anything at v1.30 or above is fine for every step in this lab.

Lab 17 · Scenario

Kallang Freight Pte Ltd is a Singapore logistics operator. Its platform team runs **TrackLane**, the shipment-tracking system that ingests bookings from three depots and publishes a rolling depot report to the operations floor.

Today the platform team has a specific complaint from operations: *"the depot report page goes blank every time you deploy."* The report is generated by a small builder process that writes an HTML file, and a web process that serves it — currently both baked into one container image, writing into the container's own filesystem. Nobody on the team can explain why the file disappears.

Your job for the next 45 minutes is to take that complaint apart. You will first **reproduce the data loss**, then rebuild the report pipeline as two containers sharing an `emptyDir`, then look at the two other places a Pod can put bytes on a node — a memory-backed `tmpfs` and the node's own filesystem via `hostPath` — and record for the platform blueprint exactly when each is appropriate.

You will carry this scenario through the whole of Day 4: Lab 18 gives the archive durable storage, Lab 19 gives the depot ledger a stable identity, Lab 20 looks at the control plane that runs it all, and Lab 21 packages it for delivery.

Lab 17 · Step-by-step procedure

Step 1 — Create the lab namespace

Everything in this lab lives inside `kcna-lab17`. Nothing you do should touch another namespace.

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab17 created
```

Set it as the default for this shell so you cannot accidentally hit default:

```
kubectl config set-context --current --namespace=kcna-lab17
Context "kind-kcna" modified.
```

Confirm:

```
kubectl config view --minify -o jsonpath='{..namespace}{"\n"}'
kcna-lab17
```

Remember this. Every remaining command in this lab still passes `-n kcna-lab17` explicitly so the lab is copy-pasteable even if you skipped this step.

Step 2 — Load the seed dataset into the cluster

`data/shipments.csv` holds 24 synthetic TrackLane bookings for 31 August 2026. Look at it first:

```
head -4 data/shipments.csv
shipment_id,booked_utc,origin_port,destination_port,mode,weight_kg,depot,status
KF-SG-100001,2026-08-31T02:14:00Z,SGSIN,MYPKG,road,1180,depot-a,in_transit
KF-SG-100002,2026-08-31T03:02:00Z,SGSIN,IDJKT,sea,24500,depot-b,at_port
KF-SG-100003,2026-08-31T03:47:00Z,SGSIN,THBKK,air,412,depot-a,cleared
```

Count the data rows (header excluded) — you will check the Pod's arithmetic against this number:

```
echo $(( $(wc -l < data/shipments.csv) - 1 ))
24
```

Project the file into the cluster as a ConfigMap:

```
kubectl -n kcna-lab17 create configmap tracklane-seed \
  --from-file=shipments.csv=data/shipments.csv
configmap/tracklane-seed created

kubectl -n kcna-lab17 get configmap tracklane-seed \
  -o jsonpath='{.data.shipments\.csv}' | head -2
shipment_id,booked_utc,origin_port,destination_port,mode,weight_kg,depot,status
KF-SG-100001,2026-08-31T02:14:00Z,SGSIN,MYPKG,road,1180,depot-a,in_transit
```

Step 3 — Reproduce the data loss (the container filesystem is ephemeral)

`manifests/01-ephemeral-scratch.yaml` declares **no volume at all**. Read it, then apply it:

```
kubectl apply -f manifests/01-ephemeral-scratch.yaml
pod/lane-scratch created

kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-scratch --timeout=90s
pod/lane-scratch condition met
```

The container wrote a note into its own filesystem. Prove the file is there:

```
kubectl -n kcna-lab17 exec lane-scratch -- cat /tmp/tracklane/notes.txt
scratch-note written at 2026-09-05T04:11:23Z
```

Now capture the identity of the container instance, because that is what actually owns the bytes:

```
kubectl -n kcna-lab17 get pod lane-scratch \
  -o jsonpath='{.status.containerStatuses[0].containerID}{"\n"}'
```

```
containerd://6f0a2b7c9e1d4a5f8c3b0e7d2a91c4f65b8e3d07a2c9f14b6e05d8371a2c9b4e
```

Delete the Pod and recreate it from the identical manifest:

```
kubectl -n kcna-lab17 delete pod lane-scratch --wait=true
pod "lane-scratch" deleted

kubectl apply -f manifests/01-ephemeral-scratch.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-scratch --timeout=90s

pod/lane-scratch created
pod/lane-scratch condition met
```

Read the note again:

```
kubectl -n kcna-lab17 exec lane-scratch -- cat /tmp/tracklane/notes.txt
scratch-note written at 2026-09-05T04:12:57Z
```

The timestamp changed. The old file did not survive; what you are reading is a brand-new file written by a brand-new container from the image. Confirm the container identity is different:

```
kubectl -n kcna-lab17 get pod lane-scratch \
-o jsonpath='{.status.containerStatuses[0].containerID}{"\n"}'
containerd://b31d7e05a9c8f24601e3b7d9c5a08f42e76b1c3d09a8f5e2c47b60d13f8a5c92
```

The mechanism. A container image is a stack of read-only layers. The runtime adds one thin **writable layer** per container, using an overlay filesystem. Every byte your process writes outside a mounted volume lands in that writable layer. When the container is removed, the runtime deletes the writable layer. A Pod restart caused by a crash keeps the Pod but replaces the container — so even a `restartPolicy: Always` crash-loop loses the data. Nothing about this is a bug; it is the reason volumes exist.

Clean up before the next step:

```
kubectl -n kcna-lab17 delete pod lane-scratch --wait=true
pod "lane-scratch" deleted
```

Step 4 — Read the `emptyDir` manifest before you apply it

```
grep -n -A 8 '^ volumes:' manifests/02-emptydir-shared.yaml
30: volumes:
31-   # The shared scratch space. No medium => node-backed (disk).
32-   - name: report
33-     emptyDir:
34-       sizeLimit: 64Mi
35-   # The seed dataset, projected read-only from a ConfigMap the learner
36-   # creates from data/shipments.csv.
37-   - name: seed
38-     configMap:
```

Three things to notice:

- `emptyDir: {}` (here with a `sizeLimit`) takes **no** identifying parameters. There is nothing to point at, because the volume has no existence before the Pod is scheduled — the kubelet creates an empty directory under `/var/lib/kubelet/pods/<pod-uid>/volumes/kubernetes.io~empty-dir/report` on whichever node wins the scheduling decision.
- Both containers list the **same volume name** in `volumeMounts`. That is the entire sharing mechanism.

- `report-web` mounts it with `readOnly: true`. The same volume can be read-write in one container and read-only in another; this is cheap least-privilege.

Step 5 — Share a filesystem between two containers with `emptyDir`

```
kubectl apply -f manifests/02-emptydir-shared.yaml
pod/lane-report created
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-report --timeout=120s
pod/lane-report condition met
kubectl -n kcna-lab17 get pod lane-report -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
lane-report	2/2	Running	0	18s	10.244.0.31	kcna-control-plane	<none>

2/2 — both containers are up. The builder logged what it wrote:

```
kubectl -n kcna-lab17 logs lane-report -c report-builder | head -1
report written to /shared/index.html (24 source rows)
```

24 source rows — matching the number you computed from `data/shipments.csv` in Step 2. The dataset really did travel ConfigMap → volume mount → builder → shared volume.

Now prove the *web* container can see a file the *builder* container wrote:

```
kubectl -n kcna-lab17 exec lane-report -c report-web -- cat /shared/index.html
<h1>TrackLane depot report</h1>
<p>built: 2026-09-05T04:15:02Z</p>
<p>pod: lane-report</p>
<p>rows: 24</p>
<p>in_transit: 7</p>
<p>held_customs: 2</p>
```

Serve it over the network to be certain the sharing is real and not a `kubectl` artefact:

```
kubectl -n kcna-lab17 port-forward pod/lane-report 8017:8080 >/tmp/pf-lab17.log 2>&1 &
sleep 3
curl -s http://127.0.0.1:8017/ | head -4
<h1>TrackLane depot report</h1>
<p>built: 2026-09-05T04:15:02Z</p>
<p>pod: lane-report</p>
<p>rows: 24</p>
```

Stop the port-forward:

```
kill %1 2>/dev/null; wait 2>/dev/null; echo "port-forward stopped"
port-forward stopped
```

Confirm the read-only mount is genuinely read-only from the web container:

```
kubectl -n kcna-lab17 exec lane-report -c report-web -- \
  sh -c 'touch /shared/tamper 2>&1 || echo "write refused (expected)"'
touch: /shared/tamper: Read-only file system
write refused (expected)
```

Finally, look at how the node presents this volume. `emptyDir` with no medium is backed by whatever filesystem holds the kubelet root directory:

```
kubectl -n kcna-lab17 exec lane-report -c report-builder -- df -h /shared
```

Filesystem	Size	Used	Available	Use%	Mounted on
overlay	58.4G	21.7G	33.7G	39%	/shared

Reality check on `sizeLimit` for disk-backed `emptyDir`. For a disk-backed `emptyDir` the `sizeLimit` is *not* a filesystem quota — `df` still reports the whole node disk. It is an **eviction threshold**: the kubelet's ephemeral-storage monitor periodically measures the directory and evicts the Pod if it exceeds the limit. For a memory-backed `emptyDir` (Step 7) the limit *is* the tmpfs size and is enforced instantly. This asymmetry catches people out constantly.

Now delete the Pod and observe that the shared report is gone too:

```
kubectl -n kcna-lab17 delete pod lane-report --wait=true
kubectl apply -f manifests/02-emptydir-shared.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-report --timeout=120s
kubectl -n kcna-lab17 exec lane-report -c report-web -- cat /shared/index.html | grep
built

pod "lane-report" deleted
pod/lane-report created
pod/lane-report condition met
<p>built: 2026-09-05T04:17:44Z</p>
```

A new build timestamp: `emptyDir` solved *container-to-container* sharing, but it did **not** solve durability. `emptyDir` shares the Pod's lifetime exactly. That is Lab 18's problem.

Step 6 — Locate the volume on the node (optional, kind only)

If you want to see the directory the kubelet created, and you have `docker` access to the kind node:

```
kubectl -n kcna-lab17 get pod lane-report -o jsonpath='{.metadata.uid}{"\n"}'
2f6c31ab-9e04-4b7d-8c15-7a0d3e59b6f1

docker exec kcna-control-plane \
  ls /var/lib/kubelet/pods/2f6c31ab-9e04-4b7d-8c15-7a0d3e59b6f1/volumes/
kubernetes.io~empty-dir/report

heartbeat.txt
index.html
```

Substitute your own Pod UID and your own kind node container name (`docker ps --format '{{.Names}}'`). This step is illustrative only — skip it if you do not have Docker access. Never modify anything under `/var/lib/kubelet`.

Step 7 — Memory-backed `emptyDir` (tmpfs)

```
kubectl apply -f manifests/03-emptydir-memory.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-cache --timeout=90s

pod/lane-cache created
pod/lane-cache condition met

kubectl -n kcna-lab17 logs lane-cache

--- mount type for /cache ---
tmpfs on /cache type tmpfs (rw,relatime,size=32768k,inode64)
--- df for /cache ---
Filesystem      Size      Used Available Use% Mounted on
tmpfs           32.0M         0       32.0M   0% /cache
```

Two facts are now on screen:

- The filesystem type is **tmpfs**, not `overlay`. The pages live in the node's page cache rather than on the kubelet's disk. (Strictly, tmpfs pages *can* be swapped if the node has swap enabled; kubeadm and kind disable swap by default, but do not build a security argument on that assumption.)
- The size is exactly **32 MiB** — the `sizeLimit` you declared. The kubelet passed it to the tmpfs mount.

Write something small and read it back:

```
kubectl -n kcna-lab17 exec lane-cache -- \
  sh -c 'printf "lane-cache warm at %s\n" "$(date -u +%H:%M:%SZ)" > /cache/warm.txt;
cat /cache/warm.txt'

lane-cache warm at 04:19:31Z
```

When to use `medium: Memory`. Unix sockets between sidecars; short-lived material you would rather not leave on the kubelet's disk; hot caches where the disk round trip dominates. **What it costs:** tmpfs pages written by a container are charged to *that container's* memory limit, so you must budget the memory allowance for whatever you actually store — a container that fills a 1 GiB tmpfs needs a 1 GiB memory allowance. It is also still ephemeral: the volume dies with the Pod, and a node reboot wipes it.

Step 8 — Read the `hostPath` manifest before you apply it

```
grep -n -A 4 'hostPath:' manifests/04-hostpath-node-logs.yaml
36:     hostPath:
37-       path: /var/log
38-       type: Directory
39-   containers:
40-     - name: peek
```

The `type` field is an **assertion the kubelet evaluates before mounting**. The values you must know:

type	Kubelet behaviour
"" (empty, the default)	No check at all. Backwards-compatible and unsafe — a typo becomes a silently-created path.
DirectoryOrCreate	Create the directory (mode 0755, owned by the kubelet) if missing, then mount.
Directory	The path must already exist and be a directory , else the mount fails.
FileOrCreate	Create an empty file if missing, then mount.
File	The path must already exist and be a file.
Socket	The path must be an existing UNIX socket (this is how <code>docker.sock</code> gets mounted).
CharDevice / BlockDevice	The path must be an existing character / block device.

Step 9 — Mount the node filesystem read-only with `hostPath`

```
kubectl apply -f manifests/04-hostpath-node-logs.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/node-log-peek --timeout=90s
```

```
pod/node-log-peek created
pod/node-log-peek condition met

kubectl -n kcna-lab17 logs node-log-peek

--- top level of the node's /var/log (read-only) ---
containers
journal
kubelet.log
kube-apiserver.log
kube-controller-manager.log
kube-scheduler.log
pods
--- can we write to it? ---
read-only mount refused the write (expected)
```

You are looking at the **node's** filesystem from inside a Pod. `containers/` and `pods/` are the symlink trees the kubelet maintains — this is exactly what a Fluent Bit or Vector DaemonSet mounts to ship logs.

Now inspect one node-level path that only exists because you crossed the boundary:

```
kubectl -n kcna-lab17 exec node-log-peek -- \
  sh -c 'ls /host-logs/containers | head -3'

etcd-kcna-control-plane_kube-system_etcd-9a4c1f7e0b3d2856ac9f4b17e05d3c82.log
kindnet-9tzqv_kube-system_kindnet-cni-4f0c8b2d19e7a536c04b81f3e9d27a05.log
kube-apiserver-kcna-control-plane_kube-system_kube-
apiserver-3e07d5b1a982c46f0b73e15d8c204a9f.log
```

Why hostPath is restricted or banned in production. Four independent reasons, all of which have caused real incidents:

1. **No isolation.** A writable hostPath at `/` gives the container the node. Mounting `/var/run/containerd/containerd.sock` or `/var/run/docker.sock` gives it the ability to start a privileged container — a full node takeover in one API call.
2. **No scheduling awareness.** The scheduler does not know whether the path exists on a candidate node. The same Pod works on node-1 and hangs in `ContainerCreating` on node-2. You will see this in Failure Injection A.
3. **No portability.** The data is pinned to one node. Reschedule the Pod and it silently starts with different data, or none.
4. **It bypasses your quota and your CSI story.** Nothing counts hostPath bytes against a PVC, a StorageClass or a ResourceQuota.

The controls: the **Pod Security Standards** `baseline` and `restricted` profiles forbid hostPath volumes outright; policy engines such as Kyverno or a `ValidatingAdmissionPolicy` can allow a specific read-only allowlist for genuine node agents. If you think you need hostPath for application data, you almost certainly want a `PersistentVolumeClaim` — Lab 18.

Lab 17 · Verification

Run the graded checks:

```
bash verification/checks.sh

== Lab 17 verification - namespace kcna-lab17 ==
[PASS] namespace kcna-lab17 exists
[PASS] configmap tracklane-seed carries 25 lines of seed data
```

```
[PASS] pod/lane-report is Running with 2/2 containers ready
[PASS] emptyDir volume 'report' is declared on pod/lane-report
[PASS] report-builder and report-web both mount the 'report' volume
[PASS] /shared/index.html reports 24 rows (matches data/shipments.csv)
[PASS] report-web sees the file written by report-builder (cross-container share)
[PASS] pod/lane-cache mounts a Memory-medium emptyDir with sizeLimit 32Mi
[PASS] /cache is a tmpfs sized 32.0M
[PASS] pod/node-log-peek mounts hostPath /var/log with type Directory
[PASS] the hostPath mount is readOnly
[PASS] no container in kcna-lab17 requests privileged: true
-----
12 passed, 0 failed
```

The full expected transcript is in `verification/expected-output.md`.

Lab 17 · Failure injection

Failure A — `hostPath` type: *Directory* assertion fails

Apply the deliberately broken Pod:

```
kubectl apply -f manifests/05-hostpath-broken.yaml
pod/archive-writer-broken created
```

Wait ~20 seconds, then look:

```
kubectl -n kcna-lab17 get pod archive-writer-broken
```

NAME	READY	STATUS	RESTARTS	AGE
archive-writer-broken	0/1	ContainerCreating	0	22s

ContainerCreating that does not progress is almost always a volume problem. The events say so:

```
kubectl -n kcna-lab17 describe pod archive-writer-broken | tail -12
```

```
Events:
  Type           Reason          Age           From              Message
  ----           -
  Normal        Scheduled       24s          default-scheduler Successfully assigned kcna-lab17/archive-writer-broken to kcna-control-plane
  Warning       FailedMount     8s (x7 over 24s)  kubelet           MountVolume.SetUp failed for volume "archive" : hostPath type check failed: /mnt/tracklane-archive is not a directory
```

Diagnosis, in order:

1. `Scheduled` succeeded — so this is *not* a scheduling, resource or affinity problem. The scheduler was happy; the kubelet is not.
2. `FailedMount` with (x7 over 24s) — the kubelet is retrying with backoff. It will retry forever; the Pod will never start on its own.
3. The message names the volume ("archive"), the failing check (`hostPath` type check), and the path (`/mnt/tracklane-archive`).
4. Cross-reference the manifest: `spec.volumes[0].hostPath.type: Directory` is an existence assertion, and `/mnt/tracklane-archive` does not exist on this node.

The three possible fixes, and which one is correct here:

Fix	Effect	Verdict
Change <code>type</code> to <code>DirectoryOrCreate</code>	Kubelet creates the directory; Pod starts	Works, but hides the real problem — you now have undeclared state on the node

Fix	Effect	Verdict
<code>docker exec <node> mkdir -p /mnt/tracklane-archive</code>	Pod starts	Works on this node only; breaks the moment the Pod reschedules
Replace the hostPath volume with a PersistentVolumeClaim	Storage becomes a first-class, schedulable, portable API object	Correct. This is exactly what you build in Lab 18

Remove the broken Pod:

```
kubectl -n kcna-lab17 delete pod archive-writer-broken --ignore-not-found
pod "archive-writer-broken" deleted
```

Failure B — overfilling a memory-backed emptyDir

lane-cache has a 32 MiB tmpfs. Try to write 40 MiB into it:

```
kubectl -n kcna-lab17 exec lane-cache -- \
  sh -c 'dd if=/dev/zero of=/cache/blob bs=1M count=40'
dd: error writing '/cache/blob': No space left on device
32+0 records in
31+0 records out
33554432 bytes (32.0MB) copied, 0.041826 seconds, 765.2MB/s
command terminated with exit code 1
```

Diagnosis. ENOSPC at exactly 33554432 bytes = 32 MiB. The kubelet sized the tmpfs to `sizeLimit`, so the write failed at the filesystem layer *before* the Pod's memory cgroup was threatened. The Pod is still Running:

```
kubectl -n kcna-lab17 get pod lane-cache
```

NAME	READY	STATUS	RESTARTS	AGE
lane-cache	1/1	Running	0	4m12s

This is the **good** failure mode, and it is why you always set `sizeLimit` on a memory-backed emptyDir. Without it the tmpfs is sized by the node's default (traditionally half the node's RAM), so the same `dd` would have kept consuming real node memory until the container hit its own memory limit and was **OOMKilled** — or, if no memory limit were set, until the node itself came under memory pressure and the kubelet began evicting *other people's* Pods.

Reclaim the space:

```
kubectl -n kcna-lab17 exec lane-cache -- sh -c 'rm -f /cache/blob; df -h /cache'
```

Filesystem	Size	Used	Available	Use%	Mounted on
tmpfs	32.0M	0	32.0M	0%	/cache

Lab 17 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
Pod stuck ContainerCreating, event hostPath type check failed: <path> is not a directory	hostPath.type: Directory asserted on a path that does not exist on the assigned node	<code>kubectl -n kcna-lab17 describe pod <pod> tail -15</code>	Use a PVC instead; or <code>DirectoryOrCreate</code> if this really is node-local agent state
Pod stuck ContainerCreating, event configmap "tracklane-seed" not found	Step 2 skipped, or the ConfigMap was created in the wrong namespace	<code>kubectl -n kcna-lab17 get cm</code>	Re-run the <code>create configmap</code> command in Step 2 with <code>-n kcna-lab17</code>

Symptom	Likely cause	Diagnostic command	Fix
lane-report shows 1/2 and report-web is in CrashLoopBackOff	report-builder has not produced /shared/index.html yet, or crashed before writing it	kubectl -n kcna-lab17 logs lane-report -c report-builder	Read the builder's error; the web container waits for a non-empty file, so fix the builder first
touch: /shared/...: Read-only file system from report-web	Working as designed — that container's volumeMount sets readOnly: true	kubectl -n kcna-lab17 get pod lane-report -o jsonpath='{.spec.containers[1].volumeMounts}'	Nothing to fix. Write from report-builder, which mounts it read-write
Data written into /shared disappears after a deploy	emptyDir is bound to the Pod lifetime, not the workload	kubectl -n kcna-lab17 get pod <pod> -o jsonpath='{.spec.volumes}'	Move the data to a PersistentVolumeClaim (Lab 18)
df -h /shared shows the whole node disk despite sizeLimit: 64Mi	Expected: for disk-backed emptyDir, sizeLimit is an eviction threshold, not a quota	kubectl -n kcna-lab17 get events --field-selector reason=Evicted	If you need hard enforcement, use medium: Memory with sizeLimit, or a PVC with a real capacity
Pod ... exceeds the limit "64Mi" Evicted event	The disk-backed emptyDir grew past sizeLimit and the kubelet evicted the Pod	kubectl -n kcna-lab17 describe pod <pod> grep -A3 Evicted	Raise sizeLimit, or stop writing bulk data into ephemeral storage

Lab 17 · Cleanup

Delete only this lab's namespace. Every object you created in this lab (Pods and the ConfigMap) is namespaced, so one delete is complete and nothing outside `kcna-lab17` is touched.

```
kubectl delete namespace kcna-lab17 --wait=true
namespace "kcna-lab17" deleted
```

Return your shell to the default namespace:

```
kubectl config set-context --current --namespace=default
Context "kind-kcna" modified.
```

Confirm nothing is left:

```
kubectl get namespace kcna-lab17
Error from server (NotFound): namespaces "kcna-lab17" not found
```

This lab created no cluster-scoped objects. No PersistentVolume, no StorageClass, no ClusterRole. The `hostPath` mount read the node's `/var/log` but wrote nothing to it — confirm with `kubectl -n kcna-lab17 logs node-log-peek` before you delete, which reports read-only mount refused the write.

Lab 17 · What you learned

- A container's writable layer belongs to the **container**, not the Pod and not the workload. Any container replacement — crash restart, image change, node reschedule — destroys it. You proved this with a changing timestamp and a changing `containerID`.

- `emptyDir` is created by the kubelet when the Pod is bound to a node and deleted when the Pod is removed from that node. It is the correct and only zero-configuration answer to *"how do two containers in one Pod share files?"* — and it is the wrong answer to *"how do I keep this data?"*.
- The same `emptyDir` can be mounted read-write in one container and read-only in another. Use that.
- **medium: Memory** turns the volume into a tmpfs: RAM-backed, charged to the writing container's memory limit, and sized to `sizeLimit` by `SizeMemoryBackedVolumes` (beta from v1.22; GA in v1.32). Do not assume the node has swap disabled. For disk-backed `emptyDir` the same field is only an eviction threshold.
- `hostPath` crosses the container boundary into the node. Its `type` field is an assertion evaluated by the kubelet at mount time, and getting it wrong produces a `FailedMount` event, not a scheduling failure — which is why `describe pod` beats `get pod` every time.
- `hostPath` is forbidden by the `baseline` and `restricted` Pod Security Standards. Legitimate uses are node agents that mount a specific path **read-only**; application data belongs on a `PersistentVolumeClaim`.

Carry into Lab 18: you now have a concrete failure (`hostPath type check failed`) whose correct fix is *"make storage a first-class API object"*. Lab 18 builds exactly that.

Lab 17 · Further reading

- Kubernetes documentation — *Volumes*: <https://kubernetes.io/docs/concepts/storage/volumes/>
- Kubernetes documentation — *Volumes: emptyDir*: <https://kubernetes.io/docs/concepts/storage/volumes/#emptydir>
- Kubernetes documentation — *Volumes: hostPath* (including the full `type` table and the security warning): <https://kubernetes.io/docs/concepts/storage/volumes/#hostpath>
- Kubernetes documentation — *Ephemeral Volumes*: <https://kubernetes.io/docs/concepts/storage/ephemeral-volumes/>
- Kubernetes documentation — *Pod Security Standards* (`hostPath` is disallowed at `baseline` and above): <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- Kubernetes documentation — *Node-pressure Eviction* (how `sizeLimit` is enforced for disk-backed volumes): <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>
- CNCF KCNA Curriculum — *Container Orchestration* → *Storage*: <https://github.com/cncf/curriculum>

Lab 18 — PersistentVolumes, Claims and StorageClasses

Field	Value
Lab ID	Lab 18
Title	PersistentVolumes, Claims and StorageClasses
Day / Topic	Day 4 · Storage
Duration	55 minutes
Namespace	kcna-lab18
Learning outcome	LO4 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 397
Repository path	courseware/labs/lab-18-pv-pvc-storageclass/

Goal. Give the TrackLane customs archive storage that outlives the Pod that wrote it — first by hand-binding a static PersistentVolume, then by letting a StorageClass provision one dynamically — and diagnose a Pending claim from describe events alone.

What you will produce:

- A statically provisioned PV bound to a namespaced PVC, seeded with 30 archive rows by one Pod and read back by a second Pod after the writer was deleted.
- A dynamically provisioned volume from kind's default `standard` StorageClass, observed transitioning from Pending (`WaitForFirstConsumer`) to Bound the moment a consumer is scheduled.
- A diagnosed Pending PVC whose root cause is a StorageClass name typo, evidenced by a `ProvisioningFailed` event.

Lab 18 · Objective

By the end of this lab you will be able to:

1. **Explain** the division of labour between a `PersistentVolume` (cluster-scoped supply, an administrator's concern) and a `PersistentVolumeClaim` (namespaced demand, a developer's concern).
2. **Bind** a hand-written static PV to a PVC using `storageClassName` and `claimRef`, and state why the bound capacity is 1Gi when the claim asked for 512Mi.
3. **Prove** persistence by writing data from one Pod, deleting that Pod, and reading the data back from a different Pod.
4. **Choose** correctly between the access modes `ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany` and `ReadWriteOncePod`, and state which of them a single-node kind cluster can actually deliver.
5. **Contrast** the reclaim policies `Retain` and `Delete`, and recognise the `Released` phase.
6. **Provision dynamically** through kind's default `standard` StorageClass, and correctly identify a `Pending` PVC caused by `volumeBindingMode: WaitForFirstConsumer` as expected behaviour rather than a fault.

7. **Diagnose** a genuinely broken `Pending` PVC — a `StorageClass` name typo — from `kubectl describe pvc` events.

Lab 18 · Prerequisites

- A single-node **kind** cluster on **Kubernetes v1.30 or later**.
- **Lab 17 completed**. This lab is the answer to the failure you diagnosed there.
- Terminal at this lab folder:

```
cd courseware/labs/lab-18-pv-pvc-storageclass
pwd
.../courseware/labs/lab-18-pv-pvc-storageclass
```

What kind gives you out of the box

Before you write anything, find out what storage this cluster already has. **This matters: the whole lab depends on it.**

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION	AGE		
standard (default)	rancher.io/local-path	Delete	WaitForFirstConsumer
41m			false

Read every column:

Column	Value	What it means for this lab
NAME	standard (default)	The (default) suffix means a PVC that omits <code>storageClassName</code> gets this class automatically.
PROVISIONER	rancher.io/local-path	kind ships the local-path-provisioner . It is <i>not</i> a CSI driver — it is a simple in-cluster controller that creates a directory under <code>/opt/local-path-provisioner</code> on the node and wraps it in a PV.
RECLAIMPOLICY	Delete	Deleting a PVC from this class destroys the underlying directory and the PV with it.
VOLUMEBINDINGMODE	WaitForFirstConsumer	Claims stay Pending until a Pod consumes them. You will see this in Step 9 and it is <i>correct</i> .
ALLOWVOLUMEEXPANSION	false	You cannot grow a local-path volume by editing the PVC.

Confirm the provisioner is actually running:

```
kubectl -n local-path-storage get pods
```

NAME	READY	STATUS	RESTARTS	AGE
local-path-provisioner-7dc846544d-mrxsq	1/1	Running	0	41m

Honesty note about this cluster. On a real production cluster the provisioner would be a **CSI driver** — `ebs.csi.aws.com`, `pd.csi.storage.gke.io`, `disk.csi.azure.com`, `rook-ceph.rbd.csi.ceph.com` — running as a controller Deployment plus a node DaemonSet, and the

volumes would be network-attached block devices or filesystems that survive node loss. kind's local-path-provisioner produces node-local directories. Everything you learn about the **PV/PVC/StorageClass API** transfers exactly; the **durability guarantees** do not. Be explicit about that distinction when you write a storage blueprint.

Lab 18 · Scenario

Kallang Freight's **TrackLane** platform has a statutory problem. Singapore Customs requires sealed export manifests to be retained for **seven years**. Right now those manifests are written into a container filesystem — the same design you proved fatal in Lab 17 — and the compliance team has flagged it as an audit finding.

The platform team has produced a retention policy (`data/retention-policy.csv`) that classifies every TrackLane dataset into gold, silver and bronze tiers with a required reclaim policy for each. Your job in this lab is to implement the two extremes of that policy:

- **customs-manifest-archive** — *gold tier, Retain*. Storage supplied deliberately by an administrator, bound to exactly one claim, and surviving the deletion of that claim.
- **tracking-render-cache** — *bronze tier, Delete*. Storage summoned on demand from a StorageClass, thrown away with its claim.

Then you will be handed a real ticket: *"the archive PVC has been Pending for twenty minutes"*.

Lab 18 · Step-by-step procedure

Step 1 — Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab18 created
```

Look at the two datasets you will use:

```
head -3 data/manifest-archive.csv
echo "rows: $(( $(wc -l < data/manifest-archive.csv) - 1 ))"

archive_id,sealed_utc,shipment_id,consignee,hs_code,declared_value_sgd,customs_ref,depot
ARC-2026-000101,2026-08-24T01:05:00Z,KF-SG-100001,Pahang Agri Bhd,0803.10,18450.00,SG-
EXP-884201,depot-a
ARC-2026-000102,2026-08-24T02:41:00Z,KF-SG-100002,Sentosa Marine
Supply,8481.80,127300.00,SG-EXP-884202,depot-b
rows: 30

column -s, -t data/retention-policy.csv | cut -c1-96
```

dataset	tier	access_mode	reclaim_policy	retention_days	
storage_class					
customs-manifest-archive	gold	ReadWriteOnce	Retain	2555	
tracklane-manual					
depot-ledger-state	gold	ReadWriteOnce	Retain	2555	
tracklane-retain					
tracking-render-cache	bronze	ReadWriteOnce	Delete	1	standard
lane-etl-scratch	bronze	ReadWriteOnce	Delete	1	standard
ops-report-snapshots	silver	ReadWriteOnce	Delete	90	standard
partner-edi-inbox	silver	ReadWriteOnce	Retain	365	
tracklane-retain					

If column is unavailable, `cat data/retention-policy.csv` shows the same content unaligned.

Load both into the cluster:

```
kubectl -n kcna-lab18 create configmap tracklane-archive-seed \
--from-file=manifest-archive.csv=data/manifest-archive.csv
kubectl -n kcna-lab18 create configmap tracklane-retention-policy \
--from-file=retention-policy.csv=data/retention-policy.csv

configmap/tracklane-archive-seed created
configmap/tracklane-retention-policy created
```

Step 2 — Understand access modes before you choose one

This is examinable and widely misunderstood. An access mode is a property of the **volume**, requested by the claim, and enforced by the **provisioner** — Kubernetes itself does not magically make a volume shareable.

Mode	Short	Meaning	Available on a single-node kind cluster?
ReadWriteOnce	RWO	Mountable read-write by Pods on one node . Multiple Pods on that <i>same</i> node may mount it.	Yes . local-path supports this.
ReadOnlyMany	ROX	Mountable read-only by Pods on many nodes simultaneously.	Not meaningfully — there is only one node, and local-path does not advertise ROX.
ReadWriteMany	RWX	Mountable read-write by Pods on many nodes simultaneously. Needs a shared filesystem: NFS, CephFS, EFS, Azure Files.	No . No block-device-backed driver can offer this, and local-path does not.
ReadWriteOncePod	RWOP	Mountable read-write by exactly one Pod in the whole cluster . Stable since v1.29; requires a CSI driver.	No . RWOP is enforced by the CSI layer; local-path is not a CSI driver.

The trap. `ReadWriteOnce` says *one node*, **not one Pod**. Two Pods scheduled to the same node can both mount an RWO volume read-write and corrupt each other. `ReadWriteOncePod` was added in v1.22 (stable v1.29) precisely to close that gap. If you need single-writer semantics, RWO is not enough.

Everything in this lab therefore uses **RWO**, which is the honest answer for this cluster.

Step 3 — Create the static PersistentVolume

Read the manifest, then apply it:

```
kubectl apply -f manifests/01-static-pv.yaml
persistentvolume/tracklane-archive-pv created

kubectl get pv tracklane-archive-pv
```


NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE		
tracklane-archive-pv	1Gi	RWO	Retain	Available	
kcna-lab18/archive-claim	tracklane-manual	<unset>			6s

Two things worth stopping on:

- **STATUS:** Available — the PV exists and is unbound.
- **CLAIM:** kcna-lab18/archive-claim is **already populated** even though no such claim exists yet. That is the `claimRef` you wrote: the PV is *reserved*. Any other PVC that tries to take it will be refused.

`VOLUMEATTRIBUTESCLASS` appears from Kubernetes v1.29 onward (the `VolumeAttributesClass` alpha/beta feature for mutable volume QoS). `<unset>` is normal. On older clients the column is absent.

Confirm it is cluster-scoped — it has no namespace:

```
kubectl get pv tracklane-archive-pv -o jsonpath='{.metadata.namespace}'[end]"{"\n"}'[end]
```

Step 4 — Create the claim and watch it bind

```
kubectl apply -f manifests/02-static-pvc.yaml
persistentvolumeclaim/archive-claim created
kubectl -n kcna-lab18 get pvc archive-claim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS	AGE				
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO	tracklane-manual
<unset>		4s			

Bound, immediately. Note this carefully, because it contrasts with Step 9:

- This PV/PVC pair uses `storageClassName: tracklane-manual`, which is a class name **no StorageClass object owns**. There is no provisioner and therefore no `volumeBindingMode`, so the PersistentVolume controller binds as soon as it finds a match. Binding is *immediate*.
- **CAPACITY** reads **1Gi**, not the 512Mi you requested. Binding is "at least as large as requested"; the Pod receives the entire PV. In a cloud, a 512Mi claim that binds to a hand-made 1Gi volume bills you for 1Gi.

Look at it from the PV's side:

```
kubectl get pv tracklane-archive-pv -o custom-columns=\
'NAME:.metadata.name,STATUS:.status.phase,CLAIM:.spec.claimRef.name,UID:.spec.claimRef.uid',
```

NAME	STATUS	CLAIM	UID
tracklane-archive-pv	Bound	archive-claim	1c9f4a02-6e5b-4d81-93a7-0b2e5f7c4d16

The controller has filled in the claim's **UID**. This is what makes binding one-to-one and permanent: even if you delete the PVC and recreate one with the identical name, the UID differs and the PV will not re-bind to it. That behaviour is the subject of Step 12.

Step 5 — Seed the archive into the volume

```
kubectl apply -f manifests/03-archive-seeder.yaml
pod/archive-seeder created
kubectl -n kcna-lab18 wait --for=jsonpath='{.status.phase}'=Succeeded \
  pod/archive-seeder --timeout=120s
pod/archive-seeder condition met
kubectl -n kcna-lab18 logs archive-seeder
wrote 30 archive rows to the PersistentVolume
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
```

30 rows — matching `data/manifest-archive.csv`. The dataset has travelled: file on disk → ConfigMap → volume mount → PersistentVolume.

Now delete the writer. This is the important move:

```
kubectl -n kcna-lab18 delete pod archive-seeder --wait=true
pod "archive-seeder" deleted
kubectl -n kcna-lab18 get pods
No resources found in kcna-lab18 namespace.
```

No Pod in the namespace. The claim, however, is untouched:

```
kubectl -n kcna-lab18 get pvc archive-claim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS	AGE				
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO	tracklane-manual
<unset>		3m12s			

Step 6 — Read the archive back from a different Pod

```
kubectl apply -f manifests/04-archive-reader.yaml
kubectl -n kcna-lab18 wait --for=condition=Ready pod/archive-auditor --timeout=120s
pod/archive-auditor created
pod/archive-auditor condition met
kubectl -n kcna-lab18 logs archive-auditor
--- provenance written by the (now deleted) seeder Pod ---
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
--- archive row count as read back ---
rows_read=30
--- Can this auditor modify the archive? ---
read-only mount refused the write (expected)
```

This is the result the whole lab exists for. `sealed_by=archive-seeder` was written by a Pod that no longer exists, and `rows_read=30` matches the source dataset byte for byte. Compare with Lab 17, where the identical experiment on an `emptyDir` produced a fresh build timestamp every time.

Step 7 — Where do the bytes actually live? (kind only, optional)

```
kubectl get pv tracklane-archive-pv -o jsonpath='{.spec.hostPath.path}'  
/tmp/tracklane-archive  
docker exec kcna-control-plane ls -l /tmp/tracklane-archive  
total 8  
-rw-r--r-- 1 root root 96 Sep 5 05:02 PROVENANCE  
-rw-r--r-- 1 root root 3341 Sep 5 05:02 manifest-archive.csv
```

Substitute your kind node container name from `docker ps --format '{{.Names}}'`. **Read only — never modify anything under a live PV from the node side.** And note the honesty point again: this is `/tmp` *inside the kind node container*, not on your laptop. `kind delete cluster` destroys it. A `Retain` reclaim policy protects you from the Kubernetes control plane, not from losing the node.

Step 8 — Register the gold-tier StorageClass

`data/retention-policy.csv` says `depot-ledger-state` and `partner-edi-inbox` need dynamic provisioning **with** `Retain`. `kind's` standard class uses `Delete`, so you need a second class.

```
kubectl apply -f manifests/05-storageclass-retain.yaml  
storageclass.storage.k8s.io/tracklane-retain created  
kubectl get storageclass  


| NAME                 | PROVISIONER           | RECLAIMPOLICY | VOLUMEBINDINGMODE    |       |
|----------------------|-----------------------|---------------|----------------------|-------|
| ALLOWVOLUMEEXPANSION | AGE                   |               |                      |       |
| standard (default)   | rancher.io/local-path | Delete        | WaitForFirstConsumer | false |
| 52m                  |                       |               |                      |       |
| tracklane-retain     | rancher.io/local-path | Retain        | WaitForFirstConsumer | false |
| 5s                   |                       |               |                      |       |


```

Same provisioner, different lifecycle. That is the whole idea of a `StorageClass`: it is a named **profile** over a driver.

```
kubectl describe storageclass tracklane-retain | head -8  
Name: tracklane-retain  
IsDefaultClass: No  
Annotations: storageclass.kubernetes.io/is-default-class=false  
Provisioner: rancher.io/local-path  
Parameters: <none>  
AllowVolumeExpansion: False  
MountOptions: <none>  
ReclaimPolicy: Retain
```

A word on CSI. In-tree storage drivers were removed from the Kubernetes codebase over the v1.21–v1.31 cycle; every cloud volume type is now an out-of-tree **CSI driver**. CSI ("Container Storage Interface") is a gRPC contract with three services — Identity, Controller and Node — that a vendor implements once and every orchestrator can consume. `kubectl get csidrivers` lists the drivers registered on your cluster. On a stock kind cluster the list is empty, because `local-path-provisioner` predates CSI and is a plain controller:

```
kubectl get csidrivers  
No resources found
```

That empty output is itself worth showing learners: kind is teaching you the API, not the production storage stack.

Step 9 — Dynamic provisioning, and the Pending you should NOT panic about

```
kubectl apply -f manifests/06-dynamic-pvc.yaml
persistentvolumeclaim/render-cache created
kubectl -n kcna-lab18 get pvc render-cache
NAME                STATUS      VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS  AGE
render-cache        Pending                                standard        <unset>
8s
```

Pending with no volume. **Do not debug this.** Read the events:

```
kubectl -n kcna-lab18 describe pvc render-cache | tail -8
Used By:          <none>
Events:
  Type            Reason              Age           From              Message
  ----            -
  Normal          WaitForFirstConsumer  3s (x2 over 10s)  persistentvolume-controller  waiting for first consumer to be created before binding
```

Reason: WaitForFirstConsumer, Type: Normal (not Warning). The controller is telling you, in as many words, that it is deliberately holding off.

Why the behaviour exists. If the provisioner created the volume immediately it would have to guess a node or a zone. In a multi-zone cluster that guess is wrong two-thirds of the time, and you get a Pod that can never be scheduled because its volume is in `ap-southeast-1a` and its only free capacity is in `1b`. `WaitForFirstConsumer` inverts the order: schedule the Pod first, *then* provision the volume where the Pod landed.

How to tell this apart from a real fault — memorise this table:

Event Type	Event Reason	Verdict
Normal	WaitForFirstConsumer	Expected. Create a Pod that uses the claim.
Normal	Provisioning / ExternalProvisioning	In progress. Wait, then check the provisioner Pod's logs.
Warning	ProvisioningFailed	Real fault. Read the message — bad class, quota, driver error.
(no events at all)	—	No matching PV and no provisioner is even looking. Usually a class name that no StorageClass owns.

Step 10 — Schedule the first consumer

```
kubectl apply -f manifests/07-render-cache-deployment.yaml
deployment.apps/render-cache created
kubectl -n kcna-lab18 rollout status deployment/render-cache --timeout=180s
```

```
Waiting for deployment "render-cache" rollout to finish: 0 of 1 updated replicas are available...
deployment "render-cache" successfully rolled out
```

```
kubectl -n kcna-lab18 get pvc render-cache
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE		
render-cache	Bound	pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54	256Mi	RWO
standard	<unset>	2m41s		

Bound, and the volume name is `pvc-<uuid>` — machine-generated, because no human wrote this PV. Look at what the provisioner created:

```
DYNPV=$(kubectl -n kcna-lab18 get pvc render-cache -o jsonpath='{.spec.volumeName}')
kubectl get pv "$DYNPV"
```

NAME	STATUS	CLAIM	STORAGECLASS	CAPACITY	ACCESS MODES	RECLAIM POLICY	VOLUMEATTRIBUTESCLASS	REASON	AGE
pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54	Bound	kcna-lab18/render-cache	standard	256Mi	RWO	Delete			34s

Note RECLAIM POLICY: Delete, inherited from the standard class. Compare with `tracklane-archive-pv`, which is Retain.

```
kubectl -n kcna-lab18 logs deployment/render-cache
--- /cache/starts.log ---
start render-cache-6c9f84b57d-x2klp 2026-09-05T05:14:08Z
--- bronze-tier datasets from the retention policy ---
tracking-render-cache
lane-etl-scratch
```

The retention policy dataset is genuinely being read inside the Pod.

Now restart the Deployment and prove the dynamic volume persists too:

```
kubectl -n kcna-lab18 rollout restart deployment/render-cache
kubectl -n kcna-lab18 rollout status deployment/render-cache --timeout=180s
kubectl -n kcna-lab18 logs deployment/render-cache | head -4

deployment.apps/render-cache restarted
deployment "render-cache" successfully rolled out
--- /cache/starts.log ---
start render-cache-6c9f84b57d-x2klp 2026-09-05T05:14:08Z
start render-cache-7fb5d9c684-qn8vt 2026-09-05T05:15:52Z
```

Two lines. The second Pod appended to a file the first Pod created. The `Recreate` strategy mattered here: with `RollingUpdate`, the new Pod would have tried to mount an RWO volume still held by the old Pod.

Step 11 — Full picture

```
kubectl -n kcna-lab18 get pvc,pod -o wide
```

NAME	STATUS	VOLUME
CAPACITY	ACCESS MODES	STORAGECLASS
AGE		
persistentvolumeclaim/archive-claim	Bound	tracklane-archive-pv
1Gi	RWO	tracklane-manual
14m		
persistentvolumeclaim/render-cache	Bound	pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54
256Mi	RWO	standard
5m		

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/archive-auditor	1/1	Running	0	9m	10.244.0.44	kcna-
control-plane						

```
pod/render-cache-7fb5d9c684-qn8vt    1/1      Running    0          2m      10.244.0.46    kcna-
control-plane
```

One claim satisfied by an administrator's hand-made volume, one satisfied by a provisioner — and the Pod specs are **identical in shape**. That is the abstraction paying off: the workload author writes `persistentVolumeClaim: {claimName: ...}` and never learns which mechanism supplied the bytes.

Step 12 — Reclaim policy in action: Retain and the Released phase

Delete the auditor and then the gold-tier claim:

```
kubectl -n kcna-lab18 delete pod archive-auditor --wait=true
kubectl -n kcna-lab18 delete pvc archive-claim --wait=true

pod "archive-auditor" deleted
persistentvolumeclaim "archive-claim" deleted

kubectl get pv tracklane-archive-pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE		
tracklane-archive-pv	1Gi	RWO	Retain	Released	
kcna-lab18/archive-claim	tracklane-manual	<unset>			17m

STATUS: Released, not Available and not deleted. **The data is still on the node**. This is exactly what a seven-year customs retention requirement needs: deleting a namespace or a claim must not destroy the archive.

A Released PV cannot be re-bound. Prove it:

```
kubectl apply -f manifests/02-static-pvc.yaml
sleep 10
kubectl -n kcna-lab18 get pvc archive-claim

persistentvolumeclaim/archive-claim created
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS	AGE				
archive-claim	Pending				tracklane-manual

```
<unset>
10s
```

The recreated claim has a **new UID**; the PV's `claimRef.uid` still points at the old one, so the controller refuses. To reclaim it an administrator must deliberately clear the stale reference:

```
kubectl patch pv tracklane-archive-pv --type=json \
  -p='[{"op":"remove","path":"/spec/claimRef/uid"}, {"op":"remove","path":"/spec/claimRef/resourceVersion"}]'
```

```
persistentvolume/tracklane-archive-pv patched

sleep 10
kubectl -n kcna-lab18 get pvc archive-claim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS	AGE				
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO	tracklane-manual

```
<unset>
33s
```

Re-bound. Now prove the *data* survived the whole release-and-readmit cycle by bringing the auditor back:

```
kubectl apply -f manifests/04-archive-reader.yaml
kubectl -n kcna-lab18 wait --for=condition=Ready pod/archive-auditor --timeout=120s
kubectl -n kcna-lab18 logs archive-auditor
```

```

pod/archive-auditor created
pod/archive-auditor condition met
--- provenance written by the (now deleted) seeder Pod ---
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
--- archive row count as read back ---
rows_read=30
--- can this auditor modify the archive? ---
read-only mount refused the write (expected)

```

`sealed_utc` is the **original seal time from Step 5**, unchanged. The claim was deleted, the volume was Released, an administrator re-admitted it, and not one byte moved. That two-step — release, then a human deliberately re-admits the volume — is the entire value of Retain.

If the patch reports `remove operation does not apply: doc is missing path`, the field was already absent; the operation is idempotent in effect. Drop the missing path from the JSON patch and re-run.

Contrast with Delete: had you deleted `render-cache`'s PVC, the local-path provisioner would have removed the directory and the PV, and the bytes would be gone with no recovery path.

Lab 18 · Verification

Run these checks **after Step 12**, with `archive-auditor` and `deployment/render-cache` both running.

```

bash verification/checks.sh

== Lab 18 verification — namespace kcna-lab18 ==
[PASS] namespace kcna-lab18 exists
[PASS] PersistentVolume tracklane-archive-pv exists and is cluster-scoped
[PASS] tracklane-archive-pv capacity=1Gi accessMode=ReadWriteOnce reclaim=Retain
[PASS] tracklane-archive-pv is pre-bound via claimRef to kcna-lab18/archive-claim
[PASS] pvc/archive-claim is Bound to tracklane-archive-pv
[PASS] archive-claim requested 512Mi but was granted the PV's full 1Gi
[PASS] the archive on the PV holds 30 rows (matches data/manifest-archive.csv)
[PASS] PROVENANCE on the PV was written by a Pod that no longer exists
[PASS] storageclass tracklane-retain exists with reclaimPolicy=Retain
[PASS] tracklane-retain uses volumeBindingMode=WaitForFirstConsumer
[PASS] pvc/render-cache is Bound to a dynamically provisioned pvc-* volume
[PASS] the dynamic PV inherited reclaimPolicy=Delete from storageclass standard
[PASS] deployment/render-cache is available and mounts pvc/render-cache
[PASS] /cache/starts.log survived a rollout restart (2+ start records)
-----
14 passed, 0 failed

```

Full transcript: `verification/expected-output.md`.

Lab 18 · Failure injection

The ticket: *"PLAT-4471 — archive PVC Pending for 20 minutes, blocking the compliance cutover."*

```

kubectl apply -f manifests/08-pvc-typo-broken.yaml
persistentvolumeclaim/archive-claim-typo created
kubectl -n kcna-lab18 get pvc

```

NAME	STATUS	VOLUME	CAPACITY	
ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE	
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO
tracklane-manual	<unset>	21m		
archive-claim-typo	Pending			
tracklane-manul	<unset>	9s		
render-cache	Bound	pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54	256Mi	RWO
standard	<unset>	11m		

get gives you Pending and nothing else. **Always escalate to describe:**

```
kubectl -n kcna-lab18 describe pvc archive-claim-typo
```

Name: archive-claim-typo
 Namespace: kcna-lab18
 StorageClass: tracklane-manul
 Status: Pending
 Volume:
 Labels: app.kubernetes.io/part-of=tracklane
 kcna.tertiaryinfotech.com/intent=failure-injection
 kcna.tertiaryinfotech.com/lab=18
 Annotations: <none>
 Finalizers: [kubernetes.io/pvc-protection]
 Capacity:
 Access Modes:
 VolumeMode: Filesystem
 Used By: <none>
 Events:

Type	Reason	Age	From	Message
Warning	ProvisioningFailed	4s (x3 over 18s)	persistentvolume-controller	storageclass.storage.k8s.io "tracklane-manul" not found

Diagnosis, step by step:

1. Type: Warning — compare with Step 9's Normal / WaitForFirstConsumer. This one is a genuine fault.
2. Reason: ProvisioningFailed from persistentvolume-controller.
3. The message is unambiguous: storageclass.storage.k8s.io "tracklane-manul" not found.
4. Confirm against reality:

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	
ALLOWVOLUMEEXPANSION	AGE			
standard (default)	rancher.io/local-path	Delete	WaitForFirstConsumer	false
71m				
tracklane-retain	rancher.io/local-path	Retain	WaitForFirstConsumer	false
19m				

tracklane-manul is not there. tracklane-manual is not there either — because for *static* provisioning the class name is just a **label used for matching**, and no StorageClass object needs to exist for it. Which raises the subtle question:

Why did archive-claim (class tracklane-manual, also not a real StorageClass) bind, while archive-claim-typo (class tracklane-manul) failed?

Because a PV exists with `storageClassName: tracklane-manual`. The controller's algorithm is: look for an unbound PV whose class, access modes, volumeMode and capacity satisfy the claim; if none exists **and** a StorageClass of that name exists, ask its provisioner. `tracklane-manul`

matches no PV *and* names no StorageClass, so both paths fail and the controller reports the second failure — the missing class. One-character typos in `storageClassName` are the single most common cause of an indefinitely Pending PVC.

Fix and confirm (do not edit the broken file; a PVC's `storageClassName` is immutable, so you must replace the object):

```
kubectl -n kcna-lab18 delete pvc archive-claim-typos
persistentvolumeclaim "archive-claim-typos" deleted
```

Prove the immutability claim for yourself first, if you like:

```
kubectl -n kcna-lab18 patch pvc archive-claim -p '{"spec":
{"storageClassName":"standard"}}'

The PersistentVolumeClaim "archive-claim" is invalid: spec: Forbidden: spec is immutable
after creation except resources.requests and volumeAttributesClassName for bound claims
```

A second, more subtle Pending to recognise. A claim can also hang because *nothing consumes it*. Compare the two events side by side — this is the single most useful diagnostic habit in Kubernetes storage:

```
kubectl -n kcna-lab18 get events --field-selector
involvedObject.kind=PersistentVolumeClaim \
--sort-by=.lastTimestamp -o custom-
columns='TYPE:.type,REASON:.reason,OBJECT:.involvedObject.name,MESSAGE:.message'
```

TYPE	REASON	OBJECT	MESSAGE
Normal	WaitForFirstConsumer	render-cache	waiting for first consumer to be created before binding
Normal	ProvisioningSucceeded	render-cache	Successfully provisioned volume
		pvc-8b2e17d4-05c9-4f36-alb7-e39d602c8a54	
Warning	ProvisioningFailed	archive-claim-typos	storageclass.storage.k8s.io "tracklane-manul" not found

Normal = wait. Warning = act.

Lab 18 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
PVC Pending, event Normal / WaitForFirstConsumer	The StorageClass uses <code>volumeBindingMode: WaitForFirstConsumer</code> and no Pod consumes the claim yet	<code>kubectl -n kcna-lab18 describe pvc <name> tail -6</code>	Nothing is broken. Create the Pod/Deployment that mounts the claim
PVC Pending, event Warning / ProvisioningFailed ... storageclass ... not found	<code>storageClassName</code> typo, or the class was never created	<code>kubectl get storageclass</code>	Delete and recreate the PVC with the correct class — <code>storageClassName</code> is immutable
PVC Pending with no events at all	Static provisioning: no PV matches on class, capacity, access mode or volumeMode	<code>kubectl get pv</code> then compare each field	Create a matching PV, or point the claim at a real StorageClass
PVC Pending, a matching PV exists but stays Released	The PV's <code>claimRef.uid</code> still names a deleted claim	<code>kubectl get pv <pv> -o jsonpath='{.spec.claimRef}'</code>	Clear <code>claimRef.uid</code> (and <code>resourceVersion</code>) with a JSON patch, as in Step 12

Symptom	Likely cause	Diagnostic command	Fix
Pod ContainerCreating, event Multi-Attach error for volume ... Volume is already exclusively attached to one node	An RWO volume is being pulled to a second node during a rolling update	kubectl -n kcna-lab18 describe pod <pod>	Set strategy.type: Recreate, or move to an RWX-capable driver
PVC deleted but stuck Terminating	The kubernetes.io/pvc-protection finalizer: a Pod still references the claim	kubectl -n kcna-lab18 describe pvc <name> grep -A2 Used	Delete the consuming Pods. Do not force-remove the finalizer — that orphans the volume
kubectl delete pvc destroyed the data	The StorageClass reclaim policy is Delete	kubectl get pv <pv> -o jsonpath='{.spec.persistentVolumeReclaimPolicy}'	Use a Retain class for anything with a retention obligation. There is no undo
PV shows Available but a same-class claim will not bind	Access mode or volumeMode mismatch, or the PV's claimRef reserves it for another claim	kubectl describe pv <pv>	Align accessModes / volumeMode, or target the reserved claim name exactly

Lab 18 · Cleanup

This lab created **two cluster-scoped objects**, which a namespace delete cannot remove. Delete them by explicit name — never with a label selector or an unscoped `kubectl delete pv --all`.

Order matters. Delete the namespace first so the claims release their volumes, then the PV, then the StorageClass.

```
kubectl delete namespace kcna-lab18 --wait=true
namespace "kcna-lab18" deleted
```

The standard-class PV for `render-cache` is reclaimed automatically because its policy is `Delete`:

```
kubectl get pv | grep kcna-lab18 || echo "no dynamically provisioned volumes remain"
tracklane-archive-pv    1Gi      RWO      Retain    Released    kcna-lab18/archive-claim
tracklane-manual        <unset>   38m
```

Only the `Retain` volume survives — precisely as designed. Remove it by name:

```
kubectl delete persistentvolume tracklane-archive-pv
persistentvolume "tracklane-archive-pv" deleted
kubectl delete storageclass tracklane-retain
storageclass.storage.k8s.io "tracklane-retain" deleted
```

Confirm the cluster is back to its original storage configuration:

```
kubectl get storageclass
kubectl get pv
kubectl get namespace kcna-lab18

NAME                                PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION    AGE
standard (default)      rancher.io/local-path Delete            WaitForFirstConsumer false
81m
No resources found
Error from server (NotFound): namespaces "kcna-lab18" not found
```

The directory `/tmp/tracklane-archive` still exists inside the kind node container. Deleting a `Retain` PV releases the Kubernetes object but never touches the backing store — reclaiming the actual bytes is a deliberate administrative act, and on this cluster it happens when you `kind delete cluster`.

Lab 18 · What you learned

- **PV and PVC split supply from demand.** A `PersistentVolume` is cluster-scoped and belongs to whoever runs the cluster; a `PersistentVolumeClaim` is namespaced and belongs to the workload. The Pod spec only ever names the claim, which is why the same Deployment YAML runs on kind, EKS and on-prem Ceph unchanged.
- **Binding is one-to-one, "at least as large as requested", and permanent.** The controller records the claim's UID in `spec.claimRef.uid`; a recreated claim with the same name gets a new UID and will not re-bind.
- **Access modes are a property of the driver, not a wish.** `RWO` means one *node*, not one *Pod* — `ReadWriteOncePod` (stable v1.29) is the mode that means one Pod. On single-node kind with `local-path`, `RWO` is all you can honestly have.
- **Reclaim policy is a data-retention decision, not a technical detail.** `Delete` destroys the backing store with the claim; `Retain` parks the PV in `Released` and demands a human to re-admit it. Map it from your retention policy, as `data/retention-policy.csv` does.
- **A `StorageClass` is a named profile over a provisioner** — driver, parameters, reclaim policy, binding mode, expansion. Two classes can share one driver and differ only in lifecycle, which is exactly what `tracklane-retain` demonstrates.
- **`WaitForFirstConsumer` produces a `Pending` PVC on purpose.** `Normal` / `WaitForFirstConsumer` means wait; `Warning` / `ProvisioningFailed` means act. Learning to read that distinction from `describe pvc` is the single highest-value storage debugging skill.
- **CSI is the plugin boundary.** In-tree drivers are gone; every real driver is out-of-tree and speaks the CSI gRPC contract. kind's `local-path-provisioner` is not CSI, which is why `kubectl get csidrivers` is empty — a useful reminder that this cluster teaches the API, not production durability.

Carry into Lab 19: a Deployment shares one PVC across all its replicas. A `StatefulSet` cannot work that way — each replica needs *its own* volume, created automatically and keyed to its ordinal. That is `volumeClaimTemplates`.

Lab 18 · Further reading

- Kubernetes documentation — *Persistent Volumes* (the authoritative reference for phases, binding, reclaiming and access modes): <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>
- Kubernetes documentation — *Storage Classes*:
<https://kubernetes.io/docs/concepts/storage/storage-classes/>
- Kubernetes documentation — *Dynamic Volume Provisioning*:
<https://kubernetes.io/docs/concepts/storage/dynamic-provisioning/>
- Kubernetes documentation — *Volume Binding Mode*:
<https://kubernetes.io/docs/concepts/storage/storage-classes/#volume-binding-mode>

- Kubernetes blog — *Kubernetes v1.29: ReadWriteOncePod access mode goes GA*: <https://kubernetes.io/blog/2023/12/18/read-write-once-pod-access-mode-ga/>
- Container Storage Interface specification: <https://github.com/container-storage-interface/spec/blob/master/spec.md>
- kind documentation — *Persistent Volumes* and the bundled local-path-provisioner: <https://kind.sigs.k8s.io/docs/user/local-registry/> and <https://github.com/rancher/local-path-provisioner>
- CNCF KCNA Curriculum — *Container Orchestration* → *Storage*: <https://github.com/cncf/curriculum>

Lab 19 — StatefulSets, Headless Services and Stable Identity

Field	Value
Lab ID	Lab 19
Title	StatefulSets, Headless Services and Stable Identity
Day / Topic	Day 4 · Storage
Duration	50 minutes
Namespace	kcna-lab19
Learning outcome	LO4 — Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
Ability	A4 Prepare a technical blueprint for a solution in a given area
Knowledge	K6 Technical blueprint design and construction process
Deck slide	Slide 411
Repository path	courseware/labs/lab-19-statefulset-headless/

Goal. Give the TrackLane depot ledger stable per-replica identity — ordinal Pod names, per-Pod DNS through a headless Service, and one automatically created PersistentVolumeClaim per ordinal — then prove that identity survives Pod deletion and that the volumes deliberately outlive the StatefulSet.

What you will produce:

- A three-replica StatefulSet whose Pods each serve a different ledger shard from their own PVC, resolvable at `<pod>.<svc>.<ns>.svc.cluster.local`.
- Evidence of ordered creation and reverse-ordered termination, and of a deleted Pod reattaching to its original volume with an incremented mount counter.
- A diagnosed NXDOMAIN caused by a StatefulSet whose serviceName names a headless Service that was never created.

Lab 19 · Objective

By the end of this lab you will be able to:

1. **State the four guarantees** a StatefulSet gives that a Deployment does not — stable ordinal names, stable per-Pod DNS, stable per-Pod storage, and ordered lifecycle — and choose correctly between the two controllers.
2. **Create a headless Service** (`clusterIP: None`) and explain what CoreDNS publishes for it that it does not publish for a ClusterIP Service.
3. **Resolve and address** an individual replica at `<pod>.<svc>.<ns>.svc.cluster.local`.
4. **Use `volumeClaimTemplates`** to have the controller create one PersistentVolumeClaim per ordinal, and demonstrate that a deleted Pod is reattached to *its own* original volume.
5. **Observe** ordered creation ($0 \rightarrow 1 \rightarrow 2$) and reverse-ordered termination ($2 \rightarrow 1 \rightarrow 0$), and switch that behaviour off with `podManagementPolicy: Parallel`.
6. **Explain** why PVCs created from `volumeClaimTemplates` are deliberately *not* deleted with the StatefulSet, and clean them up safely inside the lab namespace.
7. **Diagnose** an NXDOMAIN caused by a missing headless Service, using `nslookup` and `kubectl get endpointslices`.

Lab 19 · Prerequisites

- Single-node **kind** cluster, **Kubernetes v1.30+**, with the default `standard` StorageClass present (verify with `kubectl get sc`).
- **Labs 17 and 18 completed.** You need to be comfortable with PVC binding and `WaitForFirstConsumer` before you meet `volumeClaimTemplates`.
- Terminal at this lab folder:

```
cd courseware/labs/lab-19-statefulset-headless
pwd
.../courseware/labs/lab-19-statefulset-headless
```

Lab 19 · Scenario

Kallang Freight's **TrackLane** platform keeps a *depot ledger*: an append-only record of stock movements, sharded three ways so that each of the three physical depots owns its own slice.

The platform team originally shipped it as a Deployment with three replicas and one shared PVC. Two things went wrong in the first week:

1. All three replicas wrote to the same volume and corrupted each other. (You now know why: a `ReadWriteOnce` volume permits multiple Pods *on the same node* to mount it read-write. It is not a lock.)
2. The reconciliation job needs to talk to **shard 1 specifically** to replay a movement. With a Deployment there is no way to name a particular replica — the Service load-balances, and the Pod names are random hashes that change on every rollout.

The fix is a **StatefulSet**: three members with permanent names `depot-ledger-0`, `-1`, `-2`, each with its own volume, each individually addressable by DNS. This lab builds it and then proves each guarantee holds.

Lab 19 · Step-by-step procedure

Step 1 — Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab19 created
```

Two datasets drive this lab. First, the roster that maps an ordinal to a depot:

```
cat data/depot-roster.csv
ordinal,depot_code,depot_name,region,berths,ledger_shard
0,depot-a,Kallang Basin Depot,SG-Central,4,shard-0
1,depot-b,Tuas Mega Depot,SG-West,9,shard-1
2,depot-c,Changi Air Depot,SG-East,6,shard-2
```

Second, the ledger itself. Count the entries per shard — these are the numbers each Pod must independently produce:

```
for s in shard-0 shard-1 shard-2; do
  printf '%s = %s entries\n' "$s" "$(grep -c ",$s," data/ledger-entries.csv)"
done
```

```
shard-0 = 9 entries
shard-1 = 8 entries
shard-2 = 7 entries
```

Load both into one ConfigMap:

```
kubectl -n kcna-lab19 create configmap tracklane-ledger-seed \
  --from-file=depot-roster.csv=data/depot-roster.csv \
  --from-file=ledger-entries.csv=data/ledger-entries.csv
configmap/tracklane-ledger-seed created
```

Step 2 — Create the headless Service first

Order matters. The StatefulSet controller will not create this for you, and will not complain if it is absent.

```
kubectl apply -f manifests/01-headless-service.yaml
service/depot-ledger created
kubectl -n kcna-lab19 get service depot-ledger
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
depot-ledger	ClusterIP	None	<none>	80/TCP	5s

CLUSTER-IP: None. That single value is what makes this Service headless:

	ClusterIP Service	Headless Service (clusterIP: None)
Virtual IP allocated	Yes	No
kube-proxy rules programmed	Yes (iptables/IPVS/nftables DNAT)	No
DNS for the Service name	One A record — the VIP	One A record per Ready backing Pod
Per-Pod DNS record	No	Yes, when a StatefulSet sets <code>serviceName</code>
Load balancing	kube-proxy, per connection	Client's choice (or DNS round-robin)
Typical use	Stateless web tier	Databases, queues, quorum systems, peer discovery

Step 3 — Create the StatefulSet and watch it come up in order

Before applying, read the five identity-defining fields:

```
grep -n 'serviceName\|replicas:\|podManagementPolicy\|whenDeleted\|whenScaled'
manifests/03-statefulset.yaml
30:  serviceName: depot-ledger
31:  replicas: 3
34:  podManagementPolicy: OrderedReady
47:    whenDeleted: Retain
48:    whenScaled: Retain
```

Open a **second terminal** and start a watch:

```
kubectl -n kcna-lab19 get pods -w
```

Back in the first terminal:

```
kubectl apply -f manifests/03-statefulset.yaml
statefulset.apps/depot-ledger created
```

The watch terminal shows creation **strictly in ordinal order**, each Pod waiting for its predecessor to be Ready:

NAME	READY	STATUS	RESTARTS	AGE
depot-ledger-0	0/1	Pending	0	0s
depot-ledger-0	0/1	Init:0/1	0	2s
depot-ledger-0	0/1	PodInitializing	0	6s
depot-ledger-0	1/1	Running	0	9s
depot-ledger-1	0/1	Pending	0	0s
depot-ledger-1	0/1	Init:0/1	0	2s
depot-ledger-1	1/1	Running	0	8s
depot-ledger-2	0/1	Pending	0	0s
depot-ledger-2	0/1	Init:0/1	0	2s
depot-ledger-2	1/1	Running	0	8s

Stop the watch with Ctrl-C. Then:

```
kubectl -n kcna-lab19 rollout status statefulset/depot-ledger --timeout=240s
partitioned roll out complete: 3 new pods have been updated...
kubectl -n kcna-lab19 get statefulset,pods
```

NAME	READY	AGE
statefulset.apps/depot-ledger	3/3	52s

NAME	READY	STATUS	RESTARTS	AGE
pod/depot-ledger-0	1/1	Running	0	52s
pod/depot-ledger-1	1/1	Running	0	43s
pod/depot-ledger-2	1/1	Running	0	34s

Look at the Pod names. depot-ledger-0, -1, -2 — no random hash. Compare with a Deployment, whose Pods are named <deploy>-<replicaset-hash>-<pod-hash> and change identity on every rollout.

Step 4 — One PersistentVolumeClaim per ordinal

```
kubectl -n kcna-lab19 get pvc
```

NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
ledger-depot-ledger-0	Bound	pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3	128Mi
RWO	standard	<unset>	61s
ledger-depot-ledger-1	Bound	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771	128Mi
RWO	standard	<unset>	52s
ledger-depot-ledger-2	Bound	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28	128Mi
RWO	standard	<unset>	43s

You did not write these. The StatefulSet controller created one PVC per ordinal from the single volumeClaimTemplates entry, using the naming rule:

```
<volumeClaimTemplate.metadata.name>-<statefulset.metadata.name>-<ordinal>
ledger - depot-ledger - 0
```

Three distinct claims, three distinct dynamically provisioned PVs. This is precisely what a Deployment cannot do — its Pod template names one PVC, and every replica mounts that same one.

Confirm each Pod is wired to its own claim:

```
kubectl -n kcna-lab19 get pods -o custom-columns=\
'POD:.metadata.name,CLAIM:.spec.volumes[?
(@.name=="ledger")].persistentVolumeClaim.claimName'
```

POD	CLAIM
depot-ledger-0	ledger-depot-ledger-0

```
depot-ledger-1    ledger-depot-ledger-1
depot-ledger-2    ledger-depot-ledger-2
```

And that each `initContainer` picked up a different shard:

```
for i in 0 1 2; do
  printf 'depot-ledger-%s: ' "$i"
  kubectl -n kcna-lab19 logs "depot-ledger-$i" -c seed-shard
done

depot-ledger-0: ordinal=0 shard=shard-0 entries=9 mounts=1
depot-ledger-1: ordinal=1 shard=shard-1 entries=8 mounts=1
depot-ledger-2: ordinal=2 shard=shard-2 entries=7 mounts=1
```

9, 8, 7 — exactly the per-shard counts you computed from `data/ledger-entries.csv` in Step 1.

Each replica read the same `ConfigMap`, derived a *different* answer from its *own ordinal*, and wrote it to its *own* volume.

Step 5 — Address an individual replica by DNS

Start the in-cluster client:

```
kubectl apply -f manifests/04-ledger-client.yaml
kubectl -n kcna-lab19 wait --for=condition=Ready pod/ledger-client --timeout=90s

pod/ledger-client created
pod/ledger-client condition met
```

Resolve the headless `Service` name:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger.kcna-lab19.svc.cluster.local

Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.52
Name:   depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
Name:   depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.56
```

Three addresses for one name — the Pod IPs themselves. No VIP anywhere. Now resolve a **single member**:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local

Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
```

One name, one address, permanently attached to ordinal 1. Decompose it:

```
depot-ledger-1 . depot-ledger . kcna-lab19 . svc . cluster.local
pod name      service name  namespace      cluster domain
```

Now *use it* — fetch the ledger page from a specific replica:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  wget -qO- http://depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local/
```



```
<h1>TrackLane depot ledger</h1>
<p>pod: depot-ledger-1</p>
<p>ordinal: 1</p>
<p>depot_code: depot-b</p>
<p>depot_name: Tuas Mega Depot</p>
<p>region: SG-West</p>
<p>ledger_shard: shard-1</p>
<p>shard_entries: 8</p>
<p>volume_mount_count: 1</p>
```

Loop over all three to see three genuinely different answers:

```
for i in 0 1 2; do
  kubectl -n kcna-lab19 exec ledger-client -- \
    wget -qO- "http://depot-ledger-$i.depot-ledger.kcna-lab19.svc.cluster.local/" \
    | grep -E 'pod:|depot_name:|shard_entries:'
done

<p>pod: depot-ledger-0</p>
<p>depot_name: Kallang Basin Depot</p>
<p>shard_entries: 9</p>
<p>pod: depot-ledger-1</p>
<p>depot_name: Tuas Mega Depot</p>
<p>shard_entries: 8</p>
<p>pod: depot-ledger-2</p>
<p>depot_name: Changi Air Depot</p>
<p>shard_entries: 7</p>
```

This is the reconciliation job's problem solved. It can now say "replay into shard 1" and reach exactly one process.

Step 6 — Contrast with a normal ClusterIP Service

```
kubectl apply -f manifests/02-clusterip-service.yaml
service/depot-ledger-vip created
kubectl -n kcna-lab19 get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
depot-ledger        ClusterIP    None          <none>         80/TCP     6m
depot-ledger-vip    ClusterIP    10.96.184.207 <none>         80/TCP     4s

kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger-vip.kcna-lab19.svc.cluster.local
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:      depot-ledger-vip.kcna-lab19.svc.cluster.local
Address: 10.96.184.207
```

One address, and it is not a Pod. It is the VIP; kube-proxy DNATs it to a backend of its choosing. Prove that you cannot control which replica answers:

```
for n in 1 2 3 4 5 6; do
  kubectl -n kcna-lab19 exec ledger-client -- \
    wget -qO- http://depot-ledger-vip.kcna-lab19.svc.cluster.local/ | grep 'pod:'
done

<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-1</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-1</p>
```

Your sequence will differ and is **not** round-robin — kube-proxy in iptables mode picks a backend at random per connection. That unpredictability is exactly right for a stateless tier and exactly wrong for a ledger shard.

Both Services select the same Pods, which you can see in the EndpointSlices:

```
kubectl -n kcna-lab19 get endpointslices \
  -o custom-columns='SLICE:.metadata.name,SERVICE:.metadata.labels.kubernetes\.io/service-name,ADDRESSES:.endpoints[*].addresses'
```

SLICE	SERVICE	ADDRESSES
depot-ledger-jm4x9	depot-ledger	10.244.0.52,10.244.0.54,10.244.0.56
depot-ledger-vip-t7q2b	depot-ledger-vip	10.244.0.52,10.244.0.54,10.244.0.56

discovery.k8s.io/v1 **EndpointSlice** is the current API for Service backends. The legacy v1 Endpoints API was deprecated in Kubernetes v1.33 — `kubectl get endpointslices` is what you should reach for now.

Step 7 — Prove identity survives Pod deletion

Record the current state of ordinal 1:

```
kubectl -n kcna-lab19 get pod depot-ledger-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
depot-ledger-1	1/1	Running	0	9m	10.244.0.54	kcna-control-plane
<none>	<none>					

Delete it:

```
kubectl -n kcna-lab19 delete pod depot-ledger-1 --wait=true
pod "depot-ledger-1" deleted

kubectl -n kcna-lab19 wait --for=condition=Ready pod/depot-ledger-1 --timeout=180s
kubectl -n kcna-lab19 get pod depot-ledger-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
depot-ledger-1	1/1	Running	0	14s	10.244.0.61	kcna-control-plane
<none>	<none>					

Compare carefully:

Property	Before	After	Stable?
Pod name	depot-ledger-1	depot-ledger-1	Yes
Pod IP	10.244.0.54	10.244.0.61	No — and it does not matter
DNS name	depot-ledger-1.depot-ledger...	same	Yes
PVC	ledger-depot-ledger-1	same	Yes
Pod UID	old	new	No — it is a different Pod object

The DNS record followed the ordinal to the new IP:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local | tail -2
```

```
Name:      depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.61
```

And — the decisive evidence — the volume was **reattached, not recreated**:

```
kubectl -n kcna-lab19 logs depot-ledger-1 -c seed-shard
ordinal=1 shard=shard-1 entries=8 mounts=2

kubectl -n kcna-lab19 exec depot-ledger-1 -c web -- cat /usr/share/nginx/html/index.html |
grep volume_mount_count

<p>volume_mount_count: 2</p>
```

mounts=2. The `mounts.log` file on the PVC now has two lines: one written by the Pod you deleted, one by its replacement. The replacement Pod found the old Pod's data waiting for it. Confirm:

```
kubectl -n kcna-lab19 exec depot-ledger-1 -c web -- \
  sh -c 'cat /usr/share/nginx/html/mounts.log'

mounted depot-ledger-1 2026-09-05T06:31:12Z
mounted depot-ledger-1 2026-09-05T06:40:47Z
```

Two different timestamps, one volume. That is stable per-Pod storage.

Step 8 — Ordered termination and scaling

Scale down to 1 and watch the order. In your second terminal:

```
kubectl -n kcna-lab19 get pods -w
```

Then:

```
kubectl -n kcna-lab19 scale statefulset depot-ledger --replicas=1
statefulset.apps/depot-ledger scaled
```

The watch shows **reverse** ordinal order — highest first:

depot-ledger-2	1/1	Terminating	0	14m
depot-ledger-2	0/1	Terminating	0	14m
depot-ledger-2	0/1	Completed	0	14m
depot-ledger-1	1/1	Terminating	0	4m
depot-ledger-1	0/1	Terminating	0	4m
depot-ledger-1	0/1	Completed	0	4m

Ctrl-C the watch.

```
kubectl -n kcna-lab19 get pods

NAME           READY   STATUS    RESTARTS   AGE
depot-ledger-0 1/1     Running   0           15m
ledger-client  1/1     Running   0           9m
```

Why reverse order matters. In a quorum system (etcd, ZooKeeper, a Raft group) the members joined in ascending order. Removing the *lowest* ordinal first would take out the bootstrap/seed member while the others still depend on it. Removing the highest first preserves quorum at every intermediate step.

Now the critical observation:

```
kubectl -n kcna-lab19 get pvc

NAME           STATUS    VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
ledger-depot-ledger-0  Bound      pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3  128Mi      RWO
```

standard	<unset>		15m		
ledger-depot-ledger-1	Bound	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771	128Mi	RWO	
standard	<unset>		15m		
ledger-depot-ledger-2	Bound	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28	128Mi	RWO	
standard	<unset>		15m		

Still three PVCs, though only one Pod remains. This is `persistentVolumeClaimRetentionPolicy.whenScaled: Retain` — the default — and it is deliberate: scaling down is usually temporary, and silently destroying a database replica's data because someone typed `--replicas=1` would be catastrophic.

Scale back up and watch shard 2's data come back untouched:

```
kubectl -n kcna-lab19 scale statefulset depot-ledger --replicas=3
kubectl -n kcna-lab19 rollout status statefulset/depot-ledger --timeout=240s
kubectl -n kcna-lab19 logs depot-ledger-2 -c seed-shard

statefulset.apps/depot-ledger scaled
partitioned roll out complete: 3 new pods have been updated...
ordinal=2 shard=shard-2 entries=7 mounts=2
```

`mounts=2` for ordinal 2 as well — it re-adopted the volume it had before the scale-down.

The field to know. `spec.persistentVolumeClaimRetentionPolicy` has two independent keys:

Key	Retain (default)	Delete
<code>whenDeleted</code>	PVCs survive deletion of the StatefulSet	PVCs are garbage-collected with the StatefulSet
<code>whenScaled</code>	PVCs of removed ordinals survive	PVCs of removed ordinals are deleted on scale-down

The field reached beta and default-on in v1.27 and GA in v1.32. On an older cluster it is ignored and you get `Retain/Retain` regardless — which is why explicitly writing the defaults, as this manifest does, is good practice.

Step 9 — *podManagementPolicy: Parallel*

`OrderedReady` costs startup time proportional to the number of replicas. When members are independent, use `Parallel`.

```
kubectl apply -f manifests/06-parallel-indexer.yaml

service/lane-indexer created
statefulset.apps/lane-indexer created

kubectl -n kcna-lab19 get pods -l app.kubernetes.io/name=lane-indexer
```

NAME	READY	STATUS	RESTARTS	AGE
lane-indexer-0	0/1	ContainerCreating	0	2s
lane-indexer-1	0/1	ContainerCreating	0	2s
lane-indexer-2	0/1	ContainerCreating	0	2s

All three at AGE 2s — created simultaneously, not sequentially. Compare Step 3, where the ages were staggered by ~9 seconds each.

The identity guarantees are unchanged:

```
kubectl -n kcna-lab19 wait --for=condition=Ready pod -l app.kubernetes.io/name=lane-indexer --timeout=180s
```

```
kubectl -n kcna-lab19 exec ledger-client -- \
  wget -qO- http://lane-indexer-2.lane-indexer.kcna-lab19.svc.cluster.local/

pod/lane-indexer-0 condition met
pod/lane-indexer-1 condition met
pod/lane-indexer-2 condition met
lane-indexer ordinal 2 ready
```

Note also that `lane-indexer` has **no `volumeClaimTemplates` at all**:

```
kubectl -n kcna-lab19 get pvc -l app.kubernetes.io/name=lane-indexer

No resources found in kcna-lab19 namespace.
```

A `StatefulSet` is fundamentally about **identity**, not storage. Per-ordinal volumes are the most common reason to want that identity, not a requirement of it.

Step 10 — *StatefulSet versus Deployment: the decision*

Question	Deployment	StatefulSet
Pod names	web-6d4cf56db6-x9tzq — random, changes every rollout	depot-ledger-1 — ordinal, permanent
Individually addressable by DNS?	No	Yes , via a headless Service
Storage	All replicas share whatever PVC the template names	One PVC per ordinal , auto-created
Startup / shutdown order	Arbitrary, concurrent	Ordered (<code>OrderedReady</code>) or concurrent (<code>Parallel</code>)
Rolling update order	Governed by <code>maxSurge</code> / <code>maxUnavailable</code>	Highest ordinal first; gated by <code>partition</code>
Scale-down removes	An arbitrary replica	Always the highest ordinal
Replicas interchangeable?	Yes — that is the point	No — each has a role
Use it for	Web tiers, APIs, workers, anything stateless	Databases, brokers, quorum systems, sharded stores

Choose a Deployment unless you can name a specific guarantee above that you need.

`StatefulSets` are slower to roll, harder to debug, and leave PVCs behind. In cloud-native practice, the strongest advice is stronger still: for production databases, prefer a managed service or a mature **Operator** (CloudNativePG, Strimzi, Vitess) that wraps a `StatefulSet` with backup, failover and version-upgrade logic. A raw `StatefulSet` gives you identity and storage; it gives you nothing about consensus, backup or promotion.

Lab 19 · Verification

```
bash verification/checks.sh

== Lab 19 verification - namespace kcna-lab19 ==
[PASS] namespace kcna-lab19 exists
[PASS] service/depot-ledger is headless (clusterIP: None)
[PASS] service/depot-ledger-vip has an allocated ClusterIP
[PASS] statefulset/depot-ledger has serviceName=depot-ledger
[PASS] statefulset/depot-ledger reports 3/3 ready replicas
[PASS] pods are named by ordinal: depot-ledger-0, -1, -2
[PASS] one PVC exists per ordinal from volumeClaimTemplates
[PASS] each pod mounts its own ordinal's PVC
```

```
[PASS] shard entry counts 9/8/7 match data/ledger-entries.csv
[PASS] per-pod DNS resolves depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
[PASS] headless DNS returns 3 pod addresses, VIP DNS returns 1
[PASS] depot-ledger-1 reattached its original volume (mount count >= 2)
[PASS] persistentVolumeClaimRetentionPolicy is Retain/Retain
[PASS] statefulset/lane-indexer uses podManagementPolicy=Parallel
[PASS] no container in kcna-lab19 requests privileged: true
-----
15 passed, 0 failed
```

Full transcript: [verification/expected-output.md](#).

Lab 19 · Failure injection

The ticket: "PLAT-4488 — the ghost-ledger StatefulSet is green in the dashboard but the reconciliation job cannot reach it."

```
kubectl apply -f manifests/05-broken-no-headless-service.yaml
kubectl -n kcna-lab19 rollout status statefulset/ghost-ledger --timeout=180s

statefulset.apps/ghost-ledger created
partitioned roll out complete: 1 pods have been updated...
```

Everything looks healthy:

```
kubectl -n kcna-lab19 get statefulset ghost-ledger
kubectl -n kcna-lab19 get pod ghost-ledger-0
```

NAME	READY	AGE
ghost-ledger	1/1	31s

NAME	READY	STATUS	RESTARTS	AGE
ghost-ledger-0	1/1	Running	0	31s

1/1 Running, no events, no warnings. Now try to use the identity the StatefulSet promised:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local

Server:      10.96.0.10
Address:     10.96.0.10:53

** server can't find ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local: NXDOMAIN

command terminated with exit code 1

kubectl -n kcna-lab19 exec ledger-client -- \
  wget -T 5 -qO- http://ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local/

wget: bad address 'ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local'
command terminated with exit code 1
```

Diagnosis, in order:

1. NXDOMAIN means the name does not exist — this is a **naming/DNS** fault, not a connectivity or policy fault. A NetworkPolicy block or a crashed Pod would give a timeout or a connection refusal, not NXDOMAIN.
2. The per-Pod name has the form `<pod>.<serviceName>.<ns>.svc.cluster.local`. The middle label is `ghost-ledger`. **Is there a Service by that name?**

```
kubectl -n kcna-lab19 get service ghost-ledger

Error from server (NotFound): services "ghost-ledger" not found
```

1. There it is. Confirm the StatefulSet is asking for a Service that does not exist:

```
kubectl -n kcna-lab19 get statefulset ghost-ledger -o jsonpath='{.spec.serviceName}{"\n"}'
ghost-ledger
```

1. Corroborate from the other side — no Service means no EndpointSlice, so nothing ever gets published to DNS:

```
kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=ghost-ledger
No resources found in kcna-lab19 namespace.
```

Compare with the working set, which has one:

```
kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=depot-ledger
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-ledger-jm4x9	IPv4	80	10.244.0.52,10.244.0.61,10.244.0.66	28m

Root cause. `spec.serviceName` is a *reference by name only*. Kubernetes performs **no validation** that the named Service exists, is headless, or selects these Pods. There is no event, no warning and no status condition — because from the StatefulSet controller's point of view nothing is wrong: it created the Pods it was asked for and they are Ready.

Fix. Create the missing headless Service with a selector matching the Pods:

```
kubectl apply -f - <<'YAML'
apiVersion: v1
kind: Service
metadata:
  name: ghost-ledger
  namespace: kcna-lab19
spec:
  clusterIP: None
  selector:
    app.kubernetes.io/name: ghost-ledger
  ports:
    - name: http
      port: 80
      targetPort: http
      protocol: TCP
YAML
service/ghost-ledger created

sleep 10
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local | tail -2

Name:      ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.71
```

Resolved. Nothing about the StatefulSet changed; the missing half of the pair simply appeared.

Two near-miss variants to recognise:

Variant	Symptom	Why
Service exists but has a ClusterIP (not headless)	The service name resolves to the VIP, but <code><pod>.<svc>...</code> is still NXDOMAIN	Per-Pod records are only published for headless Services
Service exists and is headless but its selector does not match	Both names NXDOMAIN; the Service shows <code><none></code> endpoints	No EndpointSlice entries means nothing to publish

Remove the injection:

```
kubectl -n kcna-lab19 delete statefulset ghost-ledger --ignore-not-found
kubectl -n kcna-lab19 delete service ghost-ledger --ignore-not-found
```

```
statefulset.apps "ghost-ledger" deleted
service "ghost-ledger" deleted
```

Lab 19 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
<code><pod>.<svc>.<ns>.svc.cluster.local</code> returns NXDOMAIN	The Service named by <code>spec.serviceName</code> does not exist, or is not headless	<code>kubectl -n kcna-lab19 get svc <serviceName> -o jsonpath='{.spec.clusterIP}'</code>	Create the Service with <code>clusterIP: None</code> and a matching selector
Service name resolves but per-Pod names do not	The Service has a ClusterIP; only headless Services get per-Pod records	<code>kubectl -n kcna-lab19 get svc -o wide</code>	Recreate the Service with <code>clusterIP: None</code> (the field is immutable)
Headless Service resolves to nothing; <code>endpointslices</code> is empty	The Service selector does not match the Pod template labels	<code>kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=<svc></code>	Align <code>service.spec.selector</code> with <code>statefulset.spec.template.metadata.labels</code>
<code>depot-ledger-1</code> never starts; <code>-0</code> is Pending	<code>OrderedReady</code> blocks on the predecessor. The real fault is on ordinal 0	<code>kubectl -n kcna-lab19 describe pod depot-ledger-0</code>	Fix ordinal 0 (usually an unbound PVC or a failing readiness probe)
Pod Pending, event <code>pod</code> has unbound immediate <code>PersistentVolumeClaims</code>	The per-ordinal PVC has not bound. On kind this is normal for a few seconds under <code>WaitForFirstConsumer</code>	<code>kubectl -n kcna-lab19 describe pvc ledger-<sts>-<n></code>	Wait; if it persists, check the StorageClass name in <code>volumeClaimTemplates</code>
PVCs still present after deleting the StatefulSet	<code>persistentVolumeClaimRetentionPolicy.whenDeleted: Retain</code> — the default and deliberate	<code>kubectl -n kcna-lab19 get pvc</code>	Delete the PVCs explicitly by name, or set <code>whenDeleted: Delete</code> if the data is genuinely disposable
Scaling down destroyed replica data	<code>whenScaled: Delete</code> was set	<code>kubectl -n kcna-lab19 get sts <name> -o jsonpath='{.spec.persistentVolumeClaimRetentionPolicy}'</code>	Use <code>whenScaled: Retain</code> for anything with real state. There is no undo
Editing <code>volumeClaimTemplates</code> is rejected	Almost all of a StatefulSet's spec is immutable after creation	<code>kubectl -n kcna-lab19 replace --force -f manifests/03-statefulset.yaml</code>	Delete the StatefulSet with <code>--cascade=orphan</code> , change it, re-apply. The PVCs are retained
Rolling update stalls part-way through	<code>updateStrategy.rollingUpdate.partition</code> is above 0, so low ordinals are held back	<code>kubectl -n kcna-lab19 get sts <name> -o jsonpath='{.spec.updateStrategy}'</code>	Lower partition to 0 once the canary ordinal is healthy

Lab 19 · Cleanup

The point of this section is that a StatefulSet does not clean up after itself.

Delete the workloads first, then look at what is left:

```
kubectl -n kcna-lab19 delete statefulset depot-ledger lane-indexer --ignore-not-found
```



```
statefulset.apps "depot-ledger" deleted
statefulset.apps "lane-indexer" deleted
```

```
kubectl -n kcna-lab19 get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ledger-client	1/1	Running	0	41m

```
kubectl -n kcna-lab19 get pvc
```

NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE			
ledger-depot-ledger-0	Bound	pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3	128Mi
standard <unset>		44m	RWO
ledger-depot-ledger-1	Bound	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771	128Mi
standard <unset>		44m	RWO
ledger-depot-ledger-2	Bound	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28	128Mi
standard <unset>		44m	RWO

All three claims survive. The StatefulSet is gone; its storage is not. This is `whenDeleted: Retain`, and it is a feature — it is what lets you delete and recreate a database StatefulSet without losing the database.

Remove them by explicit name, inside the lab namespace only:

```
kubectl -n kcna-lab19 delete pvc \
  ledger-depot-ledger-0 ledger-depot-ledger-1 ledger-depot-ledger-2
persistentvolumeclaim "ledger-depot-ledger-0" deleted
persistentvolumeclaim "ledger-depot-ledger-1" deleted
persistentvolumeclaim "ledger-depot-ledger-2" deleted
```

Never clean up a StatefulSet's storage with `kubectl delete pvc --all`. Name the claims. On a shared cluster that flag has destroyed production databases.

The dynamically provisioned PVs are reclaimed automatically, because `standard` uses `reclaimPolicy: Delete`:

```
kubectl get pv | grep kcna-lab19 || echo "all dynamically provisioned volumes reclaimed"
all dynamically provisioned volumes reclaimed
```

Now remove the namespace, which takes everything else with it:

```
kubectl delete namespace kcna-lab19 --wait=true
namespace "kcna-lab19" deleted
kubectl get namespace kcna-lab19
Error from server (NotFound): namespaces "kcna-lab19" not found
```

This lab created no cluster-scoped objects. The only PVs involved were created and reclaimed by the `standard` StorageClass on your behalf. Deleting the namespace on its own would also have removed the PVCs — the explicit delete above is there to make the retention behaviour visible, which is the lesson.

Lab 19 · What you learned

- A **StatefulSet** gives four guarantees a Deployment does not: **stable ordinal names**, **stable per-Pod DNS**, **stable per-Pod storage**, and **ordered lifecycle**. You need a StatefulSet only when you need one of those by name.

- A **headless Service** (`clusterIP: None`) has no VIP and no kube-proxy rules. CoreDNS publishes one address per Ready backing Pod, and — when a StatefulSet names it in `serviceName` — one record per Pod at `<pod>.<svc>.<ns>.svc.cluster.local`.
- **volumeClaimTemplates creates one PVC per ordinal**, named `<template>-<statefulset>-<ordinal>`. You proved a deleted Pod is reattached to its original volume by watching the mount counter go from 1 to 2 while the Pod IP changed.
- **Identity is not the IP**. `depot-ledger-1` kept its name, its DNS record and its data across deletion; only its IP and UID changed. Every stateful protocol should address the name, never the address.
- **podManagementPolicy: OrderedReady** starts `0 → 1 → 2` and terminates `2 → 1 → 0`, which is what preserves quorum in consensus systems. **Parallel** trades that ordering for startup speed and keeps every other guarantee.
- **PVCs are deliberately not garbage-collected**. `persistentVolumeClaimRetentionPolicy` (`whenDeleted / whenScaled`, defaulting to `Retain/Retain`) is the field that governs it. Cleaning up means naming the claims — never `--all`.
- **spec.serviceName is unvalidated**. A missing or non-headless Service produces a perfectly healthy 1/1 Running StatefulSet whose only symptom is NXDOMAIN. `nslookup` plus `kubectl get endpointslices` is the diagnostic pair.
- A raw StatefulSet supplies identity and storage and nothing else. Production data systems want an **Operator** or a managed service on top.

Carry into Lab 20: everything you have built on Day 4 is stored as an object in **etcd**, and scheduled by controllers you have not yet met. Lab 20 opens the control plane.

Lab 19 · Further reading

- Kubernetes documentation — *StatefulSets*: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- Kubernetes documentation — *StatefulSet Basics* tutorial: <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/>
- Kubernetes documentation — *Headless Services*: <https://kubernetes.io/docs/concepts/services-networking/service/#headless-services>
- Kubernetes documentation — *DNS for Services and Pods* (the record formats used in Step 5): <https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>
- Kubernetes documentation — *PersistentVolumeClaim retention* for StatefulSets: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/#persistentvolumeclaim-retention>
- Kubernetes blog — *Kubernetes v1.33: Endpoints deprecation* (use EndpointSlice): <https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>
- Kubernetes documentation — *Operator pattern*: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- CNCF KCNA Curriculum — *Container Orchestration* → *Storage and Kubernetes Fundamentals* → *Core Concepts*: <https://github.com/cncf/curriculum>

Lab 20 — Control Plane Anatomy and etcd Backup/Restore

Field	Value
Lab ID	Lab 20
Title	Control Plane Anatomy and etcd Backup/Restore
Day / Topic	Day 4 · Cluster Architecture, Installation & Configuration
Duration	55 minutes
Namespace	kcna-lab20
Learning outcome	LO5 — Demonstrate Kubernetes solution for a specific business problem.
Ability	A5 Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need
Knowledge	K5 Tools and techniques for solution architecture modelling
Deck slide	Slide 437
Repository path	courseware/labs/lab-20-cluster-architecture-etcd/

Goal. Open the control plane that has been running every lab so far: read the static Pod manifests the kubelet acts on, trace how apiserver, etcd, scheduler and controller-manager interact, then take and verify a real etcd snapshot to a file — and read, without executing, the destructive restore procedure.

What you will produce:

- A verified etcd snapshot file with `etcdctl snapshot save` and a snapshot status table showing hash, revision, total keys and size.
- An annotated control-plane inventory generated from the platform team's dataset, naming the exact mirror-pod objects on this node.
- A version-controlled etcd backup policy (RPO/RTO, retention, off-node copy, what a restore does not recover) applied as a cluster object and read from inside the cluster.

Lab 20 · ⚠ Safety contract — read this before Step 1

This lab looks at the machinery that keeps the classroom cluster alive. Three rules, and they are not negotiable:

1. **Every control-plane operation in Sections 4.2 – 4.9 is read-only.** `kubectl get`, `kubectl describe`, `kubectl get --raw` against health endpoints, and reading static Pod manifests. Nothing is created, patched, restarted or deleted outside `kcna-lab20`.
2. **You will take a real etcd snapshot. You will NOT restore one.** `etcdctl snapshot save` is a client read: it streams a point-in-time copy out of a *running* etcd without pausing it, locking it or modifying a byte. That is safe and you will do it. `snapshot restore` replaces the cluster's state and is **not performed in this course**.
3. **The restore procedure is a reading exercise.** It appears in `data/etcd-restore-runbook.md` and in Section 4.12 of this README, with the real commands, so you can learn them. Those blocks are fenced as `text`, not `bash`, and carry a DO NOT RUN banner. If you want to practise a restore, build your own throwaway single-node kind cluster on your own machine, with nobody else connected, and destroy it afterwards.

The same discipline applies to `kubeadm upgrade`. Section 4.13 teaches the *concepts and sequence*. **Do not upgrade the class cluster.**

Lab 20 · Objective

By the end of this lab you will be able to:

1. **Name every control-plane component**, state what it is responsible for, and explain the one path by which all of them reach cluster state.
2. **Explain static Pods**: why the kubelet runs four control-plane components from a directory on disk, and how the resulting **mirror Pods** appear in the API server.
3. **Read** `/etc/kubernetes/manifests/etcd.yaml` field by field and extract, from the manifest alone, the four inputs `etcdctl` needs.
4. **Take** an `etcd` snapshot with `etcdctl snapshot save` and **verify** it with `snapshot status`, correctly choosing between the `etcdctl` and `etcdutl` binaries for your `etcd` version.
5. **State what an `etcd` restore does not recover**, and design a backup policy with explicit RPO, RTO, retention and off-node requirements.
6. **Describe the restore and `kubeadm upgrade` procedures accurately** without having executed either against a shared cluster.
7. **Diagnose** a control-plane fault from `kubectl get --raw='/readyz?verbose'` and the kubelet's view of a static Pod.

Lab 20 · Prerequisites

- A single-node **kind** cluster on **Kubernetes v1.30+**, provisioned by `kubeadm` (kind uses `kubeadm` internally, so its control plane is laid out exactly like a real `kubeadm` cluster).
- A `kubectl` context with cluster-admin rights — you will read `kube-system` and call cluster-scoped `--raw` endpoints.
- **Optional**: `docker` access to the kind node container. Everything essential in this lab works without it; the two optional steps that use `docker exec` are marked and have `kubectl`-only alternatives.
- Labs 17–19 completed. Every object you created in those labs is sitting in the `etcd` keyspace you are about to snapshot.

```
cd courseware/labs/lab-20-cluster-architecture-etcd
pwd
.../courseware/labs/lab-20-cluster-architecture-etcd
```

Capture your node name once — nearly every command below uses it:

```
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "NODE=$NODE"
NODE=kcna-control-plane
```

Lab 20 · Scenario

Kallang Freight has hired a second platform engineer for the TrackLane team, and the handover document has one glaring gap: nobody currently on the team can explain what actually runs the cluster, and the "backup strategy" is a wiki page that says *"AWS snapshots the volume"*.

The compliance work you did in Lab 18 made this urgent. If the cluster's state is lost, the PersistentVolume **objects** vanish even though the customs archive **bytes** survive — and nobody would know which claim owned which volume.

Your task this session: produce the control-plane section of the TrackLane technical blueprint. That means (a) a component inventory that names the real objects on this cluster, (b) a *verified* etcd snapshot proving the procedure works, and (c) a written backup policy with numbers in it. You will also read the disaster-recovery runbook — and you will not run it.

Lab 20 · Step-by-step procedure

4.1 — Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab20 created
```

Look at the platform team's inventory:

```
cut -d, -f1,2,4 data/control-plane-inventory.csv | column -s, -t
component          deployment_kind  listen_ports
kube-apiserver      StaticPod        6443
etcd                 StaticPod        2379;2380
kube-controller-manager StaticPod        10257
kube-scheduler       StaticPod        10259
kubelet             SystemdUnit      10250;10248
kube-proxy          DaemonSet        10256
coredns             Deployment       53;9153
kindnet             DaemonSet        none
local-path-provisioner Deployment       none

echo "rows=$(( $(wc -l < data/control-plane-inventory.csv) - 1 ))"
echo "static_pods=$(grep -c ',StaticPod,' data/control-plane-inventory.csv)"

rows=9
static_pods=4
```

Load both datasets into the cluster:

```
kubectl -n kcna-lab20 create configmap control-plane-inventory \
  --from-file=control-plane-inventory.csv=data/control-plane-inventory.csv
kubectl -n kcna-lab20 create configmap etcd-restore-runbook \
  --from-file=etcd-restore-runbook.md=data/etcd-restore-runbook.md

configmap/control-plane-inventory created
configmap/etcd-restore-runbook created
```

4.2 — See the control plane (read-only)

```
kubectl -n kube-system get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED NODE	READINESS GATES			
coredns-7db6d8ff4d-lc4nv	1/1	Running	0	96m	10.244.0.3
kcna-control-plane	<none>	<none>			
coredns-7db6d8ff4d-w8m2j	1/1	Running	0	96m	10.244.0.2
kcna-control-plane	<none>	<none>			
etcd-kcna-control-plane	1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>			
kindnet-8xq6r	1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>			

```
kube-apiserver-kcna-control-plane      1/1      Running    0          96m      172.18.0.2
kcna-control-plane                     <none>    <none>
kube-controller-manager-kcna-control-plane 1/1      Running    0          96m      172.18.0.2
kcna-control-plane                     <none>    <none>
kube-proxy-2wr7n                       1/1      Running    0          96m      172.18.0.2
kcna-control-plane                     <none>    <none>
kube-scheduler-kcna-control-plane       1/1      Running    0          96m      172.18.0.2
kcna-control-plane                     <none>    <none>
```

Two IP ranges are visible and the difference matters:

- 172.18.0.2 — the **node's own IP**. Every control-plane component uses `hostNetwork: true`, because the API server must be reachable before the Pod network exists. Bootstrapping order forces this.
- 10.244.0.x — the Pod network. CoreDNS is an ordinary Deployment and gets an ordinary Pod IP.

Now the mechanism that ties them together:

Component	Responsibility	Talks to etcd?	Leader-elected?
etcd	Consistent, replicated key-value store. All cluster state.	is the store	Raft (its own quorum)
kube-apiserver	REST front door. AuthN → AuthZ → Admission → validate → persist → watch.	Yes — the only component that does	No (stateless, scale horizontally)
kube-scheduler	Watches <code>Pending</code> Pods, filters and scores nodes, writes a binding.	No — through the apiserver	Yes
kube-controller-manager	~40 controller loops (Deployment, ReplicaSet, Node, PV/PVC binding, ServiceAccount, EndpointSlice...).	No — through the apiserver	Yes
cloud-controller-manager	Cloud-specific loops (LoadBalancer, node lifecycle, routes). Absent on kind.	No	Yes
kubelet	Node agent. Watches for Pods bound to its node, starts containers, reports status.	No	No
kube-proxy	Programmes node dataplane rules so Service VIPs reach Pods.	No	No

The single most important architectural fact on this page: *only the API server talks to etcd.*

The scheduler does not. The controller-manager does not. Your `kubectl` does not. Every read and every write funnels through one process that enforces authentication, authorisation, admission control and validation. Break that invariant — for example by handing an application a direct etcd client certificate — and every security control in Kubernetes is bypassed at once.

Confirm the health of each component from the API's own endpoint:

```
kubectl get --raw='/readyz?verbose'
```

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
[+]poststarthook/start-apiserver-admission-initializer ok
[+]poststarthook/generic-apiserver-start-informers ok
[+]poststarthook/rbac/bootstrap-roles ok
[+]poststarthook/scheduling/bootstrap-system-priority-classes ok
[+]poststarthook/start-cluster-authentication-info-controller ok
[+]poststarthook/start-kube-apiserver-identity-lease-controller ok
[+]shutdown ok
readyz check passed
```

The exact list of checks varies by version. The two to look for are `[+]etcd ok` and the final `readyz check passed`.

4.3 — Static Pods: where four of those Pods actually come from

Pick any control-plane Pod and ask who owns it:

```
kubectl -n kube-system get pod "etcd-$NODE" \
  -o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}'
{"\n"}
Node/kcna-control-plane
```

Owned by a Node, not by a Deployment, ReplicaSet, DaemonSet or StatefulSet. That is the signature of a **mirror Pod**. Confirm it:

```
kubectl -n kube-system get pod "etcd-$NODE" \
  -o jsonpath='{.metadata.annotations>{"\n"}' | tr ',' '\n' | grep config
"kubernetes.io/config.hash":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.mirror":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.seen":"2026-09-05T03:11:02.417392514Z"
"kubernetes.io/config.source":"file"
```

`kubernetes.io/config.source: file` is the decisive field. **This Pod did not come from the API server.** The kubelet read it from a directory on disk and then created a read-only *mirror* of it in the API so that `kubectl` can see it.

Why the bootstrap works this way. The API server is itself a Pod. Something has to start it before the API server exists to be asked. That something is the kubelet, reading YAML from `--pod-manifest-path (/etc/kubernetes/manifests)` with no API server involved at all. This is also why:

- You cannot `kubectl delete` a static Pod and have it stay deleted — the kubelet recreates the mirror within seconds. To stop a static Pod you move its **file**.
- `kubectl edit` on a mirror Pod is pointless — the kubelet overwrites it from the file.
- The four control-plane components restart automatically after a node reboot with no controller involved.

List all mirror Pods on this node:

```
kubectl -n kube-system get pods \
  -o jsonpath='{range .items[?(@.metadata.annotations.kubernetes\.io/config\.source=="file")]}{.metadata.name}\n'}
etcd-kcna-control-plane
kubeapiserver-kcna-control-plane
```



```
kube-controller-manager-kcna-control-plane
kube-scheduler-kcna-control-plane
```

Four — matching `static_pods=4` from the platform team's inventory in Step 4.1.

4.4 — Read the manifest directory (optional: needs Docker)

On kind the node is a Docker container, so the kubelet's manifest directory is inside it:

```
docker exec "$NODE" ls -l /etc/kubernetes/manifests
total 16
-rw----- 1 root root 2406 Sep  5 03:10 etcd.yaml
-rw----- 1 root root 3896 Sep  5 03:10 kube-apiserver.yaml
-rw----- 1 root root 3428 Sep  5 03:10 kube-controller-manager.yaml
-rw----- 1 root root 1463 Sep  5 03:10 kube-scheduler.yaml
```

Four files, four mirror Pods. Confirm the kubelet is watching that exact path:

```
docker exec "$NODE" grep -i staticPodPath /var/lib/kubelet/config.yaml
staticPodPath: /etc/kubernetes/manifests
```

No Docker? Skip this step. The mirror Pod in the API server carries the same spec, and `kubectl -n kube-system get pod etcd-$NODE -o yaml` shows you everything you need for Step 4.5.

4.5 — Read the etcd manifest field by field

Use whichever source you have. Docker:

```
docker exec "$NODE" cat /etc/kubernetes/manifests/etcd.yaml | sed -n '1,40p'
```

...or, equivalently and with no Docker:

```
kubectl -n kube-system get pod "etcd-$NODE" -o yaml \
| grep -E 'image:|--(advertise-client-urls|cert-file|key-file|trusted-ca-file|data-dir|
listen-client-urls|listen-peer-urls|initial-cluster)='
- --advertise-client-urls=https://172.18.0.2:2379
- --cert-file=/etc/kubernetes/pki/etcd/server.crt
- --data-dir=/var/lib/etcd
- --initial-cluster=kcna-control-plane=https://172.18.0.2:2380
- --key-file=/etc/kubernetes/pki/etcd/server.key
- --listen-client-urls=https://127.0.0.1:2379,https://172.18.0.2:2379
- --listen-peer-urls=https://172.18.0.2:2380
- --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
image: registry.k8s.io/etcd:3.5.15-0
```

Read it as an operator would:

Flag	Value	What it tells you
<code>--listen-client-urls</code>	<code>https://127.0.0.1:2379,https://172.18.0.2:2379</code>	Port 2379 serves clients. 127.0.0.1 is why an in-container <code>etcdctl</code> can use the loopback endpoint.
<code>--listen-peer-urls</code>	<code>https://172.18.0.2:2380</code>	Port 2380 is member-to-member Raft traffic. Never expose it.
<code>--data-dir</code>	<code>/var/lib/etcd</code>	The key space on disk. The actual WAL and snapshots live in <code>/var/lib/etcd/member/</code> .

Flag	Value	What it tells you
<code>--trusted-ca-file</code>	<code>/etc/kubernetes/pki/etcd/ca.crt</code>	The CA <code>etcdctl</code> must present as <code>--cacert</code> .
<code>--cert-file / --key-file</code>	<code>server.crt / server.key</code>	The client credentials <code>etcdctl</code> uses as <code>--cert / --key</code> .
<code>--initial-cluster</code>	one member	A single-member Raft group. Production wants 3 or 5 — an odd number, because quorum is $\lfloor n/2 \rfloor + 1$.
<code>image</code>	<code>registry.k8s.io/etcd:3.5.15-0</code>	Note this version. It decides which binary you use in Step 4.8.

Record the image, you will need it:

```
ETCD_IMAGE=$(kubectl -n kube-system get pod "etcd-$NODE" \
  -o jsonpath='{.spec.containers[0].image}')
echo "ETCD_IMAGE=$ETCD_IMAGE"
ETCD_IMAGE=registry.k8s.io/etcd:3.5.15-0
```

Why etcd's own TLS matters. etcd has no concept of Kubernetes RBAC. Anyone holding `server.crt/server.key` can read and write the **entire** cluster keyspace — every Secret in every namespace, in plaintext unless you have configured encryption at rest. Those files are the crown jewels. This is also why etcd runs on the control-plane node with `hostNetwork` and its listener bound where only the API server can reach it.

4.6 — Watch the scheduler and controller-manager coordinate

Both are leader-elected, and they hold their leadership in a **Lease** object you can read:

```
kubectl -n kube-system get leases

NAME                                HOLDER
AGE
apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a  apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a_0f1d6c9b-7a34-
4e02-b5c8-91d47ea36f0c    99m
kube-controller-manager              kcna-control-plane_3b8e5d21-6c47-4a09-9e13-
5f0c82ab7d64                99m
kube-scheduler                       kcna-control-plane_c7d0a493-8f52-41be-b6a7-
2e9d5310cf88                99m

kubectl -n kube-system get lease kube-scheduler \
  -o jsonpath='{.spec.holderIdentity}{ " renewed: "}{.spec.renewTime}{ "\n"}'

kcna-control-plane_c7d0a493-8f52-41be-b6a7-2e9d5310cf88 renewed: 2026-09-
05T06:52:41.883412Z
```

The mechanism. In a 3-node control plane all three schedulers are running, but only the one holding this Lease acts. It renews the Lease every few seconds; if it dies, the Lease expires and another instance takes it. The Lease is an ordinary API object — so leader election is itself stored in etcd and mediated by the API server. This is why the KCNA blueprint keeps returning to the same sentence: **everything is an object, and the API server is the only door.**

Trace one full interaction end to end using the Pods you created in Lab 19 (or any Pod):

```
kubectl -n kcna-lab20 run trace-probe --image=busybox:1.36 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "trace-
probe", "image": "busybox:1.36", "command": [ "sh", "-c", "echo scheduled; sleep
60"], "resources": { "requests": { "cpu": "50m", "memory": "64Mi" }, "limits":
```

```
{ "cpu": "200m", "memory": "128Mi" }, "securityContext":
{ "allowPrivilegeEscalation": false, "privileged": false, "capabilities": { "drop": [ "ALL" ] } } } } '

pod/trace-probe created

sleep 5
kubectl -n kcna-lab20 get events --field-selector involvedObject.name=trace-probe \
-o custom-
columns='TIME:.lastTimestamp,SOURCE:.source.component,REASON:.reason,MESSAGE:.message'

TIME                SOURCE                REASON                MESSAGE
2026-09-05T06:55:12Z default-scheduler    Scheduled             Successfully assigned
kcna-lab20/trace-probe to kcna-control-plane
2026-09-05T06:55:13Z kubelet              Pulled               Container image "busybox:1.36"
already present on machine
2026-09-05T06:55:13Z kubelet              Created              Created container: trace-probe
2026-09-05T06:55:13Z kubelet              Started              Started container trace-probe
```

Read the **SOURCE** column: `default-scheduler` decided *where*; `kubelet` did *what*. Two different components, communicating only through objects in the API server. Neither ever contacted the other.

```
kubectl -n kcna-lab20 delete pod trace-probe --ignore-not-found

pod "trace-probe" deleted
```

4.7 — Generate the concrete inventory

```
kubectl apply -f manifests/01-rbac-namespace-reader.yaml
kubectl apply -f manifests/02-inventory-job.yaml

serviceaccount/cp-inventory created
role.rbac.authorization.k8s.io/cp-inventory-reader created
rolebinding.rbac.authorization.k8s.io/cp-inventory-reader created
job.batch/cp-inventory created

kubectl -n kcna-lab20 wait --for=condition=Complete job/cp-inventory --timeout=120s
kubectl -n kcna-lab20 logs job/cp-inventory

job.batch/cp-inventory condition met
TrackLane control-plane inventory for node kcna-control-plane
generated from data/control-plane-inventory.csv
=====
COMPONENT            DEPLOYMENT    OBJECT OR PATH TO INSPECT
kube-apiserver        StaticPod     kube-system/pod/kube-apiserver-kcna-control-plane
etcd                  StaticPod     kube-system/pod/etcd-kcna-control-plane
kube-controller-manager StaticPod     kube-system/pod/kube-controller-manager-kcna-
control-plane
kube-scheduler        StaticPod     kube-system/pod/kube-scheduler-kcna-control-plane
kubelet               SystemdUnit   /etc/systemd/system/kubelet.service.d/10-
kubeadm.conf
kube-proxy            DaemonSet     api-object:kube-system/daemonset/kube-proxy
coredns               Deployment    api-object:kube-system/deployment/coredns
kindnet               DaemonSet     api-object:kube-system/daemonset/kindnet
local-path-provisioner Deployment    api-object:local-path-storage/deployment/local-
path-provisioner
=====
inventory_rows=9
static_pod_components=4
namespace=kcna-lab20
NOTE: this Job has namespaced RBAC only and cannot read kube-system.
```

The generic dataset has become the exact object names for **this** cluster. Verify the least-privilege claim in that last line:

```
kubectl auth can-i list pods --namespace=kube-system \
--as=system:serviceaccount:kcna-lab20:cp-inventory
```

```
no
kubectl auth can-i list pods --namespace=kcna-lab20 \
  --as=system:serviceaccount:kcna-lab20:cp-inventory
yes
```

Your kubectl can read the control plane; the workload you deployed cannot. That asymmetry is the correct default for platform tooling.

4.8 — Take an etcd snapshot

Choosing the method — and why.

Method	Verdict
<code>kubectl -n kube-system exec</code> into the etcd Pod	This is what we use. <code>etcdctl</code> ships <i>inside</i> the etcd container image and is not installed on the kind node or on your laptop. The TLS client certificates it needs are already mounted into that container. The operation goes through the API server, so it is subject to RBAC and lands in the audit log. It works identically on kind, on kubeadm bare metal, and on any cluster where you can <code>exec</code> into <code>kube-system</code> .
<code>docker exec</code> into the kind node	Kind-specific, bypasses the Kubernetes audit trail and RBAC entirely, requires Docker on your host — and <code>etcdctl</code> is not on the node anyway, so it would not work without extra installation. We use <code>docker</code> only in Step 4.9, to copy the finished file off the node, because <code>/var/lib/etcd</code> is a <code>hostPath</code> .
Install <code>etcdctl</code> on your laptop and dial the cluster	Requires exporting etcd's server certificate and key off the node. That is the one credential you should never move. Do not do this.

Find the etcd Pod and confirm `etcdctl` is present:

```
ETCD_POD=$(kubectl -n kube-system get pods -l component=etcd \
  -o jsonpath='{.items[0].metadata.name}')
echo "ETCD_POD=$ETCD_POD"
kubectl -n kube-system exec "$ETCD_POD" -- etcdctl version

ETCD_POD=etcd-kcna-control-plane
etcdctl version: 3.5.15
API version: 3.5
```

Now take the snapshot. Read the four TLS arguments against the manifest flags from Step 4.5 — they are the same values:

```
kubectl -n kube-system exec "$ETCD_POD" -- etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/lib/etcd/kcna-lab20-snapshot.db

{"level":"info","ts":"2026-09-05T07:02:18.441Z","caller":"snapshot/
v3_snapshot.go:65","msg":"created temporary db file","path":"/var/lib/etcd/kcna-lab20-
snapshot.db.part"}
{"level":"info","ts":"2026-09-05T07:02:18.449Z","logger":"client","caller":"v3/
maintenance.go:212","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":"2026-09-05T07:02:18.449Z","caller":"snapshot/
v3_snapshot.go:73","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
```

```
{ "level": "info", "ts": "2026-09-05T07:02:18.612Z", "logger": "client", "caller": "v3/maintenance.go:220", "msg": "completed snapshot read; closing" }
{ "level": "info", "ts": "2026-09-05T07:02:18.641Z", "caller": "snapshot/v3_snapshot.go:88", "msg": "fetched snapshot", "endpoint": "https://127.0.0.1:2379", "size": "7.4 MB", "took": "now" }
{ "level": "info", "ts": "2026-09-05T07:02:18.641Z", "caller": "snapshot/v3_snapshot.go:97", "msg": "saved", "path": "/var/lib/etcd/kcna-lab20-snapshot.db" }
Snapshot saved at /var/lib/etcd/kcna-lab20-snapshot.db
```

What just happened, and why it is safe. `snapshot save` is a *client* call to the running server's Maintenance API. `etcd` streamed a consistent copy of the keyspace at the current revision. It did not pause, lock, compact or modify anything. The cluster kept serving throughout — you can prove that:

```
kubectl get --raw='/readyz' && echo
ok
```

About the destination path. `etcd`'s actual data lives in `/var/lib/etcd/member/` (`wal/` and `snap/`). Writing a file to the *parent* directory `/var/lib/etcd/` is inert — `etcd` never reads it. We use that path because it is the one directory guaranteed to be both writable inside the container and retrievable from the node, since the manifest mounts it as a `hostPath`. **In production you would mount a separate `/var/lib/etcd-backup` `hostPath`** so that backups and live data never share a directory, and you would ship the file off the node immediately. Step 4.9 does exactly that, and Step 4.11 removes the file from the data directory.

4.9 — Verify the snapshot

A snapshot you have never verified is not a backup. `snapshot status` reads the file offline — it never contacts a server.

Which binary? This depends on your `etcd` version and is a common exam and interview point:

etcd version	snapshot save	snapshot status / snapshot restore
≤ 3.4	<code>etcdctl</code>	<code>etcdctl</code>
3.5.x	<code>etcdctl</code>	<code>etcdctl</code> — still works in <code>etcdctl</code> but prints a deprecation warning
≥ 3.6	<code>etcdctl</code>	<code>etcdctl</code> only — removed from <code>etcdctl</code>

The split exists because `save` talks to a *server* (a client operation) while `status` and `restore` operate on a *file* (an offline utility operation).

Try `etcdctl` first:

```
kubectl -n kube-system exec "$ETCD_POD" -- \
  etcdctl snapshot status /var/lib/etcd/kcna-lab20-snapshot.db -w table

+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |        1187 |    7.4 MB |
+-----+-----+-----+-----+
```

If your `etcd` image is older than 3.5.x and `etcdctl` is not present, you will see:

```
OCI runtime exec failed: exec failed: unable to start container process: exec: "etcdctl":
executable file not found in $PATH: unknown
command terminated with exit code 126
```

In that case use the deprecated `etcdctl` form, which produces the same table:

```
kubectl -n kube-system exec "$ETCD_POD" -- \
  etcdctl snapshot status /var/lib/etcd/kcna-lab20-snapshot.db -w table
```

Deprecated: Use ``etcdctl snapshot status`` instead.

```
+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |       1187 |    7.4 MB |
+-----+-----+-----+-----+
```

Read the four columns as an acceptance test:

Column	Meaning	What "bad" looks like
HASH	CRC of the file	Missing / command exits non-zero → corrupt file
REVISION	etcd's global revision at snapshot time	0 → the save failed
TOTAL KEYS	Number of keys captured	Single digits on a real cluster → wrong endpoint or empty database
TOTAL SIZE	Bytes captured	A few KB → almost certainly a failed or truncated save

TOTAL KEYS on your cluster reflects everything you have created across all 20 labs so far. Confirm the file is real from the node's point of view, and copy it off (kind-specific, needs Docker):

```
docker exec "$NODE" ls -lh /var/lib/etcd/kcna-lab20-snapshot.db
-rw----- 1 root root 7.1M Sep  5 07:02 /var/lib/etcd/kcna-lab20-snapshot.db

docker cp "$NODE:/var/lib/etcd/kcna-lab20-snapshot.db" /tmp/kcna-lab20-snapshot.db
ls -lh /tmp/kcna-lab20-snapshot.db
Successfully copied 7.42MB to /tmp/kcna-lab20-snapshot.db
-rw----- 1 you  staff  7.1M  5 Sep 07:03 /tmp/kcna-lab20-snapshot.db
```

This copy step is the whole point of a backup. A snapshot sitting on the node whose disk you are protecting against is not a backup — it is a convenience. The policy in Step 4.10 makes off-node storage mandatory for exactly this reason.

4.10 — Write the backup policy down

```
kubectl apply -f manifests/03-backup-policy.yaml
configmap/etcd-backup-policy created

kubectl -n kcna-lab20 get configmap etcd-backup-policy \
  -o jsonpath='{.data.policy\.yaml}' | sed -n '1,20p'

# Kallang Freight - TrackLane etcd backup policy
# Owner: Platform Engineering. Review: quarterly.
scope:
  cluster: tracklane-prod-sgl
  control_plane: kubeadm, 3 stacked etcd members
objectives:
  rpo_minutes: 30          # max acceptable data loss
  rto_minutes: 60          # decision -> /readyz ok
```

```

snapshot:
  interval: "*/30 * * * *"
  command: "etcdctl snapshot save"           # talks to a running server
  verify_command: "etcdctl snapshot status"   # offline, reads the file
  verify_every_snapshot: true
retention:
  hourly: 48
  daily: 14
  monthly: 12
storage:
  on_node_copy: "/var/lib/etcd-backup"       # convenience only, NOT a backup
  offsite_required: true                     # object storage, different failure domain

```

Two numbers define everything else:

- **RPO (Recovery Point Objective)** — how much data you accept losing. It is **exactly your snapshot interval**, no better. A 30-minute interval means a restore can lose 30 minutes of cluster changes.
- **RTO (Recovery Time Objective)** — how long from "we decide to restore" to `readyz ok`. You cannot claim an RTO you have not measured in a timed drill.

4.11 — Read the DR runbook from inside the cluster

```

kubectl apply -f manifests/04-runbook-reader.yaml
kubectl -n kcna-lab20 wait --for=condition=Ready pod/runbook-reader --timeout=90s
kubectl -n kcna-lab20 logs runbook-reader

pod/runbook-reader created
pod/runbook-reader condition met
=== SAFETY BANNER FROM THE RUNBOOK =====
10:## 🚫 DO NOT RUN ANY COMMAND IN SECTION 3 ON A SHARED OR CLASSROOM CLUSTER
75:## 3. 🚫 RESTORE — DESTRUCTIVE. DO NOT RUN ON A SHARED CLUSTER 🚫

=== RECOVERY OBJECTIVES FROM THE BACKUP POLICY =====
  rpo_minutes: 30           # max acceptable data loss
  rto_minutes: 60           # decision -> /readyz ok
  interval: "*/30 * * * *"
  offsite_required: true    # object storage, different failure domain

=== WHAT AN etcd RESTORE DOES *NOT* RECOVER =====
### What restore does NOT recover

* Anything written after the snapshot timestamp. Your RPO is your snapshot
  interval, full stop.
* PersistentVolume **contents**. etcd stores the PV and PVC *objects*, not the
  bytes on the disk. Volume data needs its own backup — a CSI VolumeSnapshot,
  a storage-array snapshot, or an application-level dump.
* Secrets that were encrypted at rest, unless you also still hold the
  `EncryptionConfiguration` key material. Back up the encryption keys
  separately and **never** in the same blast radius as the snapshot.
* Certificates in `/etc/kubernetes/pki`. Back these up alongside the snapshot.

=== SECTION COUNT =====
runbook_headings=6
do_not_run_banners=2

```

Note the Pod that printed this: `automountServiceAccountToken: false`, no host mounts, `readOnlyRootFilesystem: true`, all capabilities dropped. It can read the destructive procedure and cannot possibly execute it. That is the design.

Tidy the snapshot out of the etcd data directory (kind-specific; skip if you have no Docker — `kind delete cluster` removes it anyway):

```
docker exec "$NODE" rm -f /var/lib/etcd/kcna-lab20-snapshot.db
docker exec "$NODE" ls /var/lib/etcd
member
```

Only `member/` remains — etcd's real data directory, untouched throughout.

4.12 — 🚫 The restore procedure (READ ONLY — DO NOT RUN)

DO NOT EXECUTE ANY COMMAND IN THIS SECTION ON THE CLASS CLUSTER. These blocks are deliberately fenced as `text`, not `bash`, so that they are not copy-runnable by habit. Running them would stop the API server and replace cluster state for every learner in the room.

To practise this for real, create your own throwaway single-node kind cluster on your own machine — `kind create cluster --name scratch` — do the drill there, then `kind delete cluster --name scratch`.

Step R1. Stop the control plane by moving the static Pod manifests aside. The kubelet watches the directory; removing a file tears its Pod down. There is no `systemctl stop kube-apiserver`, because it is not a service.

```
mkdir -p /etc/kubernetes/manifests.bak
mv /etc/kubernetes/manifests/kube-apiserver.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/etcd.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-controller-manager.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-scheduler.yaml /etc/kubernetes/manifests.bak/
```

Step R2. Restore into a NEW directory. Never restore over a live data directory.

```
etcdctl snapshot restore /var/lib/etcd-backup/snapshot-<stamp>.db \
--name=<member-name> \
--initial-cluster=<member-name>=https://<node-ip>:2380 \
--initial-cluster-token=etcd-cluster-restored \
--initial-advertise-peer-urls=https://<node-ip>:2380 \
--data-dir=/var/lib/etcd-restored
```

Step R3. Point etcd at the restored directory — either edit the `hostPath` in `etcd.yaml`, or swap the directories:

```
mv /var/lib/etcd /var/lib/etcd-preRestore
mv /var/lib/etcd-restored /var/lib/etcd
```

Step R4. Put the manifests back. The kubelet notices within seconds and recreates the Pods.

```
mv /etc/kubernetes/manifests.bak/*.yaml /etc/kubernetes/manifests/
```

Step R5. Verify.

```
kubectl get --raw='/readyz?verbose'
kubectl get nodes
kubectl get pods -A
```

The five things that make restores go wrong, and you must be able to name them:

1. **Multi-member clusters must all be restored from the same snapshot** with matching `--initial-cluster` topology. Restoring one member while the others keep their old data corrupts the Raft group.

2. `--initial-cluster-token` **must be changed** so the restored cluster cannot accidentally join the old one.
3. **RPO is the snapshot interval.** Every write after the snapshot timestamp is gone. There is no partial restore.
4. **PersistentVolume data is not in etcd.** You restore the PV and PVC *objects*; the bytes need a CSI VolumeSnapshot or an application dump. A restored cluster with intact objects pointing at wiped volumes is a very confusing outage.
5. **Certificates and encryption keys.** `/etc/kubernetes/pki` and any `EncryptionConfiguration` key material must be backed up alongside the snapshot — and stored in a *different* blast radius, or the compromise that took the cluster takes the backup too.

4.13 — 🚫 **kubeadm upgrade concepts (READ ONLY — DO NOT RUN)**

Do not upgrade the class cluster. This section is examinable as a sequence, not as an exercise.

The kubeadm upgrade order is fixed by the version-skew policy:

Order	Action	Why here
1	Upgrade the <code>kubeadm</code> binary on the first control-plane node	The tool must know the target version before it can plan
2	<code>kubeadm upgrade plan</code>	Read-only. Shows the current versions, the available target, and any manual steps
3	<code>kubeadm upgrade apply v1.X.Y</code> on the first control-plane node	Rewrites the static Pod manifests, renews certificates, upgrades etcd
4	<code>kubeadm upgrade node</code> on every other control-plane node	Same, without re-running the cluster-wide steps
5	<code>kubectyl drain <node> → upgrade kubelet + kubectyl → systemctl restart kubelet → kubectyl uncordon <node></code>	Node components last, one node at a time
6	Repeat step 5 for every worker	Workload availability is preserved by draining one at a time

```
# READ ONLY — DO NOT RUN ON THE CLASS CLUSTER
kubeadm upgrade plan
kubeadm upgrade apply v1.31.1
kubectyl drain <node> --ignore-daemonsets
apt-get install -y kubelet=1.31.1-* kubectyl=1.31.1-*
systemctl daemon-reload && systemctl restart kubelet
kubectyl uncordon <node>
```

The three rules that govern this order:

- **Control plane before nodes, always.** A kubelet must never be newer than the API server.
- **Version skew.** The kubelet may be up to **three** minor versions *behind* the API server (widened from two in v1.28); `kubectyl` may be one minor version either side. Never skip a minor version when upgrading the control plane — go `1.30 → 1.31 → 1.32`, not `1.30 → 1.32`.
- **Snapshot etcd first.** `kubeadm upgrade apply` touches etcd. Section 4.8 is step zero of any upgrade runbook.

Lab 20 · Verification

```
bash verification/checks.sh

== Lab 20 verification — namespace kcna-lab20 ==
[PASS] namespace kcna-lab20 exists
[PASS] control plane is ready (/readyz reports ok)
[PASS] apiserver reports [+etcd ok in /readyz?verbose
[PASS] 4 static-pod components found with config.source=file
[PASS] mirror pod etcd-<node> is owned by a Node, not a controller
[PASS] etcd static pod exposes client port 2379 and peer port 2380
[PASS] configmap control-plane-inventory holds 9 data rows (matches data/)
[PASS] configmap etcd-restore-runbook carries 2 DO-NOT-RUN banners
[PASS] job/cp-inventory completed and reported static_pod_components=4
[PASS] serviceaccount cp-inventory CANNOT list pods in kube-system
[PASS] serviceaccount cp-inventory CAN list pods in kcna-lab20
[PASS] configmap etcd-backup-policy declares rpo_minutes and rto_minutes
[PASS] pod/runbook-reader runs with automountServiceAccountToken=false
[PASS] pod/runbook-reader has no hostPath volumes
[PASS] etcdctl is available inside the etcd static pod (etcd-kcna-control-plane)
[PASS] etcd endpoint status reports revision 38412 over TLS (read-only)
[PASS] no container in kcna-lab20 requests privileged: true
[PASS] no cluster-scoped object was created by this lab
-----
18 passed, 0 failed
```

Full transcript: `verification/expected-output.md`.

The verification script is strictly read-only against the control plane. Its only etcd calls are `etcdctl version` and `etcdctl endpoint status`, both pure reads. It deliberately does **not** call `endpoint health` — that commits a no-op Raft proposal — and it never calls `snapshot restore`, `defrag`, `compact` or `move-leader`. It writes no file on the node or in any container.

Lab 20 · Failure injection

The ticket: *"PLAT-4502 — `kubectl` is timing out cluster-wide. No deploys are going out. Where do I even start?"*

This lab's failure injection is **diagnostic, not destructive** — we are not going to break a shared control plane to teach you this. Instead you will learn to read the evidence, and reproduce one genuine, harmless fault.

Injection A — a static Pod that the kubelet rejects (safe, in your own namespace)

The kubelet's static-Pod path is what makes the control plane self-healing, and also what makes a typo fatal. You can reproduce the *failure signature* without touching `/etc/kubernetes/manifests` by looking at what a mirror Pod does when you attack it from the API side.

Try to delete a static Pod's mirror through the API — this is safe, and instructive:

```
kubectl -n kube-system delete pod "kube-scheduler-$NODE"
pod "kube-scheduler-kcna-control-plane" deleted
sleep 8
kubectl -n kube-system get pod "kube-scheduler-$NODE"
```

NAME	READY	STATUS	RESTARTS	AGE
kube-scheduler-kcna-control-plane	1/1	Running	0	7s

Diagnosis. The Pod is back with AGE 7s and the cluster never noticed. You deleted the **mirror**, not the Pod. The kubelet still had the file, still had the container running, and simply re-created the mirror object. RESTARTS 0 on a fresh AGE is the tell: the API object is new, the container is not.

Operational consequence. You cannot stop a control-plane component with `kubectl`. To take one out of service you move its file out of `/etc/kubernetes/manifests` — which is exactly Step R1 of the restore runbook, and exactly why that runbook works.

Injection B — read a real control-plane outage from `/readyz?verbose`

When the ticket above arrives, this is the first command, not `kubectl get pods` (which will itself hang if the API server is unwell):

```
kubectl get --raw='/readyz?verbose'
```

On a healthy cluster you saw `readyz check passed`. On a cluster whose etcd is unreachable you would see:

```
[+]ping ok
[+]log ok
[-]etcd failed: reason withheld
[-]etcd-readiness failed: reason withheld
[+]informer-sync ok
[+]poststarthook/generic-apiserver-start-informers ok
...
readyz check failed
```

Diagnosis path, in order — memorise this ladder:

Step	Command	What it distinguishes
1	<code>kubectl get --raw='/readyz?verbose'</code>	Which <i>subsystem</i> failed. <code>[-]etcd failed</code> points at storage, not at scheduling or admission
2	<code>kubectl get --raw='/livez?verbose'</code>	Whether the apiserver process itself is sick (<code>livez</code> failing → it should be restarted) or merely unable to serve (<code>readyz</code> failing)
3	<code>kubectl -n kube-system get pods</code>	Are the mirror Pods <code>Running</code> ?
4	<code>kubectl -n kube-system logs etcd-<node> --tail=50</code>	etcd's own error: disk full, corrupt WAL, cert expiry, lost quorum
5	<code>crictl ps -a / docker exec <node> crictl ps -a</code> on the node	The container-level truth, when the API server is too sick to answer
6	<code>journalctl -u kubelet -n 100</code> on the node	Whether the kubelet is even reading the manifest directory

The reason `readyz` says `reason withheld` to an unauthenticated caller and gives details to an authorised one is deliberate: health endpoints are reachable pre-auth, and leaking internal failure detail there would be an information disclosure.

The two control-plane failures you will actually meet:

Failure	Signature	First fix
Expired certificates (kubeadm certs last 1 year)	x509: certificate has expired or is not yet valid in kubelet/apiserver logs; kubectl fails to authenticate	kubeadm certs check-expiration, then kubeadm certs renew all and restart the static Pods
etcd disk full / quota exceeded	etcdserver: mvcc: database space exceeded; all writes fail, reads work	Compact and defragment, raise --quota-backend-bytes, then clear the NOSPACE alarm

Lab 20 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
kubectl -n kube-system exec etcd-... -- etcdctl ... fails with executable file not found	Wrong Pod, or a distribution whose etcd image lacks etcdctl	kubectl -n kube-system get pod \$ETCD_POD -o jsonpath='{.spec.containers[0].image}'	Confirm the image is registry.k8s.io/etcd:*; managed control planes (EKS/GKE/AKS) do not expose etcd at all
etcdctl: executable file not found	etcd older than 3.5	kubectl -n kube-system exec \$ETCD_POD -- etcdctl version	Use etcdctl snapshot status (deprecated but functional below 3.6)
snapshot save fails with context deadline exceeded	Wrong --endpoints, or the TLS material does not match the manifest	Re-read the flags in Step 4.5	Use https://127.0.0.1:2379 and the exact ca.crt / server.crt / server.key paths from the manifest
snapshot status shows TOTAL KEYS 0 or a few KB TOTAL SIZE	The save wrote to a path that was not what you verified, or failed silently	docker exec \$NODE ls -lh /var/lib/etcd/	Re-run snapshot save, then verify the <i>same</i> absolute path
A control-plane Pod reappears seconds after kubectl delete	It is a static Pod; you deleted its mirror	kubectl -n kube-system get pod <pod> -o jsonpath='{.metadata.annotations}' and look for config.source: file	Move the file in /etc/kubernetes/manifests. Never kubectl delete a mirror Pod expecting it to stay gone
A static Pod never appears at all after editing its file	YAML error; the kubelet cannot parse it and never contacts the API server	journalctl -u kubelet -n 50 on the node	Fix the YAML. There will be no event and no API object — the kubelet log is the only evidence
kubectl get --raw='/readyz?verbose' shows [-]etcd failed	etcd is down, out of disk, out of quorum, or its certs expired	kubectl -n kube-system logs etcd-\$NODE --tail=50	Address the specific etcd error. A snapshot restore is the <i>last</i> resort, not the first
x509: certificate has expired cluster-wide	kubeadm control-plane certificates expired (1-year lifetime)	kubeadm certs check-expiration on the node	kubeadm certs renew all, then restart the static Pods by touching their manifests
etcdserver: mvcc: database space exceeded	etcd hit its backend quota; all writes are rejected	etcdctl endpoint status -w table	Compact to the current revision, etcdctl defrag, disarm the NOSPACE alarm, then raise the quota

Lab 20 · Cleanup

Nothing in the control plane needs cleaning up, because nothing in the control plane was changed. The only artefacts are inside `kcna-lab20`, plus one snapshot file you already removed in Step 4.11.

```
kubectl delete namespace kcna-lab20 --wait=true
namespace "kcna-lab20" deleted
```

Confirm the snapshot is gone from the etcd data directory (skip without Docker; it is removed with the cluster in any case):

```
docker exec "$NODE" ls /var/lib/etcd
member
```

Optionally remove the copy on your own machine:

```
rm -f /tmp/kcna-lab20-snapshot.db && echo "local snapshot copy removed"
local snapshot copy removed
```

Confirm the control plane is exactly as you found it:

```
kubectl get --raw='/readyz' && echo
kubectl -n kube-system get pods --no-headers | wc -l
kubectl get namespace kcna-lab20

ok
      8
Error from server (NotFound): namespaces "kcna-lab20" not found
```

This lab created no cluster-scoped objects and modified no control-plane object. The ServiceAccount, Role and RoleBinding were namespaced by design — see `manifests/01-rbac-namespace-reader.yaml`. If you needed a ClusterRole to complete this lab, you had the wrong design.

Lab 20 · What you learned

- The control plane is **five moving parts plus a node agent**: etcd stores state, the API server is the only process that touches it, the scheduler places Pods, the controller-manager reconciles them, and the kubelet actually starts containers. They communicate **only through objects in the API server** — you saw it in the event `SOURCE` column, where `default-scheduler` and `kubelet` collaborate without ever contacting each other.
- **Only the API server talks to etcd**. That single chokepoint is where authentication, authorisation, admission and validation live. Anything that bypasses it bypasses all of them.
- **Static Pods** are how the control plane bootstraps itself: the kubelet reads `/etc/kubernetes/manifests` with no API server involved, and publishes read-only **mirror Pods** annotated `kubernetes.io/config.source: file` and owned by a `Node`. Deleting the mirror does nothing — you proved this in Injection A. To stop a static Pod you move its file.
- **Leader election is an ordinary API object**. `kubectl -n kube-system get leases` shows the scheduler and controller-manager holding and renewing Leases, which is why high availability needs no separate coordination system.
- **etcdctl snapshot save is a safe client read** against a running server; `snapshot status` and `snapshot restore` are offline file operations that moved to `etcdutl` (deprecated in `etcdctl` from

3.5, removed in 3.6). Knowing which binary owns which verb is a real operational distinction, not trivia.

- **A snapshot you have not verified is not a backup, and a snapshot on the failed node is not a backup.** `HASH`, `REVISION`, `TOTAL KEYS` and `TOTAL SIZE` are your acceptance test; off-node storage is mandatory.
- **RPO equals your snapshot interval. RTO is a number you have measured in a drill**, or it is a guess.
- **An etcd restore does not recover PersistentVolume contents, post-snapshot writes, encryption key material or certificates.** etcd holds the PV and PVC *objects*; the bytes on the disk need their own backup path — which is the direct continuation of Labs 18 and 19.
- **Restore and upgrade are one-way doors.** You now know both procedures precisely, and you performed neither against a shared cluster. That is the correct professional instinct, not a limitation of the lab.

Carry into Lab 21: you can now describe every component that runs TrackLane. Lab 21 asks the next question — how do you *deliver* changes to it repeatably, across dev and prod, without hand-editing YAML?

Lab 20 · Further reading

- Kubernetes documentation — *Kubernetes Components*:
<https://kubernetes.io/docs/concepts/overview/components/>
- Kubernetes documentation — *Static Pods*: <https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/>
- Kubernetes documentation — *Operating etcd clusters for Kubernetes* (the authoritative backup/restore reference): <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>
- Kubernetes documentation — *Kubernetes API health endpoints* (`/livez`, `/readyz`, `/healthz`):
<https://kubernetes.io/docs/reference/using-api/health-checks/>
- Kubernetes documentation — *Upgrading kubeadm clusters*:
<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>
- Kubernetes documentation — *Version skew policy*: <https://kubernetes.io/releases/version-skew-policy/>
- Kubernetes documentation — *Certificate management with kubeadm*:
<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-certs/>
- etcd documentation — *Disaster recovery* and the `etcdctl` reference: <https://etcd.io/docs/v3.5/op-guide/recovery/>
- Kubernetes documentation — *Encrypting Confidential Data at Rest*:
<https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>
- CNCF KCNA Curriculum — *Kubernetes Fundamentals* → *Administration*:
<https://github.com/cncf/curriculum>

Lab 21 — Packaging and Delivery: Helm, Kustomize and GitOps

Field	Value
Lab ID	Lab 21
Title	Packaging and Delivery: Helm, Kustomize and GitOps
Day / Topic	Day 4 · Cloud Native Application Delivery
Duration	55 minutes
Namespace	kcna-lab21
Learning outcome	LO5 — Demonstrate Kubernetes solution for a specific business problem.
Ability	A5 Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need
Knowledge	K5 Tools and techniques for solution architecture modelling
Deck slide	Slide 474
Repository path	courseware/labs/lab-21-helm-kustomize-gitops/

Goal. Replace hand-maintained TrackLane YAML with two real packaging approaches — a Helm chart rendered from environment values files, and a Kustomize base with dev and prod overlays — then read a GitOps Application manifest field by field to see how a controller turns Git into the cluster's desired state.

What you will produce:

- A minimal but complete Helm chart (Chart.yaml, values.yaml, _helpers.tpl, templates, NOTES.txt) that renders different dev and prod output from two values files, including a conditionally created PodDisruptionBudget.
- A Kustomize base plus dev and prod overlays that render to stdout with `kubectl kustomize` alone, differing in replicas, resources, config literals, image registry, probe timing and dataset size.
- An annotated Argo CD Application and the equivalent Flux GitRepository/Kustomization pair, read field by field, with the reconciliation model (prune, selfHeal, drift) explained against them.

Lab 21 · Tooling note — read this first

helm is not installed on this machine and may not be on yours. That is deliberate, and this lab is built around it:

- **Everything graded runs offline with `kubectl` alone.** `kubectl kustomize` is built into `kubectl` (it embeds Kustomize v5) and renders base + overlays to stdout with **no cluster and no network**. `verification/checks.sh` grades the *static render*.
- **The Helm sections are still fully authored and readable.** You will read `chart/` field by field, which is most of the learning. If you have `helm`, the render commands work exactly as printed and the checks grade them too. If you do not, `checks.sh` reports `[SKIP]` for those lines and still passes.

Install Helm if you want the full experience:

```
helm version
command not found: helm
```

Platform	Command
macOS (Homebrew)	<code>brew install helm</code>
Linux (script)	<code>curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 bash</code>
Any	https://helm.sh/docs/intro/install/

Argo CD and Flux are not installed either, and this lab does not install them. The reconciliation model is taught by reading a real `Application` manifest field by field — which is what you would do on the job before approving one. Those files live in `gitops/`, **outside** `manifests/`, precisely so that `kubectl apply -f manifests/` cannot pick up a kind whose CRD is absent.

Lab 21 · Objective

By the end of this lab you will be able to:

1. **Explain why hand-maintained YAML fails** at more than one environment, in terms of the specific failure it produces.
2. **Read a Helm chart** and name the purpose of `Chart.yaml`, `values.yaml`, `templates/`, `_helpers.tpl` and `NOTES.txt`, including the difference between `version` and `appVersion`.
3. **State Helm's values precedence** and predict the rendered output of `helm template` given a chart and a values file.
4. **Explain the `checksum/config` annotation** and the failure it prevents.
5. **Build a Kustomize base and two overlays**, and render both with `kubectl kustomize` — with no cluster.
6. **Choose between a strategic-merge patch and a JSON 6902 patch**, and say what each can do that the other cannot.
7. **Compare templating (Helm) with patching (Kustomize)** and defend a choice for a given situation.
8. **Read an Argo CD `Application` field by field** and explain `prune`, `selfHeal`, `targetRevision` and `ignoreDifferences` in terms of the reconciliation loop.

Lab 21 · Prerequisites

- **kubectl v1.27 or later** (it must embed Kustomize v5 for the `labels:` and `patches:` syntax used here). Check:

```
kubectl version --client
Client Version: v1.31.0
Kustomize Version: v5.4.2
```

If `Kustomize Version` reads `v4.x`, the `labels:` transformer in the overlays will be rejected. Upgrade `kubectl`, or substitute the deprecated `commonLabels:` field.

- A single-node **kind** cluster for Steps 7 and 9 only. **Steps 3–6 and 8 need no cluster at all.**
- Labs 17–20 completed.

- Terminal at this lab folder:

```
cd courseware/labs/lab-21-helm-kustomize-gitops
ls
README.md  brief.json  chart      data      gitops     kustomize  manifests
verification
```

Lab 21 · Scenario

Kallang Freight Pte Ltd has spent Day 4 hardening TrackLane's storage (Labs 17–19) and documenting the control plane that runs it (Lab 20). The last gap is how changes actually reach the cluster.

TrackLane has outgrown its deployment process. There is one directory of YAML, and to ship to the dev cluster an engineer copies it, edits the replica count, edits the log level, edits the feature flag, and trims the shipping-lane dataset to the four domestic routes so the dev fixtures load faster.

Last Thursday someone copied prod → dev, changed the replica count and the log level, and **missed the feature flag**. `FEATURE_LANE_ETA=true` went to production, the lane-ETA endpoint hit a service that is not deployed there, and the customer portal returned 500s for eleven minutes.

The post-incident action is not "be more careful". It is: **there must be exactly one place where dev and prod differ, and it must be reviewable in a pull request**.

Your job this session is to build that in both of the two mainstream ways, decide which TrackLane should adopt, and then look at the delivery model that removes the human from the apply step entirely.

The platform team has already written down the intended difference, in `data/release-inventory.csv`. That file is the contract the graded verification checks your renders against.

```
column -s, -t data/release-inventory.csv
environment  name_suffix  replicas  cpu_request  memory_request  lane_rows  log_level
feature_lane_eta
dev          -dev        1        50m         64Mi           4          debug
true
prod         -prod       3        100m        128Mi          9          warn
false
```

Lab 21 · Step-by-step procedure

Step 1 — Namespace

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab21 created
```

Step 2 — Deploy the "before" state and feel the problem

```
kubectl -n kcna-lab21 create configmap tracklane-handwritten-lanes \
  --from-file=shipping-lanes.csv=data/shipping-lanes.csv
kubectl apply -f manifests/01-tracklane-handwritten.yaml
configmap/tracklane-handwritten-lanes created
configmap/tracklane-handwritten-config created
```



```
service/tracklane-handwritten created
deployment.apps/tracklane-handwritten created

kubectl -n kcna-lab21 rollout status deployment/tracklane-handwritten --timeout=180s
deployment "tracklane-handwritten" successfully rolled out
```

Now count the work involved in producing a dev variant of this:

```
grep -c 'tracklane-handwritten' manifests/01-tracklane-handwritten.yaml
grep -c 'app.kubernetes.io/instance: tracklane-handwritten' manifests/01-tracklane-
handwritten.yaml

11
6
```

Eleven occurrences of the instance name, six of them inside label blocks, spread across three objects — and you have to change every one consistently, in a copy, without breaking the ones you must *not* change (the selector labels, which are immutable after creation). Then you must remember the feature flag.

That is the defect. Both tools below attack it, from opposite directions:

	Helm	Kustomize
Mechanism	Templating. Source files are Go templates; values are substituted at render time	Patching. Source files are plain YAML; overlays declare deltas
Source is valid YAML?	No — <code>{{ }}</code> breaks every YAML tool	Yes — editors, <code>kubeconform</code> and <code>kubectl apply</code> all work on the base
Conditional resources	Yes — <code>{{- if }}</code> can create or omit a whole object	No — you can patch a resource, not conjure one
Distribution to third parties	Yes — versioned, packaged, registry-hosted charts	Poorly — Kustomize has no package format or registry
Release lifecycle	Yes — install/upgrade/rollback/history, state in a Secret	No — it renders; <code>kubectl apply</code> does the rest
Built into kubectl	No	Yes — <code>kubectl kustomize</code> , <code>kubectl apply -k</code>
Failure mode	A logic bug renders invalid YAML at deploy time	A patch silently matches nothing and does nothing

Step 3 — Read the Helm chart

```
find chart -type f | sort

chart/.helmignore
chart/Chart.yaml
chart/templates/NOTES.txt
chart/templates/_helpers.tpl
chart/templates/configmap.yaml
chart/templates/deployment.yaml
chart/templates/service.yaml
chart/values.yaml
```

That is a complete, valid chart. Every file earns its place:

Path	Purpose
Chart.yaml	Required. Chart metadata. Its presence is what makes a directory a chart

Path	Purpose
values.yaml	Required in practice. The default values, and the chart's public interface
templates/*.yaml	Rendered into Kubernetes objects
templates/_helpers.tpl	Leading underscore ⇒ not rendered . Defines named templates used by the others
templates/NOTES.txt	Printed after <code>helm install</code> . Not an object; <code>helm template</code> does not emit it
.helmignore	Excludes files from <code>helm package</code>
charts/	(<i>absent here</i>) Vendored subcharts, populated by <code>helm dependency update</code>

version **versus** **appVersion** — the most commonly muddled pair in Helm:

```
grep -E '^(version|appVersion|apiVersion|type|kubeVersion):' chart/Chart.yaml
apiVersion: v2
type: application
version: 0.3.0
appVersion: "1.27"
```

- `version: 0.3.0` — the version of **the chart**. Bump it when templates or defaults change. Helm enforces semver.
- `appVersion: "1.27"` — the version of **the software being deployed**. Pure metadata; Helm never uses it to select an image unless a template explicitly does. Quoted, or YAML reads `1.27` as a float and drops the trailing zero on `1.20`.
- `apiVersion: v2` — Helm 3. `v1` is a Helm 2 chart.

Now the helpers — this is where the reusable logic lives:

```
grep -n '^{{- define' chart/templates/_helpers.tpl
14:{{- define "tracklane.name" -}}
24:{{- define "tracklane.fullname" -}}
41:{{- define "tracklane.chart" -}}
52:{{- define "tracklane.selectorLabels" -}}
62:{{- define "tracklane.labels" -}}
76:{{- define "tracklane.validateImage" -}}
```

The `selectorLabels` / `labels` split is the single most important idea in this file.

```
sed -n '45,56p' chart/templates/_helpers.tpl
{{/*
SELECTOR labels. These go into Deployment.spec.selector.matchLabels, which is
IMMUTABLE after creation — so this set must stay minimal and must never
include anything that changes between releases (no version, no chart version).
Getting this wrong is the single most common cause of a chart that cannot be
upgraded in place.
*/}}
{{- define "tracklane.selectorLabels" -}}
app.kubernetes.io/name: {{ include "tracklane.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/component: web
{{- end -}}
```

A chart that put `app.kubernetes.io/version` into the selector would break on the first version bump with field is immutable, and the only fix would be deleting and recreating the Deployment.

Step 4 — Render the chart for two environments

No helm? Read this step and skip the commands. The outputs shown are what the chart produces; `checks.sh` reports `[SKIP]` for these and still passes. Everything from Step 5 onward works with `kubectl` alone.

Render dev:

```
helm template tracklane-dev ./chart \
  --namespace kcna-lab21 \
  -f data/values-dev.yaml \
  --set-file lanes.csv=data/shipping-lanes.csv > /tmp/tracklane-dev.yaml
grep -E '^kind:' /tmp/tracklane-dev.yaml

kind: ConfigMap
kind: ConfigMap
kind: Service
kind: Deployment
```

Render prod:

```
helm template tracklane-prod ./chart \
  --namespace kcna-lab21 \
  -f data/values-prod.yaml \
  --set-file lanes.csv=data/shipping-lanes.csv > /tmp/tracklane-prod.yaml
grep -E '^kind:' /tmp/tracklane-prod.yaml

kind: PodDisruptionBudget
kind: ConfigMap
kind: ConfigMap
kind: Service
kind: Deployment
```

Four objects in dev, five in prod. The `PodDisruptionBudget` exists only because `data/values-prod.yaml` sets `podDisruptionBudget.enabled: true` and the template wraps it in `{{- if }}`.

That is the capability Kustomize does not have — Kustomize can patch an object into a different shape, but it cannot make one appear or disappear.

On the ordering. Helm emits manifests in its own **install order** (a fixed kind precedence: namespaces and policy objects first, then config, then services, then workloads), not in source-file order. That is why `PodDisruptionBudget` appears first even though it is written at the bottom of `templates/deployment.yaml`. Do not read anything into the sequence.

Now the artefact a reviewer actually reads:

```
diff <(grep -E 'replicas:|LOG_LEVEL|FEATURE_LANE_ETA|CACHE_TTL|cpu:|memory:|^kind:'
/tmp/tracklane-dev.yaml) \
  <(grep -E 'replicas:|LOG_LEVEL|FEATURE_LANE_ETA|CACHE_TTL|cpu:|memory:|^kind:'
/tmp/tracklane-prod.yaml)

2,4c2,4
<   CACHE_TTL_SECONDS: "5"
<   FEATURE_LANE_ETA: "true"
<   LOG_LEVEL: "debug"
---
>   CACHE_TTL_SECONDS: "300"
>   FEATURE_LANE_ETA: "false"
>   LOG_LEVEL: "warn"
9c9
<   replicas: 1
---
>   replicas: 3
```

```

11,14c11,15
<     cpu: 200m
<     memory: 128Mi
<     cpu: 50m
<     memory: 64Mi
---
>     cpu: 500m
>     memory: 256Mi
>     cpu: 100m
>     memory: 128Mi
> kind: PodDisruptionBudget

```

Every line of that diff traces to one line in `data/values-dev.yaml` or `data/values-prod.yaml`. **The environment difference is now a reviewable file, not a copy-paste ritual** — which is exactly the post-incident action item.

Values precedence, lowest to highest — you must be able to state this:

```
chart/values.yaml < -f file (left to right) < --set < --set-string / --set-file
```

Prove it:

```

helm template tracklane-dev ./chart -f data/values-dev.yaml \
  --set replicaCount=7 | grep -m1 'replicas:'

replicas: 7

```

`--set` beat the values file, which beat the chart default of 2.

Confirm the dataset really travelled from `data/`:

```

grep -c '^      SG-' /tmp/tracklane-prod.yaml
echo "source rows: $(( $(wc -l < data/shipping-lanes.csv) - 1 ))"

9
source rows: 9

```

`--set-file` read the whole CSV off disk into `.Values.lanes.csv`, and the ConfigMap template embedded it with `nindent 4`.

The checksum/config annotation — find it:

```

grep -Al 'annotations:' /tmp/tracklane-prod.yaml | grep checksum
      checksum/config: 3f7a1e5c9b02d846af1739e5c0d2b68475fa03c91d6e847b25c0af9e1d637208
grep -n 'checksum/config' chart/templates/deployment.yaml
33:      checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}

```

The failure it prevents. Without it, `helm upgrade` with a changed `logLevel` updates the ConfigMap and leaves the Pod template byte-identical. The Deployment controller sees no change, does not roll, and your Pods serve the old config indefinitely — because `envFrom` values are read once at container start. With the annotation, the config hash is part of the Pod template, so any config change forces a rollout. Four lines; catches an entire class of "I deployed it but nothing changed" incidents.

Finally, lint the chart:

```

helm lint ./chart

==> Linting ./chart
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed

```

Step 5 — Read the Kustomize base

Everything from here works with `kubectl` alone.

```
find kustomize -type f | sort
kustomize/base/deployment.yaml
kustomize/base/files/shipping-lanes.csv
kustomize/base/kustomization.yaml
kustomize/base/service.yaml
kustomize/overlays/dev/files/shipping-lanes.csv
kustomize/overlays/dev/kustomization.yaml
kustomize/overlays/dev/patch-resources.yaml
kustomize/overlays/prod/kustomization.yaml
kustomize/overlays/prod/patch-probe-timing.yaml
kustomize/overlays/prod/patch-resources.yaml
```

The defining property of the base — it is **ordinary YAML**:

```
kubectl apply --dry-run=client -f kustomize/base/deployment.yaml
deployment.apps/tracklane created (dry run)

/opt/homebrew/bin/kubeconform -strict -summary kustomize/base/deployment.yaml
kustomize/base/service.yaml

Summary: 2 resources found in 2 files - Valid: 2, Invalid: 0, Errors: 0, Skipped: 0
```

You cannot do either of those to a Helm template — `{{ .Values.replicaCount }}` is not valid YAML. That is the fundamental trade.

Now the base's `kustomization.yaml`:

```
cat kustomize/base/kustomization.yaml | grep -vE '^\\s*#|^$'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
labels:
- pairs:
    app.kubernetes.io/managed-by: kustomize
    kcna.tertiaryinfotech.com/lab: "21"
    includeSelectors: false
commonAnnotations:
    tracklane.kallangfreight.sg/owner: platform-engineering
configMapGenerator:
- name: tracklane-config
  literals:
    - LOG_LEVEL=info
    - FEATURE_LANE_ETA=false
    - CACHE_TTL_SECONDS=60
    - ENVIRONMENT=base
- name: tracklane-lanes
  files:
    - shipping-lanes.csv=files/shipping-lanes.csv
generatorOptions:
    disableNameSuffixHash: false
```

Field	What it does
resources	The plain YAML files to include
labels ... includeSelectors: false	Adds labels to <code>metadata.labels</code> only . Setting <code>true</code> would also write them into <code>spec.selector</code> — an immutable field
commonAnnotations	Same idea for annotations

Field	What it does
configMapGenerator	Builds ConfigMaps from literals or files, and appends a content hash to the name
generatorOptions.disableNameSuffixHash: false	Keeps the hash on. Turning it off reintroduces exactly the bug <code>checksum/config</code> fixes in Helm

Step 6 — Render both overlays

```
kubectl kustomize kustomize/overlays/dev | grep -E '^kind:|^  name:|^  replicas:|LOG_LEVEL|FEATURE_LANE_ETA'

kind: ConfigMap
  name: tracklane-config-dev-m889h8f92h
  FEATURE_LANE_ETA: "true"
  LOG_LEVEL: debug
kind: ConfigMap
  name: tracklane-lanes-dev-6gmkh6tdbh
kind: Service
  name: tracklane-dev
kind: Deployment
  name: tracklane-dev
  replicas: 1

kubectl kustomize kustomize/overlays/prod | grep -E '^kind:|^  name:|^  replicas:|LOG_LEVEL|FEATURE_LANE_ETA'

kind: ConfigMap
  name: tracklane-config-prod-8c5db4d86b
  FEATURE_LANE_ETA: "false"
  LOG_LEVEL: warn
kind: ConfigMap
  name: tracklane-lanes-prod-t7f75mc2d4
kind: Service
  name: tracklane-prod
kind: Deployment
  name: tracklane-prod
  replicas: 3
```

Check the whole contract at once against the platform team's inventory:

```
for env in dev prod; do
  OUT=$(kubectl kustomize "kustomize/overlays/$env")
  printf '%-5s replicas=%s cpu=%s lanes=%s image=%s probe=%s\n' "$env" \
    "$(printf '%s' "$OUT" | grep -m1 '^  replicas:' | tr -d ' ' | cut -d: -f2)" \
    "$(printf '%s' "$OUT" | grep -A2 'requests:' | grep -m1 'cpu:' | tr -d ' ' | cut -d: -f2)" \
    "$(printf '%s' "$OUT" | grep -c '^    SG-')" \
    "$(printf '%s' "$OUT" | grep -m1 'image:' | tr -d ' ' | cut -d: -f2-)" \
    "$(printf '%s' "$OUT" | grep -m1 'periodSeconds:' | tr -d ' ' | cut -d: -f2)"
done

dev  replicas=1 cpu=50m lanes=4 image=nginx:1.27-alpine probe=5
prod replicas=3 cpu=100m lanes=9 image=docker.io/library/nginx:1.27-alpine probe=10
```

Compare against `data/release-inventory.csv` from Section 3 — every value matches. **Five different mechanisms produced that line:**

Difference	Mechanism	Where
tracklane-dev / tracklane-prod	nameSuffix	overlay kustomization.yaml
replicas 1 / 3	replicas: transformer (not a patch)	overlay kustomization.yaml
cpu 50m / 100m	strategic-merge patch	patch-resources.yaml

Difference	Mechanism	Where
lanes 4 / 9	configMapGenerator with behavior: replace in dev; prod inherits the base	overlay kustomization.yaml
image bare / fully-qualified	images: transformer	prod kustomization.yaml
periodSeconds 5 / 10	JSON 6902 patch	patch-probe-timing.yaml

Strategic merge versus JSON 6902 — read them side by side:

```
cat kustomize/overlays/prod/patch-resources.yaml | grep -vE '^s*#|^$'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tracklane
spec:
  template:
    spec:
      containers:
      - name: web
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi

cat kustomize/overlays/prod/patch-probe-timing.yaml | grep -vE '^s*#|^$'
- op: replace
  path: /spec/template/spec/containers/0/readinessProbe/periodSeconds
  value: 10
- op: add
  path: /spec/template/spec/containers/0/readinessProbe/failureThreshold
  value: 3
- op: add
  path: /spec/template/spec/containers/0/readinessProbe/timeoutSeconds
  value: 3
```

	Strategic merge	JSON 6902
Shape	A YAML map with apiVersion/kind	A YAML list of {op, path, value}
Addresses list items	By merge key (containers merges on name)	By index (/containers/0)
Can remove a field	No	Yes (op: remove)
Breaks if a list is reordered	No	Yes, silently
Readability	High — looks like the object	Low — looks like a patch
Use it when	Almost always	You need remove, an exact index, or a field with no merge key

Kustomize infers which one you meant from the file's shape. There is no flag.

Finally, prove the two overlays cannot interfere with each other in the shared namespace:

```
for env in dev prod; do
  printf '%s selector: ' "$env"
  kubectl kustomize "kustomize/overlays/$env" \
    | grep -A4 '^ selector:' | grep 'instance:' | head -1 | tr -d ' '
done
```

```
dev selector: app.kubernetes.io/instance:tracklane-dev
prod selector: app.kubernetes.io/instance:tracklane-prod
```

That is `includeSelectors: true` in the overlays doing its job. With `false`, both Deployments would carry the base's selector, both Services would match all six Pods, and the two "environments" would silently merge.

Step 7 — Apply the dev overlay to the cluster

`kubectl apply -k` runs the same render and pipes it straight to `apply`:

```
kubectl apply -k kustomize/overlays/dev

configmap/tracklane-config-dev-m889h8f92h created
configmap/tracklane-lanes-dev-6gmkh6tdbh created
service/tracklane-dev created
deployment.apps/tracklane-dev created

kubectl -n kcna-lab21 rollout status deployment/tracklane-dev --timeout=180s
deployment "tracklane-dev" successfully rolled out

kubectl -n kcna-lab21 get deployment,configmap -l app.kubernetes.io/instance=tracklane-dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/tracklane-dev	1/1	1	1	24s

NAME	DATA	AGE
configmap/tracklane-config-dev-m889h8f92h	4	24s
configmap/tracklane-lanes-dev-6gmkh6tdbh	1	24s

Confirm the config actually reached the container, and that the trimmed dev dataset is what is mounted:

```
kubectl -n kcna-lab21 exec deployment/tracklane-dev -- \
  sh -c 'echo "LOG_LEVEL=$LOG_LEVEL FEATURE_LANE_ETA=$FEATURE_LANE_ETA"; \
        echo "lanes=$(( $(wc -l < /usr/share/nginx/html/lanes/shipping-lanes.csv) -
1 ))"'

LOG_LEVEL=debug FEATURE_LANE_ETA=true
lanes=4
```

Four lanes, debug logging, feature flag on — the dev contract, delivered without anyone editing a copy of the base.

Now watch the generator hash do its job. Change a literal in the dev overlay:

```
sed -i.bak 's/CACHE_TTL_SECONDS=5$/CACHE_TTL_SECONDS=15/'
kustomize/overlays/dev/kustomization.yaml
kubectl apply -k kustomize/overlays/dev

configmap/tracklane-config-dev-f2c68b9047 created
configmap/tracklane-lanes-dev-6gmkh6tdbh unchanged
service/tracklane-dev unchanged
deployment.apps/tracklane-dev configured
```

A new ConfigMap name, and deployment configured. The hash changed, so the Deployment's Pod template changed, so it rolled:

```
kubectl -n kcna-lab21 rollout status deployment/tracklane-dev --timeout=180s
kubectl -n kcna-lab21 exec deployment/tracklane-dev -- printenv CACHE_TTL_SECONDS

deployment "tracklane-dev" successfully rolled out
15
```

Restore the file:


```
mv kustomize/overlays/dev/kustomization.yaml.bak kustomize/overlays/dev/kustomization.yaml
kubectl apply -k kustomize/overlays/dev >/dev/null
grep -n 'CACHE_TTL_SECONDS' kustomize/overlays/dev/kustomization.yaml
43:      - CACHE_TTL_SECONDS=5
```

Note the leftover. The old ConfigMap `tracklane-config-dev-m889h8f92h` is still in the namespace — `apply` creates the new one and nothing deletes the old. Kustomize alone has no garbage collection. `kubectl apply --prune` and, properly, a GitOps controller's `prune: true` are what solve this. That is the bridge to Step 8.

```
kubectl -n kcna-lab21 get configmap | grep tracklane-config-dev
tracklane-config-dev-f2c68b9047    4    2m
tracklane-config-dev-m889h8f92h    4    6m
```

Step 8 — GitOps: read the reconciliation contract

Nothing so far removed the human. Somebody still runs `kubectl apply`. That means:

- The cluster's state depends on **who ran what, from which laptop, at what time**.
- Nothing detects or corrects a manual `kubectl edit` at 02:00.
- `kubectl apply -k` from a workstation needs cluster-admin **on a human's credential**.

GitOps inverts this. The four principles, as OpenGitOps states them:

1. **Declarative** — the system is described entirely by declarative state.
2. **Versioned and immutable** — that state lives in Git, with history and signatures.
3. **Pulled automatically** — agents *inside* the cluster fetch the state. Nothing is pushed in.
4. **Continuously reconciled** — agents constantly compare actual to desired and converge.

Point 3 is the security argument: the CI system never needs cluster credentials at all. Point 4 is the reliability argument.

Look at the manifest:

```
head -25 gitops/argocd-application.yaml
# =====
# Lab 21 / Step 8 - an Argo CD Application, for READING.
#
# ⚠ DO NOT APPLY THIS FILE. It is deliberately kept OUT of manifests/ so
#   that `kubectl apply -f manifests/` cannot pick it up by accident.
...
```

Confirm for yourself that it genuinely cannot be applied here:

```
kubectl apply --dry-run=client -f gitops/argocd-application.yaml
error: resource mapping not found for name: "tracklane-dev" namespace: "argocd" from
"gitops/argocd-application.yaml": no matches for kind "Application" in version
"argoproj.io/v1alpha1"
ensure CRDs are installed first
```

That error is the lesson. Argo CD is not special infrastructure — it is a controller that installs a CRD and then watches instances of it. Without the CRD, the API server has never heard of `kind: Application`. Every GitOps tool works this way.

Now read the four blocks that matter:

```
grep -vE '^\\s*#|^\\s*$|^# ' gitops/argocd-application.yaml | sed -n '1,30p'
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tracklane-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  labels:
    tracklane.kallangfreight.sg/environment: dev
spec:
  project: default
  source:
    repoURL: https://github.com/tertiarycourses/TGS-2023039343-Kubernetes-and-Cloud-
Native-Associate-KCNA-Training
    targetRevision: main
    path: courseware/labs/lab-21-helm-kustomize-gitops/kustomize/overlays/dev
    kustomize:
      namePrefix: ""
      commonAnnotations:
        tracklane.kallangfreight.sg/reconciled-by: argocd
  destination:
    server: https://kubernetes.default.svc
    namespace: kcna-lab21
```

Field	Meaning	The consequence of getting it wrong
metadata.namespace: argocd	The Application lives in Argo CD's namespace, not the target's	Argo CD only watches its own namespace by default; put it elsewhere and nothing happens, with no error
finalizers: [resources-finalizer...]	Deleting the Application also deletes what it created	Without it, deleting the Application orphans the workloads, which keep running unmanaged
spec.project	The AppProject restricting allowed repos, destinations and kinds	default permits everything — fine for a demo, wrong on a shared cluster
source.repoURL	Where the desired state lives. This is the whole of GitOps	—
source.targetRevision: main	A branch tracks the tip; a tag or SHA pins	A mutable branch makes "what is deployed?" unanswerable during an incident. Production should pin
source.path	The directory in the repo. Argo CD detects kustomization.yaml → runs kustomize; Chart.yaml → runs helm template	A wrong path renders nothing; see allowEmpty below
destination.server	https://kubernetes.default.svc = "the cluster Argo CD runs in"	—

And the reconciliation contract itself:

```
grep -A16 'syncPolicy:' gitops/argocd-application.yaml | grep -vE '^\\s*#'
```

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: false
  syncOptions:
    - CreateNamespace=true
    - ServerSideApply=true
```

```
- RespectIgnoreDifferences=true
retry:
  limit: 5
```

Setting	What the controller does	Why it matters
prune: true	Deletes cluster objects no longer present in Git	This is what fixes the orphaned ConfigMap you created in Step 7. Without prune, deleting a manifest from Git leaves the object running forever and Git stops being the source of truth
selfHeal: true	Reverts manual changes back to Git's state	The anti-drift mechanism. Someone scales a Deployment by hand; the controller scales it back and records the divergence
allowEmpty: false	Refuses to sync a render that produces zero resources	A typo'd <code>path</code> would otherwise be read as "Git says delete everything"
ServerSideApply=true	Applies with field-manager ownership tracking	Two controllers cannot silently fight over one field
ignoreDifferences: /spec/replicas	Concedes that field to another owner	Without it, an HPA changing replicas reads as permanent drift and <code>selfHeal</code> fights the HPA in a loop

The loop, in one sentence: *observe Git → render → compare to the live cluster → apply the difference → repeat forever.* There is no "deploy" step, and there is no push.

Flux expresses the identical model with different nouns:

```
grep -E '^(kind| name| interval| path| prune| targetNamespace):' gitops/flux-
kustomization.yaml

kind: GitRepository
  name: tracklane
  interval: 1m
kind: Kustomization
  name: tracklane-dev
  interval: 5m
  path: ./courseware/labs/lab-21-helm-kustomize-gitops/kustomize/overlays/dev
  prune: true
  targetNamespace: kcna-lab21
```

Argo CD	Flux
Application.spec.source	a separate <code>GitRepository</code> object
Application.spec.destination	<code>Kustomization.spec.targetNamespace</code>
syncPolicy.automated.prune	<code>Kustomization.spec.prune</code>
syncPolicy.automated.selfHeal	drift detection + force
built-in polling	explicit <code>spec.interval</code>

Argo CD packs source, destination and policy into one object and ships a web UI. Flux decomposes them into composable controllers and leans on the CLI and Git. **Both implement the same loop.** Neither is "more GitOps".

Step 9 — Decide

For TrackLane, given the incident in Section 3:

Situation	Choose	Why
Internal app, 2–3 environments, small team	Kustomize	The base stays plain YAML; the overlay diff <i>is</i> the review artefact; nothing extra to install
Shipping software other organisations install	Helm	Versioned, packaged, registry-distributable; <code>values.yaml</code> is a documented public interface
Environments that need different sets of objects	Helm	Only templating can conditionally create a resource
Installing third-party software (Prometheus, cert-manager)	Helm	That is how upstream publishes it
Consuming a third-party chart but needing 3 fields the chart does not expose	Both — render the chart, then patch it with Kustomize	This is why Argo CD and Flux both support "Helm render, then Kustomize post-render"
Any of the above, more than one cluster or more than one operator	plus GitOps	Neither tool solves drift, audit or credential distribution. That is a delivery-model problem, not a packaging one

TrackLane's answer: **Kustomize for the app, Helm for third-party dependencies, GitOps for delivery.** The overlay diff is what a reviewer reads, and `selfHeal` is what stops the next 02:00 hand-edit.

Lab 21 · Verification

This lab is graded on the static render. Run it with no cluster if you like:

```
bash verification/checks.sh

== Lab 21 verification – static render (no cluster required) ==
[PASS] chart/ contains Chart.yaml, values.yaml, _helpers.tpl and 3 templates
[PASS] Chart.yaml declares apiVersion v2, name tracklane and a semver version
[PASS] no ':latest' image tag anywhere in chart/, kustomize/ or manifests/
[PASS] kubectl kustomize renders overlays/dev
[PASS] kubectl kustomize renders overlays/prod
[PASS] both rendered overlays pass kubectl apply --dry-run=client
[PASS] overlays/dev matches data/release-inventory.csv (suffix -dev, 1 replicas, 50m/64Mi, 4 lanes, LOG_LEVEL=debug)
[PASS] overlays/prod matches data/release-inventory.csv (suffix -prod, 3 replicas, 100m/128Mi, 9 lanes, LOG_LEVEL=warn)
[PASS] the instance label was injected into selectors as well as metadata (includeSelectors: true)
[PASS] the prod images transformer rewrote the image to a fully-qualified registry path
[PASS] dev inherited the base image reference unchanged
[PASS] the prod JSON 6902 patch changed readinessProbe.periodSeconds 5 -> 10
[PASS] generated ConfigMap carries a content hash (tracklane-config-dev-m889h8f92h) and the Deployment reference was rewritten
[PASS] overlays/dev and overlays/prod render materially different output
[PASS] helm lint passes on chart/
[PASS] helm template renders the chart with data/values-dev.yaml
[PASS] helm template renders the chart with data/values-prod.yaml
[PASS] the values files drove replicaCount 1 (dev) and 3 (prod)
[PASS] the conditional PodDisruptionBudget renders in prod only
[PASS] the Deployment carries a checksum/config annotation
[PASS] --set-file carried all 9 rows of data/shipping-lanes.csv into the render
[PASS] the Argo CD Application lives in gitops/, not manifests/ (it needs a CRD)
[PASS] the Application declares repoURL, targetRevision, prune and selfHeal
[PASS] deployment/tracklane-dev is available in kcna-lab21 (applied from the dev overlay)
```

```
[PASS] no container in kcna-lab21 requests privileged: true
-----
25 passed, 0 failed, 0 skipped
```

Without helm installed the five Helm lines become `[SKIP]`; without a cluster the two cluster lines and the dry-run line become `[SKIP]`. Skips are not failures — the script exits 0:

```
-----
15 passed, 0 failed, 8 skipped
```

Full transcript, including both variants: `verification/expected-output.md`.

Lab 21 · Failure injection

Failure A — a Kustomize patch that silently matches nothing

This is Kustomize's characteristic failure and it is far more dangerous than a Helm error, because nothing goes wrong. Helm fails loudly at render time; Kustomize just... does not apply your patch.

Break the patch target names in the **prod** overlay — the environment where the patches genuinely matter:

```
cp kustomize/overlays/prod/kustomization.yaml /tmp/prod-kustomization.bak
sed -i.tmp 's|^      name: tracklane$|      name: tracklane-web|'
kustomize/overlays/prod/kustomization.yaml
grep -A6 '^patches:' kustomize/overlays/prod/kustomization.yaml | grep -vE '^\s*#'
```

```
patches:
- path: patch-resources.yaml
  target:
    group: apps
    version: v1
    kind: Deployment
    name: tracklane-web
```

Both patch targets in that file now name `tracklane-web`, which does not exist. Render it:

```
kubectl kustomize kustomize/overlays/prod >/dev/null; echo "render exit=$?"
render exit=0
```

Exit 0. No error. No warning. The render succeeded and both patches did nothing. Now look at what it produced:

```
kubectl kustomize kustomize/overlays/prod | grep -A3 'requests:'
kubectl kustomize kustomize/overlays/prod | grep 'periodSeconds:'
```

```
requests:
  cpu: 50m
  memory: 64Mi
securityContext:
  periodSeconds: 5
```

Production silently rendered with the base's resource floor and the base's probe timing. `cpu: 100m` never appeared; `periodSeconds: 10` never appeared. There was no error, `kubectl apply -k` would have succeeded, and the only symptom would be a production web tier scheduled with half the CPU it was sized for — surfacing days later as latency.

Diagnosis, in order:

1. Suspect it whenever a value you *know* you set does not appear in the render. Kustomize does not tell you.
2. **Diff the render against the base.** This is the reliable detector:

```
diff <(kubectl kustomize kustomize/base 2>/dev/null | grep -A3 'requests:') \
    <(kubectl kustomize kustomize/overlays/prod | grep -A3 'requests:') \
    && echo "NO DIFFERENCE — the prod overlay is not changing resources at all"
NO DIFFERENCE — the prod overlay is not changing resources at all
```

1. Compare the patch target against the **base** resource's name — remembering that targets match the name **before** nameSuffix:

```
grep -m1 -A1 '^metadata:' kustomize/base/deployment.yaml
grep -A6 '^patches:' kustomize/overlays/prod/kustomization.yaml | grep 'name:'

metadata:
  name: tracklane
  name: tracklane-web
```

tracklane $\neq$ tracklane-web. There is the bug.

The guard. Be clear about this: **Kustomize has no built-in option that turns an unmatched patch target into an error.** A `patches` entry whose `target` selects nothing is not a syntax error and not a warning; it is simply a no-op. Do not go looking for a flag.

The guard is therefore procedural, and it is the same one you would use for any renderer: **assert on the render in CI.** That is exactly what `verification/checks.sh` does when it compares both overlays against `data/release-inventory.csv`. Run it now with the sabotage still in place:

```
bash verification/checks.sh 2>&1 | grep -E '^\[FAIL\]|passed,'
[FAIL] overlays/prod: expected cpu request 100m
[FAIL] expected readinessProbe.periodSeconds 5 in dev and 10 in prod
13 passed, 2 failed, 8 skipped
```

Both silent no-ops were caught, and the script exits non-zero. That is the whole argument for grading on the *render* rather than on "did the apply succeed". Note the counts assume no helm and no cluster; with both present you would see the same two `[FAIL]` lines among more passes.

Restore the file and confirm you are back to green:

```
cp /tmp/prod-kustomization.bak kustomize/overlays/prod/kustomization.yaml
rm -f kustomize/overlays/prod/kustomization.yaml.tmp /tmp/prod-kustomization.bak
bash verification/checks.sh 2>&1 | tail -2
-----
15 passed, 0 failed, 8 skipped
```

Failure B — the chart's own guard rejects an unpinned tag

Requires helm. Read it if you do not have helm — the mechanism is `fail` in `_helpers.tpl`.

```
helm template tracklane-dev ./chart -f data/values-dev.yaml --set image.tag=latest
Error: execution error at (tracklane/templates/deployment.yaml:50:20): image.tag must not
be 'latest' — pin an immutable tag or a digest

Use --debug flag to render out invalid YAML
```

Diagnosis. The error names the file, the line and the reason. `_helpers.tpl` uses Helm's `fail` function:

```
sed -n '76,82p' chart/templates/_helpers.tpl
```

```

{{- define "tracklane.validateImage" -}}
{{- $tag := required "image.tag is required and must be pinned" .Values.image.tag -}}
{{- if eq $tag "latest" -}}
{{- fail "image.tag must not be 'latest' - pin an immutable tag or a digest" -}}
{{- end -}}
{{- printf "%s:%s" .Values.image.repository $tag -}}
{{- end -}}

```

Prove required fires too:

```

helm template tracklane-dev ./chart -f data/values-dev.yaml --set image.tag=null
Error: execution error at (tracklane/templates/deployment.yaml:50:20): image.tag is
required and must be pinned

```

Contrast the two failure modes, and remember it:

	Helm	Kustomize
Bad input	Fails at render time with a file, line and message	Renders successfully and quietly omits your change
Guard available	required, fail, JSON-schema values.schema.json	None built in — you must assert on the render
Where it bites	In CI, immediately	In production, later

Neither is safe without a CI step that checks the *render*. Helm just makes more mistakes impossible to ignore.

Lab 21 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
A patch has no effect; the render succeeds with exit 0	The patch <code>target</code> name/kind/group does not match any resource. Targets match the base name, before <code>nameSuffix</code>	<code>diff <(kubectl kustomize base) <(kubectl kustomize overlays/<env>)</code>	Correct the target to the base's <code>metadata.name</code> ; assert on the render in CI
<code>kubectl kustomize</code> fails with <code>json: unknown field "labels"</code>	<code>kubectl</code> embeds Kustomize v4, which predates the <code>labels: transformer</code>	<code>kubectl version --client</code>	Upgrade <code>kubectl</code> to v1.27+, or fall back to the deprecated <code>commonLabels:</code>
<code>security; file '../..data/x.csv' is not in or below '...'</code>	A generator references a file outside the customization root; Kustomize forbids this by default	<code>grep -rn 'files:' kustomize/</code>	Keep generator inputs inside the overlay directory (as this lab does), or accept <code>--load-restrictor LoadRestrictionsNone</code> and its risks
Config changed but the Pods still serve the old values	<code>envFrom</code> reads a <code>ConfigMap</code> once at container start, and nothing changed the Pod template	Helm: <code>grep checksum/config</code> . Kustomize: <code>check disableNameSuffixHash</code>	Keep the generator hash on (Kustomize) or the <code>checksum/config</code> annotation (Helm)
Old hashed <code>ConfigMaps</code> accumulate in the namespace	<code>kubectl apply -k</code> creates the new object and never deletes the old	<code>kubectl -n kcna-lab21 get cm grep tracklane-config</code>	Delete the stale ones by name, or adopt a GitOps controller with <code>prune: true</code>
<code>helm upgrade</code> fails with <code>field is immutable on spec.selector</code>	A changing label (version, chart version) leaked into the selector template	<code>helm template ... grep -A4 'selector:'</code>	Split selector labels from descriptive labels, as <code>_helpers.tpl</code> does. The only in-place remedy is to delete and recreate

Symptom	Likely cause	Diagnostic command	Fix
helm template fails with nil pointer evaluating interface {}	A template dereferences a key that has no default in values.yaml	helm template ./chart --debug	Add the default to values.yaml, or guard with {{- with }} / default
Both "environments" behave as one; Services return each other's Pods	The environment label was added with includeSelectors: false, so the selectors are identical	kubectl kustomize overlays/<env> grep -A4 '^ selector:'	Set includeSelectors: true for the instance label, as the overlays here do
no matches for kind "Application" in version "argoproj.io/v1alpha1"	Argo CD is not installed, so its CRD does not exist	kubectl get crd grep argoproj	Expected in this lab. On a real cluster, install Argo CD first
Argo CD reports permanent OutOfSync on replicas	An HPA owns that field and the controller keeps reverting it	kubectl get hpa -A	Add ignoreDifferences for /spec/replicas, as the sample Application does

Lab 21 · Cleanup

Everything this lab created is inside `kcna-lab21`. **No cluster-scoped objects, no CRDs, no controllers, no Helm releases** — `helm template` renders to stdout and never contacts a cluster.

```
kubectl delete namespace kcna-lab21 --wait=true
namespace "kcna-lab21" deleted
```

Confirm no Helm release was ever created (if you have helm):

```
helm list --all-namespaces | grep tracklane || echo "no tracklane helm release exists"
no tracklane helm release exists
```

Remove the local render artefacts and any backup files from the failure injection:

```
rm -f /tmp/tracklane-dev.yaml /tmp/tracklane-prod.yaml /tmp/prod-kustomization.bak
rm -f kustomize/overlays/*/kustomization.yaml.tmp
kustomize/overlays/*/kustomization.yaml.bak
echo "local artefacts removed"
local artefacts removed
```

Confirm the lab tree is back to its committed state and still renders:

```
kubectl kustomize kustomize/overlays/dev >/dev/null && \
kubectl kustomize kustomize/overlays/prod >/dev/null && \
echo "both overlays still render"
kubectl get namespace kcna-lab21
both overlays still render
Error from server (NotFound): namespaces "kcna-lab21" not found
```

Lab 21 · What you learned

- **Copy-paste-and-edit does not scale past one environment.** The failure it produces is not "messy YAML" — it is a config value silently taking the wrong value in production, which is exactly the incident in Section 3.

- **A Helm chart is `Chart.yaml` + `values.yaml` + `templates/`.** `version` is the chart's; `appVersion` is the software's. Files beginning with `_` define named templates and render to nothing. `NOTES.txt` prints after install and is not an object.
- **Values precedence is `values.yaml` < `-f` < `--set` < `--set-file`,** and you proved it by overriding a replica count from the command line.
- **Separate selector labels from descriptive labels.** `spec.selector` is immutable; a chart that puts a version into it cannot be upgraded in place. `_helpers.tpl` shows the correct split.
- **`checksum/config` forces a rollout when configuration changes.** Kustomize achieves the same thing with generator name hashes — and turning `disableNameSuffixHash: true` on reintroduces the bug.
- **Helm templates; Kustomize patches.** Kustomize bases stay valid YAML, so `kubeconform`, editors and `kubectl apply` all work on them. Only Helm can conditionally *create* a resource — which is why prod renders a `PodDisruptionBudget` and dev does not.
- **Strategic-merge patches address list items by merge key; JSON 6902 patches address them by index** and can `remove`. Prefer strategic merge; reach for 6902 only when you must.
- **Kustomize's characteristic failure is silence.** A patch whose target matches nothing renders exit 0 and changes nothing. The defence is to assert on the *render* in CI — which is why this lab is graded on `kubectl kustomize` output compared against `data/release-inventory.csv`, not on whether an apply succeeded.
- **GitOps is a delivery model, not a packaging tool.** Desired state in Git, pulled by an in-cluster agent, continuously reconciled. `prune` makes Git authoritative about deletion; `selfHeal` corrects manual drift; `ignoreDifferences` concedes fields to other controllers such as an HPA.
- **A GitOps controller is an ordinary controller with a CRD.** You confirmed this the honest way — by watching the API server reject `kind: Application` because the CRD is not installed.
- **Argo CD and Flux implement the same loop** with different object models. Neither is more "GitOps" than the other.

Lab 21 · Further reading

- Helm documentation — *Charts* (the authoritative reference for chart structure): <https://helm.sh/docs/topics/charts/>
- Helm documentation — *Values files and precedence*: https://helm.sh/docs/chart_template_guide/values_files/
- Helm documentation — *Named templates* (`_helpers.tpl`): https://helm.sh/docs/chart_template_guide/named_templates/
- Helm documentation — *Chart best practices: labels and annotations*: https://helm.sh/docs/chart_best_practices/labels/
- Helm — installation: <https://helm.sh/docs/intro/install/>
- Kustomize documentation — *Kustomization file reference* (labels, patches, replicas, images, generators): <https://kubectl.docs.kubernetes.io/references/kustomize/kustomization/>
- Kubernetes documentation — *Declarative management of Kubernetes objects using Kustomize*: <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>
- RFC 6902 — *JSON Patch* (the `op/path/value` format): <https://datatracker.ietf.org/doc/html/rfc6902>

- Kubernetes documentation — *Recommended labels*:
<https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>
- OpenGitOps — *GitOps Principles v1.0* (a CNCF project): <https://opengitops.dev/>
- Argo CD documentation — *Application specification reference*:
<https://argo-cd.readthedocs.io/en/stable/user-guide/application-specification/>
- Flux documentation — *Kustomization* and *GitRepository*:
<https://fluxcd.io/flux/components/kustomize/kustomizations/>
- CNCF KCNA Curriculum — *Cloud Native Application Delivery*: <https://github.com/cncf/curriculum>

Day 5 — Observability, Troubleshooting & Assessment

Day 5 Learning Focus

Learn to see the cluster and to fix it. Probes and self-healing, events, logs and the metrics pipeline, Prometheus exposition and the wider observability stack; then a disciplined troubleshooting triage across the application, node and control-plane layers. The day closes with the WSQ assessment.

Field	Value
Instructional hours	5 hours + 3 hours assessment
Class window	9:30 AM – 6:30 PM
Registered topics	7. Logging and Monitoring, 8. Troubleshooting
KCNA modules	—
Learning outcomes	LO6
Abilities / Knowledge	A6 / K7
Labs	4 — Lab 22, Lab 23, Lab 24, Lab 25
Hands-on minutes	195 minutes
Deck slides	479–568

Session structure. The table below is the timetable the trainer works to. Breaks are not counted as instructional time.

Time	Duration	Session	Mode
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:20 PM	1 h 30 m	Session 3 · Recap & Briefing for Assessment	Review · Briefing
3:20 – 3:30 PM	10 m	Afternoon Break	Break
3:30 – 4:30 PM	1 h 00 m	Assessment 1 — Written Assessment (SAQ)	Assessment
4:30 – 6:30 PM	2 h 00 m	Assessment 2 — Practical Performance (PP)	Assessment

Day 5 Concepts

Monitoring answers known questions; observability lets you ask new ones

Monitoring is the practice of collecting a predetermined set of signals and alerting on thresholds you decided in advance. **Observability** is the property of a system that lets you answer questions you did not anticipate, from its external outputs, without shipping new code. Monitoring is a subset of what observability enables. Both are built from three signals:

- **Metrics** — cheap, numeric, aggregatable time series. Best for "is it broken?" and for alerting. Poor at "why?", because cardinality limits how much context you can attach.
- **Logs** — high-detail discrete events. Best for "what exactly happened to this one request?". Expensive at volume.

- **Traces** — the causal path of a single request across services, with timing per span. Best for "which of my fourteen services is slow?".

Reach for metrics first to establish *that* something is wrong and *where*, traces to narrow it to a component, and logs to find the specific cause.

Two instrumentation recipes are worth memorising. **RED**, for request-driven services: **R**ate, **E**rrors, **D**uration. **USE**, for resources: **U**tilisation, **S**aturation, **E**rrors. Between them they cover most of what you need on a first pass.

SLI, SLO, SLA and the error budget are a chain. An **SLI** is a measured indicator — the proportion of requests served under 300 ms. An **SLO** is your internal target for that indicator — 99.9% over 28 days. An **SLA** is a *contract* with a customer, including consequences, and is always looser than the SLO. The **error budget** is $1 - \text{SLO}$: at 99.9% you may fail 0.1% of requests, which over 28 days is about 40 minutes of total unavailability. The budget is the thing that makes the conversation about shipping versus stability quantitative rather than political. And the alerting rule that follows from it: **if an alert does not correspond to something a user can feel, it is not an alert** — it is a dashboard.

Health: three probes, three consequences

The kubelet runs three probes, and the difference between them is entirely in what happens when they fail.

- **livenessProbe fails** → **the container is killed and restarted**. Use it for unrecoverable states such as a deadlock. A liveness probe that checks a downstream dependency turns one dependency's outage into a cluster-wide restart storm — this is the classic mistake.
- **readinessProbe fails** → **the Pod is removed from Service endpoints** but is left running. Use it for "temporarily cannot serve": warming a cache, a full queue, a dependency that is briefly unavailable.
- **startupProbe fails** → **the container is killed**, but while it is running it *disables* the liveness and readiness probes. Use it for slow-starting applications so you can keep a tight liveness timeout without the startup period tripping it.

Each takes `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `successThreshold` and `failureThreshold`, and one of `httpGet`, `tcpSocket`, `exec` or `grpc`. Self-healing in Kubernetes is exactly this plus the `restartPolicy` and the controller loops: a failed liveness probe restarts a container, a crash-looping Pod is backed off exponentially, a failed node's Pods are eventually evicted and recreated elsewhere by their controller.

Events, logs and the metrics pipeline

Events are ordinary API objects with a short TTL — one hour by default — that record what the control plane and kubelet did. They are the first place to look when something did not happen. They can be filtered with **field selectors** (`--field-selector`

`involvedObject.name=web-0,type=Warning`) as well as sorted (`--sort-by=.lastTimestamp`).

Because they expire, an incident older than an hour has no events left: if you want them retained, ship them.

Logs in Kubernetes are whatever a container writes to `stdout` and `stderr`. The runtime writes them to files on the node, the kubelet rotates them, and `kubectl logs` reads them back through the API. `--previous` reads the previous container instance, which is the only way to see why a crash-looping container died. Node-level log files are lost when the node is. Getting logs off the node is done by a

node-level agent as a DaemonSet (the standard approach — one agent per node, no application changes), by a **sidecar** per Pod (flexible, expensive), or by the **application shipping directly** (no infrastructure, but couples the app to the log backend).

The **resource metrics pipeline** is a narrow, specific thing: cAdvisor inside the kubelet exposes per-container CPU and memory; **metrics-server** scrapes every kubelet, keeps roughly the last minute in memory, and serves the `metrics.k8s.io` API. That is what `kubectl top` and the HorizontalPodAutoscaler read. metrics-server is **not a monitoring system** — it stores nothing, it has no history, no query language and no alerting, and it must not be used as one.

Prometheus and the observability pipeline

Prometheus scrapes an HTTP endpoint that exposes metrics in a simple text **exposition format**, stores them as time series identified by a name plus labels, and queries them with **PromQL**. The four metric types:

- **Counter** — only ever increases (or resets to zero on restart). Never graph a counter directly; wrap it in `rate()` or `increase()` to get a per-second rate that handles resets correctly.
- **Gauge** — goes up and down. Temperature, queue depth, memory in use.
- **Histogram** — observations bucketed into cumulative `_bucket` series plus `_sum` and `_count`. Quantiles are computed server-side with `histogram_quantile()`, and because the buckets are defined at instrumentation time, choosing them badly makes your p99 meaningless.
- **Summary** — quantiles computed client-side. Cheaper to query, but summaries **cannot be aggregated across instances**, which is usually disqualifying.

Prometheus **pulls**. Pull makes the scrape target's health itself a signal (`up == 0`), makes targets trivially testable with `curl`, and centralises scrape configuration. Push, via the Pushgateway, is reserved for short-lived batch jobs that will not exist when the scrape comes. Alerting rules are evaluated inside Prometheus and fired alerts are handed to **Alertmanager**, which deduplicates, groups, silences, inhibits and routes them to a receiver.

OpenTelemetry is not a rival to Prometheus. It is a vendor-neutral standard and set of SDKs and a **Collector** for *producing and shipping* traces, metrics and logs; Prometheus is a *storage and query* system for metrics. The common architecture uses OpenTelemetry instrumentation feeding a Collector, which exports metrics to Prometheus and traces to **Jaeger** or an equivalent.

Troubleshooting: phase first, cause second

The discipline that separates people who fix clusters from people who guess is refusing to theorise before establishing the phase. Work the tree, stop at the first **no**:

1. **Does the object exist?** `kubectl get` in the right namespace. Far more "missing Pods" are namespace scoping errors than deletions.
2. **Did the scheduler place it?** A Pod in `Pending` with no `nodeName` is a *scheduling verdict*, and `kubectl describe pod` prints the reason: insufficient resources, an unsatisfiable affinity, an untolerated taint, or an unbound PVC.
3. **Could the image be fetched?** `ImagePullBackOff` / `ErrImagePull` — a typo, a private registry with no `imagePullSecret`, or a rate limit. `ErrImageNeverPull` is different: it means `imagePullPolicy: Never` and the image is not on the node.

4. **Did the container start and stay up?** `CrashLoopBackOff` is a back-off timer, not a cause. `kubectl logs --previous` has the cause. Exit code **137** is a SIGKILL, usually an OOMKill — confirm in `describe` under `Last State`. Exit code **1** is the application.
5. **Is it Ready?** `Running` but 0/1 is a readiness probe failing, and that also removes it from Service endpoints.
6. **Does the Service have backends?** `kubectl get endpointslices` — empty means selector mismatch, unready Pods, or a port mismatch.
7. **Is it a network policy?** Reachable from one namespace and not another, with healthy endpoints, points at `NetworkPolicy` — or at a CNI that does not enforce it.

At the node layer, `kubectl get nodes` plus `kubectl describe node` gives you the conditions — `Ready`, `MemoryPressure`, `DiskPressure`, `PIDPressure` — and `kubectl top nodes` gives current usage. A `NotReady` node usually means the kubelet stopped reporting; the evidence is on the node (`systemctl status kubelet`, `journalctl -u kubelet`).

At the control-plane layer, **read the blast radius to identify the component**. If `kubectl` itself fails to connect, it is the API server or your kubeconfig. If everything reads fine but new Pods stay `Pending` forever with no events, it is the scheduler. If Deployments do not create `ReplicaSets`, or deleted objects are not garbage-collected, it is the controller manager. If reads are inconsistent or the API server reports storage errors, it is `etcd`. In a `kubeadm` cluster all four are static Pods in `kube-system`, so `kubectl -n kube-system get pods` and `kubectl -n kube-system logs <pod>` are the first two commands — falling back to the node's `crictl ps -a` and the files in `/etc/kubernetes/manifests/` when the API server is the thing that is down.

Finally, prefer **read-only inspection surfaces** while diagnosing: `kubectl describe`, `kubectl logs`, `kubectl get -o yaml`, `kubectl events`, `kubectl top`, `kubectl auth can-i` and `kubectl debug` with an ephemeral container. Changing things while you are still establishing the phase destroys the evidence.

Ecosystem, principles and community

The course closes where it started. CNCF projects move Sandbox → Incubating → Graduated, and each promotion requires demonstrable adopters, governance, committer diversity and — for graduation — a completed independent security audit. Participation is through **SIGs** (Special Interest Groups) that own areas of the project, **Working Groups** that span them for a time-boxed goal, and the Kubernetes Enhancement Proposal (**KEP**) process by which changes are proposed, reviewed and staged through alpha, beta and GA.

KCNA itself is a **60-question, 90-minute, multiple-choice, online proctored** exam with a **75% pass mark**, valid for **two years**, with no prerequisites, one retake included and a 12-month eligibility window from purchase. It is the foundational rung; **CKA**, **CKAD** and **CKS** are the performance-based certifications that follow it.

Day 5 Labs

Day 5 has 4 labs, listed below and then set out in full. Work through them in order; each runs in its own namespace and cleans up after itself.

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 22	Probes, Health and Self-Healing	kcna-lab22	45	LO6 / A6 / K7	498

Lab	Title	Namespace	Minutes	LO / A / K	Deck slide
Lab 23	Events, Logs, Field Selectors and the Metrics Server	kcna-lab23	50	LO6 / A6 / K7	512
Lab 24	Metrics, Prometheus Exposition and the Observability Pipeline	kcna-lab24	45	LO6 / A6 / K7	528
Lab 25	Troubleshooting Triage: Application, Node and Control Plane	kcna-lab25	55	LO6 / A6 / K7	549

Lab 22 — Probes, Health and Self-Healing

Field	Value
Lab ID	Lab 22
Title	Probes, Health and Self-Healing
Day / Topic	Day 5 · Logging and Monitoring
Duration	45 minutes
Namespace	kcna-lab22
Learning outcome	LO6 — Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.
Ability	A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework
Knowledge	K7 Interactions among various IT components
Deck slide	Slide 498
Repository path	courseware/labs/lab-22-probes-health-selfhealing/

Goal. Configure liveness, readiness and startup probes on the Depot Portal workloads and prove that each one has a different consequence: restart, endpoint withdrawal, or a grace window for a slow boot.

What you will produce:

- A depot-portal Deployment whose readiness probe demonstrably adds and removes its Pod address from the Service EndpointSlice without ever restarting the container
- A manifest-indexer Deployment that boots for 40 seconds and reaches Ready with zero restarts because a startupProbe grants it a 120-second budget
- A completed probe-tuning worksheet recording each probe's failure budget, observed timing and consequence

How to read the expected-output blocks. Every command below is followed by the output you should see. Pod name suffixes, IP addresses, `AGE` columns and timestamps are generated at runtime and **will differ on your cluster** — the *shape*, the column headers, the status strings and the event reasons are what you are matching against. Nothing in this lab asks you to match a random hash.

Lab 22 · Objective

By the end of this lab you will be able to:

1. **Distinguish** the three probe types by their *consequence*, not their syntax — a failing **liveness** probe restarts the container, a failing **readiness** probe removes the Pod from the Service's EndpointSlice, and a **startup** probe suspends both while a slow application boots.
2. **Configure** `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds` and `failureThreshold` and calculate the resulting *failure budget* in seconds.
3. **Prove** that readiness gates traffic by watching a Service's EndpointSlice lose and regain an address while the container is never restarted.

4. **Diagnose** a restart loop caused by a liveness probe that is more aggressive than the application's real startup time, and repair it with a startup probe.

Lab 22 · Prerequisites

- A running single-node **kind** cluster and a `kubectl` whose minor version is within one of the server.

Verify:

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

- Cluster-admin rights within your own namespace (the standard kind admin kubeconfig is sufficient).
- You are in the lab directory. Every relative path below is from here:

```
cd courseware/labs/lab-22-probes-health-selfhealing
```

- Images used, all **pinned**: `nginx:1.27-alpine`, `busybox:1.36`, `alpine:3.20`. Pull them once before you start so probe timings are not distorted by image download time on a slow connection:

```
kubectl run image-warm --rm -i --restart=Never --image=busybox:1.36 -n default -- true
pod "image-warm" deleted
```

Lab 22 · Scenario

Meridian Freight Pte Ltd runs the **Depot Portal**, the web application its drivers use to check in shipments at the Tuas depot. Last Thursday the Depot Platform Squad shipped release 6.0.0 and opened incident **INC-4417**.

The rollout reported "2/2 available" within seconds, the dashboard went green, and drivers immediately started receiving HTTP 502 at the gate. The new replicas had been added to the Service's backend list **before** they could serve a single request, because the Deployment had no readiness probe — Kubernetes had no way to know the difference between *the process has started* and *the process can do work*.

The follow-up ticket **DEP-1012** also flagged a second workload, the `manifest-indexer`, which spends about forty seconds rebuilding an in-memory shipment index at boot. An engineer "hardened" it by adding a tight liveness probe. It has been in `CrashLoopBackOff` ever since — and nothing is actually wrong with it.

You are the on-call platform engineer. Your job today is to make health signals mean something.

Lab 22 · Step-by-step procedure

Part 1 — Create the namespace and the content ConfigMap

Step 1.1 — Create the lab namespace.

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab22 created
```

Step 1.2 — Confirm the Pod Security Admission labels landed. The lab runs under the `baseline` profile; nothing in this lab is privileged.

```
kubectl get namespace kcna-lab22 --show-labels

NAME          STATUS    AGE      LABELS
kcna-lab22    Active    4s       app.kubernetes.io/part-of=depot-portal,kcna.tertiaryinfotech.com/day=5,kcna.tertiaryinfotech.com/lab=lab-22,kubernetes.io/metadata.name=kcna-lab22,pod-security.kubernetes.io/enforce-version=latest,pod-security.kubernetes.io/enforce=baseline
```

Step 1.3 — Build the ConfigMap from the two HTML files in `data/`. These are the real probe targets: `index.html` is what liveness asks for, `ready.html` is what readiness asks for.

```
kubectl create configmap depot-portal-content \
  --from-file=index.html=data/depot-portal-content/index.html \
  --from-file=ready.html=data/depot-portal-content/ready.html \
  -n kcna-lab22

configmap/depot-portal-content created
```

Step 1.4 — Verify the keys.

```
kubectl get configmap depot-portal-content -n kcna-lab22 -o jsonpath='{.data}' | tr ',' '\n' | cut -c1-60

{"index.html":"<!doctype html>\n<!--\n  Meridian Freight Pt\n  "ready.html":"READY depot-portal sg-tuas inbound\n# Meridia
```

Part 2 — Deploy depot-portal with readiness and liveness

Step 2.1 — Read the probe stanzas before you apply them. Open `manifests/10-depot-portal.yaml` and locate the two probes on the `portal` container. Note that they point at **different paths on purpose**:

Probe	Path	Budget before it acts	What happens when it fails
readinessProbe	/ready.html	$3 + (5 \times 2) = 13 \text{ s}$	Pod address removed from the Service EndpointSlice. Container keeps running.
livenessProbe	/index.html	$10 + (10 \times 3) = 40 \text{ s}$	Container is killed with SIGTERM and restarted. RESTARTS increments.

Step 2.2 — Apply the Deployment and its Service.

```
kubectl apply -f manifests/10-depot-portal.yaml

deployment.apps/depot-portal created
service/depot-portal created
```

Step 2.3 — Watch the Pods become Ready. The `READY` column is the *readiness* column: 1/1 means one of one containers is passing its readiness probe.

```
kubectl get pods -n kcna-lab22 -l app=depot-portal -w

NAME                                READY    STATUS             RESTARTS   AGE
depot-portal-7c9b6d4f8b-2q6x1      0/1      Init:0/1           0          2s
depot-portal-7c9b6d4f8b-h4kzp      0/1      Init:0/1           0          2s
depot-portal-7c9b6d4f8b-2q6x1      0/1      PodInitializing    0          4s
depot-portal-7c9b6d4f8b-h4kzp      0/1      PodInitializing    0          4s
depot-portal-7c9b6d4f8b-2q6x1      0/1      Running            0          6s
```

depot-portal-7c9b6d4f8b-h4kzp	0/1	Running	0	6s
depot-portal-7c9b6d4f8b-2q6xl	1/1	Running	0	9s
depot-portal-7c9b6d4f8b-h4kzp	1/1	Running	0	9s

Press **Ctrl-C** to stop watching.

Read this carefully. There is a real gap between `Running` (~6 s) and `1/1` (~9 s). That gap is `initialDelaySeconds: 3` plus one probe round trip. Without a readiness probe there would be no gap at all — the Pod would be declared `Ready` the instant it started running, which is exactly what caused INC-4417.

Step 2.4 — Confirm the init container did its job.

```
kubectl logs -n kcna-lab22 -l app=depot-portal -c seed-content --tail=5 --prefix
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] [seed] webroot populated:
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] total 8
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] -rw-r--r--      1 root      root
383 Sep  5 06:12 index.html
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] -rw-r--r--      1 root      root
341 Sep  5 06:12 ready.html
```

Part 3 — Prove that readiness gates traffic

This is the heart of the lab. You are going to break readiness on **one replica** and watch that replica's IP leave the Service backend list.

Step 3.1 — Look at the Service's backends. Since Kubernetes v1.33 the `v1 Endpoints` object is deprecated; **EndpointSlice** is the current API and is what `kube-proxy` actually consumes.

```
kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-portal-xk4t9	IPv4	80	10.244.0.14,10.244.0.15	62s

Two addresses — one per Ready replica.

Step 3.2 — Start the in-cluster client Pod. You will curl the Service from inside the cluster; nothing is exposed outside it.

```
kubectl apply -f manifests/40-probe-client.yaml
kubectl wait --for=condition=Ready pod/probe-client -n kcna-lab22 --timeout=60s

pod/probe-client created
pod/probe-client condition met
```

Step 3.3 — Confirm the Service answers. `alpine:3.20` ships `BusyBox wget`, not `curl`, so use `wget -qO-`.

```
kubectl exec -n kcna-lab22 probe-client -- wget -qO- http://depot-portal.kcna-
lab22.svc.cluster.local/ready.html

READY depot-portal sg-tuas inbound
# Meridian Freight Pte Ltd - Depot Portal readiness marker.
#
# This file is the READINESS target. The application would normally create it
# only after it has opened its database pool and warmed the shipment cache.
# In this lab you delete and restore it by hand so you can watch the Service
# EndpointSlice gain and lose this Pod, WITHOUT the container being restarted.
```

Step 3.4 — Capture the name of **one** replica into a shell variable. You will break only this one.

```
VICTIM=$(kubectl get pods -n kcna-lab22 -l app=depot-portal \
-o jsonpath='{.items[0].metadata.name}')
echo "victim = $VICTIM"

victim = depot-portal-7c9b6d4f8b-2q6x1
```

Step 3.5 — In a **second terminal**, start watching the EndpointSlice. Leave this running for the rest of Part 3.

```
kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal -w
```

Step 3.6 — Back in the first terminal, delete the readiness marker inside the victim container. Liveness still passes, because `/index.html` is untouched.

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- rm /usr/share/nginx/html/ready.html
(no output - success)
```

Step 3.7 — Within about 13 seconds, the watch in your second terminal drops an address:

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-portal-xk4t9	IPv4	80	10.244.0.14,10.244.0.15	3m
depot-portal-xk4t9	IPv4	80	10.244.0.15	3m12s

Step 3.8 — Confirm the Pod is 0/1 but that `RESTARTS` is still 0. This is the single most important observation in the lab.

```
kubectl get pods -n kcna-lab22 -l app=depot-portal
```

NAME	READY	STATUS	RESTARTS	AGE
depot-portal-7c9b6d4f8b-2q6x1	0/1	Running	0	3m20s
depot-portal-7c9b6d4f8b-h4kzp	1/1	Running	0	3m20s

Not Ready, not restarted, still Running. Readiness withdrew the Pod from service; it did not punish the container.

Step 3.9 — Read the Pod conditions and the event the kubelet raised.

```
kubectl describe pod -n kcna-lab22 "$VICTIM" | sed -n '/^Conditions:/,/^Volumes:/p'
```

```
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                         True
  Ready                              False
  ContainersReady                    False
  PodScheduled                       True
```

```
kubectl events -n kcna-lab22 --for pod/"$VICTIM" --types=Warning
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
10s (x3 over 20s)	Warning	Unhealthy	Pod/depot-portal-7c9b6d4f8b-2q6x1	Readiness probe failed: HTTP probe failed with statuscode: 404

Step 3.10 — Verify the Service still works. Traffic now only reaches the healthy replica — this is the graceful degradation that readiness buys you.

```
kubectl exec -n kcna-lab22 probe-client -- \
sh -c 'for i in 1 2 3 4 5; do wget -qO- http://depot-portal.kcna-
lab22.svc.cluster.local/ready.html | head -1; done'
```

```
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
```

Five requests, five successes, zero errors — while one of your two replicas is broken. Compare that to INC-4417, where roughly half the requests returned 502.

Step 3.11 — Restore readiness by copying the marker back from the read-only ConfigMap mount.

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- \
  cp /etc/depot/content/ready.html /usr/share/nginx/html/ready.html
cp: can't stat '/etc/depot/content/ready.html': No such file or directory
command terminated with exit code 1
```

Expected obstacle — read it, do not skip it. The ConfigMap volume is mounted into the `init` container only, not into `portal`. That is deliberate: the runtime container should not be able to see its own seed data. Recreate the file directly instead:

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- \
  sh -c 'echo "READY depot-portal sg-tuas inbound" > /usr/share/nginx/html/ready.html'
(no output — success)
```

Step 3.12 — Within ~5 seconds the address returns. Your second-terminal watch prints:

```
depot-portal-xk4t9   IPv4      80      10.244.0.15,10.244.0.14   4m10s
kubectl get pods -n kcna-lab22 -l app=depot-portal
NAME                                READY   STATUS    RESTARTS   AGE
depot-portal-7c9b6d4f8b-2q6xl      1/1     Running   0           4m15s
depot-portal-7c9b6d4f8b-h4kzp      1/1     Running   0           4m15s
```

Stop the watch in the second terminal with `Ctrl-C`.

Step 3.13 — Record what you observed in `data/probe-tuning-worksheet.csv`, column `observed_time_to_first_success_s`, row `depot-portal / readiness`.

Part 4 — A slow starter done right: the startup probe

Step 4.1 — Apply the `manifest-indexer`. It sleeps 40 seconds before it listens, and its `startupProbe` grants a 120 second budget.

```
kubectl apply -f manifests/20-manifest-indexer-startup.yaml
deployment.apps/manifest-indexer created
service/manifest-indexer created
```

Step 4.2 — Watch it. It stays `0/1 Running` for about 40 seconds and then flips to `1/1` — **with zero restarts**.

```
kubectl get pods -n kcna-lab22 -l app=manifest-indexer -w
NAME                                READY   STATUS             RESTARTS   AGE
manifest-indexer-6d84f9c7b5-vn2tq   0/1     ContainerCreating   0           1s
manifest-indexer-6d84f9c7b5-vn2tq   0/1     Running             0           3s
manifest-indexer-6d84f9c7b5-vn2tq   1/1     Running             0           45s
```

`Ctrl-C` to stop.

Step 4.3 — Confirm no restarts occurred and the startup probe is what held the line.

```
kubectl describe pod -n kcna-lab22 -l app=manifest-indexer | grep -E 'Restart Count|
Liveness|Readiness|Startup'
Restart Count: 0
Liveness:      http-get http://:http/index.html delay=0s timeout=2s period=10s
#success=1 #failure=3
```

```

    Readiness:      http-get http://:http/index.html delay=0s timeout=2s period=5s
#success=1 #failure=2
    Startup:        http-get http://:http/index.html delay=0s timeout=2s period=5s
#success=1 #failure=24

```

Step 4.4 — Read the startup events. You will see the startup probe failing repeatedly and then simply stopping — it is never run again once it succeeds.

```

kubectl events -n kcna-lab22 --for deployment/manifest-indexer 2>/dev/null | head -5
kubectl get events -n kcna-lab22 --field-selector reason=Unhealthy --sort-
by=.lastTimestamp | tail -3

```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
55s	Warning	Unhealthy	Pod/manifest-indexer-6d84f9c7b5-vn2tq	Startup probe failed: Get "http://10.244.0.17:8080/index.html": dial tcp 10.244.0.17:8080: connect: connection refused

The mechanism. `connection refused` is the correct, expected failure while the process is still sleeping. The startup probe absorbs all of it. Liveness and readiness are not evaluated at all during this window — that is the entire point of the third probe type.

Part 5 — Verification

Run the bundled checks.

```
bash verification/checks.sh
```

See `verification/expected-output.md` for the full expected transcript and the pass criteria.

Part 6 — Failure injection (mandatory)

You will now reproduce ticket **DEP-1012**: the same slow application, but with the startup probe removed and liveness tuned aggressively.

Step 6.1 — Read the arithmetic before you apply it.

```

liveness budget = initialDelaySeconds 5 + (periodSeconds 5 × failureThreshold 2)
                  = 15 seconds
real start time = 40 seconds
15 < 40 -> the container can never finish starting

```

Step 6.2 — Apply the broken Deployment.

```

kubectl apply -f manifests/30-manifest-indexer-outage.yaml
deployment.apps/manifest-indexer-outage created

```

Step 6.3 — Watch for about 90 seconds. `RESTARTS` climbs and the Pod enters `CrashLoopBackOff`.

```

kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage -w

```

NAME	READY	STATUS	RESTARTS	AGE
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	0	5s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	1 (2s ago)	22s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	2 (1s ago)	41s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	CrashLoopBackOff	2 (12s ago)	55s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	3 (21s ago)	75s

Ctrl-C to stop.

Step 6.4 — Diagnose. The `Warning Unhealthy + Killing` pair is the signature of a liveness-induced restart loop.

```
BROKEN=$(kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage \
-o jsonpath='{.items[0].metadata.name}')
kubectl describe pod -n kcna-lab22 "$BROKEN" | tail -14

Events:
  Type            Reason            Age           From            Message
  ----            -
  Normal          Scheduled         81s          default-scheduler Successfully assigned kcna-lab22/manifest-indexer-outage-5f7c8b9d6-t8pnw to kind-control-plane
  Normal          Pulled            22s (x4 over 80s) kubelet          Container image "busybox:1.36" already present on machine
  Normal          Created           22s (x4 over 80s) kubelet          Created container: indexer
  Normal          Started           22s (x4 over 80s) kubelet          Started container indexer
  Warning          Unhealthy         12s (x8 over 74s) kubelet          Liveness probe failed: Get "http://10.244.0.18:8080/index.html": dial tcp 10.244.0.18:8080: connect: connection refused
  Normal          Killing           12s (x4 over 66s) kubelet          Container indexer failed liveness probe, will be restarted
```

Step 6.5 — Confirm the application itself is innocent. Its own log shows it was always mid-warm-up when it was killed — it never got to print the second line.

```
kubectl logs -n kcna-lab22 "$BROKEN" --previous
[indexer] 06:31:04 warming shipment manifest index...
```

Exactly one line. The `index warm; serving on :8080` line never appears, because the kubelet killed the process at ~15 s and the sleep is 40 s.

Step 6.6 — Compare the two Deployments side by side to isolate the difference.

```
diff <(kubectl get deploy manifest-indexer -n kcna-lab22 -o
jsonpath='{.spec.template.spec.containers[0].livenessProbe}{"\n"}
{.spec.template.spec.containers[0].startupProbe}{"\n"}') \
<(kubectl get deploy manifest-indexer-outage -n kcna-lab22 -o
jsonpath='{.spec.template.spec.containers[0].livenessProbe}{"\n"}
{.spec.template.spec.containers[0].startupProbe}{"\n"}')
1,2c1,2
<
{"failureThreshold":3,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},"periodSeconds":10,"successThreshold":1,"timeoutSeconds":2}
<
{"failureThreshold":24,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},"periodSeconds":5,"successThreshold":1,"timeoutSeconds":2}
---
>
{"failureThreshold":2,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},"initialDelaySeconds":5,"periodSeconds":5,"successThreshold":1,"timeoutSeconds":2}
>
```

The broken Deployment has **no startup probe at all** (empty second line), and a liveness `failureThreshold` of 2 instead of 3.

Step 6.7 — Repair it. Patch a startup probe onto the running Deployment rather than re-applying a file, so you see the rollout heal in place.

```
kubectl patch deployment manifest-indexer-outage -n kcna-lab22 --type=strategic -p '
spec:
  template:
    spec:
      containers:
        - name: indexer
```

```

startupProbe:
  httpGet:
    path: /index.html
    port: http
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 24

```

```
deployment.apps/manifest-indexer-outage patched
```

Step 6.8 — Confirm the heal. The new ReplicaSet's Pod reaches 1/1 with `RESTARTS 0`.

```

kubectl rollout status deployment/manifest-indexer-outage -n kcna-lab22 --timeout=150s
kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage

Waiting for deployment "manifest-indexer-outage" rollout to finish: 1 old replicas are
pending termination...
deployment "manifest-indexer-outage" successfully rolled out
NAME                                READY   STATUS    RESTARTS   AGE
manifest-indexer-outage-6b5d497c84-w9jrx  1/1     Running   0          52s

```

Step 6.9 — Record the repair in `data/probe-tuning-worksheet.csv`, row `manifest-indexer-outage`, column `verdict`.

Step 6.10 — Remove only the injected Deployment.

```

kubectl delete deployment manifest-indexer-outage -n kcna-lab22
deployment.apps "manifest-indexer-outage" deleted

```

Lab 22 · Verification

Run the scripted checks:

```
bash verification/checks.sh
```

You have completed the lab when **all checks report PASS** and you can answer these three questions without looking them up:

1. A Pod is 0/1 `Running` with `RESTARTS 0`. Which probe is failing, and is the Service still sending it traffic? (*Readiness; no.*)
2. A Pod is 0/1 and `RESTARTS` is climbing. Which probe is failing? (*Liveness — or a startup probe whose budget has been exhausted.*)
3. Your application takes 40 s to boot and you want liveness to catch a hang within 30 s. Which probe do you add, and what must its budget exceed? (*A startup probe; its `periodSeconds` × `failureThreshold` must comfortably exceed 40 s.*)

Lab 22 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
Pod stuck 0/1 <code>Running</code> , <code>RESTARTS 0</code> , never becomes <code>Ready</code>	Readiness probe path returns non-2xx/3xx, or the port name in the probe does not match a <code>containerPort</code> name	<pre>kubectl describe pod <pod> -n kcna-lab22 grep -A2 Readiness then kubectl events -n kcna-lab22 --for pod/<pod></pre>	Correct the path or the port name; probe <code>port</code> : must match the <code>ports[].name</code> on the same container

Symptom	Likely cause	Diagnostic command	Fix
RESTARTS climbing, events show Liveness probe failed: connection refused shortly after start	Liveness budget is shorter than real startup time (the DEP-1012 bug)	<code>kubectl logs <pod> --previous — the app log stops mid-boot</code>	Add a <code>startupProbe</code> whose <code>periodSeconds × failureThreshold</code> exceeds worst-case boot time; leave liveness tight
EndpointSlice shows <unset> under ENDPOINTS while Pods look Ready	Service selector does not match the Pod template labels	<code>kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal -o yaml</code> and compare with <code>kubectl get pods --show-labels</code>	Align <code>spec.selector</code> on the Service with the Pod labels
Liveness probe failed: HTTP probe failed with statuscode: 404 immediately	Probe points at a file the image does not serve, or the webroot volume shadowed the image's default content	<code>kubectl exec <pod> -c portal -- ls /usr/share/nginx/html</code>	Point the probe at a path that exists, or seed the volume (this lab uses an init container for exactly this reason)
Probes appear to "not run" during the first minute	A <code>startupProbe</code> is present and still failing — liveness/readiness are suspended by design	<code>kubectl describe pod <pod> grep Startup</code>	Nothing to fix; this is correct behaviour. Shorten the startup budget only if boot is genuinely faster
<code>kubectl exec ... -c portal</code> returns container portal is not valid for pod	Wrong container name, or you targeted the init container after it completed	<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[*].name}'</code>	Use a name from that list; init containers cannot be <code>exec</code> -ed once they have terminated

Lab 22 · Cleanup

Delete **only** this lab's namespace. Everything you created lives inside it.

```
kubectl delete namespace kcna-lab22
namespace "kcna-lab22" deleted
kubectl get namespace kcna-lab22
Error from server (NotFound): namespaces "kcna-lab22" not found
```

This lab created **no** cluster-scoped objects and touched **nothing** in `kube-system`. Deleting the namespace is a complete cleanup.

Lab 22 · What you learned

- **Probes are three different questions with three different answers.** Liveness asks *should this container be restarted?*; readiness asks *should this Pod receive traffic?*; startup asks *has this container finished booting yet?* Confusing them causes outages in both directions — traffic sent to a cold replica (INC-4417) or a healthy replica killed forever (DEP-1012).
- **Readiness is a traffic-control mechanism, not a health mechanism.** You proved this by watching an address leave an EndpointSlice while `RESTARTS` stayed at 0, and by serving five successful requests through a degraded Deployment.

- **Failure budgets are arithmetic, not vibes.** `initialDelaySeconds + (periodSeconds × failureThreshold)` is the number that matters; compare it against the application's real worst-case timing.
- **The startup probe exists precisely so liveness can stay aggressive.** Without it you must choose between slow detection of hangs and killing slow starters. With it you get both.
- **EndpointSlice is the current API** for Service backends; the `v1 Endpoints` object was deprecated in Kubernetes v1.33 and should not be your first diagnostic in new work.

Further reading

- Kubernetes — *Configure Liveness, Readiness and Startup Probes*: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>
- Kubernetes — *Pod Lifecycle* → *Container probes*: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>
- Kubernetes — *EndpointSlices*: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- Kubernetes blog — *Endpoints deprecation (v1.33)*: <https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>
- CNCF — KCNA curriculum, *Cloud Native Architecture* → *Observability*: <https://github.com/cncf/curriculum>

Lab 23 — Events, Logs, Field Selectors and the Metrics Server

Field	Value
Lab ID	Lab 23
Title	Events, Logs, Field Selectors and the Metrics Server
Day / Topic	Day 5 · Logging and Monitoring
Duration	50 minutes
Namespace	kcna-lab23
Learning outcome	LO6 — Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.
Ability	A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework
Knowledge	K7 Interactions among various IT components
Deck slide	Slide 512
Repository path	courseware/labs/lab-23-events-logs-metrics/

Goal. Build the incident evidence trail: query Events as API objects with server-side field selectors, drive kubectl logs across containers and restarts, and establish exactly what kubectl top requires on a kind cluster that does not ship metrics-server.

What you will produce:

- A namespace emitting a real mixed event stream (Normal, Warning FailedMount, Warning BackOff) filtered with compound `--field-selector` queries
- A two-container Pod proving that only stdout/stderr reaches kubectl logs, plus a crash-looping container whose cause is only visible via `--previous`
- A completed event-triage worksheet recording each query, what it returned, and whether kubectl top was available (Path A install or Path B skip)

How to read the expected-output blocks. Pod name suffixes, IP addresses, `AGE/LAST SEEN` columns and timestamps are generated at runtime and **will differ on your cluster**. Match the *shape*, the column headers, the event reasons and the error strings.

Read the Part 5 warning before you start. `kubectl top` does **not** work on a stock kind cluster. That section is explicitly conditional.

Lab 23 · Objective

By the end of this lab you will be able to:

1. **Query Events as first-class API objects**, sort them meaningfully, and explain why the default listing order is misleading.
2. **Filter Events with `--field-selector`** on `type`, `reason`, `involvedObject.kind` and `involvedObject.name`, and state which fields are selectable and which are not.
3. **Explain the Event TTL surprise** — why an incident from two hours ago has no Events left — and name the mechanism that controls it.
4. **Drive `kubectl logs`** with `-c`, `--previous`, `--since`, `--tail`, `-f` and `--prefix`, and explain why only stdout/stderr is captured.

5. Explain why node-level log shipping exists by demonstrating a log that is invisible to `kubectl logs` until a sidecar makes it visible.
6. State precisely what `kubectl top` requires, install metrics-server on kind with the flag kind forces you to use, and explain why that flag would be a security defect in production.

Lab 23 · Prerequisites

- A running single-node **kind** cluster.

```
kubectl version  
Client Version: v1.37.0  
Kustomize Version: v5.8.1  
Server Version: v1.37.0
```

- You are in the lab directory:

```
cd courseware/labs/lab-23-events-logs-metrics
```

- Images used, all **pinned**: `busybox:1.36`, and — only if you choose to do Part 5 — `registry.k8s.io/metrics-server/metrics-server:v0.7.2`.
- **Lab 22 is a soft prerequisite.** You should already know what a Warning `Unhealthy` event looks like.

Lab 23 · Scenario

The Meridian Freight post-incident review for **INC-4417** stalled for an embarrassing reason: by the time the Depot Platform Squad opened the console the next morning, **the Events were gone**. The Slack thread contained screenshots of `kubectl get pods` and nothing else. Nobody could reconstruct the ordering of events, and the root cause had to be inferred.

The remediation ticket **DEP-1019** says, in full:

Build the evidence trail. Before the next incident we must be able to answer, from the cluster itself: what did the control plane say, in what order; what did the application print; and what was it consuming. Document what is retained, for how long, and what is not retained at all.

That is this lab.

Lab 23 · Step-by-step procedure

Part 1 — Namespace and log corpus

Step 1.1 — Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml  
namespace/kcna-lab23 created
```

Step 1.2 — Load the synthetic Depot Portal access log from `data/` into a ConfigMap. This 30-line corpus is what the `portal` container will replay, so the log stream you analyse has realistic structure: INFO, WARN and ERROR lines, request ids, upstream timeouts and a rising queue depth.

```
kubectl create configmap depot-access-log \
  --from-file=depot-portal-access.log=data/depot-portal-access.log \
  -n kcna-lab23
configmap/depot-access-log created
```

Step 1.3 — Sanity-check the corpus you just loaded.

```
grep -c ERROR data/depot-portal-access.log
grep -c WARN data/depot-portal-access.log

6
5
```

Part 2 — Generate a real event stream

Step 2.1 — Apply the three workloads. Together they will produce Normal events (scheduling, pulling, starting), Warning events (FailedMount, BackOff) and a continuous log stream.

```
kubectl apply -f manifests/10-depot-portal-logger.yaml
kubectl apply -f manifests/20-flaky-scanner.yaml
kubectl apply -f manifests/30-missing-config.yaml

deployment.apps/depot-portal created
deployment.apps/gate-scanner created
pod/depot-gate-agent created
```

Step 2.2 — Wait about 60 seconds for the event stream to develop, then look at the Pods. Note the three different states: healthy, crash-looping, and stuck before it ever started.

```
kubectl get pods -n kcna-lab23
```

NAME	READY	STATUS	RESTARTS	AGE
depot-gate-agent	0/1	ContainerCreating	0	65s
depot-portal-58d6c47f9b-jm4vd	2/2	Running	0	65s
depot-portal-58d6c47f9b-rz8nq	2/2	Running	0	65s
gate-scanner-6c4f8b7d95-w2xkc	0/1	CrashLoopBackOff	2 (18s ago)	65s

depot-gate-agent sits in ContainerCreating **forever**, not Error. A volume that cannot be mounted blocks the container from ever being created. That distinction matters in Lab 25.

Part 3 — Events as first-class objects

Step 3.1 — List Events the naive way, and notice the problem.

```
kubectl get events -n kcna-lab23 | head -6
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
41s	Normal	Pulled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Container image "busybox:1.36" already present on machine
2m6s	Normal	Scheduled	Pod/depot-portal-58d6c47f9b-jm4vd	Successfully assigned kcna-lab23/depot-portal-58d6c47f9b-jm4vd to kind-control-plane
19s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found
2m6s	Normal	Created	Pod/depot-portal-58d6c47f9b-jm4vd	Created container: portal
...				

The trap. Events are returned in the API's own order, which is *not* chronological. During an incident this is actively harmful: you will misread cause and effect.

Step 3.2 — Sort them. This is the form you should build muscle memory for.

```
kubectl get events -n kcna-lab23 --sort-by=.lastTimestamp | tail -12
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m30s	Normal	Scheduled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Successfully assigned kcna-lab23/gate-scanner-6c4f8b7d95-w2xkc to kind-control-plane
2m28s	Normal	Pulled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Container image "busybox:1.36" already present on machine
2m28s	Normal	Created	Pod/gate-scanner-6c4f8b7d95-w2xkc	Created container: scanner
2m28s	Normal	Started	Pod/gate-scanner-6c4f8b7d95-w2xkc	Started container scanner
2m16s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
30s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.3 — The modern alternative. `kubectl events` (GA since v1.29) sorts by default and can scope to one object.

```
kubectl events -n kcna-lab23 --for pod/depot-gate-agent
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m1s	Normal	Scheduled	Pod/depot-gate-agent	Successfully assigned kcna-lab23/depot-gate-agent to kind-control-plane
2m59s (x2 over 3m1s)	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Read the (x2 over 3m1s) aggregation. Kubernetes does not store one Event per occurrence. Repeated identical events are collapsed into a single object with a `count` and a `firstTimestamp/lastTimestamp` pair. Your "one" event may represent hundreds of occurrences.

Step 3.4 — Field selectors. Filter to Warnings only — the single highest-value filter in day-to-day triage.

```
kubectl get events -n kcna-lab23 --field-selector type=Warning --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m40s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
54s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.5 — Compound field selector: Warnings about Pods only. Comma means AND. In a real cluster this strips out Node, Deployment and PVC noise.

```
kubectl get events -n kcna-lab23 \
  --field-selector involvedObject.kind=Pod,type=Warning \
  --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m55s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
69s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.6 — Select by reason, then by a specific object name.

```
kubectl get events -n kcna-lab23 --field-selector reason=FailedMount
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
84s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

```
kubectl get events -n kcna-lab23 \
  --field-selector involvedObject.name=depot-gate-agent,involvedObject.kind=Pod
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m30s	Normal	Scheduled	Pod/depot-gate-agent	Successfully assigned kcna-lab23/depot-gate-agent to kind-control-plane
89s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.7 — Discover the limits of field selectors. Not every field is indexed for selection — you cannot select on arbitrary paths, and you cannot use label selectors on Events at all (Events carry no meaningful labels).

```
kubectl get events -n kcna-lab23 --field-selector message=whatever

Error from server (BadRequest): Unable to find "/v1, Resource=events" that match label selector "", field selector "message=whatever": field label not supported: message
```

The rule. For core v1 Events the selectable fields are `involvedObject.kind`, `involvedObject.namespace`, `involvedObject.name`, `involvedObject.uid`, `involvedObject.apiVersion`, `involvedObject.resourceVersion`, `involvedObject.fieldPath`, `reason`, `reportingComponent`, `source`, `type` and `metadata.name/metadata.namespace`. Anything else — including `message` — must be filtered client-side with `grep` or `jq`.

Step 3.8 — Look at a raw Event object to see the fields you have been selecting on, plus the two that explain aggregation.

```
kubectl get events -n kcna-lab23 \
--field-selector reason=FailedMount -o yaml | \
grep -E '^ (count|firstTimestamp|lastTimestamp|reason|type)|^ (kind|name):'

  kind: Pod
  name: depot-gate-agent
count: 3
firstTimestamp: "2026-09-05T06:12:04Z"
lastTimestamp: "2026-09-05T06:16:21Z"
reason: FailedMount
type: Warning
```

Step 3.9 — The Event TTL surprise. Events are stored in etcd like any other object, but the kube-apiserver garbage-collects them on a timer controlled by its `--event-ttl` flag. **The default is one hour.** Confirm the setting your cluster is actually running — read-only inspection of the static Pod manifest on the kind node:

```
docker exec kind-control-plane \
grep -E 'event-ttl' /etc/kubernetes/manifests/kube-apiserver.yaml

(no output)
```

No output is the expected result and it is the lesson. kubeadm does not set `--event-ttl`, so the kube-apiserver default of **1 hour** applies. If your control-plane node has a different name, get it with `kubectl get nodes -o name`.

Confirm the effective default from the running process:

```
kubectl -n kube-system get pod -l component=kube-apiserver \
-o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep -c event-ttl

0
```

Consequences you must be able to state. Events are **not** an audit log. They expire, they are aggregated rather than enumerated, and they are namespaced. Anything you need after an hour

must be shipped somewhere else — which is exactly why the Kubernetes Events API is a *signal source* for an observability pipeline, not the pipeline itself. This is the argument for Lab 24.

Step 3.10 — Fill in rows 3.2 through 3.7 of `data/event-triage-worksheet.csv` with what each query returned.

Part 4 — Logs

Step 4.1 — The Pod has two containers, so `kubectl logs` refuses to guess.

```
POD=$(kubectl get pods -n kcna-lab23 -l app=depot-portal \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$POD"

error: a container name must be specified for pod depot-portal-58d6c47f9b-jm4vd, choose
one of: [portal audit-sidecar]
```

Step 4.2 — Select the container with `-c`, and bound the output with `--tail`.

```
kubectl logs -n kcna-lab23 "$POD" -c portal --tail=6

2026-09-05T06:00:41Z INFO depot-portal gate=sg-tuas lane=inbound req_id=e41533 method=GET
path=/api/v1/shipments status=200 duration_ms=40 driver=DRV-2210
2026-09-05T06:00:43Z INFO depot-portal gate=sg-tuas lane=inbound req_id=f52644 method=GET
path=/api/v1/manifest/SG44140 status=200 duration_ms=57 driver=DRV-2210
2026-09-05T06:00:45Z INFO depot-portal gate=sg-tuas lane=outbound req_id=063755
method=POST path=/api/v1/dispatch status=201 duration_ms=198 driver=DRV-9042
2026-09-05T06:00:47Z INFO depot-portal gate=sg-tuas lane=inbound req_id=174866 method=GET
path=/healthz status=200 duration_ms=1
2026-09-05T06:00:49Z WARN depot-portal gate=sg-tuas lane=inbound req_id=285977 method=GET
path=/api/v1/manifest/SG44151 status=404 duration_ms=11 reason=manifest_not_indexed
2026-09-05T06:00:51Z INFO depot-portal gate=sg-tuas lane=inbound req_id=396a88
method=POST path=/api/v1/checkin status=201 duration_ms=122 driver=DRV-1187
2026-09-05T06:00:53Z ERROR depot-portal gate=sg-tuas lane=outbound req_id=4a7b99
method=POST path=/api/v1/dispatch status=502 duration_ms=3006 upstream=manifest-indexer
error=upstream_timeout
```

The timestamps in the log body are from the corpus, not from now. That is deliberate: application timestamps and cluster wall-clock time are different things, and mixing them up is a real source of confusion in incident reviews. Ask the *cluster* for its view of time with `--timestamps`.

Step 4.3 — Time-bound the query with `--since`, and add cluster-side timestamps. This is the combination you want when you know roughly when something happened.

```
kubectl logs -n kcna-lab23 "$POD" -c portal --since=30s --timestamps | tail -3

2026-09-05T06:19:12.884301Z 2026-09-05T06:00:31Z ERROR depot-portal gate=sg-tuas
lane=outbound req_id=9fc0ee method=POST path=/api/v1/dispatch status=502 duration_ms=3009
upstream=manifest-indexer error=upstream_timeout
2026-09-05T06:19:14.886117Z 2026-09-05T06:00:33Z INFO depot-portal gate=sg-tuas
lane=inbound req_id=a0dlff method=POST path=/api/v1/checkin status=201 duration_ms=131
driver=DRV-3341
2026-09-05T06:19:16.888445Z 2026-09-05T06:00:35Z WARN depot-portal gate=sg-tuas
lane=outbound req_id=ble200 method=GET path=/api/v1/queue status=200 duration_ms=1204
note=queue_depth_high depth=58
```

Two timestamps per line: the first is when the container runtime received the line, the second is what the application wrote.

Step 4.4 — Aggregate across replicas with a label selector and `--prefix`. This is how you read a Deployment rather than a Pod.


```
kubectl logs -n kcna-lab23 -l app=depot-portal -c portal --tail=2 --prefix

[pod/depot-portal-58d6c47f9b-jm4vd/portal] 2026-09-05T06:00:53Z ERROR depot-portal
gate=sg-tuas lane=outbound req_id=4a7b99 method=POST path=/api/v1/dispatch status=502
duration_ms=3006 upstream=manifest-indexer error=upstream_timeout
[pod/depot-portal-58d6c47f9b-jm4vd/portal] 2026-09-05T06:00:55Z INFO depot-portal
gate=sg-tuas lane=inbound req_id=5b8caa method=GET path=/api/v1/shipments status=200
duration_ms=36 driver=DRV-3341
[pod/depot-portal-58d6c47f9b-rz8nq/portal] 2026-09-05T06:00:19Z INFO depot-portal
gate=sg-tuas lane=inbound req_id=3f6a88 method=GET path=/api/v1/shipments status=200
duration_ms=38 driver=DRV-9042
[pod/depot-portal-58d6c47f9b-rz8nq/portal] 2026-09-05T06:00:21Z INFO depot-portal
gate=sg-tuas lane=inbound req_id=4a7b99 method=POST path=/api/v1/checkin status=201
duration_ms=127 driver=DRV-1187
```

Step 4.5 — --previous, the single most important logs flag in triage. The `gate-scanner` container is crash-looping. Its *current* container may have only just started, so the reason it died is in the *terminated* container's log.

```
SCANNER=$(kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$SCANNER" --previous

06:18:02 INFO gate-scanner starting, serial=SCN-0447
06:18:02 INFO opening barcode reader on /dev/ttyUSB0
06:18:07 WARN reader handshake retry 1/1
06:18:12 ERROR reader handshake failed: device not present
06:18:12 FATAL gate-scanner exiting with status 1
```

Compare with the current container, which is mid-boot or backing off:

```
kubectl logs -n kcna-lab23 "$SCANNER"

06:19:44 INFO gate-scanner starting, serial=SCN-0447
06:19:44 INFO opening barcode reader on /dev/ttyUSB0
```

Only one generation back is kept. `--previous` gives you the immediately preceding container, not a history. If a Pod restarts twice while you are making coffee, the original failure is gone.

Step 4.6 — Confirm the exit code that matches that log, from the API rather than the log text.

```
kubectl get pod -n kcna-lab23 "$SCANNER" \
-o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}
{.status.containerStatuses[0].lastState.terminated.reason}{"\n"}'

1
Error
```

Step 4.7 — Why node-level log shipping exists. The `audit-sidecar` container writes audit records to a file, and separately tails that file to stdout. First, look at the file directly — this is what the application actually produced:

```
kubectl exec -n kcna-lab23 "$POD" -c audit-sidecar -- tail -3 /var/log/depot/audit.log

2026-09-05T06:19:30Z AUDIT seq=14 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:35Z AUDIT seq=15 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:40Z AUDIT seq=16 actor=gate-scanner action=scan result=accepted
```

Now look at what the cluster can see:

```
kubectl logs -n kcna-lab23 "$POD" -c audit-sidecar --tail=3

2026-09-05T06:19:30Z AUDIT seq=14 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:35Z AUDIT seq=15 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:40Z AUDIT seq=16 actor=gate-scanner action=scan result=accepted
```

They match — **but only because a `tail -f` process is deliberately re-emitting the file to stdout**. Delete that sidecar behaviour and `kubectl logs` shows nothing at all, while the file keeps filling up inside the container and disappears the moment the Pod is deleted.

Step 4.8 — Confirm where the container runtime actually stores what it captured, on the node.

```
docker exec kind-control-plane sh -c 'ls /var/log/pods/kcna-lab23_'"$POD"'_*/portal/'
0.log
docker exec kind-control-plane sh -c \
'cat /etc/kubernetes/kubelet.conf >/dev/null && echo readable'
readable
```

The three facts that justify a log agent.

1. Only **stdout/stderr** is captured. Files inside the container are not.
2. The kubelet **rotates** those files — by default at 10Mi per file with 5 files retained (`containerLogMaxSize` / `containerLogMaxFiles` in the kubelet configuration). Older lines are deleted, not archived.
3. Everything is **node-local and Pod-lifetime-bound**. Delete the Pod and `kubectl logs` returns `NotFound` immediately.

A node-level agent such as **Fluent Bit** or **Fluentd** (both CNCF projects; Fluentd is CNCF-graduated) runs as a DaemonSet, tails `/var/log/pods/...` and ships lines off-node **before** rotation destroys them. That is the entire reason the pattern exists.

Step 4.9 — Prove fact 3.

```
kubectl delete pod -n kcna-lab23 "$POD"
kubectl logs -n kcna-lab23 "$POD" -c portal --tail=1
pod "depot-portal-58d6c47f9b-jm4vd" deleted
Error from server (NotFound): pods "depot-portal-58d6c47f9b-jm4vd" not found
```

The Deployment immediately replaces it, but the deleted Pod's logs are unrecoverable.

Step 4.10 — Record rows 4.3 and 4.5 in `data/event-triage-worksheet.csv`.

Part 5 — The Metrics Server (CONDITIONAL)

Read this box before running anything in Part 5

`kubectl top` does not work on a stock kind cluster. Resource metrics are served by the `metrics.k8s.io` API, which is provided by the **metrics-server add-on**. `kind` does not install it. This is not a broken cluster.

You have two valid ways to complete this part:

- **Path A — install metrics-server** using the manifests in this lab. This creates **four cluster-scoped objects** and **one additive RoleBinding in `kube-system`**, all listed and justified in the header comments of `manifests/90-metrics-server.yaml` and `manifests/91-metrics-server-kube-system-rolebinding.yaml`.
- **Path B — skip the install.** Run Steps 5.1 and 5.2 to see and understand the failure, then read Steps 5.4-5.7 without executing them. Parts 1-4 are the assessed content and are unaffected.

No `kubectl top` numbers are printed anywhere in this lab as if they were observed on your machine. Where output is shown for Path A, the numeric columns are marked as varying.

Step 5.1 — Establish the starting state. Ask for metrics before installing anything.

```
kubectl top pods -n kcna-lab23
error: Metrics API not available
```

Step 5.2 — Confirm *why*, from the API discovery layer. This is the diagnostic that distinguishes "add-on missing" from "add-on broken".

```
kubectl get apiservice v1beta1.metrics.k8s.io
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io"
not found

kubectl api-resources --api-group=metrics.k8s.io
NAME          SHORTNAMES  APIVERSION  NAMESPACE  KIND
```

An empty table. The API group does not exist.

Stop here if you are taking Path B. Everything below installs software.

Step 5.3 (Path A) — Read the two manifest headers *first*. They are the teaching content of this part, not boilerplate.

```
sed -n '1,60p' manifests/90-metrics-server.yaml
sed -n '1,35p' manifests/91-metrics-server-kube-system-rolebinding.yaml
```

Then apply, main bundle first:

```
kubectl apply -f manifests/90-metrics-server.yaml
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/kcna-lab23:metrics-server created
clusterrolebinding.rbac.authorization.k8s.io/kcna-lab23:metrics-server created
clusterrolebinding.rbac.authorization.k8s.io/kcna-lab23:metrics-server:auth-delegator
created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created

kubectl apply -f manifests/91-metrics-server-kube-system-rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/kcna-lab23-metrics-server-auth-reader created
```

Step 5.4 (Path A) — Wait for it to become Ready. Allow up to 90 seconds: the readiness probe has a 20 second initial delay and the first scrape cycle must complete.

```
kubectl rollout status deployment/metrics-server -n kcna-lab23 --timeout=120s
deployment "metrics-server" successfully rolled out

kubectl get apiservice v1beta1.metrics.k8s.io
NAME          SERVICE          AVAILABLE  AGE
v1beta1.metrics.k8s.io  kcna-lab23/metrics-server  True       75s
```

AVAILABLE True is the gate. If it reads False (FailedDiscoveryCheck), see the Troubleshooting table.

Step 5.5 (Path A) — Now `kubectl top` works.

```
kubectl top nodes
```

NAME	CPU (cores)	CPU (%)	MEMORY (bytes)	MEMORY (%)
kind-control-plane	<varies>	<varies>	<varies>	<varies>

The numbers are deliberately not written out. They depend entirely on your host machine, what else is running on it, and how long metrics-server has been collecting. Anyone who prints specific figures here is inventing them. What you must verify is that **five columns appear and the node is named** — that proves the aggregation layer is working.

Step 5.6 (Path A) — Give it something predictable to measure. Apply the CPU consumer, which is capped by a 100m CPU limit.

```
kubectl apply -f manifests/40-load-generator.yaml
kubectl rollout status deployment/manifest-indexer -n kcna-lab23 --timeout=60s

deployment.apps/manifest-indexer created
deployment "manifest-indexer" successfully rolled out
```

Wait ~60 seconds for two scrape cycles, then:

```
kubectl top pods -n kcna-lab23 --containers
```

POD	READY	NAME	CPU (cores)	MEMORY (bytes)
manifest-indexer-...	1/1	indexer	<near 100m>	<small>
depot-portal-...	2/2	portal	<near 0m>	<small>
depot-portal-...	2/2	audit-sidecar	<near 0m>	<small>
metrics-server-...	1/1	metrics-server	<varies>	<varies>

What you can legitimately predict, and why. The `indexer` container runs an unbounded arithmetic loop, so it will consume every cycle it is permitted. Its `limits.cpu` is 100m, which the kernel enforces as a CFS quota. It should therefore report **at or just below 100m** — and crucially, *never above it*. That is a claim about the enforcement mechanism, which is deterministic. The exact millicore reading is not.

Confirm the ceiling yourself:

```
kubectl get pod -n kcna-lab23 -l app=manifest-indexer \
-o jsonpath='{.items[0].spec.containers[0].resources.limits}{"\n"}'
{"cpu":"100m","memory":"64Mi"}
```

Step 5.7 (Path A) — See the raw API underneath `kubectl top`. `top` is only a formatter.

```
kubectl get --raw '/apis/metrics.k8s.io/v1beta1/namespaces/kcna-lab23/pods' \
| head -c 400; echo

{"kind":"PodMetricsList","apiVersion":"metrics.k8s.io/v1beta1","metadata":{},"items":
[{"metadata":{"name":"manifest-indexer-...","namespace":"kcna-
lab23","creationTimestamp":"..."},"timestamp":"...","window":"15.0s","containers":
[{"name":"indexer","usage":{"cpu":"...","memory":"..."}}]}...
```

Three properties to note, and remember for Lab 24. The response carries a `window` (a short sampling interval), it has **no history at all**, and it stores nothing. `metrics-server` keeps a small in-memory ring buffer to serve the current value for the `HorizontalPodAutoscaler` and for `kubectl top`. It is **not** a monitoring system, it cannot answer "what was CPU at 03:00 last night", and it must never be used as one. A time-series database — `Prometheus` — is what fills that gap, and that is Lab 24.

Step 5.8 — Record row 5.4 in `data/event-triage-worksheet.csv`, noting which path (A or B) you took.

Lab 23 · Verification

```
bash verification/checks.sh
```

The script auto-detects whether you took Path A or Path B and adjusts. See [verification/expected-output.md](#).

Lab 23 · Failure injection (mandatory)

You will now break the metrics pipeline in the way it most commonly breaks in the field: the aggregation layer points at a Service that has no healthy backend. If you took **Path B**, do the alternative injection in Step 6.6 instead.

Step 6.1 (Path A) — Scale metrics-server to zero. The APIService still exists, so the API group is still *advertised* — it just cannot be served.

```
kubectl scale deployment metrics-server -n kcna-lab23 --replicas=0
deployment.apps/metrics-server scaled
```

Step 6.2 — Wait ~30 seconds, then observe the failure mode. This is a *different* error string from Step 5.1, and the difference is the whole point.

```
kubectl top pods -n kcna-lab23
Error from server (ServiceUnavailable): the server is currently unable to handle the
request (get pods.metrics.k8s.io)
```

Step 6.3 — Diagnose from the aggregation layer.

```
kubectl get apiservice v1beta1.metrics.k8s.io
NAME                                SERVICE                                AVAILABLE      AGE
v1beta1.metrics.k8s.io             kcna-lab23/metrics-server             False (MissingEndpoints)  6m
kubectl get apiservice v1beta1.metrics.k8s.io \
  -o jsonpath='{.status.conditions[0].message}{"\n"}'
endpoints for service/metrics-server in "kcna-lab23" have no addresses with port name
"https"
```

Learn both signatures. error: Metrics API not available → the APIService does not exist; the add-on was never installed. Error from server (ServiceUnavailable) + AVAILABLE: False → the APIService exists but its backend is down. Completely different fixes.

Step 6.4 — Repair.

```
kubectl scale deployment metrics-server -n kcna-lab23 --replicas=1
kubectl rollout status deployment/metrics-server -n kcna-lab23 --timeout=120s
deployment.apps/metrics-server scaled
deployment "metrics-server" successfully rolled out
```

Step 6.5 — Confirm recovery.

```
kubectl get apiservice v1beta1.metrics.k8s.io
NAME                                SERVICE                                AVAILABLE      AGE
v1beta1.metrics.k8s.io             kcna-lab23/metrics-server             True           8m
```

Step 6.6 (Path B alternative — no install required) — Break the log evidence trail instead, which is the failure that actually cost Meridian its INC-4417 post-mortem. Force a second restart of the crash-looping scanner and watch the original evidence become unrecoverable.

```
kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].status.containerStatuses[0].restartCount}{"\n"}'
kubectl delete pod -n kcna-lab23 -l app=gate-scanner

4
pod "gate-scanner-6c4f8b7d95-w2xkc" deleted

NEW=$(kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$NEW" --previous

Error from server (BadRequest): previous terminated container "scanner" in pod "gate-scanner-..." not found
```

That is the INC-4417 failure, reproduced. A brand-new Pod has no previous container, so there is no evidence at all. Combined with the one-hour Event TTL from Step 3.9, an incident that is not investigated promptly becomes uninvestigable. Ship your logs and events off-cluster.

Lab 23 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
error: Metrics API not available	metrics-server is not installed; stock kind has no metrics.k8s.io	kubectl get apiservice v1beta1.metrics.k8s.io returns NotFound	Install it (Part 5 Path A), or accept it and take Path B
AVAILABLE: False (FailedDiscoveryCheck) on the APIService, metrics-server Pod is 0/1 and its log shows x509: cannot validate certificate ... doesn't contain any IP SANs	--kubelet-insecure-tls is missing; kind kubelets serve self-signed certificates	kubectl logs -n kcna-lab23 deploy/metrics-server tail -20	Add --kubelet-insecure-tls (already present in the supplied manifest — check you applied the supplied file)
metrics-server Pod runs but readiness never passes, log shows unable to load configmap based request-header-client-ca-file or configmaps "extension-apiserver-authentication" is forbidden	The kube-system RoleBinding was not applied	kubectl get rolebinding kcna-lab23-metrics-server-auth-reader -n kube-system	Apply manifests/91-metrics-server-kube-system-rolebinding.yaml, or take Path B
Error from server (ServiceUnavailable) from kubectl top	APIService exists but has no healthy backend	kubectl get apiservice v1beta1.metrics.k8s.io -o jsonpath='{.status.conditions[0].message}'	Scale/repair the metrics-server Deployment; check its Service targetPort name is https
field label not supported: message	You tried to field-select on a non-indexed Event field	—	Field-select on type/reason/involvedObject.* only; filter message text client-side with grep

Symptom	Likely cause	Diagnostic command	Fix
Events you expected are simply absent	kube-apiserver --event-ttl (default 1 hour) garbage-collected them	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp head -1</code> shows nothing older than ~1h	Nothing to fix in-cluster. Ship Events to durable storage if you need them
<code>kubectl logs</code> says a container name must be specified	Multi-container Pod	<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[*].name}'</code>	Add <code>-c <container></code> , or <code>--all-containers</code>
previous terminated container ... not found	The Pod was recreated, or has never restarted	<code>kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].restartCount}'</code>	Only one generation is retained; capture logs promptly or ship them off-node
Application writes to a file and <code>kubectl logs</code> shows nothing	Only stdout/stderr is captured by the container runtime	<code>kubectl exec <pod> -c <c> -- tail /path/to/file</code>	Log to stdout, or add a sidecar that tails the file (as <code>audit-sidecar</code> does here)

Lab 23 · Cleanup

Step 8.1 — Delete the lab namespace. This removes every namespaced object including metrics-server itself.

```
kubectl delete namespace kcna-lab23
namespace "kcna-lab23" deleted
```

Step 8.2 — Path A only. The cluster-scoped objects and the kube-system RoleBinding are *not* in the namespace and survive Step 8.1. Remove them by **exact name** — never with a wildcard or a bare `kubectl delete`:

```
kubectl delete apiservice v1beta1.metrics.k8s.io
kubectl delete clusterrolebinding kcna-lab23:metrics-server
kubectl delete clusterrolebinding kcna-lab23:metrics-server:auth-delegator
kubectl delete clusterrole kcna-lab23:metrics-server
kubectl delete rolebinding kcna-lab23-metrics-server-auth-reader -n kube-system

apiservice.apiregistration.k8s.io "v1beta1.metrics.k8s.io" deleted
clusterrolebinding.rbac.authorization.k8s.io "kcna-lab23:metrics-server" deleted
clusterrolebinding.rbac.authorization.k8s.io "kcna-lab23:metrics-server:auth-delegator" deleted
clusterrole.rbac.authorization.k8s.io "kcna-lab23:metrics-server" deleted
rolebinding.rbac.authorization.k8s.io "kcna-lab23-metrics-server-auth-reader" deleted
```

If Lab 24 or a later exercise needs `kubectl top`, leave metrics-server installed and run Step 8.2 at the end of the day instead. Note that leaving a dangling `APIService` whose `Service` no longer exists will make `kubectl get apiservice` report `False` and can slow down `kubectl api-resources` for everyone on the cluster — so if you delete the namespace, delete the `APIService` too.

Step 8.3 — Verify.

```
kubectl get namespace kcna-lab23
kubectl get apiservice v1beta1.metrics.k8s.io
```



```
Error from server (NotFound): namespaces "kcna-lab23" not found
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io"
not found
```

Lab 23 · What you learned

- **Events are API objects with all the properties of API objects** — and two surprising ones. They are **aggregated** (`count + firstTimestamp`), so one row can mean hundreds of occurrences; and they are **garbage-collected after `--event-ttl`, one hour by default**. Events are a live triage signal, never an audit trail.
- **`--sort-by=.lastTimestamp` is not optional**. The default listing order is not chronological and will make you misread causality during an incident.
- **Field selectors are server-side and cheap, but restricted**. `type`, `reason` and `involvedObject.*` are indexed; `message` is not. Knowing the boundary saves you from piping the whole namespace through `grep`.
- **`kubectl logs` sees `stdout/stderr` and nothing else**, keeps exactly one previous container generation, and is bounded by kubelet log rotation (10Mi × 5 files by default). Those three limits, together, are the complete argument for a node-level shipper such as Fluent Bit or Fluentd.
- **`kubectl top` is an add-on, not a feature**. It requires metrics-server serving `metrics.k8s.io` through the aggregation layer. On kind it also requires `--kubelet-insecure-tls`, because kind kubelets serve self-signed certificates — a flag that disables TLS verification and would be a genuine security defect in production, where the fix is signed kubelet serving certificates instead.
- **metrics-server has no history**. It answers "now", in a ~15 s window, for the HPA and for `kubectl top`. Any question containing the word "yesterday" needs a time-series database. That is Lab 24.
- **Two failure signatures, two different fixes**: `Metrics API not available` (never installed) versus `ServiceUnavailable + AVAILABLE: False` (installed, backend down).

Further reading

- Kubernetes — *Application Introspection and Debugging*:
<https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>
- Kubernetes — *Logging Architecture* (node-level agents, rotation, sidecars):
<https://kubernetes.io/docs/concepts/cluster-administration/logging/>
- Kubernetes — *Resource Metrics Pipeline*:
<https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>
- Kubernetes — *Field Selectors*: <https://kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/>
- metrics-server — README, including the `--kubelet-insecure-tls` caveat:
<https://github.com/kubernetes-sigs/metrics-server>
- Fluentd (CNCF **graduated**) and Fluent Bit: <https://www.cncf.io/projects/fluentd/>
- CNCF — KCNA curriculum, *Cloud Native Architecture* → *Observability*:
<https://github.com/cncf/curriculum>

Lab 24 — Metrics, Prometheus Exposition and the Observability Pipeline

Field	Value
Lab ID	Lab 24
Title	Metrics, Prometheus Exposition and the Observability Pipeline
Day / Topic	Day 5 · Logging and Monitoring
Duration	45 minutes
Namespace	kcna-lab24
Learning outcome	LO6 — Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.
Ability	A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework
Knowledge	K7 Interactions among various IT components
Deck slide	Slide 528
Repository path	courseware/labs/lab-24-prometheus-observability/

Goal. Run a real scrape target and a single self-contained Prometheus, read the text exposition format line by line, and query PromQL against metric names you produced yourself — including the error-ratio query that would have detected INC-4417.

What you will produce:

- A working exporter serving `depot_portal_*` metrics in Prometheus text format at `/metrics`, scraped by hand with `wget` before Prometheus ever touches it
- A Prometheus Pod scraping two named static targets from a scrape config you wrote, with both targets reporting `up == 1`
- A completed exposition worksheet recording counter-versus-gauge behaviour, the measured rate against its predicted band, and the `up == 0` signal during failure injection

How to read the expected-output blocks. Pod name suffixes, IP addresses, `AGE` columns and timestamps vary at runtime. **Metric values vary too** — counters depend on how long the generator has been running when you scrape. Where a number is unpredictable it is shown as `<counter>`, `<gauge>` or similar. What you match is the *structure*: metric names, `# HELP/# TYPE` lines, label sets, and the relationships between series.

Lab 24 · Scope and honesty statement — read first

This lab does **not** install `kube-prometheus-stack`. There is no Operator, no `ServiceMonitor` CRD, no Alertmanager, no Grafana and no node-exporter.

That is a deliberate teaching decision. Those components are correct for a production cluster and wrong for a 45-minute lab, because they hide the mechanism behind custom resources. You would learn to fill in a CRD without ever seeing a scrape.

Instead you get the smallest thing that is still genuinely Prometheus: **one Pod, one scrape config you wrote yourself, two targets you can name**, and a **local TSDB you query with real PromQL** over metric names you produced. Everything you observe here is real output from a real Prometheus server.

CNCF status, stated precisely — you will be asked this in the assessment:

Project	CNCF status	Role
Kubernetes	Graduated	Orchestrator
Prometheus	Graduated (the second project ever to graduate, 2018)	Metrics collection, storage, alerting
OpenTelemetry	Incubating — and the second-highest-velocity CNCF project after Kubernetes	Vendor-neutral generation and collection standard for traces, metrics and logs
Fluentd	Graduated	Log collection and forwarding
Jaeger	Graduated	Distributed tracing backend
Thanos, Cortex	Incubating	Long-term, horizontally scalable storage for Prometheus
Grafana	Not a CNCF project — open-source, Grafana Labs (AGPLv3)	Visualisation. Commonly taught alongside Prometheus; frequently and wrongly assumed to be CNCF
Loki	Not a CNCF project — Grafana Labs	Log aggregation

Sources: <https://www.cncf.io/projects/> and <https://prometheus.io/>.

Lab 24 · Objective

By the end of this lab you will be able to:

1. **Name the four observability signals** — metrics, logs, traces, events (with profiles as an emerging fifth) — and say which one answers which question.
2. **Explain pull vs push** and give the concrete operational consequence of each in a Kubernetes cluster.
3. **Read the Prometheus text exposition format line by line:** # HELP, # TYPE, metric name, label set, sample value, and the `_bucket/_sum/_count` triple that makes up a histogram.
4. **Distinguish counter, gauge and histogram** and explain from first principles why `rate()` exists and why a raw counter is almost never what you want to graph.
5. **Read a scrape config:** job, targets, interval, timeout, and where `relabel_configs` sits relative to `metric_relabel_configs`.
6. **Write PromQL against metric names you created yourself** and interpret the results.
7. **Diagnose a down scrape target** using `up` and the `/targets` endpoint.
8. **Place OpenTelemetry correctly** in the pipeline relative to Prometheus.

Lab 24 · Prerequisites

- A running single-node **kind** cluster.

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

- You are in the lab directory:

```
cd courseware/labs/lab-24-prometheus-observability
```

- Images used, all **pinned**: `busybox:1.36`, `nginx:1.27-alpine`, `alpine:3.20`, `prom/prometheus:v2.54.1`.
- **Lab 23 is a soft prerequisite.** You should already be able to say why metrics-server is not a monitoring system.

Lab 24 · Scenario

Meridian Freight's post-incident review for **INC-4417** produced one finding that nobody could argue with: *nobody could say when the error rate started rising*. The Depot Platform Squad had `kubectl top` — which, as you established in Lab 23, holds no history whatsoever — and a Slack thread.

Ticket **DEP-1024**:

Instrument the Depot Portal so that "when did this start" is answerable. Stand up a metrics pipeline in the lab cluster, expose gate throughput, queue depth and scan latency, and demonstrate a query that would have detected INC-4417 within one scrape interval.

You are the platform engineer on DEP-1024.

Lab 24 · Step-by-step procedure

Part 1 — The four signals, before you type anything

Signal	Answers	Cardinality	Retention	This course
Metrics	<i>How much? How often? How fast?</i> Aggregated numbers over time	Low — bounded by label combinations	Long (months) and cheap	This lab
Logs	<i>What exactly happened in this one request?</i> Discrete timestamped records	High — one record per event	Short and expensive	Lab 23
Traces	<i>Where did the time go across services?</i> Causally-linked spans	Very high; usually sampled	Short	Concept only
Events	<i>What did the control plane decide?</i> Kubernetes-specific state changes	Low	1 hour by default	Lab 23

The rule to remember. Metrics tell you *that* something is wrong and *when* it started. Logs and traces tell you *why*. Alert on metrics; investigate with logs and traces. Alerting on log volume is a common and expensive mistake.

Part 2 — Deploy the scrape target

Step 2.1 — Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab24 created
```

Step 2.2 — Load the generator script from `data/` into a ConfigMap. Read the script first — it is the exposition format, written out by hand, with comments.

```
sed -n '25,60p' data/generate-metrics.sh
kubectl create configmap depot-metrics-generator \
  --from-file=generate-metrics.sh=data/generate-metrics.sh \
  -n kcna-lab24
configmap/depot-metrics-generator created
```

Step 2.3 — Apply the exporter's nginx config and the workload.

```
kubectl apply -f manifests/05-exporter-nginx-config.yaml
kubectl apply -f manifests/10-depot-metrics-sim.yaml

configmap/exporter-nginx-config created
deployment.apps/depot-metrics-sim created
service/depot-metrics-sim created
```

Step 2.4 — Wait for it.

```
kubectl rollout status deployment/depot-metrics-sim -n kcna-lab24 --timeout=120s
deployment "depot-metrics-sim" successfully rolled out
kubectl get pods -n kcna-lab24
```

NAME	READY	STATUS	RESTARTS	AGE
depot-metrics-sim-6b7d9f4c58-k2mvx	2/2	Running	0	35s

2/2 — the generator writing the file, and the exporter serving it.

Part 3 — Read the exposition format, line by line

Step 3.1 — Start the in-cluster client.

```
kubectl apply -f manifests/30-metrics-client.yaml
kubectl wait --for=condition=Ready pod/metrics-client -n kcna-lab24 --timeout=60s

pod/metrics-client created
pod/metrics-client condition met
```

Step 3.2 — Scrape the target by hand. This is *exactly* what Prometheus does — an HTTP GET, nothing more.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics

# HELP depot_portal_shipments_scanned_total Shipment barcodes scanned at the depot gate
since process start.
# TYPE depot_portal_shipments_scanned_total counter
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} <counter>
# HELP depot_portal_shipments_rejected_total Shipments rejected at the gate since process
start.
# TYPE depot_portal_shipments_rejected_total counter
depot_portal_shipments_rejected_total{depot="sg-
tuas",lane="inbound",reason="manifest_not_indexed"} <counter>
# HELP depot_portal_http_requests_total HTTP requests handled by the depot portal.
# TYPE depot_portal_http_requests_total counter
depot_portal_http_requests_total{depot="sg-tuas",method="GET",status="200"} <counter>
```

```

depot_portal_http_requests_total{depot="sg-tuas",method="POST",status="502"} <counter>
# HELP depot_portal_queue_depth Shipments currently waiting in the gate queue.
# TYPE depot_portal_queue_depth gauge
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} <gauge, 12-51>
# HELP depot_portal_active_drivers Drivers currently checked in at the depot.
# TYPE depot_portal_active_drivers gauge
depot_portal_active_drivers{depot="sg-tuas"} <gauge, 4-12>
# HELP depot_portal_scan_latency_seconds Time taken to scan and validate one shipment.
# TYPE depot_portal_scan_latency_seconds histogram
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="0.5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="1"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="2.5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="+Inf"} <cumulative>
depot_portal_scan_latency_seconds_sum{depot="sg-tuas"} <sum>
depot_portal_scan_latency_seconds_count{depot="sg-tuas"} <count>
# HELP depot_portal_build_info Build metadata. Always 1; the LABELS carry the information.
# TYPE depot_portal_build_info gauge
depot_portal_build_info{version="6.0.0",revision="a41f9c2",depot="sg-tuas"} 1

```

Step 3.3 — Anatomy of one line. Take this line apart:

```

depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} 147
└────────────────────────────────┬────────────────────────────────┬────────┘
metric name                      label set                      value

```

- **Metric name** — snake_case. The `_total` suffix is the strong convention for counters. Units belong in the name (`_seconds`, `_bytes`), never in a label.
- **Label set** — key/value pairs in braces. Each distinct combination is a **separate time series**. This is where cardinality explosions come from: put a user ID or a request ID in a label and you create one series per user.
- **Value** — a 64-bit float. There is no integer type.
- Timestamps are **optional** in the exposition and almost always omitted; Prometheus stamps the sample with its own scrape time.

Step 3.4 — # HELP and # TYPE. These are metadata comment lines, one pair per metric family, and they are not decoration:

```

kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics \
  | grep '^#'

# HELP depot_portal_shipments_scanned_total Shipment barcodes scanned at the depot gate
since process start.
# TYPE depot_portal_shipments_scanned_total counter
# HELP depot_portal_shipments_rejected_total Shipments rejected at the gate since process
start.
# TYPE depot_portal_shipments_rejected_total counter
# HELP depot_portal_http_requests_total HTTP requests handled by the depot portal.
# TYPE depot_portal_http_requests_total counter
# HELP depot_portal_queue_depth Shipments currently waiting in the gate queue.
# TYPE depot_portal_queue_depth gauge
# HELP depot_portal_active_drivers Drivers currently checked in at the depot.
# TYPE depot_portal_active_drivers gauge
# HELP depot_portal_scan_latency_seconds Time taken to scan and validate one shipment.
# TYPE depot_portal_scan_latency_seconds histogram
# HELP depot_portal_build_info Build metadata. Always 1; the LABELS carry the information.
# TYPE depot_portal_build_info gauge

```

TYPE is what tells Prometheus (and you) whether `rate()` is legal on this series. # HELP is what the UI shows an engineer at 3 a.m.

Step 3.5 — Counter vs gauge, demonstrated. Scrape twice, 12 seconds apart, and compare.

```
kubectl exec -n kcna-lab24 metrics-client -- sh -c '
URL=http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics
echo "--- scrape 1"
wget -qO- $URL | grep -E "^depot_portal_(shipments_scanned_total|queue_depth)"
sleep 12
echo "--- scrape 2"
wget -qO- $URL | grep -E "^depot_portal_(shipments_scanned_total|queue_depth)"
,

--- scrape 1
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} 63
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} 47
--- scrape 2
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} 72
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} 26
```

The two exact numbers above will differ on your run — that is the point. What must hold, every time:

- the **counter went UP** (63 → 72). A counter never decreases while the process lives. Its absolute value is meaningless: it depends only on how long the process has been up.
- the **gauge moved in an arbitrary direction** (47 → 26). A gauge is a current reading. Its absolute value *is* meaningful.

Record both pairs in `data/exposition-worksheet.csv`.

Step 3.6 — Why `rate()` must exist. "147 shipments scanned" is not an answer to any operational question, because it is a lifetime total. What you want is *shipments per second, now*. That derivative — computed over a time window, and **automatically corrected for counter resets when a process restarts** — is `rate()`. There is no way to get it from a single scrape; it is inherently a query over stored history. This is precisely the capability metrics-server does not have.

Step 3.7 — Read the histogram. A histogram is not one series, it is a family:

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics \
  | grep scan_latency

depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="0.5"} 60
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="1"} 81
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="2.5"} 93
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="5"} 96
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="+Inf"} 97
depot_portal_scan_latency_seconds_sum{depot="sg-tuas"} 39
depot_portal_scan_latency_seconds_count{depot="sg-tuas"} 97
```

- `le` means **less than or equal to**. Buckets are **cumulative**: the `le="1"` bucket contains everything in `le="0.5"` too. Values must therefore be non-decreasing as `le` rises — check that in your own output.
- `le="+Inf"` always equals `_count`. If it does not, the exporter is broken.
- `_sum / _count` gives the **mean**. Means hide outliers, which is exactly why buckets exist: `histogram_quantile()` estimates p95/p99 from them.
- Quantile accuracy is bounded by your bucket boundaries. You cannot recover precision you did not define up front — the single most important practical consequence of choosing buckets.

Step 3.8 — The `_info` pattern. `depot_portal_build_info` is always 1. The *value* carries nothing; the **labels** carry version and revision. This lets you join build metadata onto other series in PromQL rather than duplicating a `version` label onto every metric (which would multiply your cardinality).

Step 3.9 — Confirm the Content-Type.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -S -qO /dev/null \
  http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics 2>&1 \
  | grep -i 'content-type'

Content-Type: text/plain; version=0.0.4; charset=utf-8
```

version=0.0.4 identifies the text exposition format version. Prometheus uses this header to select a parser.

Part 4 — Pull vs push, and the scrape config

Step 4.1 — The model. Prometheus **pulls**. It connects to each target on a schedule and does an HTTP GET. Targets do not send anything.

	Pull (Prometheus)	Push (StatsD, OTLP push, Graphite)
Who initiates	The monitoring system	The application
Target discovery	Required — service discovery or static config	Not required
Is the target up?	Free — a failed scrape <i>is</i> the signal (<code>up == 0</code>)	Needs a separate heartbeat; silence is ambiguous
Short-lived jobs	Awkward — needs a Pushgateway	Natural
Firewall direction	Monitoring must reach targets	Targets must reach monitoring
Overload behaviour	Monitoring controls the rate	Targets can flood the collector

The decisive advantage of pull, in one line: you get liveness detection for free. `up == 0` tells you a target is unreachable without the target having to do anything. You will prove this in Part 6.

Step 4.2 — Read the scrape config before you apply it.

```
cat data/prometheus-scrape.yml
```

Work through its anatomy:

Key	Meaning	Value here
<code>global.scrape_interval</code>	Default pull frequency; the resolution of every stored series	10s
<code>global.evaluation_interval</code>	How often rules are evaluated	30s
<code>global.external_labels</code>	Stamped on series leaving this server	<code>cluster, env</code>
<code>scrape_configs[].job_name</code>	Becomes the <code>job</code> label on every series from this job	<code>prometheus, depot-portal</code>
<code>static_configs[].targets</code>	<code>host:port</code> list. Production uses <code>kubernetes_sd_configs</code> instead	Service DNS name
<code>static_configs[].labels</code>	Extra labels stamped on every series from these targets	<code>depot, service</code>
<code>metrics_path</code>	Path to GET	<code>/metrics</code> (the default)
<code>scrape_timeout</code>	Must be <code><= scrape_interval</code>	5s
<code>relabel_configs</code>	Runs before the scrape; rewrites the <i>target's</i> labels	Strips <code>:9100</code> into <code>instance</code>
<code>metric_relabel_configs</code>	Runs after the scrape; can drop individual <i>series</i>	Not used here

Relabelling in outline. Both stages operate on label sets with `source_labels → regex → target_label → replacement → action`. `relabel_configs` decides *what to scrape and how to label it*; `metric_relabel_configs` decides *what to keep*. The usual production use of the second is `action: drop` on high-cardinality series that would otherwise bloat the TSDB.

Step 4.3 — Create the config ConfigMap. The key **must** be `prometheus.yml` because that is the filename `--config.file` points at.

```
kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape.yml \
  -n kcna-lab24
configmap/prometheus-config created
```

Step 4.4 — Deploy Prometheus.

```
kubectl apply -f manifests/20-prometheus.yaml
kubectl rollout status deployment/prometheus -n kcna-lab24 --timeout=180s
deployment.apps/prometheus created
service/prometheus created
deployment "prometheus" successfully rolled out
```

Step 4.5 — Confirm it loaded your config and both targets are UP. Allow ~20 seconds for the first scrape cycle.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- 'http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/targets?
state=active' \
  | tr ',' '\n' | grep -E '"job"|"health"|"scrapeUrl"'
"scrapeUrl":"http://localhost:9090/metrics"
"health":"up"
"job":"prometheus"
"scrapeUrl":"http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics"
"health":"up"
"job":"depot-portal"
```

Two targets, both `"health":"up"`.

Step 4.6 — Optionally open the web UI. Leave this running in a second terminal.

```
kubectl port-forward -n kcna-lab24 svc/prometheus 9090:9090
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
```

Browse <http://localhost:9090/targets> and <http://localhost:9090/graph>.

Part 5 — PromQL against metrics you produced

Every query below runs against the metric names **you** defined in `data/generate-metrics.sh`. Use the web UI, or the API from the client Pod. A helper to keep the commands short:

```
promql() {
  kubectl exec -n kcna-lab24 metrics-client -- \
    wget -qO- "http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/query?query=$1"
}
```


URL-encode special characters: { is %7B, } is %7D, " is %22, [is %5B,] is %5D, a space is %20. The web UI needs none of this, which is why it is easier for exploration.

Step 5.1 — Instant vector. Ask for the raw counter.

```
promql 'depot_portal_shipments_scanned_total'

{"status":"success","data":{"resultType":"vector","result":[{"metric":
{"__name__":"depot_portal_shipments_scanned_total","cluster":"kcna-kind","depot":"sg-
tuas","env":"lab","instance":"depot-metrics-sim.kcna-
lab24.svc.cluster.local","job":"depot-portal","lane":"inbound","service":"depot-
portal"},"value":[<timestamp>,<counter>"]}]}]}
```

Look at the labels you did not write. `job` came from `job_name`. `instance` was produced by your `relabel_configs` — note it has **no** `:9100`, which proves the relabel rule fired. `cluster` and `env` came from `external_labels`. `depot`, `lane` and `service` came from the exposition and the static config. Prometheus composed all of it.

Step 5.2 — `rate()`. The query that actually answers "how busy is the gate?"

```
promql 'rate(depota_portal_shipments_scanned_total%5B1m%5D)'

{"status":"success","data":{"resultType":"vector","result":[{"metric":{"depot":"sg-
tuas","env":"lab","cluster":"kcna-kind","instance":"depot-metrics-sim.kcna-
lab24.svc.cluster.local","job":"depot-portal","lane":"inbound","service":"depot-
portal"},"value":[<timestamp>,<per-second rate>"]}]}]}
```

Note the missing `__name__`. `rate()` returns a *new* series that is no longer the original metric, so the name is dropped.

Sanity-check the magnitude yourself. The generator adds between 7 and 11 to the counter every 10 seconds, so the true rate is between **0.7 and 1.1 per second**. If your result is in that band, your whole pipeline — exposition, scrape, storage, query — is provably correct end to end. This is a prediction you can verify, not a number to copy.

Step 5.3 — The `[1m]` window rule. `rate()` needs at least two samples in the window. Your `scrape_interval` is 10s, so `[1m]` gives about six — ample. Try a window narrower than the interval and watch it fail:

```
promql 'rate(depota_portal_shipments_scanned_total%5B5s%5D)'

{"status":"success","data":{"resultType":"vector","result":[]}}}
```

An empty result, not an error. **Rule of thumb: make the range at least 4× your scrape interval.**

Step 5.4 — Aggregate away a label. Sum across HTTP status codes:

```
promql 'sum%20by%20(status)%20(rate(depota_portal_http_requests_total%5B2m%5D))'

{"status":"success","data":{"resultType":"vector","result":[
{"metric":{"status":"200"},"value":[<ts>,<rate>]},
{"metric":{"status":"502"},"value":[<ts>,<rate>"]}]}]}
```

Step 5.5 — The query that would have caught INC-4417. Error ratio:

```
promql 'sum(rate(depota_portal_http_requests_total%7Bstatus%3D%22502%22%7D%5B2m%5D))
%20/%20sum(rate(depota_portal_http_requests_total%5B2m%5D))'

{"status":"success","data":{"resultType":"vector","result":[{"metric":{}, "value":
[<ts>,<ratio between 0 and 1>"]}]}]}
```

This is the shape of every good availability alert: a ratio of a filtered rate to a total rate, over the same window. It is dimensionless, so it does not need retuning when traffic grows. Alerting on absolute error *count* does. In the generator, errors accrue on one cycle in four while successes accrue every cycle, so expect a **small ratio, well under 0.1**.

Step 5.6 — `histogram_quantile()`. Estimated p95 scan latency:

```
promql 'histogram_quantile(0.95%2C%20sum%20by%20(1e)
%20(rate(depot_portal_scan_latency_seconds_bucket%5B5m%5D))) '
{"status":"success","data":{"resultType":"vector","result":[{"metric":{},"value":
[<ts>,<"seconds">]]}}
```

Read the query inside out, because this is the most misread expression in PromQL.

`rate(..._bucket[5m])` converts each cumulative bucket counter into a per-second rate; `sum by (1e)` aggregates across all series while **keeping the 1e label**, which the function requires; `histogram_quantile(0.95, ...)` then interpolates within the bucket where the 95th percentile falls. Dropping `1e` from the `by` clause is the classic mistake and yields `NaN`.

Step 5.7 — Query the gauge, and compare.

```
promql 'depot_portal_queue_depth'
{"status":"success","data":{"resultType":"vector","result":[{"metric":
{"__name__":"depot_portal_queue_depth", ...},"value":[<ts>,<"12-51">]]}}
```

No `rate()` here — and applying one would be a bug. `rate()` on a gauge is meaningless: it assumes monotonicity and treats every decrease as a counter reset. For gauges you use the value directly, or `avg_over_time()`, `max_over_time()`, `delta()`.

Step 5.8 — Prometheus scraping itself. Compare your hand-built exposition against one produced by a mature client library:

```
promql 'prometheus_http_requests_total'
{"status":"success","data":{"resultType":"vector","result":[
{"metric":{"__name__":"prometheus_http_requests_total","code":"200","handler":"/api/v1/
query","instance":"localhost:9090","job":"prometheus","component":"monitoring","cluster":"
kcna-kind","env":"lab"},"value":[<ts>,<"counter">]}],
...}}
promql 'prometheus_tsdb_head_series'
{"status":"success","data":{"resultType":"vector","result":[{"metric":
{"__name__":"prometheus_tsdb_head_series", ...},"value":[<ts>,<"active series count">]]}}
```

`prometheus_tsdb_head_series` is the **single most important capacity metric** for any Prometheus. It counts active series, which is what drives memory. A cardinality explosion shows up here first, long before an OOMKill.

Step 5.9 — The free liveness metric.

```
promql 'up'
{"status":"success","data":{"resultType":"vector","result":[
{"metric":{"__name__":"up","component":"monitoring","cluster":"kcna-
kind","env":"lab","instance":"localhost:9090","job":"prometheus"},"value":[<ts>,"1"]},
{"metric":{"__name__":"up","cluster":"kcna-kind","depot":"sg-
tuas","env":"lab","instance":"depot-metrics-sim.kcna-
lab24.svc.cluster.local","job":"depot-portal","service":"depot-portal"},"value":
[<ts>,"1"]]]}}
```

Both 1. **up** is synthesised by Prometheus, not exposed by the target — it is the free liveness signal that the pull model buys you.

Step 5.10 — Record your Part 5 results in `data/exposition-worksheet.csv`.

Part 6 — Failure injection (mandatory)

You will break the scrape config in the most common way it breaks: a correct, valid config that points at the wrong port.

Step 6.1 — Confirm the healthy baseline.

```
promql 'up%7Bjob%3D%22depot-portal%22%7D'
{"status":"success","data":{"... "value":<ts>,"1"]}}}]}
```

Step 6.2 — Swap in the broken config. Note that this is a **valid** YAML file that Prometheus will load without complaint.

```
diff data/prometheus-scrape.yml data/prometheus-scrape-broken.yml | head -20
< # Job 2: the Depot Portal metrics simulator.
...
<         - depot-metrics-sim.kcna-lab24.svc.cluster.local:9100
---
>         # WRONG PORT — the Service listens on 9100.
>         - depot-metrics-sim.kcna-lab24.svc.cluster.local:9101

kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape-broken.yml \
  -n kcna-lab24 --dry-run=client -o yaml | kubectl apply -f -

Warning: resource configmaps/prometheus-config is missing the kubectl.kubernetes.io/last-applied-configuration annotation which was created by kubectl create --save-config or kubectl apply. The missing annotation will be applied automatically.
configmap/prometheus-config configured
```

Step 6.3 — Wait for the kubelet to propagate the ConfigMap into the Pod. This is **not instant** — projected ConfigMap volumes refresh on the kubelet sync period, typically up to ~60 seconds.

```
kubectl exec -n kcna-lab24 deploy/prometheus -- \
  grep -c 9101 /etc/prometheus/prometheus.yml
1
```

If this returns 0, the file has not refreshed yet. Wait 30 seconds and retry. **Do not restart the Pod** — you would lose the TSDB and the whole point of the hot reload.

Step 6.4 — Hot-reload Prometheus. `--web.enable-lifecycle` makes this possible without losing stored samples.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- --post-data='' \
  http://prometheus.kcna-lab24.svc.cluster.local:9090/-/reload
(no output — HTTP 200)
```

Step 6.5 — Wait ~20 seconds, then observe the failure. **This is the whole lesson of the pull model.**

```
promql 'up'
{"status":"success","data":{"resultType":"vector","result":[
{"metric":
{"__name__":"up","component":"monitoring","instance":"localhost:9090","job":"prometheus",
```

```
...}, "value": [ <ts>, "1" ] },
{ "metric": { "__name__": "up", "depot": "sg-tuas", "instance": "depot-metrics-sim.kcna-
lab24.svc.cluster.local", "job": "depot-portal", ... }, "value": [ <ts>, "0" ] } ] }
```

up{job="depot-portal"} is now 0. Nobody had to report anything: the failure to connect *is* the signal.

Step 6.6 — Get the reason from the targets API.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- 'http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/targets?
state=active' \
  | tr ',' '\n' | grep -E '"health"|"lastError"'
"health": "up"
"lastError": ""
"health": "down"
"lastError": "Get \"http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9101/metrics\":
dial tcp 10.96.x.x:9101: connect: connection refused"
```

connection refused — DNS resolved and the Service exists, but nothing listens on that port. Compare with the signatures in the Troubleshooting table.

Step 6.7 — The critical observation about stale data. Query the metric itself, then query it as a range:

```
promql 'depot_portal_queue_depth'
{"status": "success", "data": {"resultType": "vector", "result": []}}
```

Empty — the series went STALE, it did not go to zero. When a scrape fails, Prometheus marks the series stale rather than inventing a value. This matters enormously for alerting: a rule written as `depot_portal_queue_depth > 40` will simply **stop firing** when the target dies, because there is no data to compare. An alert on `up == 0` is what catches that, and it is why every mature alert set has one.

Step 6.8 — Repair: restore the good config and reload.

```
kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape.yml \
  -n kcna-lab24 --dry-run=client -o yaml | kubectl apply -f -
configmap/prometheus-config configured
```

Wait for propagation, then reload:

```
kubectl exec -n kcna-lab24 deploy/prometheus -- \
  grep -c 9100 /etc/prometheus/prometheus.yml
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- --post-data=' ' \
  http://prometheus.kcna-lab24.svc.cluster.local:9090/-/reload
1
```

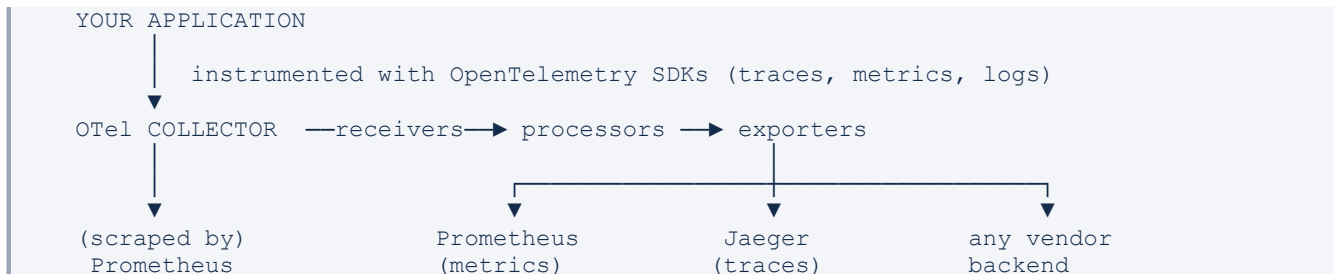
Step 6.9 — Confirm recovery, and note that the history survived.

```
promql 'up%7Bjob%3D%22depot-portal%22%7D'
{"status": "success", "data": { ... "value": [ <ts>, "1" ] } ] }
promql 'count_over_time(depot_portal_queue_depth%5B15m%5D)'
{"status": "success", "data": {"resultType": "vector", "result": [ { "metric": { ... }, "value":
[ <ts>, "<sample count>" ] } ] } }
```

The sample count spans the whole 15 minutes including the outage — the gap is visible as fewer samples than a perfect run would give. **You have just answered the INC-4417 question:** *when did it start, and how long did it last?* That is what the Depot Platform Squad could not do.

Part 7 — Where OpenTelemetry fits

Step 7.1 — Nothing to run. Fix the mental model, because this is examinable and widely muddled.



- **OpenTelemetry is a generation and collection standard, not a backend.** It defines the APIs, SDKs, semantic conventions and the OTLP wire protocol. It stores nothing and queries nothing. There is no "OpenTelemetry dashboard".
- **Prometheus is a backend:** it scrapes, stores in a TSDB, and serves PromQL.
- They are **complementary, not competing**. The common production shape is: instrument once with OTEL SDKs, run the OTEL Collector, and have it expose a Prometheus endpoint that Prometheus scrapes — or use Prometheus's native OTLP ingestion endpoint.
- **Why it matters commercially:** instrumenting with a vendor's proprietary agent locks your *code* to that vendor. Instrumenting with OTEL means changing backends is a Collector config change, not a re-instrumentation project. That vendor-neutrality is the entire reason the project exists — and why it is the second-highest-velocity project in the CNCF.

Step 7.2 — Commit these three sentences to memory for the assessment:

1. *Prometheus is CNCF **graduated**; OpenTelemetry is CNCF **incubating**; Grafana is **not a CNCF project** at all.*
2. *Prometheus **pulls** by default and synthesises `up` for free; OTLP **pushes**.*
3. `rate()` *exists because counters are monotonic lifetime totals, and it automatically handles counter resets on process restart.*

Lab 24 · Verification

```
bash verification/checks.sh
```

See `verification/expected-output.md`.

Lab 24 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
Prometheus Pod CrashLoopBackOff, log shows opening storage failed: ... permission denied	The TSDB directory is not writable by uid 65534	<code>kubectl logs -n kcna-lab24 deploy/prometheus</code>	Ensure <code>securityContext.fsGroup : 65534</code> is set on the Pod (present in the supplied manifest)
Prometheus Pod OOMKilled, RESTARTS climbing, reason: OOMKilled	Memory limit too small for the active series count	<code>kubectl get pod -n kcna-lab24 -l app=prometheus -o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated.reason}'</code>	Raise <code>limits.memory</code> ; check <code>prometheus_tsdb_head_series</code> for a cardinality explosion
Target DOWN with connect: connection refused	Port is wrong, or nothing is listening	<code>/api/v1/targets → lastError</code>	Correct the port in the scrape config and <code>POST /-/reload</code>
Target DOWN with no such host	Service DNS name is wrong or the Service does not exist	<code>kubectl exec metrics-client -- nslookup depot-metrics-sim.kcna-lab24.svc.cluster.local</code>	Use <code><svc>.<ns>.svc.cluster.local</code> ; confirm the Service exists
Target DOWN with context deadline exceeded	Target is too slow; <code>scrape_timeout</code> exceeded	<code>/api/v1/targets → lastError</code>	Raise <code>scrape_timeout</code> (must stay <code><= scrape_interval</code>), or make the exporter faster
Config edited but nothing changes	ConfigMap volume not yet refreshed, or no reload issued	<code>kubectl exec deploy/prometheus -- cat /etc/prometheus/prometheus.yml</code>	Wait for the kubelet sync (~60 s), then <code>POST /-/reload</code>
<code>POST /-/reload</code> returns 404	<code>--web.enable-lifecycle</code> is not set	<code>kubectl get deploy prometheus -o jsonpath='{.spec.template.spec.containers[0].args}'</code>	Add the flag (present in the supplied manifest)
<code>rate()</code> returns an empty result	Range window is shorter than ~2 scrape intervals, or the series is stale	Query the raw metric first	Widen the range to at least <code>4 × scrape_interval</code>
<code>histogram_quantile()</code> returns NaN	The <code>le</code> label was aggregated away	Inspect the inner <code>sum by (...)</code> clause	Always keep <code>le: sum by (le) (rate(..._bucket[5m]))</code>
Counters appear to jump backwards	More than one replica behind the scraped Service — each scrape hits a different Pod	<code>kubectl get deploy depot-metrics-sim -o jsonpath='{.spec.replicas}'</code>	Keep 1 replica for static scraping; in production discover Pods individually with <code>kubernetes_sd_configs</code>
Browser downloads <code>/metrics</code> instead of displaying it	Content-Type is <code>application/octet-stream</code>	<code>wget -S -qO /dev/null <url></code>	Set <code>default_type "text/plain; version=0.0.4"</code> (done in <code>05-exporter-nginx-config.yaml</code>)

Lab 24 · Cleanup

Delete **only** this lab's namespace.

```
kubectl delete namespace kcna-lab24
namespace "kcna-lab24" deleted
kubectl get namespace kcna-lab24
Error from server (NotFound): namespaces "kcna-lab24" not found
```

Stop any `kubectl port-forward` left running with `Ctrl-C`.

This lab created **no** cluster-scoped objects and touched **nothing** in `kube-system`. Deleting the namespace is a complete cleanup.

Lab 24 · What you learned

- **The four signals divide by question, not by tooling.** Metrics say *that* and *when*; logs and traces say *why*; Kubernetes Events say what the control plane decided. Alert on metrics, investigate with the rest.
- **The exposition format is just text over HTTP.** `# HELP`, `# TYPE`, a metric name, a label set in braces, a float. You scraped it with `wget` before Prometheus ever touched it — there is no magic in the protocol.
- **Types determine legal operations.** Counters only rise, so you must differentiate them with `rate()`. Gauges move freely, so you read them directly. Histograms are a *family* of cumulative `_bucket` series plus `_sum` and `_count`, and their quantile accuracy is fixed by bucket boundaries chosen in advance.
- **`rate()` exists because a lifetime total answers no operational question**, and because it silently corrects for counter resets across restarts.
- **Pull gives you liveness for free.** `up` is synthesised by Prometheus, not exposed by the target. You proved this by breaking one port number and watching `up` go to 0 with a precise `lastError`.
- **Failed scrapes produce staleness, not zeros.** Threshold alerts silently stop firing when a target dies, which is why `up == 0` must be alerted separately.
- **Labels are the cardinality budget.** Every distinct label combination is a separate time series; `prometheus_tsdb_head_series` is the metric that tells you when you have overspent.
- **`relabel_configs` runs before the scrape** (what to scrape, how to label it); **`metric_relabel_configs` runs after** (what to keep).
- **OpenTelemetry generates and collects; Prometheus stores and queries.** They are complementary. Prometheus and Fluentd and Jaeger are CNCF **graduated**; OpenTelemetry is **incubating**; Grafana is not a CNCF project.

Further reading

- Prometheus — *Exposition formats*: https://prometheus.io/docs/instrumenting/exposition_formats/
- Prometheus — *Metric types*: https://prometheus.io/docs/concepts/metric_types/
- Prometheus — *Querying basics* and *Query functions*:
<https://prometheus.io/docs/prometheus/latest/querying/basics/>
- Prometheus — *Configuration* → *scrape_config* and *relabel_config*:
<https://prometheus.io/docs/prometheus/latest/configuration/configuration/>

- Prometheus — *Instrumentation and naming best practices*:
<https://prometheus.io/docs/practices/naming/>
- OpenTelemetry — *What is OpenTelemetry?*: <https://opentelemetry.io/docs/what-is-opentelemetry/>
- CNCF — graduated and incubating project list: <https://www.cncf.io/projects/>
- CNCF — KCNA curriculum, *Cloud Native Architecture* → *Observability*:
<https://github.com/cncf/curriculum>

Lab 25 — Troubleshooting Triage: Application, Node and Control Plane

Field	Value
Lab ID	Lab 25
Title	Troubleshooting Triage: Application, Node and Control Plane
Day / Topic	Day 5 · Troubleshooting
Duration	55 minutes
Namespace	kcna-lab25
Learning outcome	LO6 — Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.
Ability	A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework
Knowledge	K7 Interactions among various IT components
Deck slide	Slide 549
Repository path	courseware/labs/lab-25-troubleshooting-triage/

Goal. Work a queue of four real incidents using a repeatable five-question triage method that identifies the failure phase before the cause, then inspect node and control-plane health strictly read-only.

What you will produce:

- Four diagnosed and repaired workloads: an image-pull typo, a CrashLoopBackOff exiting 127, a Pending Pod the scheduler refused, and a Service whose selector matched nothing
- A completed triage worksheet recording, per ticket, the first failing question, the decisive command, the root cause and the fix
- A reusable decision tree plus an exit-code reference the learner can carry into the practical assessment

How to read the expected-output blocks. Pod name suffixes, IPs, `AGE` columns and timestamps vary at runtime. Match the **STATUS strings**, the **event reasons** and the **error messages** — those are the diagnostic signal and they are stable.

Safety statement — read before Part 6

The control-plane section of this lab is **strictly read-only**. You will inspect static Pod manifests, read `kube-system` logs and query health endpoints. You will **never** be asked to modify, restart, scale or delete anything in `kube-system`, and you must not do so. On a shared training cluster, breaking the control plane ends the day for everyone. Control-plane *failure modes* are discussed, not induced.

Lab 25 · Objective

By the end of this lab you will be able to:

1. **Apply a repeatable five-question triage method** that identifies the *phase* a workload failed in before you form any hypothesis about why.

2. **Diagnose four distinct real failures** — image pull, CrashLoopBackOff, Pending, and a Service with no endpoints — from cluster evidence alone.
3. **Choose the right tool per phase:** `kubectl get` for phase, `kubectl describe`/`kubectl events` for scheduler and kubelet decisions, `kubectl logs [--previous]` for application behaviour, and EndpointSlice inspection for routing.
4. **Recognise container exit codes** — 0, 1, 127, 137 — and what each implies.
5. **Inspect the control plane read-only** with `--raw='/readyz?verbose'`, static Pod manifests and `kube-system` logs, and describe common control-plane failure modes without inducing them.
6. **Reuse a written decision tree** in the practical assessment.

Lab 25 · Prerequisites

- A running single-node **kind** cluster.

```
kubectl version
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

- You are in the lab directory:

```
cd courseware/labs/lab-25-troubleshooting-triage
```

- Images used, all **pinned**: `nginx:1.27-alpine`, `busybox:1.36`, `alpine:3.20`, `hashicorp/http-echo:1.0`.
- Labs 22-24 are soft prerequisites: probes, events, logs.
- Some Part 6 commands use `docker exec` against the kind node container. Get your node's name first — it is used throughout:

```
kubectl get nodes

NAME                 STATUS    ROLES    AGE   VERSION
kind-control-plane   Ready    control-plane   2d    v1.37.0

NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "node = $NODE"

node = kind-control-plane
```

Lab 25 · Scenario

It is the Monday after the Meridian Freight change freeze lifted. Over the weekend four teams pushed manifests into the **staging** depot cluster and went home. The Depot Platform Squad's on-call queue has four tickets waiting.

Read them:

```
column -s, -t data/incident-queue.csv
ticket   raised_by           priority  workload           manifest_to_apply
reported_symptom
sla_minutes
DEP-1041  gate-ops-sg-tuas    P2        depot-api          broken/01-depot-api-
imagepull.yaml      Deployed the new API build this morning. Pod never comes up. No
application logs at all.          10
DEP-1042  platform-oncall     P1        shipment-worker    broken/02-shipment-worker-
crashloop.yaml      Worker keeps restarting. It prints one line then vanishes. Restart
```

```

count is climbing fast.      10
DEP-1043 platform-oncall    P2      manifest-indexer  broken/03-manifest-indexer-
pending.yaml      Applied 20 minutes ago and nothing has happened. kubectl logs says
there is no container.      10
DEP-1044 driver-app-team    P1      depot-tracker    broken/04-depot-tracker-no-
endpoints.yaml    Pods are green, dashboard is green, but every request to the tracker
service times out.      15

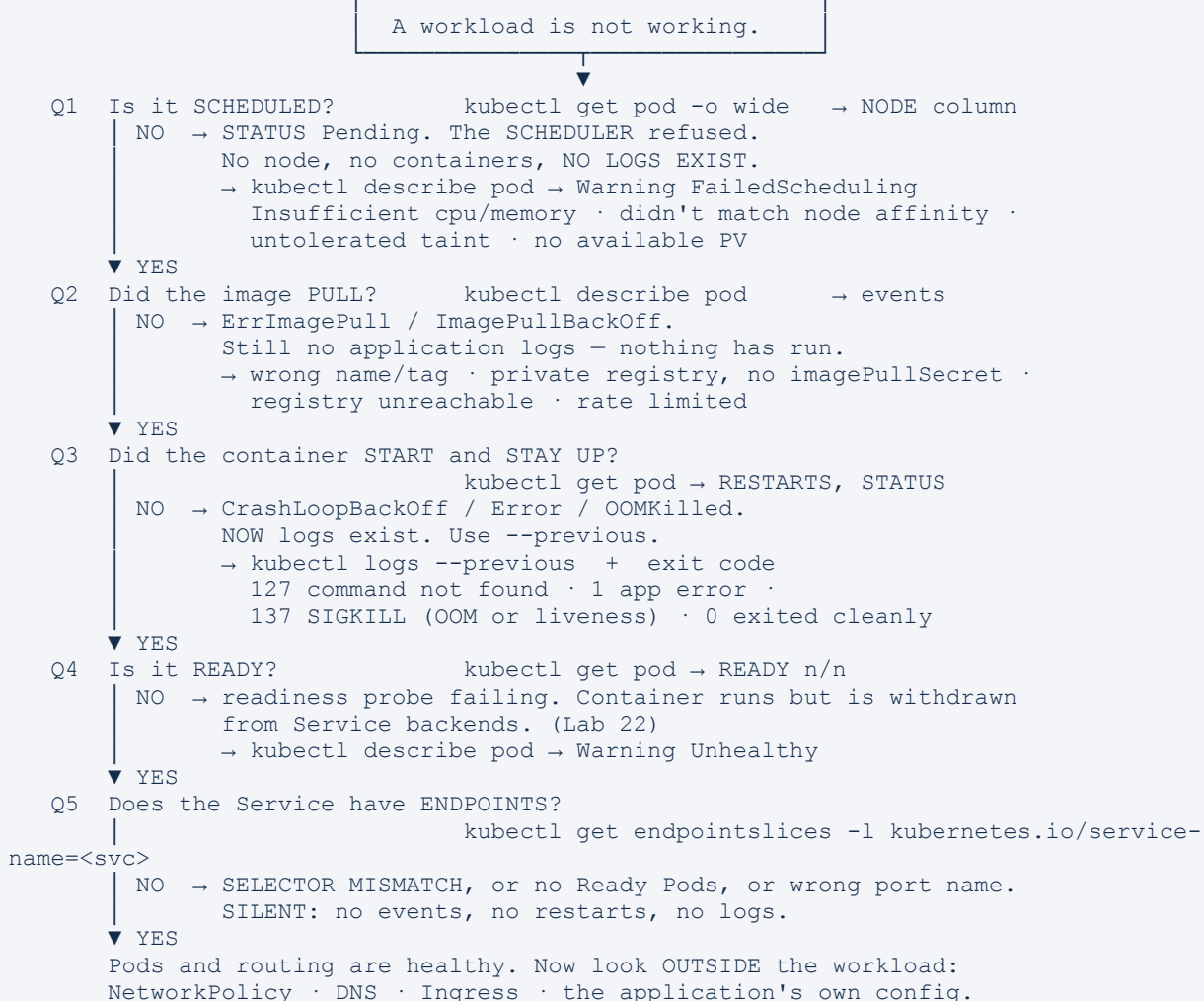
```

Every reporter has described a **symptom**, and every one of them has already guessed at a cause. None of the guesses is reliable. Your job is to work the method, not the guesses.

These four manifests are schema-valid. `kubectl apply` will accept all of them without complaint. That is deliberate and it is the most important thing about this lab: **passing validation is not the same as working.** The API server checks structure; it cannot check that an image tag is spelled correctly, that a binary exists, that a node has 512Gi of RAM, or that a selector matches anything.

Lab 25 · The triage method

Before you touch the cluster, learn the five questions. Ask them **in order** and **stop at the first NO** — that is your failure phase.



Why the order matters. Each question is only meaningful if the previous one answered YES. Asking for logs on a Pending Pod is not just useless, it is actively misleading — it tells you "no container", which sounds like a container problem and is not. **Phase first, cause second.**

Lab 25 · Step-by-step procedure

Part 1 — Namespace and the known-good baseline

Step 1.1 — Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml
namespace/kcna-lab25 created
```

Step 1.2 — Apply the healthy reference workload. Keep it running all lab: it is your control sample.

```
kubectl apply -f manifests/10-depot-portal-healthy.yaml
kubectl rollout status deployment/depot-portal -n kcna-lab25 --timeout=120s

deployment.apps/depot-portal created
service/depot-portal created
pod/triage-client created
deployment "depot-portal" successfully rolled out
```

Step 1.3 — Record what "working" looks like. All five questions answer YES.

```
kubectl get pods -n kcna-lab25 -o wide -l app=depot-portal
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-portal
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE READINESS GATES						
depot-portal-6f9c8d7b54-4nvqz	1/1	Running	0	40s	10.244.0.21	kind-
control-plane	<none>	<none>				
depot-portal-6f9c8d7b54-mzt8h	1/1	Running	0	40s	10.244.0.22	kind-
control-plane	<none>	<none>				

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-portal-4bp7q	IPv4	80	10.244.0.21,10.244.0.22	40s

Step 1.4 — Confirm it actually serves traffic.

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -qO- --timeout=5 http://depot-portal.kcna-lab25.svc.cluster.local/ | head -4

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Part 2 — Apply all four broken manifests

Step 2.1 — Apply them together. Note that all four are accepted.

```
kubectl apply -f broken/

deployment.apps/depot-api created
deployment.apps/shipment-worker created
deployment.apps/manifest-indexer created
deployment.apps/depot-tracker created
service/depot-tracker created
```

Five objects created, zero errors. Nothing here was rejected by validation. Every one of these failures is a *runtime* failure.

Step 2.2 — Wait ~90 seconds, then take one snapshot. This single command is your triage starting point for all four tickets.

```
kubectl get pods -n kcna-lab25 -o wide
```

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP
depot-api-7d4b9c6f85-x2ktp		0/1	ImagePullBackOff	0	92s	
10.244.0.23	kind-control-plane	<none>	<none>			
depot-portal-6f9c8d7b54-4nvqz		1/1	Running	0	4m	
10.244.0.21	kind-control-plane	<none>	<none>			
depot-portal-6f9c8d7b54-mzt8h		1/1	Running	0	4m	
10.244.0.22	kind-control-plane	<none>	<none>			
depot-tracker-5c8f7b9d64-h6wnr		1/1	Running	0	92s	
10.244.0.25	kind-control-plane	<none>	<none>			
depot-tracker-5c8f7b9d64-q4jsx		1/1	Running	0	92s	
10.244.0.26	kind-control-plane	<none>	<none>			
manifest-indexer-8b7d5c9f47-lm3zq		0/1	Pending	0	92s	<none>
<none>	<none>	<none>				
shipment-worker-59d4c8b7f6-w8kdt		0/1	CrashLoopBackOff	4 (21s ago)	92s	
10.244.0.24	kind-control-plane	<none>	<none>			
trriage-client		1/1	Running	0	4m	
10.244.0.20	kind-control-plane	<none>	<none>			

Step 2.3 — Read the snapshot with Q1 in mind, before diagnosing anything. Fill in the `q1_scheduled` column of `data/triage-worksheet.csv` now.

Pod	NODE	Q1 scheduled?	Earliest possible failure phase
depot-api	kind-control-plane	YES	Q2 or later
manifest-indexer	<none>	NO	Q1 — scheduling
shipment-worker	kind-control-plane	YES	Q3 or later
depot-tracker	kind-control-plane	YES	Q4 or later — and both are 1/1 Running, so Q5

Notice you have already localised all four failures to different phases, from **one** `kubectl get pods -o wide`, without a single `describe` or `logs`.

Part 3 — DEP-1041: depot-api (image pull)

Step 3.1 — Q1: scheduled? Yes — it has a node and an IP. Move to Q2.

Step 3.2 — Q2: did the image pull? `describe` is where the kubelet reports.

```
kubectl describe pod -n kcna-lab25 -l app=depot-api | tail -12
```

```
Events:
  Type     Reason      Age           From              Message
  ----     -
  Normal   Scheduled   2m12s        default-scheduler Successfully assigned
kcna-lab25/depot-api-7d4b9c6f85-x2ktp to kind-control-plane
  Normal   Pulling     47s (x4 over 2m11s)    kubelet            Pulling image "nginx:1.27-
alpine"
  Warning  Failed      45s (x4 over 2m9s)     kubelet            Failed to pull image
"nginx:1.27-alpine": failed to pull and unpack image "docker.io/library/nginx:1.27-alpine":
failed to resolve reference "docker.io/library/nginx:1.27-alpine":
```

```

docker.io/library/nginx:1.27-alpne: not found
Warning   Failed      45s (x4 over 2m9s)    kubelet           Error: ErrImagePull
Normal    BackOff      9s (x7 over 2m8s)    kubelet           Back-off pulling image
"nginx:1.27-alpne"
Warning   Failed      9s (x7 over 2m8s)    kubelet           Error: ImagePullBackOff

```

Step 3.3 — Confirm that logs are useless here, and understand why.

```

kubectl logs -n kcna-lab25 -l app=depot-api

Error from server (BadRequest): container "api" in pod "depot-api-7d4b9c6f85-x2ktp" is
waiting to start: trying and failing to pull image

```

No image means no container means no logs. Reporters who say "there are no logs" often think that is the problem. It is a *consequence* of the phase.

Step 3.4 — Extract the exact image reference from the API.

```

kubectl get deploy depot-api -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].image}'{"\n"}'

nginx:1.27-alpne

```

Step 3.5 — Compare against the known-good baseline. The diff *is* the bug.

```

kubectl get deploy depot-portal -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].image}'{"\n"}'

nginx:1.27-alpine

```

alpne versus alpine. **Root cause: misspelt image tag.**

Distinguish the three image-pull failure signatures, because the fixes are completely different:

lastError / message	Meaning	Fix
not found / manifest unknown	Name or tag is wrong	Correct the reference
unauthorized / authentication required	Private registry, no credentials	Create an <code>imagePullSecret</code> and reference it
no such host / i/o timeout	Registry unreachable from the node	Network, DNS, proxy or firewall
toomanyrequests	Docker Hub rate limit	Authenticate, or use a mirror/pull-through cache

Step 3.6 — Record DEP-1041 in `data/triage-worksheet.csv`: `first_failing_question = q2_pulled`.

Part 4 — DEP-1042 and DEP-1043

DEP-1042: shipment-worker (CrashLoopBackOff)

Step 4.1 — Q1 yes, Q2 yes (it has restarted, so the image pulled). Q3 is where it fails. Confirm from events.

```

kubectl describe pod -n kcna-lab25 -l app=shipment-worker | tail -10

Events:
  Type     Reason      Age           From              Message
  ----     -
  Normal   Scheduled   3m8s         default-scheduler Successfully assigned kcna-lab25/shipment-worker-59d4c8b7f6-w8kdt to kind-control-plane
  Normal   Pulled      94s (x5 over 3m7s)    kubelet           Container image

```

```
"busybox:1.36" already present on machine
Normal   Created    94s (x5 over 3m7s)    kubelet           Created container: worker
Normal   Started    94s (x5 over 3m7s)    kubelet           Started container worker
Warning  BackOff     12s (x14 over 3m4s)    kubelet           Back-off restarting failed
container worker in pod shipment-worker-59d4c8b7f6-w8kdt_kcna-lab25(...)
```

Pulled, Created and Started are all Normal. The image was fine and the container genuinely ran. This is unambiguously a Q3 failure, not Q2. Contrast with Step 3.2 where Pulling was followed by Failed.

Step 4.2 — Now logs are meaningful. Use `--previous`, because the current container is in back-off.

```
kubectl logs -n kcna-lab25 -l app=shipment-worker --previous --tail=10
[worker] starting shipment worker for depot sg-tuas
/bin/sh: /usr/local/bin/depot-shipment-worker: not found
```

Step 4.3 — Confirm with the exit code, which is authoritative.

```
WORKER=$(kubectl get pods -n kcna-lab25 -l app=shipment-worker \
-o jsonpath='{.items[0].metadata.name}')
kubectl get pod -n kcna-lab25 "$WORKER" -o jsonpath='{range .status.containerStatuses[*]}
{name}{ "  lastState.exitCode="}{.lastState.terminated.exitCode}{ "  reason="}
{.lastState.terminated.reason}{ "\n"}{end} '
worker lastState.exitCode=127 reason=Error
```

Exit code 127 is the shell's "command not found". Learn the four you will meet constantly:

Code	Meaning	Typical cause
0	Clean exit	Job finished; wrong <code>restartPolicy</code> for a long-running workload
1	Application error	Bad config, missing env var, failed dependency — read the log
127	Command not found	<code>command/args</code> disagree with the image contents
137	SIGKILL (128+9)	OOMKilled , or a failed liveness probe. Check <code>reason</code>

Step 4.4 — Prove the binary genuinely is not in **that image**. You cannot `exec` into the crash-looping container — it is never up long enough — so run a throwaway Pod from the *same* image and look for yourself. Note `--rm`, which removes it immediately afterwards.

```
kubectl run image-probe --rm -i --restart=Never \
--image=busybox:1.36 -n kcna-lab25 -- ls /usr/local/bin/
pod "image-probe" deleted
```

Empty listing, then the Pod is gone. Confirm the shell agrees:

```
kubectl run image-probe --rm -i --restart=Never \
--image=busybox:1.36 -n kcna-lab25 -- \
sh -c 'command -v depot-shipment-worker || echo "NOT IN IMAGE"'
NOT IN IMAGE
pod "image-probe" deleted
```

Root cause: the container command references a binary the image does not contain.

Step 4.5 — Record DEP-1042: `first_failing_question = q3_started`, exit code 127.

DEP-1043: manifest-indexer (Pending)

Step 4.6 — Q1 fails immediately: STATUS Pending, NODE <none>. Confirm that logs cannot help, so you do not waste SLA time there.

```
kubectl logs -n kcna-lab25 -l app=manifest-indexer
Error from server (BadRequest): pod manifest-indexer-8b7d5c9f47-lm3zq does not have a host assigned
```

Read that error precisely. does not have a host assigned is the unscheduled-Pod signature, and it is *different* from the image-pull Pod's waiting to start: trying and failing to pull image in Step 3.3. Two Pods, both with no logs, two completely different phases. The error text tells you which.

Step 4.7 — Go straight to the scheduler's verdict.

```
kubectl describe pod -n kcna-lab25 -l app=manifest-indexer | tail -8
Events:
  Type            Reason              Age             From              Message
  ----            -
Warning          FailedScheduling    4m5s (x2 over 4m20s)  default-scheduler  0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory.
preemption: 0/1 nodes are available: 1 No preemption victims found for incoming pod.
```

Read 0/1 nodes are available literally. It is a per-node tally of *reasons*: Insufficient cpu, Insufficient memory, node(s) had untolerated taint, node(s) didn't match Pod's node affinity/selector, node(s) had volume node affinity conflict. On a big cluster you get a breakdown per reason, and the tally tells you how much of the cluster each predicate ruled out.

Step 4.8 — Compare demand against supply. This is the arithmetic that settles it.

```
kubectl get deploy manifest-indexer -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].resources.requests}{"\n"}'
kubectl get node "$NODE" -o jsonpath='{.status.allocatable.cpu}{" cpu / "}'
{.status.allocatable.memory}{" memory\n"}'

{"cpu":"64","memory":"512Gi"}
10 cpu / 8039384Ki memory
```

64 cores and 512Gi requested; the node can offer 10 cores and roughly 7.7Gi. **Root cause: resource requests exceed any node's allocatable capacity.**

The exact allocatable figures above depend on your machine and your Docker Desktop resource allocation — yours will differ. The *relationship* is what matters: requested > allocatable.

Step 4.9 — See the alternative Pending cause without applying it. Open `broken/03-manifest-indexer-pending.yaml` and read the commented-out `nodeAffinity` block. Uncommenting it (with sane resources) produces the same Pending status but a different message:

```
0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector.
```

Same symptom, different root cause, different fix. This is precisely why the method separates *phase* from *cause*. Pending is a phase; there are at least five distinct causes.

Step 4.10 — Record DEP-1043: `first_failing_question = q1_scheduled`.

Part 5 — DEP-1044: depot-tracker (Service with no endpoints)

This is the hardest of the four, because nothing looks wrong.

Step 5.1 — Q1-Q4 all pass. Verify that for yourself:

```
kubectl get pods -n kcna-lab25 -l app=depot-tracker -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS GATES					
depot-tracker-5c8f7b9d64-h6wnr	1/1	Running	0	6m20s	10.244.0.25	kind-
control-plane	<none>					
depot-tracker-5c8f7b9d64-q4jsx	1/1	Running	0	6m20s	10.244.0.26	kind-
control-plane	<none>					

Scheduled, pulled, started, **Ready 1/1**, zero restarts. A status dashboard would be entirely green — exactly as the driver-app team reported.

Step 5.2 — Reproduce the user-facing symptom.

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -qO- --timeout=5 http://depot-tracker.kcna-lab25.svc.cluster.local/
wget: can't connect to remote host (10.96.184.22): Connection refused
command terminated with exit code 1
```

Connection refused from a ClusterIP with no backends. DNS resolved (there is an IP), so this is not a DNS fault. Note the contrast: a NetworkPolicy block or a missing route usually gives a **timeout**, whereas an endpoint-less Service typically gives an immediate **refusal**. That distinction is worth real minutes during an incident.

Step 5.3 — Q5. Ask the routing layer directly.

```
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-tracker
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-tracker-9xz4k	IPv4	<unset>	<unset>	6m45s

ENDPOINTS <unset>. Compare with the healthy baseline from Step 1.3, which listed two addresses. **This is the answer.**

Step 5.4 — Find out *why* there are no endpoints. There are three possible causes; eliminate them in order.

Cause A — no Ready Pods? Already disproved in Step 5.1.

Cause B — selector mismatch. Compare the two label sets:

```
echo "Service selects:"
kubectl get svc depot-tracker -n kcna-lab25 -o jsonpath='{.spec.selector}{"\n"}'
echo "Pods are labelled:"
kubectl get pods -n kcna-lab25 -l app=depot-tracker \
  -o jsonpath='{.items[0].metadata.labels}{"\n"}'

Service selects:
{"app":"depot-tracking"}
Pods are labelled:
{"app":"depot-tracker","app.kubernetes.io/part-of":"depot-portal","pod-template-hash":"5c8f7b9d64"}
```

depot-track**ing** versus depot-track**er**. **Root cause found.**

Step 5.5 — Confirm decisively by asking the API to select with the Service's own selector. Zero results proves the selector matches nothing.

```
kubectl get pods -n kcna-lab25 -l app=depot-tracking
No resources found in kcna-lab25 namespace.
```

Commit this one command to memory. `kubectl get pods -l <the service's selector>` answers "does this Service select anything?" in one step, and it works no matter how complex the selector is.

Step 5.6 — Cause C, for completeness: a **port name mismatch**. If

`Service.spec.ports[].targetPort` names a port the container does not declare, you get endpoints but with `PORTS <unset>` and traffic still fails. Check the healthy service to see what a correct mapping looks like:

```
kubectl get svc depot-portal -n kcna-lab25 -o jsonpath='{.spec.ports}{"\n"}'
kubectl get deploy depot-portal -n kcna-lab25 \
  -o jsonpath='{.spec.template.spec.containers[0].ports}{"\n"}'
[{"name":"http","port":80,"protocol":"TCP","targetPort":"http"}]
[{"containerPort":80,"name":"http","protocol":"TCP"}]
```

`targetPort: "http"` resolves because the container declares a port **named** `http`.

Step 5.7 — Record DEP-1044: `first_failing_question = q5_endpoints`.

Part 6 — Node and control-plane inspection (READ-ONLY)

Nothing in this Part modifies anything. Every command reads. Do not substitute a `delete`, `edit`, `patch`, `scale` or `restart` for any of them.

Step 6.1 — Node health. `Ready` is one of several conditions; the pressure conditions are what tell you a node is about to start evicting Pods.

```
kubectl get nodes -o wide
kubectl describe node "$NODE" | sed -n '/^Conditions:/,/^Addresses:/p'
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
OS-IMAGE		KERNEL-VERSION		CONTAINER-RUNTIME		
kind-control-plane	Ready	control-plane	2d	v1.37.0	172.18.0.2	<none>
Debian GNU/Linux 12 (bookworm)		6.10.14-linuxkit		containerd://2.1.4		

Conditions:

Type	Status	LastHeartbeatTime	LastTransitionTime	Reason
Message	----	-----	-----	-----

MemoryPressure	False	...	...	
KubeletHasSufficientMemory		kubelet has sufficient memory available		
DiskPressure	False	...	...	
KubeletHasNoDiskPressure		kubelet has no disk pressure		
PIDPressure	False	...	...	KubeletHasSufficientPID
kubelet has sufficient PID available				
Ready	True	...	...	KubeletReady
kubelet is posting ready status				

Node failure modes to recognise, not to induce.

- `Ready False / Unknown` — the kubelet has stopped posting status. After `--node-monitor-grace-period` the node controller taints the node and Pods are evicted. `Unknown` usually means the node or its network is gone.
- `DiskPressure True` — the kubelet starts **evicting Pods** and garbage collecting images. Very often it is the container log directory or unused images filling the disk.
- `MemoryPressure True` — eviction by QoS class: `BestEffort` first, then `Burstable`, `Guaranteed` last. This is the practical reason to set requests and limits.

Step 6.2 — Node capacity versus commitment. This is what settles "why can nothing schedule?" questions like DEP-1043.

```
kubectl describe node "$NODE" | sed -n '/Allocated resources/,/^Events/p'
```

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests          Limits
-----
cpu                 1050m (10%)       1300m (13%)
memory              690Mi (8%)        1224Mi (15%)
ephemeral-storage   0 (0%)            0 (0%)
```

The scheduler makes decisions on REQUESTS, not on live usage. A node can be 5% busy and still reject a Pod because its requests are fully committed. This is the single most common source of "there's plenty of free memory, why won't it schedule?".

Step 6.3 — Control-plane health via the API server's own endpoints. This is the modern, supported check.

```
kubectl get --raw='/readyz?verbose' | tail -20

[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
[+]poststarthook/start-apiserver-admission-initializer ok
[+]poststarthook/generic-apiserver-start-informers ok
[+]poststarthook/start-kube-apiserver-identity-lease-controller ok
[+]poststarthook/bootstrap-controller ok
[+]poststarthook/rbac/bootstrap-roles ok
[+]poststarthook/scheduling/bootstrap-system-priority-classes ok
[+]poststarthook/start-cluster-authentication-info-controller ok
[+]shutdown ok
readyz check passed

kubectl get --raw='/livez?verbose' | tail -5

[+]etcd ok
[+]autoregister-completion ok
[+]shutdown ok
livez check passed
```

readyz VS livez VS healthz. `livez` answers "is the process functional?"; `readyz` answers "should it receive traffic?" — the same distinction as container probes in Lab 22. `healthz` is the deprecated combined endpoint. `[-]etcd failed` in this listing is the single most important line to recognise: it means the API server cannot reach its datastore, and essentially everything else in the cluster is about to fail.

Step 6.4 — The deprecated check, for recognition only.

```
kubectl get componentstatuses

Warning: v1 ComponentStatus is deprecated in v1.19+
NAME                STATUS    MESSAGE    ERROR
scheduler            Healthy   ok
controller-manager   Healthy   ok
etcd-0               Healthy   ok
```

Expect this to be unhelpful or absent. ComponentStatus was deprecated in v1.19 and is not reliable on modern clusters — it may print an empty list, or the API may be gone entirely. **If it errors, that is not a fault.** Use `/readyz?verbose` instead. You should recognise `componentstatuses` in older documentation and know why not to depend on it.

Step 6.5 — Control-plane Pods. On a kubeadm/kind cluster these are **static Pods**, managed directly by the kubelet from files on disk — not by any controller.

```
kubectl -n kube-system get pods -o wide \
-l tier=control-plane
```

NAME	READY	STATUS	RESTARTS	AGE	IP
etcd-kind-control-plane	1/1	Running	0	2d	172.18.0.2
kind-control-plane					
kube-apiserver-kind-control-plane	1/1	Running	0	2d	172.18.0.2
kind-control-plane					
kube-controller-manager-kind-control-plane	1/1	Running	1	2d	172.18.0.2
kind-control-plane					
kube-scheduler-kind-control-plane	1/1	Running	1	2d	172.18.0.2
kind-control-plane					

Step 6.6 — Read the static Pod manifests on the node. Read-only — `cat`, never an editor.

```
docker exec "$NODE" ls -l /etc/kubernetes/manifests/

total 16
-rw-r--r-- 1 root root 2405 Sep  3 08:11 etcd.yaml
-rw-r--r-- 1 root root 3896 Sep  3 08:11 kube-apiserver.yaml
-rw-r--r-- 1 root root 3428 Sep  3 08:11 kube-controller-manager.yaml
-rw-r--r-- 1 root root 1463 Sep  3 08:11 kube-scheduler.yaml

docker exec "$NODE" grep -E '^\s+- --(etcd-servers|secure-port|advertise-address|service-cluster-ip-range)' \
/etc/kubernetes/manifests/kube-apiserver.yaml

- --advertise-address=172.18.0.2
- --etcd-servers=https://127.0.0.1:2379
- --secure-port=6443
- --service-cluster-ip-range=10.96.0.0/16
```

The static Pod mechanism, and its danger. The kubelet watches `/etc/kubernetes/manifests/` and runs whatever it finds there. There is no Deployment, no ReplicaSet and no API-server involvement — which is exactly how the API server itself can be started. The consequence: **editing a file in this directory restarts a control-plane component immediately**, with no rollout, no validation and no undo. A single YAML typo in `kube-apiserver.yaml` takes the cluster's API offline, and you then have no `kubectl` with which to fix it. That is why this lab only reads.

Step 6.7 — Control-plane logs, read-only.

```
kubectl -n kube-system logs kube-scheduler-"$NODE" --tail=8

I0905 06:44:02.118472      1 schedule_one.go:...] "Successfully bound pod to node"
pod="kcna-lab25/depot-portal-6f9c8d7b54-4nvqz" node="kind-control-plane"
```

```
I0905 06:45:31.884210      1 schedule_one.go:...] "Unable to schedule pod; no fit;
waiting" pod="kcna-lab25/manifest-indexer-8b7d5c9f47-lm3zq" err="0/1 nodes are available:
1 Insufficient cpu, 1 Insufficient memory."
```

Your DEP-1043 failure, from the scheduler's own perspective. The Event you read in Step 4.7 and this log line are the same decision reported through two different channels. When Events have expired (Lab 23: one-hour TTL), the component log is where the evidence still lives.

Step 6.8 — If a control-plane static Pod is not running at all, `kubectl` cannot show you its logs — so read the container runtime directly on the node. Know this command *before* you need it.

```
docker exec "$NODE" crictl ps --name kube-apiserver
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID
3f8a1c2e9b7d	alb2c3d4e5f6	2 days ago	Running	kube-apiserver	0	
9c8b7a6d5e4f	kube-apiserver-kind-control-plane					

Discussed, not induced — the control-plane failure modes.

Component down	What still works	What breaks	First check
kube-apiserver	Running Pods keep running; kube-proxy keeps routing	All <code>kubectl</code> ; every controller; all cluster state changes	<code>crictl ps</code> on the node; /etc/kubernetes/manifests/kube-apiserver.yaml; etcd reachability
etcd	Nothing meaningful — the API server goes read-only then fails	Everything	<code>kubectl get --raw='/readyz?verbose'</code> → [-]etcd failed
kube-scheduler	Existing Pods run normally	New Pods stay Pending forever with no FailedScheduling event at all	<code>kubectl -n kube-system logs kube-scheduler-<node></code>
kube-controller-manager	Existing Pods run	Deployments do not create ReplicaSets; failed nodes are never drained; endpoints stop updating	<code>kubectl -n kube-system logs kube-controller-manager-<node></code>
kubelet (on one node)	Other nodes fine	That node goes <code>NotReady</code> ; its Pods are evicted after the grace period	<code>docker exec <node> systemctl status kubelet</code>

The diagnostic that distinguishes "scheduler down" from "unschedulable Pod": a Pending Pod with **no FailedScheduling event whatsoever** means nothing is even evaluating it — suspect the scheduler. A Pending Pod **with a FailedScheduling event** means the scheduler is alive and is telling you exactly why it refused, as in DEP-1043.

Part 7 — Apply the fixes and verify

Step 7.1 — Only now, with all four root causes written into your worksheet, apply the fixes.

```
kubectl apply -f manifests/20-fixes.yaml
```

```
deployment.apps/depot-api configured
deployment.apps/shipment-worker configured
deployment.apps/manifest-indexer configured
service/depot-tracker configured
```

Note configured, not created — these are updates to the objects you already applied. Note also that the depot-tracker **Deployment** is absent from the fix file: it was never broken. Only its Service was.

Step 7.2 — Wait ~60 seconds and confirm all five phases pass everywhere.

```
kubectl get pods -n kcna-lab25 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS GATES					
depot-api-6b8d5f9c74-t7prv	1/1	Running	0	52s	10.244.0.27	
kind-control-plane	<none>	<none>				
depot-portal-6f9c8d7b54-4nvqz	1/1	Running	0	22m	10.244.0.21	
kind-control-plane	<none>	<none>				
depot-portal-6f9c8d7b54-mzt8h	1/1	Running	0	22m	10.244.0.22	
kind-control-plane	<none>	<none>				
depot-tracker-5c8f7b9d64-h6wnr	1/1	Running	0	18m	10.244.0.25	
kind-control-plane	<none>	<none>				
depot-tracker-5c8f7b9d64-q4jsx	1/1	Running	0	18m	10.244.0.26	
kind-control-plane	<none>	<none>				
manifest-indexer-7c9f8b6d54-k2xnp	1/1	Running	0	52s	10.244.0.28	
kind-control-plane	<none>	<none>				
shipment-worker-6d5c9b8f47-r3mqz	1/1	Running	0	52s	10.244.0.29	
kind-control-plane	<none>	<none>				
triage-client	1/1	Running	0	22m	10.244.0.20	
kind-control-plane	<none>	<none>				

No Pending, no ImagePullBackOff, no CrashLoopBackOff, zero restarts.

Step 7.3 — Confirm DEP-1044 specifically. The endpoints must now appear.

```
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-tracker
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-tracker-9xz4k	IPv4	5678	10.244.0.25,10.244.0.26	19m

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -qO- --timeout=5 http://depot-tracker.kcna-lab25.svc.cluster.local/
depot-tracker ok
```

Step 7.4 — Confirm DEP-1042's worker is now doing real work.

```
kubectl logs -n kcna-lab25 -l app=shipment-worker --tail=3
```

```
[worker] starting shipment worker for depot sg-tuas
[worker] 07:02:14 processed shipment batch
[worker] 07:02:29 processed shipment batch
```

Step 7.5 — Complete every column of data/triage-worksheet.csv. This is your assessment artefact.

Part 8 — Failure injection (mandatory): work one blind

Everything so far came with a ticket telling you which workload to look at. In the assessment it will not. Practise the method with no starting point.

Step 8.1 — Re-break exactly one thing, chosen at random by the shell so you do not know which:

```
CHOICE=$(( (RANDOM % 2) + 1 ))
case $CHOICE in
  1) kubectl set image deployment/depot-api api=nginx:1.27-alpne -n kcna-lab25 ;;
  2) kubectl patch service depot-tracker -n kcna-lab25 \
    -p '{"spec":{"selector":{"app":"depot-tracker"}}}' ;;
```

```
esac
echo "injected — do not scroll up"

deployment.apps/depot-api image updated
injected — do not scroll up
```

Step 8.2 — Find it using only the method. Start wide:

```
kubectl get pods -n kcna-lab25 -o wide
kubectl get endpointslices -n kcna-lab25
```

Work Q1 → Q5. Do not guess. Whichever question first answers NO tells you which of the two injections you received.

Step 8.3 — Time yourself. A competent engineer localises either of these to a phase in **under 60 seconds** with two commands.

Step 8.4 — Repair, whichever it was:

```
kubectl apply -f manifests/20-fixes.yaml
kubectl rollout status deployment/depot-api -n kcna-lab25 --timeout=120s

deployment.apps/depot-api configured
service/depot-tracker configured
deployment "depot-api" successfully rolled out
```

Step 8.5 — Verify the repair with the checks script.

```
bash verification/checks.sh
```

Lab 25 · Verification

```
bash verification/checks.sh
```

The script verifies the healthy end state and, importantly, that the four broken manifests are still **schema-valid** — see `verification/expected-output.md`.

You have completed the lab when all checks pass, `data/triage-worksheet.csv` is fully filled in, and you can reproduce the five-question decision tree from memory.

Lab 25 · Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
Pending, NODE <none>, FailedScheduling: Insufficient cpu/memory	Requests exceed any node's allocatable capacity	<code>kubectl describe node <node> sed -n '/Allocated resources/,/^Events/p'</code>	Lower resources.requests, or add capacity
Pending with didn't match Pod's node affinity/selector	nodeSelector/ nodeAffinity matches no node's labels	<code>kubectl get nodes --show-labels</code>	Correct the selector, or label the node
Pending with untolerated taint	Node is tainted; Pod has no matching toleration	<code>kubectl describe node <node> grep -i taint</code>	Add a toleration, or remove the taint (with care)
Pending and no FailedScheduling event at all	The scheduler is not running — nothing is evaluating the Pod	<code>kubectl -n kube-system get pods -l component=kube-scheduler</code>	Escalate to a cluster administrator. Do not attempt on a shared cluster
ErrImagePull / ImagePullBackOff with not found	Wrong image name or tag	<code>kubectl describe pod <pod> tail -12</code>	Correct the reference; compare with a known-good workload

Symptom	Likely cause	Diagnostic command	Fix
ImagePullBackOff with unauthorized	Private registry, no credentials	Same	Create an <code>imagePullSecret</code> and set <code>imagePullSecrets</code>
ImagePullBackOff with toomanyrequests	Registry rate limit	Same	Authenticate to the registry, or use a pull-through cache
CrashLoopBackOff, exit code 127	<code>command/args</code> reference a binary absent from the image	<code>kubectl logs <pod> --previous</code>	Fix the command, or use an image that contains the binary
CrashLoopBackOff, exit code 1	Application-level error	<code>kubectl logs <pod> --previous</code>	Read the log; usually configuration
CrashLoopBackOff, exit code 137, reason: OOMKilled	Memory limit too low	<code>kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'</code>	Raise <code>limits.memory</code> , or fix the leak
CrashLoopBackOff, exit code 137, reason: Error + Unhealthy events	Liveness probe killing a healthy-but-slow app	<code>kubectl describe pod <pod></code>	Add a <code>startupProbe</code> (Lab 22)
0/1 Running, RESTARTS 0, never Ready	Readiness probe failing	<code>kubectl describe pod <pod> grep -A3 Readiness</code>	Fix the probe path/port (Lab 22)
Pods Ready but Service ENDPOINTS <unset>	Service selector matches no Pod labels	<code>kubectl get pods -l <the service's selector></code> returns nothing	Align <code>spec.selector</code> with the Pod template labels
Endpoints present but PORTS <unset>	<code>targetPort</code> names a port the container does not declare	<code>kubectl get deploy <d> -o jsonpath='{.spec.template.spec.containers[0].ports}'</code>	Name the container port, or use the numeric port
Service resolves but times out (rather than refusing)	NetworkPolicy, or the app is not listening on that port	<code>kubectl get networkpolicy -n <ns></code>	Adjust the policy, or correct the listen port
<code>kubectl logs</code> says "waiting to start"	You are asking for logs before the container exists — a Q1/Q2 failure	<code>kubectl get pod <pod> -o wide</code>	Go back to Q1; use <code>describe</code> , not <code>logs</code>
<code>kubectl get componentstatuses</code> errors or is empty	Deprecated in v1.19+ and unreliable on modern clusters	<code>kubectl get --raw='/readyz?verbose'</code>	Not a fault — use <code>readyz/livez</code>

Lab 25 · Cleanup

Delete **only** this lab's namespace.

```
kubectl delete namespace kcna-lab25
namespace "kcna-lab25" deleted
kubectl get namespace kcna-lab25
Error from server (NotFound): namespaces "kcna-lab25" not found
```

This lab created **no** cluster-scoped objects and **modified nothing** in `kube-system` — Part 6 was entirely read-only. Deleting the namespace is a complete cleanup.

Keep your completed `data/triage-worksheet.csv`. It is the assessment artefact.

Lab 25 · What you learned

- **Phase before cause.** Five questions — scheduled, pulled, started, ready, endpoints — localise any workload failure in about two commands. Stop at the first NO. You applied four broken manifests and localised all four phases from a single `kubectl get pods -o wide`.
- **Schema-valid is not working.** All four broken manifests were accepted by the API server without a murmur. Validation checks structure; it cannot check that a tag is spelled right, a binary exists, a node has 512Gi, or a selector matches anything.
- **Match the tool to the phase.** `describe/events` for scheduler and kubelet decisions; `logs --previous` only once a container has actually run; EndpointSlice inspection for routing. Asking for logs on a `Pending` Pod returns a misleading answer.
- **Exit codes are evidence.** 127 = command not found (packaging); 1 = application error; 137 = SIGKILL, so check `reason` for `OOMKilled`; 0 = clean exit.
- **The scheduler decides on requests, not usage.** A near-idle node will still refuse a Pod whose requests do not fit.
- **Service selector mismatches are silent.** No event, no restart, no log — only absent endpoints. `kubectl get pods -l <service selector>` is the one-command test.
- **Connection refused and timeout mean different things.** Refused points at an endpoint-less Service; timeout points at NetworkPolicy or a wrong listen port.
- **Control-plane components are static Pods** run by the kubelet straight from `/etc/kubernetes/manifests/`, with no controller, no validation and no undo. Diagnose them with `/readyz?verbose` and `kube-system` logs; `componentstatuses` is deprecated. **Read, never write.**
- **A `Pending` Pod with no `FailedScheduling` event at all** is a scheduler outage, not an unschedulable Pod. Different problem, different escalation.

Further reading

- Kubernetes — *Troubleshooting Applications*: <https://kubernetes.io/docs/tasks/debug/debug-application/>
- Kubernetes — *Debug Running Pods*: <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>
- Kubernetes — *Debug Services*: <https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>
- Kubernetes — *Troubleshooting Clusters*: <https://kubernetes.io/docs/tasks/debug/debug-cluster/>
- Kubernetes — *Static Pods*: <https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/>
- Kubernetes — *Node-pressure Eviction*: <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>
- Kubernetes — *EndpointSlices*: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- CNCF — KCNA curriculum, *Kubernetes Fundamentals* and *Cloud Native Architecture*: <https://github.com/cncf/curriculum>

Quick Command Reference

Every command below is used somewhere in this course. Angle brackets are placeholders — replace them, including the brackets. Almost every command that operates on a namespaced object takes `-n <namespace>`; if you omit it you are talking to the namespace `default`, which is the single most common cause of “my object disappeared”.

119 commands in 9 groups. The groups follow the shape of the course: identity and discovery first, then objects, workloads, scheduling, networking, security, storage, observability and finally delivery and the control plane.

Context, identity and discovery

Command	What it does
<code>kubectl config get-contexts</code>	List every context in the kubeconfig; * marks the current one.
<code>kubectl config current-context</code>	Print only the current context name. Run this before every lab.
<code>kubectl config use-context <ctx></code>	Switch the active context.
<code>kubectl config view --minify</code>	Show only the current context, with credentials redacted.
<code>kubectl config set-context --current --namespace=<ns></code>	Set a default namespace so you can stop typing <code>-n</code> .
<code>kubectl version</code>	Client and server versions. They must be within one minor of each other.
<code>kubectl cluster-info</code>	API server and CoreDNS endpoints.
<code>kubectl api-versions</code>	Every API group/version pair the server serves.
<code>kubectl api-resources</code>	Every kind, its short names, API version and whether it is namespaced.
<code>kubectl api-resources --namespaced=false</code>	Only cluster-scoped kinds — the ones a namespace cannot isolate.
<code>kubectl explain <kind>.<path></code>	Read the live OpenAPI schema for a field. Never guess field names.
<code>kubectl explain <kind> --recursive</code>	The whole field tree for a kind.
<code>kubectl auth can-i <verb> <resource> -n <ns></code>	Test your own RBAC permission.
<code>kubectl auth can-i --list -n <ns></code>	Everything you are allowed to do in a namespace.
<code>kubectl auth can-i <verb> <res> --as system:serviceaccount:<ns>:<sa></code>	Test another identity's permission (impersonation).

Creating, reading and changing objects

Command	What it does
<code>kubectl apply -f <file dir></code>	Declaratively create or update from a manifest. The default verb of this course.
<code>kubectl apply -k <dir></code>	Apply a Kustomize overlay (Kustomize is built into kubectl).
<code>kubectl create <kind> <name> [flags]</code>	Imperatively create an object. Fails if it already exists.
<code>kubectl create configmap <n> --from-file=<path></code>	ConfigMap with one key named after the file's basename.
<code>kubectl create secret generic <n> --from-literal=k=v</code>	Secret from literal values (base64-encoded, not encrypted).

Command	What it does
<code>kubectl run <name> --image= --dry-run=client -o yaml</code>	Generate a starting Pod manifest without creating anything.
<code>kubectl apply -f <f> --dry-run=client</code>	Client-side validation only — no API call for the object.
<code>kubectl apply -f <f> --dry-run=server</code>	Full server-side validation, defaulting and admission, then discard.
<code>kubectl get <kind> [-n <ns>]</code>	List objects. Add <code>-A</code> for all namespaces.
<code>kubectl get <kind> <name> -o yaml</code>	Server-side truth: what you wrote plus defaults plus status.
<code>kubectl get <kind> -o wide</code>	Extra columns — node, IP, image, and more.
<code>kubectl get <kind> -o jsonpath='{.spec.field}'</code>	Extract exactly one field. Escape dots in keys as <code>\.</code>
<code>kubectl get <kind> -l <key>=<value></code>	Filter by label selector.
<code>kubectl get <kind> --field-selector <path>=<v></code>	Filter server-side on a supported field.
<code>kubectl get --raw <path></code>	Hit the REST API directly, bypassing kubectl's object model.
<code>kubectl describe <kind> <name></code>	Human-readable summary plus the object's recent events.
<code>kubectl edit <kind> <name></code>	Open the live object in an editor. Useful in a lab, not in production.
<code>kubectl patch <kind> <name> -p '<json>'</code>	Change specific fields without sending the whole object.
<code>kubectl label <kind> <name> k=v [--overwrite]</code>	Add or change a label — this is how adoption and orphaning are demonstrated.
<code>kubectl annotate <kind> <name> k=v</code>	Add or change an annotation.
<code>kubectl delete -f <file></code>	Delete exactly what a manifest created.
<code>kubectl delete namespace <ns></code>	Delete a namespace and everything namespaced inside it. The standard lab cleanup.

Workloads, controllers and rollouts

Command	What it does
<code>kubectl get deploy,rs,pods</code>	See the three-level ownership chain in one command.
<code>kubectl scale deployment <n> --replicas=<N></code>	Change the replica count. Does not create a new revision.
<code>kubectl set image deployment/<n> <c>=</code>	Change a container image — this does create a new revision.
<code>kubectl rollout status deployment/<n></code>	Block until the rollout completes or the progress deadline fires.
<code>kubectl rollout history deployment/<n></code>	List revisions.
<code>kubectl rollout history deployment/<n> --revision=<N></code>	Show one revision's Pod template.
<code>kubectl rollout undo deployment/<n></code>	Roll back to the previous revision (creating a new revision number).
<code>kubectl rollout restart deployment/<n></code>	Restart all Pods by stamping a new template annotation.
<code>kubectl rollout pause resume deployment/<n></code>	Batch several changes into one rollout.
<code>kubectl get jobs,cronjobs</code>	Job and CronJob status, completions and schedules.
<code>kubectl create job <n> --from=cronjob/<c></code>	Trigger a CronJob immediately, out of schedule.
<code>kubectl get ds -A</code>	DaemonSets — desired equals the node count, which is not your decision.
<code>kubectl get statefulset,pvc</code>	StatefulSet replicas and the per-ordinal claims that outlive them.
<code>kubectl wait --for=condition=Ready pod/<n> --timeout=90s</code>	Block until a condition is met. The correct way to wait in a script.

Scheduling and capacity

Command	What it does
<code>kubectl get nodes -o wide</code>	Node status, roles, version, IP, OS and container runtime.
<code>kubectl describe node <n></code>	Conditions, taints, allocatable capacity and the Pods already placed.
<code>kubectl label node <n> <k>=<v></code>	Label a node so a nodeSelector or affinity rule can target it.
<code>kubectl taint node <n> <k>=<v>:NoSchedule</code>	Repel Pods that do not tolerate the taint. Add a trailing - to remove.
<code>kubectl cordon <n> / kubectl uncordon <n></code>	Mark a node unschedulable, or schedulable again.
<code>kubectl drain <n> --ignore-daemonsets</code>	Evict Pods ahead of maintenance.
<code>kubectl top nodes</code>	Current CPU and memory usage per node (needs metrics-server).
<code>kubectl top pods -n <ns> --containers</code>	Current usage per Pod or container.
<code>kubectl get resourcequota,limitrange -n <ns></code>	The namespace's aggregate cap and its per-container defaults.
<code>kubectl get hpa -n <ns></code>	Autoscaler targets, current metric and replica bounds.

Services, networking and DNS

Command	What it does
<code>kubectl get svc -n <ns></code>	Service type, ClusterIP, ports and external IP.
<code>kubectl get endpointslices -n <ns></code>	The derived backend list. Empty means selector, readiness or port mismatch.
<code>kubectl get endpointslices -l kubernetes.io/service-name=<svc></code>	The slices for one Service specifically.
<code>kubectl expose deployment <n> --port=80 --target-port=8080</code>	Create a Service for an existing workload.
<code>kubectl port-forward svc/<n> 8080:80 -n <ns></code>	Tunnel a local port to a Service or Pod without exposing anything.
<code>kubectl get ingress,ingressclass -A</code>	Ingress rules, and whether any controller exists to implement them.
<code>kubectl describe ingress <n> -n <ns></code>	Rules, backends and the controller's own events.
<code>kubectl get networkpolicy -n <ns></code>	Policies in a namespace. Remember: enforcement is the CNI's job.
<code>kubectl exec -n <ns> <pod> -- nslookup <svc></code>	Resolve a Service name from inside the cluster.
<code>kubectl exec -n <ns> <pod> -- wget -qO- http://<svc>:<port></code>	Prove reachability from one Pod to another.
<code>kubectl run tmp --rm -it --image=busybox:1.36 -- restart=Never -- sh</code>	A throwaway shell inside the cluster network.

Security: identity, RBAC and configuration

Command	What it does
<code>kubectl get serviceaccounts -n <ns></code>	Workload identities in a namespace.
<code>kubectl create serviceaccount <n> -n <ns></code>	Create a workload identity.
<code>kubectl get roles,rolebindings -n <ns></code>	Namespaced permissions and grants.

Command	What it does
<code>kubectl get clusterroles,clusterrolebindings</code>	Cluster-wide permissions and grants.
<code>kubectl create role <n> --verb=get,list --resource=pods -n <ns></code>	Create a namespaced Role.
<code>kubectl create rolebinding <n> --role=<r> --serviceaccount=<ns>:<sa></code>	Grant a Role to a ServiceAccount in one namespace.
<code>kubectl describe clusterrole <n></code>	Read the rules a ClusterRole actually grants.
<code>kubectl get secret <n> -o jsonpath='{.data.<key>}' base64 -d</code>	Decode a Secret value — proof that base64 is not encryption.
<code>kubectl get cm <n> -o yaml</code>	Read a ConfigMap's keys and values.
<code>kubectl get ns --show-labels</code>	Namespace labels, including the Pod Security admission profile.

Storage

Command	What it does
<code>kubectl get pv</code>	Cluster-scoped volumes, their capacity, access modes, reclaim policy and status.
<code>kubectl get pvc -n <ns></code>	Namespaced claims and what they bound to.
<code>kubectl get storageclass</code>	Available classes, their provisioner and binding mode.
<code>kubectl describe pvc <n> -n <ns></code>	Why a claim is Pending — the events name the provisioner's reason.
<code>kubectl get csidrivers,csinodes</code>	Installed CSI drivers and the nodes that registered them.
<code>kubectl get volumeattachments</code>	Which volume is attached to which node.

Observability and troubleshooting

Command	What it does
<code>kubectl logs <pod> -n <ns></code>	stdout and stderr of a container.
<code>kubectl logs <pod> -c <container> -n <ns></code>	One container in a multi-container Pod.
<code>kubectl logs <pod> --previous</code>	The previous instance's logs — the only way to see why a crash loop died.
<code>kubectl logs -f <pod> --tail=50</code>	Follow the last 50 lines.
<code>kubectl logs -l app=<name> --all-containers</code>	Logs across every Pod matching a label.
<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>	What the control plane and kubelet actually did, oldest last.
<code>kubectl events -n <ns> --for pod/<name></code>	Events for one object.
<code>kubectl get events --field-selector type=Warning -A</code>	Every warning in the cluster.
<code>kubectl describe pod <n> -n <ns></code>	Conditions, container state, last state, exit code and events — the first command in any triage.
<code>kubectl exec -it <pod> -n <ns> -- sh</code>	A shell inside a running container.
<code>kubectl debug <pod> -it --image=busybox:1.36 --target=<c></code>	Attach an ephemeral debug container to a Pod you cannot exec into.
<code>kubectl cp <ns>/<pod>:<path> <local></code>	Copy a file out of a container.
<code>kubectl get pods -A --field-selector status.phase!=Running</code>	Everything that is not Running, cluster-wide.
<code>kubectl -n kube-system get pods</code>	Control-plane static Pods, CoreDNS, kube-proxy and the CNI agents.

Command	What it does
<code>kubectl get apiservices grep -v True</code>	Broken aggregated API extensions — a common cause of partial failures.
<code>kubectl get --raw /healthz /readyz /livez</code>	API server health endpoints.

Packaging, delivery and the control plane

Command	What it does
<code>helm template <release> <chart> -f values.yaml</code>	Render a chart locally. Installs nothing — the safe way to review.
<code>helm install <release> <chart> -n <ns> --create-namespace</code>	Create a release.
<code>helm upgrade <release> <chart> --set image.tag=<t></code>	New revision of an existing release.
<code>helm rollback <release> <revision></code>	Return a release to an earlier revision.
<code>helm list -n <ns> / helm history <release></code>	Releases and their revision history.
<code>helm lint <chart></code>	Static checks on a chart before you ship it.
<code>helm uninstall <release> -n <ns></code>	Remove a release.
<code>kubectl kustomize <dir></code>	Render a Kustomize overlay to stdout without applying it.
<code>kubectl apply -k overlays/<env></code>	Apply a rendered overlay.
<code>kind create cluster --config <file></code>	Create the local training cluster.
<code>kind get clusters / kind delete cluster --name <n></code>	List and remove local clusters. Never use <code>--all</code> .
<code>kind load docker-image --name <n></code>	Side-load a locally built image into the cluster's nodes.
<code>etcdctl snapshot save <file></code>	Back up etcd. Run on a control-plane node with the right certificates.
<code>etcdctl snapshot restore <file> --data-dir=<new></code>	Restore into a NEW data directory. A file operation, not an API call.
<code>crictl ps -a / crictl images</code>	Inspect containers and images through the CRI when the API server is down.

Assessment Preparation

The WSQ assessment for TGS-2023039343 is sat on **Day 5**, after the recap session. It carries 3 hours and uses **two instruments**. You must be judged **Competent** on both to be awarded the Statement of Attainment.

The two instruments

Instrument	Form	Items	Duration	Codes assessed
Written Assessment (SAQ)	Short-answer questions, open book	7	60 minutes	K1, K3, K2, K4, K6, K5, K7
Practical Performance (PP)	Hands-on tasks against a live cluster	6	120 minutes	A1, A3, A2, A4, A5, A6

The Written Assessment tests **knowledge** (7 questions in 60 minutes — roughly 8 minutes each). The Practical Performance tests **ability** (6 tasks in 120 minutes — roughly 20 minutes each) and is where the labs in this guide pay for themselves.

What each Written Assessment question covers

Q	Knowledge code	Knowledge statement	Revise from
1	K1	Process for developing proof of concepts	Day 1 Concepts; Labs 01–05
2	K3	Objectives of solution architecture	Day 2 Concepts; Labs 06–10
3	K2	Components of solution architecture	Day 3 Concepts; Labs 11–13
4	K4	Steps for developing solution architecture	Day 3 Concepts; Labs 11–13
5	K6	Technical blueprint design and construction process	Day 3 and Day 4 Concepts; Labs 14–19
6	K5	Tools and techniques for solution architecture modelling	Day 4 Concepts; Labs 20–21
7	K7	Interactions among various IT components	Day 5 Concepts; Labs 22–25

What each Practical Performance task covers

Task	LO	Ability code	Ability statement	Revise from
1	LO1	A1	Develop an architectural proof of concept	Day 1; Labs 01–05
2	LO2	A3	Identify technical and practical requirements as well as stakeholders' demands	Day 2; Labs 06–10

Task	LO	Ability code	Ability statement	Revise from
3	LO3	A2	Develop a solution architecture utilising appropriate tools, techniques and models of system components and interfaces	Day 3; Labs 11–13
4	LO4	A4	Prepare a technical blueprint for a solution in a given area	Day 3 and Day 4; Labs 14–19
5	LO5	A5	Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need	Day 4; Labs 20–21
6	LO6	A6	Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework	Day 5; Labs 22–25

How to prepare

1. **Re-run every lab's verification script.** `bash verification/checks.sh` in each lab folder. A lab you cannot get to a full pass is a lab you cannot do under time pressure.
2. **Re-run the failure injections without reading the diagnosis.** Break it, then work the triage tree from Day 5 before you look at the answer. This is the closest thing to the Practical Performance.
3. **Practise `kubectl explain` rather than memorising YAML.** You are allowed to read the cluster's own schema; use it.
4. **Learn the imperative generators.** `kubectl run ... --dry-run=client -o yaml` and `kubectl create deployment ... --dry-run=client -o yaml` produce a correct skeleton in seconds. Typing YAML from memory under time pressure is how people run out of time.
5. **Rehearse the namespace discipline.** Set `--namespace` on the context at the start of each task, and check `kubectl config current-context` before you touch anything.
6. **Write down the phase before the cause.** In the Practical Performance, state what you observed and which gate it failed at. Assessors mark reasoning, not just the final object.

The assessment flow, in order

Follow these five steps in this order on the day. Steps 1, 2, 4 and 5 maintain the course feedback, attendance and assessment records. Funding eligibility follows the applicable programme terms;

TRAQOM feedback is separate from attendance. Submit your evidence so the assessor can complete the competency decision.

1. **TRAQOM** — scan the TRAQOM QR code shown on the LMS and complete the SkillsFuture Singapore course-quality survey.
2. **Assessment digital attendance** — take the assessment-session digital attendance. This is separate from the daily training attendance you have already taken.
3. **Sit the assessment** — Written Assessment (SAQ) first (60 minutes), then Practical Performance (PP) (120 minutes).
4. **Submit your answers on the LMS** — upload or enter your responses at <https://lms-tms.tertiaryinfotech.com/> before the session closes.
5. **Sign the Assessment Summary Record** — review the assessor's judgement with them, ask about anything you disagree with, and sign.

Result. You are judged **Competent (C)** or **Not Yet Competent (NYC)** against each outcome. If any outcome is NYC, your assessor will explain the re-assessment arrangements before you leave.

A note on the external KCNA exam

The WSQ assessment above and the CNCF **KCNA** examination are separate things. This course prepares you for both, but sitting KCNA is optional, is booked and paid for separately through the Linux Foundation, and does not affect your Statement of Attainment. Current KCNA facts:

Item	Value
Format	Online, proctored, multiple-choice
Questions	60
Duration	90 minutes
Passing score	75%
Validity	2 years
Prerequisites	None
Retake	One retake included with the exam purchase
Eligibility window	12 months from purchase

The current KCNA blueprint, effective 24 November 2025, weights the four domains as follows. Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 5 Sep 2026.

Domain	Weight	Competencies
Kubernetes Fundamentals	44%	Kubernetes Core Concepts, Administration, Scheduling, Containerization
Container Orchestration	28%	Networking, Security, Troubleshooting, Storage
Cloud Native Application Delivery	16%	Application Delivery, Debugging
Cloud Native Architecture	12%	Observability, Cloud Native Ecosystem and Principles, Cloud Native Community and Collaboration

A practice exam is available to you at
<https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna>

Support

If anything in this guide does not work on your machine, ask during class — that is what the practical sessions are for. Outside class, use the contacts below.

Contact

Channel	Detail
Training provider	Tertiary Infotech Academy Pte Ltd
UEN	201200696W
Email	enquiry@tertiaryinfotech.com
Telephone	+65 6100 0613
Website	www.tertiarycourses.com.sg
Trainer	Dr. Alfred Ang — Founder & Principal Trainer, Tertiary Infotech Academy

Links you will need

Resource	What it is for	Link
Learning Management System	Course slides, this Learner Guide, lab access, the TRAQOM QR code, digital attendance and assessment submission	https://lms-tms.tertiaryinfotech.com/
Practice exam	Timed multiple-choice practice against the KCNA blueprint	https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna
Course page	Registered outline, dates, fees and funding eligibility	https://www.tertiarycourses.com.sg/wsq-kubernetes-and-cloud-native-associate-kcna-training.html
Lab repository	All 25 lab folders: manifests, datasets and verification scripts	https://github.com/tertiarycourses/TGS-2023039343-Kubernetes-and-Cloud-Native-Associate-KCNA-Training

Course identity

Field	Value
Course title	Kubernetes and Cloud Native Associate (KCNA) Training
TGS reference number	TGS-2023039343
TSC	Solution Architecture (ICT-DES-4006-1.1)
Duration	40 hours over 5 days (37 training + 3 assessment)
Document	Learner Guide v6.0
Effective date	5 September 2026